



דף שער

אוריה לשם - תיק פרויקט

Classify – מערכת לניהול למידה עם בדיקת עבודות בעזרת AI

תעודת זהות: 331614172

בית ספר: תיכון הרצוג כפר סבא

תאריך הגשה: 4.5.2026

שם המנחה: יוסי זהבי



Classify
EDUCATIONAL AI

1	דף שער
4	מבוא
4	ייזום
7	אפיון
14	תיאור תחום הידע
14	תיאור היכולות של צד הלקוח:
14	יכולות כלליות:
16	יכולות למשתמשים בעלי הרשאת תלמיד:
18	יכולות למשתמשים בעלי הרשאת מורה:
20	יכולות למשתמשים בעלי הרשאת מנהל:
21	יכולות למשתמשים בעלי הרשאת אדמין:
22	תיאור היכולות של צד השרת
28	ארכיטקטורת הפרויקט
28	רכיבי החומרה
29	תיאור הטכנולוגיה בפרויקט
30	תיאור זרימת המידע במערכת
30	יכולות כלליות
31	יכולות למשתמשים בעלי הרשאת תלמיד
32	יכולות למשתמשים בעלי הרשאת מורה
35	יכולות למשתמשים בעלי הרשאת מנהל:
36	יכולות למשתמשים בעלי הרשאת אדמין:
37	האלגוריתמים המרכזיים בפרויקט
39	סביבת הפיתוח
40	תיאור פרוטוקול התקשורת
47	מסכי המערכת
47	מסכים כללים:
51	מסכי תלמיד
54	מסכי מורה
59	מסכי מנהל
60	מסכי אדמין
62	תרשים מסכים

63	מבני הנתונים
70	סקירת חולשות ואיומים
73	מימוש
73	חלק א
73	סקירת כל הספריות והמודלים בפרויקט
74	מחלקות בצד השרת
81	מחלקות ומודולים בצד הלקוח
85	חלק ב
85	בעיות אלגוריתמיות
90	קטעי קוד מיוחדים
92	חלק ג
99	מדריך למשתמש
99	מבנה קבצי המערכת
99	דרישות מערכת
100	התקנת המערכת
100	הגדרת AI
100	הפעלת המערכת
101	שימוש כללי במערכת
105	רפלקציה

מבוא

ייזום

תיאור כללי:

מערכת classify היא מערכת ניהול למידה חדשנית הדומה במבנה למערכות מוכרות כגון Google Classroom ו-Microsoft Teams, אך כוללת יכולת ייחודית: בדיקת עבודות תלמידים בעזרת מנועי בינה מלאכותית (AI).

המערכת מאפשרת למורים לפתוח כיתות וקורסים, להוסיף מטלות, לעקוב אחרי הגשות, ולתלמידים – להגיש עבודות ולקבל משוב וציון.

בנוסף, כל עבודה עוברת בדיקה אוטומטית ע"י מספר מנועי AI, והמערכת מחשבת ציון ממוצע אובייקטיבי. המורה מקבל את תוצאות ה-AI ויכול לבחור האם לאשר, לערוך או לשנות את הציון, ולהוסיף משוב אישי לתלמיד.

בחרתי בפרויקט זה משום שהוא מאפשר לי להשתמש טוב ביכולות הקיימות שלי כמו תכנות ב-Python, עבודה עם רשתות DBI ועיצוב גרפי, וגם לחקור עוד בתחום העבודה עם AI. בנוסף, המערכת נותנת מענה אמיתי לצורך קיים – הקלה על מורים בתהליך בדיקת עבודות.

לקוחות המערכת:

לקוחות עיקריים: מוסדות חינוך (בתי ספר, מכללות או ארגוני הכשרה) המעוניינים לנהל קורסים ומטלות דיגיטליות.

משתמשי קצה:

מורים: יוצרים כיתות, מפרסמים מטלות, מקבלים את ציוני ה-AI, מוסיפים משוב ומנהלים את ציוני התלמידים.

תלמידים: מגישים עבודות, צופים בציונים ובמשובים, ומנהלים את המטלות שלהם בכל הקורסים.

מטרת ויעדי המערכת:

המטרה המרכזית של המערכת היא לשפר את תהליך הלמידה, על ידי הוספת AI לתהליך בדיקת העבודות. היעד המרכזי הוא שהמערכת תקצר זמני בדיקה בשביל המורים וגם תתן ציונים הוגנים ושווים לכל התלמידים.

בעיות תועלות וחסכונות:

הבעיות הן שבדיקת עבודות ידנית על ידי מורים היא תהליך ארוך, מייגע ולעיתים לא עקבי, מה שגורם לעומס רב על המורים. ובנוסף כל מורה בודק בצורה שונה במעט, ולכן כל תהליך הבדיקה אינו עקבי.

המערכת תאפשר למורים לבדוק עבודות בצורה יעילה יותר, ובכך לחסוך זמן בבדיקת העבודות וגם לבצע בדיקה יעילה יותר. תועלת נוספת היא שהמערכת תנקד באופן שוויוני כל תלמיד.

פתרונות קיימים:

כיום קיימות הרבה מערכות ממוחשבות לניהול למידה כשהמוכרות שבהן הן:

Google Classroom מערכת מבית Google המאפשרת למורים לפתוח כיתות, להוסיף מטלות ולתקשר עם תלמידים.

Microsoft Teams מערכת מקיפה לניהול כיתות ומטלות, עם אינטגרציה לשירותי Microsoft. כיום אין אף מערכת שעוזרת למורים לבדוק עבודות בעזרת AI, ובכל מערכת המורים נאלצים עדיין לעבור ולבדוק כל עובדה באופן ידני.

טכנולוגיית הפרויקט:

מערכות למידה ממוחשבות אינן דבר חדש והן נפוצות מאוד, אבל מערכת עם AI הינה דבר חדש שעוד לא נבנה. אך המערכת עדיין דורשת בדיקה ידנית של המורים משום שהAI עדיין אינו מושלם, ויכול לטעות ולתת ציון לא הוגן.

לכן מערכת כזאת שמשלבת בתוכה AI ויכולת בדיקה עצמאית היא טכנולוגיה חדשה שאינה קיימת כיום, גם לא בפתרונות הקיימים.

תחומי הפרויקט:

פיתוח תוכנה:

- פיתוח מערכת מלאה בצורת שרת-לקוח בשפת Python.
- בניית ממשק משתמש גרפי (GUI) באמצעות QML.
- כתיבת קוד צד שרת לניהול נתונים, קורסים, מטלות ובדיקות AI.

בסיסי נתונים:

- תכנון ויישום בסיס נתונים (SQLite) לאחסון משתמשים, קורסים, הגשות, ציונים ומשובים.
- שימוש בשאליות מאובטחות והבטחת שלמות הנתונים.

רשתות ותקשורת:

- עבודה עם פרוטוקול TCP להעברת נתונים בין השרת ללקוח.
- טיפול במספר חיבורים במקביל (תלמידים/מורים שונים).

אבטחת מידע:

- הצפנת סיסמאות בשמירה. (SHA256 / Argon2)
- הצפנת תעבורה בסיסית (RSA / AES) או. (HTTPS)
- הגנה מפני SQL Injection ובדיקת קבצים לפני אחסון.

בינה מלאכותית:

- שילוב מספר מנועי AI לצורך ניתוח ובדיקת עבודות.
- חישוב ציון ממוצע ממספר מנועים ומתן משוב אוטומטי ראשוני.
- אפשרות בקרה ועריכת ציון ע"י המורה.

מערכות הפעלה ותהליכים:

- שימוש בתהליכים מרובים (Multiprocessing / Threading) לבדיקת עבודות במקביל.
- ניהול משימות ברקע תוך שמירה על תגובתיות המערכת.

ממשקי משתמש וגישות:

- עיצוב ממשקים נוחים וברורים למורה ולתלמיד.
- הצגת נתוני הגשות, ציונים ומשובים בצורה פשוטה ואינטואיטיבית.

תחומים שאינם כלולים בפרויקט:

- תמיכה בגרסה אינטרנטית ובגרסה לנייד – כרגע המערכת תהיה רק כיישום Windows
- העתקות בין תלמידים – המערכת כרגע לא תבדוק אם תלמידים הגישו את אותה עבודה

אפיון

תיאור כללי והיכולות לכל סוג משתמש:

מערכת classify היא מערכת ממוחשבת לניהול למידה, המאפשרת למורים ולתלמידים לתקשר בצורה דיגיטלית, להגיש ולבדוק עבודות, ולקבל ציונים ומשוב באופן מאובטח ומבוקר.

המערכת כוללת שני צדדים עיקריים:

- צד מורה
- צד תלמיד

מאחורי הקלעים פועל שרת המטפל בבקשות המשתמשים, בהרשאות, בשמירת הנתונים, בהעלאת והורדת קבצים, ובתהליך בדיקת העבודות בעזרת בינה מלאכותית.

מורה יכול להתחבר למערכת לאחר שאושר על ידי מנהל בית הספר, לאחר ההתחברות המורה יכול:

- ליצור כיתות חדשות.
- לקבל קוד הצטרפות לכיתה ולשתף אותו עם תלמידים.
- לאשר או לדחות בקשות הצטרפות של תלמידים לכיתה.
- להשבית או להסיר תלמיד מכיתה.
- ליצור מטלות חדשות, כולל תיאור, תאריך הגשה וקובץ הנחיות לבדיקה.
- להפעיל או לבטל בדיקה בעזרת בינה מלאכותית לכל מטלה.
- לסגור מטלות כדי למנוע הגשות נוספות.
- לצפות בהגשות תלמידים ובקבצים שהוגשו.
- לצפות בציון ובמשוב שנוצרו על ידי הבינה המלאכותית.
- לקבוע ציון סופי ולהוסיף משוב אישי לתלמיד.
- להעלות חומרי לימוד לכיתה.
- לשלוח הודעות לתלמידי הכיתה.

תלמיד יכול להירשם למערכת ולהתחבר לאחר שאושר על ידי מנהל בית הספר, לאחר ההתחברות התלמיד יכול:

- לצפות בכיתות שבהן הוא חבר פעיל.
- להצטרף לכיתה בעזרת קוד הצטרפות.
- לצפות במטלות שהמורה פרסם.
- להגיש עבודה כקובץ או כטקסט.
- למחוק הגשה קיימת ולהגיש מחדש, כל עוד המטלה פתוחה.
- להוריד קבצים הקשורים למטלות ולחומרי לימוד.
- לצפות בציונים ובמשוב שקיבל.
- לצפות בהודעות מהמורה.
- לסמן הודעות וציונים כנקראו.
- לעזוב כיתה.

תלמידים ומורים יכולים להירשם לבית הספר שלהם, אך החשבון שלהם יישאר במצב המתנה עד שמנהל בית הספר יאשר אותם.

מנהל בית ספר נוצר כמשתמש פעיל בזמן ההרשמה.

בנוסף למערכת יהיו עוד שני סוגי הרשאות:

- מנהל
- אדמין

מנהל בית ספר אחראי על ניהול המשתמשים והמידע של בית הספר שלו, מנהל יכול:

- לאשר משתמשים חדשים שנרשמו לבית הספר.
- לחסום משתמשים פעילים.
- לבטל חסימה של משתמשים חסומים.
- לצפות ברשימת המשתמשים של בית הספר.
- לצפות בכיתות של בית הספר.
- לצפות במטלות של בית הספר ובכמות ההגשות לכל מטלה.
- לעדכן פרטי בית ספר.

אדמין הוא משתמש בעל הרשאה כללית לכל המערכת, האדמין יכול:

- לצפות בכל בתי הספר, המשתמשים, הכיתות, המטלות, ההגשות, הציונים, המשובים, חומרי הלימוד וההודעות.
- ליצור בית ספר חדש.
- לעדכן או למחוק בית ספר.
- ליצור משתמשים.
- לעדכן או למחוק משתמשים.
- לעדכן כיתות ומטלות.
- לקבל תמונת מצב כללית על פעילות המערכת.

פירוט הבדיקות המתוכננות:

בדיקות הקופסה השחורה יתמקדו בבדיקת המערכת מנקודת מבט המשתמש, ללא בחינת המימוש הפנימי של הקוד.

הבדיקות יבוצעו עבור הפונקציונליות המרכזית במערכת: הרשמה והתחברות, ניהול קורסים, ניהול מטלות והגשות, בדיקות AI והרשאות משתמשים.

1. בדיקת הרשמה של משתמש חדש

מטרת הבדיקה: לבדוק שמשתמש חדש יכול להירשם למערכת.
אופן ביצוע: מילוי שם מלא, דואר אלקטרוני, סיסמה, סוג משתמש ומזהה בית ספר, ולחיצה על כפתור ההרשמה.
תוצאה צפויה: המשתמש נוצר במערכת, ותוצג הודעה מתאימה. תלמידים ומורים ימתינו לאישור מנהל בית הספר.

2. בדיקת הרשמה עם שדות חסרים

מטרת הבדיקה: לבדוק שהמערכת אינה מאפשרת הרשמה לא תקינה.

אופן ביצוע: ניסיון הרשמה כאשר אחד או יותר מהשדות ריקים.
תוצאה צפויה: המערכת תמנע את ההרשמה ותציג הודעת שגיאה מתאימה.

3. **בדיקת התחברות עם פרטים תקינים**

מטרת הבדיקה: לבדוק שמשתמש פעיל יכול להתחבר למערכת.
אופן ביצוע: הזנת דואר אלקטרוני וסיסמה תקינים במסך ההתחברות.
תוצאה צפויה: המשתמש יעבור למסך הבית המתאים להרשאה שלו.

4. **בדיקת התחברות עם פרטים שגויים**

מטרת הבדיקה: לבדוק שהמערכת מונעת כניסה לא מורשית.
אופן ביצוע: הזנת דואר אלקטרוני או סיסמה שגויים.
תוצאה צפויה: המערכת לא תאפשר כניסה ותציג הודעת שגיאה.

5. **בדיקת התחברות של משתמש שלא אושר**

מטרת הבדיקה: לבדוק שמשתמש במצב המתנה לא יכול להיכנס למערכת.
אופן ביצוע: ניסיון התחברות של תלמיד או מורה שנרשם אך עדיין לא אושר.
תוצאה צפויה: המערכת תמנע את הכניסה ותציג הודעה שהחשבון ממתין לאישור.

6. **בדיקת יצירת כיתה על ידי מורה**

מטרת הבדיקה: לבדוק שמורה יכול ליצור כיתה חדשה.
אופן ביצוע: התחברות כמורה, פתיחת מסך יצירת כיתה, מילוי שם ותיאור, ולחיצה על יצירה.
תוצאה צפויה: הכיתה תתווסף לרשימת הכיתות של המורה ויווצר לה קוד הצטרפות.

7. **בדיקת הצטרפות תלמיד לכיתה בעזרת קוד**

מטרת הבדיקה: לבדוק שתלמיד יכול לשלוח בקשת הצטרפות לכיתה.
אופן ביצוע: התחברות כתלמיד, הזנת קוד כיתה תקין ושליחת בקשה.
תוצאה צפויה: הבקשה תישמר ותופיע אצל המורה כרשומה הממתנה לאישור.

8. **בדיקת אישור תלמיד לכיתה**

מטרת הבדיקה: לבדוק שמורה יכול לאשר תלמיד שהצטרף לכיתה.
אופן ביצוע: התחברות כמורה, פתיחת רשימת בקשות ההצטרפות ואישור תלמיד.
תוצאה צפויה: התלמיד יהפוך לחבר פעיל בכיתה ויראה את הכיתה והמטלות שלה.

9. **בדיקת יצירת מטלה חדשה**

מטרת הבדיקה: לבדוק שמורה יכול ליצור מטלה בכיתה.
אופן ביצוע: התחברות כמורה, בחירת כיתה, הזנת כותרת, תיאור ותאריך הגשה, וצירוף קובץ הנחיות במקרה של בדיקה בעזרת בינה מלאכותית.
תוצאה צפויה: המטלה תישמר ותופיע אצל המורה ואצל תלמידי הכיתה.

10. **בדיקת יצירת מטלה עם בדיקת בינה מלאכותית ללא קובץ הנחיות**

מטרת הבדיקה: לבדוק שהמערכת אינה מאפשרת בדיקה אוטומטית ללא הנחיות.
אופן ביצוע: ניסיון ליצור מטלה כאשר בדיקת בינה מלאכותית פעילה אך לא צורף קובץ הנחיות.

תוצאה צפויה: המערכת תמנע את יצירת המטלה ותציג הודעה שיש לצרף קובץ הנחיות.

11. בדיקת הגשת עבודה על ידי תלמיד

מטרת הבדיקה: לבדוק שתלמיד יכול להגיש עבודה למטלה פתוחה. אופן ביצוע: התחברות כתלמיד, בחירת מטלה פתוחה, העלאת קובץ או כתיבת תשובה טקסטואלית ושליחת ההגשה. תוצאה צפויה: ההגשה תישמר במערכת ותופיע אצל המורה ברשימת ההגשות.

12. בדיקת הגשה למטלה סגורה

מטרת הבדיקה: לבדוק שהמערכת מונעת הגשות לאחר סגירת מטלה. אופן ביצוע: מורה סוגר מטלה, ולאחר מכן תלמיד מנסה להגיש אליה עבודה. תוצאה צפויה: המערכת תמנע את ההגשה ותציג הודעת שגיאה.

13. בדיקת העלאת קובץ גדול מדי

מטרת הבדיקה: לבדוק שהמערכת מגבילה העלאת קבצים. אופן ביצוע: ניסיון להעלות קובץ שגודלו גדול מהמגבלה שהוגדרה במערכת. תוצאה צפויה: ההעלאה תיחסם ותוצג הודעת שגיאה.

14. בדיקת תהליך בדיקה בעזרת בינה מלאכותית

מטרת הבדיקה: לבדוק שהגשה שנשלחה לבדיקה עוברת תהליך בדיקה אוטומטי. אופן ביצוע: תלמיד מגיש עבודה למטלה שבה בדיקת בינה מלאכותית פעילה. לאחר מכן המורה פותח את רשימת ההגשות. תוצאה צפויה: לאחר סיום התהליך יוצגו למורה ציון ומשוב ראשוני שנוצרו על ידי המערכת.

15. בדיקת עריכת ציון ומשוב על ידי מורה

מטרת הבדיקה: לבדוק שהמורה יכול לקבוע את הציון הסופי. אופן ביצוע: מורה פותח הגשה, בודק את הציון והמשוב שנוצרו, משנה את הציון או מוסיף משוב אישי ושומר. תוצאה צפויה: הציון הסופי והמשוב נשמרים ומוצגים לתלמיד.

16. בדיקת העלאת חומר לימוד

מטרת הבדיקה: לבדוק שמורה יכול להעלות חומרי לימוד לכיתה. אופן ביצוע: התחברות כמורה, בחירת כיתה, יצירת חומר לימוד חדש וצירוף קובץ או תיאור. תוצאה צפויה: חומר הלימוד נשמר ומופיע לתלמידים בכיתה.

17. בדיקת שליחת הודעה לכיתה

מטרת הבדיקה: לבדוק שמורה יכול לפרסם הודעה לתלמידי הכיתה. אופן ביצוע: התחברות כמורה, כתיבת נושא ותוכן הודעה ושליחתה לכיתה. תוצאה צפויה: ההודעה תופיע אצל תלמידי הכיתה במסך ההודעות.

18. בדיקת סימון הודעה כנקראה

מטרת הבדיקה: לבדוק שהמערכת שומרת מצב קריאה של הודעות.

אופן ביצוע: תלמיד פותח הודעה חדשה במסך ההודעות.
תוצאה צפויה: ההודעה תסומן כנקראה ולא תופיע כהודעה חדשה.

19. בדיקת הרשאות תלמיד

מטרת הבדיקה: לבדוק שתלמיד לא יכול לבצע פעולות של מורה.
אופן ביצוע: התחברות כתלמיד וניסיון לבצע פעולה שאינה שייכת לתלמיד, כגון יצירת מטלה או ניהול תלמידים בכיתה.
תוצאה צפויה: המערכת תמנע את הפעולה ולא תאפשר גישה לא מורשית.

20. בדיקת הרשאות מנהל בית ספר

מטרת הבדיקה: לבדוק שמנהל רואה רק את המידע של בית הספר שלו.
אופן ביצוע: התחברות כמנהל בית ספר וצפייה במשתמשים, כיתות ומטלות.
תוצאה צפויה: יוצגו רק נתונים השייכים לבית הספר של אותו מנהל.

21. בדיקת הרשאות אדמין

מטרת הבדיקה: לבדוק שלאדמין יש גישה כללית למערכת.
אופן ביצוע: התחברות כאדמין ופתיחת מסך הניהול הכללי.
תוצאה צפויה: האדמין יוכל לצפות בכלל בתי הספר, המשתמשים, הכיתות, המטלות, ההגשות והנתונים המרכזיים במערכת.

לוח זמנים:

מס' אבן דרך	תאריך יעד מתוכנן	תאריך בפועל	תיאור המשימה	הערות / פירוט
1	10.09.2025	10.09.2025	בחירת נושא והגדרת כיוון לפרויקט	בחירת נושא, בדיקת צרכים של מערכת LMS
2	20.09.2025	20.09.2025	מחקר: טכנולוגיות AI, תקשורת DBI-	למידה על TCP, DB, QML, שירותי AI
3	05.10.2025	5.10.2025	כתיבת מבוא וייזום	תיאור המערכת, מטרות, לקוחות, בעיות ופתרונות
4	25.11.2025	25.11.2025	כתיבת אפיון מפורט	פירוט יכולות, תרחישים, הרשאות, בדיקות קופסה שחורה
5	30.11.2025	30.11.2025	תכנון לו"ז וניהול סיכונים	השלמת חלקי האפיון הנדרשים
6	20.12.2025	20.12.2025	ארכיטקטורת מערכת + תרשימי זרימה	תכנון שרת-לקוח, פרוטוקול, מודולים

מס' אבן דרך	תאריך יעד מתוכנן	תאריך בפועל	תיאור המשימה	הערות / פירוט
7	30.12.2025	30.12.2025	תכנון בסיס נתונים (DB)	טבלאות: משתמשים, קורסים, מטלות, הגשות, ציוני AI
8	20.01.2026	20.01.2026	פיתוח צד שרת – שלב א'	הרשמה, התחברות, ניהול משתמשים
9	15.02.2026	15.02.2026	פיתוח צד שרת – שלב ב'	ניהול קורסים ומטלות, תקשורת TCP
10	28.02.2026	28.02.2026	פיתוח צד לקוח – ממשק תלמיד ומורה	בניית ממשק משתמש בעזרת QML ו-PySide6
11	15.03.2026	15.03.2026	שילוב מנועי AI במערכת	בדיקת עבודות, חישוב ציון, משוב אוטומטי
12	01.04.2026	01.04.2026	בדיקות מערכת (QA)	בדיקות קופסה שחורה, עומסים, הרשאות AI,
13	20.04.2026	20.04.2026	תיקוני באגים ושיפורים	תיקונים לפי תוצאות בדיקות
14	01.05.2026	20.04.2026	כתיבת חלק המימוש בתיק	תיעוד מחלקות, מודולים, קטעי קוד
15	סוף מאי 2026	04.05.2026	השלמת תיק פרויקט והגשה	מדריך משתמש, ביבליוגרפיה, רפלקציה

הסיכונים בפרויקט:

מס'	תיאור הסיכון	השפעה על הפרויקט	דרכי התמודדות מתוכננות	מה בוצע בפועל
1	מנועי ה-AI נותנים תוצאות לא עקביות / לא אמינות	ציונים שגויים, משוב לא מתאים	שימוש במספר מנועי AI וחישוב ממוצע; אפשרות לעריכת ציון ע"י מורה	בוצע איחוד תוצאות ממנועי הבינה המלאכותית הזמינים, והמורה יכול לערוך את הציון והמשוב לפני פרסום לתלמיד.
2	עומס על השרת בזמן בדיקות AI מרובות	האטת המערכת, קריסות	שימוש ב-Threading/Multiprocessing; הגבלת כמות בדיקות במקביל	בדיקות הבינה המלאכותית מתבצעות ברקע בעזרת תור הגשות, כך שהשרת ממשיך לטפל בבקשות משתמשים.
3	בעיות אבטחת מידע (SQL Injection, פריצות)	דליפת נתונים, פגיעה במשתמשים	שימוש ב-Prepared Statements, סיסמאות, בדיקת קבצים לפני אחסון	בוצעו הצפנת תקשורת, גיבוב סיסמאות, בדיקות הרשאה בשרת ושימוש בשאליות מאובטחות מול בסיס הנתונים.
4	תקלות ב-GUI עקב עומס או פעולות רקע	ממשק קופא או קורס	הרצת עבודות AI בתהליכים נפרדים UI ; נשאר רספונסיבי	פעולות תקשורת ובדיקות כבדות מתבצעות ברקע, כדי לשמור על ממשק משתמש פעיל ולא קפוא.
5	תלמידים מעלים קבצים גדולים מדי / קבצים פגומים	עומס על השרת, תקלות בדיקה	הגבלת גודל קובץ, בדיקת תקינות לפני שליחה ל-AI	נוספו בדיקות לגודל קובץ, קובץ ריק, תוכן לא תקין וסריקת קבצים לפני שמירה בשרת.
6	חוסר התאמה בין הרשאות (תלמיד/מורה/מנהל)	חשיפה למידע לא נכון	בדיקות הרשאות בכל פעולה בשרת; בדיקות QA	נוספה בדיקת הרשאות מרכזית בשרת לכל בקשה רגישה לפי סוג המשתמש והשיוך שלו.
7	תקלה באחד ממנועי ה-AI	אי יכולת להוציא ציון	מעבר אוטומטי למנוע חלופי, הודעת שגיאה למורה	המערכת ממשיכה עם מנועים זמינים אחרים, ואם כל המנועים נכשלים ההגשה מסומנת כשגיאה.

תיאור תחום הידע

תיאור היכולות של צד הלקוח:

יכולות כלליות:

הרשמה למערכת:

פירוט	נושא
הרשמה למערכת – (Sign Up) צד לקוח	שם היכולת
מאפשרת למשתמש חדש (תלמיד / מורה / מנהל) ליצור חשבון במערכת classify ע"י מילוי פרטים אישיים ושליחתם לשרת לאישור.	מהות היכולת
• הצגת מסך הרשמה עם שדות חובה (שם, מייל, סיסמה, סוג משתמש, בית ספר). • בדיקת תקינות ראשונית בצד לקוח (שדות לא ריקים, סיסמה באורך מינימלי, פורמט מייל). • הצפנת הסיסמה לפני השליחה לשרת. • שליחת בקשת הרשמה לשרת דרך פרוטוקול התקשורת (TCP). • קבלת תשובה מהשרת (הצלחה / כישלון / ממתין לאישור מנהל). • הצגת הודעת הצלחה או הודעת שגיאה למשתמש.	אוסף היכולות / פעולות הנדרשות למימוש
• טפסי GUI ב־ Pyside6 qml (תיבות טקסט, כפתור "הרשמה"). • מודול תקשורת לקוח-שרת (socket/TCP) • מודול הצפנה בצד הלקוח (לסיסמאות).	אובייקטים נחוצים

התחברות למערכת:

פירוט	נושא
התחברות למערכת – (Login) צד לקוח	שם היכולת
מאפשרת למשתמש קיים להיכנס למערכת בעזרת מייל/שם משתמש וסיסמה, ולקבל גישה למסכים המתאימים להרשאה שלו (תלמיד/מורה/מנהל/אדמין).	מהות היכולת

פירוט	נושא
• הצגת מסך התחברות עם שדות מייל/שם משתמש וסיסמה. • בדיקת שדות ריקים בצד לקוח. • הצפנת הסיסמה לפני שליחה לשרת. • שליחת בקשת התחברות לשרת (כולל סוג הלקוח אם צריך). • קבלת תשובה מהשרת (התחברות הצליחה / סיסמה שגויה / משתמש לא מאושר). • מעבר למסך הראשי המתאים לפי ההרשאה (מסך תלמיד / מסך מורה וכו'). • טיפול בשגיאה והצגת הודעה מתאימה.	אוסף היכולות / פעולות הנדרשות למימוש
• טופס GUI של התחברות ב-PySide6 qml • מודול תקשורת עם השרת. • מודול הצפנה בצד הלקוח (לסיסמאות).	אובייקטים נחוצים

התנתקות מהמערכת

פירוט	נושא
התנתקות מהמערכת (Logout) - צד לקוח	שם היכולת
מאפשרת לכל משתמש לסיים את השימוש במערכת, לנקות את נתוני ההתחברות בצד הלקוח, ולחזור למסך ההתחברות הראשי.	מהות היכולת
• הצגת כפתור התנתקות במסך הראשי של המשתמש. • ניקוי נתוני המשתמש המחובר בצד הלקוח. • סגירת מצב ההתחברות המקומי. • מעבר חזרה למסך ההתחברות. • במקרה של תקלה או ניתוק מהשרת, המשתמש עדיין חוזר למסך ההתחברות.	אוסף היכולות / פעולות הנדרשות למימוש
• כפתור התנתקות בממשק המשתמש. • אובייקט בצד הלקוח ששומר את פרטי המשתמש המחובר. • מסך התחברות.	אובייקטים נחוצים

יכולות למשתמשים בעלי הרשאת תלמיד:

צפייה בכיתות, מטלות, חומרי לימוד, הודעות וציונים:

פירוט	נושא
צפייה בנתוני תלמיד – צד לקוח	שם היכולת
מאפשרת לתלמיד לראות את כל המידע הרלוונטי אליו במערכת: הכיתות שבהן הוא חבר פעיל, המטלות של אותן כיתות, חומרי הלימוד, הודעות מהמורה, ציונים ומשובים.	מהות היכולת
• שליחת בקשה לשרת לקבלת נתוני התלמיד המחובר. • קבלת רשימת הכיתות שבהן התלמיד חבר פעיל. • הצגת המטלות השייכות לכיתות של התלמיד, כולל תאריך הגשה, מצב הגשה וקבצים מצורפים. • הצגת חומרי לימוד שהמורה העלה לכיתה. • הצגת הודעות שנשלחו לכיתה על ידי המורה. • הצגת ציונים ומשובים עבור מטלות שנבדקו. • רענון הנתונים לאחר פעולות כמו הגשה, הצטרפות לכיתה או סימון הודעה כנקראה.	אוסף היכולות / פעולות הנדרשות למימוש
• מסך בית תלמיד. • רשימות תצוגה לכיתות, מטלות, חומרי לימוד, הודעות וציונים. • מודול תקשורת לקוח-שרת. • אובייקט משתמש מחובר.	אובייקטים נחוצים

הצטרפות ועזיבת כיתה:

פירוט	נושא
הצטרפות לכיתה בעזרת קוד ועזיבת כיתה – צד לקוח	שם היכולת
מאפשרת לתלמיד להצטרף לכיתה באמצעות קוד כיתה שקיבל מהמורה, וכן לעזוב כיתה שבה הוא כבר נמצא.	מהות היכולת
• הצגת שדה להזנת קוד כיתה. • שליחת קוד הכיתה לשרת יחד עם מזהה התלמיד.	אוסף היכולות / פעולות הנדרשות למימוש

פירוט	נושא
• קבלת תשובה מהשרת האם בקשת ההצטרפות נשמרה או נדחתה. • הצגת מצב שבו הבקשה ממתנה לאישור המורה. • שליחת בקשה לעזיבת כיתה קיימת. • רענון רשימת הכיתות לאחר הצטרפות או עזיבה.	
• מסך הצטרפות לכיתה. • שדה להזנת קוד כיתה. • קוד כיתה. • מזהה תלמיד. • כפתור שליחת בקשת הצטרפות. • כפתור עזיבת כיתה. • רשימת כיתות של התלמיד. • מודול תקשורת מול השרת.	אובייקטים נחוצים

הגשת עבודה חדשה

פירוט	נושא
הגשת עבודה למטלה – תלמיד	שם היכולת
מאפשרת לתלמיד להגיש עבודה למטלה פתוחה, באמצעות העלאת קובץ מהמחשב או כתיבת תשובה טקסטואלית. בנוסף, התלמיד יכול למחוק הגשה קיימת ולהגיש מחדש כל עוד המטלה עדיין פתוחה.	מהות היכולת
• בחירת מטלה מרשימת המטלות הקיימות. • בדיקה שהמטלה פתוחה להגשה. • בחירת קובץ מהמחשב או כתיבת תשובה טקסטואלית. • בדיקת תקינות בסיסית של ההגשה, כגון קובץ ריק או קובץ גדול מדי. • שליחת ההגשה לשרת יחד עם פרטי התלמיד והמטלה. • קבלת תשובה מהשרת האם ההגשה נשמרה או נדחתה.	אוסף היכולות / פעולות הנדרשות למימוש

פירוט	נושא
• עדכון סטטוס המטלה בצד הלקוח. • אפשרות למחיקת הגשה קיימת והגשה מחדש.	
• מסך מטלות תלמיד. • מסך או חלון הגשת עבודה. • דיאלוג בחירת קובץ. • שדה כתיבת תשובה טקסטואלית. • מזהה תלמיד. • מזהה מטלה. • מודול תקשורת לקוח-שרת.	אובייקטים נחוצים

יכולות למשתמשים בעלי הרשאת מורה:

ניהול הקורסים והמטלות (הוספת משימה, מחיקת משימה, עדכון משימה, צפייה במשימות)

פירוט	נושא
ניהול קורסים ומטלות – מורה	שם היכולת
מאפשרת למורה ליצור כיתות, לפרסם מטלות, לצפות בתלמידים המשויכים לכיתה, ולאשר או להסיר תלמידים לפי בקשות הצטרפות שנשלחו בעזרת קוד כיתה.	מהות היכולת
• הצגת רשימת הכיתות של המורה לאחר ההתחברות. • יצירת כיתה חדשה עם שם ותיאור. • הצגת קוד הצטרפות לכיתה לצורך שיתוף עם תלמידים. • הצגת רשימת תלמידים ובקשות הצטרפות לכיתה. • אישור, דחייה, השבתה או הסרה של תלמיד מכיתה. • יצירת מטלה חדשה הכוללת שם, תיאור, תאריך הגשה וקובץ הנחיות במקרה הצורך. • סגירת מטלה כדי למנוע הגשות נוספות. • מחיקת מטלה או כיתה לפי הצורך. • שליחת הפעולות לשרת וקבלת תשובת הצלחה או שגיאה.	אוסף היכולות / פעולות הנדרשות למימוש

נושא	פירוט
אובייקטים נחוצים	• מסך בית מורה. • רשימת כיתות. • קוד כיתה. • רשימת תלמידים ובקשות הצטרפות. • טופס יצירת כיתה. • טופס יצירת מטלה. • מודול תקשורת לקוח-שרת.

שליחת ציון לתלמיד אחרי בדיקה

נושא	פירוט
שם היכולת	עריכת ציון AI והוספת משוב – מורה
מהות היכולת	מאפשרת למורה לעבור על הציון שניתן ע"י מנועי ה-AI, לשנות אותו במידת הצורך ולהוסיף משוב אישי לתלמיד.
אוסף היכולות / פעולות הנדרשות למימוש	• הצגת רשימת ההגשות למטלה מסוימת, כולל ציון ה-AI • אפשרות לשנות את הציון ולהקליד משוב טקסטואלי. • שליחת הציון הסופי והמשוב לשרת. • עדכון המידע בצד הלקוח (שינוי הציון, הצגת "ציון סופי נשלח לתלמיד").
אובייקטים נחוצים	• טופס GUI לעריכת ציון ומשוב. • מודול תקשורת לעדכון השרת. • מודל נתונים להצגת ההגשות וציוניהן.

יכולות למשתמשים בעלי הרשאת מנהל:

ניהול בית הספר (אישור וחסימת משתמשים, צפייה בכיתות ובמטלות)

נושא	פירוט
שם היכולת	ניהול משתמשים ברמת בית ספר – מנהל
מהות היכולת	מאפשרת למנהל בית הספר לאשר משתמשים חדשים, לחסום משתמשים פעילים או לבטל חסימה של משתמשים השייכים לבית הספר שלו בלבד, דרך ממשק גרפי.
אוסף היכולות / פעולות הנדרשות למימוש	• הצגת רשימת משתמשים ממתינים לאישור (שהגדירו את בית הספר שלו בהרשמה). • כפתור אישור, חסימה או ביטול חסימה בהתאם לסטטוס המשתמש. • הצגת רשימת כל המשתמשים הפעילים בבית הספר (תלמידים ומורים). • אפשרות לחסום/להסיר משתמש קיים (לדוגמה: תלמיד שסיים ללמוד). • שליחת בקשות מתאימות לשרת (אישור, שינוי סטטוס, מחיקה). • עדכון הרשימות בצד הלקוח בהתאם לתשובת השרת.
אובייקטים נחוצים	• מסך GUI למנהל המציג טבלאות של משתמשים (ממתינים / פעילים). • מודול תקשורת ל-API ניהול משתמשים בשרת. • מודל נתונים בצד הלקוח המייצג משתמשים והרשאותיהם.

צפייה בפרטים על בית הספר:

נושא	פירוט
שם היכולת	צפייה בסטטיסטיקות ברמת בית הספר – מנהל
מהות היכולת	מאפשרת למנהל לקבל תמונת מצב על פעילות המערכת בבית הספר: מספר משתמשים, מספר כיתות, מספר מטלות וכמות הגשות.
אוסף היכולות / פעולות הנדרשות למימוש	• שליחת בקשה לשרת לקבלת סטטיסטיקות לבית ספר מסוים (על פי מזהה בית ספר של המנהל). • קבלת נתונים מסוכמים מהשרת • הצגת הנתונים בצורה גרפית • רענון נתונים בלחיצת כפתור "רענן".

פירוט	נושא
• מסך מעוצב להצגת כל הסטטיסטיקות על בית הספר בצורה גרפית • מודול תקשורת לשליפת הדוחות מהשרת. • מודל נתונים לסיכומי סטטיסטיקה בצד הלקוח.	אובייקטים נחוצים

יכולות למשתמשים בעלי הרשאת אדמין:

ניהול המערכת כולה (מחיקת או שינוי הרשאה של כל משתמש, צפייה בכל הפרטים על המערכת המשתמשים הקורסים והמשימות)

פירוט	נושא
ניהול גלובלי של בתי ספר ומשתמשים – אדמין	שם היכולת
מאפשרת למשתמש בעל הרשאת אדמין לנהל את כלל המערכת: יצירה/מחיקה של בתי ספר, צפייה בכל המשתמשים, וטיפול במקרים מיוחדים.	מהות היכולת
• צפייה ברשימת כל בתי הספר במערכת (שם, מזהה, מספר משתמשים). • יצירת בית ספר חדש והגדרת פרטים ראשוניים (שם, כתובת, איש קשר וכו'). • מחיקת בית ספר (אם אין בו משתמשים / לפי הגבלה). • צפייה ברשימת כל המשתמשים בכל המערכת, עם אפשרות חיפוש/סינון לפי בית ספר/תפקיד. • שינוי הרשאות למנהלים / מורים • שליחת כל הפעולות האלו לשרת ועדכון תצוגה בהתאם.	אוסף היכולות / פעולות הנדרשות למימוש
• מסך GUI של אדמין עם טבלאות/כרטיסיות (בתי ספר, משתמשים). • מודול תקשורת ל-API ניהול גלובלי בשרת. • מודל נתונים לייצוג בתי ספר ומשתמשים ברמה מערכתית.	אובייקטים נחוצים

תיאור היכולות של צד השרת

פירוט	נושא
ניהול הרשמה והתחברות	שם היכולת
מאפשר לשרת לקבל בקשות הרשמה והתחברות מהלקוחות, לבצע אימות, ולנהל יצירת חשבונות חדשים תוך שמירה על אבטחה.	מהות היכולת
• קבלת בקשת הרשמה מהלקוח והוצאת בדיקות תקינות. • בדיקה האם המשתמש קיים מראש במערכת. • קבלת סיסמה שעברה Hash בצד הלקוח ושמירתה בסיס הנתונים. • סימון משתמש כ"לא מאושר" עד אישורי מנהל בית ספר. • קבלת בקשות התחברות והשוואת ה-Hash של הסיסמה לערך השמור בסיס הנתונים. • בדיקת הרשאות ותפקיד. • החזרת תשובת התחברות ללקוח (OK / ERROR / WAITING_APPROVAL).	אוסף היכולות / פעולות הנדרשות למימוש
• טבלת Users ב-DB • מודול Hashing לסיסמאות. • מודול תקשורת • מודול הרשאות	אובייקטים נחוצים

פירוט	נושא
ניהול משתמשים והרשאות	שם היכולת
קובע אילו פעולות מותר לבצע לכל משתמש בהתאם לתפקידו (תלמיד / מורה / מנהל / אדמין).	מהות היכולת
• בדיקת הרשאות בכל בקשה שמגיעה לשרת.	אוסף היכולות / פעולות הנדרשות למימוש

פירוט	נושא
- מתן גישה לפונקציות על פי תפקיד המשתמש. - ניהול שינויי הרשאות ע"י אדמין או מנהל. - טיפול במשתמשים ממתנים לאישור מנהל. - אישור, חסימה, ביטול חסימה או מחיקה של משתמשים בהתאם להרשאת המשתמש המבצע.	
• שדה role בטבלת users	אובייקטים נחוצים

פירוט	נושא
ניהול קורסים	שם היכולת
אחראי על יצירת קורסים, מחיקתם, שיוך מורה לקורס וניהול תלמידים המצטרפים לקורס.	מהות היכולת
- קבלת בקשה ממורה ליצירת קורס חדש. - יצירת רשומת קורס חדשה ב-DB - שמירת שיוך מורה לקורס. - ניהול הצטרפות תלמידים לקורס באמצעות קוד כיתה, כולל שמירת בקשות ואישור או הסרה לפי צורך. - שליפת כל קורסי המורה / התלמיד. - טיפול בהרשאות (תלמיד לא יכול ליצור קורס, למשל).	אוסף היכולות / פעולות הנדרשות למימוש
- טבלת Courses - טבלת CourseMembers - מודול תקשורת. - מנגנון הרשאות.	אובייקטים נחוצים

נושא	פירוט
שם היכולת	ניהול מטלות
מהות היכולת	אחראי על יצירה, אחסון, שליפה וניהול של מטלות לכל קורס.
אוסף היכולות / פעולות הנדרשות למימוש	• קבלת בקשה ליצירת מטלה חדשה מהמורה. • יצירת רשומה בטבלת Assignments. • קישור מטלה לקורס המתאים. • הצגת המטלה לתלמידים במסך המטלות לאחר שליפת הנתונים מהשרת. • שליפה והצגת כל המטלות לפי קורס/משתמש.
אובייקטים נחוצים	• טבלת Assignments • מודול תקשורת ללקוח. • File Manager (לקבצים נלווים למטלה).

נושא	פירוט
שם היכולת	ניהול הגשות
מהות היכולת	מקבל הגשות מהתלמידים, כקובץ או כטקסט, שומר אותן בצורה מאובטחת, ומעביר אותן להמשך טיפול (AI).
אוסף היכולות / פעולות הנדרשות למימוש	• קבלת קובץ או תשובה טקסטואלית מהתלמיד דרך פרוטוקול התקשורת. • בדיקה שגודל הקובץ עומד בטווח המותר. • שמירת ההגשה בתיקייה "יעודית לפי קורס/מטלה/תלמיד. • יצירת רשומת הגשה ב-DB • שליחת המשימה להמשך בדיקה אוטומטית.

פירוט	נושא
• טבלת Submissions • מערך אחסון/תיקיות לפי מבנה מוגדר. • File Manager (לקבצים נלווים למטלה).	אובייקטים נחוצים

פירוט	נושא
מנוע בדיקת עבודות בעזרת AI	שם היכולת
מפעיל מספר מודלי AI על כל עבודה שהוגשה, מחשב ציון אוטומטי, ומפיק משוב.	מהות היכולת
• שליפת קובץ ההגשה מתיקיית הקבצים. • קריאה למנוע ה-AI • קבלת תוצאת ציון + ניתוח טקסטואלי. • שילוב תוצאות ממנועי AI זמינים לציון ומשוב סופי. • עדכון טבלת Grades • שליחת התוצאה למורה.	אוסף היכולות / פעולות הנדרשות למימוש
• טבלת Grades • API ל-AI • תהליך רקע (Thread).	אובייקטים נחוצים

פירוט	נושא
ניהול ציונים ומשובים	שם היכולת
שומר את ציוני ה-AI ואת הציון הסופי של המורה, ומנהל את המשוב שחוזר לתלמיד.	מהות היכולת
• קבלת ציון AI מהמודול הקודם. • שמירת ציון AI בצד. • קבלת עדכון מורה לשינוי ציון.	אוסף היכולות / פעולות הנדרשות למימוש

פירוט	נושא
• שמירת הציון הסופי והמשוב. • שליפה והצגה לתלמידים ומורים על פי בקשה.	
• טבלת Grades ו Feedback • מודול הרשאות. • מודול תקשורת.	אובייקטים נחוצים

פירוט	נושא
תקשורת מאובטחת	שם היכולת
אחראי על קבלת בקשות, TCP פענוח הצפנה, ושליחת תגובות.	מהות היכולת
• פתיחת שרת TCP שמקבל חיבורים. • קבלת הודעות מהלקוחות. • פענוח הודעות באמצעות AES • שליחת תגובה מוצפנת בחזרה. • טיפול בניתוקים timeouts, ושגיאות.	אוסף היכולות / פעולות הנדרשות למימוש
• socket server. • החלפת מפתחות בעזרת DH והצפנת הודעות בעזרת AES. • מודול הצפנה. • Threading לחיבורים מרובים.	אובייקטים נחוצים

פירוט	נושא
מנהל DB	שם היכולת
אחראי על כל הפעולות מול בסיס הנתונים: קריאה, כתיבה, עדכון, מחיקה.	מהות היכולת

נושא	פירוט
אוסף היכולות / פעולות הנדרשות למימוש	• התחברות ל־ DB בעת עליית השרת. • ביצוע שאילתות לכל יכולות המערכת. • מניעת SQL Injection באמצעות prepared statements • סגירת קישורים לאחר שימוש. • טיפול בשגיאות DB
אובייקטים נחוצים	• טבלאות בסיס הנתונים. • שאילתות Prepared Statements.

ארכיטקטורת הפרויקט

רכיבי החומרה

מחשב שרת

מחשב סטנדרטי עליו רץ שרת ה־ Python מסד הנתונים ומערכת הקבצים.
השרת מחובר לרשת ומאפשר ללקוחות להתחבר אליו.

מחשבי לקוח

מחשבים עליהם רצה אפליקציית ה־ GUI של המשתמשים (תלמידים/מורים/מנהלים).
מחשבים אלו צריכים חיבור לרשת לצורך תקשורת עם השרת.

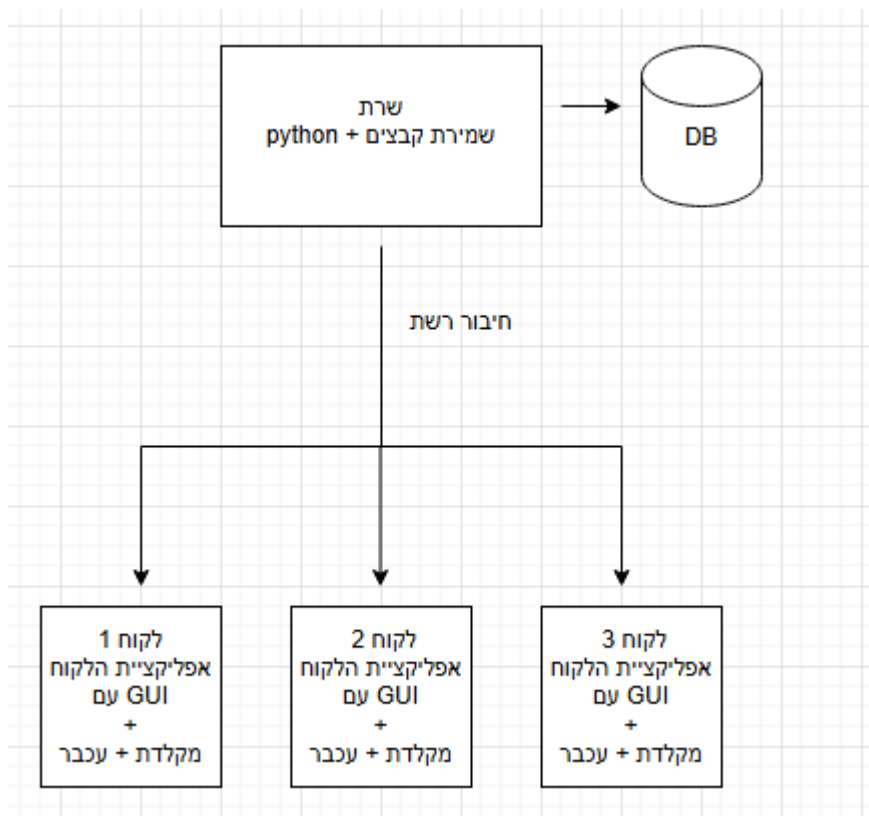
בנוסף במחשבים אלו צריך להיות מקלדת + עכבר

רכיב אחסון

אחסון מקומי או פנימי בשרת לצורך שמירת קבצי ההגשות ונתוני המערכת.

ציוד רשת

נתב / מתג וכרטיסי רשת המאפשרים תקשורת TCP בין השרת ללקוחות.



תיאור הטכנולוגיה בפרויקט

המערכת classify פותחה בשפת Python ומורכבת משרת המטפל בבקשות הלקוחות, בגישה למסד הנתונים ובהפעלת מנועי ה-AI.

השרת פועל בסביבת Windows ומשתמש במנגנון multi-threading כדי לטפל במספר לקוחות במקביל.

הלקוחות מפעילים אפליקציית Desktop שנבנתה באמצעות qml6 pyside, המהווה ממשק גרפי נגיש לתלמידים, מורים ומנהלים.

מסד הנתונים של המערכת הוא SQLite, ונמצא כולו בצד השרת לצורך שמירת משתמשים, קורסים, מטלות, הגשות וציונים.

התקשורת בין הלקוח לשרת מתבצעת בפרוטוקול TCP באמצעות Sockets, כדי להבטיח העברת מידע אמינה וסדורה.

קבצי ההגשות מאוחסנים במערכת הקבצים של השרת ומנוהלים על ידי קוד צד השרת.

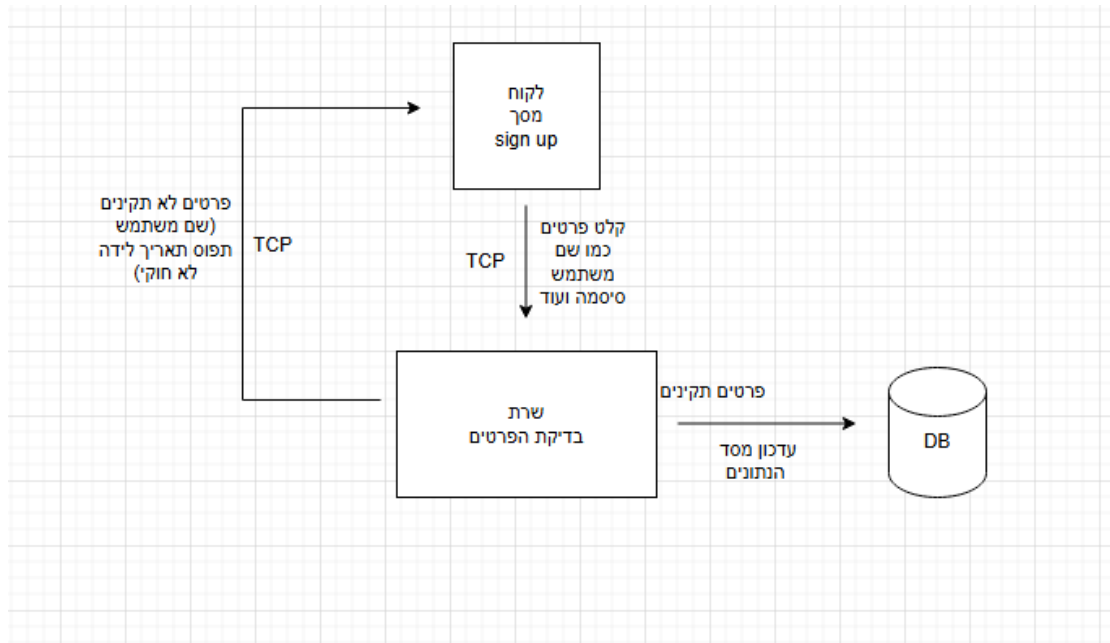
תהליך בדיקת העבודות נעשה על ידי מספר מודלי AI, הפועלים דרך קריאות API מהשרת.

בנוסף נעשה שימוש במנגנוני אבטחה כגון Hash לסיסמאות, החלפת מפתחות והצפנת התקשורת בין הלקוח לשרת.

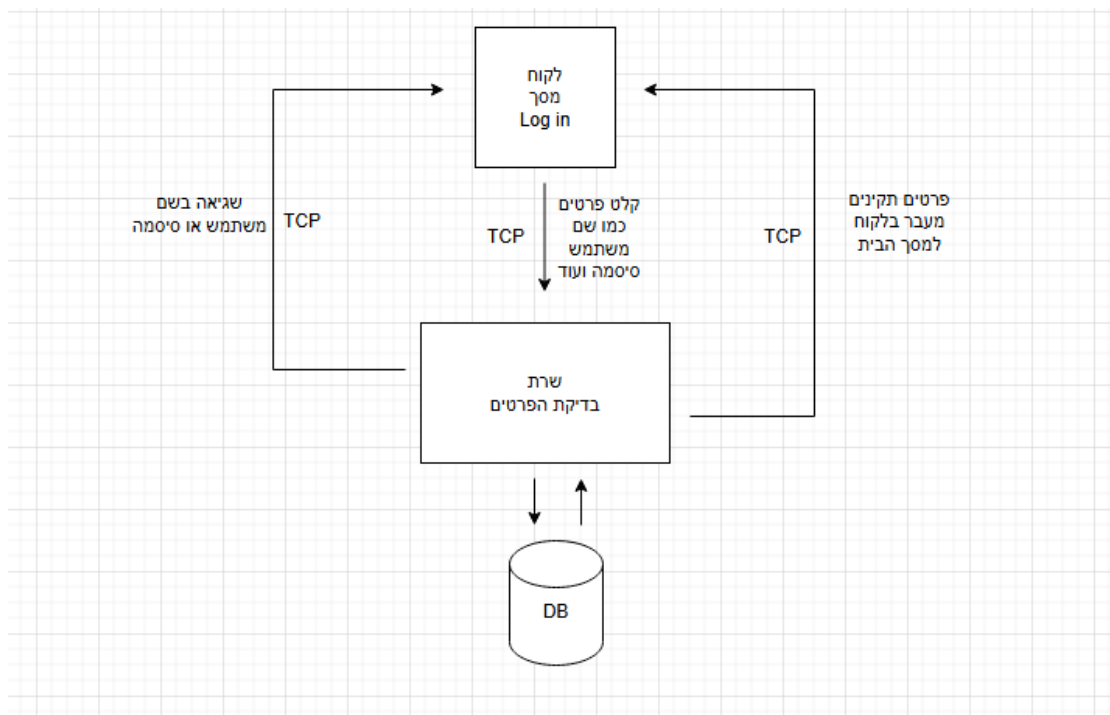
תיאור זרימת המידע במערכת

יכולות כלליות

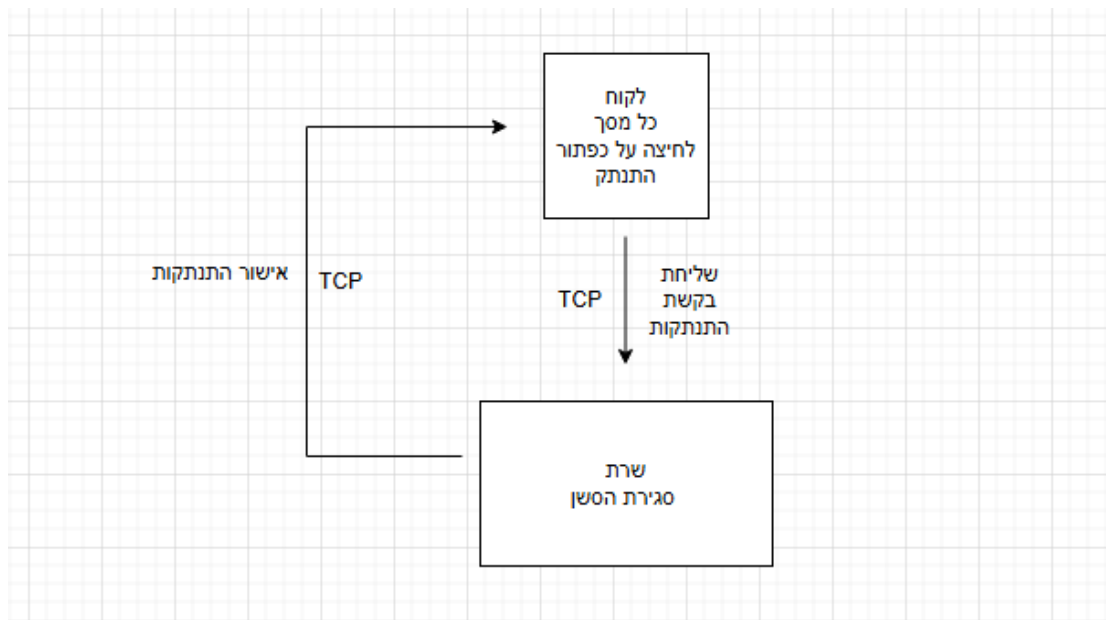
sign up יכולת



Log in יכולת

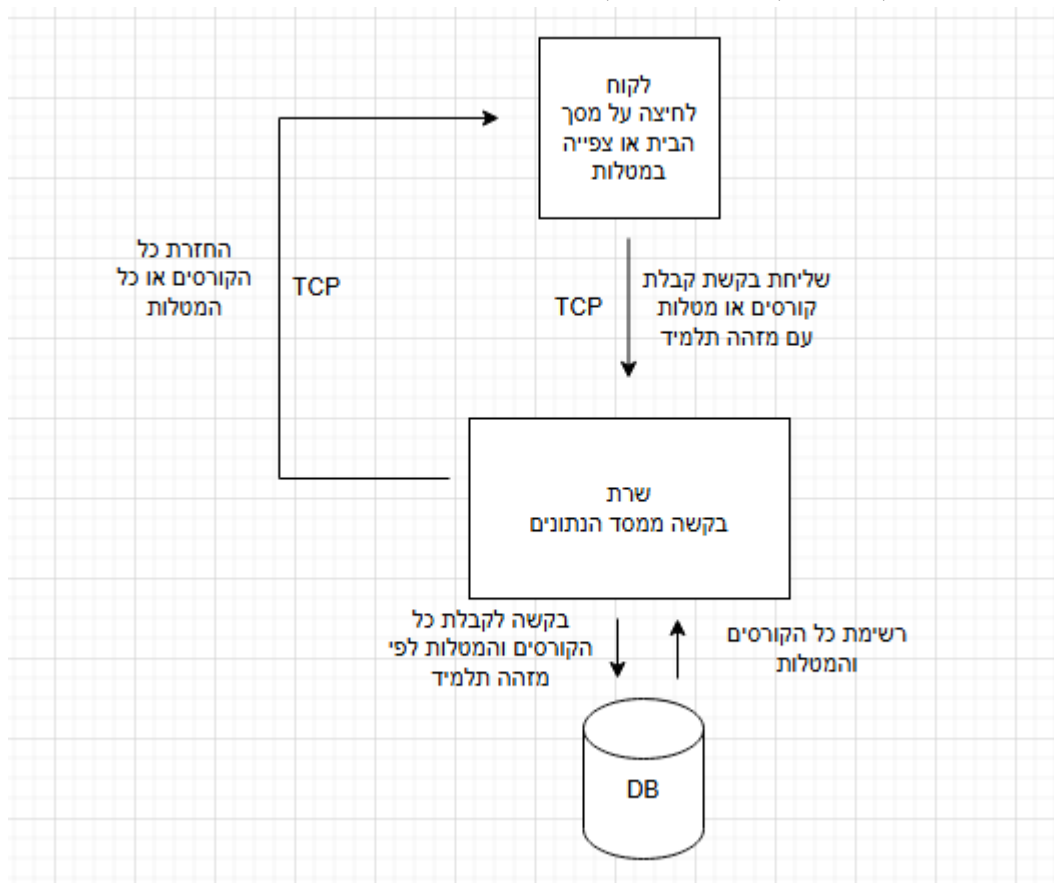


יכולת התנתקות

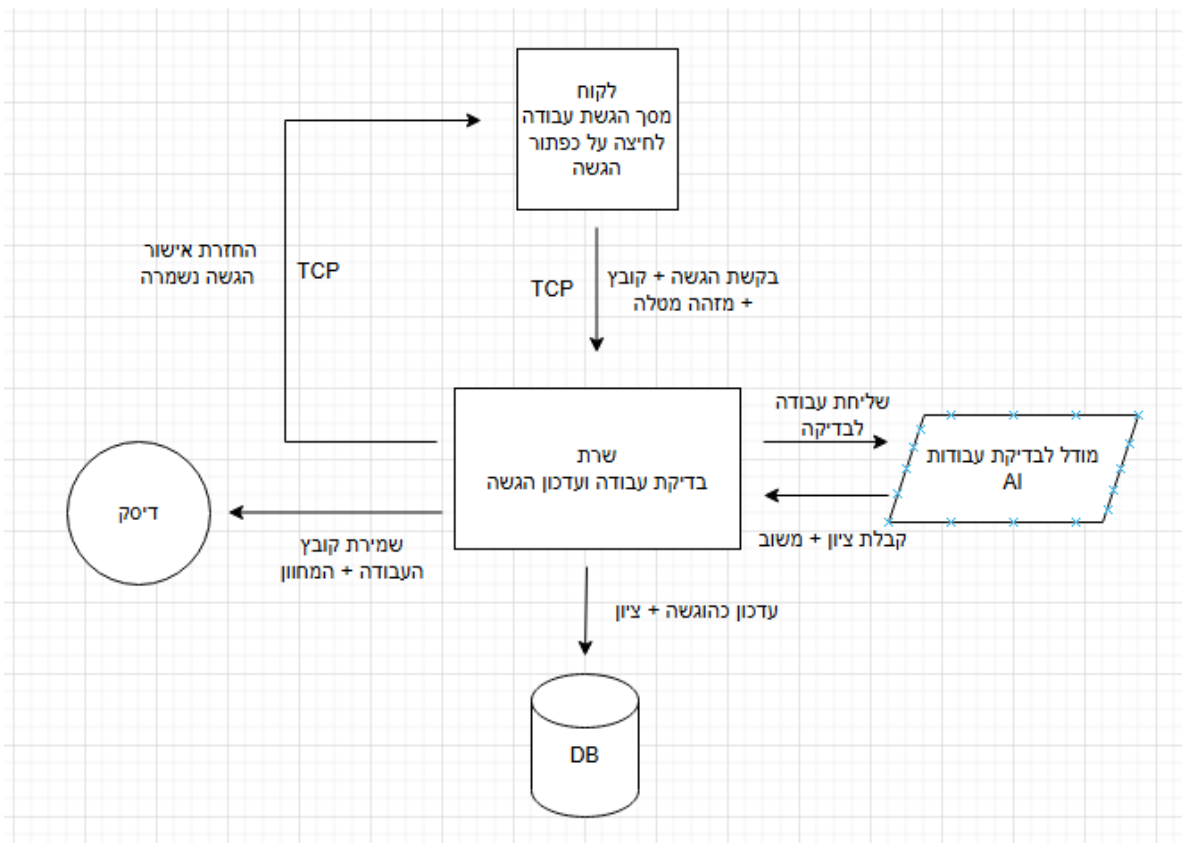


יכולות למשתמשים בעלי הרשאת תלמיד

צפייה בכיתות, מטלות, חומרי לימוד, הודעות וציונים

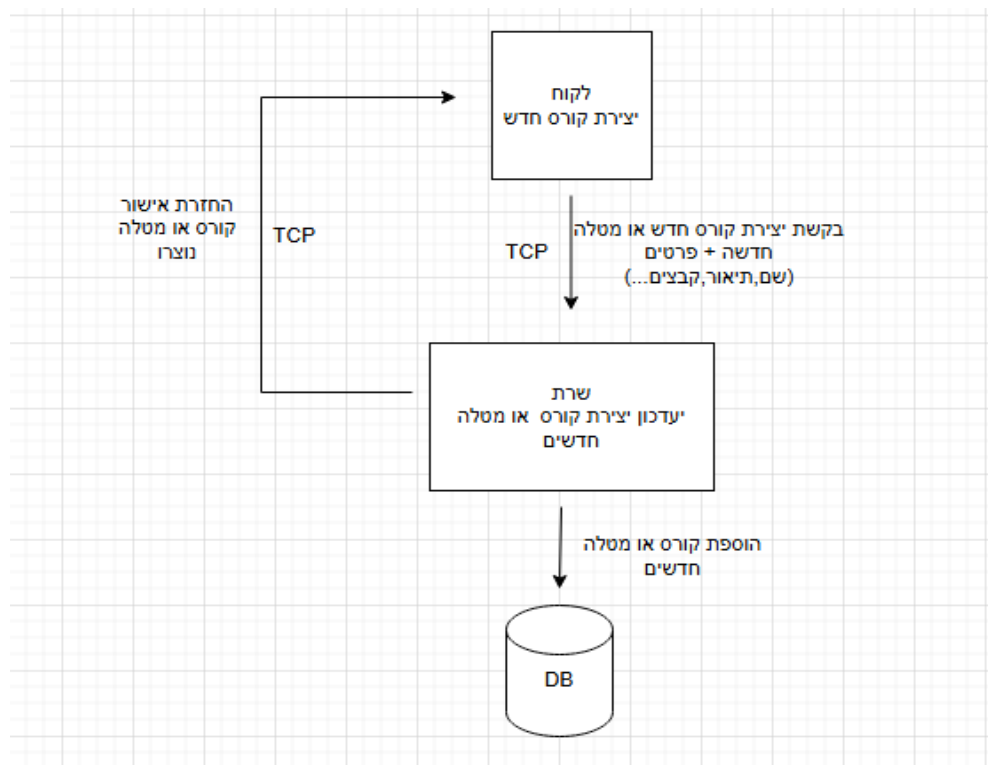


הגשת עבודה

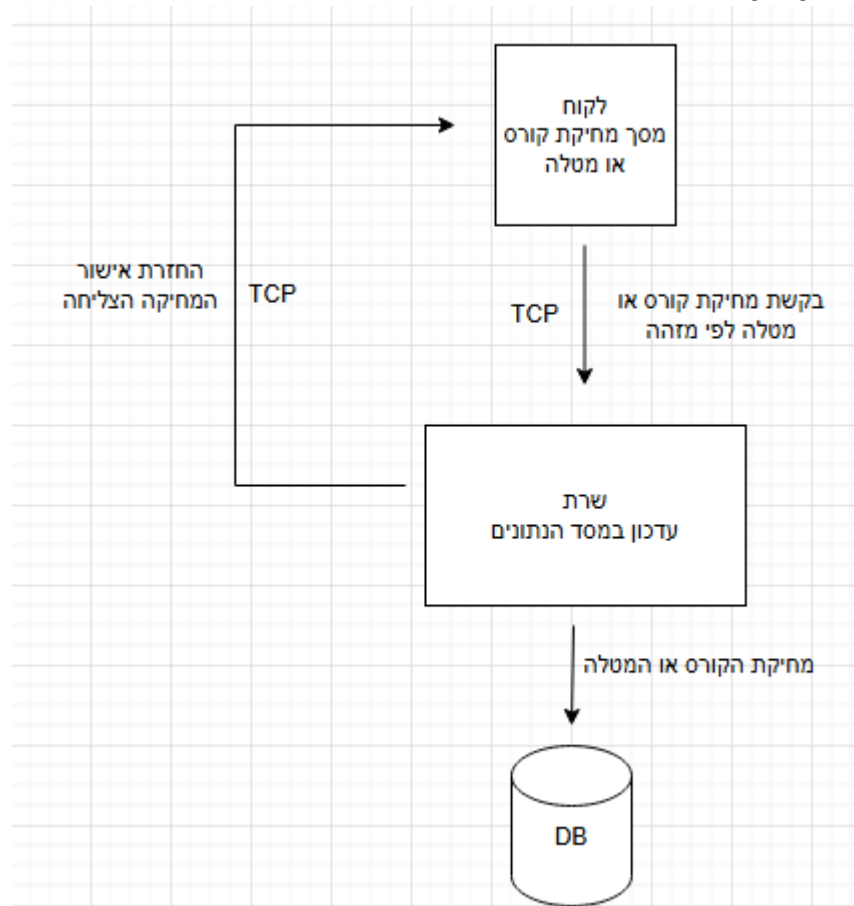


יכולות למשתמשים בעלי הרשאת מורה

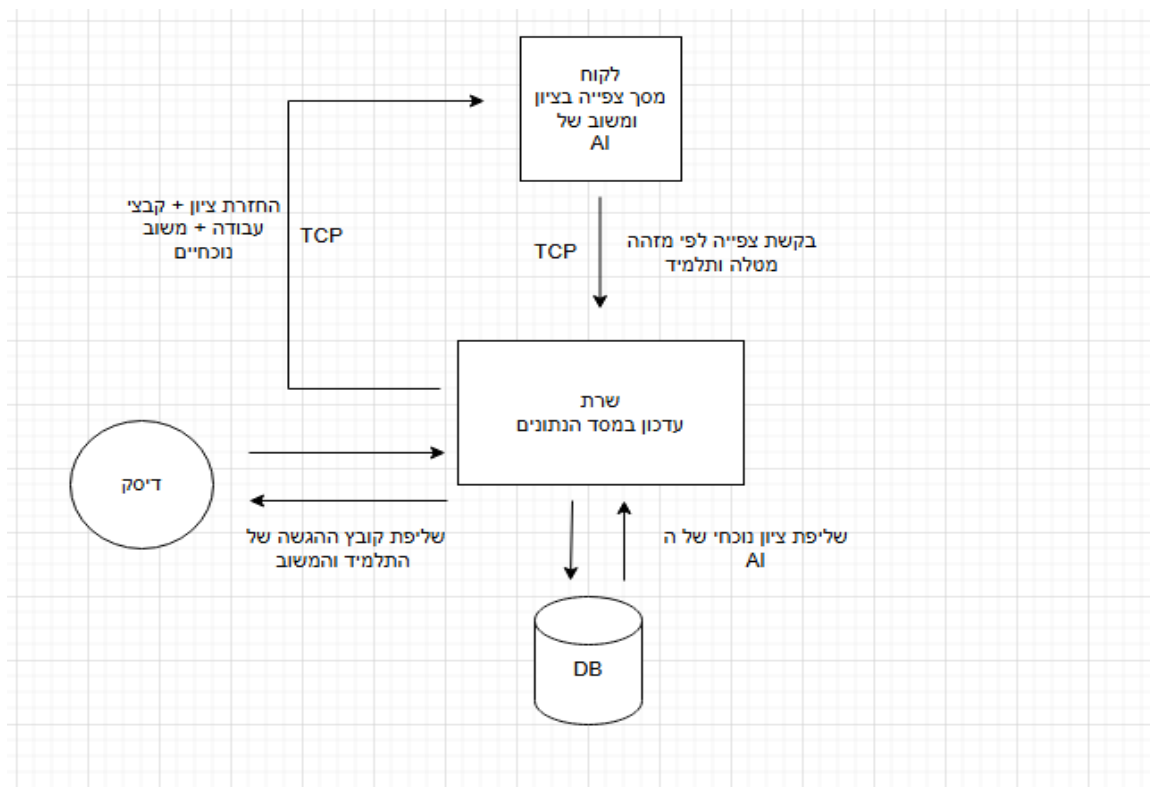
יצירת כיתה ומטלה



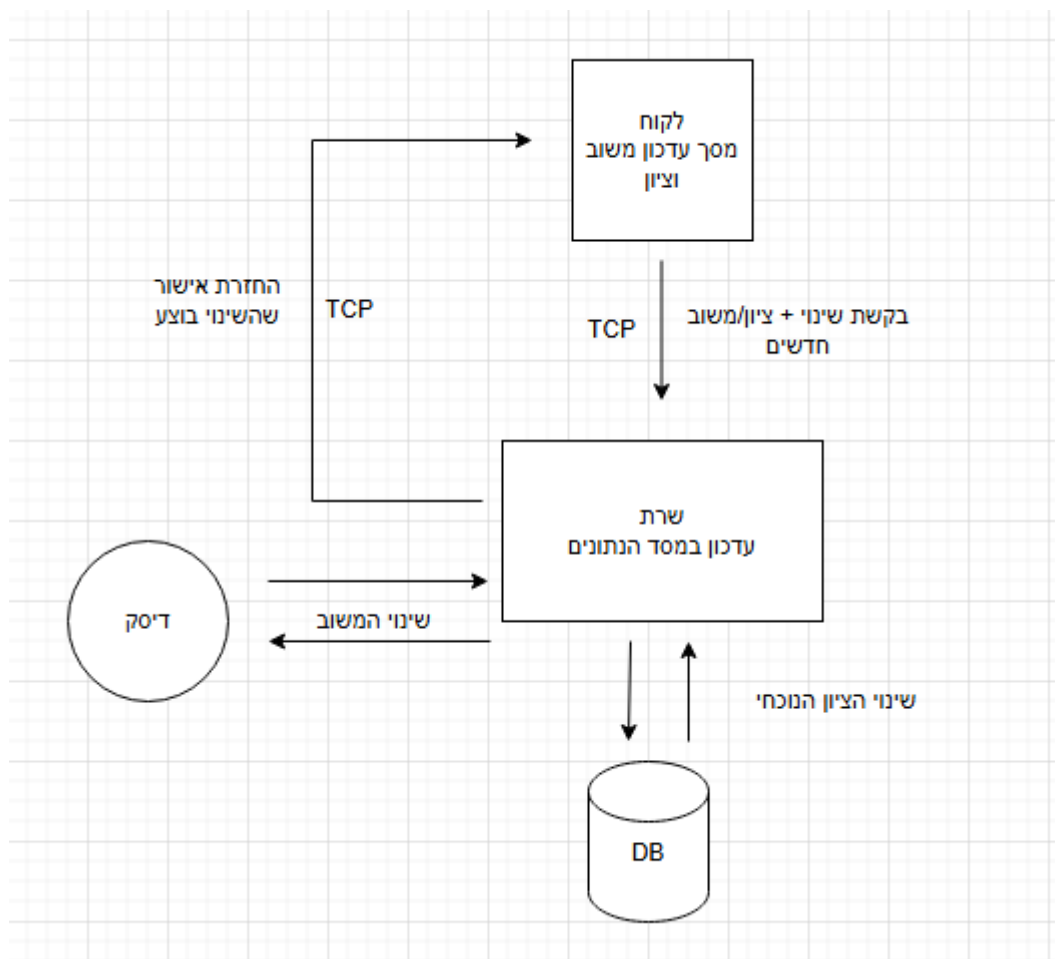
מחיקת קורס/מטלה



צפייה בציון ומשוב הAI ועבודת התלמיד.

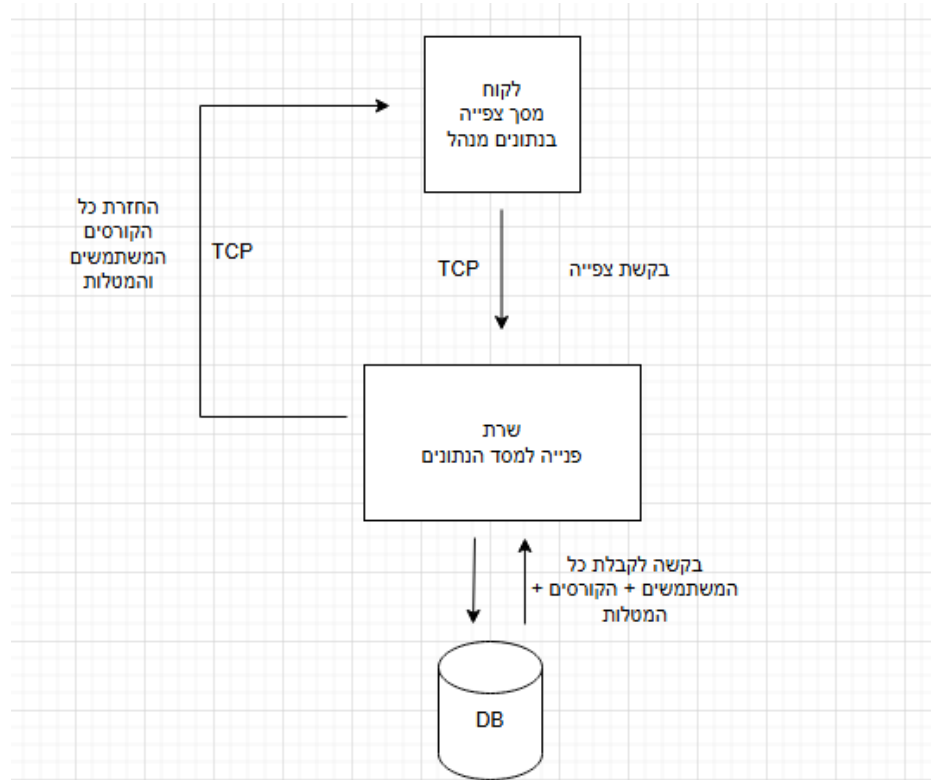


עדכון משוב/ציון של עבודת תלמיד.

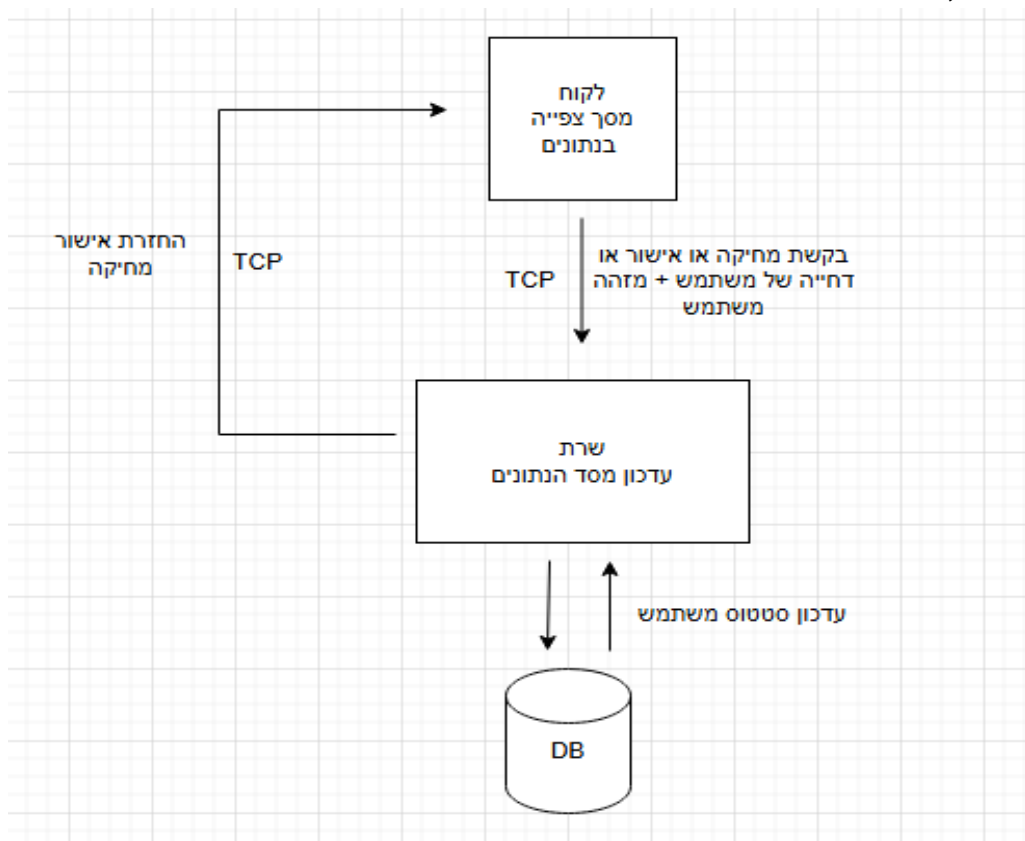


יכולות למשתמשים בעלי הרשאת מנהל:

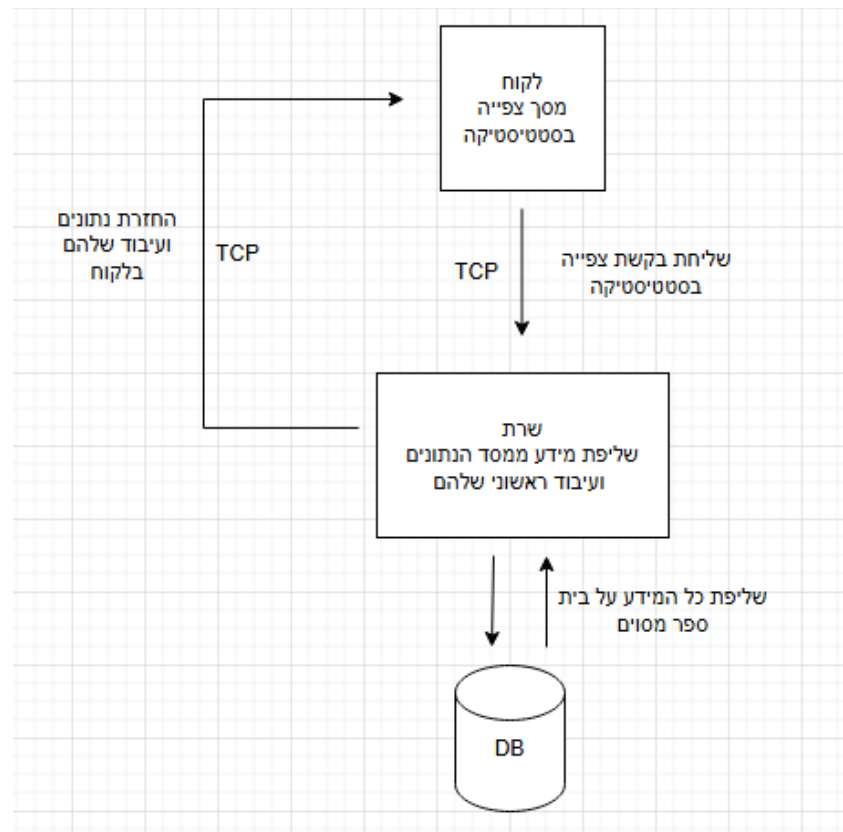
צפייה בכל המשתמשים הקיימים + המטלות + הקורסים



אישור, חסימה או ביטול חסימה של משתמשים

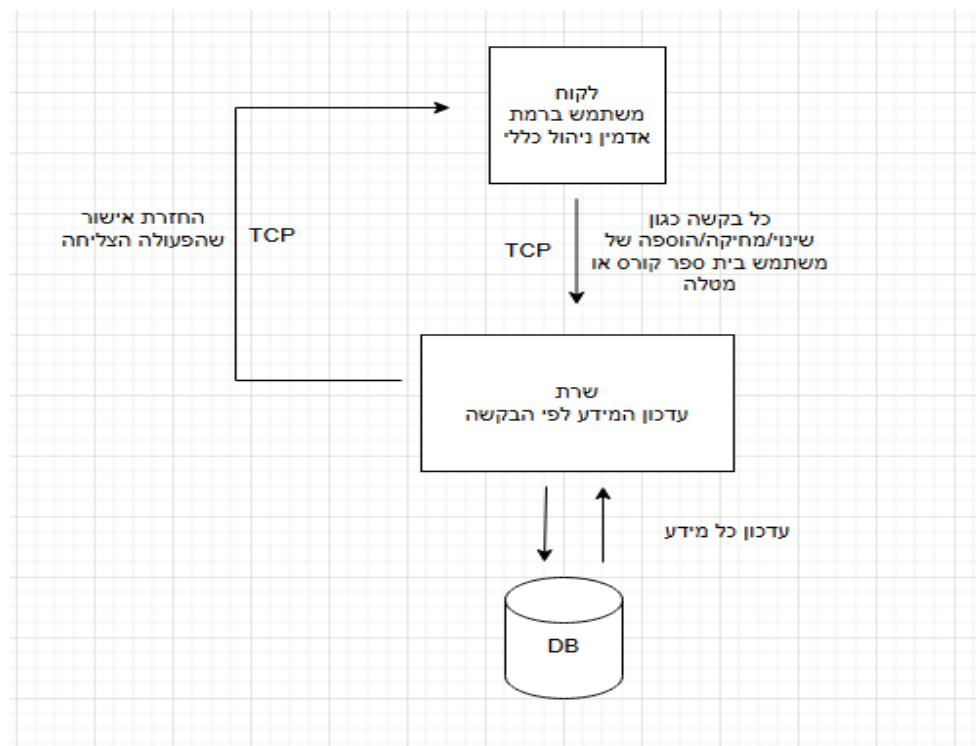


צפייה בסטטיסטיקות בית ספר



יכולות למשתמשים בעלי הרשאת אדמין:

ביצוע כל שינוי הוספה או מחיקה בנתונים של השרת.



האלגוריתמים המרכזיים בפרויקט

בפרויקט קיימים מספר תהליכים אלגוריתמיים בסיסיים, למשל ניהול הרשאות משתמשים, שמירת הגשות בבסיס הנתונים וניהול קורסים. תהליכים אלו מבוססים על פעולות סטנדרטיות של קריאה וכתיבה לבסיס הנתונים ולכן אינם מהווים קושי אלגוריתמי מיוחד.

האלגוריתם המרכזי והמורכב בפרויקט הוא תהליך בדיקת העבודות בעזרת מספר מנועי AI וחישוב ציון אוטומטי, ולכן פרק זה יתמקד בו.

תיאור אלגוריתם מרכזי – מנוע בדיקת עבודות בעזרת AI

1. ניסוח הבעיה האלגוריתמית

כאשר תלמיד מגיש עבודה, המערכת צריכה:

- לשלוח את העבודה למספר מנועי AI שונים.
- לקבל מכל מנוע ציון ומשוב טקסטואלי.
- לאחד את התוצאות לציון אחד סופי, שיהיה כמה שיותר אובייקטיבי ויציב.
- לשמור את התוצאות בבסיס הנתונים, כדי שהמורה יוכל לראות אותן, לערוך את הציון במידת הצורך, ולשלוח לתלמיד את הציון הסופי.

הבעיה האלגוריתמית היא איך לחבר כמה מקורות הערכה שונים (מנועי AI) לציון אחד הגיוני, ולנהל את כל התהליך בצורה מסודרת ואמינה.

2. פתרונות אפשריים

נבדקו כמה גישות אפשריות:

שימוש במנוע AI יחיד

הציון הסופי הוא הציון שמחזיר מודל אחד.

יתרון: מימוש פשוט.

חסרון: אם המודל טועה או מוטה – כל הציון שגוי.

חישוב ציון לפי Rubric קבוע

הגדרת קריטריונים קבועים (תוכן, שפה, מבנה וכו') ונתינת ניקוד לכל קריטריון.

יתרון: שקיפות גבוהה.

חסרון: דורש לבנות ידנית rubric לכל סוג משימה, פחות גמיש.

שילוב מספר מנועי (Ensemble) AI

הפעלת כמה מודלים שונים על אותה עבודה.

קבלת ציון מכל מודל וחישוב ציון משולב (למשל ממוצע).

יתרון: מפחית השפעה של טעות או הטיה של מודל אחד, ונותן ציון יותר יציב.

3. תיאור הפתרון שנבחר במערכת classify

בפרויקט בחרתי בגישת שילוב מספר מנועי AI זמינים וחישוב ציון משולב:

1. לאחר הגשה, השרת מוסיף את העבודה **לתור בדיקה**.

2. תהליך ברקע שולף עבודות מהתור, ובשביל כל עבודה:

- שומר את הקובץ במיקום קבוע בשרת.
- ממיר את העבודה לפורמט טקסטואלי מתאים למנועי ה-AI
- שולח את הטקסט **לכמה מנועי AI שונים** דרך קריאות API.

3. מכל מנוע AI מתקבל:

- ציון מספרי בתחום 0–100.
- משוב טקסטואלי (הערות, חוזקות/חולשות).

4. השרת:

- מחשב ציון AI משולב מתוך תוצאות המנועים הזמינים.
- שומר בבסיס הנתונים:
 - הציונים מכל מנוע בנפרד.
 - הציון המשולב כ-"ציון AI"
 - המשוב האוטומטי.

5. בממשק המורה:

- המורה רואה את ציון ה-AI ואת המשוב.
- יכול לשנות את הציון ולהוסיף משוב אישי.

6. לאחר שמירת העריכה של המורה:

- נשמר **ציון סופי** לתלמיד.
- התלמיד רואה את הציון הסופי ואת המשוב בממשק שלו.

4. נימוק לבחירת הפתרון

שימוש בכמה מנועי AI מאפשר לקבל **תמונה מאוזנת יותר** ולא להיות תלוי במודל יחיד. שילוב תוצאות ממספר מנועים הוא פתרון יציב יחסית, מפחית תלות במודל יחיד, ועדיין קל להסביר למשתמשים. עדיין נשמרת **שליטה מלאה למורה** – האלגוריתם נותן המלצה אוטומטית, אבל הציון הסופי הוא החלטת המורה. הפתרון מתאים למטרה של הפרויקט: להקל על עומס בדיקת עבודות, בלי לוותר על אחריות פדגוגית.

סביבת הפיתוח

פיתוח המערכת בוצע בסביבת עבודה מבוססת Windows, כאשר השרת והלקוח מפותחים בשפת Python. הפיתוח התבצע באופן מקומי, אך המערכת תוכננה כך שתוכל לפעול גם בסביבה מבוצרת (שרת מרוחק ומספר לקוחות).

כלי הפיתוח

כדי לפתח את המערכת נדרשו הכלים הבאים:

1. שפת תכנות

- **Python 3.10** ומעלה משמשת לפיתוח צד השרת וצד הלקוח. השפה נבחרה בזכות פשטות, תמיכה בריבוי תהליכים, ספריות רשת, עבודה נוחה עם SQLite ושילוב קל עם שירותי AI.

2. סביבת פיתוח (IDE)

- **PyCharm / VSCode** משמשות לכתיבת הקוד, דיבוג, ניהול חבילות וניהול סביבת העבודה הווירטואלית.

3. ספריות Python מרכזיות

- **socket** למימוש פרוטוקול TCP.
- **threading** לטיפול במספר חיבורי לקוח במקביל ולהרצת בדיקות AI ברקע.
- **sqlite3** עבודה מול מסד הנתונים.
- **hashlib** לביצוע Hash לסיסמאות.
- **PySide6** ו-**QML** לפיתוח ממשק המשתמש הגרפי בלקוח.
- **openai, anthropic, google-gemini** לביצוע קריאות API למנועי ה-AI.

4. מודולים משלימים

- **json** פורמט המידע המועבר בין הלקוח לשרת.
- **os** לניהול תיקיות קבצי ההגשות.
- **logging** רישום פעולות ושגיאות בשרת לתוך קובץ.

סביבת מסד נתונים

SQLite

מסד נתונים קל משקל שמשולב בקובץ אחד על השרת. נבחר מכיוון שהפרויקט אינו דורש DB מורכב או שרת חיצוני. מאפשר פיתוח מהיר וקל על מחשב אחד.

תיאור פרוטוקול התקשורת

התקשורת בין הלקוח לשרת במערכת classify מתבצעת באמצעות פרוטוקול TCP, מעליו מוגדר פרוטוקול לוגי פנימי המבוסס על הודעות JSON.

כל הודעה הנשלחת בין הלקוח לשרת היא הודעת JSON מוצפנת. לפני תוכן ההודעה נשלח גודל ההודעה, כדי שהצד המקבל יוכל לדעת כמה בתים עליו לקרוא.

מבנה זה מאפשר שליחה וקבלה של מידע בצורה פשוטה, קריאה ומאובטחת.

מבנה כללי של הודעה

לכל הודעה במערכת יש מבנה קבוע:

- type – סוג הפעולה (למשל: LOGIN_REQUEST).
- שדות נוספים – כל הודעה כוללת את השדות הדרושים לפעולה, לדוגמה, email, password, user_id או course_id.
- status – בשלב תגובה בלבד: OK / ERROR.
- message – מידע נוסף או הסבר שגיאה.

דוגמה:

שרת → לקוח:

```
{"type": "LOGIN_REQUEST", "email": "test@test.com", "password": "hash_value"}
```

לקוח → שרת :

```
{status": "OK", "role": "TEACHER", "user_id": 7, "school_id": 1}
```

סוגי ההודעות ופירוט

הודעות כלליות:

שם ההודעה	מי שולח	למי	שדות עיקריים	תיאור
SIGNUP_REQUEST	לקוח	שרת	name, email, password_hash, role, school_id	בקשה להרשמת משתמש חדש.

שם ההודעה	מי שולח	למי	שדות עיקריים	תיאור
LOGIN_REQUEST	לקוח	שרת	email, password_hash	ניסיון התחברות למערכת.
DOWNLOAD_FILE_REQUEST	לקוח	שרת	file_path	בקשה להורדת קובץ מהשרת

הודעות של פעולות תלמיד:

שם ההודעה	מי שולח	למי	שדות עיקריים	תיאור
GET_STUDENT_DASHBOARD_REQUEST	תלמיד	שרת	student_id	קבלת נתוני התלמיד: כיתות, מטלות, חומרי לימוד והודעות
JOIN_COURSE_BY_CODE_REQUEST	תלמיד	שרת	student_id, class_code	בקשת הצטרפות לכיתה בעזרת קוד כיתה
LEAVE_COURSE_REQUEST	תלמיד	שרת	course_id, student_id	עזיבת כיתה
UPLOAD_SUBMISSION_REQUEST	תלמיד	שרת	assignment_id, student_id, file_name, file_content_b64	הגשת עבודה כקובץ או כטקסט
DELETE_STUDENT_SUBMISSION_REQUEST	תלמיד	שרת	assignment_id, student_id	מחיקת הגשה קיימת
MARK_MESSAGE_READ_REQUEST	תלמיד	שרת	student_id, message_id	סימון הודעה כנקראה
MARK_ALL_MESSAGES_READ_REQUEST	תלמיד	שרת	student_id	סימון כל ההודעות כנקראו

שם ההודעה	מי שולח	למי	שדות עיקריים	תיאור
MARK_GRADE_READ_REQUEST	תלמיד	שרת	student_id, submission_id	סימון ציון כנקרא

הודעות של פעולות מורה:

שם ההודעה	מי שולח	למי	שדות עיקריים	תיאור
GET_TEACHER_COURSES_REQUEST	מורה	שרת	teacher_id	בקשה לקבלת כל הקורסים של המורה.
CREATE_COURSE_REQUEST	מורה	שרת	name, description, teacher_id, school_id	יצירת קורס חדש
DELETE_COURSE_REQUEST	מורה	שרת	course_id	מחיקת קורס
GET_COURSE_ASSIGNMENTS_REQUEST	מורה	שרת	course_id	בקשה לקבלת מטלות בקורס
CREATE_ASSIGNMENT_REQUEST	מורה	שרת	course_id, title, description, due_date, ai_enabled	יצירת מטלה חדשה
SET_ASSIGNMENT_CLOSED_REQUEST	מורה	שרת	assignment_id, is_closed	סגירה או פתיחה של מטלה להגשות

שם ההודעה	מי שולח	למי	שדות עיקריים	תיאור
DELETE_ASSIGNMENT_REQUEST	מורה	שרת	assignment_id	מחיקת מטלה
GET_COURSE_STUDENTS_REQUEST	מורה	שרת	course_id	קבלת תלמידים ובקשות הצטרפות לקורס
UPDATE_COURSE_MEMBER_STATUS_REQUEST	מורה	שרת	course_id, student_id, member_status	שינוי סטטוס תלמיד בקורס
DELETE_COURSE_MEMBER_REQUEST	מורה	שרת	course_id, student_id	הסרת תלמיד מקורס
GET_SUBMISSIONS_FOR_ASSIGNMENT_REQUEST	מורה	שרת	assignment_id	קבלת הגשות למטלה
UPDATE_SUBMISSION_REVIEW_REQUEST	מורה	שרת	submission_id, final_score, teacher_feedback, updated_by	שמירת ציון סופי ומשוב מורה

הודעות של חומרי לימוד והודעות כיתה:

שם ההודעה	מי שולח	למי	שדות עיקריים	תיאור
GET_COURSE_MATERIALS_REQUEST	מורה	שרת	course_id	קבלת חומרי לימוד של קורס
CREATE_MATERIAL_REQUEST	מורה	שרת	course_id, title, description,	יצירת חומר

שם ההודעה	מי שולח	למי	שדות עיקריים	תיאור
			material_type, created_by	לימוד חדש
DELETE_MATERIAL_REQUEST	מורה	שרת	material_id	מחיקת חומר לימוד
SAVE_COURSE_MESSAGE_REQUEST	מורה	שרת	course_id, sender_id, subject, body, state	שליחת הודעה לקורס
GET_COURSE_MESSAGES_REQUEST	מורה	שרת	course_id	קבלת הודעות של קורס
DELETE_MESSAGE_REQUEST	מורה	שרת	message_id	מחיקת הודעה

הודעות של פעולות מנהל:

שם ההודעה	מי שולח	למי	שדות עיקריים	תיאור
GET_USERS_REQUEST	מנהל	שרת	school_id	בקשה לקבלת משתמשי בית הספר
APPROVE_USER_REQUEST	מנהל	שרת	user_id	אישור משתמש חדש
BLOCK_USER_REQUEST	מנהל	שרת	user_id	חסימת משתמש
UNBLOCK_USER_REQUEST	מנהל	שרת	user_id	ביטול חסימת משתמש

שם ההודעה	מי שולח	למי	שדות עיקריים	תיאור
GET_SCHOOL_COURSES_REQUEST	מנהל	שרת	school_id	קבלת קורסים של בית הספר
GET_SCHOOL_ASSIGNMENTS_REQUEST	מנהל	שרת	school_id	קבלת מטלות של בית הספר
UPDATE_SCHOOL_REQUEST	מנהל	שרת	school_id, name, address, contact_name, contact_email	עדכון פרטי בית ספר

הודעות של פעולות אדמין:

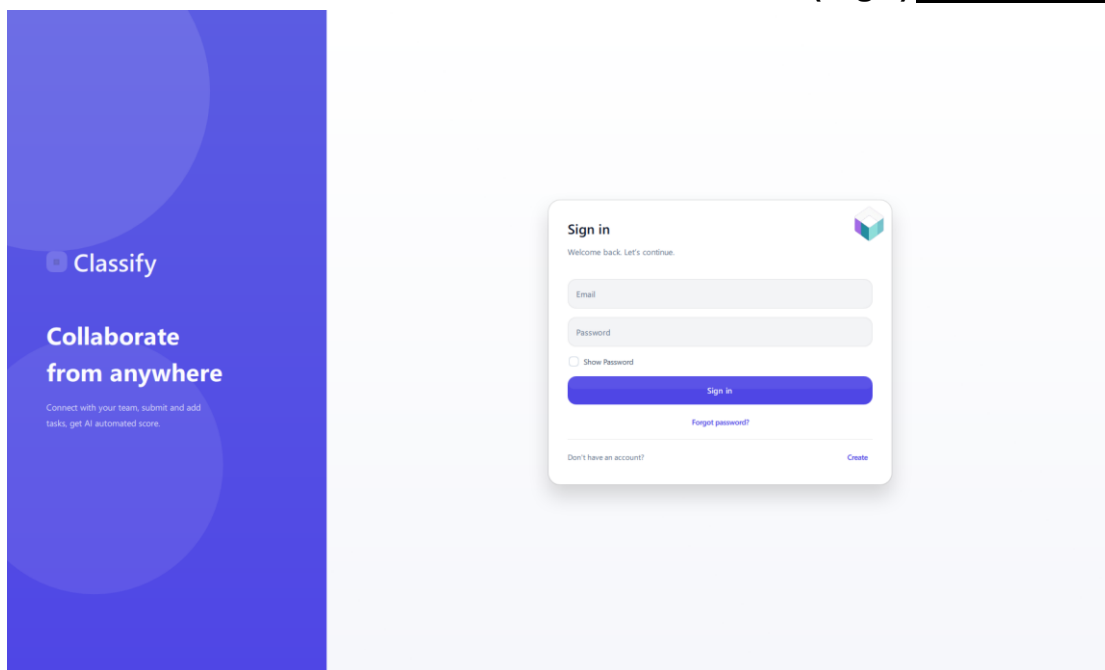
שם ההודעה	מי שולח	למי	שדות עיקריים	תיאור
GET_ADMIN_OVERVIEW_REQUEST	אדמין	שרת	אין	קבלת תמונת מצב מלאה של המערכת
ADMIN_CREATE_SCHOOL_REQUEST	אדמין	שרת	name, address, contact_name, contact_email	יצירת בית ספר
ADMIN_UPDATE_SCHOOL_REQUEST	אדמין	שרת	school_id, name, address, contact_name, contact_email	עדכון בית ספר
ADMIN_DELETE_SCHOOL_REQUEST	אדמין	שרת	school_id	מחיקת בית ספר

שם ההודעה	מי שולח	למי	שדות עיקריים	תיאור
ADMIN_CREATE_USER_REQUEST	אדמין	שרת	name, email, password, role, school_id, status	יצירת משתמש
ADMIN_UPDATE_USER_REQUEST	אדמין	שרת	user_id, name, email, role, school_id, status	עדכון משתמש
ADMIN_DELETE_USER_REQUEST	אדמין	שרת	user_id	מחיקת משתמש
ADMIN_UPDATE_COURSE_REQUEST	אדמין	שרת	course_id, name, description, teacher_id, school_id	עדכון קורס
ADMIN_UPDATE_ASSIGNMENT_REQUEST	אדמין	שרת	assignment_id, title, description, due_date, ai_enabled, is_closed	עדכון מטלה

מסכי המערכת

מסכים כללים:

מסך התחברות (Login)



שם המסך: מסך התחברות

תפקיד המסך: מאפשר למשתמש קיים (תלמיד / מורה / מנהל / אדמין) להיכנס למערכת באמצעות דוא"ל וסיסמה.

מידע שמוצג במסך:

- שדה דוא"ל
- שדה סיסמה
- כפתור "התחברות"
- כפתור "הרשמה" למשתמשים חדשים
- הודעות שגיאה (למשל: "פרטים שגויים", "החשבון ממתין לאישור מנהל")

פעולות אפשריות:

- הזנת פרטי התחברות ולחיצה על "התחברות"
- מעבר למסך ההרשמה בלחיצה על "עדיין לא רשום? הירשם"
- קבלת הודעת שגיאה במקרה של התחברות לא תקינה

מסך הרשמה (Sign Up)

שם המסך: מסך הרשמה

תפקיד המסך: מאפשר למשתמש חדש ליצור חשבון במערכת ולבחור את סוג ההרשאה שלו (תלמיד/מורה/מנהל), בצירוף שיוך לבית ספר.

מידע שמוצג במסך:

- שדות קלט: שם מלא, דוא"ל, סיסמה, בחירת תפקיד (תלמיד/מורה/מנהל), בחירת מזהה בית ספר
- כפתור "הרשמה"
- קישור חזרה למסך התחברות
- הודעות שגיאה (שדות חסרים, פורמט דוא"ל לא תקין, סיסמה קצרה מדי וכו')
- הודעת הצלחה/המתנה לאישור מנהל

פעולות אפשריות:

- מילוי פרטי הרשמה ולחיצה על "הרשמה"
- מעבר חזרה למסך התחברות
- קבלת הודעה "ההרשמה נקלטה, החשבון ממתין לאישור מנהל בית הספר"

מסך ראשי לאחר התחברות

לאחר התחברות מוצלחת, המשתמש עובר למסך ראשי שונה בהתאם להרשאה שלו:

תלמיד – מסך בית תלמיד

The screenshot shows the 'Classify' student dashboard for user Noam. The interface has a purple header with the logo, 'Welcome back, Noam', and 'Refresh' and 'Logout' buttons. A left sidebar contains a 'Student menu' with links to Dashboard, My classes, Assignments, Study materials, Grades, Messages, and a 'Join a class' button. The main area is titled 'Overview' and features a summary row with: 'My classes: 3', 'Open tasks: 1', 'Unread grades: 0', and 'Unread messages: 0'. Below this, the 'My classes' section lists three classes: 'Mathematics A' (Teacher Noam Levi, Code 12GBND), 'Biology' (Teacher Dana Cohen, Code 10M8NA), and 'History' (Uriya Ieshem, Code GOABES), each with an 'Open' button. A 'Next things to do' section shows a task 'temp long' due on 19/05/2026 at 23:59 with an 'Open' button. An 'Up next' section at the bottom left shows the same task. A 'Join class' button is located at the top right of the 'My classes' section.

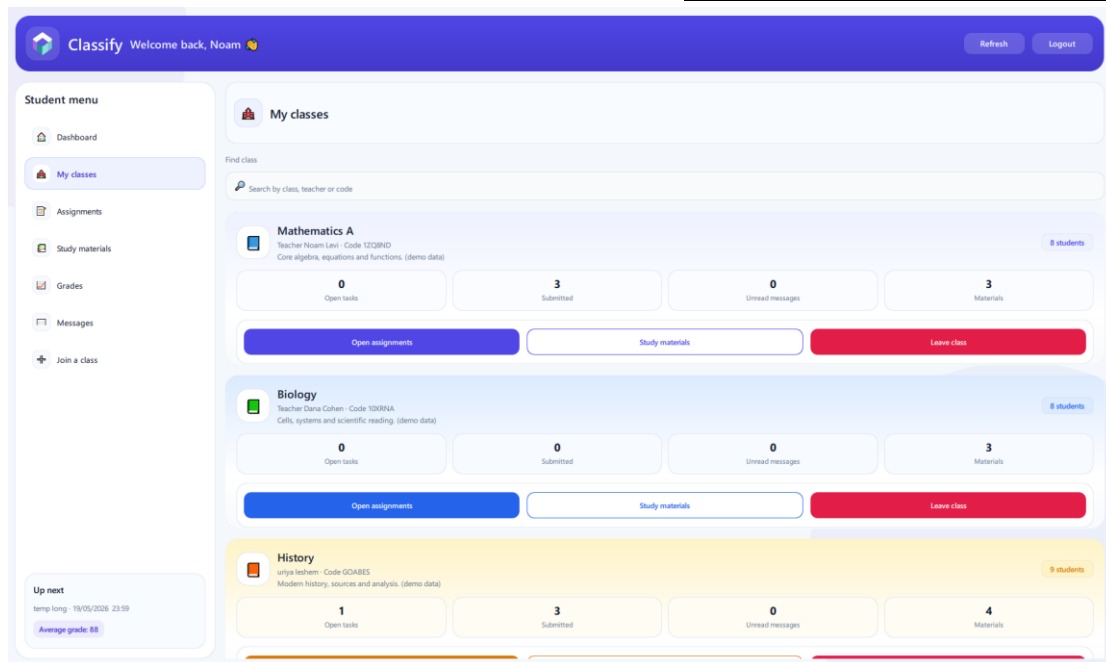
מורה – מסך בית מורה

The screenshot shows the 'Classify' teacher dashboard for user Uriya. The interface has a purple header with the logo, 'Welcome back, uriya', and 'Refresh' and 'Logout' buttons. A left sidebar contains a 'Quick actions' menu with links to Dashboard, Create Class, All Classes, Assignments, Students, Materials, Submission Review, Messages, and Reports. The main area is titled 'Overview' and features a summary row with: 'Active classes: 2', 'Total students: 20', 'Open assignments: 2', and 'Average grade: 69'. Below this, the 'Class snapshot' section lists two classes: 'English Writing' (Class code: 40X0SL) and 'History' (Class code: GOABES), each showing '10 students' and '0 open tasks'. A 'Awaiting approval' section shows '1 student'. A 'Need review' section shows '10 submissions'. A 'Top class by average' section shows 'English Writing' with an '81 avg'. A 'Most open workload' section shows 'History' with '2 open'. At the bottom, there are two sections: 'All assignments' with '11' and 'Materials' with '5'.

מנהל – מסך ניהול בית ספר

אדמין – מסך ניהול מערכת

מסך בית תלמיד – רשימת קורסים



שם המסך: מסך בית – תלמיד / רשימת קורסים
תפקיד המסך: מאפשר לתלמיד לראות את כל הקורסים אליהם שובץ.

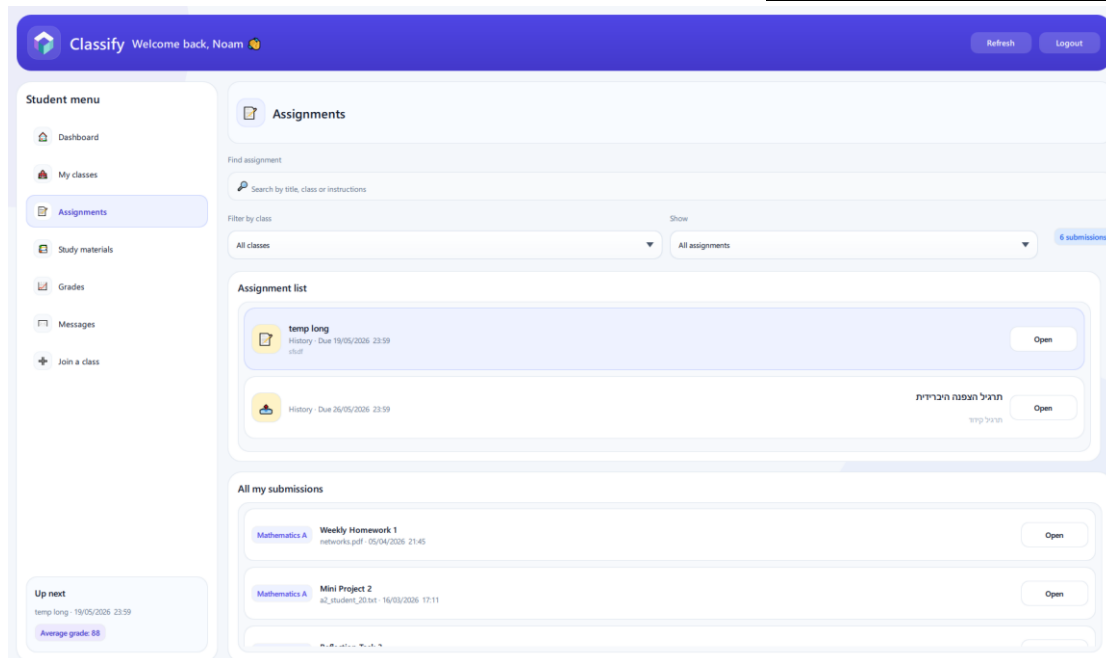
מידע שמוצג במסך:

- רשימת קורסים (שם קורס, שם המורה)
- עבור כל קורס – סטטוס כללי (מספר מטלות פתוחות, מספר מטלות שהוגשו)
- כפתור "כניסה לקורס" לכל קורס

פעולות אפשריות:

- בחירת קורס מהרשימה ומעבר למסך המטלות של אותו קורס
- רענון הרשימה (כפתור "רענן")

מסך מטלות בקורס לתלמיד



שם המסך: מסך מטלות בקורס – תלמיד
תפקיד המסך: מציג לתלמיד את כל המטלות בקורס מסוים, כולל סטטוס ההגשה.

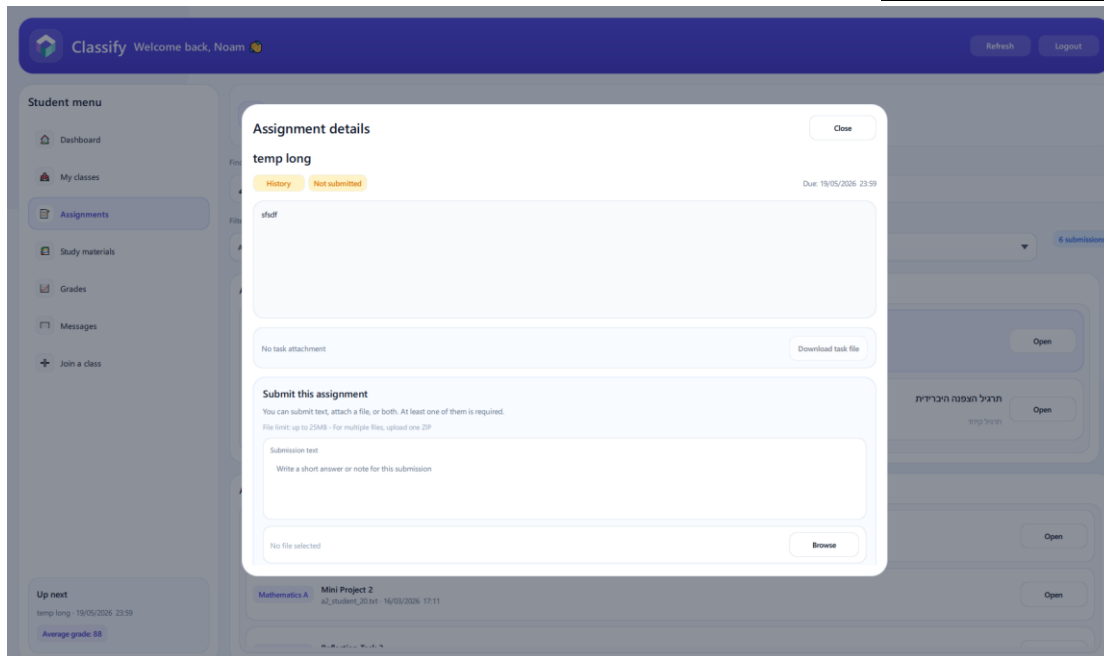
מידע שמוצג במסך:

- כותרת עם שם הקורס
- טבלת מטלות הכוללת:
 - שם המטלה
 - תאריך יעד
 - סטטוס (לא הוגשה / הוגשה / נבדקה)
 - ציון סופי (אם קיים)
- כפתור "הגש עבודה" לכל מטלה שעדיין פתוחה

פעולות אפשריות:

- בחירת מטלה וצפייה בפרטי המטלה
- מעבר למסך הגשת עבודה עבור מטלה פתוחה
- צפייה בציון ובמשוב עבור מטלה שנבדקה

מסך הגשת עבודה



שם המסך: מסך הגשת עבודה – תלמיד
תפקיד המסך: מאפשר לתלמיד לבחור קובץ מהמחשב ולהגישו כמענה למטלה.

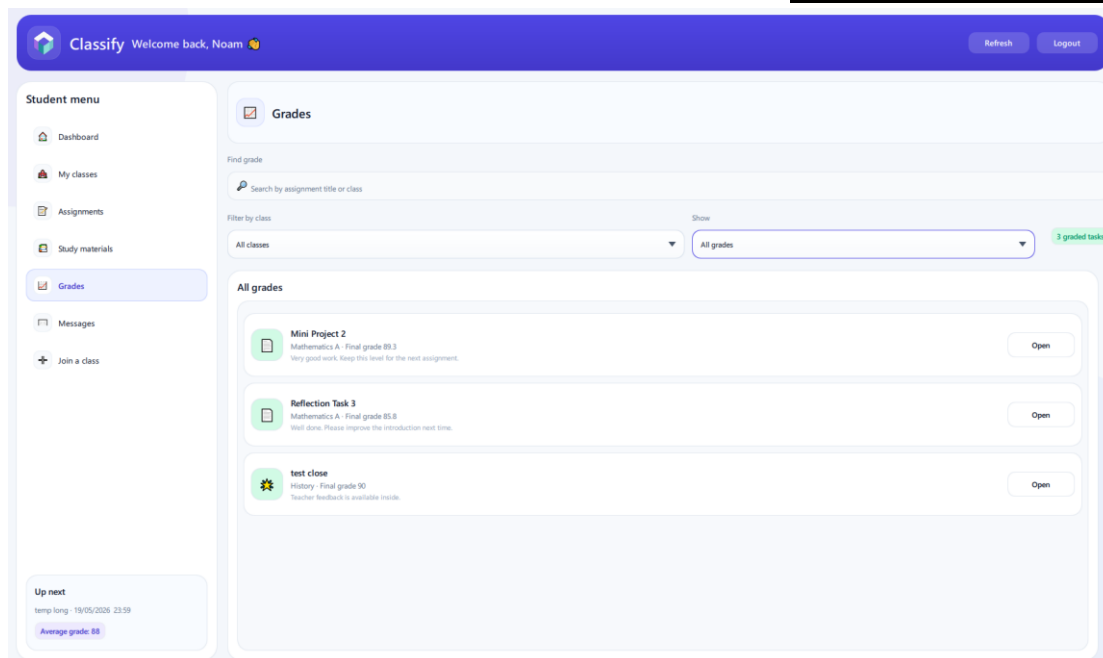
מידע שמוצג במסך:

- שם המטלה ותיאור קצר
- תאריך יעד
- שדה תצוגת שם הקובץ שנבחר
- כפתור "בחר קובץ"
- כפתור "הגש"
- הודעת סטטוס (למשל: "ההגשה נקלטה בהצלחה")

פעולות אפשריות:

- בחירת קובץ מהמחשב המקומי
- ביצוע הגשה לשרת
- צפייה בהודעת הצלחה או שגיאה

מסך צפייה בציונים ומשוב



שם המסך: מסך ציונים ומשובים – תלמיד
תפקיד המסך: מציג לתלמיד את הציון הסופי שקיבל על מטלה ואת המשוב מהמורה

מידע שמוצג במסך:

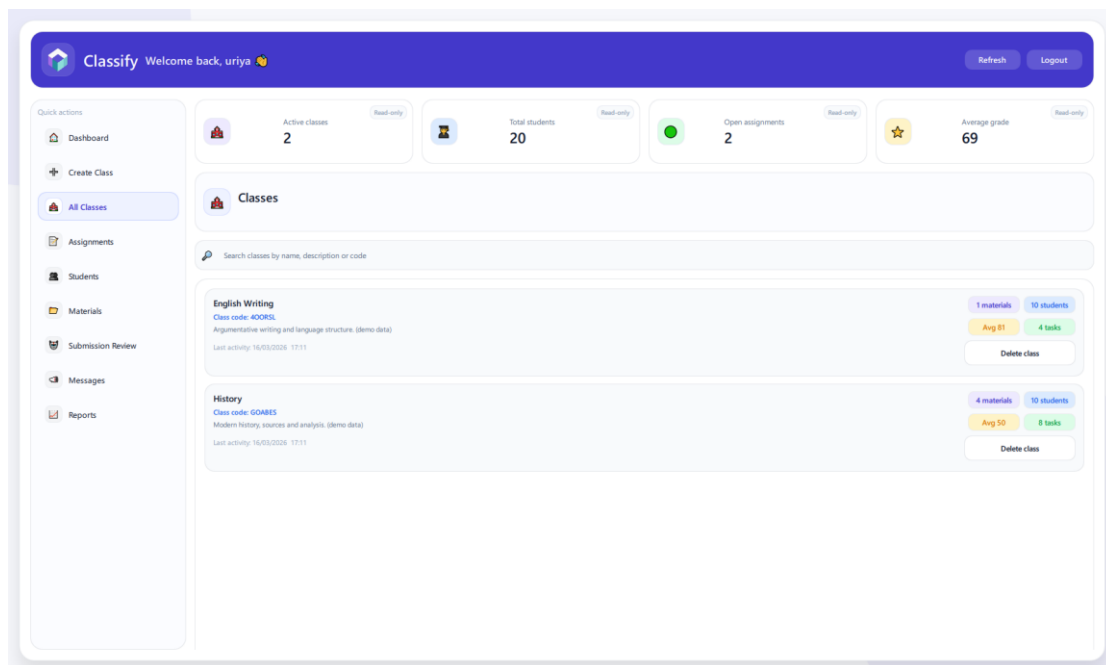
- שם הקורס ושם המטלה
- ציון סופי
- משוב אישי מהמורה

פעולות אפשריות:

- מעבר בין מטלות שונות באמצעות רשימה/נפתח
- צפייה בהיסטוריית ציונים ומטלות

מסכי מורה

מסך בית מורה – רשימת קורסים



שם המסך: מסך בית – מורה / ניהול קורסים
תפקיד המסך: מאפשר למורה לראות את כל הקורסים שבאחריותו ולנהל אותם.

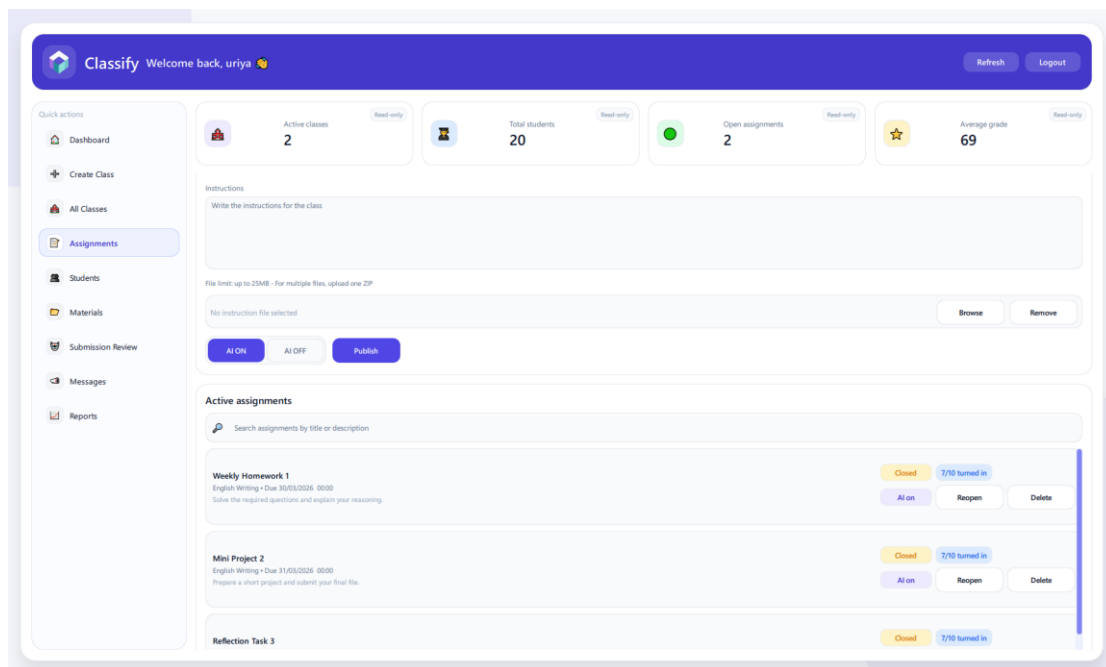
מידע שמוצג במסך:

- רשימת קורסים (שם, תיאור קצר)
- לכל קורס: מספר מטלות, מספר תלמידים
- כפתור "צפה בקורס"
- כפתור "יצירת קורס חדש"

פעולות אפשריות:

- כניסה לניהול קורס ספציפי
- יצירת קורס חדש
- מחיקת הקורס

מסך ניהול מטלות בקורס



שם המסך: מסך ניהול מטלות – מורה
תפקיד המסך: מאפשר למורה להוסיף, לערוך ולמחוק מטלות בקורס.

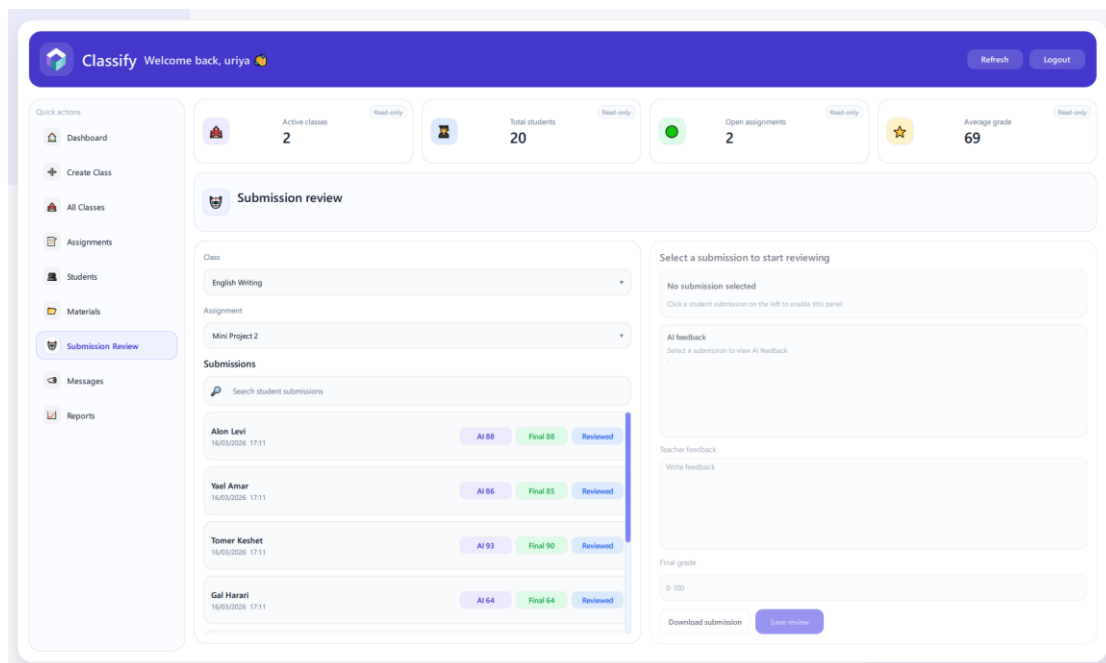
מידע שמוצג במסך:

- שם הקורס
- טבלת מטלות קיימות: שם, תאריך יעד, מספר הגשות
- כפתור "הוסף מטלה חדשה"
- כפתורי "מחיקה" לכל מטלה

פעולות אפשריות:

- יצירת מטלה חדשה עם שם, תיאור, תאריך יעד, קובץ מצורף אם צריך
- מחיקת מטלה
- מעבר למסך צפייה בהגשות עבור מטלה נבחרת

מסך רשימת הגשות למטלה



שם המסך: מסך הגשות – מורה
תפקיד המסך: מאפשר למורה לראות את כל הגשות התלמידים למטלה מסוימת, כולל מצב בדיקת ה-AI

מידע שמוצג במסך:

- שם הקורס ושם המטלה
- טבלת הגשות:
 - שם התלמיד
 - תאריך ושעת ההגשה
 - סטטוס בדיקת AI (ממתין/נבדק)
 - ציון AI (אם קיים)
 - ציון סופי (אם כבר הוזן)

פעולות אפשריות:

- פתיחת הגשה מסוימת לצפייה בקובץ
- מעבר למסך עריכת ציון ומשוב לתלמיד

מסך עריכת ציון ומשוב

Submission review

Class

English Writing

Assignment

Mini Project 2

Submissions

Search student submissions

Alon Levi
16/03/2026 17:11
AI 88 Final 88 Reviewed

Yael Amar
16/03/2026 17:11
AI 86 Final 85 Reviewed

Tomer Keshet
16/03/2026 17:11
AI 93 Final 90 Reviewed

Gal Harari
16/03/2026 17:11
AI 64 Final 64 Reviewed

Alon Levi review

a2_student_34.txt
AI score: 88 • Final: 88

AI feedback
Strong answer with clear organization and relevant examples

Teacher feedback
Good progress. Check the final section again.

Final grade
88

Download submission Save review

שם המסך: מסך עריכת ציון ומשוב – מורה
תפקיד המסך: מאפשר למורה לעבור על תוצאות ה-AI, לעדכן ציון סופי ולהוסיף משוב אישי.

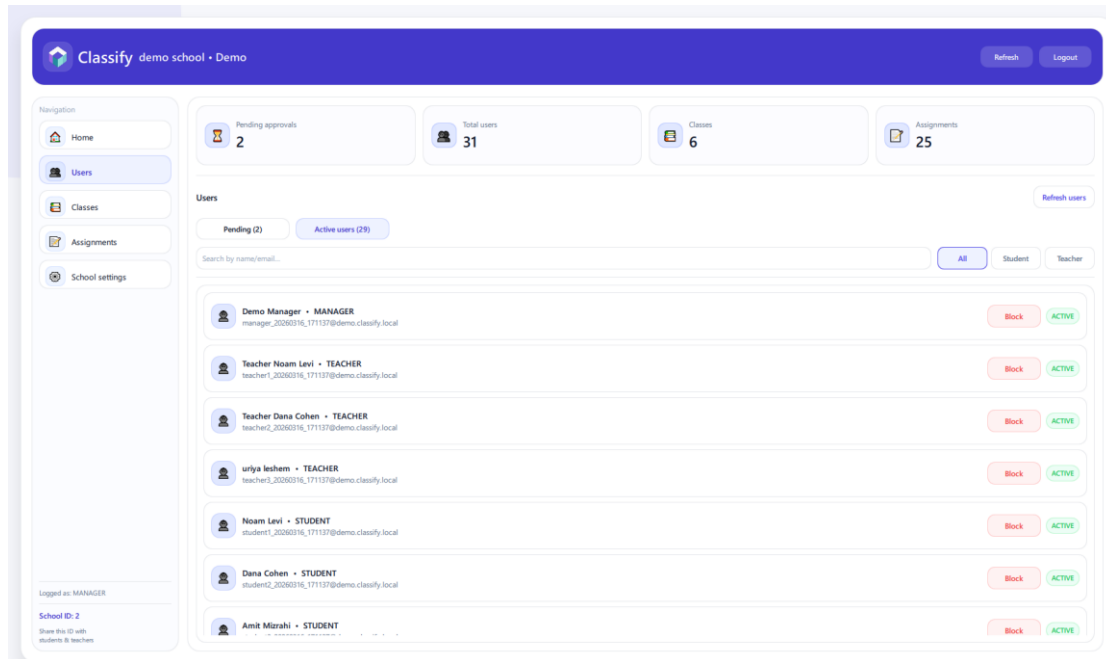
מידע שמוצג במסך:

- שם התלמיד, הקורס והמטלה
- ציון AI המשולב
- תקציר משוב AI (טקסט)
- שדה להקלדת ציון סופי
- שדה להקלדת משוב מהמורה
- כפתור "שמירת ציון ומשוב"

פעולות אפשריות:

- שינוי הציון שהציע ה-AI
- הוספת משוב אישי
- שמירת הציון והמשוב ושליחתם לתלמיד

מסך ניהול משתמשים בבית הספר



שם המסך: מסך ניהול משתמשים – מנהל

תפקיד המסך: מאפשר למנהל לאשר/לדחות משתמשים חדשים ולנהל משתמשים קיימים בבית הספר.

מידע שמוצג במסך:

- טבלת משתמשים ממתינים לאישור: שם, דוא"ל, תפקיד מבוקש
- טבלת משתמשים פעילים: שם, תפקיד, סטטוס (פעיל/חסום)
- כפתורי "אשר" למשתמשים ממתינים
- כפתורי "חסום" למשתמשים פעילים

פעולות אפשריות:

- אישור משתמש חדש
- דחיית בקשת משתמש
- חסימת משתמש קיים

מסך סטטיסטיקות בית ספר



שם המסך: מסך סטטיסטיקה – מנהל
תפקיד המסך: מציג למנהל תמונת מצב על פעילות המערכת בתוך בית הספר.

מידע שמוצג במסך:

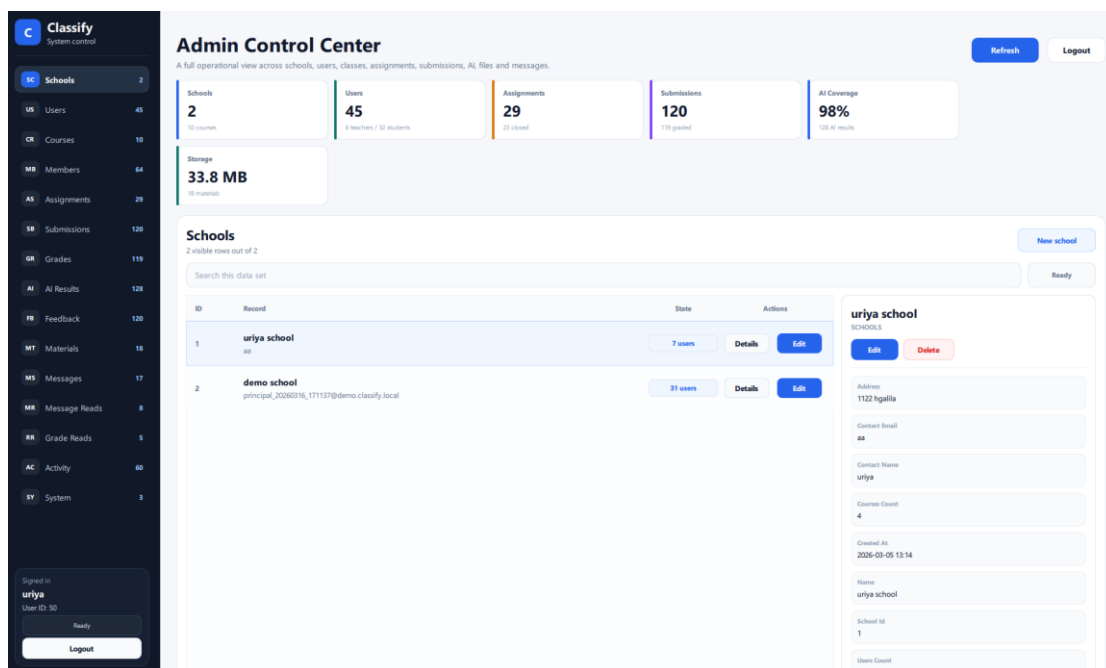
- מספר קורסים פעילים
- מספר תלמידים ומורים
- כמות משתמשים ממתינים לאישור
- כמות משימות פעילות

פעולות אפשריות:

- רענון הנתונים

מסכי אדמין

מסך ניהול בתי ספר



שם המסך: מסך ניהול בתי ספר – אדמין
תפקיד המסך: מאפשר לאדמין לראות את כל בתי הספר במערכת ולנהל אותם.

מידע שמוצג במסך:

- רשימת בתי ספר: שם, מזהה, מספר משתמשים

- כפתור "הוסף בית ספר"
- כפתורי "עריכה" / "מחיקה" לבית ספר

פעולות אפשריות:

- יצירת בית ספר חדש
- עריכת פרטי בית ספר
- מחיקת בית ספר (בכפוף לתנאים)

מסך ניהול משתמשים גלובלי

Admin Control Center
A full operational view across schools, users, classes, assignments, submissions, AI, files and messages.

Summary:
 Schools: 2 (10 courses)
 Users: 45 (6 teachers / 39 students)
 Assignments: 29 (23 closed)
 Submissions: 120 (119 graded)
 AI Coverage: 98% (138 AI results)
 Storage: 33.8 MB (18 materials)

Users
45 visible rows out of 45

ID	Record	State	Actions
4	uriya lesheim uriya@gmail.com / uriya school	PENDING	Details Edit
5	uriya lesheim test1@ / -	ACTIVE	Details Edit
6	uriya lesheim test2@ / uriya school	ACTIVE	Details Edit
7	uriya lesheim test3@ / -	PENDING	Details Edit
8	keren lesheim test4@ / uriya school	ACTIVE	Details Edit
9	uriya lesheim test5@ / -	ACTIVE	Details Edit
10	uriya lesheim test42@ / -	ACTIVE	Details Edit
11	uriya lesheim test5@ / -	ACTIVE	Details Edit

uriya lesheim
(users)

uriya@gmail.com / uriya school

Created At: 2026-02-09 12:19

Email: uriya@gmail.com

AI: 4

Memberships Count: 0

Name: uriya lesheim

Role: STUDENT

School id: 1

שם המסך: מסך ניהול משתמשים – אדמין

תפקיד המסך: מאפשר לאדמין לצפות בכל משתמשי המערכת ולשנות הרשאות במידת הצורך.

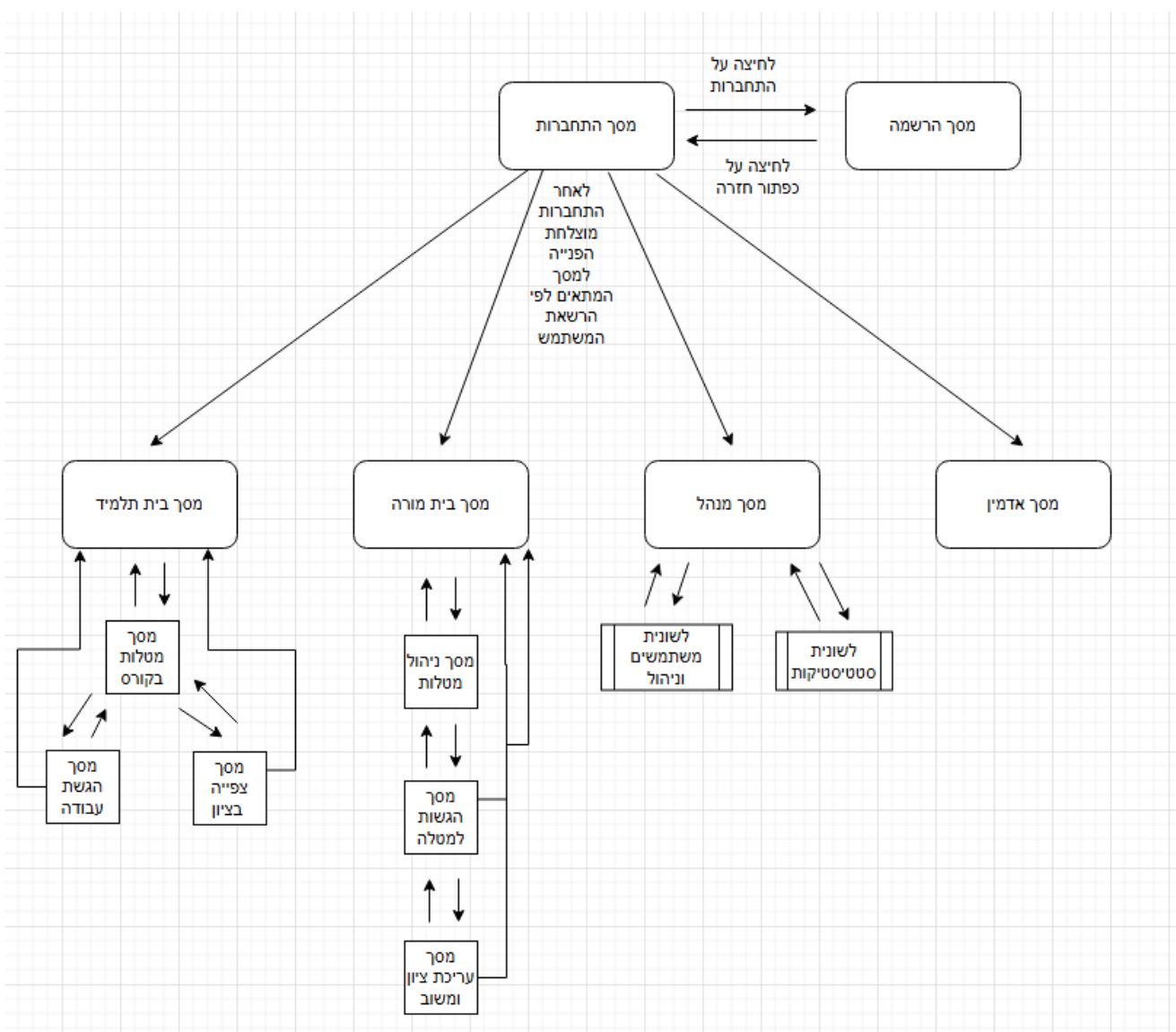
מידע שמוצג במסך:

- טבלה עם כל המשתמשים: שם, דוא"ל, בית ספר, תפקיד נוכחי, סטטוס
- אפשרות סינון לפי בית ספר / תפקיד
- שדה לשינוי תפקיד (תלמיד/מורה/מנהל)

פעולות אפשריות:

- שינוי הרשאת משתמש
- חסימת משתמש
- חיפוש משתמש לפי שם/דוא"ל

תרשים מסכים



מבני הנתונים

במערכת **Classify** נעשה שימוש במבני נתונים שונים לצורך שמירת משתמשים, בתי ספר, קורסים, מטלות, הגשות, ציונים ומשובים. מבני הנתונים נשמרים בשילוב של:

- **מסד נתונים – (SQLite)** עבור נתונים מובנים: משתמשים, קורסים, מטלות, הגשות, ציונים ועוד.
- **מערכת קבצים בשרת** – עבור קבצי ההגשות של התלמידים וקבצים מצורפים למטלות.

טבלת – Schools בתי ספר

טבלת בתי הספר מייצגת כל מוסד חינוכי המשתמש במערכת.

שם השדה	טיפוס נתון	תיאור	דוגמה
school_id	INTEGER	מפתח ראשי, מזהה ייחודי לכל בית ספר	1
name	TEXT	שם בית הספר	"תיכון הרצוג"
address	TEXT	כתובת בית הספר (אופציונלי)	"כפר סבא, רח"..."
contact_name	TEXT	שם איש קשר ראשי	"אוריה לשם"
contact_email	TEXT	דוא"ל איש קשר	"uriya@herzog.sch"
Created_at	TEXT	תאריך יצירה	"2025-10-01" 12:30:00

מפתח ראשי: school_id

טבלת – Users משתמשים

טבלת המשתמשים שומרת את כל המשתמשים במערכת: תלמידים, מורים, מנהלים ואדמין.

שם השדה	טיפוס נתון	תיאור	דוגמה
user_id	INTEGER	מפתח ראשי, מזהה משתמש ייחודי	10
name	TEXT	שם מלא	"אוריה לשם"
email	TEXT	דוא"ל ייחודי	"uria@classroom.com"
password_hash	TEXT	Hash של הסיסמה	"A1B2C3D4..."
role	TEXT	תפקיד המשתמש במערכת	"STUDENT" / "TEACHER" / "MANAGER" / "ADMIN"
schoolID	INTEGER	מזהה בית הספר אליו שייך המשתמש (אם רלוונטי)	1
status	TEXT	סטטוס משתמש: פעיל / ממתין / חסום	"ACTIVE" / "PENDING" / "BLOCKED"
created_at	DATETIME	תאריך יצירת המשתמש	"2025-10-01 12:30:00"

מפתח ראשי:

• user_id

טבלת – Courses קורסים

טבלה זו מייצגת כיתות שנוצרו על ידי מורים.

שם השדה	טיפוס נתון	תיאור	דוגמה
course_id	INTEGER	מפתח ראשי, מזהה קורס	100
name	TEXT	שם הקורס	"פייתון – מבוא"
description	TEXT	תיאור קצר של הקורס	"קורס מבוא לתכנות ב-Python"
teacher_id	INTEGER	מזהה המורה האחראי על הקורס	10
schoolID	INTEGER	מזהה בית הספר שבו מתנהל הקורס	1
created_at	DATETIME	תאריך יצירת הקורס	"2025-11-10 09:00:00"
Class_code	TEXT	קוד כיתה ייחודי שבעזרתו מצטרפים	"MPFZ3X"

מפתח ראשי:

• course_id

טבלת CourseMembers – שיוך תלמידים לקורסים

טבלת חיבור (Many-to-Many) בין תלמידים לבין קורסים.

שם השדה	טיפוס נתון	תיאור	דוגמה
id	INTEGER	מפתח ראשי	1
course_id	INTEGER	מזהה הקורס	100
student_id	INTEGER	מזהה התלמיד	25
joined_at	DATETIME	תאריך הצטרפות התלמיד לקורס	"2025-11-15 14:00:00"
member_status	TEXT	סטטוס השיוך של התלמיד לקורס	ACTIVE" / "PENDING" / "INACTIVE"

מפתח ראשי:

id •

טבלת – Assignments מטלות

טבלת המטלות שומרת את כל המטלות שנוצרו בכל קורס.

שם השדה	טיפוס נתון	תיאור	דוגמה
assignment_id	INTEGER	מפתח ראשי, מזהה מטלה	500
course_id	INTEGER	מזהה הקורס אליו שייכת המטלה	100
title	TEXT	שם המטלה	"תרגיל 1 – פונקציות"
description	TEXT	תיאור מטלה	"לכתוב 3 פונקציות..."
due_date	DATETIME	תאריך יעד להגשה	"2025-12-01 23:59:59"
attachment_path	TEXT	נתיב לקובץ מטלת לדוגמה (אם קיים, אופציונלי)	"assignments/100/500/task.pdf"
created_at	DATETIME	תאריך יצירת המטלה	"2025-11-20 10:00:00"
ai_enabled	INTEGER	האם המטלה נבדקת בעזרת AI	0 / 1

שם השדה	טיפוס נתון	תיאור	דוגמה
is_closed	INTEGER	אם המטלה סגורה להגשות	0 / 1

מפתח ראשי:

• assignment_id

טבלת – Submissions הגשות

טבלת ההגשות שומרת כל הגשה שבוצעה על ידי תלמיד עבור מטלה מסוימת. ההגשה יכולה להיות קובץ או תשובה טקסטואלית, ובפועל נשמר נתיב לקובץ שנשמר בשרת.

שם השדה	טיפוס נתון	תיאור	דוגמה
submission_id	INTEGER	מפתח ראשי, מזהה הגשה	1000
assignment_id	INTEGER	מזהה המטלה	500
student_id	INTEGER	מזהה התלמיד שהגיש	25
file_path	TEXT	נתיב לקובץ ההגשה במערכת הקבצים של השרת	"submissions/100/500/25/work1.docx"
submitted_at	DATETIME	תאריך ושעת ההגשה	"2025-11-25 21:15:00"
status	TEXT	PENDING_AI / "AI_IN_PROGRESS" / "AI_DONE" / "ERROR"	"PENDING_AI" / "AI_DONE" / "ERROR"

מפתח ראשי:

• submission_id

טבלת – AiResults תוצאות מנועי הAI

טבלה זו שומרת את תוצאת כל מנוע AI בנפרד, לכל הגשה. כך ניתן להשתמש במספר מנועים במקביל.

שם השדה	טיפוס נתון	תיאור	דוגמה
ai_result_id	INTEGER	מפתח ראשי	1
submission_id	INTEGER	מזהה ההגשה	1000
engine_name	TEXT	שם מנוע ה-AI	"GPT4" / "Claude" / "LocalAI1"
score	REAL	ציון מספרי מהמנוע (0-100)	87.5
feedback	TEXT	טקסט משוב שנוצר ע"י המנוע	"העבודה ברורה, אך חסרים..."

מפתח ראשי:

• ai_result_id

טבלת Grades – ציונים

טבלת ציונים מרכזת את הציון האוטומטי (AI) והציון הסופי שנקבע על ידי המורה לכל הגשה.

שם השדה	טיפוס נתון	תיאור	דוגמה
grade_id	INTEGER	מפתח ראשי	2000
submission_id	INTEGER	מזהה ההגשה	1000
ai_score	REAL	ציון AI משולב שחושב לפי תוצאות מנועי ה-AI	88.3
final_score	REAL	ציון סופי כפי שנקבע על ידי המורה	92
updated_by	INTEGER	מזהה המורה שעדכן את הציון (אם רלוונטי)	10
updated_at	DATETIME	מועד עדכון הציון הסופי	"2025-11-26 19:30:00"

מפתח ראשי:

• grade_id

טבלת Feedback – משובים

טבלת משובים שומרת את משוב ה-AI ואת משוב המורה עבור ההגשה.

שם השדה	טיפוס נתון	תיאור	דוגמה
feedback_id	INTEGER	מפתח ראשי	3000
submission_id	INTEGER	מזהה ההגשה	1000
ai_feedback	TEXT	משוב אוטומטי שנוצר ע"י מנועי ה-AI	"הניסוח טוב, אבל חסר..."
teacher_feedback	TEXT	משוב אישי מהמורה	"עבודה יפה, רק לשפר..."

מפתח ראשי:

• feedback_id

מבני נתונים במערכת הקבצים

בנוסף למסד הנתונים, המערכת משתמשת במבנה תיקיות קבוע תחת התיקייה server_storage לצורך שמירת קבצים שהועלו למערכת.

תיקיית הגשות:

server_storage/submissions/course_<course_id>/assignment_<assignment_id>/<student_<student_id>/<filename

לדוגמה:

server_storage/submissions/course_100/assignment_500/student_25/work1.docx

תיקיית קבצים מצורפים למטלות:

<server_storage/assignments/course_<course_id>/attachments/<filename

לדוגמה:

server_storage/assignments/course_100/attachments/task.pdf

תיקיית חומרי לימוד:

<server_storage/materials/course_<course_id>/<filename

לדוגמה:

server_storage/materials/course_100/lesson1.pdf

סקירת חולשות ואיומים

עבודה עם בסיס הנתונים – מניעת SQL Injection

איומים:

תוקף מנסה להחדיר קוד SQL זדוני דרך שדות קלט (למשל, דוא"ל או שם משתמש), כדי לגרום לפקודות לא רצויות בבסיס הנתונים – גניבת מידע, מחיקת טבלאות ועוד.

טיפול במערכת Classify

- שימוש ב־ **Prepared Statements** ולא בבניית מחרוזות SQL באמצעות חיבור טקסט. (string concatenation)
- בדיקות תקינות בצד השרת לכל הקלטים (אורך מקסימלי, פורמטים קבועים כמו דוא"ל).
- בדיקות הרשאה בצד השרת לפני ביצוע פעולות על בסיס הנתונים.

דוגמה:

במקום:

```
"SELECT * FROM Users WHERE email = " + email + " AND password = " + pwd + ""
```

נשתמש ב:

```
"SELECT * FROM Users WHERE email = ? AND password_hash = ?"
```

תהליך Login והרשאות

איומים:

- ניסיון התחברות עם סיסמאות גנובות או מנחשים. (Brute Force)
- גישה למשאבים לא מתאימים (תלמיד שמנסה לבצע פעולת מורה, מורה שמנסה לבצע פעולת אדמין וכו').

טיפול במערכת:

- שמירת סיסמאות כ־Hash מסוג SHA-256, ולא כסיסמאות גלויות. ה־Hash נוצר בצד הלקוח ונשמר בבסיס הנתונים.
- בדיקת ההרשאה (role) של המשתמש בכל פעולה בשרת, ולא רק בצד הלקוח.
- בדיקת סטטוס משתמש (ACTIVE, PENDING, BLOCKED) לפני ביצוע פעולה.
- מגבלה על מספר ניסיונות התחברות כושלים (Lockout זמני).

העלאת קבצים (קבצי הגשות)

איום:

- תלמיד מעלה קובץ גדול מדי וגורם לעומס על השרת.
- העלאת קובץ לא תקין או קובץ מסוג מסוכן.
- ניסיון להעלות קובץ שמכיל קוד זדוני.

טיפול במערכת:

- המערכת מגבילה את גודל הקובץ ל-25MB, מונעת העלאת קובץ ריק, מנקה את שם הקובץ, ושומרת אותו בנתיב מבווקר תחת server_storage. הגבלת גודל מקסימלי לקובץ (למשל עד X MB).
- בדיקות תקינות בסיסיות לפני שמירת הקובץ (אימות שהקובץ אכן הועבר בהצלחה).

יירוט מידע (Sniffing / MITM)

איום:

- תוקף שנמצא באותה רשת מנסה להאזין לחבילות TCP ולקרוא נתונים רגישים (סיסמאות, ציונים, פרטי תלמידים).
- תקיפת – Man-In-The-Middle שינוי הנתונים בדרך.

טיפול במערכת:

- שימוש ב־הצפנה בשכבת האפליקציה:
 - המידע שנשלח בין הלקוח לשרת עטוף במבנה JSON, ולאחר מכן מוצפן לפני שליחתו (למשל בעזרת AES).
 - מפתח ההצפנה מחולק בצורה מאובטחת (בעזרת DH)
- סיסמאות אינן נשלחות כטקסט רגיל אלא כ־Hash, ולכן גם אם התעבורה נחשפת – אי אפשר לראות את הסיסמה המקורית.
- כל הודעה כוללת שדות קבועים (type, data, status) כך שאפשר לוודא שהמבנה תקין ולא שונה בדרך.

תקיפות / DOS עומס על השרת

איום:

- משתמש זדוני (או באג במערכת) שולח כמות גדולה מאוד של בקשות לשרת, מה שיכול לגרום לעומס, האטה או קריסה.

טיפול במערכת:

- הגבלת מספר החיבורים במקביל שהשרת מקבל.
- שימוש ב־ Threading בצורה מבוקרת, כך שכל חיבור מטופל בתורו.
- הגדרת Timeout לחיבורים לא פעילים – סגירת חיבור שנשאר פתוח ללא שימוש לאורך זמן.
- אפשרות להוסיף בעתיד סינון כתובות IP או Rate Limit (הגבלת מספר הבקשות לדקה ללקוח).

מימוש

חלק א

סקירת כל הספריות והמודלים בפרויקט

socket – מימוש תקשורת TCP בין הלקוח לשרת.

threading – טיפול במספר לקוחות במקביל והרצת פעולות רקע כמו בדיקות AI בלי לחסום את השרת או הממשק.

sqlite3 – עבודה מול מסד הנתונים SQLite לשמירת משתמשים, בתי ספר, קורסים, מטלות, הגשות, ציונים, משוברים והודעות.

json – ייצוג ההודעות הלוגיות בין הלקוח לשרת בפורמט JSON.

hashlib – יצירת Hash מסוג SHA-256 לסיסמאות לפני שליחתן ושמירתן.

secrets ו **hmac** – יצירת ואימות Auth Token פנימי לבדיקת זהות המשתמש לאחר התחברות.

base64 – קידוד ופענוח קבצים לצורך העברה בתוך הודעות JSON.

os ו **pathlib** – עבודה עם נתיבי קבצים, תיקיות, משתני סביבה ומיקום קבצים בשרת.

datetime ו **time** – ניהול זמני יצירה, זמני הגשה, תאריכי יעד, Timeout ומנגנוני חסימה זמנית.

re – בדיקות פורמט וניקוי טקסט, למשל ניקוי שמות קבצים ובדיקות קלט.

random ו **string** – יצירת קוד כיתה ייחודי להצטרפות תלמידים לקורס.

tempfile, shutil, subprocess ו **shlex** – טיפול בקבצים זמניים, מחיקת תיקיות, סריקת קבצים והרצת כלי מערכת חיצוניים בעת הצורך.

zipfile ו **tarfile** – חילוץ וקריאת קבצי ארכיון שהועלו כחלק מהגשה.

io ו **html** – קריאת מידע בינארי מהזיכרון וניקוי טקסט שחולץ מקבצים.

statistics – חישוב ערכים סטטיסטיים כמו median בשילוב תוצאות ממספר מנועי AI.

urllib.request – שליפת כתובת שרת מתוך קובץ הגדרה חיצוני במידת הצורך.

PySide6 ו **QML** – בניית ממשק המשתמש הגרפי בצד הלקוח והצגת מסכי תלמיד, מורה, מנהל ואדמין.

cryptography – מימוש הצפנת התקשורת באמצעות DH ליצירת מפתח משותף ו-AES להצפנת ההודעות.

python-dotenv – טעינת משתני סביבה עבור מפתחות API והגדרות AI.

openai – חיבור למודל AI של OpenAI לצורך בדיקת הגשות.

anthropic – חיבור למודל AI של Anthropic לצורך בדיקת הגשות.

google-gemini – חיבור למודל Gemini לצורך בדיקת הגשות.

pypdf / PyPDF2 – חילוץ טקסט מקבצי PDF שהועלו לבדיקה.

rarfile ו-py7zr – חילוץ קבצי RAR ו-Z7 במקרה שהוגשה עבודה כארכיון.

tcp_by_size – מודול פנימי לשליחה וקבלה של הודעות TCP לפי גודל ההודעה.

DH – מודול פנימי לביצוע תהליך Diffie-Hellman בין הלקוח לשרת.

AES_e – מודול פנימי להצפנה ופענוח של הודעות בעזרת AES.

db – מודול פנימי לניהול בסיס הנתונים וכל פעולות הקריאה, הכתיבה, העדכון והמחיקה.

AI_Grader – מודול פנימי לבדיקת הגשות בעזרת מנועי AI, איחוד התוצאות ושמירת ציון ומשוב.

resources_rc – מודול משאבים של Qt הכולל קבצים גרפיים ומשאבים המשמשים את ממשק המשתמש.

מחלקות בצד השרת

במימוש בפועל צד השרת בנוי בעיקר ממודולים ופונקציות, ולא ממחלקות Manager נפרדות. החלוקה הלוגית נעשית לפי קבצים: מודול שרת, מודול בסיס נתונים, מודול בדיקות AI ומודולי תקשורת והצפנה.

Classify_Server.py 1

תפקיד המודול:

אחראי על הפעלת שרת ה-TCP, קבלת חיבורים מלקוחות, פענוח הודעות מוצפנות, בדיקת הרשאות, ניתוב בקשות לפונקציות המתאימות ושליחת תשובות מוצפנות חזרה ללקוח.

תכונות ונתונים מרכזיים:

HOST / PORT – כתובת ופורט ההאזנה של השרת.

STORAGE_ROOT – תיקיית שמירת הקבצים שהועלו למערכת.

MAX_FILE_SIZE – גודל קובץ מקסימלי להעלאה.

ROUTES – מילון שמקשר בין קוד הודעה לבין פונקציית הטיפול המתאימה.

FAILED_LOGINS_BY_SOURCE – שמירת ניסיונות התחברות כושלים לצורך חסימה זמנית.

פעולות מרכזיות:

main()

כניסה: ללא פרמטרים.

יציאה: מפעילה את השרת, מאזינה לחיבורים חדשים ופותחת Thread לכל לקוח.

handle_client(client_sock, addr)

כניסה: socket של לקוח וכתובת הלקוח.

יציאה: מבצעת DH, מקבלת הודעות מוצפנות, מפענחת אותן, מעבירה לטיפול ושולחת תגובה מוצפנת.

handle_request(req, client_ip)

כניסה: הודעת JSON שהתקבלה מהלקוח וכתובת הלקוח.

יציאה: מחזירה תשובת JSON לפי סוג הבקשה.

authorize_request(req)

כניסה: בקשת משתמש הכוללת Auth Token ופרטי פעולה.

יציאה: מחזירה האם המשתמש מורשה לבצע את הפעולה.

handle_upload_submission(req)

כניסה: פרטי מטלה, תלמיד וקובץ/תוכן הגשה.

יציאה: שומרת את ההגשה, מעדכנת את בסיס הנתונים ומחזירה מזהה הגשה.

handle_login(req)

כניסה: דוא"ל ו-Hash של סיסמה.

יציאה: מחזירה פרטי משתמש ו-Auth Token במקרה של התחברות תקינה, או הודעת שגיאה.

תפקיד המודול:

אחראי על כל העבודה מול בסיס הנתונים SQLite: יצירת הטבלאות, קריאה, כתיבה, עדכון ומחיקה של נתונים.

תכונות ונתונים מרכזיים:

classify.db – קובץ בסיס הנתונים.

טבלאות מרכזיות – schools, users, courses, course_members, assignments, submissions, ai_results, grades, feedback, materials, messages.

Prepared Statements – שימוש בפרמטרים בשאילתות כדי למנוע SQL Injection.

פעולות מרכזיות:

signup_user(name, email, password_hash, role, school_id)

כניסה: פרטי משתמש חדש.

יציאה: יוצר משתמש חדש בסטטוס מתאים ומחזיר הצלחה או שגיאה.

login_user(email, password_hash)

כניסה: דוא"ל ו-Hash של סיסמה.

יציאה: מחזיר את פרטי המשתמש אם ההתחברות תקינה.

create_course(name, description, teacher_id, school_id)

כניסה: פרטי קורס ומזהה מורה.

יציאה: יוצר קורס חדש ומחזיר מזהה קורס.

join_course_by_code(student_id, code_value)

כניסה: מזהה תלמיד וקוד כיתה.

יציאה: מוסיף בקשת הצטרפות לקורס או מחזיר שגיאה מתאימה.

save_submission(assignment_id, student_id, file_path, status)

כניסה: פרטי הגשה ונתיב הקובץ שנשמר.

יציאה: שומר הגשה חדשה ומחליף הגשה קודמת אם קיימת.

save_grade(submission_id, ai_score, final_score, updated_by)

כניסה: מזהה הגשה וציונים.

יציאה: שומר או מעדכן ציון בבסיס הנתונים.

save_feedback(submission_id, ai_feedback, teacher_feedback)

כניסה: מזהה הגשה ומשובים.

יציאה: שומר או מעדכן משוב AI ומשוב מורה.

(admin_get_overview)

כניסה: ללא פרמטרים.

יציאה: מחזיר לאדמין תמונת מצב מלאה של נתוני המערכת.

AI Grader.py (3)

תפקיד המודול:

אחראי על בדיקת הגשות בעזרת מנועי AI, חילוץ טקסט מקבצים, איחוד תוצאות ממספר מודלים ושמירת ציון ומשוב בבסיס הנתונים.

תכונות ונתונים מרכזיים:

ENABLED_PROVIDER_NAMES – רשימת ספקי AI פעילים.

DEFAULT_OPENAI_MODEL / DEFAULT_ANTHROPIC_MODEL /
DEFAULT_GEMINI_MODEL – שמות המודלים.

מגבלות גודל – הגבלת כמות הטקסט והקבצים שנשלחים לבדיקה.

פעולות מרכזיות:

`run_ai_grading_worker(stop_event)`

כניסה: אובייקט עצירה אופציונלי.

יציאה: מריץ תהליך רקע שבודק הגשות ממתכות.

`process_one_submission(providers, summary_provider)`

כניסה: רשימת ספקי AI וספק לסיכום.

יציאה: בודק הגשה אחת, שומר תוצאות ומעדכן סטטוס.

`load_submission_payload(submission)`

כניסה: פרטי הגשה מבסיס הנתונים.

יציאה: מחלץ את תוכן ההגשה והמטלה לטקסט שניתן לשלוח ל-AI.

`aggregate_results(payload, results, summary_provider)`

כניסה: תוצאות מכמה מנועי AI.

יציאה: יוצר תוצאה משולבת אחת הכוללת ציון ומשוב.

GraderProvider (4

תפקיד המחלקה:

מחלקת בסיס לכל ספק AI. היא מגדירה את הפעולות שכל ספק AI חייב לממש.

תכונות:

name – שם הספק.

פעולות:

`is_available()`

כניסה: ללא פרמטרים.

יציאה: מחזירה האם הספק זמין לשימוש.

grade(payload)

כניסה: תוכן ההגשה והמטלה.

יציאה: מחזירה ציון ומשוב במבנה אחיד.

OpenAIGraderProvider / AnthropicGraderProvider / (5 GeminiGraderProvider

תפקיד המחלקות:

מחלקות שמחברות את המערכת למנועי AI חיצוניים: OpenAI, Anthropic ו-Gemini.

תכונות:

api_key – מפתח API של הספק.

model – שם המודל שבו משתמשים.

client – אובייקט החיבור לספק.

פעולות:

is_available()

כניסה: ללא פרמטרים.

יציאה: מחזירה האם קיימים מפתח וחיבור תקינים לספק.

grade(payload)

כניסה: תוכן ההגשה, קובץ המטלה והנחיות הבדיקה.

יציאה: מחזירה ציון, משוב, חוזקות, נקודות לשיפור ופירוט לפי קריטריונים.

ProviderRegistry (6

תפקיד המחלקה:

ניהול ספקי ה-AI הזמינים במערכת ובחירת הספקים הפעילים לבדיקה.

תכונות:

providers – מילון של ספקי AI לפי שם.

פעולות:

register(provider)

כניסה: ספק AI.

יציאה: מוסיף את הספק לרשימת הספקים.

get_available(names)

כניסה: רשימת שמות ספקים.

יציאה: מחזירה את הספקים הזמינים מתוכה.

get_first_available(preferred_name)

כניסה: שם ספק מועדף.

יציאה: מחזירה ספק זמין לסיכום תוצאות.

7) מודולי תקשורת והצפנה פנימיים

tcp_by_size.py

תפקיד: שליחה וקבלה של הודעות TCP לפי גודל הודעה, כדי למנוע קריאה חלקית של מידע.

פעולות מרכזיות: send_with_size(sock, data), recv_by_size(sock).

DH.py

תפקיד: ביצוע Diffie-Hellman בין הלקוח לשרת לצורך יצירת מפתח משותף.

פעולות מרכזיות: DH_client(sock), DH_server(sock).

AES_e.py

תפקיד: הצפנה ופענוח של ההודעות לאחר יצירת המפתח.

פעולות מרכזיות: encrypt_message(aes_key, data), decrypt_message(aes_key, data).

מחלקות ומודולים בצד הלקוח

AuthClient (1)

תפקיד המחלקה:

אחראית על תקשורת הלקוח מול השרת: פתיחת חיבור TCP, ביצוע DH, הצפנת הודעות, שליחת בקשות וקבלת תשובות.

תכונות:

host / port – כתובת השרת.

timeout_seconds – זמן המתנה מקסימלי לחיבור.

sock – חיבור ה-TCP לשרת.

aes_key – מפתח ההצפנה שנוצר לאחר DH.

current_user – פרטי המשתמש המחובר.

lock_ – מנגנון נעילה כדי למנוע שליחת כמה הודעות במקביל על אותו socket.

פעולות:

connect()

כניסה: ללא פרמטרים.

יציאה: פותחת חיבור לשרת ומייצרת מפתח AES.

send_request(req)

כניסה: בקשת JSON.

יציאה: שולחת את הבקשה לשרת ומחזירה תשובת JSON.

login(email, password)

כניסה: דוא"ל וסיסמה.

יציאה: שולחת בקשת התחברות לאחר Hash לסיסמה ושומרת את פרטי המשתמש אם הצליחה.

signup(name, email, password, role, school_id)

כניסה: פרטי הרשמה.

יציאה: שולחת בקשת הרשמה לשרת ומחזירה תשובה.

upload_submission(assignment_id, student_id, file_path)

כניסה: מזהה מטלה, מזהה תלמיד ונתיב קובץ.

יציאה: מקודדת את הקובץ ב-base64 ושולחת אותו לשרת.

upload_submission_text(assignment_id, student_id, submission_text)

כניסה: מזהה מטלה, מזהה תלמיד וטקסט תשובה.

יציאה: הופכת את הטקסט לקובץ טקסט ושולחת אותו כהגשה.

download_file(file_path)

כניסה: הפניה לקובץ בשרת.

יציאה: מורידה את הקובץ ושומרת אותו במחשב הלקוח.

Auth (2)

תפקיד המחלקה:

מחלקת גישור בין קבצי QML לבין AuthClient. המחלקה נחשפת לממשק בשם auth ומאפשרת למסכים להפעיל פעולות שרת בצורה אסינכרונית.

תכונות:

client_ – מופע של AuthClient שמבצע את התקשורת בפועל.

loginResult, signupResult, למשל, QML, תוצאות למסכי QML, למשל, loginResult, signupResult, uploadSubmissionResult, downloadFileResult, adminActionResult

פעולות:

login(email, password, keepsign)

כניסה: פרטי התחברות מהמסך.

יציאה: מפעילה Thread להתחברות ומחזירה תוצאה דרך loginResult.

signup(name, email, password, role, school_id)

כניסה: פרטי הרשמה מהמסך.

יציאה: שולחת הרשמה ומחזירה תוצאה דרך signupResult.

logout()

כניסה: ללא פרמטרים.

יציאה: מנקה את פרטי המשתמש המחובר בצד הלקוח.

get_student_dashboard(student_id)

כניסה: מזהה תלמיד.

יציאה: מביאה מהשרת כיתות, מטלות, חומרי לימוד והודעות לתלמיד.

create_course / create_assignment / create_material / save_course_message

כניסה: פרטי הפעולה מתוך מסכי המורה.

יציאה: שולחות בקשות יצירה לשרת ומחזירות תוצאה למסך.

admin_create_user / admin_update_user / admin_create_school /
admin_update_school

כניסה: נתונים ממסך האדמין.

יציאה: שולחות פעולות ניהול לשרת ומחזירות תוצאה למסך.

QML רכיבי (3)

תפקיד הרכיבים:

קבצי QML אחראים להצגת ממשק המשתמש, ניווט בין מסכים, קליטת נתונים מהמשתמש והפעלת פעולות דרך `auth`.

רכיבים מרכזיים:

`main.qml` – יוצר את חלון האפליקציה ומפעיל `StackView` למסכים.

`Login.qml` – מסך התחברות.

`Signup.qml` – מסך הרשמה.

`StudentHome.qml` – מסכי תלמיד: כיתות, מטלות, הגשות, ציונים, הודעות וחומרי לימוד.

`TeacherHome.qml` – מסכי מורה: כיתות, מטלות, תלמידים, חומרי לימוד, הודעות ובדיקת הגשות.

`ManagerHome.qml` – מסכי מנהל בית ספר: אישור משתמשים, חסימה, כיתות, מטלות והגדרות בית ספר.

`AdminHome.qml` – מסך אדמין לניהול כלל נתוני המערכת.

פעולות כלליות:

ניווט בין מסכים באמצעות `StackView`.

הצגת נתונים שמתקבלים מהשרת.

שליחת פעולות משתמש למחלקת `Auth`.

עדכון המסך לפי `Signals` שחוזרים מהשרת.

חלק ב

בעיות אלגוריתמיות

אלגוריתם בדיקת עבודות בעזרת AI

תיאור הבעיה האלגוריתמית

כאשר תלמיד מגיש עבודה במערכת, המערכת צריכה:

- לקבל את קובץ ההגשה מהתלמיד.
- להעביר את העבודה לכמה מנועי בינה מלאכותית שונים.
- לקבל מכל מנוע AI ציון ומשוב.
- לחבר את כל התוצאות לציון אחד סופי, כמה שיותר אובייקטיבי ויציב.
- לשמור את הציון והמשוב בבסיס הנתונים, כדי שהמורה יוכל לראות, לערוך, ולאשר לתלמיד.

האתגר הוא לנהל בצורה מסודרת את כל השלבים האלו, כך שגם אם אחד ממנועי ה-AI נכשל או מחזיר תוצאה מוזרה – עדיין יהיה ציון סופי הגיוני.

רעיון הפתרון שנבחר

במערכת Classify נבחרה גישה של **שילוב מספר מנועי AI**. במקום להסתמך רק על תשובה ממודל אחד, המערכת יכולה לשלוח את ההגשה למספר ספקי AI זמינים, כגון OpenAI, Anthropic ו-Gemini. כל ספק מחזיר ציון ומשוב במבנה אחיד, והמערכת מאחדת את התוצאות לציון AI משולב ולמשוב מסכם.

כאשר מתקבלת יותר מתוצאה אחת, הציון המשולב מחושב לפי median של הציונים התקינים, כדי להפחית השפעה של תוצאה חריגה ממודל אחד. בנוסף, אם קיים ספק AI מתאים לסיכום, הוא יכול לעזור בבניית משוב מאוחד. אם רק ספק אחד זמין, משתמשים בתוצאה שלו.

המורה רואה את הציון והמשוב האוטומטי, ויכול לשנות את הציון ולהוסיף משוב אישי לפני שליחה לתלמיד.

שלבי האלגוריתם

אלגוריתם בדיקת עבודה אחת: (Submission)

1. קבלת מזהה ההגשה

האלגוריתם מטפל בהגשה שממתינה לבדיקה. ההגשה מזהה באמצעות submission_id, שנשמר בטבלת submissions בבסיס הנתונים.

2. שליפת פרטי ההגשה

- המערכת משתמשת במודול db כדי לשלוף את פרטי ההגשה:
- נתיב קובץ ההגשה בשרת.
- מזהה המטלה שאליה ההגשה שייכת.
- פרטי המטלה וקובץ ההנחיות/המחונן אם קיים.

3. המרת ההגשה לטקסט

- המערכת קוראת את קובץ ההגשה ומנסה להמיר אותו לטקסט. קיימת תמיכה בקבצי טקסט, PDF, קבצי Office כמו PPTX, DOCX ו-XLSX, וכן קבצי ארכיון כמו ZIP, TAR, RAR ו-Z7. במקרה של ארכיון, המערכת מנסה לחלץ ממנו קבצים קריאים ולצרף את התוכן שלהם לבדיקה.

4. שליחת העבודה למנועי ה-AI

מוגדרת רשימת ספקי AI פעילים לפי הגדרות המערכת. לדוגמה: OpenAI, Gemini ו-Antropic.

עבור כל ספק AI זמין:

נבנית בקשת בדיקה הכוללת את טקסט ההגשה, פרטי המטלה, קובץ ההנחיות/המחונן והוראות להחזרת תשובה במבנה JSON אחיד.

הספק מחזיר ציון מספרי בטווח 0-100, משוב טקסטואלי, חוזקות, נקודות לשיפור ופירוט לפי קריטריונים.

התוצאה נשמרת בטבלת ai_results.

5. טיפול בשגיאות AI

אם אחד מספקי ה-AI נכשל, למשל בגלל שגיאת API, חוסר זמינות או תשובה לא תקינה, המערכת לא מפילה את כל תהליך הבדיקה. השגיאה נשמרת כהערה פנימית, והמערכת ממשיכה להשתמש בתוצאות התקינות שהתקבלו מספקים אחרים.

אם כל ספקי ה-AI נכשלו, ההגשה מסומנת בסטטוס ERROR, ונשמר משוב שגיאה כדי שהמורה יוכל להבין שהבדיקה האוטומטית לא הצליחה.

6. חישוב ציון משולב

לאחר קבלת התוצאות התקינות, המערכת אוספת את הציונים שהתקבלו ממנועי ה-AI. אם קיימת תוצאה אחת בלבד, היא משמשת כציון ה-AI.

אם קיימות כמה תוצאות, המערכת מחשבת את ה-median של הציונים. בחירה ב-median עוזרת לצמצם השפעה של ציון חריג שמודל אחד החזיר, ולכן מתאימה יותר מממוצע פשוט במקרה של מספר מנועים.

הציון שמתקבל נשמר כ-ai_score בטבלת grades.

7. בניית משולב משולב

המערכת מאחדת את המשובים שהתקבלו ממנועי ה-AI לתוצאה אחת מסודרת. המשוב המשולב כולל סיכום כללי, חוזקות, נקודות לשיפור, פירוט לפי קריטריונים ודגלים מיוחדים במקרה שהמערכת זיהתה בעיה כמו חוסר התאמה או חלקים חסרים בהגשה. אם קיימים כמה מנועים זמינים, המערכת יכולה להשתמש בספק AI נוסף לצורך יצירת סיכום מאוחד. אם אין ספק סיכום זמין, המערכת מאחדת את המידע באופן פנימי מתוך התוצאות שהתקבלו.

8. שמירת התוצאה בבסיס הנתונים

לאחר חישוב הציון והמשוב, המערכת שומרת את התוצאות בבסיס הנתונים: בטבלת ai_results נשמרת כל תוצאה שהתקבלה ממנוע AI בנפרד, כולל שם המנוע, הציון והמשוב.

בטבלת grades נשמר הציון המשולב בשדה ai_score. בשלב זה final_score עדיין יכול להישאר ריק, עד שהמורה יאשר או יעדכן את הציון.

בטבלת feedback נשמר המשוב האוטומטי בשדה ai_feedback.

לאחר שמירה תקינה, סטטוס ההגשה בטבלת submissions מתעדכן ל-AI_DONE. אם התהליך נכשל, הסטטוס מתעדכן ל-ERROR.

9. שליחה לממשק המורה

○ כאשר המורה נכנס למסך בדיקת ההגשות, המערכת מציגה לו:

▪ את ציון ה-AI

▪ את המשוב האוטומטי.

○ המורה יכול לשנות את הציון ולהוסיף משוב אישי – הציון הסופי שנשמר לתלמיד הוא final_score, כלומר הציון שהמורה אישר או ערך.

לולאה ראשית לטיפול בבקשות לקוח

בנוסף לאלגוריתם בדיקת ההגשות, קיימת בשרת לולאה מרכזית שמטפלת בכל לקוח שמתחבר למערכת. לולאה זו אחראית לקבל הודעות מהלקוח, לפענח אותן, להפעיל את הפעולה המתאימה ולהחזיר תשובה.

1. קבלת חיבור מלקוח

השרת מאזין לחיבורי TCP. כאשר לקוח חדש מתחבר, השרת פותח עבורו Thread נפרד, כך שמספר לקוחות יכולים לעבוד במקביל.

2. יצירת מפתח הצפנה

בתחילת החיבור הלקוח והשרת מבצעים תהליך DH ליצירת מפתח משותף. לאחר מכן כל ההודעות בין הצדדים מוצפנות בעזרת AES.

3. תחילת לולאת תקשורת

כל עוד הלקוח מחובר, השרת ממתין להודעה חדשה. ההודעות נשלחות באמצעות `tcp_by_size`, כלומר לפני תוכן ההודעה נשלח גודל ההודעה, כדי שהשרת ידע כמה בתים עליו לקרוא.

4. פענוח ההודעה

לאחר קבלת ההודעה המוצפנת, השרת מפענח אותה באמצעות מפתח ה-AES שנוצר בתחילת החיבור. לאחר מכן ההודעה מומרת מ-JSON למבנה dictionary של Python.

5. זיהוי סוג הבקשה

השרת קורא את השדה `type` בהודעה. שדה זה מגדיר את סוג הפעולה המבוקשת, לדוגמה `LOGIN_REQUEST`, `SIGNUP_REQUEST`, `UPLOAD_SUBMISSION_REQUEST` או `GET_STUDENT_DASHBOARD_REQUEST`.

6. בדיקת הרשאות

עבור פעולות שדורשות משתמש מחובר, השרת בודק את ה-Auth Token ואת הרשאות המשתמש. הבדיקה מתבצעת בצד השרת ולא רק בצד הלקוח, כדי למנוע מצב שבו משתמש מפעיל פעולה שאינה מתאימה לתפקיד שלו.

7. ניתוב לפונקציית הטיפול המתאימה

לאחר זיהוי סוג ההודעה, השרת משתמש במילון `ROUTES` שמקשר בין `type` לבין פונקציית הטיפול המתאימה. לדוגמה, בקשת התחברות מועברת לפונקציית `handle_login`, בקשת העלאת הגשה מועברת ל-`handle_upload_submission`, ובקשת נתוני תלמיד מועברת ל-`handle_get_student_dashboard`.

8. ביצוע הפעולה

פונקציית הטיפול מבצעת את הפעולה הנדרשת: קריאה או עדכון בבסיס הנתונים, שמירת קובץ בשרת, בדיקת הרשאות נוספת, או החזרת מידע למסך המשתמש.

9. בניית תגובת JSON

בסיום הטיפול השרת בונה תגובה במבנה JSON. התגובה כוללת בדרך כלל שדה status, שדה message, ושדות מידע נוספים לפי הצורך, כמו user_id, role, רשימת קורסים, רשימת מטלות או מזהה הגשה.

10. הצפנה ושליחת תשובה

התגובה מומרת ל-JSON, מוצפנת בעזרת AES ונשלחת חזרה ללקוח באמצעות tcp_by_size.

11. חזרה להמתנה להודעה נוספת

לאחר שליחת התגובה השרת חוזר לתחילת הלולאה וממתין להודעה הבאה מאותו לקוח. אם הלקוח מתנתק או מתרחשת שגיאת תקשורת, הלולאה מסתיימת וה-Thread של הלקוח נסגר.

קטעי קוד מיוחדים

קטע הקוד הבא מציג את איחוד תוצאות ה-AI. הפונקציה מקבלת רשימת תוצאות ממנועי AI שונים, מחשבת ציון משולב בעזרת median, מאחדת קריטריונים, חוזקות, נקודות לשיפור ודגלי בדיקה, ומחזירה תוצאה אחת אחידה.

```
def aggregate_without_llm(results: List[Dict[str, Any]]) -> Dict[str, Any]:
    scores = [max(0.0, min(100.0, safe_float(item.get("score", 0.0)))) for item in results]
    score_value = round(float(median(scores)), 2)

    criterion_map: Dict[str, List[Dict[str, Any]]] = {}
    for result in results:
        for row in result.get("criterion_scores") or []:
            key = normalize_criterion_name(row.get("criterion", ""))
            if not key:
                continue
            criterion_map.setdefault(key, []).append(row)

    aggregated_criteria = []
    for key, rows in criterion_map.items():
        max_scores = [safe_float(r.get("max_score", 0.0)) for r in rows]
        row_scores = [safe_float(r.get("score", 0.0)) for r in rows]
        comments = [str(r.get("comment", "")).strip() for r in rows if str(r.get("comment", "")).strip()]
        label = rows[0].get("criterion", key)
        aggregated_criteria.append({
            "criterion": label,
            "score": round(float(median(row_scores)), 2),
            "max_score": round(float(median(max_scores)), 2),
            "comment": choose_best_comment(comments),
        })

    strengths = merge_unique_lines(results, field_name="strengths", limit=5)
    improvements = merge_unique_lines(results, field_name="improvements", limit=5)

    flags = {
        "missing_submission_parts": any((r.get("flags") or {}).get("missing_submission_parts", False) for r in results),
        "off_topic": any((r.get("flags") or {}).get("off_topic", False) for r in results),
        "unclean_rubric": any((r.get("flags") or {}).get("unclean_rubric", False) for r in results),
        "low_confidence": any((r.get("flags") or {}).get("low_confidence", False) for r in results),
    }

    summary = choose_best_comment([str(r.get("summary", "")).strip() for r in results if str(r.get("summary", "")).strip()])
    final_feedback = choose_best_comment([str(r.get("final_feedback", "")).strip() for r in results if str(r.get("final_feedback", "")).strip()])

    return normalize_grading_result({
        "score": score_value,
        "summary": summary,
        "strengths": strengths,
        "improvements": improvements,
        "criterion_scores": aggregated_criteria,
        "flags": flags,
        "final_feedback": final_feedback,
        "engine_name": "consensus",
    })
```

בקטע זה ניתן לראות שהמערכת לא מחשבת ממוצע פשוט, אלא משתמשת ב-median עבור הציון הכללי וגם עבור ציוני הקריטריונים. בנוסף, הפונקציה מאחדת הערות, חוזקות ונקודות לשיפור ממספר מנועים, ומחזירה מבנה אחיד שאפשר לשמור בבסיס הנתונים ולהציג למורה.

קטע הקוד הבא מציג את לולאת הטיפול בלקוח בצד השרת. הפונקציה מקבלת חיבור TCP של לקוח, מבצעת החלפת מפתח באמצעות DH, מפענחת הודעות מוצפנות, מעבירה אותן לטיפול לפי סוג הבקשה, מצפינה את התגובה ושולחת אותה חזרה ללקוח.

```
def handle_client(client_sock, addr): 1 usage 1 uniyā
    print(f"[+] Connected: {addr}")
    try:
        client_sock.settimeout(HANDSHAKE_TIMEOUT_SECONDS)
        aes_key = DH_server(client_sock)
        client_sock.settimeout(None)

        while True:
            encrypted_req = recv_by_size(client_sock)
            if not encrypted_req:
                break

            plain_req_bytes = decrypt_message(aes_key, encrypted_req)

            try:
                req = json.loads(plain_req_bytes.decode("utf-8"))
            except json.JSONDecodeError:
                resp = err("BAD_JSON")
            else:
                resp = handle_request(req, addr[0])

            plain_resp = json.dumps(resp).encode("utf-8")
            encrypted_resp = encrypt_message(aes_key, plain_resp)
            send_with_size(client_sock, encrypted_resp)

    except Exception as e:
        print(f"[!] Error {addr}: {e}")
    finally:
        client_sock.close()
        print(f"[-] Disconnected: {addr}")
```

בקטע זה ניתן לראות את רצף הטיפול בבקשת לקוח: קבלת הודעה מוצפנת, פענוח, המרה מ-JSON, ניתוב לפונקציית הטיפול המתאימה, בניית תשובה, הצפנה ושליחה חזרה. הלולאה ממשיכה כל עוד הלקוח מחובר, וכל לקוח מטופל ב-Thread נפרד.

חלק ג

בחלק זה מוצגות בדיקות שבוצעו למערכת לאחר שלבי המימוש. מטרת הבדיקות הייתה לוודא שהפעולות המרכזיות במערכת פועלות כראוי עבור כל סוגי המשתמשים, שההרשאות נאכפות בצד השרת, ושהמערכת מתמודדת בצורה תקינה עם קלטים שגויים, קבצים ובעיות תקשורת.

בדיקה 1 – התחברות תקינה למערכת

מטרת הבדיקה: לוודא שמשתמש קיים יכול להתחבר למערכת ולקבל את המסך המתאים לפי התפקיד שלו.

מה בוצע בפועל: הוזן דוא"ל וסיסמה של משתמש קיים. לאחר לחיצה על Sign in נשלחה בקשת LOGIN_REQUEST לשרת.

תוצאה בפועל: המשתמש התחבר בהצלחה, התקבל Auth Token, והמערכת פתחה את המסך המתאים לפי התפקיד: תלמיד, מורה, מנהל או אדמין.
בעיות שהתגלו / כיצד נפתרו: לא התגלו בעיות בבדיקה זו.

בדיקה 2 – התחברות עם פרטים שגויים

מטרת הבדיקה: לוודא שהמערכת לא מאפשרת התחברות עם דוא"ל או סיסמה שגויים.

מה בוצע בפועל: הוזנה סיסמה שגויה עבור משתמש קיים, ולאחר מכן נשלחה בקשת התחברות.

תוצאה בפועל: המערכת החזירה שגיאה מתאימה ולא אפשרה מעבר למסכי המערכת.
בעיות שהתגלו / כיצד נפתרו: נבדק גם מנגנון חסימה זמנית לאחר מספר ניסיונות התחברות כושלים. המנגנון פעל כראוי.

בדיקה 3 – הרשמה למערכת

מטרת הבדיקה: לוודא שמשתמש חדש יכול להירשם למערכת עם שם, דוא"ל, סיסמה, תפקיד ומזהה בית ספר.

מה בוצע בפועל: נפתח מסך ההרשמה, מולאו כל השדות הנדרשים ונשלחה בקשת SIGNUP_REQUEST לשרת.

תוצאה בפועל: המשתמש נוצר בבסיס הנתונים. משתמש מסוג תלמיד או מורה נשמר בסטטוס PENDING עד לאישור מנהל.

בעיות שהתגלו / כיצד נפתרו: לא התגלו בעיות מהותיות. נבדקה גם הרשמה ללא School ID לתלמיד/מורה והמערכת הציגה שגיאה מתאימה.

בדיקה 4 – אישור משתמש על ידי מנהל

מטרת הבדיקה: לוודא שמנהל בית ספר יכול לאשר משתמשים שממתינים לאישור.
מה בוצע בפועל: התחבר משתמש מסוג MANAGER, נפתח מסך ניהול המשתמשים, ונבחר משתמש בסטטוס PENDING. לאחר מכן נלחץ כפתור Approve.
תוצאה בפועל: סטטוס המשתמש השתנה ל-ACTIVE, והמשתמש יכול היה להתחבר למערכת.
בעיות שהתגלו / כיצד נפתרו: לא התגלו בעיות בבדיקה זו.

בדיקה 5 – חסימה וביטול חסימה של משתמש

מטרת הבדיקה: לוודא שמנהל יכול לחסום משתמש פעיל ולבטל חסימה.
מה בוצע בפועל: במסך ניהול המשתמשים נבחר משתמש פעיל, נלחץ Block, ולאחר מכן נבדקה התחברות המשתמש. בהמשך נלחץ Unblock.
תוצאה בפועל: לאחר החסימה המשתמש לא הצליח להתחבר למערכת. לאחר ביטול החסימה המשתמש הצליח להתחבר שוב.
בעיות שהתגלו / כיצד נפתרו: לא התגלו בעיות בבדיקה זו.

בדיקה 6 – יצירת כיתה על ידי מורה

מטרת הבדיקה: לוודא שמורה יכול ליצור כיתה חדשה במערכת.
מה בוצע בפועל: התחבר משתמש מסוג TEACHER, נפתח מסך Create Class, הוזנו שם ותיאור כיתה ונשלחה בקשת CREATE_COURSE_REQUEST.
תוצאה בפועל: הכיתה נוצרה ונוספה לרשימת הכיתות של המורה. בנוסף נוצר קוד כיתה ייחודי להצטרפות תלמידים.
בעיות שהתגלו / כיצד נפתרו: לא התגלו בעיות בבדיקה זו.

בדיקה 7 – הצטרפות תלמיד לכיתה בעזרת קוד

מטרת הבדיקה: לוודא שתלמיד יכול לשלוח בקשת הצטרפות לכיתה באמצעות קוד כיתה.
מה בוצע בפועל: התחבר משתמש מסוג STUDENT, נפתח מסך Join a class, הוזן קוד כיתה תקין ונשלחה בקשת JOIN_COURSE_BY_CODE_REQUEST.
תוצאה בפועל: בקשת ההצטרפות נשמרה במערכת והופיעה אצל המורה כבקשה הממתינה לאישור.
בעיות שהתגלו / כיצד נפתרו: נבדק גם קוד לא תקין, והמערכת החזירה הודעת שגיאה מתאימה.

בדיקה 8 – אישור תלמיד בכיתה על ידי מורה

מטרת הבדיקה: לוודא שמורה יכול לאשר תלמיד שביקש להצטרף לכיתה.
מה בוצע בפועל: המורה נכנס למסך Students, בחר תלמיד בסטטוס PENDING ולחץ על אישור.
תוצאה בפועל: סטטוס התלמיד בכיתה השתנה ל-ACTIVE, והכיתה הופיעה אצל התלמיד ברשימת הכיתות שלו.
בעיות שהתגלו / כיצד נפתרו: לא התגלו בעיות בבדיקה זו.

בדיקה 9 – יצירת מטלה

מטרת הבדיקה: לוודא שמורה יכול ליצור מטלה חדשה בכיתה.
מה בוצע בפועל: המורה נכנס למסך Assignments, בחר כיתה, הזין שם מטלה, תיאור, תאריך יעד והגדרת AI, וצירף קובץ הנחיות במידת הצורך.
תוצאה בפועל: המטלה נוצרה ונשמרה בבסיס הנתונים. תלמידים בכיתה יכלו לראות אותה במסך Assignments.
בעיות שהתגלו / כיצד נפתרו: כאשר AI היה פעיל ללא קובץ הנחיות, המערכת הציגה שגיאה ולא אפשרה יצירת מטלה, בהתאם למימוש.

בדיקה 10 – הגשת עבודה כקובץ

מטרת הבדיקה: לוודא שתלמיד יכול להגיש קובץ למטלה פתוחה.
מה בוצע בפועל: תלמיד פתח מטלה, בחר קובץ מהמחשב ולחץ Submit.
תוצאה בפועל: הקובץ נשלח לשרת, נשמר תחת server_storage, ונרשמה הגשה חדשה ב-submissions.
בעיות שהתגלו / כיצד נפתרו: לא התגלו בעיות בבדיקה זו.

בדיקה 11 – הגשת תשובה טקסטואלית

מטרת הבדיקה: לוודא שתלמיד יכול להגיש תשובה טקסטואלית ללא בחירת קובץ.
מה בוצע בפועל: תלמיד פתח מטלה, כתב תשובה בשדה הטקסט ולחץ Submit.
תוצאה בפועל: המערכת יצרה קובץ טקסט מהתשובה, שלחה אותו לשרת ושמרה אותו כהגשה רגילה.
בעיות שהתגלו / כיצד נפתרו: לא התגלו בעיות בבדיקה זו.

בדיקה 12 – החלפה ומחיקה של הגשה

מטרת הבדיקה: לוודא שתלמיד יכול להחליף הגשה קיימת או למחוק אותה.
מה בוצע בפועל: תלמיד הגיש עבודה למטלה, לאחר מכן העלה הגשה חדשה, ובהמשך בדק גם מחיקה של ההגשה.
תוצאה בפועל: ההגשה החדשה החליפה את ההגשה הקודמת. בעת מחיקה, ההגשה נמחקה מבסיס הנתונים והקובץ נמחק מהשרת.
בעיות שהתגלו / כיצד נפתרו: לא התגלו בעיות בבדיקה זו.

בדיקה 13 – מניעת העלאת קובץ גדול מדי

מטרת הבדיקה: לוודא שהמערכת מגבילה העלאת קבצים גדולים מדי.
מה בוצע בפועל: נבחר קובץ גדול מ-MB25 וניסו להגיש אותו.
תוצאה בפועל: המערכת דחתה את הקובץ והציגה הודעת שגיאה מתאימה.
בעיות שהתגלו / כיצד נפתרו: לא התגלו בעיות בבדיקה זו.

בדיקה 14 – בדיקת AI להגשה

מטרת הבדיקה: לוודא שהגשה שנשלחה לבדיקה עוברת לתהליך AI ונשמר לה ציון ומשוב.
מה בוצע בפועל: נוצרה מטלה עם AI פעיל וקובץ הנחיות, תלמיד הגיש עבודה, והשרת הפעיל את תהליך בדיקת ה-AI ברקע.
תוצאה בפועל: ההגשה עברה מסטטוס PENDING_AI ל-AI_DONE. נשמר ציון AI בטבלת grades, משוב AI בטבלת feedback, ותוצאות מנועים בטבלת ai_results.
בעיות שהתגלו / כיצד נפתרו: כאשר ספק AI לא היה זמין, המערכת המשיכה עם ספקים זמינים אחרים. אם לא היה ספק זמין כלל, ההגשה סומנה כ-ERROR.

בדיקה 15 – בדיקת מורה להגשה ועדכון ציון

מטרת הבדיקה: לוודא שמורה יכול לצפות בתוצאות AI, לערוך ציון ולהוסיף משוב אישי.
מה בוצע בפועל: המורה פתח את מסך Submission Review, בחר הגשה, בדק את ציון ה-AI והמשוב, הזין final_score ומשוב מורה ושמר.
תוצאה בפועל: הציון הסופי נשמר בטבלת grades בשדה final_score, ומשוב המורה נשמר בטבלת feedback בשדה teacher_feedback.

בעיות שהתגלו / כיצד נפתרו: נבדקה הזנת ציון מחוץ לטווח 0-100, והממשק מנע שמירה של ערך לא תקין.

בדיקה 16 – צפייה בציון ומשוב על ידי תלמיד

מטרת הבדיקה: לוודא שתלמיד יכול לראות את הציון הסופי והמשוב לאחר בדיקת המורה. מה בוצע בפועל: לאחר שהמורה שמר ציון סופי, התלמיד נכנס למסך Grades ופתח את פרטי הציון.

תוצאה בפועל: התלמיד ראה את final_score ואת משוב המורה. לאחר פתיחת הציון, המערכת סימנה את הציון כנקרא.

בעיות שהתגלו / כיצד נפתרו: לא התגלו בעיות בבדיקה זו.

בדיקה 17 – חומרי לימוד

מטרת הבדיקה: לוודא שמורה יכול לפרסם חומר לימוד ותלמיד יכול לצפות בו.

מה בוצע בפועל: המורה יצר חומר לימוד בכיתה וצירף קובץ. לאחר מכן תלמיד מאותה כיתה נכנס למסך Study materials.

תוצאה בפועל: חומר הלימוד הופיע לתלמיד, והקובץ היה זמין להורדה.

בעיות שהתגלו / כיצד נפתרו: לא התגלו בעיות בבדיקה זו.

בדיקה 18 – הודעות כיתה

מטרת הבדיקה: לוודא שמורה יכול לשלוח הודעה לכיתה ותלמיד יכול לקרוא אותה.

מה בוצע בפועל: המורה יצר הודעה במסך Messages. תלמיד מאותה כיתה נכנס למסך Messages ופתח את ההודעה.

תוצאה בפועל: ההודעה הופיעה לתלמיד. לאחר פתיחתה, היא סומנה כנקראה.

בעיות שהתגלו / כיצד נפתרו: לא התגלו בעיות בבדיקה זו.

בדיקה 19 – בדיקת הרשאות תלמיד

מטרת הבדיקה: לוודא שתלמיד לא יכול לבצע פעולות שמיועדות למורה, מנהל או אדמין.

מה בוצע בפועל: נבדקו פעולות כמו יצירת כיתה, יצירת מטלה וניהול משתמשים מתוך משתמש תלמיד או באמצעות בקשה לא מתאימה.

תוצאה בפועל: השרת דחה את הפעולות והחזיר שגיאת הרשאה.

בעיות שהתגלו / כיצד נפתרו: לא התגלו בעיות בבדיקה זו, מכיוון שהבדיקות מתבצעות בצד השרת ולא רק בממשק.

בדיקה 20 – בדיקת הרשאות מנהל בית ספר

מטרת הבדיקה: לוודא שמנהל בית ספר רואה ומנהל רק נתונים של בית הספר שלו.
מה בוצע בפועל: התחבר משתמש MANAGER ונבדקו מסכי משתמשים, כיתות ומטלות.
תוצאה בפועל: המנהל ראה משתמשים, כיתות ומטלות של בית הספר שלו בלבד. פעולות על נתונים מחוץ לבית הספר נדחו.
בעיות שהתגלו / כיצד נפתרו: לא התגלו בעיות בבדיקה זו.

בדיקה 21 – פעולות אדמין

מטרת הבדיקה: לוודא שאדמין יכול לנהל נתונים כלליים במערכת.
מה בוצע בפועל: התחבר משתמש ADMIN ונבדקו יצירה, עדכון ומחיקה של בתי ספר ומשתמשים, וכן צפייה בנתוני מערכת נוספים.
תוצאה בפועל: הפעולות בוצעו בהצלחה, והנתונים עודכנו במסך האדמין ובבסיס הנתונים.
בעיות שהתגלו / כיצד נפתרו: לא התגלו בעיות בבדיקה זו.

בדיקה 22 – הורדת קובץ

מטרת הבדיקה: לוודא שמשתמש מורשה יכול להוריד קובץ מהשרת.
מה בוצע בפועל: תלמיד ומורה ניסו להוריד קבצים הקשורים לכיתות שלהם, כמו קובץ מטלה, חומר לימוד או הגשה.
תוצאה בפועל: הקובץ ירד בהצלחה רק כאשר למשתמש הייתה הרשאה מתאימה.
בעיות שהתגלו / כיצד נפתרו: ניסיון להוריד קובץ ללא הרשאה נדחה על ידי השרת.

בדיקה 23 – סגירת מטלה להגשות

מטרת הבדיקה: לוודא שמטלה סגורה לא מאפשרת הגשות חדשות.
מה בוצע בפועל: המורה סגר מטלה ולאחר מכן תלמיד ניסה להגיש אליה עבודה.
תוצאה בפועל: המערכת לא אפשרה הגשה חדשה למטלה סגורה.
בעיות שהתגלו / כיצד נפתרו: לא התגלו בעיות בבדיקה זו.

בדיקה 24 – תקשורת מוצפנת בין לקוח לשרת

מטרת הבדיקה: לוודא שהלקוח והשרת משתמשים בתהליך DH וב-AES להעברת הודעות.

מה בוצע בפועל: הופעלו הלקוח והשרת ונבדקה התחברות רגילה. נבדק בקוד שהחיבור מתחיל ב-DH ולאחר מכן כל הודעה מוצפנת ונשלחת דרך `tcp_by_size`.

תוצאה בפועל: התקשורת פעלה כראוי, והודעות JSON לא נשלחו כטקסט רגיל אלא לאחר הצפנה.

בעיות שהתגלו / כיצד נפתרו: לא התגלו בעיות בבדיקה זו.

מדריך למשתמש

מטרת המדריך

מדריך זה מסביר כיצד להתקין, להפעיל ולהשתמש במערכת Classify. המערכת מיועדת לארבעה סוגי משתמשים: תלמיד, מורה, מנהל בית ספר ואדמין. כל משתמש מקבל מסכים ופעולות בהתאם להרשאות שלו.

מבנה קבצי המערכת

קבצי המערכת המרכזיים הם:

Classify_Server.py – קובץ השרת. אחראי על תקשורת TCP, קבלת בקשות, בדיקת הרשאות, עבודה מול בסיס הנתונים, שמירת קבצים והפעלת בדיקות AI.

Classify_Client.py – קובץ הלקוח. אחראי על הפעלת ממשק המשתמש, התחברות לשרת ושליחת פעולות המשתמש.

AI_Grader.py – מודול בדיקת ההגשות בעזרת מנועי AI, חילוץ טקסט מקבצים ואיחוד תוצאות.

db.py – מודול בסיס הנתונים. כולל יצירת טבלאות ופעולות קריאה, כתיבה, עדכון ומחיקה.

DH.py – מודול יצירת מפתח משותף בין הלקוח לשרת.

AES_e.py – מודול הצפנה ופענוח של הודעות.

tcp_by_size.py – מודול לשליחה וקבלה של הודעות TCP לפי גודל ההודעה.

main.qml – קובץ ראשי שמפעיל את ממשק המשתמש.

data/screens – תיקיית מסכי המערכת.

server_data – תיקייה שבה נשמרים בסיס הנתונים וקובץ הלוג של השרת.

server_storage – תיקייה שבה נשמרים קבצים שהועלו למערכת, כגון הגשות, חומרי לימוד וקבצי מטלות.

דרישות מערכת

המערכת פותחה בסביבת Windows ודורשת:

Python 3.10 ומעלה.

חיבור רשת בין הלקוח לשרת, או הרצה מקומית על אותו מחשב.

התקנת הספריות הדרושות: cryptography, PySide6, python-dotenv ו-pyenv.

לשימוש בבדיקות AI יש להתקין גם את ספריות ספקי ה-AI הרלוונטיות, כגון, openai, anthropic ו-google-genai.
לקריאת קבצים מסוגים שונים ניתן להשתמש גם ב-rarfile, PyPDF2 / pypdf ו-py7zr.

התקנת המערכת

תחילה יש לפתוח תיקייה של הפרויקט ולהתקין סביבת עבודה וירטואלית:

```
python -m venv .venv
```

```
venv\Scripts\Activate.ps1\.
```

לאחר מכן מתקינים את הספריות הבסיסיות:

```
pip install PySide6 cryptography python-dotenv
```

אם משתמשים בבדיקות AI ובחילוץ קבצים מתקדמים, מתקינים גם:

```
pip install openai anthropic google-genai pypdf PyPDF2 rarfile py7zr
```

הגדרת AI

בדיקות ה-AI הן אופציונליות. אם רוצים להשתמש בהן, יש להגדיר קובץ .env בתיקיית הפרויקט עם מפתחות API מתאימים, לדוגמה:

```
OPENAI_API_KEY=your_openai_key
```

```
ANTHROPIC_API_KEY=your_anthropic_key
```

```
GEMINI_API_KEY=your_gemini_key
```

```
AI_GRADING_PROVIDERS=openai
```

```
AI_SUMMARIZER_PROVIDER=openai
```

אם לא מוגדר ספק AI זמין, המערכת עדיין יכולה לפעול, אך בדיקת ההגשות האוטומטית לא תתבצע עד להגדרת ספק מתאים.

הפעלת המערכת

יש להפעיל קודם את השרת:

python Classify_Server.py

השרת מאזין כברירת מחדל בכתובת:

0.0.0.0:5555

לאחר מכן מפעילים את הלקוח:

python Classify_Client.py

כברירת מחדל, הלקוח מתחבר לשרת בכתובת:

127.0.0.1:5555

אם השרת נמצא במחשב אחר, ניתן לעדכן את כתובת השרת בקובץ client_config.json.

נתונים התחלתיים

בקוד המערכת לא מוגדרים משתמשי ברירת מחדל קשיחים. משתמשים נוצרים דרך מסך ההרשמה או דרך מסך האדמין. מנהל בית ספר יכול להירשם דרך מסך ההרשמה, ולאחר מכן תלמידים ומורים יכולים להירשם באמצעות מזהה בית הספר. תלמידים ומורים נכנסים תחילה בסטטוס PENDING, עד שמנהל בית הספר מאשר אותם.

שימוש כללי במערכת

לאחר הפעלת הלקוח מוצג מסך התחברות. משתמש קיים מזין דוא"ל וסיסמה ולוחץ על Sign in. משתמש חדש עובר למסך ההרשמה, ממלא שם מלא, דוא"ל, סיסמה, תפקיד ומזהה בית ספר במידת הצורך.

לאחר התחברות מוצג למשתמש מסך הבית המתאים לתפקיד שלו. כל משתמש רואה רק את הפעולות המותרות לו לפי ההרשאה שלו.

משתמש מסוג תלמיד

תלמיד יכול לבצע את הפעולות הבאות:

לצפות בכיתות שאליהן הוא משויך.

להצטרף לכיתה באמצעות קוד כיתה.

לצפות במטלות פתוחות וסגורות.

להגיש עבודה כקובץ או כתשובה טקסטואלית.

להחליף או למחוק הגשה קיימת.

לצפות בחומרי לימוד.

לקרוא הודעות שנשלחו על ידי המורה.

לצפות בציונים סופיים ובמשוב המורה.

אופן שימוש בסיסי לתלמיד:

לאחר התחברות, התלמיד נכנס למסך Join a class כדי להזין קוד כיתה שקיבל מהמורה. לאחר אישור המורה, הכיתה מופיעה במסך My classes. מתוך מסך Assignments ניתן לפתוח מטלה, לצרף קובץ או לכתוב תשובה, ולשלוח את ההגשה. לאחר שהמורה בודק את העבודה, הציון מופיע במסך Grades.

משתמש מסוג מורה

מורה יכול לבצע את הפעולות הבאות:

ליצור כיתה חדשה.

לראות את רשימת הכיתות שלו.

לצפות בקוד הכיתה ולשתף אותו עם תלמידים.

לאשר או להסיר תלמידים שביקשו להצטרף.

ליצור מטלות ולצרף קובץ הנחיות.

לסגור או לפתוח מטלות להגשה.

לצפות בהגשות תלמידים.

לראות ציון ומשוב AI.

לעדכן final_score ולהוסיף משוב אישי.

לפרסם חומרי לימוד.

לשלוח הודעות לכיתה.

לצפות בדוחות כלליים.

אופן שימוש בסיסי למורה:

לאחר התחברות, המורה יוצר כיתה במסך Create Class. לאחר יצירת הכיתה מתקבל קוד כיתה, שאותו ניתן להעביר לתלמידים. כאשר תלמידים שולחים בקשת הצטרפות, המורה מאשר אותם במסך Students. במסך Assignments המורה יוצר מטלות, ובמסך Submission Review הוא בודק את ההגשות, רואה את תוצאת ה-AI, קובע ציון סופי ומוסיף משוב.

משתמש מסוג מנהל בית ספר

מנהל בית ספר יכול לבצע את הפעולות הבאות:

לראות משתמשים השייכים לבית הספר שלו.

לאשר משתמשים חדשים.

לחסום משתמשים ולבטל חסימה.

לצפות בכיתות ובמטלות של בית הספר.

לעדכן את פרטי בית הספר.

אופן שימוש בסיסי למנהל:

לאחר התחברות, המנהל נכנס למסך המשתמשים. משתמשים חדשים מופיעים בסטטוס PENDING, והמנהל יכול לאשר אותם. משתמש בעייתי ניתן לחסום, ולאחר מכן לבטל את החסימה במידת הצורך. בנוסף, המנהל יכול לצפות בנתוני הכיתות והמטלות של בית הספר ולעדכן פרטי בית ספר.

משתמש מסוג אדמין

אדמין הוא משתמש בעל הרשאות מערכת רחבות. הוא יכול:

לצפות בנתוני כלל המערכת.

ליצור, לעדכן ולמחוק בתי ספר.

ליצור, לעדכן ולמחוק משתמשים.

לעדכן כיתות ומטלות.

לצפות בטבלאות מידע נוספות כגון הגשות, ציונים, תוצאות AI, חומרי לימוד והודעות.

אופן שימוש בסיסי לאדמין:

לאחר התחברות, האדמין נכנס למסך AdminHome. במסך זה ניתן לבחור את סוג הנתונים הרצוי, לחפש רשומות, לפתוח פרטים, לבצע עדכון או מחיקה בהתאם להרשאות, ולנהל את נתוני המערכת בצורה מרוכזת.

שמירת קבצים

קבצים שמועלים למערכת אינם נשמרים בתוך בסיס הנתונים עצמו, אלא בתיקיית server_storage. בבסיס הנתונים נשמר הנתב לקובץ. לדוגמה:

הגשות תלמידים נשמרות תחת submissions.

קבצי מטלות נשמרים תחת assignments.

חומרי לימוד נשמרים תחת materials.

אבטחה והרשאות

המערכת משתמשת בהצפנה בין הלקוח לשרת. בתחילת החיבור מתבצע תהליך DH ליצירת מפתח משותף, ולאחר מכן ההודעות מוצפנות בעזרת AES. סיסמאות אינן נשמרות כטקסט רגיל אלא כ-Hash. בנוסף, פעולות משתמש נבדקות בצד השרת לפי role ו-Auth Token, כדי לוודא שמשתמש לא יבצע פעולה שאינה מתאימה להרשאה שלו.

סיום עבודה

בסיום השימוש ניתן להתנתק דרך כפתור Logout. פעולה זו מנקה את פרטי המשתמש המחובר בצד הלקוח ומחזירה את המשתמש למסך ההתחברות.

רפלקציה

במהלך העבודה על הפרויקט למדתי הרבה גם מבחינה טכנית וגם מבחינת דרך עבודה. בהתחלה היה לי רעיון די ברור למה אני רוצה שהמערכת תעשה, אבל ככל שהתקדמתי הבנתי שיש פער גדול בין רעיון כללי לבין מערכת שעובדת באמת. הייתי צריך לחשוב על מבנה הנתונים, על הרשאות, על תקשורת בין Client ל-Server, על שמירת קבצים, על בדיקות, ועל איך לחבר את כל החלקים האלה בצורה מסודרת.

אחד האתגרים המרכזיים בפרויקט היה לשמור על התאמה בין התכנון לבין הקוד בפועל. היו דברים שתכננתי בצורה מסוימת, אבל בזמן המימוש גיליתי שצריך לשנות אותם כדי שהמערכת תהיה נוחה יותר או נכונה יותר. למשל, בחלק מהמסכים, בקודי ההודעות ובדרך שבה נשמרים ציונים ומשובים, הייתי צריך לעדכן את התכנון לפי מה שבאמת נבנה בקוד.

בנוסף, העבודה עם AI הייתה חלק משמעותי בפרויקט. לא רציתי שהמערכת רק תציג ציון, אלא שתדע לקבל הגשה, לשלוח אותה לבדיקה, לשמור תוצאות, ולאפשר למורה לראות את הציון והמשוב לפני קביעת הציון הסופי. זה דרש ממני להבין איך לעבוד עם כמה ספקי AI, איך לשמור תוצאות בצורה מסודרת, ואיך להתמודד עם מצב שבו חלק מהבדיקות מצליחות וחלק נכשלות. הכול כדי להגיע לאיכות בדיקה שמורה יכול לסמוך עליה, ולא לחשוש בציון ה AI אלא לאשר אותו ברוב המקרים.

גם בצד של האבטחה למדתי לא מעט. הבנתי שכשעובדים עם משתמשים, סיסמאות וקבצים, אי אפשר להתייחס לזה כאל פרט קטן. לכן שילבתי Hash לסיסמאות, הרשאות לפי סוג משתמש, תקשורת מוצפנת בין הלקוח לשרת, ובדיקות שמוודאות שכל משתמש יכול לבצע רק את הפעולות שמתאימות לו.

אם הייתי מתחיל את הפרויקט מחדש, כנראה שהייתי משקיע יותר זמן כבר בהתחלה בתכנון מדויק של הטבלאות, קודי ההודעות והמסכים. זה היה חוסך לי חלק מהשינויים שעשיתי בהמשך. מצד שני, אני חושב שגם השינויים האלה היו חלק חשוב מהלמידה, כי הם גרמו לי להבין יותר טוב איך מערכת אמיתית מתפתחת תוך כדי עבודה.

בסופו של דבר אני מרוצה מהפרויקט, כי הצלחתי לבנות מערכת שמשלבת כמה תחומים ביחד: ממשק משתמש, שרת, מסד נתונים, ניהול הרשאות, קבצים, תקשורת מוצפנת ובדיקת עבודות בעזרת AI. הפרויקט דרש ממני התמדה, פתרון בעיות וחשיבה מסודרת, ואני מרגיש שהוא שיפר משמעותית את היכולת שלי לתכנן ולממש מערכת תוכנה מלאה

נספחים

Classify_Server.py

import socket

import threading

import json

import os

import re

import base64

import hashlib

import hmac

import shlex

import shutil

import secrets

import subprocess

import tempfile

import time

from pathlib import Path

from datetime import datetime

from typing import Optional

from tcp_by_size import send_with_size, recv_by_size

from DH import DH_server

from AES_e import encrypt_message, decrypt_message

import db

from AI_Grader import run_ai_grading_worker

HOST = "0.0.0.0"

PORT = 5555

```

STORAGE_ROOT = Path(__file__).resolve().parent / "server_storage"

MAX_FILE_SIZE = 25 * 1024 * 1024 # 25MB

LOG_FILE = Path(__file__).resolve().parent / "server_data/server_log.txt"

UPLOAD_SCAN_TMP_DIR = Path(__file__).resolve().parent / "server_data" /
"upload_scan_tmp"


PUBLIC_PATH_PREFIX = "storage"//:

HANDSHAKE_TIMEOUT_SECONDS = 30


MAX_FAILED_LOGIN_ATTEMPTS =
int(os.getenv("CLASSIFY_MAX_FAILED_LOGIN_ATTEMPTS", "20"))

FAILED_LOGIN_WINDOW_SECONDS =
int(os.getenv("CLASSIFY_FAILED_LOGIN_WINDOW_SECONDS", "900"))

LOGIN_BLOCK_SECONDS =
int(os.getenv("CLASSIFY_LOGIN_BLOCK_SECONDS", "1800"))


ANTIVIRUS_SCAN_ENABLED = os.getenv("CLASSIFY_AV_SCAN_ENABLED",
"1").strip().lower() not in {"0", "false", "no", "off"}

ANTIVIRUS_FAIL_OPEN = os.getenv("CLASSIFY_AV_FAIL_OPEN",
"0").strip().lower() in {"1", "true", "yes", "on"}

ANTIVIRUS_SCAN_TIMEOUT_SECONDS =
int(os.getenv("CLASSIFY_AV_SCAN_TIMEOUT_SECONDS", "60"))

ANTIVIRUS_SCAN_COMMAND = os.getenv("CLASSIFY_AV_SCAN_COMMAND",
 "").strip()

EICAR_TEST_SIGNATURE = b"EICAR-STANDARD-ANTIVIRUS-TEST-FILE"


LOG_REDACTED_VALUE = "<redacted">

LOG_BINARY_VALUE = "<binary content omitted">

LOG_MAX_TEXT_CHARS = 800

LOG_SUMMARY_KEYS} =
"  schools", "users", "courses", "members", "assignments", "submissions",

```

```

"    ai_results", "grades", "feedback", "materials", "messages,"
"    message_reads", "grade_reads", "activity"
{
LOG_SENSITIVE_KEYS} =
"    password", "password_hash", "token", "secret", "api_key", "authorization,"
_"    auth_token", "file_content_b64", "attachment_content_b64", "content_b64"
{

```

```

AUTH_TOKEN_SECRET = secrets.token_bytes(32)

```

```

FAILED_LOGIN_LOCK = threading.Lock()

```

```

FAILED_LOGINS_BY_SOURCE{} =

```

```

def auth_token_signature(payload: str) -> str:

```

```

    digest = hmac.new(AUTH_TOKEN_SECRET, payload.encode("utf-8"),
hashlib.sha256).digest()

```

```

    return base64.urlsafe_b64encode(digest).decode("ascii").rstrip("=")

```

```

def create_auth_token(user_id: int) -> str:

```

```

    payload = str(int(user_id))

```

```

    return f"{payload}.{auth_token_signature(payload)}"

```

```

def verify_auth_token(token: str, user_id: int) -> bool:

```

```

    token_text = str(token or "")

```

```

    if "." not in token_text:

```

```

        return False

```

```

    payload, signature = token_text.rsplit(1, ".")

```

```

if payload != str(int(user_id)):
    return False

expected = auth_token_signature(payload)

return hmac.compare_digest(signature, expected)


def resolve_storage_path(file_ref: str) -> Optional[Path]:
    file_ref = str(file_ref or "").strip()

    if not file_ref:
        return None

    storage_root = STORAGE_ROOT.resolve()

    if file_ref.startswith(PUBLIC_PATH_PREFIX):
        relative_value = file_ref[len(PUBLIC_PATH_PREFIX):].strip().lstrip("\\")
        target = (storage_root / relative_value).resolve()
    else:
        target = Path(file_ref).expanduser()
        if not target.is_absolute():
            target = (storage_root / file_ref.lstrip("\\")).resolve()
        else:
            target = target.resolve()

    if target != storage_root and storage_root not in target.parents:
        return None

    return target

```

```

def to_public_file_ref(file_path: str) -> str:
    path_text = str(file_path or "").strip()
    if not path_text:
        return ""

    resolved = resolve_storage_path(path_text)
    if resolved is None:
        return ""

    try:
        relative_value = resolved.relative_to(STORAGE_ROOT.resolve()).as_posix()
    except Exception:
        return ""

    return PUBLIC_PATH_PREFIX + relative_value

```

```

def normalize_public_file_refs(value):
    if isinstance(value, list):
        return [normalize_public_file_refs(item) for item in value]
    if isinstance(value, dict):
        fixed{} =
        for key, item in value.items():
            if key in {"attachment_path", "file_path", "submissionPath",
"attachmentPath", "filePath", "submission_path"}:
                fixed[key] = to_public_file_ref(item)
            else:
                fixed[key] = normalize_public_file_refs(item)

```

```

        return fixed

    return value

----- #
#Helpers: validate + responses
----- #

def redact_for_log(value, parent_key=""):
    key_name = str(parent_key or "").lower()

    if key_name in LOG_SENSITIVE_KEYS:
        if "b64" in key_name or "content" in key_name:
            return LOG_BINARY_VALUE
        return LOG_REDACTED_VALUE

    if isinstance(value, dict):
        return {key: redact_for_log(item, key) for key, item in value.items()}

    if isinstance(value, list):
        if key_name in LOG_SUMMARY_KEYS:
            return f"<list: {len(value)} item(s)"<

        return [redact_for_log(item, parent_key) for item in value[:25]] +
        (["<truncated>"] if len(value) > 25 else [])

    if isinstance(value, str):
        if len(value) > LOG_MAX_TEXT_CHARS:
            return value[:LOG_MAX_TEXT_CHARS] + "... <truncated"<

    return value

```

return value

```
def log_action(req_type, req_data, resp_data, client_ip="UNKNOWN",
user_id=None, user_name=None):

    try:

        timestamp = datetime.now().strftime("%d/%m/%Y %H:%M:%S")

        safe_req = redact_for_log(req_data)

        safe_resp = redact_for_log(resp_data)

        log_entry) =

            f"\n{' '*60}\n"

            f"TIME: {timestamp}\n"

            f"IP: {client_ip}\n"

            f"USER_ID: {user_id}\n"

            f"USER_NAME: {user_name}\n"

            f"ACTION: {req_type}\n"

            f"REQUEST:\n{json.dumps(safe_req, indent=2, ensure_ascii=False)}\n"

            f"RESPONSE:\n{json.dumps(safe_resp, indent=2,
ensure_ascii=False)}\n"

        (

            with open(LOG_FILE, "a", encoding="utf-8") as f:

                f.write(log_entry)

    except Exception as e:

        print("[LOG ERROR]", e)
```



```

def ok(**kwargs):
    resp = {"status": "OK"}
    resp.update(kwargs)
    return resp

def err(message: str, **kwargs):
    resp = {"status": "ERROR", "message": message}
    resp.update(kwargs)
    return resp

def require_fields(req: dict, fields: list[str]):
    missing = [f for f in fields if f not in req]
    if missing:
        return False, err("MISSING_FIELDS", missing=missing)
    return True, None

def normalize_login_source(client_ip: str) -> str:
    source = str(client_ip or "").strip()
    return source or "UNKNOWN"

def get_login_block_status(client_ip: str):
    source = normalize_login_source(client_ip)
    now = time.monotonic()
    with FAILED_LOGIN_LOCK:

```

```

state = FAILED_LOGINS_BY_SOURCE.get(source)

if not state:

    return False, 0

blocked_until = float(state.get("blocked_until", 0))

if blocked_until > now:

    return True, max(1, int(blocked_until - now + 0.999))

first_failed_at = float(state.get("first_failed_at", now))

if now - first_failed_at > max(1, FAILED_LOGIN_WINDOW_SECONDS):

    FAILED_LOGINS_BY_SOURCE.pop(source, None)

    return False, 0

if blocked_until:

    state["blocked_until"] = 0

return False, 0


def record_failed_login(client_ip: str):

    source = normalize_login_source(client_ip)

    now = time.monotonic()

    window_seconds = max(1, FAILED_LOGIN_WINDOW_SECONDS)

    max_attempts = max(1, MAX_FAILED_LOGIN_ATTEMPTS)

    block_seconds = max(1, LOGIN_BLOCK_SECONDS)

    with FAILED_LOGIN_LOCK:

        state = FAILED_LOGINS_BY_SOURCE.get(source)

```

```

        if not state or now - float(state.get("first_failed_at", now)) >
window_seconds:

        state = {"count": 0, "first_failed_at": now, "blocked_until": 0}

        FAILED_LOGINS_BY_SOURCE[source] = state

state["count"] = int(state.get("count", 0)) + 1
if state["count"] >= max_attempts:

    state["blocked_until"] = now + block_seconds

    return True, block_seconds

return False, max_attempts - state["count"]

```

```

def reset_failed_login_source(client_ip: str):

    source = normalize_login_source(client_ip)

    with FAILED_LOGIN_LOCK:

        FAILED_LOGINS_BY_SOURCE.pop(source, None)

```

```

def sanitize_filename(filename: str) -> str:

    if not filename:

        return "file.bin"

    clean = os.path.basename(filename)

    clean = re.sub(r"[^A-Za-z0-9._-]", "_", clean)

    return clean[:180] if clean else "file.bin"

```

```

def decode_file_content(file_content_b64: str):

    try:

```

```

        file_bytes = base64.b64decode(file_content_b64.encode("utf-8"),
validate=True)

        return True, file_bytes

    except Exception:

        return False, b""


def find_windows_defender_scanner:()

    candidates[] =

    program_files = os.environ.get("ProgramFiles")

    if program_files:

        candidates.append(Path(program_files) / "Windows Defender" /
"MpCmdRun.exe")

    program_data = os.environ.get("ProgramData")

    if program_data:

        platform_root = Path(program_data) / "Microsoft" / "Windows Defender" /
"Platform"

        try:

            platform_candidates = [p for p in platform_root.glob("*MpCmdRun.exe")
if p.is_file()]

            platform_candidates.sort(key=lambda p: p.stat().st_mtime,
reverse=True)

            candidates.extend(platform_candidates)

        except Exception:

            pass

    for candidate in candidates:

        try:

```

```

        if candidate.is_file():
            return str(candidate)
    except Exception:
        continue

    return shutil.which("MpCmdRun.exe")

def build_antivirus_command(scan_path: Path):
    if ANTIVIRUS_SCAN_COMMAND:
        try:
            command_parts = shlex.split(ANTIVIRUS_SCAN_COMMAND,
posix=(os.name != "nt"))
        except ValueError:
            return [], ""
        if not command_parts:
            return [], ""
        scan_path_text = str(scan_path)
        if any("{path}" in part for part in command_parts):
            command_parts = [part.replace("{path}", scan_path_text) for part in
command_parts]
        else:
            command_parts.append(scan_path_text)
        return "custom", command_parts

    clamdscan = shutil.which("clamdscan")
    if clamdscan:
        return "clamav", [clamdscan, "--no-summary", str(scan_path)]

```

```

clamscan = shutil.which("clamscan")

if clamscan:
    return "clamav", [clamscan, "--no-summary", str(scan_path)]

defender = find_windows_defender_scanner()

if defender:
    return "defender", [defender, "-Scan", "-ScanType", "3", "-File",
str(scan_path), "-DisableRemediation"]

return[] ,""

def antivirus_output_indicates_malware(scanner_name: str, return_code: int,
output_text: str) -> bool:
    text = (output_text or "").lower()

    if scanner_name == "clamav:":
        return return_code == 1 or "found" in text or "infected files: 1" in text

    if scanner_name == "defender:":
        threat_markers) =
"        threat,"
"        found,"
"        malware,"
"        virus,"
"        infected,"
"        detected,"
"        eicar,"
(
        clean_markers) =
"        no threats,"

```

```

"        no threat,"
"        found no threats,"
"        threats found: 0,"
"        found 0 threats,"
"        scan completed successfully,"
(
    return any(marker in text for marker in threat_markers) and not any(marker
in text for marker in clean_markers)

    return return_code == 1 or "malware" in text or "virus" in text or "infected" in
text

```

```

def scan_file_bytes_for_malware(file_bytes: bytes, original_name: str):

    if not ANTIVIRUS_SCAN_ENABLED:

        return True, "SCAN_DISABLED"

    if EICAR_TEST_SIGNATURE in file_bytes:

        return False, "FILE_REJECTED_MALWARE"

    UPLOAD_SCAN_TMP_DIR.mkdir(parents=True, exist_ok=True)

    suffix = Path(sanitize_filename(original_name)).suffix[:24] or ".upload"

    fd, temp_path_text = tempfile.mkstemp(prefix="classify_scan_", suffix=suffix,
dir=str(UPLOAD_SCAN_TMP_DIR))

    temp_path = Path(temp_path_text)

    try:

        with os.fdopen(fd, "wb") as f:

            f.write(file_bytes)

```

```

scanner_name, command = build_antivirus_command(temp_path)

if not command:

    if ANTIVIRUS_FAIL_OPEN:

        print("[AV] No antivirus scanner found; upload allowed because
CLASSIFY_AV_FAIL_OPEN is enabled.")

        return True, "SCAN_UNAVAILABLE"

    return False, "FILE_SCAN_UNAVAILABLE"

try:

    completed = subprocess.run)

        command,

        stdout=subprocess.PIPE,

        stderr=subprocess.PIPE,

        timeout=max(1, ANTIVIRUS_SCAN_TIMEOUT_SECONDS),

        check=False,

        text=True,

        encoding="utf-8,"

        errors="replace,"

(

except subprocess.TimeoutExpired:

    return False, "FILE_SCAN_FAILED"

except Exception as exc:

    print(f"[AV] Scanner could not run: {exc}")

    return False, "FILE_SCAN_FAILED"

output_text = f"{completed.stdout or ''}\n{completed.stderr or ''}"

if antivirus_output_indicates_malware(scanner_name,
completed.returncode, output_text):

    return False, "FILE_REJECTED_MALWARE"

```



```

        if completed.returncode == 0:
            return True, "SCAN_CLEAN"

        print(f"[AV] Scanner failed with exit code {completed.returncode}:
{output_text[500:]}")

        return False, "FILE_SCAN_FAILED"

    finally:

        try:

            temp_path.unlink(missing_ok=True)

        except Exception:

            pass

def save_base64_file(file_content_b64: str, original_name: str, relative_dir: Path):

    ok_decode, file_bytes = decode_file_content(file_content_b64)

    if not ok_decode:

        return False, "BAD_FILE_CONTENT"

    if len(file_bytes) == 0:

        return False, "EMPTY_FILE"

    if len(file_bytes) > MAX_FILE_SIZE:

        return False, "FILE_TOO_LARGE"

    ok_scan, scan_result = scan_file_bytes_for_malware(file_bytes,
original_name)

    if not ok_scan:

        return False, scan_result

```

```

safe_name = sanitize_filename(original_name)

target_dir = STORAGE_ROOT / relative_dir
target_dir.mkdir(parents=True, exist_ok=True)

target_path = target_dir / safe_name
stem = target_path.stem
suffix = target_path.suffix
counter = 1
while target_path.exists:()
    target_path = target_dir / f"{stem}_{counter}{suffix}"
    counter += 1

with open(target_path, "wb") as f:
    f.write(file_bytes)

return True, str(target_path)

def delete_stored_file(file_path: str):
    try:
        if not file_path:
            return True

        target = Path(file_path).resolve()
        storage_root = STORAGE_ROOT.resolve()
        if storage_root not in target.parents and target != storage_root:
            return False

        if target.exists() and target.is_file:()
            target.unlink()

```

```

        return True
    except Exception:
        return False

def is_safe_storage_path(file_path: str):
    try:
        return resolve_storage_path(file_path) is not None
    except Exception:
        return False

def delete_stored_tree(path_value: str):
    try:
        if not path_value:
            return True

        target = Path(path_value).resolve()
        storage_root = STORAGE_ROOT.resolve()
        if target != storage_root and storage_root not in target.parents:
            return False

        if target.exists() and target.is_dir():
            shutil.rmtree(target, ignore_errors=True)

        return True
    except Exception:
        return False

----- #

```

#Authorization helpers

----- #

```
UNAUTHENTICATED_REQUESTS = {"LOGIN_REQUEST", "SIGNUP_REQUEST"}
```

```
ADMIN_ONLY_REQUESTS =
```

```
"  GET_ADMIN_OVERVIEW_REQUEST,"
"  ADMIN_CREATE_SCHOOL_REQUEST,"
"  ADMIN_UPDATE_SCHOOL_REQUEST,"
"  ADMIN_DELETE_SCHOOL_REQUEST,"
"  ADMIN_CREATE_USER_REQUEST,"
"  ADMIN_UPDATE_USER_REQUEST,"
"  ADMIN_DELETE_USER_REQUEST,"
"  ADMIN_UPDATE_COURSE_REQUEST,"
"  ADMIN_UPDATE_ASSIGNMENT_REQUEST,"
{
```

```
def build_auth_context_from_request(req: dict) -> dict:
```

```
    try:
```

```
        user_id = int(req.get("_auth_user_id", -1))
```

```
    except (TypeError, ValueError):
```

```
        return{}
```

```
    if user_id < 0:
```

```
        return{}
```

```
    if not verify_auth_token(req.get("_auth_token", ""), user_id):
```

```
        return{}
```

```

ok_user, user_ctx = db.get_user_context(user_id)

if not ok_user:
    return {}

status = str(user_ctx.get("status", "")).upper()
if status != "ACTIVE:"
    return {}

return {}

"    user_id": int(user_ctx.get("user_id", user_id)),
"    role": str(user_ctx.get("role", "")).upper(),
"    school_id": int(user_ctx.get("school_id", -1)),
"    name": user_ctx.get("name", ""),
{

def authorize_request(req: dict):
    req_type = req.get("type", "")
    if req_type in UNAUTHENTICATED_REQUESTS:
        return True, None {} ,

    auth_context = build_auth_context_from_request(req)
    if not auth_context or int(auth_context.get("user_id", -1)) < 0:
        return False, err("AUTH_REQUIRED") {} ,

    role = str(auth_context.get("role", "")).upper()
    user_id = int(auth_context.get("user_id", -1))

```

```

school_id = int(auth_context.get("school_id", -1))

if role == "ADMIN:"
    return True, None, auth_context

def deny(msg="FORBIDDEN"):
    return False, err(msg), auth_context

if req_type in ADMIN_ONLY_REQUESTS:
    return deny()

if req_type == "GET_TEACHER_COURSES_REQUEST:"
    return (True, None, auth_context) if role in {"TEACHER", "ADMIN"} and
int(req.get("teacher_id", -999)) == user_id else deny()

if req_type in {"GET_STUDENT_DASHBOARD_REQUEST",
"JOIN_COURSE_BY_CODE_REQUEST", "LEAVE_COURSE_REQUEST",
"DELETE_STUDENT_SUBMISSION_REQUEST",
"MARK_MESSAGE_READ_REQUEST", "MARK_ALL_MESSAGES_READ_REQUEST",
"MARK_GRADE_READ_REQUEST"}:
    if role not in {"STUDENT", "ADMIN"}:
        return deny()

    target_id = int(req.get("student_id", user_id))

    return (True, None, auth_context) if role == "ADMIN" or target_id == user_id
else deny()

if req_type == "UPLOAD_SUBMISSION_REQUEST:"
    if role not in {"STUDENT", "ADMIN"}:
        return deny()

```

```

        return (True, None, auth_context) if role == "ADMIN" or
int(req.get("student_id", -1)) == user_id else deny()

    if req_type in {"APPROVE_USER_REQUEST", "BLOCK_USER_REQUEST",
"UNBLOCK_USER_REQUEST", "GET_USERS_REQUEST",
"GET_SCHOOL_COURSES_REQUEST",
"GET_SCHOOL_ASSIGNMENTS_REQUEST", "UPDATE_SCHOOL_REQUEST"}:

        if role not in {"MANAGER", "ADMIN"}:

            return deny()

            target_school = int(req.get("school_id", school_id)) if req_type !=
"APPROVE_USER_REQUEST" and req_type != "BLOCK_USER_REQUEST" and
req_type != "UNBLOCK_USER_REQUEST" else school_id

            if role != "ADMIN" and target_school != school_id:

                return deny()

            if req_type in {"APPROVE_USER_REQUEST", "BLOCK_USER_REQUEST",
"UNBLOCK_USER_REQUEST"}:

                ok_user, user_ctx = db.get_user_context(int(req.get("user_id", -1)))

                if not ok_user:

                    return False, err(user_ctx), auth_context

                if role != "ADMIN" and int(user_ctx.get("school_id", -1)) != school_id:

                    return deny()

            return True, None, auth_context

    if req_type in {"CREATE_COURSE_REQUEST"}:

        if role not in {"TEACHER", "ADMIN"}:

            return deny()

            if role != "ADMIN" and (int(req.get("teacher_id", -1)) != user_id or
int(req.get("school_id", -1)) != school_id):

                return deny()

            return True, None, auth_context

```

```

    if req_type in {"GET_COURSE_ASSIGNMENTS_REQUEST",
"GET_COURSE_STUDENTS_REQUEST", "GET_COURSE_MATERIALS_REQUEST",
"GET_COURSE_MESSAGES_REQUEST"}:

        course_id = int(req.get("course_id", -1))

        if course_id < 0:

            return deny("MISSING_COURSE_ID")

        return (True, None, auth_context) if db.can_user_access_course(user_id,
role, school_id, course_id) else deny()


    if req_type in {"DELETE_COURSE_REQUEST",
"CREATE_ASSIGNMENT_REQUEST", "CREATE_MATERIAL_REQUEST",
"SAVE_COURSE_MESSAGE_REQUEST",
"UPDATE_COURSE_MEMBER_STATUS_REQUEST",
"DELETE_COURSE_MEMBER_REQUEST"}:

        if role not in {"TEACHER", "ADMIN"}:

            return deny()

        course_id = int(req.get("course_id", -1))

        if course_id < 0:

            return deny("MISSING_COURSE_ID")

        if not db.can_user_access_course(user_id, role, school_id, course_id):

            return deny()

        if req_type == "CREATE_MATERIAL_REQUEST" and role == "TEACHER" and
int(req.get("created_by", user_id) or user_id) != user_id:

            return deny()

        if req_type == "SAVE_COURSE_MESSAGE_REQUEST" and role ==
"TEACHER" and int(req.get("sender_id", user_id) or user_id) != user_id:

            return deny()

    return True, None, auth_context

```



```

    if req_type in {"SET_ASSIGNMENT_CLOSED_REQUEST",
"DELETE_ASSIGNMENT_REQUEST",
"GET_SUBMISSIONS_FOR_ASSIGNMENT_REQUEST"}:

        ok_course, course_id =
db.get_assignment_course_id(int(req.get("assignment_id", -1)))

        if not ok_course:

            return False, err(course_id), auth_context

        return (True, None, auth_context) if db.can_user_access_course(user_id,
role, school_id, course_id) and role in {"TEACHER", "ADMIN"} else deny()


    if req_type == "UPDATE_SUBMISSION_REVIEW_REQUEST:"

        ok_course, course_id =
db.get_submission_course_id(int(req.get("submission_id", -1)))

        if not ok_course:

            return False, err(course_id), auth_context

        if role == "ADMIN:"

            return True, None, auth_context

        if role != "TEACHER" or int(req.get("updated_by", -1)) != user_id:

            return deny()

        return (True, None, auth_context) if db.can_user_access_course(user_id,
role, school_id, course_id) else deny()


    if req_type in {"DELETE_MATERIAL_REQUEST"}:

        ok_course, course_id = db.get_material_course_id(int(req.get("material_id",
-1)))

        if not ok_course:

            return False, err(course_id), auth_context

        return (True, None, auth_context) if db.can_user_access_course(user_id,
role, school_id, course_id) and role in {"TEACHER", "ADMIN"} else deny()


    if req_type in {"DELETE_MESSAGE_REQUEST"}:

```

```

        ok_course, course_id =
db.get_message_course_id(int(req.get("message_id", -1)))

        if not ok_course:

            return False, err(course_id), auth_context

        return (True, None, auth_context) if db.can_user_access_course(user_id,
role, school_id, course_id) and role in {"TEACHER", "ADMIN"} else deny()

if req_type == "DOWNLOAD_FILE_REQUEST:"

    resolved = resolve_storage_path(req.get("file_path", ""))

    if resolved is None:

        return False, err("INVALID_FILE_PATH"), auth_context

    allowed = db.can_user_access_file(user_id, role, school_id, str(resolved))

    return (True, None, auth_context) if allowed else deny()

return True, None, auth_context

```

```

----- #
#Handlers (one per request type)
----- #

def handle_signup(req: dict) -> dict:

    valid, e = require_fields(req, ["name", "email", "password", "role", "school_id"])

    if not valid:

        return e

# checks

if len(req["name"].split(" ")) < 2:

    return err("enter FULL name")

if "@" not in req["email"]:

    return err("enter valid email")

```

```
    ok_db, msg = db.signup_user(req["name"], req["email"], req["password"],
req["role"], req["school_id"])
```

```
    if ok_db:
```

```
        return ok(message=msg)
```

```
    return err(msg)
```

```
def handle_login(req: dict, client_ip="UNKNOWN") -> dict:
```

```
    blocked, retry_after_seconds = get_login_block_status(client_ip)
```

```
    if blocked:
```

```
        return err("LOGIN_RATE_LIMITED",
retry_after_seconds=retry_after_seconds)
```

```
    valid, e = require_fields(req, ["email", "password"])
```

```
    if not valid:
```

```
        return e
```

```
    ok_db, role, user_id, uname, school_id, school_name, school_address,
school_contact_email = db.login_user(req["email"], req["password"])
```

```
    if ok_db:
```

```
        reset_failed_login_source(client_ip)
```

```
        return ok(role=role, user_id=user_id, name=uname, school_id=school_id,
school_name=school_name,
```

```
            school_address=school_address,
school_contact_email=school_contact_email,
```

```
            auth_token=create_auth_token(user_id)(
```

```
    if role == "INVALID_CREDENTIALS:"
```

```
        source_blocked, result = record_failed_login(client_ip)
```

```

    if source_blocked:
        return err("LOGIN_RATE_LIMITED", retry_after_seconds=result)
    return err(role, remaining_attempts=result)

reset_failed_login_source(client_ip)
return err(role)

def handle_approve(req: dict) -> dict:
    valid, e = require_fields(req, ["user_id"])
    if not valid:
        return e

    ok_db, msg = db.approve_user(req["user_id"])
    if ok_db:
        return ok(message=msg)
    return err(msg)

def handle_block(req: dict) -> dict:
    valid, e = require_fields(req, ["user_id"])
    if not valid:
        return e

    ok_db, msg = db.block_user(req["user_id"])
    if ok_db:
        return ok(message=msg)
    return err(msg)

```

```

def handle_unblock(req: dict) -> dict:
    valid, e = require_fields(req, ["user_id"])
    if not valid:
        return e

    ok_db, msg = db.unblock_user(req["user_id"])
    if ok_db:
        return ok(message=msg)
    return err(msg)

```

```

def handle_school_update(req: dict) -> dict:
    valid, e = require_fields(req, ["school_id"])
    if not valid:
        return e

    ok_db, msg = db.update_school(
        req["school_id"],
        req.get("name"),
        req.get("address"),
        req.get("contact_name"),
        req.get("contact_email")
    )

    if ok_db:
        return ok(message=msg)

```

```
return err(msg)
```

```
def handle_get_users(req: dict) -> dict:  
    valid, e = require_fields(req, ["school_id"])  
    if not valid:  
        return e  
  
    ok_db, result = db.get_school_users(req["school_id"])  
    if ok_db:  
        return ok(users=result)  
    return err(result)
```

```
def handle_school_courses(req: dict) -> dict:  
    valid, e = require_fields(req, ["school_id"])  
    if not valid:  
        return e  
  
    ok_db, result = db.get_courses_for_school(req["school_id"])  
    if ok_db:  
        return ok(courses=result)  
    return err(result)
```

```
def handle_school_assignments(req: dict) -> dict:  
    valid, e = require_fields(req, ["school_id"])
```

```
if not valid:
```

```
    return e
```

```
ok_db, result = db.get_assignments_for_school(req["school_id"])
```

```
if ok_db:
```

```
    return ok(assignments=result)
```

```
return err(result)
```

```
def handle_get_teacher_courses(req: dict) -> dict:
```

```
    valid, e = require_fields(req, ["teacher_id"])
```

```
    if not valid:
```

```
        return e
```

```
ok_db, result = db.get_courses_for_teacher(req["teacher_id"])
```

```
if ok_db:
```

```
    return ok(courses=result)
```

```
return err(result)
```

```
def handle_create_course(req: dict) -> dict:
```

```
    valid, e = require_fields(req, ["name", "description", "teacher_id", "school_id"])
```

```
    if not valid:
```

```
        return e
```

```
    ok_db, result = db.create_course(req["name"], req.get("description", ""),  
req["teacher_id"], req["school_id"])
```

```
    if ok_db:
```

```

        return ok(course_id=result)

    return err(result)


def handle_delete_course(req: dict) -> dict:
    valid, e = require_fields(req, ["course_id"])

    if not valid:
        return e

    course_id = int(req["course_id"])

    ok_paths, file_paths = db.get_course_file_paths(course_id)

    if not ok_paths:
        return err(file_paths)

    ok_db, result = db.delete_course(course_id)

    if not ok_db:
        return err(result)

    for file_path in file_paths:
        delete_stored_file(file_path)

    delete_stored_tree(str(STORAGE_ROOT / "assignments" /
f"course_{course_id}"))

    delete_stored_tree(str(STORAGE_ROOT / "materials" / f"course_{course_id}"))

    delete_stored_tree(str(STORAGE_ROOT / "submissions" /
f"course_{course_id}"))

    return ok(message=result, course_id=course_id)

```



```

def handle_get_course_assignments(req: dict) -> dict:
    valid, e = require_fields(req, ["course_id"])
    if not valid:
        return e

    ok_db, result = db.get_assignments_for_course(req["course_id"])
    if ok_db:
        return ok(course_id=req["course_id"],
assignments=normalize_public_file_refs(result))
    return err(result)


def handle_create_assignment(req: dict) -> dict:
    valid, e = require_fields(req, ["course_id", "title", "description", "due_date"])
    if not valid:
        return e

    ai_enabled = bool(req.get("ai_enabled", True))
    attachment_path = req.get("attachment_path")
    attachment_name = req.get("attachment_name", "")
    attachment_b64 = req.get("attachment_content_b64", "")

    if attachment_b64:
        ok_file, file_result = save_base64_file(
            attachment_b64,
            attachment_name,
            Path("assignments") / f"course_{int(req['course_id'])}" / "attachments"

```

```

(
    if not ok_file:
        return err(file_result)

    attachment_path = file_result

    if ai_enabled and not attachment_path:
        return err("RUBRIC_ATTACHMENT_REQUIRED")

    ok_db, result = db.create_assignment)
        req["course_id"],
        req["title"],
        req.get("description", ""),
        req.get("due_date", ""),
        attachment_path,
        ai_enabled
    (
        if ok_db:
            return ok(assignment_id=result, course_id=req["course_id"],
attachment_path=to_public_file_ref(attachment_path or ""))

            delete_stored_file(attachment_path or "")

            return err(result)

def handle_get_course_students(req: dict) -> dict:
    valid, e = require_fields(req, ["course_id"])

    if not valid:
        return e

```

```
ok_db, result = db.get_students_in_course(req["course_id"])
```

```
if ok_db:
```

```
    return ok(course_id=req["course_id"], students=result)
```

```
return err(result)
```

```
def handle_get_submissions_for_assignment(req: dict) -> dict:
```

```
    valid, e = require_fields(req, ["assignment_id"])
```

```
    if not valid:
```

```
        return e
```

```
    assignment_id = req["assignment_id"]
```

```
    ok_db, result = db.get_submissions_for_assignment(assignment_id)
```

```
    if not ok_db:
```

```
        return err(result)
```

```
    enriched[] =
```

```
    for row in result:
```

```
        item = dict(row)
```

```
        ok_feedback, feedback = db.get_feedback(row["submission_id"])
```

```
        if ok_feedback:
```

```
            item["ai_feedback"] = feedback.get("ai_feedback")
```

```
            item["teacher_feedback"] = feedback.get("teacher_feedback")
```

```
        else:
```

```
            item["ai_feedback"] = ""
```

```
            item["teacher_feedback"] = ""
```

```
    enriched.append(item)
```

```

    return ok(assignment_id=assignment_id,
submissions=normalize_public_file_refs(enriched))

def handle_update_submission_review(req: dict) -> dict:
    valid, e = require_fields(req, ["submission_id", "final_score",
"teacher_feedback", "updated_by"])

    if not valid:
        return e

    submission_id = int(req["submission_id"])
    final_score = float(req["final_score"])
    teacher_feedback = req.get("teacher_feedback", "")
    updated_by = int(req["updated_by"])

    ok_grade, _grade = db.get_grade(submission_id)

    if ok_grade:
        ok_db, msg = db.update_final_grade(submission_id, final_score,
updated_by)
    else:
        ok_db, msg = db.save_grade(submission_id, 0, final_score, updated_by)

    if not ok_db:
        return err(msg)

    ok_feedback, feedback_msg = db.save_feedback(submission_id, None,
teacher_feedback)

    if not ok_feedback:
        return err(feedback_msg)

```

```
return ok(message="REVIEW_UPDATED", submission_id=submission_id)
```

```
def handle_get_course_materials(req: dict) -> dict:
```

```
    valid, e = require_fields(req, ["course_id"])
```

```
    if not valid:
```

```
        return e
```

```
    ok_db, result = db.get_materials_for_course(req["course_id"])
```

```
    if ok_db:
```

```
        return ok(course_id=req["course_id"],  
materials=normalize_public_file_refs(result))
```

```
    return err(result)
```

```
def handle_create_material(req: dict) -> dict:
```

```
    valid, e = require_fields(req, ["course_id", "title"])
```

```
    if not valid:
```

```
        return e
```

```
    file_path = req.get("file_path")
```

```
    file_name = req.get("file_name", "")
```

```
    file_b64 = req.get("file_content_b64", "")
```

```
    course_id = int(req["course_id"])
```

```
    if file_b64:
```

```
        ok_file, file_result = save_base64_file)
```

```
        file_b64,
```

```

        file_name,
        Path("materials") / f"course_{course_id}"
    (
        if not ok_file:
            return err(file_result)
        file_path = file_result

    ok_db, result = db.create_material(
        course_id,
        req["title"],
        req.get("description", ""),
        req.get("material_type", "FILE"),
        req.get("created_by"),
        file_path
    (
        if ok_db:
            return ok(material_id=result, course_id=course_id,
file_path=to_public_file_ref(file_path or ""))
            delete_stored_file(file_path or "")
            return err(result)

def handle_save_course_message(req: dict) -> dict:
    valid, e = require_fields(req, ["course_id", "sender_id", "subject", "body",
"state"])
    if not valid:
        return e

    ok_db, result = db.create_message)

```

```

    req["course_id"],
    req["sender_id"],
    req["subject"],
    req["body"],
    req.get("state", "sent")
(
    if ok_db:
        return ok(message_id=result, course_id=req["course_id"])
    return err(result)

def handle_get_course_messages(req: dict) -> dict:
    valid, e = require_fields(req, ["course_id"])
    if not valid:
        return e

    ok_db, result = db.get_messages_for_course(req["course_id"])
    if ok_db:
        return ok(course_id=req["course_id"], messages=result)
    return err(result)

def handle_update_course_member_status(req: dict) -> dict:
    valid, e = require_fields(req, ["course_id", "student_id", "member_status"])
    if not valid:
        return e

    member_status = str(req["member_status"]).upper()

```

```

    if member_status not in ("ACTIVE", "INACTIVE", "PENDING"):
        return err("INVALID_MEMBER_STATUS")

    ok_db, result = db.update_course_member_status(int(req["course_id"]),
int(req["student_id"]), member_status)

    if ok_db:
        return ok(message=result, course_id=int(req["course_id"]),
student_id=int(req["student_id"]), member_status=member_status)

    return err(result)

def handle_delete_course_member(req: dict) -> dict:
    valid, e = require_fields(req, ["course_id", "student_id"])

    if not valid:
        return e

    ok_db, result = db.delete_course_member(int(req["course_id"]),
int(req["student_id"]))

    if ok_db:
        return ok(message=result, course_id=int(req["course_id"]),
student_id=int(req["student_id"]))

    return err(result)

def handle_set_assignment_closed(req: dict) -> dict:
    valid, e = require_fields(req, ["assignment_id", "is_closed"])

    if not valid:
        return e

```



```

assignment_id = int(req["assignment_id"])
is_closed = bool(req["is_closed"])
ok_db, result = db.set_assignment_closed(assignment_id, is_closed)
if ok_db:
    return ok(message=result, assignment_id=assignment_id,
is_closed=is_closed)
    return err(result)

```

```

def handle_delete_assignment(req: dict) -> dict:
    valid, e = require_fields(req, ["assignment_id"])
    if not valid:
        return e

    assignment_id = int(req["assignment_id"])
    ok_assignment, assignment = db.get_assignment(assignment_id)
    if not ok_assignment:
        return err(assignment)
    course_id = int(assignment["course_id"])

    ok_db, result = db.delete_assignment(assignment_id)
    if not ok_db:
        return err(result)

    delete_stored_file(result.get("attachment_path", ""))
    for file_path in result.get("submission_paths", []):
        delete_stored_file(file_path)

    delete_stored_tree(str(STORAGE_ROOT / "submissions" /
f"course_{course_id}" / f"assignment_{assignment_id}"))

```

```
    return ok(message="ASSIGNMENT_DELETED", assignment_id=assignment_id,
course_id=course_id)
```

```
def handle_delete_material(req: dict) -> dict:
```

```
    valid, e = require_fields(req, ["material_id"])
```

```
    if not valid:
```

```
        return e
```

```
    material_id = int(req["material_id"])
```

```
    ok_db, result = db.delete_material(material_id)
```

```
    if not ok_db:
```

```
        return err(result)
```

```
    delete_stored_file(result.get("file_path", ""))
```

```
    return ok(message="MATERIAL_DELETED", material_id=material_id,
course_id=int(result.get("course_id", -1)))
```

```
def handle_delete_message(req: dict) -> dict:
```

```
    valid, e = require_fields(req, ["message_id"])
```

```
    if not valid:
```

```
        return e
```

```
    message_id = int(req["message_id"])
```

```
    ok_db, result = db.delete_message(message_id)
```

```
    if not ok_db:
```

```
        return err(result)
```

```
    return ok(message="MESSAGE_DELETED", message_id=message_id,
course_id=int(result.get("course_id", -1)))
```

```
def handle_download_file(req: dict) -> dict:
```

```
    valid, e = require_fields(req, ["file_path"])
```

```
    if not valid:
```

```
        return e
```

```
    path_obj = resolve_storage_path(req.get("file_path", ""))
```

```
    if path_obj is None:
```

```
        return err("INVALID_FILE_PATH")
```

```
    if not path_obj.exists() or not path_obj.is_file():
```

```
        return err("FILE_NOT_FOUND")
```

```
    file_bytes = path_obj.read_bytes()
```

```
    if len(file_bytes) > MAX_FILE_SIZE:
```

```
        return err("FILE_TOO_LARGE")
```

```
    return ok(file_name=path_obj.name,
file_content_b64=base64.b64encode(file_bytes).decode("utf-8"),
file_path=to_public_file_ref(str(path_obj)))
```

```
def handle_upload_submission(req: dict) -> dict:
```

```
    valid, e = require_fields(req, ["assignment_id", "student_id", "file_name",
"file_content_b64"])
```

```
    if not valid:
```

```
        return e
```

```

assignment_id = int(req["assignment_id"])
student_id = int(req["student_id"])

ok_assignment, assignment = db.get_assignment(assignment_id)
if not ok_assignment:
    return err(assignment)

if bool(assignment.get("is_closed", 0)):
    return err("ASSIGNMENT_CLOSED")

course_id = int(assignment["course_id"])

if hasattr(db, "is_student_active_in_course") and not
db.is_student_active_in_course(course_id, student_id):
    return err("STUDENT_INACTIVE_IN_CLASS")

ok_file, file_result = save_base64_file)

    req["file_content_b64"],
    req["file_name"],

    Path("submissions") / f"course_{course_id}" /
f"assignment_{assignment_id}" / f"student_{student_id}"
(
    if not ok_file:
        return err(file_result)

ai_enabled = bool(assignment.get("ai_enabled", 1))
submission_status = "PENDING_AI" if ai_enabled else "AI_DONE"

```

```
    ok_db, result = db.save_submission(assignment_id, student_id, file_result,
submission_status)
```

```
    if not ok_db:
```

```
        delete_stored_file(file_result)
```

```
        return err(result)
```

```
    submission_id = int(result.get("submission_id", -1))
```

```
    for old_path in result.get("replaced_file_paths", []):
```

```
        delete_stored_file(old_path)
```

```
    if not ai_enabled:
```

```
        ok_grade, grade_msg = db.save_grade(submission_id, 0.0, 0.0, None)
```

```
        if not ok_grade:
```

```
            db.delete_submission_for_student(assignment_id, student_id)
```

```
            delete_stored_file(file_result)
```

```
            return err(grade_msg)
```

```
        ok_feedback, feedback_msg = db.save_feedback(submission_id, "AI is
disabled for this assignment.", None)
```

```
        if not ok_feedback:
```

```
            db.delete_submission_for_student(assignment_id, student_id)
```

```
            delete_stored_file(file_result)
```

```
            return err(feedback_msg)
```

```
    return ok(submission_id=submission_id, assignment_id=assignment_id,
file_path=to_public_file_ref(file_result))
```

```
--- #Added: student-home request handlers---
```

```
def handle_get_student_dashboard(req: dict) -> dict:
```

```

valid, e = require_fields(req, ["student_id"])

if not valid:

    return e


ok_db, result = db.get_student_dashboard(int(req["student_id"]))

if ok_db:

    return ok(student_id=int(req["student_id"]), classes=result.get("classes",
[]), assignments=normalize_public_file_refs(result.get("assignments", [])),
materials=normalize_public_file_refs(result.get("materials", [])),
messages=result.get("messages", []))

    return err(result)


def handle_join_course_by_code(req: dict) -> dict:

    valid, e = require_fields(req, ["student_id", "class_code"])

    if not valid:

        return e


    ok_db, result = db.join_course_by_code(int(req["student_id"]),
req.get("class_code", ""))

    if ok_db:

        return ok(message=result, student_id=int(req["student_id"]),
class_code=req.get("class_code", ""))

        return err(result)


def handle_leave_course(req: dict) -> dict:

    valid, e = require_fields(req, ["student_id", "course_id"])

    if not valid:

        return e

```

```
    ok_db, result = db.leave_course_for_student(int(req["course_id"]),
int(req["student_id"]))
```

```
    if ok_db:
```

```
        return ok(message=result, student_id=int(req["student_id"]),
course_id=int(req["course_id"]))
```

```
    return err(result)
```

```
def handle_delete_student_submission(req: dict) -> dict:
```

```
    valid, e = require_fields(req, ["assignment_id", "student_id"])
```

```
    if not valid:
```

```
        return e
```

```
    ok_db, result = db.delete_submission_for_student(int(req["assignment_id"]),
int(req["student_id"]))
```

```
    if not ok_db:
```

```
        return err(result)
```

```
    for file_path in result.get("file_paths", []):
```

```
        delete_stored_file(file_path)
```

```
    return ok(message="SUBMISSION_DELETED",
assignment_id=int(req["assignment_id"]), student_id=int(req["student_id"]),
submission_id=int(result.get("submission_id", -1)))
```

```
def handle_mark_message_read(req: dict) -> dict:
```

```
    valid, e = require_fields(req, ["student_id", "message_id"])
```

```
    if not valid:
```

```
        return e
```

```

    ok_db, result = db.mark_message_read(int(req["student_id"]),
int(req["message_id"]))

    if ok_db:

        return ok(message=result, student_id=int(req["student_id"]),
message_id=int(req["message_id"]))

    return err(result)

```

```

def handle_mark_all_messages_read(req: dict) -> dict:

    valid, e = require_fields(req, ["student_id"])

    if not valid:

        return e

    ok_db, result = db.mark_all_messages_read(int(req["student_id"]))

    if ok_db:

        return ok(message=result, student_id=int(req["student_id"]))

    return err(result)

```

```

def handle_mark_grade_read(req: dict) -> dict:

    valid, e = require_fields(req, ["student_id", "submission_id"])

    if not valid:

        return e

    ok_db, result = db.mark_grade_read(int(req["student_id"]),
int(req["submission_id"]))

    if ok_db:

```



```

        return ok(message=result, student_id=int(req["student_id"]),
submission_id=int(req["submission_id"]))

    return err(result)

```

```

----- #

```

```

#Admin request handlers

```

```

----- #

```

```

def _admin_only(req: dict) -> bool:

```

```

    # Real authorization is enforced in authorize_request using the authenticated
    user role.

```

```

    return True

```

```

def handle_admin_overview(req: dict) -> dict:

```

```

    ok_db, result = db.admin_get_overview()

```

```

    if ok_db:

```

```

        return ok(**result)

```

```

    return err(result)

```

```

def handle_admin_create_school(req: dict) -> dict:

```

```

    valid, e = require_fields(req, ["name"])

```

```

    if not valid:

```

```

        return e

```

```

    ok_db, result = db.create_school(req.get("name", ""), req.get("address", ""),
req.get("contact_name", ""), req.get("contact_email", ""))

```

```

    if ok_db:

```

```

        return ok(message="SCHOOL_CREATED", school_id=int(result))

    return err(result)


def handle_admin_update_school(req: dict) -> dict:

    valid, e = require_fields(req, ["school_id"])

    if not valid:

        return e

    ok_db, result = db.update_school(int(req["school_id"]), req.get("name"),
    req.get("address"), req.get("contact_name"), req.get("contact_email"))

    if ok_db:

        return ok(message=result)

    return err(result)


def handle_admin_delete_school(req: dict) -> dict:

    valid, e = require_fields(req, ["school_id"])

    if not valid:

        return e

    ok_db, result = db.admin_delete_school(int(req["school_id"]))

    if ok_db:

        return ok(message=result)

    return err(result)


def handle_admin_create_user(req: dict) -> dict:

    valid, e = require_fields(req, ["name", "email", "password", "role", "school_id",
    "status"])

    if not valid:

```

```

        return e

    ok_db, result = db.admin_create_user(req["name"], req["email"],
req["password"], req["role"], int(req["school_id"]), req["status"])

    if ok_db:

        return ok(message=result)

    return err(result)


def handle_admin_update_user(req: dict) -> dict:

    valid, e = require_fields(req, ["user_id", "name", "email", "role", "school_id",
"status"])

    if not valid:

        return e

    ok_db, result = db.admin_update_user(int(req["user_id"]), req["name"],
req["email"], req["role"], int(req["school_id"]), req["status"])

    if ok_db:

        return ok(message=result)

    return err(result)


def handle_admin_delete_user(req: dict) -> dict:

    valid, e = require_fields(req, ["user_id"])

    if not valid:

        return e

    ok_db, result = db.delete_user(int(req["user_id"]))

    if ok_db:

        return ok(message=result)

    return err(result)

```

```

def handle_admin_update_course(req: dict) -> dict:

    valid, e = require_fields(req, ["course_id", "name", "description", "teacher_id",
    "school_id"])

    if not valid:

        return e

    ok_db, result = db.admin_update_course(int(req["course_id"]), req["name"],
    req.get("description", ""), int(req["teacher_id"]), int(req["school_id"]))

    if ok_db:

        return ok(message=result)

    return err(result)


def handle_admin_update_assignment(req: dict) -> dict:

    valid, e = require_fields(req, ["assignment_id", "title", "description", "due_date",
    "ai_enabled", "is_closed"])

    if not valid:

        return e

    ok_db, result = db.admin_update_assignment(int(req["assignment_id"]),
    req["title"], req.get("description", ""), req.get("due_date", ""),
    bool(req.get("ai_enabled")), bool(req.get("is_closed")))

    if ok_db:

        return ok(message=result)

    return err(result)


----- #
#Router: map type -> handler
----- #

ROUTES} =

"  GET_ADMIN_OVERVIEW_REQUEST": handle_admin_overview,

```

" ADMIN_CREATE_SCHOOL_REQUEST": handle_admin_create_school,

" ADMIN_UPDATE_SCHOOL_REQUEST": handle_admin_update_school,

" ADMIN_DELETE_SCHOOL_REQUEST": handle_admin_delete_school,

" ADMIN_CREATE_USER_REQUEST": handle_admin_create_user,

" ADMIN_UPDATE_USER_REQUEST": handle_admin_update_user,

" ADMIN_DELETE_USER_REQUEST": handle_admin_delete_user,

" ADMIN_UPDATE_COURSE_REQUEST": handle_admin_update_course,

" ADMIN_UPDATE_ASSIGNMENT_REQUEST":
handle_admin_update_assignment,

" SIGNUP_REQUEST": handle_signup,

" LOGIN_REQUEST": handle_login,

" APPROVE_USER_REQUEST": handle_approve,

" UPDATE_SCHOOL_REQUEST": handle_school_update,

" GET_USERS_REQUEST": handle_get_users,

" BLOCK_USER_REQUEST": handle_block,

" UNBLOCK_USER_REQUEST": handle_unblock,

" GET_SCHOOL_COURSES_REQUEST": handle_school_courses,

" GET_SCHOOL_ASSIGNMENTS_REQUEST": handle_school_assignments,

" GET_TEACHER_COURSES_REQUEST": handle_get_teacher_courses,

" CREATE_COURSE_REQUEST": handle_create_course,

" DELETE_COURSE_REQUEST": handle_delete_course,

" GET_COURSE_ASSIGNMENTS_REQUEST": handle_get_course_assignments,

" CREATE_ASSIGNMENT_REQUEST": handle_create_assignment,

" SET_ASSIGNMENT_CLOSED_REQUEST": handle_set_assignment_closed,

" DELETE_ASSIGNMENT_REQUEST": handle_delete_assignment,

" GET_COURSE_STUDENTS_REQUEST": handle_get_course_students,

" UPDATE_COURSE_MEMBER_STATUS_REQUEST":
handle_update_course_member_status,

" DELETE_COURSE_MEMBER_REQUEST": handle_delete_course_member,

```

" GET_SUBMISSIONS_FOR_ASSIGNMENT_REQUEST":
handle_get_submissions_for_assignment,

" UPDATE_SUBMISSION_REVIEW_REQUEST":
handle_update_submission_review,

" GET_COURSE_MATERIALS_REQUEST": handle_get_course_materials,
" CREATE_MATERIAL_REQUEST": handle_create_material,
" DELETE_MATERIAL_REQUEST": handle_delete_material,
" SAVE_COURSE_MESSAGE_REQUEST": handle_save_course_message,
" GET_COURSE_MESSAGES_REQUEST": handle_get_course_messages,
" DELETE_MESSAGE_REQUEST": handle_delete_message,
" UPLOAD_SUBMISSION_REQUEST": handle_upload_submission,
" DOWNLOAD_FILE_REQUEST": handle_download_file,
" GET_STUDENT_DASHBOARD_REQUEST": handle_get_student_dashboard,
" JOIN_COURSE_BY_CODE_REQUEST": handle_join_course_by_code,
" LEAVE_COURSE_REQUEST": handle_leave_course,
" DELETE_STUDENT_SUBMISSION_REQUEST":
handle_delete_student_submission,

" MARK_MESSAGE_READ_REQUEST": handle_mark_message_read,
" MARK_ALL_MESSAGES_READ_REQUEST": handle_mark_all_messages_read,
" MARK_GRADE_READ_REQUEST": handle_mark_grade_read,
{

```

```

def handle_request(req: dict, client_ip="UNKNOWN") -> dict:

```

```

    req_type = req.get("type")

```

```

    user_id = req.get("user_id") or req.get("sender_id") or req.get("updated_by")

```

```

    user_name = req.get("name")

```

```

    if not req_type:

```

```

    resp = err("MISSING_TYPE")

    log_action("UNKNOWN", req, resp, client_ip, user_id, user_name)

    return resp

handler = ROUTES.get(req_type)

if not handler:

    resp = err("UNKNOWN_REQUEST", type=req_type)

    log_action(req_type, req, resp, client_ip, user_id, user_name)

    return resp

auth_ok, auth_resp, auth_context = authorize_request(req)

user_id = auth_context.get("user_id") or user_id

user_name = auth_context.get("name") or user_name

if not auth_ok:

    log_action(req_type, req, auth_resp, client_ip, user_id, user_name)

    return auth_resp

try:

    if req_type == "LOGIN_REQUEST:"

        resp = handler(req, client_ip)

    else:

        resp = handler(req)

except Exception as e:

    print("[!] Handler exception:", e)

    resp = err("SERVER_ERROR")

log_action(req_type, req, resp, client_ip, user_id, user_name)

```

```

    return resp

----- #

#Client connection: DH + AES + request loop
----- #

def handle_client(client_sock, addr):

    print(f"[+] Connected: {addr}")

    try:

        client_sock.settimeout(HANDSHAKE_TIMEOUT_SECONDS)

        aes_key = DH_server(client_sock)

        client_sock.settimeout(None)

    while True:

        encrypted_req = recv_by_size(client_sock)

        if not encrypted_req:

            break

        plain_req_bytes = decrypt_message(aes_key, encrypted_req)

        try:

            req = json.loads(plain_req_bytes.decode("utf-8"))

        except json.JSONDecodeError:

            resp = err("BAD_JSON")

        else:

            resp = handle_request(req, addr[0])

        plain_resp = json.dumps(resp).encode("utf-8")

        encrypted_resp = encrypt_message(aes_key, plain_resp)

        send_with_size(client_sock, encrypted_resp)

```



```

except Exception as e:
    print(f"[!] Error {addr}: {e}")
finally:
    client_sock.close()
    print(f"[-] Disconnected: {addr}")

def main:()
    STORAGE_ROOT.mkdir(parents=True, exist_ok=True)

    threading.Thread(target=run_ai_grading_worker, daemon=True,
name="AIGraderWorker").start()

    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.bind((HOST, PORT))
    s.listen()
    print(f"[SERVER] Listening on {HOST}:{PORT}")

    while True:
        conn, addr = s.accept()
        threading.Thread(target=handle_client, args=(conn, addr),
daemon=True).start()

if __name__ == "__main__":
    main()

```

```

Classify_Client.py

import sys
import socket
import json
import hashlib
import threading
import os
import base64
import urllib.request
from pathlib import Path
from PySide6.QtGui import QIcon
from PySide6.QtCore import QObject, Signal, Slot, QUrl
from PySide6.QtGui import QApplication
from PySide6.QtQml import QQmlApplicationEngine
from tcp_by_size import send_with_size, recv_by_size
from DH import DH_client
from AES_e import encrypt_message, decrypt_message
import resources_rc

APP_DIR = Path(sys.executable).resolve().parent if getattr(sys, "frozen", False)
else Path(__file__).resolve().parent

DEFAULT_CLASSIFY_HOST = "127.0.0.1"

DEFAULT_CLASSIFY_PORT = 5555

DEFAULT_DISCOVERY_TIMEOUT_SECONDS = 3

def client_config_path() -> Path:

```

```

override_path = os.getenv("CLASSIFY_CLIENT_CONFIG", "").strip()

if override_path:
    return Path(override_path)

candidates] =
    APP_DIR / "client_config.json,"
    APP_DIR / "_internal" / "client_config.json,"
[
for candidate in candidates:
    if candidate.exists() and candidate.is_file:()
        return candidate
return candidates[0]

def load_client_config() -> dict:
    path = client_config_path()
    if not path.exists() or not path.is_file:()
        return{}
    try:
        with path.open("r", encoding="utf-8") as file:
            data = json.load(file)
            return data if isinstance(data, dict) else{}
    except Exception as exc:
        print(f"[DISCOVERY] Could not read {path}: {exc}")
        return{}

def safe_port(value, default_port=DEFAULT_CLASSIFY_PORT) -> int:

```

```

try:
    port = int(value)
except (TypeError, ValueError):
    return int(default_port)
return port if 1 <= port <= 65535 else int(default_port)

def server_address_from_payload(payload: dict, default_host: str, default_port:
int):
    if not isinstance(payload, dict):
        return default_host, default_port

    source = payload
    for key in ("classify_server", "server"):
        nested = payload.get(key)
        if isinstance(nested, dict):
            source = nested
            break

    host) =
        source.get("host")
        or source.get("ip")
        or source.get("domain")
        or source.get("server_host")
        or default_host
    (
        port = source.get("port", source.get("server_port", default_port))
        return str(host).strip() or default_host, safe_port(port, default_port)

```

```

def fetch_discovery_server_address(discovery_url: str, timeout_seconds: int,
default_host: str, default_port: int):

    request = urllib.request.Request(discovery_url, headers={"Accept":
"application/json"})

    with urllib.request.urlopen(request, timeout=timeout_seconds) as response:

        raw = response.read(1024 * 64)

        payload = json.loads(raw.decode("utf-8"))

        return server_address_from_payload(payload, default_host, default_port)

```

```

def
resolve_configured_server_address(default_host=DEFAULT_CLASSIFY_HOST,
default_port=DEFAULT_CLASSIFY_PORT):

    config = load_client_config()

    fallback_host = str(config.get("fallback_host") or config.get("host") or
default_host).strip() or default_host

    fallback_port = safe_port(config.get("fallback_port", config.get("port",
default_port)), default_port)

    discovery_url = str(config.get("discovery_url", "")).strip()

    timeout_seconds = safe_port(config.get("discovery_timeout_seconds",
DEFAULT_DISCOVERY_TIMEOUT_SECONDS),
DEFAULT_DISCOVERY_TIMEOUT_SECONDS)

    if discovery_url:

        try:

            return fetch_discovery_server_address(discovery_url, timeout_seconds,
fallback_host, fallback_port)

        except Exception as exc:

```

```
print(f"[DISCOVERY] Discovery failed ({discovery_url}): {exc}. Using  
fallback {fallback_host}:{fallback_port}")
```

```
return fallback_host, fallback_port
```

```
def prettify_message(message: str) -> str:
```

```
    raw = (message or "").strip()
```

```
    if not raw:
```

```
        return 'An unexpected error occurred. Please try again'.
```

```
    mapping =
```

```
    '    OK': 'Done successfully, '.  
    '    DOWNLOAD_OK': 'File downloaded successfully, '.  
    '    UNKNOWN_ERROR': 'An unexpected error occurred. Please try again, '.  
    '    INVALID_CREDENTIALS': 'The email or password is incorrect, '.  
    '    LOGIN_RATE_LIMITED': 'Too many failed sign-in attempts from this  
network. Please try again later, '.  
    '    WAITING_APPROVAL': 'Your account is waiting for approval from your  
school manager, '.  
    '    USER_BLOCKED': 'This account has been blocked. Please contact your  
school manager, '.  
    '    USER_ALREADY_EXISTS': 'An account with this email already exists, '.  
    '    USER_CREATED': 'Your account was created successfully, '.  
    '    USER_APPROVED': 'The user was approved successfully, '.  
    '    USER_UNBLOCKED': 'The user was unblocked successfully, '.  
    '    USER_DELETED': 'The user was deleted successfully, '.  
    '    MISSING_FIELDS': 'Some required fields are missing, '.
```

' SCHOOL_ID_REQUIRED': 'Please enter your school ID before continuing,'.

' INVALID_SCHOOL_ID': 'Please enter a valid school ID,'.

' SCHOOL_NOT_FOUND': 'We could not find a school with that ID,'.

' SCHOOL_CREATED': 'The school was created successfully,'.

' SCHOOL_UPDATED': 'School details were updated successfully,'.

' SCHOOL_DELETED': 'The school was deleted successfully,'.

' NO_FIELDS_TO_UPDATE': 'There is nothing to update,'.

' INVALID_ROLE': 'Please choose a valid role,'.

' INVALID_STATUS': 'Please choose a valid status,'.

' TEACHER_NOT_FOUND': 'Please choose a valid teacher,'.

' COURSE_NOT_FOUND': 'The class could not be found,'.

' COURSE_DELETED': 'The class was deleted successfully,'.

' CLASS_NOT_FOUND': 'The class code was not found,'.

' INVALID_CLASS_CODE': 'Please enter a valid class code,'.

' STUDENT_ADDED': 'The student was added successfully,'.

' STUDENT_REMOVED': 'The student was removed successfully,'.

' MEMBER_STATUS_UPDATED': 'The student status was updated successfully,'.

' ASSIGNMENT_NOT_FOUND': 'The assignment could not be found,'.

' ASSIGNMENT_CLOSED': 'This assignment is closed and can no longer be submitted,'.

' ASSIGNMENT_UPDATED': 'The assignment was updated successfully,'.

' ASSIGNMENT_DELETED': 'The assignment was deleted successfully,'.

' ASSIGNMENT_ALREADY_PAST_DUE': 'This due date has already passed. Please choose a future date,'.

' RUBRIC_ATTACHMENT_REQUIRED': 'Please attach instructions file before publishing an AI-checked assignment,'.

' RUBRIC_ATTACHMENT_MISSING': 'This AI assignment is missing its rubric file,'.

' MATERIAL_NOT_FOUND': 'The study material could not be found,'.

' MATERIAL_DELETED': 'The material was deleted successfully,'.

' MESSAGE_NOT_FOUND': 'The message could not be found,'.

' MESSAGE_DELETED': 'The message was deleted successfully,'.

' MESSAGE_MARKED_READ': 'The message was marked as read,'.

' ALL_MESSAGES_MARKED_READ': 'All messages were marked as read,'.

' SUBMISSION_NOT_FOUND': 'The submission could not be found,'.

' SUBMISSION_DELETED': 'Your submission was deleted successfully,'.

' REVIEW_UPDATED': 'The review was updated successfully,'.

' STATUS_UPDATED': 'The status was updated successfully,'.

' GRADE_SAVED': 'The grade was saved successfully,'.

' GRADE_UPDATED': 'The grade was updated successfully,'.

' GRADE_MARKED_READ': 'The grade was marked as read,'.

' AI_RESULTS_DELETED': 'The AI results were deleted successfully,'.

' NO_PENDING_SUBMISSIONS': 'There are no pending submissions right now,'.

' CLAIM_CONFLICT': 'This submission is already being processed. Please try again in a moment,'.

' NO_PERMISSION': 'You do not have permission to perform this action,'.

' NOT_AUTHENTICATED': 'Please sign in before continuing,'.

' AUTH_REQUIRED': 'Please sign in before continuing,'.

' FORBIDDEN': 'You do not have permission to perform this action,'.

' UNKNOWN_REQUEST': 'The app sent an unsupported request to the server,'.

' FILE_NOT_FOUND': 'The selected file could not be found,'.

' EMPTY_FILE': 'The selected file is empty,'.

' EMPTY_FILE_CONTENT': 'The downloaded file is empty,'.

' FILE_TOO_LARGE': 'File is too large. Maximum allowed size is 25MB,'.

' BAD_FILE_CONTENT': 'The file content is invalid,'.


```

'    FILE_REJECTED_MALWARE': 'This file was blocked by the server security
scan,',
'    FILE_SCAN_UNAVAILABLE': 'The server could not scan this file for viruses
right now,',
'    FILE_SCAN_FAILED': 'The server security scan failed. Please try again
later,',
'    DOWNLOAD_NOT_ALLOWED': 'You do not have permission to download
this file,',
'    INVALID_FILE_REFERENCE': 'The file reference is invalid,',
'    INVALID_FILE_PATH': 'The selected file could not be accessed,',
'    NETWORK_ERROR': 'Could not connect to the server. Please try again,',
'    NETWORK_TIMEOUT': 'The server took too long to respond. Please try
again,',
'    SERVER_UNAVAILABLE': 'The server is currently unavailable. Please try
again later,',

```

```
{
```

```
    if raw in mapping:
```

```
        return mapping[raw]
```

```
    if raw.startswith('VALIDATION_ERROR'):
```

```
        return 'The selected file could not be validated'.
```

```
    if raw.startswith('enter FULL name'):
```

```
        return 'Please enter your full name'.
```

```
    if raw.startswith('enter valid email'):
```

```
        return 'Please enter a valid email address'.
```

```
    if raw.startswith('UNIQUE constraint failed'):
```

```
        return 'This item already exists'.
```

```
    if raw.startswith('FOREIGN KEY constraint failed'):
```

```
        return 'The requested action could not be completed because related data
is missing'.
```

```
    if raw in {'Server closed connection', 'Connection reset by peer'}:
```

```
        return 'The connection to the server was lost. Please try again'.
```

```
    return raw.replace(' ', '_')
```

```
----- #
```

```
#TCP Auth Client (persistent connection)
```

```
----- #
```

```
class AuthClient:
```

```
    def __init__(self, host=DEFAULT_CLASSIFY_HOST,
port=DEFAULT_CLASSIFY_PORT):
```

```
        self.fallback_host = host or DEFAULT_CLASSIFY_HOST
```

```
        self.fallback_port = safe_port(port, DEFAULT_CLASSIFY_PORT)
```

```
        self.host, self.port = resolve_configured_server_address(self.fallback_host,
self.fallback_port)
```

```
        self.timeout_seconds = 30
```

```
        self.sock = None
```

```
        self.aes_key = None
```

```
        self.current_user{} =
```

```
        self._lock = threading.Lock()
```

```
    def refresh_server_address(self):
```

```
        host, port = resolve_configured_server_address(self.fallback_host,
self.fallback_port)
```

```
        if host != self.host or port != self.port:
```

```

        print(f"[DISCOVERY] Classify server changed to {host}:{port}")

    self.host = host

    self.port = port

def connect(self):

    if self.sock is not None and self.aes_key is not None:

        return

    self.refresh_server_address()

    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    try:

        sock.settimeout(self.timeout_seconds)

        sock.connect((self.host, self.port))

        aes_key = DH_client(sock)

    except Exception:

        try:

            sock.close()

        except Exception:

            pass

        raise

    self.sock = sock

    self.aes_key = aes_key

def close(self):

    if self.sock is not None:

        try:

            self.sock.close()

```

```

        except Exception:
            pass

    self.sock = None

    self.aes_key = None

def _send_request_once(self, req: dict) -> dict:
    with self._lock:
        if self.sock is None or self.aes_key is None:
            self.connect()

        outgoing = dict(req)

        if self.current_user and outgoing.get("type") not in {"LOGIN_REQUEST",
"SIGNUP_REQUEST"}:
            outgoing["_auth_user_id"] = int(self.current_user.get("user_id", -1))
            outgoing["_auth_token"] = str(self.current_user.get("auth_token", ""))

        plain = json.dumps(outgoing).encode("utf-8")
        encrypted = encrypt_message(self.aes_key, plain)
        send_with_size(self.sock, encrypted)

        encrypted_resp = recv_by_size(self.sock)
        if not encrypted_resp:
            raise ConnectionError("Server closed connection")

        plain_resp = decrypt_message(self.aes_key,
encrypted_resp).decode("utf-8")
        return json.loads(plain_resp)

def send_request(self, req: dict) -> dict:

```

```

try:
    return self._send_request_once(req)
except Exception:
    self.close()
    self.connect()
    return self._send_request_once(req)

def clear_user(self):
    with self._lock:
        self.current_user{} =

def _load_file_payload(self, local_path: str):
    path = (local_path or "").replace("file:///", "")
    path = os.path.normpath(path)
    with open(path, "rb") as f:
        file_bytes = f.read()

    return os.path.basename(path),
base64.b64encode(file_bytes).decode("utf-8")

----- #   APIs-----

def login(self, email: str, password: str) -> dict:
    email = email.strip().lower()
    password_hash = hashlib.sha256(password.encode("utf-8")).hexdigest()

    req = {"type": "LOGIN_REQUEST", "email": email, "password":
password_hash}

    resp = self.send_request(req)

    if resp.get("status") == "OK:"

        self.current_user{} =

"        user_id": int(resp.get("user_id", -1)),

```

```

"        role": str(resp.get("role", "")).upper,()
"        school_id": int(resp.get("school_id", -1)),
"        auth_token": str(resp.get("auth_token", "")),
{
    else:
        self.current_user{} =
    return resp

def signup(self, name, email: str, password: str, role: str, school_id) -> dict:
    email = email.strip().lower()
    password_hash = hashlib.sha256(password.encode("utf-8")).hexdigest()
    req = {"type": "SIGNUP_REQUEST", "name": name, "email": email,
"password": password_hash, "role": role, "school_id": school_id}
    return self.send_request(req)

def approve_user(self, user_id: int) -> dict:
    req = {"type": "APPROVE_USER_REQUEST", "user_id": int(user_id)}
    return self.send_request(req)

def get_teacher_courses(self, teacher_id: int) -> dict:
    return self.send_request({"type": "GET_TEACHER_COURSES_REQUEST",
"teacher_id": int(teacher_id)})

def create_course(self, name: str, description: str, teacher_id: int, school_id:
int) -> dict:
    req} =
"        type": "CREATE_COURSE_REQUEST",
"        name": name,
"        description": description,

```

```

        "teacher_id": int(teacher_id),
        "school_id": int(school_id)
    }

    return self.send_request(req)

def delete_course(self, course_id: int) -> dict:

    return self.send_request({"type": "DELETE_COURSE_REQUEST",
"course_id": int(course_id)})

def get_course_assignments(self, course_id: int) -> dict:

    return self.send_request({"type":
"GET_COURSE_ASSIGNMENTS_REQUEST", "course_id": int(course_id)})

def create_assignment(self, course_id: int, title: str, description: str, due_date:
str, ai_enabled: bool = True, attachment_path: str = "") -> dict:

    req =
    "type": "CREATE_ASSIGNMENT_REQUEST",
    "course_id": int(course_id),
    "title": title,
    "description": description,
    "due_date": due_date,
    "ai_enabled": bool(ai_enabled),
    "attachment_path": "" :
    {
        if attachment_path:

            attachment_name, attachment_b64 =
self._load_file_payload(attachment_path)

            req["attachment_name"] = attachment_name

            req["attachment_content_b64"] = attachment_b64

```

```

        return self.send_request(req)

    def set_assignment_closed(self, assignment_id: int, is_closed: bool) -> dict:
        return self.send_request({"type": "SET_ASSIGNMENT_CLOSED_REQUEST",
                                   "assignment_id": int(assignment_id), "is_closed": bool(is_closed)})

    def delete_assignment(self, assignment_id: int) -> dict:
        return self.send_request({"type": "DELETE_ASSIGNMENT_REQUEST",
                                   "assignment_id": int(assignment_id)})

    def get_course_students(self, course_id: int) -> dict:
        return self.send_request({"type": "GET_COURSE_STUDENTS_REQUEST",
                                   "course_id": int(course_id)})

    def get_submissions_for_assignment(self, assignment_id: int) -> dict:
        return self.send_request({"type":
                                   "GET_SUBMISSIONS_FOR_ASSIGNMENT_REQUEST", "assignment_id":
                                   int(assignment_id)})

    def update_submission_review(self, submission_id: int, final_score: float,
                                  teacher_feedback: str, updated_by: int) -> dict:
        req = {
            "type": "UPDATE_SUBMISSION_REVIEW_REQUEST",
            "submission_id": int(submission_id),
            "final_score": float(final_score),
            "teacher_feedback": teacher_feedback,
            "updated_by": int(updated_by)
        }
        return self.send_request(req)

```



```

def get_course_materials(self, course_id: int) -> dict:

    return self.send_request({"type": "GET_COURSE_MATERIALS_REQUEST",
"course_id": int(course_id)})

def create_material(self, course_id: int, title: str, description: str,
material_type: str, created_by: int, file_path: str = "") -> dict:

    req = {
        "type": "CREATE_MATERIAL_REQUEST",
        "course_id": int(course_id),
        "title": title,
        "description": description,
        "material_type": material_type,
        "created_by": int(created_by),
        "file_path": "" :
    }

    if file_path:
        file_name, file_b64 = self._load_file_payload(file_path)

        req["file_name"] = file_name

        req["file_content_b64"] = file_b64

    return self.send_request(req)

def delete_material(self, material_id: int) -> dict:

    return self.send_request({"type": "DELETE_MATERIAL_REQUEST",
"material_id": int(material_id)})

def save_course_message(self, course_id: int, sender_id: int, subject: str,
body: str, state: str) -> dict:

    req = {
        "type": "SAVE_COURSE_MESSAGE_REQUEST",

```

```

        "course_id": int(course_id),
        "sender_id": int(sender_id),
        "subject": subject,
        "body": body,
        "state": state
    }

    return self.send_request(req)

def get_course_messages(self, course_id: int) -> dict:
    return self.send_request({"type": "GET_COURSE_MESSAGES_REQUEST",
"course_id": int(course_id)})

def delete_message(self, message_id: int) -> dict:
    return self.send_request({"type": "DELETE_MESSAGE_REQUEST",
"message_id": int(message_id)})

def upload_submission(self, assignment_id: int, student_id: int, file_path: str)
-> dict:
    file_name, file_b64 = self._load_file_payload(file_path)
    req =
    "type": "UPLOAD_SUBMISSION_REQUEST",
    "assignment_id": int(assignment_id),
    "student_id": int(student_id),
    "file_name": file_name,
    "file_content_b64": file_b64
    {
        return self.send_request(req)

```

```
def update_course_member_status(self, course_id: int, student_id: int,
member_status: str) -> dict:
```

```
    return self.send_request})

"    type": "UPDATE_COURSE_MEMBER_STATUS_REQUEST,"
"    course_id": int(course_id),
"    student_id": int(student_id),
"    member_status": member_status
({
```

```
def delete_course_member(self, course_id: int, student_id: int) -> dict:
```

```
    return self.send_request})

"    type": "DELETE_COURSE_MEMBER_REQUEST,"
"    course_id": int(course_id),
"    student_id": int(student_id)
({
```

--- # Added: student-home APIs---

```
def get_student_dashboard(self, student_id: int) -> dict:
```

```
    return self.send_request({"type": "GET_STUDENT_DASHBOARD_REQUEST",
"student_id": int(student_id)})
```

```
def join_course_by_code(self, student_id: int, class_code: str) -> dict:
```

```
    return self.send_request({"type": "JOIN_COURSE_BY_CODE_REQUEST",
"student_id": int(student_id), "class_code": class_code})
```

```
def leave_course(self, student_id: int, course_id: int) -> dict:
```

```
    return self.send_request({"type": "LEAVE_COURSE_REQUEST",
"student_id": int(student_id), "course_id": int(course_id)})
```

```
def delete_student_submission(self, assignment_id: int, student_id: int) -> dict:
```

```
    return self.send_request({"type":  
"DELETE_STUDENT_SUBMISSION_REQUEST", "assignment_id":  
int(assignment_id), "student_id": int(student_id)})
```

```
def mark_message_read(self, student_id: int, message_id: int) -> dict:
```

```
    return self.send_request({"type": "MARK_MESSAGE_READ_REQUEST",  
"student_id": int(student_id), "message_id": int(message_id)})
```

```
def mark_all_messages_read(self, student_id: int) -> dict:
```

```
    return self.send_request({"type":  
"MARK_ALL_MESSAGES_READ_REQUEST", "student_id": int(student_id)})
```

```
def mark_grade_read(self, student_id: int, submission_id: int) -> dict:
```

```
    return self.send_request({"type": "MARK_GRADE_READ_REQUEST",  
"student_id": int(student_id), "submission_id": int(submission_id)})
```

```
def upload_submission_text(self, assignment_id: int, student_id: int,  
submission_text: str) -> dict:
```

```
    file_name = f"submission_{int(assignment_id)}_{int(student_id)}.txt"  
  
    file_b64 = base64.b64encode((submission_text or "").encode("utf-  
8")).decode("utf-8")
```

```
    req =  
"  
    type": "UPLOAD_SUBMISSION_REQUEST",  
"  
    assignment_id": int(assignment_id),  
"  
    student_id": int(student_id),  
"  
    file_name": file_name,  
"  
    file_content_b64": file_b64  
{
```

```
    return self.send_request(req)
```

```

def download_file(self, file_path: str, save_dir: str = "") -> tuple[dict, str]:

    resp = self.send_request({"type": "DOWNLOAD_FILE_REQUEST",
"file_path": file_path})

    if resp.get("status") != "OK:"

        return resp"" ,

    file_name = resp.get("file_name", "downloaded_file")

    file_b64 = resp.get("file_content_b64", "")

    if not file_b64:

        return {"status": "ERROR", "message": "EMPTY_FILE_CONTENT"}"" ,

    target_dir = save_dir.strip() if save_dir else
os.path.join(os.path.expanduser("~"), "Downloads", "ClassifyDownloads")

    os.makedirs(target_dir, exist_ok=True)

    file_bytes = base64.b64decode(file_b64.encode("utf-8"))

    target_path = os.path.join(target_dir, file_name)

    base_name, ext = os.path.splitext(target_path)

    counter = 1

    while os.path.exists(target_path):

        target_path = f"{base_name}_{counter}{ext}"

        counter += 1

    with open(target_path, "wb") as f:

        f.write(file_bytes)

    return resp, target_path

```

```

----- #

#QML-facing object (exposed as "auth")

----- #

class Auth(QObject):

    loginResult = Signal(bool, str, str, int, str, int, str, str, str)

    signupResult = Signal(bool, str)

    schoolUpdateResult = Signal(bool, str)

    approveResult = Signal(bool, str)

    getUsersResult = Signal(bool, str, list)

    getCoursesResult = Signal(bool, str, list)

    getAssignmentsResult = Signal(bool, str, list)

    teacherCoursesResult = Signal(bool, str, list)

    courseAssignmentsResult = Signal(bool, str, int, list)

    courseStudentsResult = Signal(bool, str, int, list)

    submissionsResult = Signal(bool, str, int, list)

    createCourseResult = Signal(bool, str, int)

    createAssignmentResult = Signal(bool, str, int, int)

    setAssignmentClosedResult = Signal(bool, str, int, bool)

    deleteAssignmentResult = Signal(bool, str, int, int)

    updateSubmissionReviewResult = Signal(bool, str, int)

    courseMaterialsResult = Signal(bool, str, int, list)

    createMaterialResult = Signal(bool, str, int, int)

    deleteMaterialResult = Signal(bool, str, int, int)

    courseMessagesResult = Signal(bool, str, int, list)

    saveCourseMessageResult = Signal(bool, str, int, int)

```

```

deleteMessageResult = Signal(bool, str, int, int)
uploadSubmissionResult = Signal(bool, str, int, int)
updateCourseMemberStatusResult = Signal(bool, str, int, int, str)
deleteCourseMemberResult = Signal(bool, str, int, int)
deleteCourseResult = Signal(bool, str, int)
downloadFileResult = Signal(bool, str, str, str)
localFileValidationResult = Signal(bool, str, str, str, int)
studentDashboardResult = Signal(bool, str, int, list, list, list, list)
joinCourseResult = Signal(bool, str, int, str)
leaveCourseResult = Signal(bool, str, int, int)
deleteSubmissionResult = Signal(bool, str, int, int)
markMessageReadResult = Signal(bool, str, int, int)
markAllMessagesReadResult = Signal(bool, str, int)
markGradeReadResult = Signal(bool, str, int, int)
adminOverviewResult = Signal(bool, str, dict)
adminActionResult = Signal(bool, str, str)

```

```

def __init__(self, host="127.0.0.1", port=5555):
    super().__init__()
    self._client = AuthClient(host, port)

    try:
        self._client.connect()
    except Exception as e:
        print(" Initial connect failed:", e)

```

```
@ Slot()
```

```
def logout(self):
```

```

        self._client.clear_user()

    @ Slot(int, str, str, str, str)
    def updateSchool(self, school_id, name, address, contact_name,
contact_email):
        try:
            payload =
"            type": "UPDATE_SCHOOL_REQUEST,"
"            school_id": int(school_id),
"            name": name,
"            address": address,
"            contact_name": contact_name,
"            contact_email": contact_email
{
            resp = self._client.send_request(payload)
            if resp.get("status") == "OK:"
                self.schoolUpdateResult.emit(True,
prettify_message(resp.get("message", "OK")))
            else:
                self.schoolUpdateResult.emit(False,
prettify_message(resp.get("message", "UNKNOWN_ERROR")))
        except Exception as e:
            self.schoolUpdateResult.emit(False, prettify_message(str(e)))

    @ Slot(int)
    def get_school_assignments(self, school_id):
        threading.Thread(target=self._worker_get_school_assignments,
args=(school_id,), daemon=True).start()

```



```

def _worker_get_school_assignments(self, school_id):
    try:
        req = {"type": "GET_SCHOOL_ASSIGNMENTS_REQUEST", "school_id":
school_id}

        resp = self._client.send_request(req)

        if resp.get("status") == "OK:"

            self.getAssignmentsResult.emit(True, "OK", resp.get("assignments",
[]))

        else:

            self.getAssignmentsResult.emit(False,
prettify_message(resp.get("message", "ERROR")), [])

    except Exception as e:

        self.getAssignmentsResult.emit(False, f"NETWORK_ERROR: {e}", [])

```

@ Slot(int)

```

def get_school_courses(self, school_id):

    threading.Thread(target=self._worker_get_school_courses,
args=(school_id,), daemon=True).start()

```

```

def _worker_get_school_courses(self, school_id):
    try:
        req = {"type": "GET_SCHOOL_COURSES_REQUEST", "school_id":
school_id}

        resp = self._client.send_request(req)

        if resp.get("status") == "OK:"

            self.getCoursesResult.emit(True, "OK", resp.get("courses", []))

        else:

            self.getCoursesResult.emit(False,
prettify_message(resp.get("message", "ERROR")), [])

    except Exception as e:

```

```
self.getCoursesResult.emit(False, f"NETWORK_ERROR: {e}", [])
```

```
@ Slot(int)
```

```
def approveUser(self, uid):
```

```
    threading.Thread(target=self._worker_approve_user, args=(uid,),  
daemon=True).start()
```

```
def _worker_approve_user(self, uid):
```

```
    try:
```

```
        req = {"type": "APPROVE_USER_REQUEST", "user_id": uid}
```

```
        self._client.send_request(req)
```

```
    except Exception as e:
```

```
        print("ERROR:", e)
```

```
@ Slot(int)
```

```
def blockUser(self, uid):
```

```
    threading.Thread(target=self._worker_block_user, args=(uid,),  
daemon=True).start()
```

```
def _worker_block_user(self, uid):
```

```
    try:
```

```
        req = {"type": "BLOCK_USER_REQUEST", "user_id": uid}
```

```
        self._client.send_request(req)
```

```
    except Exception as e:
```

```
        print("ERROR:", e)
```

```
@ Slot(int)
```

```
def unblockUser(self, uid):
```

```

        threading.Thread(target=self._worker_unblock_user, args=(uid,),
daemon=True).start()

```

```

def _worker_unblock_user(self, uid):
    try:
        req = {"type": "UNBLOCK_USER_REQUEST", "user_id": uid}
        self._client.send_request(req)
    except Exception as e:
        print("ERROR:", e)

```

@ Slot(int)

```

def getUsers(self, school_id):
    threading.Thread(target=self._worker_get_users, args=(school_id,),
daemon=True).start()

```

```

def _worker_get_users(self, school_id):
    try:
        req = {"type": "GET_USERS_REQUEST", "school_id": school_id}
        resp = self._client.send_request(req)
        if resp.get("status") == "OK:"
            self.getUsersResult.emit(True, "OK", resp.get("users", []))
        else:
            self.getUsersResult.emit(False, prettify_message(resp.get("message",
"ERROR")), [])
    except Exception as e:
        self.getUsersResult.emit(False, f"NETWORK_ERROR: {e}", [])

```

@ Slot(int)

```

def get_teacher_courses(self, teacher_id):

```

```

        threading.Thread(target=self._worker_get_teacher_courses,
args=(teacher_id,), daemon=True).start()

```

```

def _worker_get_teacher_courses(self, teacher_id):
    try:
        resp = self._client.get_teacher_courses(teacher_id)
        if resp.get("status") == "OK:"
            self.teacherCoursesResult.emit(True, "OK", resp.get("courses", []))
        else:
            self.teacherCoursesResult.emit(False,
prettify_message(resp.get("message", "ERROR")), [])
    except Exception as e:
        self.teacherCoursesResult.emit(False, f"NETWORK_ERROR: {e}", [])

```

@ Slot(str, str, int, int)

```

def create_course(self, name, description, teacher_id, school_id):
    threading.Thread(target=self._worker_create_course, args=(name,
description, teacher_id, school_id), daemon=True).start()

def _worker_create_course(self, name, description, teacher_id, school_id):
    try:
        resp = self._client.create_course(name, description, teacher_id,
school_id)
        if resp.get("status") == "OK:"
            self.createCourseResult.emit(True, "OK", int(resp.get("course_id", -1)))
        else:
            self.createCourseResult.emit(False,
prettify_message(resp.get("message", "ERROR")), -1)
    except Exception as e:
        self.createCourseResult.emit(False, f"NETWORK_ERROR: {e}", -1)

```

```

@ Slot(int, int)

def delete_course_member(self, course_id, student_id):

    threading.Thread(target=self._worker_delete_course_member,
args=(course_id, student_id), daemon=True).start()


def _worker_delete_course_member(self, course_id, student_id):

    try:

        resp = self._client.delete_course_member(course_id, student_id)

        if resp.get("status") == "OK:"

            self.deleteCourseMemberResult.emit(True,
prettify_message(resp.get("message", "OK")), int(resp.get("course_id",
course_id)), int(resp.get("student_id", student_id)))

        else:

            self.deleteCourseMemberResult.emit(False,
prettify_message(resp.get("message", "ERROR")), int(course_id), int(student_id))

    except Exception as e:

        self.deleteCourseMemberResult.emit(False, f"NETWORK_ERROR: {e}",
int(course_id), int(student_id))

```

```

@ Slot(int)

def delete_course(self, course_id):

    threading.Thread(target=self._worker_delete_course, args=(course_id,),
daemon=True).start()


def _worker_delete_course(self, course_id):

    try:

        resp = self._client.delete_course(course_id)

        if resp.get("status") == "OK:"

```

```

        self.deleteCourseResult.emit(True,
prettify_message(resp.get("message", "OK")), int(resp.get("course_id",
course_id)))

    else:

        self.deleteCourseResult.emit(False,
prettify_message(resp.get("message", "ERROR")), int(course_id))

    except Exception as e:

        self.deleteCourseResult.emit(False, f"NETWORK_ERROR: {e}",
int(course_id))

```

@ Slot(str, str)

```

def validate_local_file(self, file_path, target):

    try:

        path = (file_path or "").replace("file:///", "")

        path = os.path.normpath(path)

        if not path or not os.path.exists(path):

            self.localFileValidationResult.emit(False, target, file_path,
prettify_message("FILE_NOT_FOUND"), 0)

            return

        size_bytes = os.path.getsize(path)

        max_size = 25 * 1024 * 1024

        if size_bytes <= 0:

            self.localFileValidationResult.emit(False, target, file_path,
prettify_message("EMPTY_FILE"), size_bytes)

            return

        if size_bytes > max_size:

            self.localFileValidationResult.emit(False, target, file_path,
prettify_message("FILE_TOO_LARGE"), size_bytes)

            return

```

```

        self.localFileValidationResult.emit(True, target, file_path,
prettify_message("OK"), size_bytes)

```

```

    except Exception as e:

```

```

        self.localFileValidationResult.emit(False, target, file_path,
prettify_message(f"VALIDATION_ERROR: {e}"), 0)

```

```

@ Slot(int)

```

```

    def get_course_assignments(self, course_id):

```

```

        threading.Thread(target=self._worker_get_course_assignments,
args=(course_id,), daemon=True).start()

```

```

    def _worker_get_course_assignments(self, course_id):

```

```

        try:

```

```

            resp = self._client.get_course_assignments(course_id)

```

```

            if resp.get("status") == "OK:"

```

```

                self.courseAssignmentsResult.emit(True, "OK",
int(resp.get("course_id", course_id)), resp.get("assignments", []))

```

```

            else:

```

```

                self.courseAssignmentsResult.emit(False,
prettify_message(resp.get("message", "ERROR")), int(course_id), [])

```

```

        except Exception as e:

```

```

            self.courseAssignmentsResult.emit(False, f"NETWORK_ERROR: {e}",
int(course_id), [])

```

```

@ Slot(int, str, str, str, bool, str)

```

```

    def create_assignment(self, course_id, title, description, due_date,
ai_enabled=True, attachment_path=""):

```

```

        threading.Thread(target=self._worker_create_assignment, args=(course_id,
title, description, due_date, ai_enabled, attachment_path),
daemon=True).start()

```

```

def _worker_create_assignment(self, course_id, title, description, due_date,
ai_enabled, attachment_path):

    try:

        if attachment_path:

            path = os.path.normpath((attachment_path or "").replace("file:///", ""))

            if os.path.exists(path) and os.path.getsize(path) > 25 * 1024 * 1024:

                self.createAssignmentResult.emit(False,
prettify_message("FILE_TOO_LARGE"), int(course_id), -1)

                return

            resp = self._client.create_assignment(course_id, title, description,
due_date, ai_enabled, attachment_path)

            if resp.get("status") == "OK:"

                self.createAssignmentResult.emit(True, "OK", int(resp.get("course_id",
course_id)), int(resp.get("assignment_id", -1)))

            else:

                self.createAssignmentResult.emit(False,
prettify_message(resp.get("message", "ERROR")), int(course_id), -1)

        except Exception as e:

            self.createAssignmentResult.emit(False, f"NETWORK_ERROR: {e}",
int(course_id), -1)

```

@ Slot(int, bool)

```

def set_assignment_closed(self, assignment_id, is_closed):

    threading.Thread(target=self._worker_set_assignment_closed,
args=(assignment_id, is_closed), daemon=True).start()

def _worker_set_assignment_closed(self, assignment_id, is_closed):

    try:

        resp = self._client.set_assignment_closed(assignment_id, is_closed)

        if resp.get("status") == "OK:"

```



```

        self.setAssignmentClosedResult.emit(True,
prettify_message(resp.get("message", "OK")), int(resp.get("assignment_id",
assignment_id)), bool(resp.get("is_closed", is_closed)))

    else:

        self.setAssignmentClosedResult.emit(False,
prettify_message(resp.get("message", "ERROR")), int(assignment_id),
bool(is_closed))

    except Exception as e:

        self.setAssignmentClosedResult.emit(False, f"NETWORK_ERROR: {e}",
int(assignment_id), bool(is_closed))

```

@ Slot(int)

```

def delete_assignment(self, assignment_id):

    threading.Thread(target=self._worker_delete_assignment,
args=(assignment_id,), daemon=True).start()

def _worker_delete_assignment(self, assignment_id):

    try:

        resp = self._client.delete_assignment(assignment_id)

        if resp.get("status") == "OK:"

            self.deleteAssignmentResult.emit(True,
prettify_message(resp.get("message", "OK")), int(resp.get("assignment_id",
assignment_id)), int(resp.get("course_id", -1)))

        else:

            self.deleteAssignmentResult.emit(False,
prettify_message(resp.get("message", "ERROR")), int(assignment_id), -1)

    except Exception as e:

        self.deleteAssignmentResult.emit(False, f"NETWORK_ERROR: {e}",
int(assignment_id), -1)

```

@ Slot(int)

```

def get_course_students(self, course_id):

    threading.Thread(target=self._worker_get_course_students,
args=(course_id,), daemon=True).start()

def _worker_get_course_students(self, course_id):

    try:

        resp = self._client.get_course_students(course_id)

        if resp.get("status") == "OK:"

            self.courseStudentsResult.emit(True, "OK", int(resp.get("course_id",
course_id)), resp.get("students", []))

        else:

            self.courseStudentsResult.emit(False,
prettify_message(resp.get("message", "ERROR")), int(course_id), [])

    except Exception as e:

        self.courseStudentsResult.emit(False, f"NETWORK_ERROR: {e}",
int(course_id), [])

```

@ Slot(int)

```

def get_submissions_for_assignment(self, assignment_id):

    threading.Thread(target=self._worker_get_submissions_for_assignment,
args=(assignment_id,), daemon=True).start()

def _worker_get_submissions_for_assignment(self, assignment_id):

    try:

        resp = self._client.get_submissions_for_assignment(assignment_id)

        if resp.get("status") == "OK:"

            self.submissionsResult.emit(True, "OK", int(resp.get("assignment_id",
assignment_id)), resp.get("submissions", []))

        else:

```

```

        self.submissionsResult.emit(False,
prettify_message(resp.get("message", "ERROR")), int(assignment_id), [])

    except Exception as e:

        self.submissionsResult.emit(False, f"NETWORK_ERROR: {e}",
int(assignment_id), [])

```

```

@ Slot(int, str, str, int)

```

```

    def update_submission_review(self, submission_id, final_score,
teacher_feedback, updated_by):

```

```

        threading.Thread(target=self._worker_update_submission_review,
args=(submission_id, final_score, teacher_feedback, updated_by),
daemon=True).start()

```

```

    def _worker_update_submission_review(self, submission_id, final_score,
teacher_feedback, updated_by):

```

```

        try:

            resp = self._client.update_submission_review(submission_id,
final_score, teacher_feedback, updated_by)

            if resp.get("status") == "OK:"

                self.updateSubmissionReviewResult.emit(True,
prettify_message(resp.get("message", "OK")), int(resp.get("submission_id",
submission_id)))

```

```

        else:

            self.updateSubmissionReviewResult.emit(False,
prettify_message(resp.get("message", "ERROR")), int(submission_id))

```

```

    except Exception as e:

        self.updateSubmissionReviewResult.emit(False, f"NETWORK_ERROR:
{e}", int(submission_id))

```

```

@ Slot(int)

```

```

    def get_course_materials(self, course_id):

```

```

        threading.Thread(target=self._worker_get_course_materials,
args=(course_id,), daemon=True).start()

```

```

def _worker_get_course_materials(self, course_id):
    try:
        resp = self._client.get_course_materials(course_id)
        if resp.get("status") == "OK:":
            self.courseMaterialsResult.emit(True, "OK", int(resp.get("course_id",
course_id)), resp.get("materials", []))
        else:
            self.courseMaterialsResult.emit(False,
prettify_message(resp.get("message", "ERROR")), int(course_id), [])
    except Exception as e:
        self.courseMaterialsResult.emit(False, f"NETWORK_ERROR: {e}",
int(course_id), [])

```

```

@ Slot(int, str, str, str, int, str)

```

```

def create_material(self, course_id, title, description, material_type,
created_by, file_path=""):
    threading.Thread(target=self._worker_create_material, args=(course_id,
title, description, material_type, created_by, file_path), daemon=True).start()

```

```

def _worker_create_material(self, course_id, title, description, material_type,
created_by, file_path):
    try:
        if file_path:
            path = os.path.normpath((file_path or "").replace("file:///", ""))
            if os.path.exists(path) and os.path.getsize(path) > 25 * 1024 * 1024:
                self.createMaterialResult.emit(False,
prettify_message("FILE_TOO_LARGE"), int(course_id), -1)
        return

```

```

        resp = self._client.create_material(course_id, title, description,
material_type, created_by, file_path)

        if resp.get("status") == "OK:"

            self.createMaterialResult.emit(True, "OK", int(resp.get("course_id",
course_id)), int(resp.get("material_id", -1)))

        else:

            self.createMaterialResult.emit(False,
prettify_message(resp.get("message", "ERROR")), int(course_id), -1)

        except Exception as e:

            self.createMaterialResult.emit(False, f"NETWORK_ERROR: {e}",
int(course_id), -1)

@ Slot(int)

def delete_material(self, material_id):

    threading.Thread(target=self._worker_delete_material, args=(material_id,),
daemon=True).start()

def _worker_delete_material(self, material_id):

    try:

        resp = self._client.delete_material(material_id)

        if resp.get("status") == "OK:"

            self.deleteMaterialResult.emit(True,
prettify_message(resp.get("message", "OK")), int(resp.get("material_id",
material_id)), int(resp.get("course_id", -1)))

        else:

            self.deleteMaterialResult.emit(False,
prettify_message(resp.get("message", "ERROR")), int(material_id), -1)

        except Exception as e:

            self.deleteMaterialResult.emit(False, f"NETWORK_ERROR: {e}",
int(material_id), -1)

```

```

@ Slot(int, int, str, str, str)

def save_course_message(self, course_id, sender_id, subject, body, state):

    threading.Thread(target=self._worker_save_course_message,
args=(course_id, sender_id, subject, body, state), daemon=True).start()

def _worker_save_course_message(self, course_id, sender_id, subject, body,
state):

    try:

        resp = self._client.save_course_message(course_id, sender_id, subject,
body, state)

        if resp.get("status") == "OK:"

            self.saveCourseMessageResult.emit(True, "OK",
int(resp.get("course_id", course_id)), int(resp.get("message_id", -1)))

        else:

            self.saveCourseMessageResult.emit(False,
prettyfy_message(resp.get("message", "ERROR")), int(course_id), -1)

    except Exception as e:

        self.saveCourseMessageResult.emit(False, f"NETWORK_ERROR: {e}",
int(course_id), -1)

```

```

@ Slot(int)

def get_course_messages(self, course_id):

    threading.Thread(target=self._worker_get_course_messages,
args=(course_id,), daemon=True).start()

def _worker_get_course_messages(self, course_id):

    try:

        resp = self._client.get_course_messages(course_id)

        if resp.get("status") == "OK:"

            self.courseMessagesResult.emit(True, "OK", int(resp.get("course_id",
course_id)), resp.get("messages", []))

```

```

        else:

            self.courseMessagesResult.emit(False,
            prettify_message(resp.get("message", "ERROR")), int(course_id), [])

        except Exception as e:

            self.courseMessagesResult.emit(False, f"NETWORK_ERROR: {e}",
            int(course_id), [])

    @ Slot(int)

    def delete_message(self, message_id):

        threading.Thread(target=self._worker_delete_message, args=(message_id,),
        daemon=True).start()

    def _worker_delete_message(self, message_id):

        try:

            resp = self._client.delete_message(message_id)

            if resp.get("status") == "OK:"

                self.deleteMessageResult.emit(True,
            prettify_message(resp.get("message", "OK")), int(resp.get("message_id",
            message_id)), int(resp.get("course_id", -1)))

            else:

                self.deleteMessageResult.emit(False,
            prettify_message(resp.get("message", "ERROR")), int(message_id), -1)

            except Exception as e:

                self.deleteMessageResult.emit(False, f"NETWORK_ERROR: {e}",
            int(message_id), -1)

    @ Slot(int, int, str)

    def upload_submission(self, assignment_id, student_id, file_path):

        threading.Thread(target=self._worker_upload_submission,
        args=(assignment_id, student_id, file_path), daemon=True).start()

```

```

@ Slot(int, int, str)

def upload_submission_text(self, assignment_id, student_id,
submission_text):

    threading.Thread(target=self._worker_upload_submission_text,
args=(assignment_id, student_id, submission_text), daemon=True).start()


def _worker_upload_submission(self, assignment_id, student_id, file_path):

    try:

        if file_path:

            path = os.path.normpath((file_path or "").replace("file:///", ""))

            if os.path.exists(path) and os.path.getsize(path) > 25 * 1024 * 1024:

                self.uploadSubmissionResult.emit(False,
prettify_message("FILE_TOO_LARGE"), int(assignment_id), -1)

                return

            resp = self._client.upload_submission(assignment_id, student_id,
file_path)

            if resp.get("status") == "OK:"

                self.uploadSubmissionResult.emit(True, "OK",
int(resp.get("assignment_id", assignment_id)), int(resp.get("submission_id", -1)))

            else:

                self.uploadSubmissionResult.emit(False,
prettify_message(resp.get("message", "ERROR")), int(assignment_id), -1)

        except Exception as e:

            self.uploadSubmissionResult.emit(False, f"NETWORK_ERROR: {e}",
int(assignment_id), -1)


def _worker_upload_submission_text(self, assignment_id, student_id,
submission_text):

    try:

        resp = self._client.upload_submission_text(assignment_id, student_id,
submission_text)

```



```

        if resp.get("status") == "OK:"

            self.uploadSubmissionResult.emit(True, "OK",
int(resp.get("assignment_id", assignment_id)), int(resp.get("submission_id", -1)))

        else:

            self.uploadSubmissionResult.emit(False,
prettify_message(resp.get("message", "ERROR")), int(assignment_id), -1)

        except Exception as e:

            self.uploadSubmissionResult.emit(False, f"NETWORK_ERROR: {e}",
int(assignment_id), -1)

```

@ Slot(int, int, str)

```

def update_course_member_status(self, course_id, student_id,
member_status):

    threading.Thread(target=self._worker_update_course_member_status,
args=(course_id, student_id, member_status), daemon=True).start()

def _worker_update_course_member_status(self, course_id, student_id,
member_status):

    try:

        resp = self._client.update_course_member_status(course_id,
student_id, member_status)

        if resp.get("status") == "OK:"

            self.updateCourseMemberStatusResult.emit(True,
prettify_message(resp.get("message", "OK")), int(resp.get("course_id",
course_id)), int(resp.get("student_id", student_id)), resp.get("member_status",
member_status))

        else:

            self.updateCourseMemberStatusResult.emit(False,
prettify_message(resp.get("message", "ERROR")), int(course_id), int(student_id),
member_status)

        except Exception as e:

```

```

        self.updateCourseMemberStatusResult.emit(False,
f"NETWORK_ERROR: {e}", int(course_id), int(student_id), member_status)

--- #   Added: student-home signals/slots---

@   Slot(int)

    def get_student_dashboard(self, student_id):

        threading.Thread(target=self._worker_get_student_dashboard,
args=(student_id,), daemon=True).start()

    def _worker_get_student_dashboard(self, student_id):

        try:

            resp = self._client.get_student_dashboard(student_id)

            if resp.get("status") == "OK:"

                self.studentDashboardResult.emit(True, "OK",
int(resp.get("student_id", student_id)), resp.get("classes", []),
resp.get("assignments", []), resp.get("materials", []), resp.get("messages", []))

            else:

                self.studentDashboardResult.emit(False,
prettify_message(resp.get("message", "ERROR")), int(student_id), [], [], [], [])

        except Exception as e:

            self.studentDashboardResult.emit(False, f"NETWORK_ERROR: {e}",
int(student_id), [], [], [], [])

@   Slot(int, str)

    def join_course_by_code(self, student_id, class_code):

        threading.Thread(target=self._worker_join_course_by_code,
args=(student_id, class_code), daemon=True).start()

    def _worker_join_course_by_code(self, student_id, class_code):

        try:

```

```

        resp = self._client.join_course_by_code(student_id, class_code)

        if resp.get("status") == "OK:":

            self.joinCourseResult.emit(True,
prettify_message(resp.get("message", "OK")), int(student_id), class_code)

        else:

            self.joinCourseResult.emit(False,
prettify_message(resp.get("message", "ERROR")), int(student_id), class_code)

    except Exception as e:

        self.joinCourseResult.emit(False, f"NETWORK_ERROR: {e}",
int(student_id), class_code)

```

```

@ Slot(int, int)

```

```

    def leave_course(self, student_id, course_id):

        threading.Thread(target=self._worker_leave_course, args=(student_id,
course_id), daemon=True).start()

```

```

    def _worker_leave_course(self, student_id, course_id):

        try:

            resp = self._client.leave_course(student_id, course_id)

            if resp.get("status") == "OK:":

                self.leaveCourseResult.emit(True,
prettify_message(resp.get("message", "OK")), int(student_id), int(course_id))

            else:

                self.leaveCourseResult.emit(False,
prettify_message(resp.get("message", "ERROR")), int(student_id), int(course_id))

        except Exception as e:

            self.leaveCourseResult.emit(False, f"NETWORK_ERROR: {e}",
int(student_id), int(course_id))

```

```

@ Slot(int, int)

```

```

def delete_student_submission(self, assignment_id, student_id):

    threading.Thread(target=self._worker_delete_student_submission,
args=(assignment_id, student_id), daemon=True).start()

def _worker_delete_student_submission(self, assignment_id, student_id):

    try:

        resp = self._client.delete_student_submission(assignment_id,
student_id)

        if resp.get("status") == "OK:":

            self.deleteSubmissionResult.emit(True,
prettify_message(resp.get("message", "OK")), int(assignment_id), int(student_id))

        else:

            self.deleteSubmissionResult.emit(False,
prettify_message(resp.get("message", "ERROR")), int(assignment_id),
int(student_id))

    except Exception as e:

        self.deleteSubmissionResult.emit(False, f"NETWORK_ERROR: {e}",
int(assignment_id), int(student_id))

```

@ Slot(int, int)

```

def mark_message_read(self, student_id, message_id):

    threading.Thread(target=self._worker_mark_message_read,
args=(student_id, message_id), daemon=True).start()

def _worker_mark_message_read(self, student_id, message_id):

    try:

        resp = self._client.mark_message_read(student_id, message_id)

        if resp.get("status") == "OK:":

            self.markMessageReadResult.emit(True,
prettify_message(resp.get("message", "OK")), int(student_id), int(message_id))

        else:

```

```

        self.markMessageReadResult.emit(False,
prettify_message(resp.get("message", "ERROR")), int(student_id),
int(message_id))

    except Exception as e:

        self.markMessageReadResult.emit(False, f"NETWORK_ERROR: {e}",
int(student_id), int(message_id))

```

@ Slot(int)

```

def mark_all_messages_read(self, student_id):

    threading.Thread(target=self._worker_mark_all_messages_read,
args=(student_id,), daemon=True).start()

```

```

def _worker_mark_all_messages_read(self, student_id):

    try:

        resp = self._client.mark_all_messages_read(student_id)

        if resp.get("status") == "OK:"

            self.markAllMessagesReadResult.emit(True,
prettify_message(resp.get("message", "OK")), int(student_id))

        else:

            self.markAllMessagesReadResult.emit(False,
prettify_message(resp.get("message", "ERROR")), int(student_id))

    except Exception as e:

        self.markAllMessagesReadResult.emit(False, f"NETWORK_ERROR: {e}",
int(student_id))

```

@ Slot(int, int)

```

def mark_grade_read(self, student_id, submission_id):

    threading.Thread(target=self._worker_mark_grade_read, args=(student_id,
submission_id), daemon=True).start()

```

```

def _worker_mark_grade_read(self, student_id, submission_id):

```

```

try:

    resp = self._client.mark_grade_read(student_id, submission_id)

    if resp.get("status") == "OK:"

        self.markGradeReadResult.emit(True,
prettify_message(resp.get("message", "OK")), int(student_id), int(submission_id))

    else:

        self.markGradeReadResult.emit(False,
prettify_message(resp.get("message", "ERROR")), int(student_id),
int(submission_id))

except Exception as e:

    self.markGradeReadResult.emit(False, f"NETWORK_ERROR: {e}",
int(student_id), int(submission_id))

```

--- # Added: admin APIs---

```

@ Slot()

def get_admin_overview(self):

    threading.Thread(target=self._worker_get_admin_overview,
daemon=True).start()

def _worker_get_admin_overview(self):

    try:

        resp = self._client.send_request({"type":
"GET_ADMIN_OVERVIEW_REQUEST"})

        if resp.get("status") == "OK:"

            payload} =

"        schools": resp.get("schools", []),
"        users": resp.get("users", []),
"        courses": resp.get("courses", []),
"        members": resp.get("members", []),

```

```

"        assignments": resp.get("assignments", []),
"        submissions": resp.get("submissions", []),
"        ai_results": resp.get("ai_results", []),
"        grades": resp.get("grades", []),
"        feedback": resp.get("feedback", []),
"        materials": resp.get("materials", []),
"        messages": resp.get("messages", []),
"        message_reads": resp.get("message_reads", []),
"        grade_reads": resp.get("grade_reads", []),
"        activity": resp.get("activity", []),
"        system": resp.get("system", []),
"        stats": resp.get("stats", {})
{
    self.adminOverviewResult.emit(True, "OK", payload)
else:
    self.adminOverviewResult.emit(False,
    prettify_message(resp.get("message", "ERROR")), {})
except Exception as e:
    self.adminOverviewResult.emit(False, f"NETWORK_ERROR: {e}", {})

@ Slot(str, str, str, str)
def admin_create_school(self, name, address, contact_name,
contact_email):
    threading.Thread(target=self._worker_admin_create_school, args=(name,
address, contact_name, contact_email), daemon=True).start()

def _worker_admin_create_school(self, name, address, contact_name,
contact_email):

```

```

        self._admin_action("create_school", {"type":
"ADMIN_CREATE_SCHOOL_REQUEST", "name": name, "address": address,
"contact_name": contact_name, "contact_email": contact_email})

```

```

@ Slot(int, str, str, str, str)

```

```

    def admin_update_school(self, school_id, name, address, contact_name,
contact_email):

```

```

        threading.Thread(target=self._worker_admin_update_school,
args=(school_id, name, address, contact_name, contact_email),
daemon=True).start()

```

```

    def _worker_admin_update_school(self, school_id, name, address,
contact_name, contact_email):

```

```

        self._admin_action("update_school", {"type":
"ADMIN_UPDATE_SCHOOL_REQUEST", "school_id": int(school_id), "name":
name, "address": address, "contact_name": contact_name, "contact_email":
contact_email})

```

```

@ Slot(int)

```

```

    def admin_delete_school(self, school_id):

```

```

        threading.Thread(target=self._worker_admin_delete_school,
args=(school_id,), daemon=True).start()

```

```

    def _worker_admin_delete_school(self, school_id):

```

```

        self._admin_action("delete_school", {"type":
"ADMIN_DELETE_SCHOOL_REQUEST", "school_id": int(school_id)})

```

```

@ Slot(str, str, str, str, int, str)

```

```

    def admin_create_user(self, name, email, password, role, school_id, status):

```

```

        threading.Thread(target=self._worker_admin_create_user, args=(name,
email, password, role, school_id, status), daemon=True).start()

```



```
def _worker_admin_create_user(self, name, email, password, role, school_id,
status):
```

```
    password_hash = hashlib.sha256(password.encode("utf-8")).hexdigest()
```

```
    self._admin_action("create_user", {"type":
"ADMIN_CREATE_USER_REQUEST", "name": name, "email": email.strip().lower(),
"password": password_hash, "role": role, "school_id": int(school_id), "status":
status})
```

```
@ Slot(int, str, str, str, int, str)
```

```
def admin_update_user(self, user_id, name, email, role, school_id, status):
```

```
    threading.Thread(target=self._worker_admin_update_user, args=(user_id,
name, email, role, school_id, status), daemon=True).start()
```

```
def _worker_admin_update_user(self, user_id, name, email, role, school_id,
status):
```

```
    self._admin_action("update_user", {"type":
"ADMIN_UPDATE_USER_REQUEST", "user_id": int(user_id), "name": name,
"email": email.strip().lower(), "role": role, "school_id": int(school_id), "status":
status})
```

```
@ Slot(int)
```

```
def admin_delete_user(self, user_id):
```

```
    threading.Thread(target=self._worker_admin_delete_user, args=(user_id,),
daemon=True).start()
```

```
def _worker_admin_delete_user(self, user_id):
```

```
    self._admin_action("delete_user", {"type":
"ADMIN_DELETE_USER_REQUEST", "user_id": int(user_id)})
```

```
@ Slot(int, str, str, int, int)
```

```
def admin_update_course(self, course_id, name, description, teacher_id,
school_id):
```

```

        threading.Thread(target=self._worker_admin_update_course,
args=(course_id, name, description, teacher_id, school_id),
daemon=True).start()

```

```

def _worker_admin_update_course(self, course_id, name, description,
teacher_id, school_id):

```

```

        self._admin_action("update_course", {"type":
"ADMIN_UPDATE_COURSE_REQUEST", "course_id": int(course_id), "name":
name, "description": description, "teacher_id": int(teacher_id), "school_id":
int(school_id)})

```

```

@ Slot(int, str, str, str, bool, bool)

```

```

def admin_update_assignment(self, assignment_id, title, description,
due_date, ai_enabled, is_closed):

```

```

        threading.Thread(target=self._worker_admin_update_assignment,
args=(assignment_id, title, description, due_date, ai_enabled, is_closed),
daemon=True).start()

```

```

def _worker_admin_update_assignment(self, assignment_id, title,
description, due_date, ai_enabled, is_closed):

```

```

        self._admin_action("update_assignment", {"type":
"ADMIN_UPDATE_ASSIGNMENT_REQUEST", "assignment_id":
int(assignment_id), "title": title, "description": description, "due_date": due_date,
"ai_enabled": bool(ai_enabled), "is_closed": bool(is_closed)})

```

```

def _admin_action(self, action_name, req):

```

```

    try:

        resp = self._client.send_request(req)

        if resp.get("status") == "OK:"

            self.adminActionResult.emit(True,
prettyfy_message(resp.get("message", "OK")), action_name)

    else:

```

```

        self.adminActionResult.emit(False,
prettify_message(resp.get("message", "ERROR")), action_name)

    except Exception as e:

        self.adminActionResult.emit(False, f"NETWORK_ERROR: {e}",
action_name)

@ Slot(str)

def download_file(self, file_path):

    threading.Thread(target=self._worker_download_file, args=(file_path,),
daemon=True).start()

def _worker_download_file(self, file_path):

    try:

        resp, saved_path = self._client.download_file(file_path)

        if resp.get("status") == "OK:"

            self.downloadFileResult.emit(True,
prettify_message("DOWNLOAD_OK"), file_path, saved_path)

        else:

            self.downloadFileResult.emit(False,
prettify_message(resp.get("message", "ERROR")), file_path, "")

    except Exception as e:

        self.downloadFileResult.emit(False, f"NETWORK_ERROR: {e}", file_path,
"")

@ Slot(str, str, bool)

def login(self, email, password, keepsign):

    threading.Thread(target=self._worker_login, args=(email, password,
keepsign), daemon=True).start()

def _worker_login(self, email, password, keepsign):

```

```

try:
    resp = self._client.login(email, password)
    if resp.get("status") == "OK:"
        role = resp.get("role", "")
        user_id = int(resp.get("user_id", -1))
        name = resp.get("name", "unknown")
        school_id = int(resp.get("school_id", -1))
        school_name = resp.get("school_name", "")
        school_address = resp.get("school_address", "")
        school_contact_email = resp.get("school_contact_email", "")
        self.loginResult.emit(True, "OK", role, user_id, name, school_id,
school_name, school_address, school_contact_email)
    else:
        self.loginResult.emit(False, prettify_message(resp.get("message",
"ERROR")), "", -1, "", -1, "", "", "")
except Exception as e:
    self.loginResult.emit(False, f"NETWORK_ERROR: {e}", "", -1, "", -1, "", "", "")

```

@ Slot(str, str, str, str, str)

```

def signup(self, name, email, password, role, school_id):
    threading.Thread(target=self._worker_signup, args=(name, email,
password, role, school_id), daemon=True).start()

def _worker_signup(self, name, email, password, role, school_id):
    try:
        resp = self._client.signup(name, email, password, role, school_id)
        if resp.get("status") == "OK:"
            self.signupResult.emit(True, prettify_message(resp.get("message",
"OK")))

```

```

        else:

            self.signupResult.emit(False, prettify_message(resp.get("message",
"ERROR"))))

        except Exception as e:

            self.signupResult.emit(False, f"NETWORK_ERROR: {e}")

    @ Slot(int)

    def approve(self, user_id):

        threading.Thread(target=self._worker_approve, args=(user_id,),
daemon=True).start()

    def _worker_approve(self, user_id):

        try:

            resp = self._client.approve_user(user_id)

            if resp.get("status") == "OK:"

                self.approveResult.emit(True, prettify_message(resp.get("message",
"OK"))))

            else:

                self.approveResult.emit(False, prettify_message(resp.get("message",
"ERROR"))))

        except Exception as e:

            self.approveResult.emit(False, f"NETWORK_ERROR: {e}")

def main:()

    os.environ["QT_QUICK_CONTROLS_STYLE"] = "Basic"

    app = QtGuiApplication(sys.argv)

    app.setWindowIcon(QIcon(":/data/screens/png/AppIcon.png"))

```

```

engine = QQmlApplicationEngine()

auth = Auth()
engine.rootContext().setContextProperty("auth", auth)
if getattr(sys, "frozen", False):
    engine.load(QUrl("qrc:/main.qml"))
else:
    engine.load(QUrl.fromLocalFile(str(APP_DIR / "main.qml")))

if not engine.rootObjects:()
    print("Failed to load QML (no root objects)")
    return 1

return app.exec()

```

```

if __name__ == "__main: "__
    main()

```

```

AI_Grader.py
import sqlite3
import random
import string
import re
from datetime import datetime
from pathlib import Path

```

```

--- #Added: course/class code helpers---

def _generate_course_code(length: int = 6) -> str:
    alphabet = string.ascii_uppercase + string.digits
    return ''.join(random.choices(alphabet, k=length))

def _generate_unique_course_code(conn) -> str:
    cur = conn.cursor()
    while True:
        code = _generate_course_code(6)
        cur.execute("SELECT 1 FROM courses WHERE class_code = ? LIMIT 1",
        (code,))
        if cur.fetchone() is None:
            return code

def _ensure_course_codes(conn):
    cur = conn.cursor()
    cur.execute("PRAGMA table_info(courses)")
    course_cols = [row[1] for row in cur.fetchall()]
    if "class_code" not in course_cols:
        conn.execute("ALTER TABLE courses ADD COLUMN class_code TEXT")

    conn.execute("CREATE UNIQUE INDEX IF NOT EXISTS
idx_courses_class_code_unique ON courses(class_code)")

    cur.execute("SELECT course_id FROM courses WHERE class_code IS NULL
OR TRIM(class_code) = '')")
    missing_rows = cur.fetchall()

```

```

for (course_id,) in missing_rows:

    code = _generate_unique_course_code(conn)

    conn.execute("UPDATE courses SET class_code = ? WHERE course_id = ?",
(code, course_id))

```

```

def _conn:()

```

```

    conn = sqlite3.connect("server_data/classify.db")

```

```

    conn.execute("""

```

```

        CREATE TABLE IF NOT EXISTS schools)

```

```

            school_id  INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

            name      TEXT NOT NULL,

```

```

            address   TEXT,

```

```

            contact_name TEXT,

```

```

            contact_email TEXT,

```

```

            created_at TEXT NOT NULL

```

```

(

```

```

(

```

```

    conn.execute("""

```

```

        CREATE TABLE IF NOT EXISTS users)

```

```

            id        INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

            name      TEXT NOT NULL,

```

```

            email     TEXT UNIQUE NOT NULL,

```

```

            password_hash TEXT NOT NULL,

```

```

            role      TEXT NOT NULL,

```

```

            schoolID  INTEGER NOT NULL,

```

```

            status    TEXT NOT NULL,

```



```

        created_at TEXT NOT NULL
    (
    ("""

conn.execute("""
    CREATE TABLE IF NOT EXISTS courses)
        course_id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        description TEXT,
        teacher_id INTEGER NOT NULL,
        schoolID INTEGER NOT NULL,
        created_at TEXT NOT NULL
    (
    ("""

```

```

conn.execute("""
    CREATE TABLE IF NOT EXISTS course_members)
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        course_id INTEGER NOT NULL,
        student_id INTEGER NOT NULL,
        joined_at TEXT NOT NULL,
        member_status TEXT NOT NULL DEFAULT 'ACTIVE,'
        UNIQUE(course_id, student_id)
    (
    ("""

```

```

conn.execute("""
    CREATE TABLE IF NOT EXISTS assignments)

```

```

        assignment_id INTEGER PRIMARY KEY AUTOINCREMENT,
        course_id    INTEGER NOT NULL,
        title        TEXT NOT NULL,
        description   TEXT,
        due_date     TEXT,
        attachment_path TEXT,
        ai_enabled    INTEGER NOT NULL DEFAULT 1,
        is_closed    INTEGER NOT NULL DEFAULT 0,
        created_at   TEXT NOT NULL
    (
    ("""

```

```

conn.execute("""

```

```

CREATE TABLE IF NOT EXISTS submissions)

```

```

        submission_id INTEGER PRIMARY KEY AUTOINCREMENT,
        assignment_id INTEGER NOT NULL,
        student_id    INTEGER NOT NULL,
        file_path     TEXT NOT NULL,
        submitted_at  TEXT NOT NULL,
        status        TEXT NOT NULL DEFAULT 'PENDING_AI'
    (
    ("""

```

```

conn.execute("""

```

```

CREATE TABLE IF NOT EXISTS ai_results)

```

```

        ai_result_id INTEGER PRIMARY KEY AUTOINCREMENT,
        submission_id INTEGER NOT NULL,
        engine_name   TEXT NOT NULL,

```

```

        score    REAL NOT NULL,
        feedback TEXT
    (
    ("""

conn.execute("""

CREATE TABLE IF NOT EXISTS grades)

    grade_id    INTEGER PRIMARY KEY AUTOINCREMENT,
    submission_id INTEGER NOT NULL UNIQUE,
    ai_score    REAL,
    final_score REAL,
    updated_by  INTEGER,
    updated_at  TEXT
    (
    ("""

conn.execute("""

CREATE TABLE IF NOT EXISTS feedback)

    feedback_id  INTEGER PRIMARY KEY AUTOINCREMENT,
    submission_id INTEGER NOT NULL UNIQUE,
    ai_feedback  TEXT,
    teacher_feedback TEXT
    (
    ("""

conn.execute("""

CREATE TABLE IF NOT EXISTS materials)

    material_id  INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

        course_id INTEGER NOT NULL,
        title TEXT NOT NULL,
        description TEXT,
        type TEXT,
        created_by INTEGER,
        file_path TEXT,
        created_at TEXT NOT NULL
    (
    ("""

conn.execute("""
    CREATE TABLE IF NOT EXISTS messages)
        message_id INTEGER PRIMARY KEY AUTOINCREMENT,
        course_id INTEGER NOT NULL,
        sender_id INTEGER,
        subject TEXT NOT NULL,
        body TEXT NOT NULL,
        state TEXT NOT NULL,
        created_at TEXT NOT NULL
    (
    ("""

# Lightweight migrations for older databases
cur = conn.cursor()
_ ensure_course_codes(conn)
cur.execute("PRAGMA table_info(course_members)")
course_member_cols = [row[1] for row in cur.fetchall()]
if "member_status" not in course_member_cols:

```

```

conn.execute("ALTER TABLE course_members ADD COLUMN
member_status TEXT NOT NULL DEFAULT 'ACTIVE'")

cur.execute("PRAGMA table_info(assignments)")

assignment_cols = [row[1] for row in cur.fetchall()]

if "is_closed" not in assignment_cols:

    conn.execute("ALTER TABLE assignments ADD COLUMN is_closed
INTEGER NOT NULL DEFAULT 0")

if "ai_enabled" not in assignment_cols:

    conn.execute("ALTER TABLE assignments ADD COLUMN ai_enabled
INTEGER NOT NULL DEFAULT 1")

```

```

conn.execute("""

CREATE TABLE IF NOT EXISTS message_reads)

    id    INTEGER PRIMARY KEY AUTOINCREMENT,

    message_id INTEGER NOT NULL,

    student_id INTEGER NOT NULL,

    read_at TEXT NOT NULL,

    UNIQUE(message_id, student_id)

(
"""

```

```

conn.execute("""

CREATE TABLE IF NOT EXISTS grade_reads)

    id    INTEGER PRIMARY KEY AUTOINCREMENT,

    submission_id INTEGER NOT NULL,

    student_id INTEGER NOT NULL,

    read_at TEXT NOT NULL,

    UNIQUE(submission_id, student_id)

(

```

```

("""

conn.commit()

return conn

def _normalize_due_date_text(value):
    text = (value or "").strip()

    if not text:
        return ""

    text = text.replace("T", " ")
    text = re.sub(r"\s+", " ", text).strip()

    m = re.match(r"^\d{4}-\d{2}-\d{2}$", text)
    if m:
        return f"{m.group(1)}-{m.group(2)}-{m.group(3)} 00:00"

    m = re.match(r"^\d{4}-\d{2}-\d{2} (\d{2}):(\d{2})(?::\d{2})?$", text)
    if m:
        return f"{m.group(1)}-{m.group(2)}-{m.group(3)} {m.group(4)}:{m.group(5)}"

    return text

def _parse_due_datetime(value):
    normalized = _normalize_due_date_text(value)

    if not normalized:

```

```

return None

for fmt in ("%Y-%m-%d %H:%M", "%Y-%m-%d %H:%M:%S", "%Y-%m-%d"):
    try:
        return datetime.strptime(normalized, fmt)
    except ValueError:
        continue
return None

```

```

def close_expired_assignments(course_id=None, assignment_id=None,
conn=None):

    own_conn = conn is None
    conn = conn or _conn()

    try:
        cur = conn.cursor()

        sql = "SELECT assignment_id, due_date, is_closed FROM assignments
WHERE 1=1"

        params[] =

        if course_id is not None:
            sql += " AND course_id"=?=
            params.append(int(course_id))

        if assignment_id is not None:
            sql += " AND assignment_id"=?=
            params.append(int(assignment_id))

        cur.execute(sql, params)

        rows = cur.fetchall()

        now = datetime.now()

```

```

changed = False

for row in rows:
    current_assignment_id = int(row[0])
    due_dt = _parse_due_datetime(row[1])
    is_closed = int(row[2] or 0)

    if due_dt is not None and due_dt <= now and is_closed == 0:
        cur.execute("UPDATE assignments SET is_closed=1 WHERE
assignment_id=?", (current_assignment_id,))
        changed = True

if changed:
    conn.commit()

return True

finally:
    if own_conn:
        conn.close()

def _assignment_row_to_dict(row):
    return {
        "assignment_id": int(row[0]),
        "course_id": int(row[1]),
        "title": row[2],
        "description": row[3],
        "due_date": _normalize_due_date_text(row[4]),
        "attachment_path": row[5]
    }

```



```
"    ai_enabled": row,[6]
"    is_closed": row,[7]
"    created_at": row[8]
{
```

```
#
```

```
#SCHOOLS
```

```
#
```

```
def create_school(name: str, address=None, contact_name=None,
contact_email=None):
    (שגיאה) False או (True, school_id) יצירת בית ספר חדש. מחזיר
    conn = _conn()
    try:
        created_at = datetime.now().isoformat(timespec="seconds")
        cur = conn.cursor()
        cur.execute)
"        INSERT INTO schools (name, address, contact_name, contact_email,
created_at) VALUES,(? ,? ,? ,? ,?)
)        name, address, contact_name, contact_email, created_at(
(
        conn.commit()
        return True, cur.lastrowid
    except sqlite3.Error as e:
        return False, str(e)
    finally:
        conn.close()
```

```

def update_school(school_id: int, name=None, address=None,
contact_name=None, contact_email=None):
    מתעדכנים. None"""
    conn = _conn()
    try:
        cur = conn.cursor()
        fields, values[] ,[] =
        if name is not None:
            fields.append("name = ?");    values.append(name)
        if address is not None:
            fields.append("address = ?");    values.append(address)
        if contact_name is not None:
            fields.append("contact_name = ?"); values.append(contact_name)
        if contact_email is not None:
            fields.append("contact_email = ?"); values.append(contact_email)
        if not fields:
            return False, "NO_FIELDS_TO_UPDATE"
        values.append(int(school_id))
        cur.execute(f"UPDATE schools SET {' '.join(fields)} WHERE school_id = ?",
values)
        conn.commit()
        if cur.rowcount == 0:
            return False, "SCHOOL_NOT_FOUND"
        return True, "SCHOOL_UPDATED"
    finally:
        conn.close()

```

```

def get_school(school_id: int):
    , שגיאה). (False) או (True, dict). מחזיר ID) ""שליפת פרטי בית ספר לפי
    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute(
            "SELECT school_id, name, address, contact_name, contact_email FROM
            schools WHERE school_id, "? =
        )
        school_id(,
        (
            row = cur.fetchone()

            if not row:

                return False, "SCHOOL_NOT_FOUND"

            return True, {"school_id": row[0], "name": row[1], "address": row[2],[
            "
            contact_name": row[3], "contact_email": row{[4]

        finally:

            conn.close()

```

```

def school_exists(school_id: int):
    ""בדיקה מהירה האם מזהה בית ספר קיים.""
    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute("SELECT 1 FROM schools WHERE school_id = ? LIMIT 1",
        (int(school_id),))

        return cur.fetchone() is not None

    finally:

        conn.close()

```

```

def get_all_schools:()
) (True, list)"""שליפת כל בתי הספר (לשימוש אדמין). מחזיר (
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("SELECT school_id, name, address, contact_name,
contact_email FROM schools")
        rows = cur.fetchall()
        schools = [{"school_id": r[0], "name": r[1], "address": r[2],
"
            contact_name": r[3], "contact_email": r [4]for r in rows[
        return True, schools
    finally:
        conn.close()

```

```

#
#USERS
#

```

```

def signup_user(name, email: str, password_hash: str, role: str, school_id):
"""רישום משתמש חדש. מנהל יוצר בית ספר אוטומטית."""
    conn = _conn()
    email = email.strip().lower()
    created_at = datetime.now().isoformat(timespec="seconds")
    stat = "ACTIVE" if role == "MANAGER" else "PENDING"

    school_id_int = None

```

```

if school_id is not None:
    s = str(school_id).strip()
    if s:
        try:
            school_id_int = int(s)
        except ValueError:
            return False, "INVALID_SCHOOL_ID"

    try:
        if role == "MANAGER:"
            cur = conn.cursor()
            cur.execute(
                "INSERT INTO schools (name, address, contact_name, contact_email,
                created_at) VALUES, "(? ,? ,? ,? ,? ,?)
            ")
            New School", None, name, email, created_at(
            (
                school_id_int = cur.lastrowid
                cur.execute("UPDATE schools SET name=? WHERE school_id,""?=
                    (f"School #{school_id_int}", school_id_int)(

        if school_id_int is None:
            return False, "SCHOOL_ID_REQUIRED"

        if not school_exists(school_id_int):
            return False, "SCHOOL_NOT_FOUND"

        conn.execute(
            "INSERT INTO users (name, email, password_hash, role, schoolID,
            status, created_at) VALUES, "(? ,? ,? ,? ,? ,? ,? ,?)

```

```

)      name, email, password_hash, role, school_id_int, stat, created_at(
(
    conn.commit()
    return True, "USER_CREATED"
except sqlite3.IntegrityError:
    return False, "USER_ALREADY_EXISTS"
finally:
    conn.close()

```

```

def login_user(email: str, password_hash: str):

```

```

)      התחברות. מחזיר ( True, role, user_id, name, school_id, school_name,
school_address, school_contact_email".(

```

```

    conn = _conn()

```

```

    email = email.strip().lower()

```

```

    try:

```

```

        cur = conn.cursor()

```

```

        cur.execute)

```

```

"      SELECT id, role, status, name, schoolID FROM users WHERE email=?
AND password_hash, "?=

```

```

)      email, password_hash(

```

```

(

```

```

    row = cur.fetchone()

```

```

    if not row:

```

```

        return False, "INVALID_CREDENTIALS", -1, "", -1 "", "", "", ,

```

```

    user_id, role, status, name, school_id = row

```

```

    name = name.split[0](" ")

```

```

if status == "PENDING:"
    return False, "WAITING_APPROVAL", -1, "", -1 "", "", "",
if status == "BLOCKED:"
    return False, "USER_BLOCKED", -1, "", -1 "", "", "",

school_name = school_address = school_contact_email "" =

cur.execute("SELECT name, address, contact_email FROM schools WHERE
school_id = ?", (school_id,))

srow = cur.fetchone()

if srow:
    school_name      = srow[0] or ""
    school_address   = srow[1] or ""
    school_contact_email = srow[2] or ""

    return True, role, user_id, name, school_id, school_name, school_address,
    school_contact_email

finally:
    conn.close()

```

```

def get_user_context(user_id: int):
    וּשֵׁם שֶׁל מִשְׁתַּמֵּשׁ לִפִּי מִזְהָה. "" role, school_id, status "" מִחְזִיר

    conn = _conn()

    try:
        cur = conn.cursor()

        cur.execute("SELECT id, role, schoolID, status, name, email FROM users
WHERE id=?", (int(user_id),))

        row = cur.fetchone()

```

```

        if not row:
            return False, "USER_NOT_FOUND"
        return True} ,
"        user_id": int(row[0]),
"        role": row[1] or, ""
"        school_id": int(row[2]),
"        status": row[3] or, ""
"        name": row[4] or, ""
"        email": row[5] or, ""
{
    finally:
        conn.close()

def is_teacher_of_course(teacher_id: int, course_id: int):
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("SELECT 1 FROM courses WHERE course_id=? AND
teacher_id=?", (int(course_id), int(teacher_id)))
        return cur.fetchone() is not None
    finally:
        conn.close()

def is_course_in_school(course_id: int, school_id: int):
    conn = _conn()
    try:

```



```

        cur = conn.cursor()

        cur.execute("SELECT 1 FROM courses WHERE course_id=? AND
schoolID=?", (int(course_id), int(school_id)))

        return cur.fetchone() is not None

    finally:

        conn.close()

```

```

def get_assignment_course_id(assignment_id: int):

    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute("SELECT course_id FROM assignments WHERE
assignment_id=?", (int(assignment_id),))

        row = cur.fetchone()

        if not row:

            return False, "ASSIGNMENT_NOT_FOUND"

        return True, int(row[0])

    finally:

        conn.close()

```

```

def get_submission_course_id(submission_id: int):

    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute("""

            SELECT a.course_id

            FROM submissions s

```

```

        JOIN assignments a ON s.assignment_id = a.assignment_id
        WHERE s.submission_id=?
, """      (int(submission_id),)
    row = cur.fetchone()
    if not row:
        return False, "SUBMISSION_NOT_FOUND"
    return True, int(row[0])
finally:
    conn.close()

def get_message_course_id(message_id: int):
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("SELECT course_id FROM messages WHERE message_id=?",
(int(message_id),))
        row = cur.fetchone()
        if not row:
            return False, "MESSAGE_NOT_FOUND"
        return True, int(row[0])
    finally:
        conn.close()

def get_material_course_id(material_id: int):
    conn = _conn()
    try:

```

```

        cur = conn.cursor()

        cur.execute("SELECT course_id FROM materials WHERE material_id=?",
(int(material_id),))

        row = cur.fetchone()

        if not row:

            return False, "MATERIAL_NOT_FOUND"

        return True, int(row[0])

    finally:

        conn.close()

```

```

def can_user_access_course(user_id: int, role: str, school_id: int, course_id: int):

    role = (role or "").upper()

    if role == "ADMIN:"

        return True

    if role == "TEACHER:"

        return is_teacher_of_course(user_id, course_id)

    if role == "MANAGER:"

        return is_course_in_school(course_id, school_id)

    if role == "STUDENT:"

        return is_student_active_in_course(course_id, user_id)

    return False

```

```

def can_user_access_file(user_id: int, role: str, school_id: int, file_path: str):

    """בדיקת הרשאה כללית לקובץ לפי הנתוב בשורת. """

    path_text = str(file_path or "")

    course_id = None

```

```

match = re.search(r"course_(\d+)", path_text)

if match:
    course_id = int(match.group(1))

if course_id is None:
    return False

return can_user_access_course(user_id, role, school_id, course_id)


def get_school_users(school_id: int):
    ) כל המשתמשים של בית ספר. מחזיר ( True, list"".
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute(
            "SELECT id, name, email, role, status FROM users WHERE schoolID, "? =
            school_id(,
            (
                users = [{"id": r[0], "name": r[1], "email": r[2], "role": r[3], "status": r[4]{
                    for r in cur.fetchall()
                return True, users
    except sqlite3.Error as e:
        return False, str(e)
    finally:
        conn.close()

```

```

def approve_user(user_id: int):
) אישור משתמש ממתיין PENDING → ACTIVE"". (
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("UPDATE users SET status=? WHERE id=? AND status, " ?=
            ("ACTIVE", user_id, "PENDING")(
        conn.commit()
        if cur.rowcount == 0:
            return False, "USER_NOT_FOUND_OR_ALREADY_ACTIVE"
        return True, "USER_APPROVED"
    finally:
        conn.close()

```

```

def block_user(user_id: int):
) חסימת משתמש פעיל ACTIVE → BLOCKED"". (
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("UPDATE users SET status=? WHERE id=? AND status, " ?=
            ("BLOCKED", user_id, "ACTIVE")(
        conn.commit()
        if cur.rowcount == 0:
            return False, "USER_NOT_FOUND_OR_ALREADY_BLOCKED"
        return True, "USER_BLOCKED"
    finally:

```

```
conn.close()
```

```
def unblock_user(user_id: int):
```

```
) """ביטול חסימת משתמש""" BLOCKED → ACTIVE""".(
```

```
    conn = _conn()
```

```
    try:
```

```
        cur = conn.cursor()
```

```
        cur.execute("UPDATE users SET status=? WHERE id=? AND status, " ?=
```

```
            ("ACTIVE", user_id, "BLOCKED")
```

```
        conn.commit()
```

```
        if cur.rowcount == 0:
```

```
            return False, "USER_NOT_FOUND_OR_ALREADY_UNBLOCKED"
```

```
        return True, "USER_UNBLOCKED"
```

```
    finally:
```

```
        conn.close()
```

```
def delete_user(user_id: int):
```

```
"""מחיקת משתמש לחלוטין מהמערכת."""
```

```
    conn = _conn()
```

```
    try:
```

```
        cur = conn.cursor()
```

```
        cur.execute("DELETE FROM users WHERE id=?", (user_id,))
```

```
        conn.commit()
```

```
        if cur.rowcount == 0:
```

```
            return False, "USER_NOT_FOUND"
```

```
        return True, "USER_DELETED"
```

```
finally:
    conn.close()
```

```
def get_pending_users:()
) כל המשתמשים הממתינים לאישור ( id, email, role"").(
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("SELECT id, email, role FROM users WHERE status=?",
("PENDING",))
        return cur.fetchall()
    finally:
        conn.close()
```

```
#
#COURSES
#
```

```
def create_course(name: str, description: str, teacher_id: int, school_id: int):
) שגיאה""). (False"") או (True, course_id) "יצירת קורס חדש. מחזיר (
    conn = _conn()
    try:
        created_at = datetime.now().isoformat(timespec="seconds")
        class_code = _generate_unique_course_code(conn) # --- Added: real 6-
char join code---
        cur = conn.cursor()
        cur.execute)
```

```

"        INSERT INTO courses (name, description, teacher_id, schoolID,
created_at, class_code) VALUES, "(? ,? ,? ,? ,? ,?)"
)        name, description, teacher_id, school_id, created_at, class_code(
(
    conn.commit()

    return True, cur.lastrowid

except sqlite3.Error as e:

    return False, str(e)

finally:

    conn.close()


def get_courses_for_teacher(teacher_id: int):
    """כל הקורסים של מורה מסוים."""
    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute)

"        SELECT course_id, name, description, schoolID, created_at, class_code
FROM courses WHERE teacher_id, "=?
)        teacher_id(
(
    courses = [{"course_id": r[0], "name": r[1], "description": r[2],[
"        school_id": r[3], "created_at": r[4], "class_code": r[5] or ""} for r in
cur.fetchall]()

    return True, courses

finally:

    conn.close()

```



```

def get_courses_for_student(student_id: int):
    """כל הקורסים הפעילים שתלמיד שויך אליהם."""
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("""
            SELECT c.course_id, c.name, c.description, c.teacher_id, c.created_at,
            c.class_code
            FROM courses c
            JOIN course_members cm ON c.course_id = cm.course_id
            WHERE cm.student_id = ? AND COALESCE(cm.member_status,
            'ACTIVE') = 'ACTIVE'
            , """)
        (student_id,)(
            courses = [{"course_id": r[0], "name": r[1], "description": r[2], [
            "teacher_id": r[3], "created_at": r[4], "class_code": r[5] or ""} for r in
            cur.fetchall()]
            return True, courses
    finally:
        conn.close()

```

```

def get_courses_for_school(school_id: int):
    """כל הקורסים של בית ספר (לשימוש מנהל)."""
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("""
            SELECT c.course_id, c.name, c.description, c.teacher_id, c.created_at,

```

```

COUNT(CASE WHEN COALESCE(cm.member_status, 'ACTIVE') =
'ACTIVE' THEN cm.student_id END) as student_count,

u.name as teacher_name

FROM courses c

LEFT JOIN course_members cm ON c.course_id = cm.course_id

LEFT JOIN users u ON c.teacher_id = u.id

WHERE c.schoolID? =

GROUP BY c.course_id

, "" (school_id,)(
courses = [{"course_id": r[0], "name": r[1], "description": r[2],[
"
teacher_id": r[3], "created_at": r[4], "students": r,[5]
"
teacher_name": r[6] or "Unknown{"
for r in cur.fetchall()

return True, courses

finally:

conn.close()

```

```

def get_course_teacher(course_id: int):

"""מחזיר את מזהה המורה של קורס מסוים."""

conn = _conn()

try:

cur = conn.cursor()

cur.execute("SELECT teacher_id FROM courses WHERE course_id=?",
(course_id,))

row = cur.fetchone()

if not row:

```

```

        return False, "COURSE_NOT_FOUND"

    return True, row[0]

finally:

    conn.close()


def get_course_school(course_id: int):
    """מחזיר את מזהה בית הספר של קורס מסוים."""
    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute("SELECT schoolID FROM courses WHERE course_id=?",
(course_id,))

        row = cur.fetchone()

        if not row:

            return False, "COURSE_NOT_FOUND"

        return True, row[0]

    finally:

        conn.close()


def get_course_file_paths(course_id: int):
    """כל נתיבי הקבצים המשויכים לקורס."""
    conn = _conn()

    try:

        cur = conn.cursor()

        paths[] =

```

```

        cur.execute("SELECT attachment_path FROM assignments WHERE
course_id=? AND attachment_path IS NOT NULL AND attachment_path <> '',
(course_id,))

        paths.extend([row[0] for row in cur.fetchall() if row[0]])

        cur.execute("SELECT file_path FROM materials WHERE course_id=? AND
file_path IS NOT NULL AND file_path <> '', (course_id,))

        paths.extend([row[0] for row in cur.fetchall() if row[0]])

    cur.execute("""
        SELECT s.file_path
        FROM submissions s
        JOIN assignments a ON s.assignment_id = a.assignment_id
        WHERE a.course_id=? AND s.file_path IS NOT NULL AND s.file_path" <>
, """      (course_id,)(
        paths.extend([row[0] for row in cur.fetchall() if row[0]])

    return True, paths

finally:
    conn.close()

def delete_course(course_id: int):
    """מחיקת קורס וכל המידע המשויך אליו."""
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("SELECT course_id FROM courses WHERE course_id=?",
(course_id,))

```

```

if not cur.fetchone():

    return False, "COURSE_NOT_FOUND"


cur.execute("SELECT assignment_id FROM assignments WHERE
course_id=?", (course_id,))

assignment_ids = [row[0] for row in cur.fetchall()]

submission_ids[] =

if assignment_ids:

    placeholders = ",".join(["?"] * len(assignment_ids))

    cur.execute(f"SELECT submission_id FROM submissions WHERE
assignment_id IN ({placeholders})", assignment_ids)

    submission_ids = [row[0] for row in cur.fetchall()]


if submission_ids:

    placeholders = ",".join(["?"] * len(submission_ids))

    cur.execute(f"DELETE FROM ai_results WHERE submission_id IN
({placeholders})", submission_ids)

    cur.execute(f"DELETE FROM grades WHERE submission_id IN
({placeholders})", submission_ids)

    cur.execute(f"DELETE FROM feedback WHERE submission_id IN
({placeholders})", submission_ids)

    cur.execute(f"DELETE FROM grade_reads WHERE submission_id IN
({placeholders})", submission_ids)


if assignment_ids:

    placeholders = ",".join(["?"] * len(assignment_ids))

    cur.execute(f"DELETE FROM submissions WHERE assignment_id IN
({placeholders})", assignment_ids)

    cur.execute(f"DELETE FROM assignments WHERE assignment_id IN
({placeholders})", assignment_ids)

```

```

        cur.execute("DELETE FROM message_reads WHERE message_id IN
(SELECT message_id FROM messages WHERE course_id=?)", (course_id,))

        cur.execute("DELETE FROM materials WHERE course_id=?", (course_id,))

        cur.execute("DELETE FROM messages WHERE course_id=?", (course_id,))

        cur.execute("DELETE FROM course_members WHERE course_id=?",
(course_id,))

        cur.execute("DELETE FROM courses WHERE course_id=?", (course_id,))

        conn.commit()

        return True, "COURSE_DELETED"

except sqlite3.Error as e:

    conn.rollback()

    return False, str(e)

finally:

    conn.close()

```

```

def get_course(course_id: int):

```

שליפת פרטי קורס לפי ID.

```

    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute)

"        SELECT course_id, name, description, teacher_id, schoolID, created_at
FROM courses WHERE course_id, "?=

)        course_id(,

(

        row = cur.fetchone()

        if not row:

```

```

        return False, "COURSE_NOT_FOUND"

    return True} ,

    "        course_id": row,[0]
    "        name": row,[1]
    "        description": row,[2]
    "        teacher_id": row,[3]
    "        school_id": row,[4]
    "        created_at": row[5]
    {
    finally:
        conn.close()

```

```

#
#COURSE MEMBERS
#

```

```

def add_student_to_course(course_id: int, student_id: int, member_status: str =
"ACTIVE"):
    """שיוך תלמיד לקורס."""
    conn = _conn()

    try:
        joined_at = datetime.now().isoformat(timespec="seconds")

        conn.execute(

            "        INSERT OR IGNORE INTO course_members (course_id, student_id,
joined_at, member_status) VALUES,(? ,? ,? ,?)

            )        course_id, student_id, joined_at, str(member_status or
"ACTIVE").upper()

            (

```

```

        conn.commit()

        return True, "STUDENT_ADDED"

except sqlite3.Error as e:

    return False, str(e)

finally:

    conn.close()


def remove_student_from_course(course_id: int, student_id: int):
    """הסרת תלמיד מקורס."""
    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute("DELETE FROM course_members WHERE course_id=? AND
student_id,"?=

                    (course_id, student_id)(

        conn.commit()

        if cur.rowcount == 0:

            return False, "MEMBER_NOT_FOUND"

        return True, "STUDENT_REMOVED"

    finally:

        conn.close()


def delete_course_member(course_id: int, student_id: int):
    """מחיקת בקשת הצטרפות או תלמיד מקורס."""
    return remove_student_from_course(course_id, student_id)

```



```

def get_students_in_course(course_id: int):
    """כל התלמידים בקורס מסוים."""
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("""
            SELECT u.id, u.name, u.email, COALESCE(cm.member_status,
'ACTIVE'), cm.joined_at
            FROM users u
            JOIN course_members cm ON u.id = cm.student_id
            WHERE cm.course_id? =
            ORDER BY u.name COLLATE NOCASE
        , """)
        (course_id,)(
            students = [{"id": r[0], "name": r[1], "email": r[2], "member_status": r[3],
"joined_at": r[4] or ""} for r in cur.fetchall()]
            return True, students
    finally:
        conn.close()

```

```

def update_course_member_status(course_id: int, student_id: int,
member_status: str):
    """עדכון סטטוס תלמיד בתוך קורס ) ACTIVE / INACTIVE / PENDING""".(
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute)

```

```
"          UPDATE course_members SET member_status=? WHERE course_id=?
AND student_id,"?=
```

```
)          member_status, course_id, student_id(
```

```
(
```

```
    conn.commit()
```

```
    if cur.rowcount == 0:
```

```
        return False, "MEMBER_NOT_FOUND"
```

```
    return True, "MEMBER_STATUS_UPDATED"
```

```
finally:
```

```
    conn.close()
```

```
def get_course_member_status(course_id: int, student_id: int):
```

```
    """מחזיר את סטטוס החברות של תלמיד בכיתה."""
```

```
    conn = _conn()
```

```
    try:
```

```
        cur = conn.cursor()
```

```
        cur.execute)
```

```
"          SELECT COALESCE(member_status, 'ACTIVE') FROM course_members
WHERE course_id=? AND student_id,"?=
```

```
)          course_id, student_id(
```

```
(
```

```
    row = cur.fetchone()
```

```
    if not row:
```

```
        return False, "MEMBER_NOT_FOUND"
```

```
    return True, row[0]
```

```
finally:
```

```
    conn.close()
```

```

def is_student_active_in_course(course_id: int, student_id: int):

    ok_status, status = get_course_member_status(course_id, student_id)

    if not ok_status:

        return False

    return str(status).upper() == "ACTIVE"

_____#

#ASSIGNMENTS

_____#

def create_assignment(course_id: int, title: str, description: str, due_date: str,
attachment_path=None, ai_enabled: bool = True):

    , שגיאה) (False) או (True, assignment_id) יצירת מטלה חדשה בקורס. מחזיר )

    conn = _conn()

    try:

        created_at = datetime.now().isoformat(timespec="seconds")

        cur = conn.cursor()

        cur.execute)

"        INSERT INTO assignments (course_id, title, description, due_date,
attachment_path, ai_enabled, is_closed, created_at) VALUES (?, ?, ?, ?, ?, ?)
, "(?, ?

)        course_id, title, description, due_date, attachment_path, 1 if ai_enabled
else 0, 0, created_at(

(

        conn.commit()

        return True, cur.lastrowid

    except sqlite3.Error as e:

        return False, str(e)

```

```

finally:
    conn.close()

def get_assignments_for_course(course_id: int):
    """כל המטלות של קורס מסוים."""
    conn = _conn()
    try:
        close_expired_assignments(course_id=course_id, conn=conn)

        cur = conn.cursor()

        cur.execute)

        "
            SELECT assignment_id, course_id, title, description, due_date,
            attachment_path, ai_enabled, is_closed, created_at FROM assignments WHERE
            course_id=? ORDER BY COALESCE(due_date, created_at), assignment_id,"
        )
        course_id(,
        (

        assignments = [_assignment_row_to_dict(r) for r in cur.fetchall()]

        return True, assignments

    finally:
        conn.close()

def get_assignment(assignment_id: int):
    """ID""".שליפת מטלה בודדת לפי

    conn = _conn()

    try:

        close_expired_assignments(assignment_id=assignment_id, conn=conn)

        cur = conn.cursor()

        cur.execute)

```

```
"        SELECT assignment_id, course_id, title, description, due_date,
attachment_path, ai_enabled, is_closed, created_at FROM assignments WHERE
assignment_id,"?="
```

```
)        assignment_id(,
```

```
(
```

```
    row = cur.fetchone()
```

```
    if not row:
```

```
        return False, "ASSIGNMENT_NOT_FOUND"
```

```
    return True, _assignment_row_to_dict(row)
```

```
finally:
```

```
    conn.close()
```

```
def set_assignment_closed(assignment_id: int, is_closed: bool):
```

```
    """עדכון מצב פתוח/סגור של מטלה."""
```

```
    conn = _conn()
```

```
    try:
```

```
        close_expired_assignments(assignment_id=assignment_id, conn=conn)
```

```
        cur = conn.cursor()
```

```
        cur.execute)
```

```
"        SELECT is_closed, due_date FROM assignments WHERE
assignment_id,"?="
```

```
)        assignment_id(,
```

```
(
```

```
    row = cur.fetchone()
```

```
    if not row:
```

```
        return False, "ASSIGNMENT_NOT_FOUND"
```

```
    due_dt = _parse_due_datetime(row[1])
```

```
    if not bool(is_closed) and due_dt is not None and due_dt <=
datetime.now:()
```

```
        return False, "ASSIGNMENT_ALREADY_PAST_DUE"
```

```
        cur.execute("UPDATE assignments SET is_closed=? WHERE
assignment_id=?", (1 if is_closed else 0, assignment_id))
```

```
        conn.commit()
```

```
        if cur.rowcount == 0:
```

```
            return False, "ASSIGNMENT_NOT_FOUND"
```

```
        return True, "ASSIGNMENT_UPDATED"
```

```
    finally:
```

```
        conn.close()
```

```
def delete_assignment(assignment_id: int):
```

```
    """מחיקת מטלה וכל המידע המשויך אליה."""
```

```
    conn = _conn()
```

```
    try:
```

```
        cur = conn.cursor()
```

```
        cur.execute("SELECT attachment_path FROM assignments WHERE
assignment_id=?", (assignment_id,))
```

```
        row = cur.fetchone()
```

```
        if not row:
```

```
            return False, "ASSIGNMENT_NOT_FOUND"
```

```
        attachment_path = row[0] or ""
```

```
        cur.execute("SELECT submission_id, file_path FROM submissions WHERE
assignment_id=?", (assignment_id,))
```

```
        submission_rows = cur.fetchall()
```

```

submission_ids = [int(r[0]) for r in submission_rows]
submission_paths = [r[1] for r in submission_rows if r[1]]

if submission_ids:
    placeholders = ",".join(["?"] * len(submission_ids))

    cur.execute(f"DELETE FROM ai_results WHERE submission_id IN
({placeholders})", submission_ids)

    cur.execute(f"DELETE FROM grades WHERE submission_id IN
({placeholders})", submission_ids)

    cur.execute(f"DELETE FROM feedback WHERE submission_id IN
({placeholders})", submission_ids)

    cur.execute(f"DELETE FROM grade_reads WHERE submission_id IN
({placeholders})", submission_ids)

    cur.execute("DELETE FROM submissions WHERE assignment_id=?",
(assignment_id,))

    cur.execute("DELETE FROM assignments WHERE assignment_id=?",
(assignment_id,))

    conn.commit()

    return True, {"message": "ASSIGNMENT_DELETED", "attachment_path":
attachment_path, "submission_paths": submission_paths}

except sqlite3.Error as e:

    conn.rollback()

    return False, str(e)

finally:

    conn.close()

def get_assignments_for_school(school_id: int):
    """כל המשימות של בית ספר מסוים (לשימוש מנהל)."""

    conn = _conn()

    try:

```

```

cur = conn.cursor()

cur.execute("SELECT course_id FROM courses WHERE schoolID=?",
(school_id,))

for (course_id,) in cur.fetchall():

    close_expired_assignments(course_id=int(course_id), conn=conn)

cur.execute("""
    SELECT a.assignment_id, a.title, a.description, a.due_date,
           c.name as course_name, u.name as teacher_name,
           COUNT(s.submission_id) as submission_count
    FROM assignments a
    JOIN courses c ON a.course_id = c.course_id
    JOIN users u ON c.teacher_id = u.id
    LEFT JOIN submissions s ON a.assignment_id = s.assignment_id
    WHERE c.schoolID? =
    GROUP BY a.assignment_id
    ORDER BY COALESCE(a.due_date, a.created_at), a.assignment_id
, """)
    (school_id,)(
assignments] =
}
"    assignment_id": r,[0]
"    title":    r,[1]
"    description": r,[2]
"    due_date":  _normalize_due_date_text(r[3]),
"    course_name": r,[4]
"    teacher_name": r,[5]
"    submission_count": r[6]
{

```



```

        for r in cur.fetchall()

[
    return True, assignments
except sqlite3.Error as e:
    return False, str(e)
finally:
    conn.close()

_____#
#SUBMISSIONS
_____#

def save_submission(assignment_id: int, student_id: int, file_path: str, status: str
= "PENDING_AI"):
    """שמירת הגשה חדשה, תוך החלפת כל הגשה קודמת של אותו תלמיד לאותה
    מטלה."""
    conn = _conn()
    try:
        submitted_at = datetime.now().isoformat(timespec="seconds")
        cur = conn.cursor()

        cur.execute("""
            SELECT submission_id, file_path
            FROM submissions
            WHERE assignment_id=? AND student_id=?
            ,"" (int(assignment_id), int(student_id))(
        previous_rows = cur.fetchall()
        previous_ids = [int(r[0]) for r in previous_rows]
        previous_file_paths = [r[1] or "" for r in previous_rows if r[1]]

```

```

    if previous_ids:

        placeholders = ",".join(["?"] * len(previous_ids))

        cur.execute(f"DELETE FROM ai_results WHERE submission_id IN
({placeholders})", previous_ids)

        cur.execute(f"DELETE FROM grades WHERE submission_id IN
({placeholders})", previous_ids)

        cur.execute(f"DELETE FROM feedback WHERE submission_id IN
({placeholders})", previous_ids)

        cur.execute(f"DELETE FROM grade_reads WHERE submission_id IN
({placeholders})", previous_ids)

        cur.execute(f"DELETE FROM submissions WHERE submission_id IN
({placeholders})", previous_ids)

    cur.execute)

    "        INSERT INTO submissions (assignment_id, student_id, file_path,
submitted_at, status) VALUES, "(? ,? ,? ,? ,? )
)        assignment_id, student_id, file_path, submitted_at, status(
(

    conn.commit()

    return True, {"submission_id": int(cur.lastrowid), "replaced_file_paths":
previous_file_paths}

except sqlite3.Error as e:

    conn.rollback()

    return False, str(e)

finally:

    conn.close()

def get_submissions_for_assignment(assignment_id: int):

```

"""כל ההגשות למטלה (לשימוש מורה)."""

```
conn = _conn()
```

```
try:
```

```
    cur = conn.cursor()
```

```
    cur.execute("""
```

```
        SELECT s.submission_id, s.student_id, u.name, s.file_path,
        s.submitted_at, s.status,
```

```
            g.ai_score, g.final_score
```

```
        FROM submissions s
```

```
        JOIN users u ON s.student_id = u.id
```

```
        LEFT JOIN grades g ON s.submission_id = g.submission_id
```

```
        WHERE s.assignment_id? =
```

```
            AND s.submission_id) =
```

```
            SELECT MAX(s2.submission_id)
```

```
            FROM submissions s2
```

```
            WHERE s2.assignment_id = s.assignment_id AND s2.student_id =
s.student_id
```

```
(
```

```
    ORDER BY s.submitted_at DESC, s.submission_id DESC
```

```
, """    (assignment_id,)(
```

```
    submissions = [{"submission_id": r[0], "student_id": r[1], "student_name":
r[2],[
```

```
"        file_path": r[3], "submitted_at": r[4], "status": r,[5]
```

```
"        ai_score": r[6], "final_score": r[[7]
```

```
        for r in cur.fetchall()
```

```
    return True, submissions
```

```
finally:
```

```
    conn.close()
```

```

def get_submission(submission_id: int):
    """פרטי הגשה ספציפית."""
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute(
            "SELECT submission_id, assignment_id, student_id, file_path,
            submitted_at, status FROM submissions WHERE submission_id, " +=
            submission_id(,
            (
                row = cur.fetchone()
                if not row:
                    return False, "SUBMISSION_NOT_FOUND"
                return True, {"submission_id": row[0], "assignment_id": row[1], "student_id":
row[2], [
            "file_path": row[3], "submitted_at": row[4], "status": row[5]
        ]
    finally:
        conn.close()

```

```

def update_submission_status(submission_id: int, status: str):
    """עדכון סטטוס הגשה PENDING_AI / AI_DONE / ERROR"""
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("UPDATE submissions SET status=? WHERE submission_id=?",
(status, submission_id))
        conn.commit()

```

```

        if cur.rowcount == 0:

            return False, "SUBMISSION_NOT_FOUND"

        return True, "STATUS_UPDATED"

    finally:

        conn.close()

_____  
#

#AI RESULTS

_____  
#

def save_ai_result(submission_id: int, engine_name: str, score: float, feedback:
str):

    אחד. AI שמיירת תוצאה ממנוע

    conn = _conn()

    try:

        conn.execute)

        "          INSERT INTO ai_results (submission_id, engine_name, score, feedback)
VALUES,"(? ,? ,? ,?)

    )          submission_id, engine_name, score, feedback(

    (

        conn.commit()

        return True, "AI_RESULT_SAVED"

    except sqlite3.Error as e:

        return False, str(e)

    finally:

        conn.close()

```

```

def get_ai_results(submission_id: int):
    להגשה מסוימת. AI""" כל תוצאות ה-
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute(
            "SELECT engine_name, score, feedback FROM ai_results WHERE
            submission_id, "=?=
        )
        submission_id(,
        (
            results = [{"engine": r[0], "score": r[1], "feedback": r[2]} for r in cur.fetchall()]
            return True, results
    finally:
        conn.close()

```

#

#GRADES

#

```

def save_grade(submission_id: int, ai_score: float, final_score=None,
updated_by=None):
    ראשוני (ו/או ציון סופי) להגשה. AI""" שמירת ציון
    conn = _conn()
    try:
        updated_at = datetime.now().isoformat(timespec="seconds")
        conn.execute("""
            INSERT INTO grades (submission_id, ai_score, final_score, updated_by,
            updated_at)

```

```

VALUES(?, ?, ?, ?, ?)

ON CONFLICT(submission_id) DO UPDATE SET

    ai_score = excluded.ai_score,

    final_score = excluded.final_score,

    updated_by = excluded.updated_by,

    updated_at = excluded.updated_at

, "" (submission_id, ai_score, final_score, updated_by, updated_at)(

    conn.commit()

    return True, "GRADE_SAVED"

except sqlite3.Error as e:

    return False, str(e)

finally:

    conn.close()

```

```

def update_final_grade(submission_id: int, final_score: float, updated_by: int):

```

```

    """עדכון ציון סופי על ידי מורה."""

```

```

    conn = _conn()

```

```

    try:

```

```

        updated_at = datetime.now().isoformat(timespec="seconds")

```

```

        cur = conn.cursor()

```

```

        cur.execute("""

```

```

            UPDATE grades SET final_score=?, updated_by=?, updated_at=?=

```

```

            WHERE submission_id=?

```

```

, "" (final_score, updated_by, updated_at, submission_id)(

```

```

    conn.commit()

```

```

    if cur.rowcount == 0:

```

```

        return False, "GRADE_NOT_FOUND"

```

```

        return True, "GRADE_UPDATED"

    finally:

        conn.close()


def get_grade(submission_id: int):
    """שליפת ציון להגשה ספציפית."""
    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute(
            "
                SELECT ai_score, final_score, updated_by, updated_at FROM grades
                WHERE submission_id,?=
            )
            submission_id(,
            (
                row = cur.fetchone()

                if not row:

                    return False, "GRADE_NOT_FOUND"

                return True, {"ai_score": row[0], "final_score": row[1],[
            "
                updated_by": row[2], "updated_at": row{[3]

    finally:

        conn.close()


#
#FEEDBACK
#

```



```

def save_feedback(submission_id: int, ai_feedback=None,
teacher_feedback=None):

    ומשוב מורה. AI""" שמירה/עדכון משוב

    conn = _conn()

    try:

        conn.execute("""

            INSERT INTO feedback (submission_id, ai_feedback, teacher_feedback)

            VALUES(?, ?, ?)

            ON CONFLICT(submission_id) DO UPDATE SET

                ai_feedback = COALESCE(excluded.ai_feedback, ai_feedback),

                teacher_feedback = COALESCE(excluded.teacher_feedback,

teacher_feedback)

, "" (submission_id, ai_feedback, teacher_feedback)(

        conn.commit()

        return True, "FEEDBACK_SAVED"

    except sqlite3.Error as e:

        return False, str(e)

    finally:

        conn.close()

```

```

def get_feedback(submission_id: int):

    """שליפת משוב להגשה ספציפית.

    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute)

    " SELECT ai_feedback, teacher_feedback FROM feedback WHERE

submission_id,"?="

```

```

)      submission_id(,
(
    row = cur.fetchone()
    if not row:
        return False, "FEEDBACK_NOT_FOUND"
    return True, {"ai_feedback": row[0], "teacher_feedback": row[1]}
finally:
    conn.close()

```

```
#
```

```
#MATERIALS
```

```
#
```

```

def create_material(course_id: int, title: str, description: str = "", material_type:
str = "FILE", created_by=None, file_path=None):

```

```

"""יצירת חומר לימוד לקורס."""

```

```

    conn = _conn()
    try:
        created_at = datetime.now().isoformat(timespec="seconds")
        cur = conn.cursor()
        cur.execute(
            "INSERT INTO materials (course_id, title, description, type, created_by,
file_path, created_at) VALUES, "(? ,? ,? ,? ,? ,? ,? )"
        )
        course_id, title, description, material_type, created_by, file_path,
created_at(
        (
            conn.commit()

```

```

        return True, cur.lastrowid

except sqlite3.Error as e:
    return False, str(e)

finally:
    conn.close()


def get_materials_for_course(course_id: int):
    """כל חומרי הלימוד של קורס מסוים."""
    conn = _conn()

    try:
        cur = conn.cursor()

        cur.execute("""
            SELECT m.material_id, m.course_id, m.title, m.description, m.type,
                   m.created_by, m.file_path, m.created_at,
                   COALESCE(u.name, 'Teacher') as created_by_name
            FROM materials m
            LEFT JOIN users u ON m.created_by = u.id
            WHERE m.course_id=?
            ORDER BY m.created_at DESC
        """, (course_id,))

        materials = []

        "    material_id": r,[0]
        "    course_id": r,[1]
        "    title": r,[2]
        "    description": r,[3]
        "    type": r,[4]
        "    created_by": r,[5]

```

```

"        file_path": r,[6]
"        created_at": r,[7]
"        created_by_name": r[8]
{        for r in cur.fetchall():
        return True, materials
finally:
        conn.close()

```

```

def get_material(material_id: int):

```

```

    """שליפת חומר לימוד בודד."""

```

```

    conn = _conn()

```

```

    try:

```

```

        cur = conn.cursor()

```

```

        cur.execute("""

```

```

            SELECT material_id, course_id, title, description, type, created_by,
file_path, created_at

```

```

            FROM materials

```

```

            WHERE material_id=?

```

```

, """)
        (material_id,)(

```

```

        row = cur.fetchone()

```

```

        if not row:

```

```

            return False, "MATERIAL_NOT_FOUND"

```

```

        return True} ,

```

```

"        material_id": row,[0]

```

```

"        course_id": row,[1]

```

```

"        title": row,[2]

```

```

"        description": row,[3]

```

```

"        type": row,[4]
"        created_by": row,[5]
"        file_path": row,[6]
"        created_at": row[7]
{
    finally:
        conn.close()

```

```

def delete_material(material_id: int):

```

```

    """מחיקת חומר לימוד."""

```

```

    conn = _conn()

```

```

    try:

```

```

        cur = conn.cursor()

```

```

        cur.execute("SELECT file_path, course_id FROM materials WHERE
material_id=?", (material_id,))

```

```

        row = cur.fetchone()

```

```

        if not row:

```

```

            return False, "MATERIAL_NOT_FOUND"

```

```

        file_path = row[0] or ""

```

```

        course_id = int(row[1])

```

```

        cur.execute("DELETE FROM materials WHERE material_id=?",
(material_id,))

```

```

        conn.commit()

```

```

        return True, {"message": "MATERIAL_DELETED", "file_path": file_path,
"course_id": course_id}

```

```

    finally:

```

```

        conn.close()

```

```

#MESSAGES
#

def create_message(course_id: int, sender_id: int, subject: str, body: str, state:
str = "sent"):
    """שמירת הודעה/טיוטה לקורס."""
    conn = _conn()
    try:
        created_at = datetime.now().isoformat(timespec="seconds")
        cur = conn.cursor()
        cur.execute(
            "INSERT INTO messages (course_id, sender_id, subject, body, state,
            created_at) VALUES, "(? , ? , ? , ? , ? , ?)
        )
        course_id, sender_id, subject, body, state, created_at(
        (
            conn.commit()
            return True, cur.lastrowid
    except sqlite3.Error as e:
        return False, str(e)
    finally:
        conn.close()

```

```

def get_messages_for_course(course_id: int):
    """שליפת הודעות של קורס מסוים."""
    conn = _conn()
    try:

```

```

cur = conn.cursor()

cur.execute("""

    SELECT m.message_id, m.course_id, m.sender_id, m.subject, m.body,
m.state, m.created_at,

        COALESCE(u.name, 'Teacher') as sender_name

    FROM messages m

    LEFT JOIN users u ON m.sender_id = u.id

    WHERE m.course_id=?

    ORDER BY m.created_at DESC

, """)
    (course_id,)(
        messages}}] =
"        message_id": r,[0]
"        course_id": r,[1]
"        sender_id": r,[2]
"        subject": r,[3]
"        body": r,[4]
"        state": r,[5]
"        created_at": r,[6]
"        sender_name": r[7]
{        for r in cur.fetchall()

        return True, messages

    finally:

        conn.close()

def delete_message(message_id: int):

    """מחיקת הודעה."""

    conn = _conn()

```

```

try:
    cur = conn.cursor()
    cur.execute("SELECT course_id FROM messages WHERE message_id=?",
(message_id,))
    row = cur.fetchone()
    if not row:
        return False, "MESSAGE_NOT_FOUND"
    course_id = int(row[0])
    cur.execute("DELETE FROM message_reads WHERE message_id=?",
(message_id,))
    cur.execute("DELETE FROM messages WHERE message_id=?",
(message_id,))
    conn.commit()
    return True, {"message": "MESSAGE_DELETED", "course_id": course_id}
finally:
    conn.close()

```

```

--- #Added: student dashboard / read-state helpers---
def get_student_dashboard(student_id: int):
    """מחזיר את כל הנתונים למסך תלמיד: קורסים, מטלות, חומרים והודעות."""
    ok_courses, courses = get_courses_for_student(student_id)
    if not ok_courses:
        return False, courses

    conn = _conn()
    try:

```



```

cur = conn.cursor()

course_ids = [int(c["course_id"]) for c in courses]
class_list[] =

for idx, course in enumerate(courses):

    teacher_name = "Teacher"

    try:

        cur.execute("SELECT name FROM users WHERE id=?",
(int(course.get("teacher_id", -1)),))

        row = cur.fetchone()

        if row and row[0]:

            teacher_name = row[0]

    except Exception:

        pass


    cur.execute("SELECT COUNT(*) FROM course_members WHERE
course_id=? AND COALESCE(member_status, 'ACTIVE')='ACTIVE'",
(int(course["course_id"]),))

    students_count = cur.fetchone[0]()

    palette] =

#4")          F46E5", "#EEF2FF,("📖", "
#2563")          EB", "#DBEAFE,("📝", "
#" )          D97706", "#FEF3C7,("📅", "
#" ,"#059669")          D1FAE5,("📌", "
#7")          C3AED", "#EDE9FE,("📁", "
[

    accent, accent_soft, icon_text = palette[idx % len(palette)]

    class_list.append})

"        classId": int(course["course_id"]),

```

```

"        name": course.get("name", "Course"),
"        teacherName": teacher_name,
"        room,"—" : "
"        description": course.get("description") or, ""
"        studentsCount": students_count,
"        code": (course.get("class_code") or ""), # --- Modified: real stored
class code---
"        accent": accent,
"        accentSoft": accent_soft,
"        iconText": icon_text,
({

```

```

assignments[] =

```

```

materials[] =

```

```

messages[] =

```

```

if course_ids:

```

```

    placeholders = ",".join(["?"] * len(course_ids))

```

```

    for course_id in course_ids:

```

```

        close_expired_assignments(course_id=int(course_id), conn=conn)

```

```

    cur.execute(f"""

```

```

        SELECT a.assignment_id, a.course_id, a.title, a.description,
a.due_date, a.attachment_path, a.created_at, a.is_closed,
            s.submission_id, s.file_path, s.submitted_at,
            g.final_score,
            f.teacher_feedback,
            CASE WHEN gr.id IS NULL THEN 0 ELSE 1 END AS grade_read

```

```

FROM assignments a
LEFT JOIN submissions s ON s.submission_id) =
    SELECT s2.submission_id
    FROM submissions s2
    WHERE s2.assignment_id = a.assignment_id AND s2.student_id =
?

    ORDER BY s2.submission_id DESC

    LIMIT 1

(

    LEFT JOIN grades g ON g.submission_id = s.submission_id
    LEFT JOIN feedback f ON f.submission_id = s.submission_id
    LEFT JOIN grade_reads gr ON gr.submission_id = s.submission_id AND
gr.student_id? =

    WHERE a.course_id IN ({placeholders})
    ORDER BY COALESCE(a.due_date, a.created_at), a.assignment_id
, ""
    [int(student_id), int(student_id), *course_ids](
for row in cur.fetchall:()

    attachment_name = Path(row[5]).name if row[5] else ""
    submission_name = Path(row[9]).name if row[9] else ""
    assignments.append})

"        assignmentId": int(row[0]),
"        classId": int(row[1]),
"        title": row[2] or, ""
"        description": row[3] or, ""
"        instructions": row[3] or, ""
"        dueText": _normalize_due_date_text(row[4]),
"        isClosed": bool(row[7]),
"        submitted": row[8] is not None,
"        submissionId": int(row[8]) if row[8] is not None else -1,

```

```

"        finalGrade": float(row[11]) if row[11] is not None else -1,
"        teacherFeedback": row[12] or ""
"        attachmentName": attachment_name,
"        attachmentPath": row[5] or ""
"        submissionFileName": submission_name,
"        submissionPath": row[9] or ""
"        submittedAt": (row[10] or "").replace("T", " "),
"        gradeRead": bool(row[13]),
"        submissionText, "" :""
({

cur.execute(f"""

        SELECT m.material_id, m.course_id, m.type, m.title, m.description,
m.file_path, m.created_at,

        COALESCE(u.name, 'Teacher')

        FROM materials m

        LEFT JOIN users u ON m.created_by = u.id

        WHERE m.course_id IN ({placeholders})

        ORDER BY m.created_at DESC, m.material_id DESC

, """"
        course_ids(
for row in cur.fetchall():
        materials.append})

"        materialId": int(row[0]),
"        classId": int(row[1]),
"        type": row[2] or "FILE,"
"        title": row[3] or ""
"        byText": row[7] or "Teacher,"
"        description": row[4] or ""

```

```

"        viewText": row[4] or, ""
"        filePath": row[5] or, ""
"        whenText": (row[6] or "").replace("T", " "),
({

    cur.execute(f"""
        SELECT msg.message_id, msg.course_id, COALESCE(u.name,
'Teacher'), msg.subject, msg.body, msg.created_at,
        CASE WHEN mr.id IS NULL THEN 0 ELSE 1 END AS is_read
        FROM messages msg
        LEFT JOIN users u ON msg.sender_id = u.id
        LEFT JOIN message_reads mr ON mr.message_id = msg.message_id
AND mr.student_id? =
        WHERE msg.course_id IN ({placeholders}) AND COALESCE(msg.state,
'sent') <> 'draft'
        ORDER BY msg.created_at DESC, msg.message_id DESC
, """)
    [int(student_id), *course_ids](
for row in cur.fetchall:()
    messages.append})
"        messageId": int(row[0]),
"        classId": int(row[1]),
"        fromText": row[2] or "Teacher, "
"        subject": row[3] or, ""
"        body": row[4] or, ""
"        whenText": (row[5] or "").replace("T", " "),
"        read": bool(row[6]),
({

return True} ,

```

```

"        classes": class_list,
"        assignments": assignments,
"        materials": materials,
"        messages": messages,
{
    finally:
        conn.close()

```

```

def join_course_by_code(student_id: int, code_value: str):

```

```

    text = (code_value or "").strip().upper()

```

```

    if not text:

```

```

        return False, "INVALID_CLASS_CODE"

```

```

    conn = _conn()

```

```

    try:

```

```

        cur = conn.cursor()

```

```

        cur.execute("SELECT course_id FROM courses WHERE UPPER(class_code)
= ? LIMIT 1", (text,))

```

```

        row = cur.fetchone()

```

```

--- #      Added: backward compatibility for old numeric / CLS-0001 style
codes---

```

```

    if row is None:

```

```

        course_id = None

```

```

    if text.startswith("CLS-"):

```

```

        try:

```

```

            course_id = int(text.split("-", 1)[1])

```

```

        except ValueError:

```

```

        course_id = None

    elif text.isdigit():

        course_id = int(text)

    if course_id is not None:

        cur.execute("SELECT course_id FROM courses WHERE course_id = ?
LIMIT 1", (course_id,))

        row = cur.fetchone()

    if row is None:

        return False, "CLASS_NOT_FOUND"

    course_id = int(row[0])

    cur.execute)
"        SELECT COALESCE(member_status, 'ACTIVE') FROM course_members
WHERE course_id=? AND student_id, "?=
)        course_id, student_id(
(
    existing = cur.fetchone()

    if existing:

        existing_status = str(existing[0]).upper()

        if existing_status == "INACTIVE:"

            return False, "You were blocked from this class. Please contact the
class teacher".

        if existing_status == "PENDING:"

            return False, "Your join request is already waiting for approval".

        return False, "You are already enrolled in this class".

```

```

        return add_student_to_course(course_id, student_id, "PENDING")

    finally:
        conn.close()

def leave_course_for_student(course_id: int, student_id: int):
    return remove_student_from_course(course_id, student_id)

def delete_submission_for_student(assignment_id: int, student_id: int):
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("""
            SELECT submission_id, file_path
            FROM submissions
            WHERE assignment_id=? AND student_id=?
            ORDER BY submission_id DESC
            , """, (assignment_id, student_id))
        rows = cur.fetchall()
        if not rows:
            return False, "SUBMISSION_NOT_FOUND"

        submission_ids = [int(r[0]) for r in rows]
        file_paths = [r[1] or "" for r in rows if r[1]]
        latest_submission_id = submission_ids[0]
        placeholders = ",".join(["?"] * len(submission_ids))

```



```

        cur.execute(f"DELETE FROM ai_results WHERE submission_id IN
({placeholders})", submission_ids)

        cur.execute(f"DELETE FROM grades WHERE submission_id IN
({placeholders})", submission_ids)

        cur.execute(f"DELETE FROM feedback WHERE submission_id IN
({placeholders})", submission_ids)

        cur.execute(f"DELETE FROM grade_reads WHERE submission_id IN
({placeholders}) AND student_id=?", [*submission_ids, int(student_id)])

        cur.execute(f"DELETE FROM submissions WHERE submission_id IN
({placeholders})", submission_ids)

        conn.commit()

        return True, {"submission_id": latest_submission_id, "file_paths":
file_paths}

    except sqlite3.Error as e:

        conn.rollback()

        return False, str(e)

    finally:

        conn.close()

```

```

def mark_message_read(student_id: int, message_id: int):

    conn = _conn()

    try:

        read_at = datetime.now().isoformat(timespec="seconds")

        conn.execute(

            "        INSERT INTO message_reads (message_id, student_id, read_at) VALUES
            (?, ?, ?) ON CONFLICT(message_id, student_id) DO NOTHING,"

        )

        message_id, student_id, read_at(

        (

            conn.commit()

```

```

        return True, "MESSAGE_MARKED_READ"
except sqlite3.Error as e:
    return False, str(e)
finally:
    conn.close()

def mark_all_messages_read(student_id: int):
    ok_dash, data = get_student_dashboard(student_id)
    if not ok_dash:
        return False, data
    conn = _conn()
    try:
        read_at = datetime.now().isoformat(timespec="seconds")
        for item in data.get("messages", []):
            conn.execute(
                "
                INSERT INTO message_reads (message_id, student_id, read_at)
VALUES (?, ?, ?) ON CONFLICT(message_id, student_id) DO NOTHING,"
            )
            int(item["messageId"]), int(student_id), read_at(
            (
                conn.commit()
                return True, "ALL_MESSAGES_MARKED_READ"
except sqlite3.Error as e:
    return False, str(e)
finally:
    conn.close()

```

```

def mark_grade_read(student_id: int, submission_id: int):
    conn = _conn()
    try:
        read_at = datetime.now().isoformat(timespec="seconds")
        conn.execute(
            "      INSERT INTO grade_reads (submission_id, student_id, read_at) VALUES
            (?, ?, ?) ON CONFLICT(submission_id, student_id) DO NOTHING,"
        )
        conn.commit()
        return True, "GRADE_MARKED_READ"
    except sqlite3.Error as e:
        return False, str(e)
    finally:
        conn.close()

```

#

#STATS(לשימוש מנהל)

#

```

def get_school_stats(school_id: int):
    """סטטיסטיקות כלליות לבית ספר: מספר משתמשים, קורסים, הגשות, ממוצע ציון."""
    conn = _conn()
    try:
        cur = conn.cursor()

        cur.execute("SELECT COUNT(*) FROM users WHERE schoolID=? AND
        role='STUDENT'", (school_id,))

        num_students = cur.fetchone()[0]()
    
```

```

        cur.execute("SELECT COUNT(*) FROM users WHERE schoolID=? AND
role='TEACHER'", (school_id,))

```

```

        num_teachers = cur.fetchone[0]()

```

```

        cur.execute("SELECT COUNT(*) FROM courses WHERE schoolID=?",
(school_id,))

```

```

        num_courses = cur.fetchone[0]()

```

```

cur.execute("""

```

```

    SELECT COUNT(*) FROM submissions s

```

```

    JOIN assignments a ON s.assignment_id = a.assignment_id

```

```

    JOIN courses c ON a.course_id = c.course_id

```

```

    WHERE c.schoolID=?

```

```

, """)
    (school_id,)(

```

```

    num_submissions = cur.fetchone[0]()

```

```

cur.execute("""

```

```

    SELECT AVG(g.final_score) FROM grades g

```

```

    JOIN submissions s ON g.submission_id = s.submission_id

```

```

    JOIN assignments a ON s.assignment_id = a.assignment_id

```

```

    JOIN courses c ON a.course_id = c.course_id

```

```

    WHERE c.schoolID=? AND g.final_score IS NOT NULL

```

```

, """)
    (school_id,)(

```

```

    avg_grade = cur.fetchone[0]()

```

```

return True} ,

```

```

"    num_students": num_students,

```

```

"    num_teachers": num_teachers,

```

```

"    num_courses": num_courses,
"    num_submissions": num_submissions,
"    avg_grade": round(avg_grade, 1) if avg_grade else None
{
    finally:
        conn.close()

```

```

#
#AI WORKER HELPERS
#

```

```

def claim_next_pending_submission:()
-ל- תפיסת ההגשה הממתינה הבאה בצורה אוטומית והעברתה ל- AI_IN_PROGRESS"".
    conn = _conn()
    try:
        conn.execute("BEGIN IMMEDIATE")
        cur = conn.cursor()
        cur.execute("""
            SELECT submission_id
            FROM submissions
            WHERE status = 'PENDING_AI'
            ORDER BY submission_id ASC
            LIMIT 1
        """)
        row = cur.fetchone()
        if not row:
            conn.commit()
            return False, "NO_PENDING_SUBMISSIONS"

```

```

        submission_id = int(row[0])

        cur.execute)

"        UPDATE submissions SET status=? WHERE submission_id=? AND
status='PENDING_AI',"
")        AI_IN_PROGRESS", submission_id(
(
        if cur.rowcount == 0:

            conn.rollback()

            return False, "CLAIM_CONFLICT"

        conn.commit()

        return get_submission(submission_id)
except sqlite3.Error as e:

    conn.rollback()

    return False, str(e)
finally:

    conn.close()

def list_pending_submission_ids(limit: int = 20):
    """רשימת הגשות ממתינות, ללא תפיסה שלהן לעיבוד."""
    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute)

"        SELECT submission_id FROM submissions WHERE
status='PENDING_AI' ORDER BY submission_id ASC LIMIT, "?"
)        int(limit)(,

```

```

(
    return True, [int(r[0]) for r in cur.fetchall()]
finally:
    conn.close()

def reset_in_progress_submissions():
    """
    בעת PENDING_AI בחזרה ל-AI_IN_PROGRESS מאפס עבודות שנתקעו במצב
    עליית השרת.
    """
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("UPDATE submissions SET status='PENDING_AI' WHERE
status='AI_IN_PROGRESS'")
        conn.commit()
        return True, cur.rowcount
    except sqlite3.Error as e:
        conn.rollback()
        return False, str(e)
    finally:
        conn.close()

def delete_ai_results_for_submission(submission_id: int):
    """
    קודמות להגשה, לפני ריצה מחדש. AI"""
    """ניקוי תוצאות"""
    conn = _conn()
    try:
        conn.execute("DELETE FROM ai_results WHERE submission_id=?",
(submission_id,))

```

```

        conn.commit()

        return True, "AI_RESULTS_DELETED"

except sqlite3.Error as e:

    conn.rollback()

    return False, str(e)

finally:

    conn.close()

_____#

#ADMIN HELPERS

_____#

def admin_get_overview():

    """ Full system snapshot for ADMIN dashboard"""

    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute("""

            SELECT s.school_id, s.name, s.address, s.contact_name,
s.contact_email, COALESCE(s.created_at, ''),

                COUNT(DISTINCT u.id) as users_count,

                COUNT(DISTINCT c.course_id) as courses_count

            FROM schools s

            LEFT JOIN users u ON s.school_id = u.schoolID

            LEFT JOIN courses c ON s.school_id = c.schoolID

            GROUP BY s.school_id

            ORDER BY s.school_id

        """)

```



```

schools}}] =
"    school_id": r,[0]
"    name": r[1] or, ""
"    address": r[2] or, ""
"    contact_name": r[3] or, ""
"    contact_email": r[4] or, ""
"    created_at": r[5] or, ""
"    users_count": int(r[6] or 0),
"    courses_count": int(r[7] or 0)
{    for r in cur.fetchall()

cur.execute("""
    SELECT u.id, u.name, u.email, u.role, u.schoolID, u.status, u.created_at,
           COALESCE(s.name, "") as school_name,
           COUNT(DISTINCT cm.course_id) as memberships_count
    FROM users u
    LEFT JOIN schools s ON u.schoolID = s.school_id
    LEFT JOIN course_members cm ON u.id = cm.student_id
    GROUP BY u.id
    ORDER BY u.id
    (""
    users}}] =
"    id": r,[0]
"    name": r[1] or, ""
"    email": r[2] or, ""
"    role": r[3] or, ""
"    school_id": r,[4]
"    status": r[5] or, ""

```

```

"        created_at": r[6] or ""
"        school_name": r[7] or ""
"        memberships_count": int(r[8] or 0)
{        for r in cur.fetchall()

cur.execute("""

        SELECT c.course_id, c.name, c.description, c.teacher_id, c.schoolID,
c.created_at, c.class_code,

                COALESCE(u.name, '') as teacher_name, COALESCE(s.name, '') as
school_name,

                COUNT(DISTINCT CASE WHEN COALESCE(cm.member_status,
'ACTIVE') = 'ACTIVE' THEN cm.student_id END) as students,

                COUNT(DISTINCT a.assignment_id) as assignments_count,

                COUNT(DISTINCT m.material_id) as materials_count,

                COUNT(DISTINCT msg.message_id) as messages_count

FROM courses c

LEFT JOIN users u ON c.teacher_id = u.id

LEFT JOIN schools s ON c.schoolID = s.school_id

LEFT JOIN course_members cm ON c.course_id = cm.course_id

LEFT JOIN assignments a ON c.course_id = a.course_id

LEFT JOIN materials m ON c.course_id = m.course_id

LEFT JOIN messages msg ON c.course_id = msg.course_id

GROUP BY c.course_id

ORDER BY c.course_id

("""
courses}] =
"        course_id": r,[0]
"        name": r[1] or ""
"        description": r[2] or ""

```

```

"        teacher_id": r,[3]
"        school_id": r,[4]
"        created_at": r[5] or ""
"        class_code": r[6] or ""
"        teacher_name": r[7] or ""
"        school_name": r[8] or ""
"        students": int(r[9] or 0),
"        assignments_count": int(r[10] or 0),
"        materials_count": int(r[11] or 0),
"        messages_count": int(r[12] or 0)
{        for r in cur.fetchall()

cur.execute("""

        SELECT cm.id, cm.course_id, COALESCE(c.name, "") as course_name,
        COALESCE(c.class_code, "") as class_code,

                c.schoolID, COALESCE(s.name, "") as school_name, cm.student_id,

                COALESCE(u.name, "") as student_name, COALESCE(u.email, "") as
student_email,

                COALESCE(cm.member_status, 'ACTIVE') as member_status,
cm.joined_at

        FROM course_members cm

        LEFT JOIN courses c ON cm.course_id = c.course_id

        LEFT JOIN schools s ON c.schoolID = s.school_id

        LEFT JOIN users u ON cm.student_id = u.id

        ORDER BY cm.id

        (""
members}} =
"        member_id": r,[0]
"        course_id": r,[1]

```

```

"    course_name": r[2] or, ""
"    class_code": r[3] or, ""
"    school_id": r,[4]
"    school_name": r[5] or, ""
"    student_id": r,[6]
"    student_name": r[7] or, ""
"    student_email": r[8] or, ""
"    member_status": r[9] or, ""
"    joined_at": r[10] or ""
{    for r in cur.fetchall()

cur.execute("""

    SELECT a.assignment_id, a.course_id, a.title, a.description, a.due_date,
a.attachment_path, a.ai_enabled, a.is_closed, a.created_at,

        COALESCE(c.name, "") as course_name, COALESCE(u.name, "") as
teacher_name,

        COALESCE(s.name, "") as school_name, c.schoolID,

        COUNT(DISTINCT sub.submission_id) as submission_count,

        COUNT(DISTINCT g.grade_id) as graded_count

FROM assignments a

LEFT JOIN courses c ON a.course_id = c.course_id

LEFT JOIN users u ON c.teacher_id = u.id

LEFT JOIN schools s ON c.schoolID = s.school_id

LEFT JOIN submissions sub ON a.assignment_id = sub.assignment_id

LEFT JOIN grades g ON sub.submission_id = g.submission_id

GROUP BY a.assignment_id

ORDER BY a.assignment_id

("""

assignments]] =

```

```

"    assignment_id": r,[0]
"    course_id": r,[1]
"    title": r[2] or, ""
"    description": r[3] or, ""
"    due_date": _normalize_due_date_text(r[4]),
"    attachment_path": r[5] or, ""
"    ai_enabled": int(r[6] or 0),
"    is_closed": int(r[7] or 0),
"    created_at": r[8] or, ""
"    course_name": r[9] or, ""
"    teacher_name": r[10] or, ""
"    school_name": r[11] or, ""
"    school_id": r,[12]
"    submission_count": int(r[13] or 0),
"    graded_count": int(r[14] or 0)
{    for r in cur.fetchall():

cur.execute("""

    SELECT sub.submission_id, sub.assignment_id, COALESCE(a.title, '') as
assignment_title,

        sub.student_id, COALESCE(u.name, '') as student_name,
COALESCE(u.email, '') as student_email,

        sub.file_path, sub.submitted_at, sub.status,

        COALESCE(c.course_id, -1) as course_id, COALESCE(c.name, '') as
course_name,

        COALESCE(s.school_id, -1) as school_id, COALESCE(s.name, '') as
school_name,

        g.ai_score, g.final_score, g.updated_at,

        COUNT(DISTINCT ar.ai_result_id) as ai_results_count

```

```

FROM submissions sub
LEFT JOIN assignments a ON sub.assignment_id = a.assignment_id
LEFT JOIN courses c ON a.course_id = c.course_id
LEFT JOIN schools s ON c.schoolID = s.school_id
LEFT JOIN users u ON sub.student_id = u.id
LEFT JOIN grades g ON sub.submission_id = g.submission_id
LEFT JOIN ai_results ar ON sub.submission_id = ar.submission_id
GROUP BY sub.submission_id
ORDER BY sub.submission_id

("""
    submissions}}] =
"    submission_id": r,[0]
"    assignment_id": r,[1]
"    assignment_title": r[2] or, ""
"    student_id": r,[3]
"    student_name": r[4] or, ""
"    student_email": r[5] or, ""
"    file_path": r[6] or, ""
"    submitted_at": r[7] or, ""
"    status": r[8] or, ""
"    course_id": r,[9]
"    course_name": r[10] or, ""
"    school_id": r,[11]
"    school_name": r[12] or, ""
"    ai_score": r,[13]
"    final_score": r,[14]
"    updated_at": r[15] or, ""
"    ai_results_count": int(r[16] or 0)

```

```

{    for r in cur.fetchall()

        cur.execute("""

            SELECT ar.ai_result_id, ar.submission_id, ar.engine_name, ar.score,
ar.feedback,

                COALESCE(a.title, '') as assignment_title, COALESCE(c.name, '') as
course_name,

                COALESCE(u.name, '') as student_name

            FROM ai_results ar

            LEFT JOIN submissions sub ON ar.submission_id = sub.submission_id

            LEFT JOIN assignments a ON sub.assignment_id = a.assignment_id

            LEFT JOIN courses c ON a.course_id = c.course_id

            LEFT JOIN users u ON sub.student_id = u.id

            ORDER BY ar.ai_result_id

        """)

        ai_results}}] =
"        ai_result_id": r,[0]
"        submission_id": r,[1]
"        engine_name": r[2] or, ""
"        score": r,[3]
"        feedback": r[4] or, ""
"        assignment_title": r[5] or, ""
"        course_name": r[6] or, ""
"        student_name": r[7] or ""
{    for r in cur.fetchall()

        cur.execute("""

            SELECT g.grade_id, g.submission_id, g.ai_score, g.final_score,
g.updated_by,

```

```

        COALESCE(editor.name, '') as updated_by_name, g.updated_at,
        COALESCE(a.title, '') as assignment_title, COALESCE(c.name, '') as
course_name,

        COALESCE(student.name, '') as student_name

FROM grades g

LEFT JOIN submissions sub ON g.submission_id = sub.submission_id

LEFT JOIN assignments a ON sub.assignment_id = a.assignment_id

LEFT JOIN courses c ON a.course_id = c.course_id

LEFT JOIN users student ON sub.student_id = student.id

LEFT JOIN users editor ON g.updated_by = editor.id

ORDER BY g.grade_id

("""
    grades}}] =
"    grade_id": r,[0]
"    submission_id": r,[1]
"    ai_score": r,[2]
"    final_score": r,[3]
"    updated_by": r,[4]
"    updated_by_name": r[5] or, ""
"    updated_at": r[6] or, ""
"    assignment_title": r[7] or, ""
"    course_name": r[8] or, ""
"    student_name": r[9] or, ""
{    for r in cur.fetchall[()

cur.execute("""

SELECT f.feedback_id, f.submission_id, f.ai_feedback,
f.teacher_feedback,

```



```

        COALESCE(a.title, '') as assignment_title, COALESCE(c.name, '') as
course_name,

```

```

        COALESCE(u.name, '') as student_name

```

```

FROM feedback f

```

```

LEFT JOIN submissions sub ON f.submission_id = sub.submission_id

```

```

LEFT JOIN assignments a ON sub.assignment_id = a.assignment_id

```

```

LEFT JOIN courses c ON a.course_id = c.course_id

```

```

LEFT JOIN users u ON sub.student_id = u.id

```

```

ORDER BY f.feedback_id

```

```

("""

```

```

    feedback}] =

```

```

"        feedback_id": r,[0]

```

```

"        submission_id": r,[1]

```

```

"        ai_feedback": r[2] or, ""

```

```

"        teacher_feedback": r[3] or, ""

```

```

"        assignment_title": r[4] or, ""

```

```

"        course_name": r[5] or, ""

```

```

"        student_name": r[6] or ""

```

```

{        for r in cur.fetchall()

```

```

    cur.execute("""

```

```

        SELECT m.material_id, m.course_id, COALESCE(c.name, '') as
course_name,

```

```

        COALESCE(s.name, '') as school_name, m.title, m.description,
m.type,

```

```

        m.created_by, COALESCE(u.name, '') as created_by_name,

```

```

        m.file_path, m.created_at

```

```

FROM materials m

```

```

LEFT JOIN courses c ON m.course_id = c.course_id

```

```

LEFT JOIN schools s ON c.schoolID = s.school_id
LEFT JOIN users u ON m.created_by = u.id
ORDER BY m.material_id

("""
    materials}} =
    "    material_id": r,[0]
    "    course_id": r,[1]
    "    course_name": r[2] or, ""
    "    school_name": r[3] or, ""
    "    title": r[4] or, ""
    "    description": r[5] or, ""
    "    type": r[6] or, ""
    "    created_by": r,[7]
    "    created_by_name": r[8] or, ""
    "    file_path": r[9] or, ""
    "    created_at": r[10] or ""
    {    for r in cur.fetchall()

cur.execute("""

SELECT msg.message_id, msg.course_id, COALESCE(c.name, '') as
course_name,

        COALESCE(s.name, '') as school_name, msg.sender_id,
        COALESCE(u.name, '') as sender_name, msg.subject, msg.body,
        msg.state, msg.created_at, COUNT(mr.id) as read_count

FROM messages msg

LEFT JOIN courses c ON msg.course_id = c.course_id
LEFT JOIN schools s ON c.schoolID = s.school_id
LEFT JOIN users u ON msg.sender_id = u.id

```

```

LEFT JOIN message_reads mr ON msg.message_id = mr.message_id

GROUP BY msg.message_id

ORDER BY msg.message_id

("""
    messages}}] =
    "    message_id": r,[0]
    "    course_id": r,[1]
    "    course_name": r[2] or, ""
    "    school_name": r[3] or, ""
    "    sender_id": r,[4]
    "    sender_name": r[5] or, ""
    "    subject": r[6] or, ""
    "    body": r[7] or, ""
    "    state": r[8] or, ""
    "    created_at": r[9] or, ""
    "    read_count": int(r[10] or 0)
    {    for r in cur.fetchall[()]

cur.execute("""

    SELECT mr.id, mr.message_id, COALESCE(msg.subject, "") as
message_subject,

        mr.student_id, COALESCE(u.name, "") as student_name,

        COALESCE(c.name, "") as course_name, mr.read_at

FROM message_reads mr

LEFT JOIN messages msg ON mr.message_id = msg.message_id

LEFT JOIN courses c ON msg.course_id = c.course_id

LEFT JOIN users u ON mr.student_id = u.id

ORDER BY mr.id

```

```

("""
    message_reads}}] =
"        read_id": r,[0]
"        message_id": r,[1]
"        message_subject": r[2] or, ""
"        student_id": r,[3]
"        student_name": r[4] or, ""
"        course_name": r[5] or, ""
"        read_at": r[6] or ""
{    for r in cur.fetchall[()

cur.execute("""

    SELECT gr.id, gr.submission_id, COALESCE(a.title, '') as assignment_title,
           gr.student_id, COALESCE(u.name, '') as student_name,
           COALESCE(c.name, '') as course_name, gr.read_at

    FROM grade_reads gr

    LEFT JOIN submissions sub ON gr.submission_id = sub.submission_id

    LEFT JOIN assignments a ON sub.assignment_id = a.assignment_id

    LEFT JOIN courses c ON a.course_id = c.course_id

    LEFT JOIN users u ON gr.student_id = u.id

    ORDER BY gr.id

""")

grade_reads}}] =
"        read_id": r,[0]
"        submission_id": r,[1]
"        assignment_title": r[2] or, ""
"        student_id": r,[3]
"        student_name": r[4] or, ""

```

```

"        course_name": r[5] or ""
"        read_at": r[6] or ""
{        for r in cur.fetchall():

def table_count(table_name):
    cur.execute(f"SELECT COUNT(*) FROM {table_name}")
    return int(cur.fetchone()[0] or 0)

def grouped_counts(table_name, column_name):
    cur.execute(f"SELECT COALESCE({column_name}, ''), COUNT(*) FROM
{table_name} GROUP BY {column_name}")

    return {str(row[0] or "UNKNOWN"): int(row[1] or 0) for row in
cur.fetchall()}

db_path = Path("server_data/classify.db")
storage_root = Path("server_storage")
storage_file_count = 0
storage_size_bytes = 0
if storage_root.exists():
    for file_path in storage_root.rglob("*")
        if file_path.is_file():
            storage_file_count += 1
            storage_size_bytes += file_path.stat().st_size

activity[] =

for row in users:
    activity.append({"type": "User", "title": row["name"], "detail":
row["email"], "at": row["created_at"]})

for row in courses:

```

```

        activity.append({"type": "Course", "title": row["name"], "detail":
row["school_name"], "at": row["created_at"]})

    for row in assignments:

        activity.append({"type": "Assignment", "title": row["title"], "detail":
row["course_name"], "at": row["created_at"]})

    for row in submissions:

        activity.append({"type": "Submission", "title": row["student_name"],
"detail": row["assignment_title"], "at": row["submitted_at"]})

    for row in materials:

        activity.append({"type": "Material", "title": row["title"], "detail":
row["course_name"], "at": row["created_at"]})

    activity = sorted([item for item in activity if item.get("at")], key=lambda item:
item["at"], reverse=True)[60:]

```

```

stats} =

"    counts} :."

"        schools": table_count("schools"),
"        users": table_count("users"),
"        courses": table_count("courses"),
"        course_members": table_count("course_members"),
"        assignments": table_count("assignments"),
"        submissions": table_count("submissions"),
"        ai_results": table_count("ai_results"),
"        grades": table_count("grades"),
"        feedback": table_count("feedback"),
"        materials": table_count("materials"),
"        messages": table_count("messages"),
"        message_reads": table_count("message_reads"),
"        grade_reads": table_count("grade_reads")

,{

```

```

"        users_by_role": grouped_counts("users", "role"),
"        users_by_status": grouped_counts("users", "status"),
"        submissions_by_status": grouped_counts("submissions", "status"),
"        system} :."
"        db_size_bytes": db_path.stat().st_size if db_path.exists() else 0,
"        storage_file_count": storage_file_count,
"        storage_size_bytes": storage_size_bytes
{
{

    system_rows] =

"}        metric": "Database file", "value": str(stats["system"]["db_size_bytes"]),
"unit": "bytes", "source": str(db_path),{

"}        metric": "Storage files", "value": str(storage_file_count), "unit": "files",
"source": str(storage_root),{

"}        metric": "Storage size", "value": str(storage_size_bytes), "unit": "bytes",
"source": str(storage_root){

[

    return True} ,

"        schools": schools,
"        users": users,
"        courses": courses,
"        members": members,
"        assignments": assignments,
"        submissions": submissions,
"        ai_results": ai_results,
"        grades": grades,
"        feedback": feedback,

```

```

"    materials": materials,
"    messages": messages,
"    message_reads": message_reads,
"    grade_reads": grade_reads,
"    activity": activity,
"    system": system_rows,
"    stats": stats
{
except sqlite3.Error as e:
    return False, str(e)
finally:
    conn.close()

```

```

def admin_create_user(name, email, password_hash, role, school_id,
status="ACTIVE"):
    conn = _conn()
    try:
        role = (role or "").upper().strip()
        status = (status or "ACTIVE").upper().strip()
        if role not in {"ADMIN", "MANAGER", "TEACHER", "STUDENT"}:
            return False, "INVALID_ROLE"
        if status not in {"ACTIVE", "PENDING", "BLOCKED"}:
            return False, "INVALID_STATUS"
        if role != "ADMIN" and not school_exists(int(school_id)):
            return False, "SCHOOL_NOT_FOUND"
        conn.execute("INSERT INTO users (name, email, password_hash, role,
schoolID, status, created_at) VALUES, "(? ,? ,? ,? ,? ,? ,? )

```



```

        (name, email.strip().lower(), password_hash, role, int(school_id),
status, datetime.now().isoformat(timespec="seconds"))(
    conn.commit()

    return True, "USER_CREATED"
except sqlite3.IntegrityError:
    return False, "USER_ALREADY_EXISTS"
except sqlite3.Error as e:
    return False, str(e)
finally:
    conn.close()

```

```

def admin_update_user(user_id, name, email, role, school_id, status):
    conn = _conn()
    try:
        role = (role or "").upper().strip()
        status = (status or "").upper().strip()
        if role not in {"ADMIN", "MANAGER", "TEACHER", "STUDENT"}:
            return False, "INVALID_ROLE"
        if status not in {"ACTIVE", "PENDING", "BLOCKED"}:
            return False, "INVALID_STATUS"
        if role != "ADMIN" and not school_exists(int(school_id)):
            return False, "SCHOOL_NOT_FOUND"
        cur = conn.cursor()
        cur.execute("UPDATE users SET name=?, email=?, role=?, schoolID=?,
status=? WHERE id=?", (name, email.strip().lower(), role, int(school_id), status,
int(user_id)))
        conn.commit()
        if cur.rowcount == 0:

```

```

        return False, "USER_NOT_FOUND"

    return True, "USER_UPDATED"

except sqlite3.IntegrityError:

    return False, "USER_ALREADY_EXISTS"

except sqlite3.Error as e:

    return False, str(e)

finally:

    conn.close()


def admin_delete_school(school_id):

    conn = _conn()

    try:

        cur = conn.cursor()

        sid = int(school_id)

        cur.execute("SELECT course_id FROM courses WHERE schoolID=?", (sid,))

        course_ids = [int(r[0]) for r in cur.fetchall()]

        for cid in course_ids:

            delete_course(cid)

        cur.execute("DELETE FROM users WHERE schoolID=?", (sid,))

        cur.execute("DELETE FROM schools WHERE school_id=?", (sid,))

        conn.commit()

        if cur.rowcount == 0:

            return False, "SCHOOL_NOT_FOUND"

        return True, "SCHOOL_DELETED"

    except sqlite3.Error as e:

        conn.rollback()

        return False, str(e)

```

```

finally:
    conn.close()

def admin_update_course(course_id, name, description, teacher_id, school_id):
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("SELECT role, schoolID FROM users WHERE id=?",
(int(teacher_id),))
        row = cur.fetchone()
        if not row or (row[0] or "").upper() not in {"TEACHER", "ADMIN"}:
            return False, "TEACHER_NOT_FOUND"
        cur.execute("UPDATE courses SET name=?, description=?, teacher_id=?,
schoolID=? WHERE course_id=?", (name, description, int(teacher_id),
int(school_id), int(course_id)))
        conn.commit()
        if cur.rowcount == 0:
            return False, "COURSE_NOT_FOUND"
        return True, "COURSE_UPDATED"
    except sqlite3.Error as e:
        return False, str(e)
    finally:
        conn.close()

def admin_update_assignment(assignment_id, title, description, due_date,
ai_enabled, is_closed):
    conn = _conn()

```

```

try:
    cur = conn.cursor()

    cur.execute("UPDATE assignments SET title=?, description=?, due_date=?,
ai_enabled=?, is_closed=? WHERE assignment_id, "?"=

                (title, description, _normalize_due_date_text(due_date), 1 if
ai_enabled else 0, 1 if is_closed else 0, int(assignment_id))(

    conn.commit()

    if cur.rowcount == 0:

        return False, "ASSIGNMENT_NOT_FOUND"

    return True, "ASSIGNMENT_UPDATED"

except sqlite3.Error as e:

    return False, str(e)

finally:

    conn.close()

```

db.py

```

import sqlite3
import random
import string
import re

from datetime import datetime
from pathlib import Path

```

```

--- #Added: course/class code helpers---

def _generate_course_code(length: int = 6) -> str:

    alphabet = string.ascii_uppercase + string.digits

    return ''.join(random.choices(alphabet, k=length))

```

```

def _generate_unique_course_code(conn) -> str:
    cur = conn.cursor()
    while True:
        code = _generate_course_code(6)
        cur.execute("SELECT 1 FROM courses WHERE class_code = ? LIMIT 1",
        (code,))
        if cur.fetchone() is None:
            return code

def _ensure_course_codes(conn):
    cur = conn.cursor()
    cur.execute("PRAGMA table_info(courses)")
    course_cols = [row[1] for row in cur.fetchall()]
    if "class_code" not in course_cols:
        conn.execute("ALTER TABLE courses ADD COLUMN class_code TEXT")

    conn.execute("CREATE UNIQUE INDEX IF NOT EXISTS
idx_courses_class_code_unique ON courses(class_code)")

    cur.execute("SELECT course_id FROM courses WHERE class_code IS NULL
OR TRIM(class_code) = '')")
    missing_rows = cur.fetchall()
    for (course_id,) in missing_rows:
        code = _generate_unique_course_code(conn)
        conn.execute("UPDATE courses SET class_code = ? WHERE course_id = ?",
        (code, course_id))

```

```

def _conn:()

    conn = sqlite3.connect("server_data/classify.db")

    conn.execute("""
        CREATE TABLE IF NOT EXISTS schools)
            school_id  INTEGER PRIMARY KEY AUTOINCREMENT,
            name      TEXT NOT NULL,
            address   TEXT,
            contact_name TEXT,
            contact_email TEXT,
            created_at TEXT NOT NULL
    (
    (""

    conn.execute("""

        CREATE TABLE IF NOT EXISTS users)
            id      INTEGER PRIMARY KEY AUTOINCREMENT,
            name     TEXT NOT NULL,
            email    TEXT UNIQUE NOT NULL,
            password_hash TEXT NOT NULL,
            role     TEXT NOT NULL,
            schoolID INTEGER NOT NULL,
            status   TEXT NOT NULL,
            created_at TEXT NOT NULL
    (
    (""

    conn.execute("""

```

```

CREATE TABLE IF NOT EXISTS courses)

    course_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    description TEXT,
    teacher_id INTEGER NOT NULL,
    schoolID INTEGER NOT NULL,
    created_at TEXT NOT NULL

(
(''

conn.execute('')

CREATE TABLE IF NOT EXISTS course_members)

    id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_id INTEGER NOT NULL,
    student_id INTEGER NOT NULL,
    joined_at TEXT NOT NULL,
    member_status TEXT NOT NULL DEFAULT 'ACTIVE,'
    UNIQUE(course_id, student_id)

(
(''

conn.execute('')

CREATE TABLE IF NOT EXISTS assignments)

    assignment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_id INTEGER NOT NULL,
    title TEXT NOT NULL,
    description TEXT,
    due_date TEXT,

```

```

        attachment_path TEXT,
        ai_enabled    INTEGER NOT NULL DEFAULT 1,
        is_closed    INTEGER NOT NULL DEFAULT 0,
        created_at    TEXT NOT NULL
    (
    (
conn.execute("""

CREATE TABLE IF NOT EXISTS submissions)

        submission_id INTEGER PRIMARY KEY AUTOINCREMENT,
        assignment_id INTEGER NOT NULL,
        student_id    INTEGER NOT NULL,
        file_path     TEXT NOT NULL,
        submitted_at  TEXT NOT NULL,
        status        TEXT NOT NULL DEFAULT 'PENDING_AI'
    (
    (
conn.execute("""

CREATE TABLE IF NOT EXISTS ai_results)

        ai_result_id INTEGER PRIMARY KEY AUTOINCREMENT,
        submission_id INTEGER NOT NULL,
        engine_name   TEXT NOT NULL,
        score         REAL NOT NULL,
        feedback      TEXT
    (
    (

```



```

conn.execute("""
CREATE TABLE IF NOT EXISTS grades)
    grade_id    INTEGER PRIMARY KEY AUTOINCREMENT,
    submission_id INTEGER NOT NULL UNIQUE,
    ai_score    REAL,
    final_score REAL,
    updated_by  INTEGER,
    updated_at  TEXT
(
("""

conn.execute("""
CREATE TABLE IF NOT EXISTS feedback)
    feedback_id    INTEGER PRIMARY KEY AUTOINCREMENT,
    submission_id  INTEGER NOT NULL UNIQUE,
    ai_feedback    TEXT,
    teacher_feedback TEXT
(
("""

conn.execute("""
CREATE TABLE IF NOT EXISTS materials)
    material_id  INTEGER PRIMARY KEY AUTOINCREMENT,
    course_id    INTEGER NOT NULL,
    title        TEXT NOT NULL,
    description   TEXT,
    type         TEXT,
    created_by   INTEGER,

```

```

        file_path TEXT,
        created_at TEXT NOT NULL
    (
    ("""

conn.execute("""
    CREATE TABLE IF NOT EXISTS messages)
        message_id INTEGER PRIMARY KEY AUTOINCREMENT,
        course_id INTEGER NOT NULL,
        sender_id INTEGER,
        subject TEXT NOT NULL,
        body TEXT NOT NULL,
        state TEXT NOT NULL,
        created_at TEXT NOT NULL
    (
    ("""

# Lightweight migrations for older databases
cur = conn.cursor()
_ ensure_course_codes(conn)
cur.execute("PRAGMA table_info(course_members)")
course_member_cols = [row[1] for row in cur.fetchall()]
if "member_status" not in course_member_cols:
    conn.execute("ALTER TABLE course_members ADD COLUMN
member_status TEXT NOT NULL DEFAULT 'ACTIVE'")
cur.execute("PRAGMA table_info(assignments)")
assignment_cols = [row[1] for row in cur.fetchall()]
if "is_closed" not in assignment_cols:

```

```
conn.execute("ALTER TABLE assignments ADD COLUMN is_closed
INTEGER NOT NULL DEFAULT 0")
```

```
if "ai_enabled" not in assignment_cols:
```

```
conn.execute("ALTER TABLE assignments ADD COLUMN ai_enabled
INTEGER NOT NULL DEFAULT 1")
```

```
conn.execute("""
```

```
CREATE TABLE IF NOT EXISTS message_reads)
```

```
id    INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
message_id INTEGER NOT NULL,
```

```
student_id INTEGER NOT NULL,
```

```
read_at  TEXT NOT NULL,
```

```
UNIQUE(message_id, student_id)
```

```
(
```

```
("""
```

```
conn.execute("""
```

```
CREATE TABLE IF NOT EXISTS grade_reads)
```

```
id    INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
submission_id INTEGER NOT NULL,
```

```
student_id  INTEGER NOT NULL,
```

```
read_at    TEXT NOT NULL,
```

```
UNIQUE(submission_id, student_id)
```

```
(
```

```
("""
```

```
conn.commit()
```

```
return conn
```

```

def _normalize_due_date_text(value):
    text = (value or "").strip()

    if not text:
        return ""

    text = text.replace("T", " ")
    text = re.sub(r"\s+", " ", text).strip()

    m = re.match(r"^\d{4}-\d{2}-\d{2}$", text)
    if m:
        return f"{m.group(1)}-{m.group(2)}-{m.group(3)} 00:00"

    m = re.match(r"^\d{4}-\d{2}-\d{2} (\d{2}):(\d{2})(?:\d{2})?$", text)
    if m:
        return f"{m.group(1)}-{m.group(2)}-{m.group(3)} {m.group(4)}:{m.group(5)}"

    return text


def _parse_due_datetime(value):
    normalized = _normalize_due_date_text(value)

    if not normalized:
        return None

    for fmt in ("%Y-%m-%d %H:%M", "%Y-%m-%d %H:%M:%S", "%Y-%m-%d"):
        try:
            return datetime.strptime(normalized, fmt)

```

```

        except ValueError:

            continue

    return None


def close_expired_assignments(course_id=None, assignment_id=None,
conn=None):

    own_conn = conn is None

    conn = conn or _conn()

    try:

        cur = conn.cursor()

        sql = "SELECT assignment_id, due_date, is_closed FROM assignments
WHERE 1=1"

        params[] =

        if course_id is not None:

            sql += " AND course_id"=?=

            params.append(int(course_id))

        if assignment_id is not None:

            sql += " AND assignment_id"=?=

            params.append(int(assignment_id))


        cur.execute(sql, params)

        rows = cur.fetchall()

        now = datetime.now()

        changed = False


        for row in rows:

            current_assignment_id = int(row[0])

            due_dt = _parse_due_datetime(row[1])

```

```

        is_closed = int(row[2] or 0)

        if due_dt is not None and due_dt <= now and is_closed == 0:

            cur.execute("UPDATE assignments SET is_closed=1 WHERE
assignment_id=?", (current_assignment_id,))

            changed = True

    if changed:

        conn.commit()

    return True

finally:

    if own_conn:

        conn.close()

def _assignment_row_to_dict(row):

    return{

        "assignment_id": int(row[0]),
        "course_id": int(row[1]),
        "title": row,[2]
        "description": row,[3]
        "due_date": _normalize_due_date_text(row[4]),
        "attachment_path": row,[5]
        "ai_enabled": row,[6]
        "is_closed": row,[7]
        "created_at": row[8]
    }

```

```

#SCHOOLS
#

def create_school(name: str, address=None, contact_name=None,
contact_email=None):
    (שגיאה) False או True, school_id) יצירת בית ספר חדש. מחזיר )
    conn = _conn()
    try:
        created_at = datetime.now().isoformat(timespec="seconds")
        cur = conn.cursor()
        cur.execute(
            "INSERT INTO schools (name, address, contact_name, contact_email,
            created_at) VALUES, "(? , ? , ? , ? , ?)"
        )
        name, address, contact_name, contact_email, created_at(
        (
            conn.commit()
            return True, cur.lastrowid
        except sqlite3.Error as e:
            return False, str(e)
        finally:
            conn.close()

def update_school(school_id: int, name=None, address=None,
contact_name=None, contact_email=None):
    מתעדכנים. None) עדכון פרטי בית ספר. רק שדות שאינם
    conn = _conn()

```

```

try:
    cur = conn.cursor()
    fields, values[] ,[] =
    if name is not None:
        fields.append("name = ?");    values.append(name)
    if address is not None:
        fields.append("address = ?");    values.append(address)
    if contact_name is not None:
        fields.append("contact_name = ?"); values.append(contact_name)
    if contact_email is not None:
        fields.append("contact_email = ?"); values.append(contact_email)
    if not fields:
        return False, "NO_FIELDS_TO_UPDATE"
    values.append(int(school_id))
    cur.execute(f"UPDATE schools SET {' '.join(fields)} WHERE school_id = ?",
values)
    conn.commit()
    if cur.rowcount == 0:
        return False, "SCHOOL_NOT_FOUND"
    return True, "SCHOOL_UPDATED"
finally:
    conn.close()

```

```

def get_school(school_id: int):
    , שגיאה) (False) או (True, dict). מחזיר ID ""שליפת פרטי בית ספר לפי
    conn = _conn()
    try:

```



```

        cur = conn.cursor()

        cur.execute)

"        SELECT school_id, name, address, contact_name, contact_email FROM
schools WHERE school_id, "?" =
)        school_id(,
(
        row = cur.fetchone()

        if not row:

            return False, "SCHOOL_NOT_FOUND"

        return True, {"school_id": row[0], "name": row[1], "address": row[2],[
"                contact_name": row[3], "contact_email": row{[4]

        finally:

            conn.close()

```

```

def school_exists(school_id: int):
    """בדיקה מהירה האם מזהה בית ספר קיים."""
    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute("SELECT 1 FROM schools WHERE school_id = ? LIMIT 1",
(int(school_id),))

        return cur.fetchone() is not None

    finally:

        conn.close()

```

```

def get_all_schools:()
    """שליפת כל בתי הספר (לשימוש אדמין). מחזיר (
    True, list""".(

```

```

conn = _conn()

try:
    cur = conn.cursor()

    cur.execute("SELECT school_id, name, address, contact_name,
contact_email FROM schools")

    rows = cur.fetchall()

    schools = [{"school_id": r[0], "name": r[1], "address": r[2],[
"
        contact_name": r[3], "contact_email": r {[4]for r in rows[

    return True, schools

finally:
    conn.close()

```

```

#
#USERS

```

```

def signup_user(name, email: str, password_hash: str, role: str, school_id):
    """רישום משתמש חדש. מנהל יוצר בית ספר אוטומטית."""

    conn = _conn()

    email = email.strip().lower()

    created_at = datetime.now().isoformat(timespec="seconds")

    stat = "ACTIVE" if role == "MANAGER" else "PENDING"

    school_id_int = None

    if school_id is not None:
        s = str(school_id).strip()

        if s:

```

```

try:
    school_id_int = int(s)
except ValueError:
    return False, "INVALID_SCHOOL_ID"

try:
    if role == "MANAGER:"
        cur = conn.cursor()
        cur.execute(
            "          INSERT INTO schools (name, address, contact_name, contact_email,
created_at) VALUES, "(? ,? ,? ,? ,?)
")          New School", None, name, email, created_at(
(
    school_id_int = cur.lastrowid
    cur.execute("UPDATE schools SET name=? WHERE school_id, "?=
        (f"School #{school_id_int}", school_id_int)(

    if school_id_int is None:
        return False, "SCHOOL_ID_REQUIRED"

    if not school_exists(school_id_int):
        return False, "SCHOOL_NOT_FOUND"

    conn.execute()

    "          INSERT INTO users (name, email, password_hash, role, schoolID,
status, created_at) VALUES, "(? ,? ,? ,? ,? ,? ,?)
)          name, email, password_hash, role, school_id_int, stat, created_at(
(
    conn.commit()

```

```

        return True, "USER_CREATED"

except sqlite3.IntegrityError:

    return False, "USER_ALREADY_EXISTS"

finally:

    conn.close()


def login_user(email: str, password_hash: str):

    ) התחברות. מחזיר ( True, role, user_id, name, school_id, school_name,
    school_address, school_contact_email".(

    conn = _conn()

    email = email.strip().lower()

    try:

        cur = conn.cursor()

        cur.execute)

"        SELECT id, role, status, name, schoolID FROM users WHERE email=?
AND password_hash,"?=

)        email, password_hash(

(

        row = cur.fetchone()

        if not row:

            return False, "INVALID_CREDENTIALS", -1, "", -1"" , "" , "" ,

        user_id, role, status, name, school_id = row

        name = name.split[0](" ")

        if status == "PENDING:"

            return False, "WAITING_APPROVAL", -1, "", -1"" , "" , "" ,

```

```

if status == "BLOCKED:"

    return False, "USER_BLOCKED", -1, "", -1 "", "", "",

    school_name = school_address = school_contact_email "" =

    cur.execute("SELECT name, address, contact_email FROM schools WHERE
school_id = ?", (school_id,))

    srow = cur.fetchone()

    if srow:

        school_name      = srow[0] or ""
        school_address    = srow[1] or ""
        school_contact_email = srow[2] or ""

    return True, role, user_id, name, school_id, school_name, school_address,
school_contact_email

finally:

    conn.close()

```

```

def get_user_context(user_id: int):

    ושם של משתמש לפי מזהה. "" role, school_id, status "" מחזיר

    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute("SELECT id, role, schoolID, status, name, email FROM users
WHERE id=?", (int(user_id),))

        row = cur.fetchone()

        if not row:

            return False, "USER_NOT_FOUND"

    return True} ,

```

```

"        user_id": int(row[0]),
"        role": row[1] or "",
"        school_id": int(row[2]),
"        status": row[3] or "",
"        name": row[4] or "",
"        email": row[5] or "",

```

```

{

```

```

    finally:

```

```

        conn.close()

```

```

def is_teacher_of_course(teacher_id: int, course_id: int):

```

```

    conn = _conn()

```

```

    try:

```

```

        cur = conn.cursor()

```

```

        cur.execute("SELECT 1 FROM courses WHERE course_id=? AND
teacher_id=?", (int(course_id), int(teacher_id)))

```

```

        return cur.fetchone() is not None

```

```

    finally:

```

```

        conn.close()

```

```

def is_course_in_school(course_id: int, school_id: int):

```

```

    conn = _conn()

```

```

    try:

```

```

        cur = conn.cursor()

```

```

        cur.execute("SELECT 1 FROM courses WHERE course_id=? AND
schoolID=?", (int(course_id), int(school_id)))

```

```

        return cur.fetchone() is not None

```

```

finally:
    conn.close()

def get_assignment_course_id(assignment_id: int):
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("SELECT course_id FROM assignments WHERE
assignment_id=?", (int(assignment_id),))
        row = cur.fetchone()
        if not row:
            return False, "ASSIGNMENT_NOT_FOUND"
        return True, int(row[0])
    finally:
        conn.close()

def get_submission_course_id(submission_id: int):
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("""
            SELECT a.course_id
            FROM submissions s
            JOIN assignments a ON s.assignment_id = a.assignment_id
            WHERE s.submission_id=?
, """)
        (int(submission_id),)(

```

```

        row = cur.fetchone()

        if not row:

            return False, "SUBMISSION_NOT_FOUND"

        return True, int(row[0])

    finally:

        conn.close()


def get_message_course_id(message_id: int):

    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute("SELECT course_id FROM messages WHERE message_id=?",
(int(message_id),))

        row = cur.fetchone()

        if not row:

            return False, "MESSAGE_NOT_FOUND"

        return True, int(row[0])

    finally:

        conn.close()


def get_material_course_id(material_id: int):

    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute("SELECT course_id FROM materials WHERE material_id=?",
(int(material_id),))

        row = cur.fetchone()

```



```

    if not row:

        return False, "MATERIAL_NOT_FOUND"

    return True, int(row[0])

finally:

    conn.close()

```

```

def can_user_access_course(user_id: int, role: str, school_id: int, course_id: int):

    role = (role or "").upper()

    if role == "ADMIN:":

        return True

    if role == "TEACHER:":

        return is_teacher_of_course(user_id, course_id)

    if role == "MANAGER:":

        return is_course_in_school(course_id, school_id)

    if role == "STUDENT:":

        return is_student_active_in_course(course_id, user_id)

    return False

```

```

def can_user_access_file(user_id: int, role: str, school_id: int, file_path: str):

    """בדיקת הרשאה כללית לקובץ לפי הנתביב השמור בשרת."""

    path_text = str(file_path or "")

    course_id = None

    match = re.search(r"course_(\d+)", path_text)

    if match:

        course_id = int(match.group(1))

```

```

if course_id is None:
    return False

return can_user_access_course(user_id, role, school_id, course_id)

def get_school_users(school_id: int):
    """כל המשתמשים של בית ספר. מחזיר (True, list)"""
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute(
            "SELECT id, name, email, role, status FROM users WHERE schoolID, '?' = "
            "school_id, "
            "
            users = [{"id": r[0], "name": r[1], "email": r[2], "role": r[3], "status": r[4]}
                    for r in cur.fetchall()]
        return True, users
    except sqlite3.Error as e:
        return False, str(e)
    finally:
        conn.close()

def approve_user(user_id: int):
    """אישור משתמש ממתין PENDING → ACTIVE"""
    conn = _conn()

```

```

try:
    cur = conn.cursor()
    cur.execute("UPDATE users SET status=? WHERE id=? AND status,"?=
                ("ACTIVE", user_id, "PENDING")(
    conn.commit()
    if cur.rowcount == 0:
        return False, "USER_NOT_FOUND_OR_ALREADY_ACTIVE"
    return True, "USER_APPROVED"
finally:
    conn.close()

```

```

def block_user(user_id: int):
) חסימת משתמש פעיל" "" ACTIVE → BLOCKED"".(
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("UPDATE users SET status=? WHERE id=? AND status,"?=
                    ("BLOCKED", user_id, "ACTIVE")(
        conn.commit()
        if cur.rowcount == 0:
            return False, "USER_NOT_FOUND_OR_ALREADY_BLOCKED"
        return True, "USER_BLOCKED"
    finally:
        conn.close()

```

```

def unblock_user(user_id: int):

```

```

) """ביטול חסימת משתמש""" BLOCKED → ACTIVE""".(
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("UPDATE users SET status=? WHERE id=? AND status, "=?=
            ("ACTIVE", user_id, "BLOCKED")
        conn.commit()
        if cur.rowcount == 0:
            return False, "USER_NOT_FOUND_OR_ALREADY_UNBLOCKED"
        return True, "USER_UNBLOCKED"
    finally:
        conn.close()

```

```

def delete_user(user_id: int):
    """מחיקת משתמש לחלוטין מהמערכת."""
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("DELETE FROM users WHERE id=?", (user_id,))
        conn.commit()
        if cur.rowcount == 0:
            return False, "USER_NOT_FOUND"
        return True, "USER_DELETED"
    finally:
        conn.close()

```

```

def get_pending_users:()
) כל המשתמשים הממתינים לאישור ( id, email, role"").(
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("SELECT id, email, role FROM users WHERE status=?",
("PENDING",))
        return cur.fetchall()
    finally:
        conn.close()

_____ #
#COURSES
_____ #

def create_course(name: str, description: str, teacher_id: int, school_id: int):
) שגיאה""). (False"") או (True, course_id) יצירת קורס חדש. מחזיר (
    conn = _conn()
    try:
        created_at = datetime.now().isoformat(timespec="seconds")
        class_code = _generate_unique_course_code(conn) # --- Added: real 6-
char join code---
        cur = conn.cursor()
        cur.execute)
"      INSERT INTO courses (name, description, teacher_id, schoolID,
created_at, class_code) VALUES,"(? ,? ,? ,? ,? ,?)
)      name, description, teacher_id, school_id, created_at, class_code(
(

```

```

        conn.commit()

        return True, cur.lastrowid

except sqlite3.Error as e:

    return False, str(e)

finally:

    conn.close()


def get_courses_for_teacher(teacher_id: int):
    """כל הקורסים של מורה מסוים."""
    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute(

            "
                SELECT course_id, name, description, schoolID, created_at, class_code
            FROM courses WHERE teacher_id, " +=

            teacher_id,

            (

                courses = [{"course_id": r[0], "name": r[1], "description": r[2], [

            "
                school_id": r[3], "created_at": r[4], "class_code": r[5] or ""} for r in
            cur.fetchall()]

            return True, courses

        finally:

            conn.close()


def get_courses_for_student(student_id: int):
    """כל הקורסים הפעילים שתלמיד שויך אליהם."""
    conn = _conn()

```

```

try:
    cur = conn.cursor()
    cur.execute("""
        SELECT c.course_id, c.name, c.description, c.teacher_id, c.created_at,
c.class_code
        FROM courses c
        JOIN course_members cm ON c.course_id = cm.course_id
        WHERE cm.student_id = ? AND COALESCE(cm.member_status,
'ACTIVE') = 'ACTIVE'
, """)
    (student_id,)(
        courses = [{"course_id": r[0], "name": r[1], "description": r[2], [
"
        teacher_id": r[3], "created_at": r[4], "class_code": r[5] or ""} for r in
cur.fetchall()]()
        return True, courses
finally:
    conn.close()

```

```

def get_courses_for_school(school_id: int):
    """כל הקורסים של בית ספר (לשימוש מנהל)."""
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("""
            SELECT c.course_id, c.name, c.description, c.teacher_id, c.created_at,
            COUNT(CASE WHEN COALESCE(cm.member_status, 'ACTIVE') =
'ACTIVE' THEN cm.student_id END) as student_count,
            u.name as teacher_name
            FROM courses c
            LEFT JOIN course_members cm ON c.course_id = cm.course_id

```

```

        LEFT JOIN users u ON c.teacher_id = u.id

        WHERE c.schoolID? =

        GROUP BY c.course_id

, "" (school_id,)(
    courses = [{"course_id": r[0], "name": r[1], "description": r[2],[
"        teacher_id": r[3], "created_at": r[4], "students": r,[5]
"        teacher_name": r[6] or "Unknown{"
        for r in cur.fetchall()

    return True, courses

finally:

    conn.close()

```

```

def get_course_teacher(course_id: int):
    ""מחזיר את מזהה המורה של קורס מסוים.""
    conn = _conn()

    try:
        cur = conn.cursor()
        cur.execute("SELECT teacher_id FROM courses WHERE course_id=?",
(course_id,))

        row = cur.fetchone()

        if not row:
            return False, "COURSE_NOT_FOUND"

        return True, row[0]

    finally:
        conn.close()

```



```

def get_course_school(course_id: int):
    """מחזיר את מזהה בית הספר של קורס מסוים."""
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("SELECT schoolID FROM courses WHERE course_id=?",
(course_id,))
        row = cur.fetchone()
        if not row:
            return False, "COURSE_NOT_FOUND"
        return True, row[0]
    finally:
        conn.close()

def get_course_file_paths(course_id: int):
    """כל נתיבי הקבצים המשויכים לקורס."""
    conn = _conn()
    try:
        cur = conn.cursor()
        paths[] =

        cur.execute("SELECT attachment_path FROM assignments WHERE
course_id=? AND attachment_path IS NOT NULL AND attachment_path <> '',
(course_id,))

        paths.extend([row[0] for row in cur.fetchall() if row[0]])

```

```

        cur.execute("SELECT file_path FROM materials WHERE course_id=? AND
file_path IS NOT NULL AND file_path <> '', (course_id,))

        paths.extend([row[0] for row in cur.fetchall() if row[0]])

    cur.execute("""

        SELECT s.file_path

        FROM submissions s

        JOIN assignments a ON s.assignment_id = a.assignment_id

        WHERE a.course_id=? AND s.file_path IS NOT NULL AND s.file_path" <>
, """"      (course_id,)(

        paths.extend([row[0] for row in cur.fetchall() if row[0]])

    return True, paths

finally:

    conn.close()

```

```

def delete_course(course_id: int):

    """מחיקת קורס וכל המידע המשויך אליו."""

    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute("SELECT course_id FROM courses WHERE course_id=?",
(course_id,))

        if not cur.fetchone():

            return False, "COURSE_NOT_FOUND"

        cur.execute("SELECT assignment_id FROM assignments WHERE
course_id=?", (course_id,))

```

```

assignment_ids = [row[0] for row in cur.fetchall()]

submission_ids[] =

if assignment_ids:

    placeholders = ",".join(["?"] * len(assignment_ids))

    cur.execute(f"SELECT submission_id FROM submissions WHERE
assignment_id IN ({placeholders})", assignment_ids)

    submission_ids = [row[0] for row in cur.fetchall()]

if submission_ids:

    placeholders = ",".join(["?"] * len(submission_ids))

    cur.execute(f"DELETE FROM ai_results WHERE submission_id IN
({placeholders})", submission_ids)

    cur.execute(f"DELETE FROM grades WHERE submission_id IN
({placeholders})", submission_ids)

    cur.execute(f"DELETE FROM feedback WHERE submission_id IN
({placeholders})", submission_ids)

    cur.execute(f"DELETE FROM grade_reads WHERE submission_id IN
({placeholders})", submission_ids)

if assignment_ids:

    placeholders = ",".join(["?"] * len(assignment_ids))

    cur.execute(f"DELETE FROM submissions WHERE assignment_id IN
({placeholders})", assignment_ids)

    cur.execute(f"DELETE FROM assignments WHERE assignment_id IN
({placeholders})", assignment_ids)

    cur.execute("DELETE FROM message_reads WHERE message_id IN
(SELECT message_id FROM messages WHERE course_id=?)", (course_id,))

    cur.execute("DELETE FROM materials WHERE course_id=?", (course_id,))

    cur.execute("DELETE FROM messages WHERE course_id=?", (course_id,))

```

```

        cur.execute("DELETE FROM course_members WHERE course_id=?",
(course_id,))

        cur.execute("DELETE FROM courses WHERE course_id=?", (course_id,))

        conn.commit()

        return True, "COURSE_DELETED"

except sqlite3.Error as e:

    conn.rollback()

    return False, str(e)

finally:

    conn.close()


def get_course(course_id: int):
    ID""".שליפת פרטי קורס לפי
    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute)

"        SELECT course_id, name, description, teacher_id, schoolID, created_at
FROM courses WHERE course_id,"?="

)        course_id(,

(

    row = cur.fetchone()

    if not row:

        return False, "COURSE_NOT_FOUND"

    return True} ,

"        course_id": row,[0]

"        name": row,[1]

"        description": row,[2]

```

```

"        teacher_id": row,[3]
"        school_id": row,[4]
"        created_at": row[5]
{
    finally:
        conn.close()

```

```

#
#COURSE MEMBERS
#

```

```

def add_student_to_course(course_id: int, student_id: int, member_status: str =
"ACTIVE"):
    """שיוך תלמיד לקורס."""
    conn = _conn()
    try:
        joined_at = datetime.now().isoformat(timespec="seconds")
        conn.execute(
            "        INSERT OR IGNORE INTO course_members (course_id, student_id,
joined_at, member_status) VALUES, "(? ,? ,? ,?)
        )
        course_id, student_id, joined_at, str(member_status or
"ACTIVE").upper()
        (
            conn.commit()
            return True, "STUDENT_ADDED"
        except sqlite3.Error as e:
            return False, str(e)
    finally:

```

```
conn.close()
```

```
def remove_student_from_course(course_id: int, student_id: int):
```

```
    """הסרת תלמיד מקורס."""
```

```
    conn = _conn()
```

```
    try:
```

```
        cur = conn.cursor()
```

```
        cur.execute("DELETE FROM course_members WHERE course_id=? AND  
student_id,"?=
```

```
                    (course_id, student_id)(
```

```
        conn.commit()
```

```
        if cur.rowcount == 0:
```

```
            return False, "MEMBER_NOT_FOUND"
```

```
        return True, "STUDENT_REMOVED"
```

```
    finally:
```

```
        conn.close()
```

```
def delete_course_member(course_id: int, student_id: int):
```

```
    """מחיקת בקשת הצטרפות או תלמיד מקורס."""
```

```
    return remove_student_from_course(course_id, student_id)
```

```
def get_students_in_course(course_id: int):
```

```
    """כל התלמידים בקורס מסוים."""
```

```
    conn = _conn()
```

```
    try:
```

```
        cur = conn.cursor()
```

```

cur.execute("""
    SELECT u.id, u.name, u.email, COALESCE(cm.member_status,
'ACTIVE'), cm.joined_at
    FROM users u
    JOIN course_members cm ON u.id = cm.student_id
    WHERE cm.course_id? =
    ORDER BY u.name COLLATE NOCASE
, """)
    (course_id,)(
        students = [{"id": r[0], "name": r[1], "email": r[2], "member_status": r[3],
"joined_at": r[4] or ""} for r in cur.fetchall()]
        return True, students
    finally:
        conn.close()

def update_course_member_status(course_id: int, student_id: int,
member_status: str):
    ) עדיכון סטטוס תלמיד בתוך קורס ( ACTIVE / INACTIVE / PENDING """).(
        conn = _conn()
        try:
            cur = conn.cursor()
            cur.execute)
            "
            UPDATE course_members SET member_status=? WHERE course_id=?
            AND student_id, "?=
            )
            member_status, course_id, student_id(
            (
                conn.commit()
                if cur.rowcount == 0:
                    return False, "MEMBER_NOT_FOUND"

```

```

        return True, "MEMBER_STATUS_UPDATED"

    finally:
        conn.close()

def get_course_member_status(course_id: int, student_id: int):
    """מחזיר את סטטוס החברות של תלמיד בכיתה."""
    conn = _conn()

    try:
        cur = conn.cursor()
        cur.execute(
            "SELECT COALESCE(member_status, 'ACTIVE') FROM course_members\n"
            "WHERE course_id=? AND student_id,"?=
        )
        cur.execute(
            "SELECT COALESCE(member_status, 'ACTIVE') FROM course_members\n"
            "WHERE course_id=? AND student_id,"?=
        )
        row = cur.fetchone()

        if not row:
            return False, "MEMBER_NOT_FOUND"

        return True, row[0]

    finally:
        conn.close()

def is_student_active_in_course(course_id: int, student_id: int):
    ok_status, status = get_course_member_status(course_id, student_id)

    if not ok_status:
        return False

    return str(status).upper() == "ACTIVE"

```

```

#

#ASSIGNMENTS

#

def create_assignment(course_id: int, title: str, description: str, due_date: str,
attachment_path=None, ai_enabled: bool = True):
    """שגיאה) (False או (True, assignment_id) יצירת מטלה חדשה בקורס. מחזיר
    conn = _conn()

    try:

        created_at = datetime.now().isoformat(timespec="seconds")

        cur = conn.cursor()

        cur.execute)

        "      INSERT INTO assignments (course_id, title, description, due_date,
attachment_path, ai_enabled, is_closed, created_at) VALUES (?, ?, ?, ?, ?, ?)
        ,(?, ?

    )      course_id, title, description, due_date, attachment_path, 1 if ai_enabled
else 0, 0, created_at(

    (

        conn.commit()

        return True, cur.lastrowid

    except sqlite3.Error as e:

        return False, str(e)

    finally:

        conn.close()

def get_assignments_for_course(course_id: int):
    """כל המטלות של קורס מסוים.
    conn = _conn()

```

```

try:
    close_expired_assignments(course_id=course_id, conn=conn)

    cur = conn.cursor()

    cur.execute)

"        SELECT assignment_id, course_id, title, description, due_date,
attachment_path, ai_enabled, is_closed, created_at FROM assignments WHERE
course_id=? ORDER BY COALESCE(due_date, created_at), assignment_id,"
)        course_id(,
(
    assignments = [_assignment_row_to_dict(r) for r in cur.fetchall()]

    return True, assignments

finally:
    conn.close()

```

```

def get_assignment(assignment_id: int):
    ID"".שליפת מטלה בודדת לפי

    conn = _conn()

    try:

        close_expired_assignments(assignment_id=assignment_id, conn=conn)

        cur = conn.cursor()

        cur.execute)

"        SELECT assignment_id, course_id, title, description, due_date,
attachment_path, ai_enabled, is_closed, created_at FROM assignments WHERE
assignment_id,"?="
)        assignment_id(,
(
    row = cur.fetchone()

    if not row:

```

```

        return False, "ASSIGNMENT_NOT_FOUND"

    return True, _assignment_row_to_dict(row)

finally:

    conn.close()


def set_assignment_closed(assignment_id: int, is_closed: bool):
    """עדכון מצב פתוח/סגור של מטלה."""
    conn = _conn()

    try:

        close_expired_assignments(assignment_id=assignment_id, conn=conn)

        cur = conn.cursor()

        cur.execute(
            "
                SELECT is_closed, due_date FROM assignments WHERE
                assignment_id, "=?
            )
            assignment_id(,
            (

        row = cur.fetchone()

        if not row:

            return False, "ASSIGNMENT_NOT_FOUND"

        due_dt = _parse_due_datetime(row[1])

        if not bool(is_closed) and due_dt is not None and due_dt <=
datetime.now:()

            return False, "ASSIGNMENT_ALREADY_PAST_DUE"

        cur.execute("UPDATE assignments SET is_closed=? WHERE
assignment_id=?", (1 if is_closed else 0, assignment_id))

        conn.commit()

```

```

    if cur.rowcount == 0:

        return False, "ASSIGNMENT_NOT_FOUND"

    return True, "ASSIGNMENT_UPDATED"

finally:

    conn.close()


def delete_assignment(assignment_id: int):

    """מחיקת מטלה וכל המידע המשויך אליה."""

    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute("SELECT attachment_path FROM assignments WHERE
assignment_id=?", (assignment_id,))

        row = cur.fetchone()

        if not row:

            return False, "ASSIGNMENT_NOT_FOUND"

        attachment_path = row[0] or ""

        cur.execute("SELECT submission_id, file_path FROM submissions WHERE
assignment_id=?", (assignment_id,))

        submission_rows = cur.fetchall()

        submission_ids = [int(r[0]) for r in submission_rows]

        submission_paths = [r[1] for r in submission_rows if r[1]]

        if submission_ids:

            placeholders = ",".join(["?"] * len(submission_ids))

            cur.execute(f"DELETE FROM ai_results WHERE submission_id IN
({placeholders})", submission_ids)

```

```

        cur.execute(f"DELETE FROM grades WHERE submission_id IN
({placeholders})", submission_ids)

        cur.execute(f"DELETE FROM feedback WHERE submission_id IN
({placeholders})", submission_ids)

        cur.execute(f"DELETE FROM grade_reads WHERE submission_id IN
({placeholders})", submission_ids)


        cur.execute("DELETE FROM submissions WHERE assignment_id=?",
(assignments_id,))

        cur.execute("DELETE FROM assignments WHERE assignment_id=?",
(assignments_id,))

        conn.commit()

        return True, {"message": "ASSIGNMENT_DELETED", "attachment_path":
attachment_path, "submission_paths": submission_paths}

    except sqlite3.Error as e:

        conn.rollback()

        return False, str(e)

    finally:

        conn.close()

def get_assignments_for_school(school_id: int):
    """כל המשימות של בית ספר מסוים (לשימוש מנהל)."""
    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute("SELECT course_id FROM courses WHERE schoolID=?",
(school_id,))

        for (course_id,) in cur.fetchall():

            close_expired_assignments(course_id=int(course_id), conn=conn)

        cur.execute("""

```

```

SELECT a.assignment_id, a.title, a.description, a.due_date,
       c.name as course_name, u.name as teacher_name,
       COUNT(s.submission_id) as submission_count
FROM assignments a
JOIN courses c ON a.course_id = c.course_id
JOIN users u ON c.teacher_id = u.id
LEFT JOIN submissions s ON a.assignment_id = s.assignment_id
WHERE c.schoolID? =
GROUP BY a.assignment_id
ORDER BY COALESCE(a.due_date, a.created_at), a.assignment_id
, "" (school_id,)(
assignments] =
}
"    assignment_id": r,[0]
"    title":    r,[1]
"    description": r,[2]
"    due_date":  _normalize_due_date_text(r[3]),
"    course_name": r,[4]
"    teacher_name": r,[5]
"    submission_count": r[6]
{
    for r in cur.fetchall()
[
    return True, assignments
except sqlite3.Error as e:
    return False, str(e)
finally:
    conn.close()

```

```

#SUBMISSIONS

```

```

def save_submission(assignment_id: int, student_id: int, file_path: str, status: str
= "PENDING_AI"):

    """שמירת הגשה חדשה, תוך החלפת כל הגשה קודמת של אותו תלמיד לאותה
    מטלה."""

    conn = _conn()

    try:

        submitted_at = datetime.now().isoformat(timespec="seconds")

        cur = conn.cursor()

        cur.execute("""

            SELECT submission_id, file_path

            FROM submissions

            WHERE assignment_id=? AND student_id=?

            , """, (int(assignment_id), int(student_id))

        previous_rows = cur.fetchall()

        previous_ids = [int(r[0]) for r in previous_rows]

        previous_file_paths = [r[1] or "" for r in previous_rows if r[1]]

        if previous_ids:

            placeholders = ",".join(["?"] * len(previous_ids))

            cur.execute(f"DELETE FROM ai_results WHERE submission_id IN
({placeholders})", previous_ids)

            cur.execute(f"DELETE FROM grades WHERE submission_id IN
({placeholders})", previous_ids)

```

```

        cur.execute(f"DELETE FROM feedback WHERE submission_id IN
({placeholders})", previous_ids)

        cur.execute(f"DELETE FROM grade_reads WHERE submission_id IN
({placeholders})", previous_ids)

        cur.execute(f"DELETE FROM submissions WHERE submission_id IN
({placeholders})", previous_ids)

```

```

        cur.execute)

"        INSERT INTO submissions (assignment_id, student_id, file_path,
submitted_at, status) VALUES, "(? ,? ,? ,? ,?)
)        assignment_id, student_id, file_path, submitted_at, status(
(
        conn.commit()

        return True, {"submission_id": int(cur.lastrowid), "replaced_file_paths":
previous_file_paths}

except sqlite3.Error as e:

    conn.rollback()

    return False, str(e)

finally:

    conn.close()

```

```

def get_submissions_for_assignment(assignment_id: int):

```

```

    """כל ההגשות למטלה (לשימוש מורה)."""

```

```

    conn = _conn()

```

```

    try:

```

```

        cur = conn.cursor()

```

```

        cur.execute("""

```

```

            SELECT s.submission_id, s.student_id, u.name, s.file_path,
s.submitted_at, s.status,

```



```

        g.ai_score, g.final_score
    FROM submissions s
    JOIN users u ON s.student_id = u.id
    LEFT JOIN grades g ON s.submission_id = g.submission_id
    WHERE s.assignment_id? =
        AND s.submission_id) =
        SELECT MAX(s2.submission_id)
        FROM submissions s2
        WHERE s2.assignment_id = s.assignment_id AND s2.student_id =
s.student_id
(
    ORDER BY s.submitted_at DESC, s.submission_id DESC
, "" (assignment_id,)(
    submissions = [{"submission_id": r[0], "student_id": r[1], "student_name":
r[2],[
"        file_path": r[3], "submitted_at": r[4], "status": r,[5]
"        ai_score": r[6], "final_score": r{[7]
        for r in cur.fetchall()
    return True, submissions
finally:
    conn.close()

```

```

def get_submission(submission_id: int):

```

```

    """פרטי הגשה ספציפית."""

```

```

    conn = _conn()

```

```

    try:

```

```

        cur = conn.cursor()

```

```

        cur.execute)

```

```

"          SELECT submission_id, assignment_id, student_id, file_path,
submitted_at, status FROM submissions WHERE submission_id, "=?
)          submission_id(,
(
    row = cur.fetchone()
    if not row:
        return False, "SUBMISSION_NOT_FOUND"
    return True, {"submission_id": row[0], "assignment_id": row[1], "student_id":
row[2],[
"          file_path": row[3], "submitted_at": row[4], "status": row[[5]
    finally:
        conn.close()

```

```

def update_submission_status(submission_id: int, status: str):
) עדכון סטטוס הגשה "PENDING_AI / AI_DONE / ERROR".(
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("UPDATE submissions SET status=? WHERE submission_id=?",
(status, submission_id))
        conn.commit()
        if cur.rowcount == 0:
            return False, "SUBMISSION_NOT_FOUND"
        return True, "STATUS_UPDATED"
    finally:
        conn.close()

```

```

#AI RESULTS

```

```

def save_ai_result(submission_id: int, engine_name: str, score: float, feedback:
str):
    אחד. AI שמירת תוצאה ממנוע
    conn = _conn()
    try:
        conn.execute)
    "      INSERT INTO ai_results (submission_id, engine_name, score, feedback)
VALUES,(? ,? ,? ,?)
    )      submission_id, engine_name, score, feedback(
    (
        conn.commit()
        return True, "AI_RESULT_SAVED"
    except sqlite3.Error as e:
        return False, str(e)
    finally:
        conn.close()

def get_ai_results(submission_id: int):
    להגשה מסוימת. AI כל תוצאות ה-
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute)

```

```

"        SELECT engine_name, score, feedback FROM ai_results WHERE
submission_id,"?="
)        submission_id(,
(
    results = [{"engine": r[0], "score": r[1], "feedback": r[2]} for r in cur.fetchall()]
    return True, results
finally:
    conn.close()

```

```

#
#GRADES
#

```

```

def save_grade(submission_id: int, ai_score: float, final_score=None,
updated_by=None):
    ראשוני (ו/או ציון סופי) להגשה. AI "" "" שמירת ציון
    conn = _conn()
    try:
        updated_at = datetime.now().isoformat(timespec="seconds")
        conn.execute("""
            INSERT INTO grades (submission_id, ai_score, final_score, updated_by,
updated_at)
            VALUES(?, ?, ?, ?, ?)
            ON CONFLICT(submission_id) DO UPDATE SET
                ai_score = excluded.ai_score,
                final_score = excluded.final_score,
                updated_by = excluded.updated_by,
                updated_at = excluded.updated_at

```

```
, """      (submission_id, ai_score, final_score, updated_by, updated_at)(
    conn.commit()
    return True, "GRADE_SAVED"
except sqlite3.Error as e:
    return False, str(e)
finally:
    conn.close()
```

```
def update_final_grade(submission_id: int, final_score: float, updated_by: int):
```

```
    """עדכון ציון סופי על ידי מורה."""
```

```
    conn = _conn()
```

```
    try:
```

```
        updated_at = datetime.now().isoformat(timespec="seconds")
```

```
        cur = conn.cursor()
```

```
        cur.execute("""
```

```
            UPDATE grades SET final_score=?, updated_by=?, updated_at=?=
```

```
            WHERE submission_id=?
```

```
, """      (final_score, updated_by, updated_at, submission_id)(
```

```
    conn.commit()
```

```
    if cur.rowcount == 0:
```

```
        return False, "GRADE_NOT_FOUND"
```

```
    return True, "GRADE_UPDATED"
```

```
finally:
```

```
    conn.close()
```

```
def get_grade(submission_id: int):
```

שליפת ציון להגשה ספציפית.

```
conn = _conn()

try:
    cur = conn.cursor()
    cur.execute(
        "SELECT ai_score, final_score, updated_by, updated_at FROM grades
        WHERE submission_id, "=?=
    )
    submission_id(,
    (
        row = cur.fetchone()

        if not row:
            return False, "GRADE_NOT_FOUND"

        return True, {"ai_score": row[0], "final_score": row[1],[
        "
            updated_by": row[2], "updated_at": row{[3]

    finally:
        conn.close()
```

#

#FEEDBACK

#

```
def save_feedback(submission_id: int, ai_feedback=None,
teacher_feedback=None):
```

ומשוב מורה. AI שמירה/עדכון משוב

```
conn = _conn()

try:
    conn.execute("""
        INSERT INTO feedback (submission_id, ai_feedback, teacher_feedback)
```

```

VALUES(?, ?, ?)

ON CONFLICT(submission_id) DO UPDATE SET

    ai_feedback = COALESCE(excluded.ai_feedback, ai_feedback),

    teacher_feedback = COALESCE(excluded.teacher_feedback,
teacher_feedback)

, """      (submission_id, ai_feedback, teacher_feedback)(

    conn.commit()

    return True, "FEEDBACK_SAVED"

except sqlite3.Error as e:

    return False, str(e)

finally:

    conn.close()

```

```

def get_feedback(submission_id: int):

    """שליפת משוב להגשה ספציפית."""

    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute(

            "      SELECT ai_feedback, teacher_feedback FROM feedback WHERE
submission_id, " ?=

        )      submission_id(,

        (

            row = cur.fetchone()

            if not row:

                return False, "FEEDBACK_NOT_FOUND"

            return True, {"ai_feedback": row[0], "teacher_feedback": row[1]}

        finally:

```

```

conn.close()

#
#MATERIALS
#

def create_material(course_id: int, title: str, description: str = "", material_type:
str = "FILE", created_by=None, file_path=None):
    """יצירת חומר לימוד לקורס."""
    conn = _conn()
    try:
        created_at = datetime.now().isoformat(timespec="seconds")
        cur = conn.cursor()
        cur.execute(
            "INSERT INTO materials (course_id, title, description, type, created_by,
file_path, created_at) VALUES, "(? ,? ,? ,? ,? ,? ,? )"
            )
        course_id, title, description, material_type, created_by, file_path,
created_at(
        (
            conn.commit()
            return True, cur.lastrowid
        except sqlite3.Error as e:
            return False, str(e)
        finally:
            conn.close()

```



```

def get_materials_for_course(course_id: int):
    """כל חומרי הלימוד של קורס מסוים."""
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("""
            SELECT m.material_id, m.course_id, m.title, m.description, m.type,
                   m.created_by, m.file_path, m.created_at,
                   COALESCE(u.name, 'Teacher') as created_by_name
            FROM materials m
            LEFT JOIN users u ON m.created_by = u.id
            WHERE m.course_id=?
            ORDER BY m.created_at DESC
        """, (course_id,))
        materials = []
        for r in cur.fetchall():
            materials.append({
                "material_id": r[0],
                "course_id": r[1],
                "title": r[2],
                "description": r[3],
                "type": r[4],
                "created_by": r[5],
                "file_path": r[6],
                "created_at": r[7],
                "created_by_name": r[8]
            })
        return True, materials
    finally:
        conn.close()

```

```

def get_material(material_id: int):
    """שליפת חומר לימוד בודד."""
    conn = _conn()

    try:
        cur = conn.cursor()
        cur.execute("""
            SELECT material_id, course_id, title, description, type, created_by,
file_path, created_at
            FROM materials
            WHERE material_id=?
, """)
        row = cur.fetchone()
        if not row:
            return False, "MATERIAL_NOT_FOUND"
        return True, {
            "material_id": row[0]
            "course_id": row[1]
            "title": row[2]
            "description": row[3]
            "type": row[4]
            "created_by": row[5]
            "file_path": row[6]
            "created_at": row[7]
        }
    finally:
        conn.close()

```

```

def delete_material(material_id: int):
    """מחיקת חומר לימוד."""
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("SELECT file_path, course_id FROM materials WHERE
material_id=?", (material_id,))
        row = cur.fetchone()
        if not row:
            return False, "MATERIAL_NOT_FOUND"
        file_path = row[0] or ""
        course_id = int(row[1])
        cur.execute("DELETE FROM materials WHERE material_id=?",
(material_id,))
        conn.commit()
        return True, {"message": "MATERIAL_DELETED", "file_path": file_path,
"course_id": course_id}
    finally:
        conn.close()

```

#

#MESSAGES

#

```

def create_message(course_id: int, sender_id: int, subject: str, body: str, state:
str = "sent"):

```

"""שמירת הודעה/טיוטה לקורס."""

```
conn = _conn()
```

```
try:
```

```
    created_at = datetime.now().isoformat(timespec="seconds")
```

```
    cur = conn.cursor()
```

```
    cur.execute)
```

```
    "        INSERT INTO messages (course_id, sender_id, subject, body, state,
created_at) VALUES, "(? ,? ,? ,? ,? ,?)"
```

```
)        course_id, sender_id, subject, body, state, created_at(
```

```
(
```

```
    conn.commit()
```

```
    return True, cur.lastrowid
```

```
except sqlite3.Error as e:
```

```
    return False, str(e)
```

```
finally:
```

```
    conn.close()
```

```
def get_messages_for_course(course_id: int):
```

"""שליפת הודעות של קורס מסוים."""

```
conn = _conn()
```

```
try:
```

```
    cur = conn.cursor()
```

```
    cur.execute("""
```

```
        SELECT m.message_id, m.course_id, m.sender_id, m.subject, m.body,
m.state, m.created_at,
```

```
        COALESCE(u.name, 'Teacher') as sender_name
```

```
        FROM messages m
```

```
        LEFT JOIN users u ON m.sender_id = u.id
```

```

        WHERE m.course_id=?

        ORDER BY m.created_at DESC

, """
    (course_id,)(
        messages}] =
    "    message_id": r,[0]
    "    course_id": r,[1]
    "    sender_id": r,[2]
    "    subject": r,[3]
    "    body": r,[4]
    "    state": r,[5]
    "    created_at": r,[6]
    "    sender_name": r[7]
    {    for r in cur.fetchall()
        return True, messages
    finally:
        conn.close()

def delete_message(message_id: int):
    """מחיקת הודעה."""
    conn = _conn()

    try:
        cur = conn.cursor()

        cur.execute("SELECT course_id FROM messages WHERE message_id=?",
(message_id,))

        row = cur.fetchone()

        if not row:

            return False, "MESSAGE_NOT_FOUND"

```

```

        course_id = int(row[0])

        cur.execute("DELETE FROM message_reads WHERE message_id=?",
(message_id,))

        cur.execute("DELETE FROM messages WHERE message_id=?",
(message_id,))

        conn.commit()

        return True, {"message": "MESSAGE_DELETED", "course_id": course_id}

    finally:

        conn.close()

```

--- #Added: student dashboard / read-state helpers---

```

def get_student_dashboard(student_id: int):
    """מחזיר את כל הנתונים למסך תלמיד: קורסים, מטלות, חומרים והודעות."""
    ok_courses, courses = get_courses_for_student(student_id)

    if not ok_courses:

        return False, courses

    conn = _conn()

    try:

        cur = conn.cursor()

        course_ids = [int(c["course_id"]) for c in courses]

        class_list[] =

        for idx, course in enumerate(courses):

            teacher_name = "Teacher"

            try:

```

```

        cur.execute("SELECT name FROM users WHERE id=?",
(int(course.get("teacher_id", -1))),)

        row = cur.fetchone()

        if row and row[0]:

            teacher_name = row[0]

    except Exception:

        pass

    cur.execute("SELECT COUNT(*) FROM course_members WHERE
course_id=? AND COALESCE(member_status, 'ACTIVE')='ACTIVE'",
(int(course["course_id"]),))

    students_count = cur.fetchone[0]()

    palette =

#4")          F46E5", "#EEF2FF,("📅", "
#2563")          EB", "#DBEAFE,("📅", "
#"")          D97706", "#FEF3C7,("📅", "
#" ,"#059669")          D1FAE5,("📅", "
#7")          C3AED", "#EDE9FE,("📅", "
[

    accent, accent_soft, icon_text = palette[idx % len(palette)]

    class_list.append})

"        classId": int(course["course_id"]),
"        name": course.get("name", "Course"),
"        teacherName": teacher_name,
"        room,"—" : "
"        description": course.get("description") or, ""
"        studentsCount": students_count,
"        code": (course.get("class_code") or ""), # --- Modified: real stored
class code---

```

```

"        accent": accent,
"        accentSoft": accent_soft,
"        iconText": icon_text,
({

assignments[] =
materials[] =
messages[] =

if course_ids:
    placeholders = ",".join(["?"] * len(course_ids))

    for course_id in course_ids:
        close_expired_assignments(course_id=int(course_id), conn=conn)

    cur.execute(f"""
        SELECT a.assignment_id, a.course_id, a.title, a.description,
a.due_date, a.attachment_path, a.created_at, a.is_closed,
            s.submission_id, s.file_path, s.submitted_at,
            g.final_score,
            f.teacher_feedback,
            CASE WHEN gr.id IS NULL THEN 0 ELSE 1 END AS grade_read
        FROM assignments a
        LEFT JOIN submissions s ON s.submission_id) =
            SELECT s2.submission_id
            FROM submissions s2
            WHERE s2.assignment_id = a.assignment_id AND s2.student_id =
?

        ORDER BY s2.submission_id DESC

```



```

LIMIT 1

(
    LEFT JOIN grades g ON g.submission_id = s.submission_id
    LEFT JOIN feedback f ON f.submission_id = s.submission_id
    LEFT JOIN grade_reads gr ON gr.submission_id = s.submission_id AND
gr.student_id? =

    WHERE a.course_id IN ({placeholders})

    ORDER BY COALESCE(a.due_date, a.created_at), a.assignment_id
, ""
    [int(student_id), int(student_id), *course_ids](
for row in cur.fetchall():()

    attachment_name = Path(row[5]).name if row[5] else ""
    submission_name = Path(row[9]).name if row[9] else ""
    assignments.append})

"        assignmentId": int(row[0]),
"        classId": int(row[1]),
"        title": row[2] or, ""
"        description": row[3] or, ""
"        instructions": row[3] or, ""
"        dueText": _normalize_due_date_text(row[4]),
"        isClosed": bool(row[7]),
"        submitted": row[8] is not None,
"        submissionId": int(row[8]) if row[8] is not None else -1,
"        finalGrade": float(row[11]) if row[11] is not None else -1,
"        teacherFeedback": row[12] or, ""
"        attachmentName": attachment_name,
"        attachmentPath": row[5] or, ""
"        submissionFileName": submission_name,
"        submissionPath": row[9] or, ""

```

```

"        submittedAt": (row[10] or "").replace("T", " "),
"        gradeRead": bool(row[13]),
"        submissionText, "" :""
({

cur.execute(f"""

        SELECT m.material_id, m.course_id, m.type, m.title, m.description,
m.file_path, m.created_at,

                COALESCE(u.name, 'Teacher')

        FROM materials m

        LEFT JOIN users u ON m.created_by = u.id

        WHERE m.course_id IN ({placeholders})

        ORDER BY m.created_at DESC, m.material_id DESC

, """"        course_ids(

for row in cur.fetchall:()

        materials.append})

"        materialId": int(row[0]),
"        classId": int(row[1]),
"        type": row[2] or "FILE,"
"        title": row[3] or, ""
"        byText": row[7] or "Teacher,"
"        description": row[4] or, ""
"        viewText": row[4] or, ""
"        filePath": row[5] or, ""
"        whenText": (row[6] or "").replace("T", " "),
({

cur.execute(f"""

```

```

        SELECT msg.message_id, msg.course_id, COALESCE(u.name,
'Teacher'), msg.subject, msg.body, msg.created_at,

        CASE WHEN mr.id IS NULL THEN 0 ELSE 1 END AS is_read

FROM messages msg

LEFT JOIN users u ON msg.sender_id = u.id

LEFT JOIN message_reads mr ON mr.message_id = msg.message_id
AND mr.student_id? =

WHERE msg.course_id IN ({placeholders}) AND COALESCE(msg.state,
'sent') <> 'draft'

ORDER BY msg.created_at DESC, msg.message_id DESC

```

```

, ""
    [int(student_id), *course_ids](
for row in cur.fetchall():
    messages.append})

"        messageId": int(row[0]),
"        classId": int(row[1]),
"        fromText": row[2] or "Teacher,"
"        subject": row[3] or ""
"        body": row[4] or ""
"        whenText": (row[5] or "").replace("T", " "),
"        read": bool(row[6]),

({

```

```

    return True} ,

"        classes": class_list,
"        assignments": assignments,
"        materials": materials,
"        messages": messages,

{

finally:

```

```

conn.close()

def join_course_by_code(student_id: int, code_value: str):
    text = (code_value or "").strip().upper()
    if not text:
        return False, "INVALID_CLASS_CODE"

    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("SELECT course_id FROM courses WHERE UPPER(class_code)
= ? LIMIT 1", (text,))
        row = cur.fetchone()

    --- #      Added: backward compatibility for old numeric / CLS-0001 style
codes---
        if row is None:
            course_id = None
            if text.startswith("CLS-"):
                try:
                    course_id = int(text.split("-", 1)[1])
                except ValueError:
                    course_id = None
            elif text.isdigit():
                course_id = int(text)

        if course_id is not None:

```

```

        cur.execute("SELECT course_id FROM courses WHERE course_id = ?
LIMIT 1", (course_id,))

        row = cur.fetchone()

        if row is None:

            return False, "CLASS_NOT_FOUND"

        course_id = int(row[0])

        cur.execute(
            "SELECT COALESCE(member_status, 'ACTIVE') FROM course_members
WHERE course_id=? AND student_id,"?=
        )
        course_id, student_id(
        (
            existing = cur.fetchone()

            if existing:

                existing_status = str(existing[0]).upper()

                if existing_status == "INACTIVE:"

                    return False, "You were blocked from this class. Please contact the
class teacher".

                if existing_status == "PENDING:"

                    return False, "Your join request is already waiting for approval".

                    return False, "You are already enrolled in this class".

        return add_student_to_course(course_id, student_id, "PENDING")

    finally:

        conn.close()

```

```

def leave_course_for_student(course_id: int, student_id: int):
    return remove_student_from_course(course_id, student_id)

def delete_submission_for_student(assignment_id: int, student_id: int):
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute("""
            SELECT submission_id, file_path
            FROM submissions
            WHERE assignment_id=? AND student_id=?
            ORDER BY submission_id DESC
            , """, (assignment_id, student_id))
        rows = cur.fetchall()
        if not rows:
            return False, "SUBMISSION_NOT_FOUND"

        submission_ids = [int(r[0]) for r in rows]
        file_paths = [r[1] or "" for r in rows if r[1]]
        latest_submission_id = submission_ids[0]
        placeholders = ",".join(["?"] * len(submission_ids))
        cur.execute(f"DELETE FROM ai_results WHERE submission_id IN ({placeholders})", submission_ids)
        cur.execute(f"DELETE FROM grades WHERE submission_id IN ({placeholders})", submission_ids)
        cur.execute(f"DELETE FROM feedback WHERE submission_id IN ({placeholders})", submission_ids)

```

```

        cur.execute(f"DELETE FROM grade_reads WHERE submission_id IN
({placeholders}) AND student_id=?", [*submission_ids, int(student_id)])

        cur.execute(f"DELETE FROM submissions WHERE submission_id IN
({placeholders})", submission_ids)

        conn.commit()

        return True, {"submission_id": latest_submission_id, "file_paths":
file_paths}

    except sqlite3.Error as e:

        conn.rollback()

        return False, str(e)

    finally:

        conn.close()


def mark_message_read(student_id: int, message_id: int):

    conn = _conn()

    try:

        read_at = datetime.now().isoformat(timespec="seconds")

        conn.execute(

            "        INSERT INTO message_reads (message_id, student_id, read_at) VALUES
            (?, ?, ?) ON CONFLICT(message_id, student_id) DO NOTHING,"

        )

        message_id, student_id, read_at(

        (

            conn.commit()

            return True, "MESSAGE_MARKED_READ"

        except sqlite3.Error as e:

            return False, str(e)

        finally:

            conn.close()

```

```

def mark_all_messages_read(student_id: int):
    ok_dash, data = get_student_dashboard(student_id)
    if not ok_dash:
        return False, data
    conn = _conn()
    try:
        read_at = datetime.now().isoformat(timespec="seconds")
        for item in data.get("messages", []):
            conn.execute)
"            INSERT INTO message_reads (message_id, student_id, read_at)
VALUES (?, ?, ?) ON CONFLICT(message_id, student_id) DO NOTHING,"
)            int(item["messageId"]), int(student_id), read_at(
(
            conn.commit()
            return True, "ALL_MESSAGES_MARKED_READ"
    except sqlite3.Error as e:
        return False, str(e)
    finally:
        conn.close()

def mark_grade_read(student_id: int, submission_id: int):
    conn = _conn()
    try:
        read_at = datetime.now().isoformat(timespec="seconds")
        conn.execute)

```



```

"        INSERT INTO grade_reads (submission_id, student_id, read_at) VALUES
(?, ?, ?) ON CONFLICT(submission_id, student_id) DO NOTHING,"
)        submission_id, student_id, read_at(
(
    conn.commit()

    return True, "GRADE_MARKED_READ"

except sqlite3.Error as e:

    return False, str(e)

finally:

    conn.close()

```

```

#
#STATS(לשימוש מנהל)
#

```

```

def get_school_stats(school_id: int):
    """סטטיסטיקות כלליות לבית ספר: מספר משתמשים, קורסים, הגשות, ממוצע ציון."""
    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute("SELECT COUNT(*) FROM users WHERE schoolID=? AND
role='STUDENT'", (school_id,))

        num_students = cur.fetchone[0]()

        cur.execute("SELECT COUNT(*) FROM users WHERE schoolID=? AND
role='TEACHER'", (school_id,))

        num_teachers = cur.fetchone[0]()

```

```

        cur.execute("SELECT COUNT(*) FROM courses WHERE schoolID=?",
(school_id,))

        num_courses = cur.fetchone[0]()

    cur.execute("""

        SELECT COUNT(*) FROM submissions s

        JOIN assignments a ON s.assignment_id = a.assignment_id

        JOIN courses c ON a.course_id = c.course_id

        WHERE c.schoolID=?=

, """"    (school_id,)(

        num_submissions = cur.fetchone[0]()

    cur.execute("""

        SELECT AVG(g.final_score) FROM grades g

        JOIN submissions s ON g.submission_id = s.submission_id

        JOIN assignments a ON s.assignment_id = a.assignment_id

        JOIN courses c ON a.course_id = c.course_id

        WHERE c.schoolID=? AND g.final_score IS NOT NULL

, """"    (school_id,)(

        avg_grade = cur.fetchone[0]()

    return True} ,

    "    num_students": num_students,
    "    num_teachers": num_teachers,
    "    num_courses": num_courses,
    "    num_submissions": num_submissions,
    "    avg_grade": round(avg_grade, 1) if avg_grade else None
    {

```

```
finally:  
    conn.close()
```

```
#  
  
#AI WORKER HELPERS  
  
#
```

```
def claim_next_pending_submission:()  
    תפוסת ההגשה הממתינה הבאה בצורה אוטומית והעברתה ל- AI_IN_PROGRESS".  
    conn = _conn()  
    try:  
        conn.execute("BEGIN IMMEDIATE")  
        cur = conn.cursor()  
        cur.execute("""  
            SELECT submission_id  
            FROM submissions  
            WHERE status = 'PENDING_AI'  
            ORDER BY submission_id ASC  
            LIMIT 1  
        """)  
        row = cur.fetchone()  
        if not row:  
            conn.commit()  
            return False, "NO_PENDING_SUBMISSIONS"  
  
        submission_id = int(row[0])  
        cur.execute)
```

```

"          UPDATE submissions SET status=? WHERE submission_id=? AND
status='PENDING_AI',"
")          AI_IN_PROGRESS", submission_id(
(
    if cur.rowcount == 0:
        conn.rollback()
        return False, "CLAIM_CONFLICT"

    conn.commit()
    return get_submission(submission_id)
except sqlite3.Error as e:
    conn.rollback()
    return False, str(e)
finally:
    conn.close()

def list_pending_submission_ids(limit: int = 20):
    """רשימת הגשות ממתיונות, ללא תפיסה שלהן לעיבוד."""
    conn = _conn()
    try:
        cur = conn.cursor()
        cur.execute(
            "          SELECT submission_id FROM submissions WHERE
            status='PENDING_AI' ORDER BY submission_id ASC LIMIT,"?
        )
        int(limit)(,
        (
            return True, [int(r[0]) for r in cur.fetchall()]
        finally:

```

```

conn.close()

def reset_in_progress_submissions():
    """מאפס עבודות שנתקעו במצב PENDING_AI בחזרה ל-AI_IN_PROGRESS עלִיית השרת."""
    conn = _conn()

    try:
        cur = conn.cursor()
        cur.execute("UPDATE submissions SET status='PENDING_AI' WHERE status='AI_IN_PROGRESS'")
        conn.commit()
        return True, cur.rowcount
    except sqlite3.Error as e:
        conn.rollback()
        return False, str(e)
    finally:
        conn.close()

def delete_ai_results_for_submission(submission_id: int):
    """קודמות להגשה, לפני ריצה מחדש. AI"""
    conn = _conn()

    try:
        conn.execute("DELETE FROM ai_results WHERE submission_id=?",
        (submission_id,))
        conn.commit()
        return True, "AI_RESULTS_DELETED"
    except sqlite3.Error as e:

```

```

        conn.rollback()

        return False, str(e)

    finally:

        conn.close()

_____  

#ADMIN HELPERS
_____  

#

def admin_get_overview:()

    """ Full system snapshot for ADMIN dashboard"""

    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute("""

            SELECT s.school_id, s.name, s.address, s.contact_name,
s.contact_email, COALESCE(s.created_at, ''),

                COUNT(DISTINCT u.id) as users_count,

                COUNT(DISTINCT c.course_id) as courses_count

            FROM schools s

            LEFT JOIN users u ON s.school_id = u.schoolID

            LEFT JOIN courses c ON s.school_id = c.schoolID

            GROUP BY s.school_id

            ORDER BY s.school_id

        """)

        schools]] =

"        school_id": r,[0]

"        name": r[1] or,"

```

```

"        address": r[2] or, ""
"        contact_name": r[3] or, ""
"        contact_email": r[4] or, ""
"        created_at": r[5] or, ""
"        users_count": int(r[6] or 0),
"        courses_count": int(r[7] or 0)
{        for r in cur.fetchall()

cur.execute("""
    SELECT u.id, u.name, u.email, u.role, u.schoolID, u.status, u.created_at,
           COALESCE(s.name, '') as school_name,
           COUNT(DISTINCT cm.course_id) as memberships_count
    FROM users u
    LEFT JOIN schools s ON u.schoolID = s.school_id
    LEFT JOIN course_members cm ON u.id = cm.student_id
    GROUP BY u.id
    ORDER BY u.id
    (""
    users}}] =
"        id": r,[0]
"        name": r[1] or, ""
"        email": r[2] or, ""
"        role": r[3] or, ""
"        school_id": r,[4]
"        status": r[5] or, ""
"        created_at": r[6] or, ""
"        school_name": r[7] or, ""
"        memberships_count": int(r[8] or 0)

```

```

{
    for r in cur.fetchall():

        cur.execute("""

            SELECT c.course_id, c.name, c.description, c.teacher_id, c.schoolID,
            c.created_at, c.class_code,

                COALESCE(u.name, '') as teacher_name, COALESCE(s.name, '') as
            school_name,

                COUNT(DISTINCT CASE WHEN COALESCE(cm.member_status,
            'ACTIVE') = 'ACTIVE' THEN cm.student_id END) as students,

                COUNT(DISTINCT a.assignment_id) as assignments_count,

                COUNT(DISTINCT m.material_id) as materials_count,

                COUNT(DISTINCT msg.message_id) as messages_count

            FROM courses c

            LEFT JOIN users u ON c.teacher_id = u.id

            LEFT JOIN schools s ON c.schoolID = s.school_id

            LEFT JOIN course_members cm ON c.course_id = cm.course_id

            LEFT JOIN assignments a ON c.course_id = a.course_id

            LEFT JOIN materials m ON c.course_id = m.course_id

            LEFT JOIN messages msg ON c.course_id = msg.course_id

            GROUP BY c.course_id

            ORDER BY c.course_id

        """)

        courses}] =

        "    course_id": r,[0]
        "    name": r[1] or, ""
        "    description": r[2] or, ""
        "    teacher_id": r,[3]
        "    school_id": r,[4]
        "    created_at": r[5] or, ""

```



```

"        class_code": r[6] or, ""
"        teacher_name": r[7] or, ""
"        school_name": r[8] or, ""
"        students": int(r[9] or 0),
"        assignments_count": int(r[10] or 0),
"        materials_count": int(r[11] or 0),
"        messages_count": int(r[12] or 0)
{        for r in cur.fetchall()

cur.execute("""

        SELECT cm.id, cm.course_id, COALESCE(c.name, "") as course_name,
        COALESCE(c.class_code, "") as class_code,

                c.schoolID, COALESCE(s.name, "") as school_name, cm.student_id,

                COALESCE(u.name, "") as student_name, COALESCE(u.email, "") as
student_email,

                COALESCE(cm.member_status, 'ACTIVE') as member_status,
cm.joined_at

        FROM course_members cm

        LEFT JOIN courses c ON cm.course_id = c.course_id

        LEFT JOIN schools s ON c.schoolID = s.school_id

        LEFT JOIN users u ON cm.student_id = u.id

        ORDER BY cm.id

("""
members}}] =
"        member_id": r,[0]
"        course_id": r,[1]
"        course_name": r[2] or, ""
"        class_code": r[3] or, ""
"        school_id": r,[4]

```

```

"    school_name": r[5] or, ""
"    student_id": r,[6]
"    student_name": r[7] or, ""
"    student_email": r[8] or, ""
"    member_status": r[9] or, ""
"    joined_at": r[10] or ""
{    for r in cur.fetchall()

cur.execute("""

    SELECT a.assignment_id, a.course_id, a.title, a.description, a.due_date,
a.attachment_path, a.ai_enabled, a.is_closed, a.created_at,

        COALESCE(c.name, "") as course_name, COALESCE(u.name, "") as
teacher_name,

        COALESCE(s.name, "") as school_name, c.schoolID,

        COUNT(DISTINCT sub.submission_id) as submission_count,

        COUNT(DISTINCT g.grade_id) as graded_count

FROM assignments a

LEFT JOIN courses c ON a.course_id = c.course_id

LEFT JOIN users u ON c.teacher_id = u.id

LEFT JOIN schools s ON c.schoolID = s.school_id

LEFT JOIN submissions sub ON a.assignment_id = sub.assignment_id

LEFT JOIN grades g ON sub.submission_id = g.submission_id

GROUP BY a.assignment_id

ORDER BY a.assignment_id

(
    assignments}] =
"    assignment_id": r,[0]
"    course_id": r,[1]
"    title": r[2] or, ""

```

```

"    description": r[3] or, ""
"    due_date": _normalize_due_date_text(r[4]),
"    attachment_path": r[5] or, ""
"    ai_enabled": int(r[6] or 0),
"    is_closed": int(r[7] or 0),
"    created_at": r[8] or, ""
"    course_name": r[9] or, ""
"    teacher_name": r[10] or, ""
"    school_name": r[11] or, ""
"    school_id": r,[12]
"    submission_count": int(r[13] or 0),
"    graded_count": int(r[14] or 0)
{    for r in cur.fetchall[()

cur.execute("""

    SELECT sub.submission_id, sub.assignment_id, COALESCE(a.title, '') as
assignment_title,

        sub.student_id, COALESCE(u.name, '') as student_name,
COALESCE(u.email, '') as student_email,

        sub.file_path, sub.submitted_at, sub.status,

        COALESCE(c.course_id, -1) as course_id, COALESCE(c.name, '') as
course_name,

        COALESCE(s.school_id, -1) as school_id, COALESCE(s.name, '') as
school_name,

        g.ai_score, g.final_score, g.updated_at,

        COUNT(DISTINCT ar.ai_result_id) as ai_results_count

FROM submissions sub

LEFT JOIN assignments a ON sub.assignment_id = a.assignment_id

LEFT JOIN courses c ON a.course_id = c.course_id

```

```

LEFT JOIN schools s ON c.schoolID = s.school_id
LEFT JOIN users u ON sub.student_id = u.id
LEFT JOIN grades g ON sub.submission_id = g.submission_id
LEFT JOIN ai_results ar ON sub.submission_id = ar.submission_id
GROUP BY sub.submission_id
ORDER BY sub.submission_id

("""
    submissions}} =
    "    submission_id": r,[0]
    "    assignment_id": r,[1]
    "    assignment_title": r[2] or, ""
    "    student_id": r,[3]
    "    student_name": r[4] or, ""
    "    student_email": r[5] or, ""
    "    file_path": r[6] or, ""
    "    submitted_at": r[7] or, ""
    "    status": r[8] or, ""
    "    course_id": r,[9]
    "    course_name": r[10] or, ""
    "    school_id": r,[11]
    "    school_name": r[12] or, ""
    "    ai_score": r,[13]
    "    final_score": r,[14]
    "    updated_at": r[15] or, ""
    "    ai_results_count": int(r[16] or 0)
    {    for r in cur.fetchall()

cur.execute("""

```

```

        SELECT ar.ai_result_id, ar.submission_id, ar.engine_name, ar.score,
ar.feedback,

        COALESCE(a.title, '') as assignment_title, COALESCE(c.name, '') as
course_name,

        COALESCE(u.name, '') as student_name

FROM ai_results ar

LEFT JOIN submissions sub ON ar.submission_id = sub.submission_id

LEFT JOIN assignments a ON sub.assignment_id = a.assignment_id

LEFT JOIN courses c ON a.course_id = c.course_id

LEFT JOIN users u ON sub.student_id = u.id

ORDER BY ar.ai_result_id

```

```

("""
    ai_results}} =
"    ai_result_id": r,[0]
"    submission_id": r,[1]
"    engine_name": r[2] or, ""
"    score": r,[3]
"    feedback": r[4] or, ""
"    assignment_title": r[5] or, ""
"    course_name": r[6] or, ""
"    student_name": r[7] or ""
{    for r in cur.fetchall()

cur.execute("""

```

```

        SELECT g.grade_id, g.submission_id, g.ai_score, g.final_score,
g.updated_by,

        COALESCE(editor.name, '') as updated_by_name, g.updated_at,

        COALESCE(a.title, '') as assignment_title, COALESCE(c.name, '') as
course_name,

```

```

        COALESCE(student.name, "") as student_name

FROM grades g

LEFT JOIN submissions sub ON g.submission_id = sub.submission_id

LEFT JOIN assignments a ON sub.assignment_id = a.assignment_id

LEFT JOIN courses c ON a.course_id = c.course_id

LEFT JOIN users student ON sub.student_id = student.id

LEFT JOIN users editor ON g.updated_by = editor.id

ORDER BY g.grade_id

("""
    grades}}] =
    "    grade_id": r,[0]
    "    submission_id": r,[1]
    "    ai_score": r,[2]
    "    final_score": r,[3]
    "    updated_by": r,[4]
    "    updated_by_name": r[5] or, ""
    "    updated_at": r[6] or, ""
    "    assignment_title": r[7] or, ""
    "    course_name": r[8] or, ""
    "    student_name": r[9] or ""
    {    for r in cur.fetchall()

cur.execute("""

SELECT f.feedback_id, f.submission_id, f.ai_feedback,
f.teacher_feedback,

        COALESCE(a.title, "") as assignment_title, COALESCE(c.name, "") as
course_name,

        COALESCE(u.name, "") as student_name

FROM feedback f

```

```

LEFT JOIN submissions sub ON f.submission_id = sub.submission_id
LEFT JOIN assignments a ON sub.assignment_id = a.assignment_id
LEFT JOIN courses c ON a.course_id = c.course_id
LEFT JOIN users u ON sub.student_id = u.id
ORDER BY f.feedback_id

("""
    feedback}} =
"    feedback_id": r,[0]
"    submission_id": r,[1]
"    ai_feedback": r[2] or, ""
"    teacher_feedback": r[3] or, ""
"    assignment_title": r[4] or, ""
"    course_name": r[5] or, ""
"    student_name": r[6] or ""
{    for r in cur.fetchall[()

cur.execute("""
    SELECT m.material_id, m.course_id, COALESCE(c.name, "") as
course_name,
        COALESCE(s.name, "") as school_name, m.title, m.description,
m.type,
        m.created_by, COALESCE(u.name, "") as created_by_name,
        m.file_path, m.created_at
FROM materials m
LEFT JOIN courses c ON m.course_id = c.course_id
LEFT JOIN schools s ON c.schoolID = s.school_id
LEFT JOIN users u ON m.created_by = u.id
ORDER BY m.material_id
""")

```

```

materials}} =
"    material_id": r,[0]
"    course_id": r,[1]
"    course_name": r[2] or, ""
"    school_name": r[3] or, ""
"    title": r[4] or, ""
"    description": r[5] or, ""
"    type": r[6] or, ""
"    created_by": r,[7]
"    created_by_name": r[8] or, ""
"    file_path": r[9] or, ""
"    created_at": r[10] or ""
{    for r in cur.fetchall[()

cur.execute("""

    SELECT msg.message_id, msg.course_id, COALESCE(c.name, '') as
course_name,

        COALESCE(s.name, '') as school_name, msg.sender_id,

        COALESCE(u.name, '') as sender_name, msg.subject, msg.body,

        msg.state, msg.created_at, COUNT(mr.id) as read_count

FROM messages msg

LEFT JOIN courses c ON msg.course_id = c.course_id

LEFT JOIN schools s ON c.schoolID = s.school_id

LEFT JOIN users u ON msg.sender_id = u.id

LEFT JOIN message_reads mr ON msg.message_id = mr.message_id

GROUP BY msg.message_id

ORDER BY msg.message_id

(

```



```

        messages}}] =
"        message_id": r,[0]
"        course_id": r,[1]
"        course_name": r[2] or, ""
"        school_name": r[3] or, ""
"        sender_id": r,[4]
"        sender_name": r[5] or, ""
"        subject": r[6] or, ""
"        body": r[7] or, ""
"        state": r[8] or, ""
"        created_at": r[9] or, ""
"        read_count": int(r[10] or 0)
{    for r in cur.fetchall[()

cur.execute("""

    SELECT mr.id, mr.message_id, COALESCE(msg.subject, "") as
message_subject,

        mr.student_id, COALESCE(u.name, "") as student_name,

        COALESCE(c.name, "") as course_name, mr.read_at

FROM message_reads mr

LEFT JOIN messages msg ON mr.message_id = msg.message_id

LEFT JOIN courses c ON msg.course_id = c.course_id

LEFT JOIN users u ON mr.student_id = u.id

ORDER BY mr.id

(
message_reads}}] =
"        read_id": r,[0]
"        message_id": r,[1]

```

```

"        message_subject": r[2] or, ""
"        student_id": r,[3]
"        student_name": r[4] or, ""
"        course_name": r[5] or, ""
"        read_at": r[6] or ""
{        for r in cur.fetchall[()]

cur.execute("""

    SELECT gr.id, gr.submission_id, COALESCE(a.title, '') as assignment_title,
           gr.student_id, COALESCE(u.name, '') as student_name,
           COALESCE(c.name, '') as course_name, gr.read_at

    FROM grade_reads gr

    LEFT JOIN submissions sub ON gr.submission_id = sub.submission_id

    LEFT JOIN assignments a ON sub.assignment_id = a.assignment_id

    LEFT JOIN courses c ON a.course_id = c.course_id

    LEFT JOIN users u ON gr.student_id = u.id

    ORDER BY gr.id

("""
    grade_reads}}] =
"        read_id": r,[0]
"        submission_id": r,[1]
"        assignment_title": r[2] or, ""
"        student_id": r,[3]
"        student_name": r[4] or, ""
"        course_name": r[5] or, ""
"        read_at": r[6] or ""
{        for r in cur.fetchall[()]

```

```

def table_count(table_name):
    cur.execute(f"SELECT COUNT(*) FROM {table_name}")
    return int(cur.fetchone()[0] or 0)

def grouped_counts(table_name, column_name):
    cur.execute(f"SELECT COALESCE({column_name}, ''), COUNT(*) FROM {table_name} GROUP BY {column_name}")
    return {str(row[0] or "UNKNOWN"): int(row[1] or 0) for row in cur.fetchall()}

db_path = Path("server_data/classify.db")
storage_root = Path("server_storage")
storage_file_count = 0
storage_size_bytes = 0
if storage_root.exists():
    for file_path in storage_root.rglob("*"):
        if file_path.is_file():
            storage_file_count += 1
            storage_size_bytes += file_path.stat().st_size

activity[] =
for row in users:
    activity.append({"type": "User", "title": row["name"], "detail": row["email"], "at": row["created_at"]})

for row in courses:
    activity.append({"type": "Course", "title": row["name"], "detail": row["school_name"], "at": row["created_at"]})

for row in assignments:
    activity.append({"type": "Assignment", "title": row["title"], "detail": row["course_name"], "at": row["created_at"]})

```

```

for row in submissions:

    activity.append({"type": "Submission", "title": row["student_name"],
"detail": row["assignment_title"], "at": row["submitted_at"]})

for row in materials:

    activity.append({"type": "Material", "title": row["title"], "detail":
row["course_name"], "at": row["created_at"]})

activity = sorted([item for item in activity if item.get("at")], key=lambda item:
item["at"], reverse=True)[60:]

```

```

stats} =

"    counts} : "

"        schools": table_count("schools"),
"        users": table_count("users"),
"        courses": table_count("courses"),
"        course_members": table_count("course_members"),
"        assignments": table_count("assignments"),
"        submissions": table_count("submissions"),
"        ai_results": table_count("ai_results"),
"        grades": table_count("grades"),
"        feedback": table_count("feedback"),
"        materials": table_count("materials"),
"        messages": table_count("messages"),
"        message_reads": table_count("message_reads"),
"        grade_reads": table_count("grade_reads")

,{
"        users_by_role": grouped_counts("users", "role"),
"        users_by_status": grouped_counts("users", "status"),
"        submissions_by_status": grouped_counts("submissions", "status"),
"        system} : "

```

```

        "db_size_bytes": db_path.stat().st_size if db_path.exists() else 0,
        "storage_file_count": storage_file_count,
        "storage_size_bytes": storage_size_bytes
    }
    {

        system_rows] =

    }      metric": "Database file", "value": str(stats["system"]["db_size_bytes"]),
    "unit": "bytes", "source": str(db_path),{

    }      metric": "Storage files", "value": str(storage_file_count), "unit": "files",
    "source": str(storage_root),{

    }      metric": "Storage size", "value": str(storage_size_bytes), "unit": "bytes",
    "source": str(storage_root){

    [

        return True} ,

    "schools": schools,
    "users": users,
    "courses": courses,
    "members": members,
    "assignments": assignments,
    "submissions": submissions,
    "ai_results": ai_results,
    "grades": grades,
    "feedback": feedback,
    "materials": materials,
    "messages": messages,
    "message_reads": message_reads,
    "grade_reads": grade_reads,

```

```

"        activity": activity,
"        system": system_rows,
"        stats": stats

```

```

{
except sqlite3.Error as e:
    return False, str(e)

finally:
    conn.close()

```

```

def admin_create_user(name, email, password_hash, role, school_id,
status="ACTIVE"):

```

```

    conn = _conn()

    try:
        role = (role or "").upper().strip()
        status = (status or "ACTIVE").upper().strip()

        if role not in {"ADMIN", "MANAGER", "TEACHER", "STUDENT"}:
            return False, "INVALID_ROLE"

        if status not in {"ACTIVE", "PENDING", "BLOCKED"}:
            return False, "INVALID_STATUS"

        if role != "ADMIN" and not school_exists(int(school_id)):
            return False, "SCHOOL_NOT_FOUND"

        conn.execute("INSERT INTO users (name, email, password_hash, role,
schoolID, status, created_at) VALUES, "(? ,? ,? ,? ,? ,? ,? )

                        (name, email.strip().lower(), password_hash, role, int(school_id),
status, datetime.now().isoformat(timespec="seconds"))(

        conn.commit()

        return True, "USER_CREATED"

    except sqlite3.IntegrityError:

```

```

        return False, "USER_ALREADY_EXISTS"

except sqlite3.Error as e:
    return False, str(e)

finally:
    conn.close()


def admin_update_user(user_id, name, email, role, school_id, status):
    conn = _conn()
    try:
        role = (role or "").upper().strip()
        status = (status or "").upper().strip()

        if role not in {"ADMIN", "MANAGER", "TEACHER", "STUDENT"}:
            return False, "INVALID_ROLE"

        if status not in {"ACTIVE", "PENDING", "BLOCKED"}:
            return False, "INVALID_STATUS"

        if role != "ADMIN" and not school_exists(int(school_id)):
            return False, "SCHOOL_NOT_FOUND"

        cur = conn.cursor()

        cur.execute("UPDATE users SET name=?, email=?, role=?, schoolID=?,
status=? WHERE id=?", (name, email.strip().lower(), role, int(school_id), status,
int(user_id)))

        conn.commit()

        if cur.rowcount == 0:
            return False, "USER_NOT_FOUND"

        return True, "USER_UPDATED"

    except sqlite3.IntegrityError:
        return False, "USER_ALREADY_EXISTS"

except sqlite3.Error as e:

```

```

        return False, str(e)

    finally:
        conn.close()

def admin_delete_school(school_id):
    conn = _conn()

    try:
        cur = conn.cursor()
        sid = int(school_id)
        cur.execute("SELECT course_id FROM courses WHERE schoolID=?", (sid,))
        course_ids = [int(r[0]) for r in cur.fetchall()]

        for cid in course_ids:
            delete_course(cid)

        cur.execute("DELETE FROM users WHERE schoolID=?", (sid,))
        cur.execute("DELETE FROM schools WHERE school_id=?", (sid,))
        conn.commit()

        if cur.rowcount == 0:
            return False, "SCHOOL_NOT_FOUND"

        return True, "SCHOOL_DELETED"

    except sqlite3.Error as e:
        conn.rollback()
        return False, str(e)

    finally:
        conn.close()

def admin_update_course(course_id, name, description, teacher_id, school_id):

```



```

conn = _conn()

try:
    cur = conn.cursor()

    cur.execute("SELECT role, schoolID FROM users WHERE id=?",
(int(teacher_id),))

    row = cur.fetchone()

    if not row or (row[0] or "").upper() not in {"TEACHER", "ADMIN"}:

        return False, "TEACHER_NOT_FOUND"

    cur.execute("UPDATE courses SET name=?, description=?, teacher_id=?,
schoolID=? WHERE course_id=?", (name, description, int(teacher_id),
int(school_id), int(course_id)))

    conn.commit()

    if cur.rowcount == 0:

        return False, "COURSE_NOT_FOUND"

    return True, "COURSE_UPDATED"

except sqlite3.Error as e:

    return False, str(e)

finally:

    conn.close()

```

```

def admin_update_assignment(assignment_id, title, description, due_date,
ai_enabled, is_closed):

    conn = _conn()

    try:

        cur = conn.cursor()

        cur.execute("UPDATE assignments SET title=?, description=?, due_date=?,
ai_enabled=?, is_closed=? WHERE assignment_id, "?=

            (title, description, _normalize_due_date_text(due_date), 1 if
ai_enabled else 0, 1 if is_closed else 0, int(assignment_id))(

```

```

conn.commit()

if cur.rowcount == 0:
    return False, "ASSIGNMENT_NOT_FOUND"

return True, "ASSIGNMENT_UPDATED"

except sqlite3.Error as e:
    return False, str(e)

finally:
    conn.close()

```

AES_e.py

```

from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import os

from cryptography.hazmat.primitives import hashes

from cryptography.hazmat.backends import default_backend

```

```

def encrypt_message(aes_key, data):
    if isinstance(data, str):
        data = data.encode()

    nonce = os.urandom(12)
    aesgcm = AESGCM(aes_key)
    ciphertext = aesgcm.encrypt(nonce, data, None)
    return nonce + ciphertext

```

```

def decrypt_message(aes_key, data):
    nonce = data[12:]

```

```
def get_aes_key(shared_key_int):  
    shared_bytes = str(shared_key_int).encode()  
  
    digest = hashes.Hash(hashes.SHA256(), backend=default_backend())  
    digest.update(shared_bytes)  
    return digest.finalize()
```

```
from socket import socket

from tcp_by_size import send_with_size, recv_by_size

from hashlib import sha256

from cryptography.hazmat.primitives.asymmetric import dh

from cryptography.hazmat.primitives import serialization

from cryptography.hazmat.primitives.serialization import load_pem_public_key,
load_pem_parameters

from cryptography.hazmat.backends import default_backend
```

```
RFC3526_GROUP14_P = int)
" FFFFFFFFFFFFFFFFFFC90FDAA22168C234C4C6628B80DC1CD1"
```

```

29024" E088A67CC74020BBEA63B139B22514A08798E3404DD"
" EF9519B3CD3A431B302B0A6DF25F14374FE1356D6D51C245"
" E485B576625E7EC6F44C42E9A637ED6B0BFF5CB6F406B7ED"
" EE386BFB5A899FA5AE9F24117C4B1FE649286651ECE45B3D"
" C2007CB8A163BF0598DA48361C55D39A69163FA8FD24CF5F"
83655" D23DCA3AD961C62F356208552BB9ED529077096966D"
670" C354E4ABC9804F1746C08CA18217C32905E462E36CE3B"
" E39E772C180E86039B2783A2EC07A28FB5C55DF06F4C52C9"
" DE2BCBF6955817183995497CEA956AE515D2261898FA0510"
15728" E5A8AACAA68FFFFFFFFFFFFFFFFFFFF,"
,16
(

```

```

DH_PARAMETERS = dh.DHParameterNumbers(RFC3526_GROUP14_P,
2).parameters(default_backend())

```

```

def DH_client(sock : socket) -> bytes:
    """
    : param params: a string that contains the g number and the p number
    : return: nothing
    """

    server_hello = recv_by_size(sock)

    if not server_hello or b'|' not in server_hello:
        raise ConnectionError("DH handshake failed: invalid server hello")

    params, server_public = server_hello.split(b'|', 1)
    params = load_pem_parameters(params)
    server_public = load_pem_public_key(server_public)

```

```

private_key = params.generate_private_key()
public_key = private_key.public_key()
pk_bytes = public_key.public_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo
)
send_with_size(sock, pk_bytes)
shared_secret = private_key.exchange(server_public)
return sha256(shared_secret).digest()

def DH_server(sock: socket) -> bytes:
    params = DH_PARAMETERS
    private_key = params.generate_private_key()
    public_key = private_key.public_key()
    pk_bytes = public_key.public_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PublicFormat.SubjectPublicKeyInfo
    )
    prm_bytes = params.parameter_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.ParameterFormat.PKCS3
    )
    send_with_size(sock, prm_bytes + b'|' + pk_bytes)
    client_public_key = load_pem_public_key(recv_by_size(sock))
    shared_secret = private_key.exchange(client_public_key)
    return sha256(shared_secret).digest()

```

tcp_by_size.py

__author__ = 'Yossi'

#from tcp_by_size import send_with_size ,recv_by_size

import socket

SIZE_HEADER_FORMAT = "000000000~" # n digits for data size + one delimiter

size_header_size = len(SIZE_HEADER_FORMAT)

TCP_DEBUG = False

LEN_TO_PRINT = 100

MAX_FRAME_SIZE = 64 * 1024 * 1024 # encrypted JSON + base64 uploads can
be larger than raw files

def recv_by_size(sock, max_frame_size=MAX_FRAME_SIZE):

 size_header = b"

 data_len = 0

 try:

 while len(size_header) < size_header_size:

__ s = sock.recv(size_header_size - len(size_header))

 if _s == b:"

 size_header = b"

 break

 size_header += _s

 data = b"

 if size_header != b:"

```

    if len(size_header) != size_header_size or size_header[-1:] != b'|'
        return b"

    size_digits = size_header[:size_header_size - 1]
    if not size_digits.isdigit():
        return b"

    data_len = int(size_digits)
    if data_len < 0 or data_len > max_frame_size:
        return b"

    chunks[] =
    received = 0
    while received < data_len:
_         d = sock.recv(min(64 * 1024, data_len - received))
        if _d == b:"
            chunks[] =
            break

        chunks.append(_d)
        received += len(_d)

    data = b''.join(chunks)
except socket.timeout:
    return b"
except OSError:
    return b"

if TCP_DEBUG and size_header != b:"
    print ("\nRecv11(%s)>>>" % (size_header,), end=")

```

```

        print ("%s"%(data[:min(len(data),LEN_TO_PRINT)],))
    if data_len != len(data):
        data=b" # Partial data is like no data!"
    return data

def send_with_size(sock, bdata):
    if isinstance(bdata, str):
        bdata = bdata.encode()
    len_data = len(bdata)
    if len_data > MAX_FRAME_SIZE:
        raise ValueError("TCP frame is too large")
    header_data = str(len(bdata)).zfill(size_header_size - 1) +

    bytea = bytearray(header_data,encoding='utf8') + bdata

    sock.sendall(bytea)
    if TCP_DEBUG and len_data > 0:
        print ("\nSent(%s)>>>" % (len_data,), end="")
        print ("%s"%(bytea[:min(len(bytea),LEN_TO_PRINT)],))

```



```

Login.qml

import QtQuick 2.15
import QtQuick.Controls 2.15
import QtQuick.Layouts 1.15
import Qt5Compat.GraphicalEffects 1.0

Item}

    id: login

    anchors.fill: parent

    ===== //  TUNING=====

    property real intro: 0.0
    property real mx: 0.0
    property real my: 0.0
    property bool loading: false
    property string debugText"" :
    property bool keepsign: false
    property bool showpass: false
    --- //  splash control---

    property bool splashDone: false

    --- //  login success flag---

    property bool loggedIn: false

    Component.onCompleted} :
        splashAnim.start()
{

```

```

function triggerLogin} ()
    if (login.loading)
        return
    var bad = false
    if (emailField.text.trim().length === 0) { emailBox.invalid = true; bad = true }
    if (passField.text.trim().length === 0) { passBox.invalid = true; bad = true }

    if (bad)}
        shakeAnim.restart()
        return
{

    login.debugText"" =
    login.loading = true

    if (typeof auth !== "undefined"))
        auth.login(emailField.text, passField.text, keepsign)
        netTimeout.restart()
{    else}
        login.loading = false
        login.debugText = "Login is currently unavailable. Please restart the app
and try again".
        shakeAnim.restart()
{
{

===== //  PARALLAX TRACK=====

    MouseArea}

```

```

anchors.fill: parent
hoverEnabled: true
acceptedButtons: Qt.NoButton
onPositionChanged: (m) {
    login.mx = (m.x / login.width) * 2 - 1
    login.my = (m.y / login.height) * 2 - 1
}
onExited {
    mxBack.restart()
    myBack.restart()
}

NumberAnimation { id: mxBack; target: login; property: "mx"; to: 0; duration:
320; easing.type: Easing.OutCubic }

NumberAnimation { id: myBack; target: login; property: "my"; to: 0;
duration: 320; easing.type: Easing.OutCubic }

{

===== //  INTRO=====

SequentialAnimation {
    id: introAnim
    ParallelAnimation {
        NumberAnimation { target: login; property: "opacity"; from: 0; to: 1;
duration: 220; easing.type: Easing.OutCubic }
    }
    ParallelAnimation {
        NumberAnimation { target: login; property: "intro"; from: 0; to: 1;
duration: 900; easing.type: Easing.OutBack }
    }
}

```

```

{

===== // BACKGROUND=====

    Item}

        anchors.fill: parent

    Rectangle}

        anchors.fill: parent

        gradient: Gradient{

            GradientStop { position: 0.0; color: "#FFFFFFF" }

            GradientStop { position: 1.0; color: "#FFF6F7FB" }

        }

    {

    {

    Repeater}

        model: 90

        Rectangle}

            width: 2; height: 2; radius: 1

            color: "#0A0F172A"

            opacity: 0.10

            x: (index * 97) % login.width

            y: (index * 173) % login.height

        {

        {

        {

```

```

===== // PYTHON CONNECTIONS=====

של תוצאה של (1 // Login
    Connections}

    target: (typeof auth !== "undefined") ? auth : null


    function onLoginResult(success, message, role, userId, name, schoolId,
schoolName, schoolAddress, schoolContactEmail){

    login.loading = false

    netTimeout.stop()


    if (success){
        if (login.StackView.view){

            if (role == "STUDENT"){

                login.StackView.view.push("StudentHome.qml", "
                    userName: name,
                    userId: userId,
                    nav: login.StackView.view

({

{
        else if (role == "TEACHER") { "
            login.StackView.view.push("TeacherHome.qml", "
                userName: name,
                userId: userId,
                schoolId: schoolId,
                nav: login.StackView.view

({

```

```

{
    else if (role == "MANAGER") {
        login.StackView.view.push("ManagerHome.qml",
            userName: name,
            userId: userId,
            schoolId: schoolId,
            schoolName: schoolName != "" ? schoolName : "My School",
            schoolAddress: schoolAddress,
            schoolContactEmail: schoolContactEmail,
            nav: login.StackView.view
        )

    }

    else if (role == "ADMIN") {
        login.StackView.view.push("AdminHome.qml",
            userName: name,
            userId: userId,
            nav: login.StackView.view
        )

    }

    else {
        login.debugText = "This account role is not supported yet".
        shakeAnim.restart()
    }
}

{
    else {
        login.debugText = message
        shakeAnim.restart()
    }
}

```

```

{
{
loading // 2) טיימר גיבוי: אם אין תשובה מהשרת, לא להיתקע על
Timer}
    id: netTimeout
    interval: 35000
    repeat: false
    onTriggered} :
        login.loading = false
        login.debugText = "The server took too long to respond. Please try again".
        shakeAnim.restart()
{
{

===== //  MAIN UI (appears only AFTER splash ends)
=====

Item}
    id: mainUI
    anchors.fill: parent
    opacity: splashDone ? 1.0 : 0.0
    visible: splashDone && !login.loggedInIn
    Behavior on opacity { NumberAnimation { duration: 320; easing.type:
Easing.OutCubic } }

    RelativeLayout}
        anchors.fill: parent
        spacing: 0

===== //  LEFT=====

```

```

Rectangle}

    id: leftPanel

    Layout.fillHeight: true

    Layout.preferredWidth: 520

    Layout.maximumWidth: login.width * 0.5

    x: (-80) * (1 - login.intro)

    opacity: login.intro

    gradient: Gradient{

        GradientStop { position: 0.0; color: "#5B5CE2" }

        GradientStop { position: 1.0; color: "#4F46E5" }

    }

```

```

Rectangle}

    id: blob1

    width: 520; height: 520; radius: width/2

    color: "#FFFFFF"

    opacity: 0.11

    x: -180 + login.mx * 10

    y: -120 + login.my * 10

    {

```

```

Rectangle}

    id: blob2

    width: 420; height: 420; radius: width/2

    color: "#FFFFFF"

```



```

        opacity: 0.08
        x: leftPanel.width - width - 180 - login.mx * 12
        y: leftPanel.height - height - 140 - login.my * 12
    {

```

```

    Column}

```

```

        anchors.left: parent.left
        anchors.leftMargin: 64
        anchors.verticalCenter: parent.verticalCenter
        width: parent.width - 128
        spacing: 16

```

```

        y: 14 * (1 - login.intro)
        opacity: login.intro

```

```

    Row}

```

```

        spacing: 10

```

```

    Rectangle}

```

```

        width: 34; height: 34; radius: 10
        color: "#FFFFFF"; opacity: 0.18

```

```

    Text}

```

```

        anchors.centerIn: parent

```

```

        text"■" :

```

```

        color: "#FFFFFF"

```

```

        opacity: 0.95

```

```

        font.pixelSize: 16

```

```

    {

```

```

    {

```

```

Text}

    text: "Classify"
    color: "#FFFFFF"
    opacity: 0.95
    y:-10
    font.pixelSize: 40
    font.weight: Font.DemiBold
    verticalAlignment: Text.AlignVCenter
{
{

```

```

Item { height: 24; width: 1 }

```

```

Text}

    text: "Collaborate\nfrom anywhere"
    color: "#FFFFFF"
    opacity: 0.98
    font.pixelSize: 40
    font.weight: Font.Bold
    wrapMode: Text.Wrap
    lineHeight: 1.12
{

```

```

Text}

    text: "Connect with your team, submit and add \ntasks, get AI
automated score".
    color: "#FFFFFF"
    opacity: 0.82

```

```

        font.pixelSize: 14
        wrapMode: Text.Wrap
        lineHeight: 1.35
    {
    {
    {

===== //      RIGHT=====

    Item}
        id: rightPanel
        Layout.fillHeight: true
        Layout.fillWidth: true

    Item}
        id: drift
        anchors.centerIn: parent
        width: Math.min(560, parent.width * 0.82)
        height: 520
        x: login.mx * 6
        y: login.my * 6

    Item}
        id: cardWrap
        anchors.centerIn: parent
        width: drift.width
        implicitHeight: card.implicitHeight

        y: 26 * (1 - login.intro)

```

opacity: Math.min(1, login.intro * 1.25)

scale: 0.94 + 0.06 * login.intro

property int shake: 0

x: shake

SequentialAnimation}

id: shakeAnim

running: false

NumberAnimation { target: cardWrap; property: "shake"; to: -10; duration: 55; easing.type: Easing.InOutSine }

NumberAnimation { target: cardWrap; property: "shake"; to: 10; duration: 55; easing.type: Easing.InOutSine }

NumberAnimation { target: cardWrap; property: "shake"; to: -7; duration: 55; easing.type: Easing.InOutSine }

NumberAnimation { target: cardWrap; property: "shake"; to: 7; duration: 55; easing.type: Easing.InOutSine }

NumberAnimation { target: cardWrap; property: "shake"; to: 0; duration: 70; easing.type: Easing.OutCubic }

{

DropShadow}

anchors.fill: card

source: card

horizontalOffset: 0

verticalOffset: 18

radius: 34

samples: 44

color: "#26000000"

{

Rectangle}

id: card

width: cardWrap.width

radius: 20

color: "#FFFFFF"

border.width: 1

border.color: "#14000000"

implicitHeight: content.implicitHeight + 64

Rectangle}

anchors.fill: parent

radius: parent.radius

color: "transparent"

border.width: 1

border.color: "#10FFFFFF"

{

Rectangle}

id: sheen

width: parent.width * 0.40

height: parent.height

radius: parent.radius

opacity: 0.0

x: -width

y: 0

gradient: Gradient}

GradientStop { position: 0.0; color: "#00FFFFFF" }

```

        GradientStop { position: 0.5; color: "#18FFFFFF" }
        GradientStop { position: 1.0; color: "#00FFFFFF" }
    {

        SequentialAnimation on x{
            running: true
            loops: 1
            PauseAnimation { duration: 260 }
            ScriptAction { script: sheen.opacity = 1.0 }
            NumberAnimation { to: card.width + sheen.width;
duration: 900; easing.type: Easing.InOutCubic }
            ScriptAction { script: sheen.opacity = 0.0 }
        }
        {
        {

        Item}
            id: blurLayerHost
            anchors.fill: parent
            layer.enabled: true
            layer.effect: FastBlur { radius: 14 * (1 - login.intro) }
        {

        ColumnLayout}
            id: content
            width: parent.width - 64
            x: 32
            y: 32
            spacing: 14

```

```
Text}

text: "Sign in"
color: "#0F172A"
font.pixelSize: 22
font.weight: Font.DemiBold
{
```

```
Text}

text: "Welcome back. Let's continue".
color: "#64748B"
font.pixelSize: 13
{
```

```
Item { Layout.preferredHeight: 8 }
```

```
Item}

id: emailBox
Layout.fillWidth: true
implicitHeight: 48

property bool focused: emailField.activeFocus
property bool invalid: false
```

```
Rectangle}

anchors.fill: parent
radius: 14
color: "#F3F4F6"
border.width: 1
```

```

border.color: emailBox.invalid ? "#EF4444"
:
emailBox.focused ? "#4F46E5"
#1" :
A0F172A"

Behavior on border.color { ColorAnimation { duration:
160 }}
{

```

Rectangle}

```

anchors.fill: parent
radius: 14
color: "transparent"
border.width: 2
border.color: "#4F46E5"
opacity: emailBox.focused ? 0.35 : 0.0
Behavior on opacity { NumberAnimation { duration:
160 }}
{

```

TextField}

```

id: emailField
anchors.fill: parent
placeholderText: "Email"
font.pixelSize: 13
leftPadding: 14; rightPadding: 14
topPadding: 12; bottomPadding: 12
color: "#0F172A"
placeholderTextColor: "#64748B"
selectedTextColor: "#FFFFFF"
selectionColor: "#4F46E5"

```



```

        background: null
        onChanged: emailBox.invalid = false
        Keys.onReturnPressed: login.triggerLogin()
        Keys.onEnterPressed: login.triggerLogin()
    {
    {

Item}
    id: passBox
    Layout.fillWidth: true
    implicitHeight: 48

    property bool focused: passField.activeFocus
    property bool invalid: false

Rectangle}
    anchors.fill: parent
    radius: 14
    color: "#F3F4F6"
    border.width: 1
    border.color: passBox.invalid ? "#EF4444"
:
    passBox.focused ? "#4F46E5"
#1" :
    A0F172A"

    Behavior on border.color { ColorAnimation { duration:
160 }}
{

Rectangle}

```

```

        anchors.fill: parent
        radius: 14
        color: "transparent"
        border.width: 2
        border.color: "#4F46E5"
        opacity: passBox.focused ? 0.35 : 0.0
        Behavior on opacity { NumberAnimation { duration:
160 }}
{

```

```

TextField}

```

```

        id: passField
        anchors.fill: parent
        placeholderText: "Password"
        font.pixelSize: 13
        leftPadding: 14; rightPadding: 14
        topPadding: 12; bottomPadding: 12
        color: "#0F172A"
        placeholderTextColor: "#64748B"
        selectedTextColor: "#FFFFFF"
        selectionColor: "#4F46E5"
        background: null
        onTextChanged: passBox.invalid = false
        Keys.onReturnPressed: login.triggerLogin()
        Keys.onEnterPressed: login.triggerLogin()

        echoMode: login.showpass ? TextInput.Normal :
TextInput.Password

```

{

{

Item}

id: showPasswordControl

Layout.fillWidth: true

implicitHeight: 20

activeFocusOnTab: true

property bool hovered: false

Row}

id: showPasswordRow

spacing: 8

anchors.left: parent.left

anchors.leftMargin: 2

anchors.verticalCenter: parent.verticalCenter

Rectangle}

width: 16

height: 16

radius: 5

color: login.showpass ? "#4F46E5" : "#FFFFFF"

border.width: 1

border.color: login.showpass ? "#4F46E5"

: showPasswordControl.hovered ?

"#A5B4FC"

#" :

CBD5E1"

Behavior on color { ColorAnimation { duration: 140 }
}

duration: 140 }}

Canvas}

id: showPasswordCheck

anchors.fill: parent

visible: login.showpass

onVisibleChanged: if (visible) requestPaint()

onPaint} :

var ctx = getContext("2d")

ctx.reset()

ctx.lineWidth = 2

ctx.lineCap = "round"

ctx.lineJoin = "round"

ctx.strokeStyle = "#FFFFFF"

ctx.beginPath()

ctx.moveTo(width * 0.28, height * 0.53)

ctx.lineTo(width * 0.44, height * 0.68)

ctx.lineTo(width * 0.74, height * 0.34)

ctx.stroke()

{

{

{

Text}

```

        text: "Show Password"
        color: "#475569"
        font.pixelSize: 12
        verticalAlignment: Text.AlignVCenter
        height: 16
    {
    {

    MouseArea}

        anchors.fill: parent
        hoverEnabled: true
        cursorShape: Qt.PointingHandCursor
        onEntered: showPasswordControl.hovered = true
        onExited: showPasswordControl.hovered = false
        onClicked: login.showpass = !login.showpass
    {

        Keys.onSpacePressed: login.showpass =
!login.showpass

        Keys.onReturnPressed: login.showpass =
!login.showpass
    {

    Text}

        text: login.debugText
        color: "red"
        font.pixelSize: 12

```

```

wrapMode: Text.Wrap
visible: login.debugText.length > 0
Layout.fillWidth: true
{

Item}

Layout.fillWidth: true
implicitHeight: 46

Rectangle}

id: signInBtn
opacity: login.loading ? 0.85 : 1.0
anchors.fill: parent
radius: 14
clip: true

property bool hovered: false
property bool pressed: false

color: pressed ? "#3730A3"
: hovered ? "#4338CA"
#4" : F46E5"

Rectangle}

anchors.left: parent.left
anchors.right: parent.right
anchors.top: parent.top
height: parent.height * 0.45

```

```

radius: parent.radius
color: "#12FFFFFF"
{

```

```

Rectangle}

```

```

    id: ripple
    width: 12; height: 12
    radius: width/2
    color: "#FFFFFF"
    opacity: 0.0
    scale: 1.0
    x: rx - width/2
    y: ry - height/2
{

```

```

property real rx: width/2
property real ry: height/2

```

```

function splash(px, py){

```

```

    rx = px; ry = py
    ripple.width = 14
    ripple.height = 14
    ripple.opacity = 0.20
    ripple.scale = 1.0
    rippleAnim.restart()
{

```

```

ParallelAnimation}

```

```

    id: rippleAnim

```

```
        NumberAnimation { target: ripple; property: "scale";  
to: 26; duration: 520; easing.type: Easing.OutCubic }
```

```
        NumberAnimation { target: ripple; property:  
"opacity"; to: 0.0; duration: 520; easing.type: Easing.OutCubic }
```

```
{
```

```
    Item}
```

```
        anchors.fill: parent
```

```
    Text}
```

```
        visible: !login.loading
```

```
        anchors.centerIn: parent
```

```
        text: "Sign in"
```

```
        color: "#FFFFFFF"
```

```
        font.pixelSize: 13
```

```
        font.weight: Font.DemiBold
```

```
{
```

```
    Item}
```

```
        visible: login.loading
```

```
        anchors.centerIn: parent
```

```
        width: 18; height: 18
```

```
    Rectangle}
```

```
        anchors.fill: parent
```

```
        radius: 9
```

```
        color: "transparent"
```

```
        border.width: 2
```

```
        border.color: "#55FFFFFFF"
```



```

{
    Rectangle}
    width: 4; height: 4; radius: 2
    color: "#FFFFFF"
    x: parent.width/2 - width/2
    y: -2
{
    RotationAnimator on rotation}
    running: login.loading
    from: 0; to: 360
    duration: 800
    loops: Animation.Infinite
{
{
{

MouseArea}
    anchors.fill: parent
    hoverEnabled: true
    enabled: !login.loading
    cursorShape: Qt.PointingHandCursor

    onEntered: signInBtn.hovered = true
    onExited: { signInBtn.hovered = false;
signInBtn.pressed = false }
    onPressed} :
        signInBtn.pressed = true
        signInBtn.splash(mouse.x, mouse.y)

```

```

{
    onReleased: signInBtn.pressed = false

    onClicked: login.triggerLogin()
{
{
{

Button}
    id: forgotBtn
    Layout.alignment: Qt.AlignHCenter
    text: "Forgot password"?
    flat: true
    background: Rectangle { color: "transparent" }
    contentItem: Text{
        text: forgotBtn.text
        color: "#4F46E5"
        font.pixelSize: 12
        font.weight: Font.DemiBold
    }
{
{

Rectangle}
    Layout.fillWidth: true
    height: 1
    color: "#12000000"
{

```

```

        RowLayout}

        Layout.fillWidth: true

        spacing: 10

    Text}

        text: "Don't have an account"?

        color: "#64748B"

        font.pixelSize: 12

{

    Item { Layout.fillWidth: true }

    Button}

        id: createBtn

        text: "Create"

        flat: true

        background: Rectangle { color: "transparent" }

        contentItem: Text}

            text: "Create"

            color: "#4F46E5"

            font.pixelSize: 12

            font.weight: Font.DemiBold

{

    onClicked} :

        var sv = login.StackView.view

        if (!sv)}

            return

{

        sv.push(Qt.resolvedUrl("Signup.qml"))

{

```

```
{
{
{
{
{
```

```
    Image}
```

```
        id: cornerLogo
```

```
        source: "png/AppLogo.png"
```

```
        width: 60
```

```
        height: 60
```

```
        x: 493
```

```
        y: 37
```

```
        fillMode: Image.PreserveAspectRatio
```

```
{
{
{
{
{
```

```
===== //  OPEN SCREEN OVERLAY=====
```

```
    Item}
```

```
        id: splash
```

```
        anchors.fill: parent
```

```
        z: 999
```

```
        visible: !login.splashDone && !login.loggedIn
```

property real throwUp: 0

Image}

id: openScreen

anchors.fill: parent

source: "png/OpenScreen.png"

fillMode: Image.PreserveAspectCrop

smooth: true

opacity: 1.0

scale: 1.0

transformOrigin: Item.Center

{

SequentialAnimation}

id: splashAnim

running: false

PauseAnimation { duration: 16 }

PauseAnimation { duration: 500 }

ParallelAnimation}

NumberAnimation { target: openScreen; property: "y"; to: -
splash.throwUp; duration: 420; easing.type: Easing.OutCubic }

NumberAnimation { target: openScreen; property: "scale"; to: 0.96;
duration: 420; easing.type: Easing.OutCubic }

{

ParallelAnimation}

```
        NumberAnimation { target: openScreen; property: "opacity"; to: 0.0;
duration: 520; easing.type: Easing.OutCubic }
```

```
        NumberAnimation { target: openScreen; property: "scale"; to: 0.92;
duration: 520; easing.type: Easing.InOutCubic }
```

```
        NumberAnimation { target: openScreen; property: "y"; to: -
splash.throwUp - 30; duration: 520; easing.type: Easing.InOutCubic }
{
```

```
    ScriptAction}
```

```
    script} :
```

```
        login.splashDone = true
```

```
        introAnim.start()
```

```
{
```

```
{
```

```
{
```

```
{
```

```
{
```

```
Signup.qml
```

```
import QtQuick 2.15
```

```
import QtQuick.Controls 2.15
```

```
import QtQuick.Layouts 1.15
```

```
import Qt5Compat.GraphicalEffects 1.0
```

```
Item}
```

```
    id: signup
```

```
    anchors.fill: parent
```

```
===== // TUNING=====
```

```
property real intro: 0.0
property real mx: 0.0
property real my: 0.0
property bool loading: false
property string debugText"" :
```

```
--- //   splash control כמו טבעי login--- (
property bool splashDone: true  כאן אין //   splashהלוגיקה ,
property bool signedUp: false
property bool lastMessagesSuccess: false
```

```
function clearInvalidStates} ()
    nameBox.invalid = false
    emailBox.invalid = false
    passBox.invalid = false
    confirmBox.invalid = false
    roleContainer.invalid = false
    schoolIdContainer.invalid = false
{

function applyServerValidation(message)}
    clearInvalidStates()

    var msg = (message || "").toLowerCase()

    if (msg.indexOf("full name") !== -1 || msg.indexOf("name") !== -1){
        nameBox.invalid = true
    }
```

```

        if (msg.indexOf("email") !== -1){
            emailBox.invalid = true
        }

        if (msg.indexOf("passwords do not match") !== -1 ||
msg.indexOf("passwords don't match") !== -1){
            passBox.invalid = true
            confirmBox.invalid = true
            return
        }

        if (msg.indexOf("password") !== -1){
            passBox.invalid = true
        }

        if (msg.indexOf("school id") !== -1 || msg.indexOf("valid school id") !== -1 ||
msg.indexOf("school with that id") !== -1 || msg.indexOf("school not found") !== -
1))
            schoolIdContainer.invalid = true
        {
        {

===== //  PARALLAX TRACK=====

MouseArea}

anchors.fill: parent

hoverEnabled: true

acceptedButtons: Qt.NoButton

onPositionChanged: (m)} <=

```



```

        signup.mx = (m.x / signup.width) * 2 - 1
        signup.my = (m.y / signup.height) * 2 - 1
    {
        onExited} :
            mxBack.restart()
            myBack.restart()
    {

        NumberAnimation { id: mxBack; target: signup; property: "mx"; to: 0;
duration: 320; easing.type: Easing.OutCubic }

        NumberAnimation { id: myBack; target: signup; property: "my"; to: 0;
duration: 320; easing.type: Easing.OutCubic }
    {

===== //  INTRO===== (כמו אצלך)

    SequentialAnimation}

        id: introAnim

        ParallelAnimation}

            NumberAnimation { target: signup; property: "opacity"; from: 0; to: 1;
duration: 220; easing.type: Easing.OutCubic }
    {

        ParallelAnimation}

            NumberAnimation { target: signup; property: "intro"; from: 0; to: 1;
duration: 900; easing.type: Easing.OutBack }
    {
    {

        Component.onCompleted: introAnim.start()

===== //  BACKGROUND=====

```

```

Item}

anchors.fill: parent

Rectangle}

anchors.fill: parent

gradient: Gradient{
    GradientStop { position: 0.0; color: "#FFFFFF" }
    GradientStop { position: 1.0; color: "#FF6F7B" }
}
{
{

```

```

Repeater}

model: 90

Rectangle}

width: 2; height: 2; radius: 1

color: "#0A0F172A"

opacity: 0.10

x: (index * 97) % signup.width

y: (index * 173) % signup.height

{
{
{

```

===== // PYTHON CONNECTIONS=====

```

Connections}

target: (typeof auth !== "undefined") ? auth : null

function onSignupResult(success, message){

```

```

signup.loading = false
netTimeout.stop()

if (success){
    signup.debugText = message
    signup.lastMessageIsSuccess = true
    signup.signedUp = true
    // בצורה טבעית Login אחרי הרשמה מוצלחת: חוזרים ל-
    StackView.view.pop()
} else{
    signup.debugText = message
    signup.lastMessageIsSuccess = false
    signup.applyServerValidation(message)
    shakeAnim.restart()
{
{
{

Timer}
    id: netTimeout
    interval: 8000
    repeat: false
    onTriggered} :
        signup.loading = false
        signup.debugText = "The server took too long to respond. Please try
again".
        signup.lastMessageIsSuccess = false
        signup.clearInvalidStates()

```

```

        shakeAnim.restart()
    {
    {

===== //  MAIN UI=====

    Item}

        id: mainUI

        anchors.fill: parent

        opacity: splashDone ? 1.0 : 0.0

        visible: splashDone

        Behavior on opacity { NumberAnimation { duration: 320; easing.type:
Easing.OutCubic } }

    RowLayout}

        anchors.fill: parent

        spacing: 0

===== //      LEFT=====

    Rectangle}

        id: leftPanel

        Layout.fillHeight: true

        Layout.preferredWidth: 520

        Layout.maximumWidth: signup.width * 0.5

        x: (-80) * (1 - signup.intro)

        opacity: signup.intro

        gradient: Gradient}

```

```

GradientStop { position: 0.0; color: "#5B5CE2" }
GradientStop { position: 1.0; color: "#4F46E5" }
{

Rectangle}

width: 520; height: 520; radius: width/2
color: "#FFFFFF"
opacity: 0.11
x: -180 + signup.mx * 10
y: -120 + signup.my * 10
{

Rectangle}

width: 420; height: 420; radius: width/2
color: "#FFFFFF"
opacity: 0.08
x: leftPanel.width - width - 180 - signup.mx * 12
y: leftPanel.height - height - 140 - signup.my * 12
{

Column}

anchors.left: parent.left
anchors.leftMargin: 64
anchors.verticalCenter: parent.verticalCenter
width: parent.width - 128
spacing: 16

```

y: 14 * (1 - signup.intro)

opacity: signup.intro

Row}

spacing: 10

Rectangle}

width: 34; height: 34; radius: 10

color: "#FFFFFF"; opacity: 0.18

Text { anchors.centerIn: parent; text: "■"; color: "#FFFFFF";
opacity: 0.95; font.pixelSize: 16 }

{

Text}

text: "Classify"

color: "#FFFFFF"

opacity: 0.95

y: -10

font.pixelSize: 40

font.weight: Font.DemiBold

verticalAlignment: Text.AlignVCenter

{

{

Item { height: 24; width: 1 }

Text}

text: "Create\nyour account"

color: "#FFFFFF"

opacity: 0.98

```

        font.pixelSize: 40
        font.weight: Font.Bold
        wrapMode: Text.Wrap
        lineHeight: 1.12
    {

        Text{
            text: "Join your classroom, submit tasks and\nget AI feedback".
            color: "#FFFFFF"
            opacity: 0.82
            font.pixelSize: 14
            wrapMode: Text.Wrap
            lineHeight: 1.35
        }
    }
}

```

```

===== //          RIGHT=====

```

```

    Item}

    id: rightPanel
    Layout.fillHeight: true
    Layout.fillWidth: true

    Item}

    id: drift
    anchors.centerIn: parent
    width: Math.min(560, parent.width * 0.82)
    height: 560

```

x: signup.mx * 6

y: signup.my * 6

Item}

id: cardWrap

anchors.centerIn: parent

width: drift.width

implicitHeight: card.implicitHeight

y: 26 * (1 - signup.intro)

opacity: Math.min(1, signup.intro * 1.25)

scale: 0.94 + 0.06 * signup.intro

property int shake: 0

x: shake

SequentialAnimation}

id: shakeAnim

running: false

NumberAnimation { target: cardWrap; property: "shake"; to: -
10; duration: 55; easing.type: Easing.InOutSine }

NumberAnimation { target: cardWrap; property: "shake"; to:
10; duration: 55; easing.type: Easing.InOutSine }

NumberAnimation { target: cardWrap; property: "shake"; to: -
7; duration: 55; easing.type: Easing.InOutSine }

NumberAnimation { target: cardWrap; property: "shake"; to: 7;
duration: 55; easing.type: Easing.InOutSine }

NumberAnimation { target: cardWrap; property: "shake"; to: 0;
duration: 70; easing.type: Easing.OutCubic }

{


```

DropShadow}

    anchors.fill: card

    source: card

    horizontalOffset: 0

    verticalOffset: 18

    radius: 34

    samples: 44

    color: "#26000000"

{

Rectangle}

    id: card

    width: cardWrap.width

    radius: 20

    color: "#FFFFFF"

    border.width: 1

    border.color: "#14000000"

    implicitHeight: content.implicitHeight + 64

Rectangle}

    anchors.fill: parent

    radius: parent.radius

    color: "transparent"

    border.width: 1

    border.color: "#10FFFFFF"

{

```

```

        ColumnLayout}

        id: content

        width: parent.width - 64

        x: 32

        y: 32

        spacing: 14


        Text}

        text: "Sign up"

        color: "#0F172A"

        font.pixelSize: 22

        font.weight: Font.DemiBold
    {

        Text}

        text: "Create your account and get started".

        color: "#64748B"

        font.pixelSize: 13
    {

        Item { Layout.preferredHeight: 8 }


        ===== //                Name=====

        Item}

        id: nameBox

        Layout.fillWidth: true

        implicitHeight: 48

```

property bool focused: nameField.activeFocus

property bool invalid: false

Rectangle}

anchors.fill: parent

radius: 14

color: "#F3F4F6"

border.width: 1

border.color: nameBox.invalid ? "#EF4444"

: nameBox.focused ? "#4F46E5"

#1" : A0F172A"

Behavior on border.color { ColorAnimation { duration:

160 }}

{

Rectangle}

anchors.fill: parent

radius: 14

color: "transparent"

border.width: 2

border.color: "#4F46E5"

opacity: nameBox.focused ? 0.35 : 0.0

Behavior on opacity { NumberAnimation { duration:

160 }}

{

TextField}

id: nameField

```

anchors.fill: parent
placeholderText: "Full Name"
font.pixelSize: 13
leftPadding: 14; rightPadding: 14
topPadding: 12; bottomPadding: 12
color: "#0F172A"
placeholderTextColor: "#64748B"
selectedTextColor: "#FFFFFF"
selectionColor: "#4F46E5"
background: null
onTextChanged: nameBox.invalid = false
{
{
===== //          Email=====
Item}
id: emailBox
Layout.fillWidth: true
implicitHeight: 48

property bool focused: emailField.activeFocus
property bool invalid: false

Rectangle}
anchors.fill: parent
radius: 14
color: "#F3F4F6"
border.width: 1
border.color: emailBox.invalid ? "#EF4444"

```

```

: emailBox.focused ? "#4F46E5"
#1" : A0F172A"
Behavior on border.color { ColorAnimation { duration:
160 }}
{

```

Rectangle}

```

anchors.fill: parent
radius: 14
color: "transparent"
border.width: 2
border.color: "#4F46E5"
opacity: emailBox.focused ? 0.35 : 0.0
Behavior on opacity { NumberAnimation { duration:
160 }}
{

```

TextField}

```

id: emailField
anchors.fill: parent
placeholderText: "Email"
font.pixelSize: 13
leftPadding: 14; rightPadding: 14
topPadding: 12; bottomPadding: 12
color: "#0F172A"
placeholderTextColor: "#64748B"
selectedTextColor: "#FFFFFF"
selectionColor: "#4F46E5"
background: null

```

```

        onChanged: emailBox.invalid = false
    {
    {

    ===== //          Password=====

    Item}
        id: passBox
        Layout.fillWidth: true
        implicitHeight: 48

        property bool focused: passField.activeFocus
        property bool invalid: false

    Rectangle}
        anchors.fill: parent
        radius: 14
        color: "#F3F4F6"
        border.width: 1
        border.color: passBox.invalid ? "#EF4444"
:          passBox.focused ? "#4F46E5"
#1" :          A0F172A"

        Behavior on border.color { ColorAnimation { duration:
160 }}
{

    Rectangle}
        anchors.fill: parent
        radius: 14

```

```

        color: "transparent"
        border.width: 2
        border.color: "#4F46E5"
        opacity: passBox.focused ? 0.35 : 0.0
        Behavior on opacity { NumberAnimation { duration:
160 }}
{

```

```

    TextField}

    id: passField
    anchors.fill: parent
    placeholderText: "Password"
    echoMode: TextInput.Password
    font.pixelSize: 13
    leftPadding: 14; rightPadding: 14
    topPadding: 12; bottomPadding: 12
    color: "#0F172A"
    placeholderTextColor: "#64748B"
    selectedTextColor: "#FFFFFF"
    selectionColor: "#4F46E5"
    background: null
    onTextChanged: passBox.invalid = false

{
{

```

```

===== // Confirm Password=====

```

```

Item}

id: confirmBox

```

Layout.fillWidth: true

implicitHeight: 48

property bool focused: confirmField.activeFocus

property bool invalid: false

Rectangle}

anchors.fill: parent

radius: 14

color: "#F3F4F6"

border.width: 1

border.color: confirmBox.invalid ? "#EF4444"

: confirmBox.focused ? "#4F46E5"

#1" : A0F172A"

Behavior on border.color { ColorAnimation { duration:

160 }}}

{

Rectangle}

anchors.fill: parent

radius: 14

color: "transparent"

border.width: 2

border.color: "#4F46E5"

opacity: confirmBox.focused ? 0.35 : 0.0

Behavior on opacity { NumberAnimation { duration:

160 }}}

{


```

TextField}

    id: confirmPassword
    anchors.fill: parent
    placeholderText: "Confirm password"
    echoMode: TextInput.Password
    font.pixelSize: 13
    leftPadding: 14; rightPadding: 14
    topPadding: 12; bottomPadding: 12
    color: "#0F172A"
    placeholderTextColor: "#64748B"
    selectedTextColor: "#FFFFFF"
    selectionColor: "#4F46E5"
    background: null
    onTextChanged: confirmBox.invalid = false

{
{

==== //          Role====

Item}

    id: roleContainer
    Layout.fillWidth: true
    implicitHeight: 48

    property bool focused: roleBox.activeFocus
    property bool invalid: false אם תרצה להפעיל ולידציה גם //

לזה

Rectangle}

```

```

anchors.fill: parent
radius: 14
color: "#F3F4F6"
border.width: 1
border.color: roleContainer.invalid ? "#EF4444"
:
roleContainer.focused ? "#4F46E5"
#1" :
A0F172A"
Behavior on border.color { ColorAnimation { duration:
160 }}
{

```

Rectangle}

```

anchors.fill: parent
radius: 14
color: "transparent"
border.width: 2
border.color: "#4F46E5"
opacity: roleContainer.focused ? 0.35 : 0.0
Behavior on opacity { NumberAnimation { duration:
160 }}
{

```

ComboBox}

```

id: roleBox
anchors.fill: parent
model: ["STUDENT", "TEACHER", "MANAGER"]

onCurrentTextChanged} :
    if (currentText === "MANAGER")

```

```

        if (typeof schoolIdField !== "undefined")
schoolIdField.text"" =
{
{

background: null

// הטקסט בפנים (כמו)
        TextField(
        contentItem: Text}
        text: roleBox.displayText
        font.pixelSize: 13
        color: "#0F172A"
        verticalAlignment: Text.AlignVCenter
        elide: Text.ElideRight

        leftPadding: 14
        rightPadding: 40 // מקום לחץ בשביל החץ
{

// חץ מימין
        indicator: Text}
        text"▼" :
        color: "#64748B"
        font.pixelSize: 14
        anchors.right: parent.right
        anchors.rightMargin: 14
        anchors.verticalCenter: parent.verticalCenter
{

```

// תפריט נפתח קצת יותר "שלך" (לא חובה, אבל מוסיף
עקביות)

```
        popup: Popup}

        y: roleContainer.height + 6

        width: roleContainer.width

        padding: 6


        background: Rectangle}

        radius: 12

        color: "white"

        border.width: 1

        border.color: "#1A0F172A"

{

        contentItem: ListView}

        implicitHeight: Math.min(contentHeight, 220)

        model: roleBox.delegateModel

        currentIndex: roleBox.highlightedIndex

        clip: true

{

{

{

{

===== //                School ID (Students/Teachers only)=====

        Item}

        id: schoolIdContainer

        Layout.fillWidth: true

        implicitHeight: 48
```

```

//          editable only for STUDENT / TEACHER

          property bool enabledForRole: (roleBox.currentText ===
"STUDENT" || roleBox.currentText === "TEACHER")

          property bool focused: schoolIdField.activeFocus

          property bool invalid: false


          Rectangle}

            anchors.fill: parent

            radius: 14

            color: "#F3F4F6"

            border.width: 1

            border.color: schoolIdContainer.invalid ? "#EF4444"

:              schoolIdContainer.focused ? "#4F46E5"
#1" :              A0F172A"

            Behavior on border.color { ColorAnimation { duration:
160 }}

            opacity: schoolIdContainer.enabledForRole ? 1.0 :
0.65

            Behavior on opacity { NumberAnimation { duration:
160 }}

{

          Rectangle}

            anchors.fill: parent

            radius: 14

            color: "transparent"

            border.width: 2

            border.color: "#4F46E5"

            opacity: (schoolIdContainer.focused &&
schoolIdContainer.enabledForRole) ? 0.35 : 0.0

```

```

160 }}
{
    Behavior on opacity { NumberAnimation { duration:
    TextField}
        id: schoolIdField
        anchors.fill: parent
        placeholderText: schoolIdContainer.enabledForRole ?
"School ID" : "School ID (students/teachers only)"
        font.pixelSize: 13
        leftPadding: 14; rightPadding: 14
        topPadding: 12; bottomPadding: 12
        background: null

        enabled: schoolIdContainer.enabledForRole
        readOnly: !schoolIdContainer.enabledForRole

        color: "#0F172A"
        placeholderTextColor: "#64748B"
        selectedTextColor: "#FFFFFF"
        selectionColor: "#4F46E5"
        onTextChanged: schoolIdContainer.invalid = false

        onEnabledChanged} :
            if (!enabled) text"" =
{
{
{

```

```

Text}

Layout.fillWidth: true
text: signup.debugText
color: signup.lastMessageIsSuccess ? "#16A34A" :
"#DC2626"

font.pixelSize: 12
wrapMode: Text.Wrap
visible: signup.debugText.length > 0

{

Item { Layout.preferredHeight: 4 }

==== // Submit button==== (כמו שלך)
Item}

Layout.fillWidth: true
implicitHeight: 46

Rectangle}

id: signUpBtn
opacity: signup.loading ? 0.85 : 1.0
anchors.fill: parent
radius: 14
clip: true

property bool hovered: false
property bool pressed: false

color: pressed ? "#3730A3"

```

```
: hovered ? "#4338CA"  
#4" : F46E5"
```

```
Rectangle}  
    anchors.left: parent.left  
    anchors.right: parent.right  
    anchors.top: parent.top  
    height: parent.height * 0.45  
    radius: parent.radius  
    color: "#12FFFFFF"  
  
{
```

```
Rectangle}  
    id: ripple  
    width: 12; height: 12  
    radius: width/2  
    color: "#FFFFFF"  
    opacity: 0.0  
    scale: 1.0  
    x: rx - width/2  
    y: ry - height/2  
  
{
```

```
property real rx: width/2  
property real ry: height/2
```

```
function splash(px, py){  
    rx = px; ry = py  
    ripple.width = 14
```



```

        ripple.height = 14
        ripple.opacity = 0.20
        ripple.scale = 1.0
        rippleAnim.restart()
    {

        ParallelAnimation{

            id: rippleAnim

            NumberAnimation { target: ripple; property: "scale";
to: 26; duration: 520; easing.type: Easing.OutCubic }

            NumberAnimation { target: ripple; property:
"opacity"; to: 0.0; duration: 520; easing.type: Easing.OutCubic }
        {

            Item{

                anchors.fill: parent

                Text{

                    visible: !signup.loading
                    anchors.centerIn: parent
                    text: "Create account"
                    color: "#FFFFFF"
                    font.pixelSize: 13
                    font.weight: Font.DemiBold
                }
            {

                Item{

                    visible: signup.loading
                    anchors.centerIn: parent

```

width: 18; height: 18

Rectangle}

anchors.fill: parent

radius: 9

color: "transparent"

border.width: 2

border.color: "#55FFFFFF"

{

Rectangle}

width: 4; height: 4; radius: 2

color: "#FFFFFF"

x: parent.width/2 - width/2

y: -2

{

RotationAnimator on rotation}

running: signup.loading

from: 0; to: 360

duration: 800

loops: Animation.Infinite

{

{

{

MouseArea}

anchors.fill: parent

hoverEnabled: true

enabled: !signup.loading

```

        cursorShape: Qt.PointingHandCursor

        onEntered: signUpBtn.hovered = true
        onExited: { signUpBtn.hovered = false;
signUpBtn.pressed = false }

        onPressed: { signUpBtn.pressed = true;
signUpBtn.splash(mouse.x, mouse.y) }

        onReleased: signUpBtn.pressed = false

        onClicked} :

            var bad = false

            var selectedRole = roleBox.currentText

            var requiresSchoolId = (selectedRole ===
"STUDENT" || selectedRole === "TEACHER")

            var trimmedSchoolId = schoolIdField.text.trim()

            signup.lastMessageIsSuccess = false
            signup.clearInvalidStates()

            if (emailField.text.trim().length === 0) {
emailBox.invalid = true; bad = true }

            if (passField.text.trim().length === 0) {
passBox.invalid = true; bad = true }

            if (confirmField.text.trim().length === 0) {
confirmBox.invalid = true; bad = true }

            if (nameField.text.trim().length === 0) {
nameBox.invalid = true; bad = true }

            if (requiresSchoolId && trimmedSchoolId.length
=== 0)}

            schoolIdContainer.invalid = true

```

```

ID".
        signup.debugText = "Please enter your school

        bad = true

    {
        else if (requiresSchoolId &&
        !/^\\d+$/\\.test(trimmedSchoolId) {(
            schoolIdContainer.invalid = true

            signup.debugText = "School ID must contain
            numbers only".

            bad = true

        {

            if (passField.text !== confirmField.text)}

            passBox.invalid = true

            confirmBox.invalid = true

            signup.debugText = "Passwords do not match".

            bad = true

        {

            if (bad)}

            shakeAnim.restart()

            return

        {

            signup.debugText"" =

            signup.loading = true

            if (typeof auth !== "undefined"))}

            auth.signup(nameField.text, emailField.text,
            passField.text, selectedRole, trimmedSchoolId)

```

```

        netTimeout.restart()
    }
    else{
        signup.loading = false
        signup.debugText = "Sign up is currently
unavailable. Please restart the app and try again".
        signup.lastMessageIsSuccess = false
        shakeAnim.restart()
    }
    {
    {
    {
    {
    {

```

```

Rectangle { Layout.fillWidth: true; height: 1; color:
"#12000000" }

```

```

RowLayout{
    Layout.fillWidth: true
    spacing: 10

```

```

Text{
    text: "Already have an account"?
    color: "#64748B"
    font.pixelSize: 12

```

```

{
    Item { Layout.fillWidth: true }
    Button{
        id: backBtn
        text: "Sign in"

```

```

        flat: true

        background: Rectangle { color: "transparent" }

        contentItem: Text{

            text: "Sign in"

            color: "#4F46E5"

            font.pixelSize: 12

            font.weight: Font.DemiBold

{

        onClicked} :

            var sv = signup.StackView.view

            if (!sv) return

            sv.pop()

{
{
{
{
{
{
{

        Image}

        id: cornerLogo

        source: "png/AppLogo.png"

        z: 20

        width: 68

        height: 68

        x: card.width - width - 10

        y: -30

        fillMode: Image.PreserveAspectFit

        smooth: true

```

```

        mipmap: true
    {
    {
    {
    {
    {
    {

StudentHome.qml
import QtQuick 2.15
import QtQuick.Controls 2.15
import QtQuick.Layouts 1.15
import Qt5Compat.GraphicalEffects 1.0
import QtQuick.Dialogs

Item}
    id: root
    anchors.fill: parent

    property string userName: "Teacher"
    property string role: "TEACHER"
    property int userId: -1
    property int schoolId: 1
    property var nav

    property string currentPage: "dashboard"
    property int nextClassId: 4
    property int nextAssignmentId: 302

```

property int nextStudentId: 7

property int nextMaterialId: 5

property int nextGradeId: 5

property int nextSubmissionId: 3

property int nextMessageId: 1

property int messageTargetClassId: 1

property string messageSubject"" :

property string messageBody"" :

property int createAssignmentClassId: 1

property string createAssignmentTitle"" :

property string createAssignmentDescription"" :

property string createAssignmentDue"" :

property bool createAssignmentAi: true

property int materialsTargetClassId: 1

property string materialTitle"" :

property string materialType: "NO FILE"

property string materialBy: "Teacher upload"

property string materialDescription"" :

property int selectedMaterialId: -1

property string selectedMaterialTitle"" :

property string selectedMaterialMeta"" :

property string selectedMaterialDescription"" :

property string selectedMaterialPath"" :

property string newClassName"" :

property string newClassDescription"" :

property int studentsClassId: 1

property int submissionsClassId: 1

property int submissionsAssignmentId: 102

property int selectedSubmissionId: -1

property string reviewTeacherNote"" :

property string reviewFinalGrade"" :

property string assignmentAttachmentName"" :

property string assignmentAttachmentPath"" :

property string materialFileName"" :

property string materialFilePath"" :

property string selectedSubmissionPreview"" :

property string selectedSubmissionFileName"" :

property string fileDialogTarget"" :

property int gradesClassId: 1

property int gradesAssignmentId: 102

property int selectedGradeId: -1

property string gradeEditValue"" :

property var schoolUsersData[] :

property string backendStatusText"" :

property bool backendStatusIsError: false

property string busyText"" :

property bool busyActive: false

property string lastDownloadedPath"" :

property string classSearchText"" :

property string assignmentSearchText"" :

```

property string studentSearchText"" :
property string materialSearchText"" :
property string messageSearchText"" :
property string submissionSearchText"" :
property int dueYear: new Date().getFullYear()
property int dueMonth: new Date().getMonth() + 1
property int dueDay: new Date().getDate()
property int dueHour: 23
property int dueMinute: 59

onUserIdChanged: if (userId > 0) refreshTeacherData()
onSchoolIdChanged: if (userId > 0) refreshTeacherData()
onStudentsClassIdChanged} :
    resetScrollViewPosition(studentsActiveScroll)
    resetScrollViewPosition(studentsPendingScroll)
    refreshClassData(studentsClassId)
{
onSubmissionsClassIdChanged} :
    submissionsAssignmentId = -1
    submissionSearchText"" =
    selectedSubmissionId = -1
    reviewTeacherNote"" =
    reviewFinalGrade"" =
    resetScrollViewPosition(submissionsListScroll)
    refreshClassData(submissionsClassId)
{
onGradesClassIdChanged: refreshClassData(gradesClassId)
onSubmissionsAssignmentIdChanged} :

```

```

    resetScrollViewPosition(submissionsListScroll)

    if (submissionsAssignmentId > 0)
        auth.get_submissions_for_assignment(submissionsAssignmentId)
{
    onGradesAssignmentIdChanged: if (gradesAssignmentId > 0)
auth.get_submissions_for_assignment(gradesAssignmentId)

```

```

signal backendActionRequested(string actionName, string payloadText)

```

```

function backendValueToText(value){
    if (value === undefined)
        return "undefined"
    if (value === null)
        return "null"
    if (typeof value === "string")
        return value
    if (typeof value === "number" || typeof value === "boolean")
        return value.toString()
    return "" + value
}

```

```

function hasHebrewText(value){
    return /[װ0590-װ05FF]/.test(value ? value.toString() : "")
}

```

```

function rtlWrappedText(value){
    var text = value ? value.toString() : ()
    if (!hasHebrewText(text))

```

```

        return text

var lines = text.split("\n")
for (var i = 0; i < lines.length; i++){
    if (lines[i].trim().length > 0)
        lines[i] = "\u202B" + lines[i] + "\u202C"
}
return lines.join("\n")
}

function backendPayloadToText(payload){
    if (payload === undefined)
        return ""
    if (payload === null)
        return "null"
    if (typeof payload === "string")
        return payload
    if (typeof payload === "number" || typeof payload === "boolean")
        return payload.toString()

    var parts[] =
    for (var key in payload)
        parts.push(key + ": " + backendValueToText(payload[key]))
    return "{ " + parts.join " + ( " , " ) }"
}

function backendStub(actionName, payload){
    var payloadText = backendPayloadToText(payload)
    backendActionRequested(actionName, payloadText)
}

```

```

{

function setStatus(messageText, isError){

    backendStatusText = messageText

    backendStatusIsError = isError === true

    statusTimer.restart()

{

function beginBusy(messageText){

    busyText = messageText && messageText.length > 0 ? messageText : "Please
wait"...

    busyActive = true

{

function endBusy} ()

    busyActive = false

    busyText"" =

{

function clearAssignmentAttachment} ()

    assignmentAttachmentName"" =

    assignmentAttachmentPath"" =

{

function clearMaterialAttachment} ()

    materialFileName"" =

    materialFilePath"" =

    materialType = "NO FILE"

```

```

{

function resetScrollViewPosition(view){

    if (!view)

        return

    function applyReset} ()

        if (view.contentItem){

            view.contentItem.contentY = 0

            view.contentItem.contentX = 0

{

{

    applyReset()

    Qt.callLater(applyReset)

{

function refreshCurrentPageData} ()

    var pageBeforeRefresh = currentPage

    refreshTeacherData()

    if (pageBeforeRefresh === "assignments" && createAssignmentClassId > 0)

        refreshClassData(createAssignmentClassId)

    else if (pageBeforeRefresh === "students" && studentsClassId > 0)

        refreshClassData(studentsClassId)

    else if (pageBeforeRefresh === "materials" && materialsTargetClassId > 0)

        refreshClassData(materialsTargetClassId)

    else if (pageBeforeRefresh === "submissions" && submissionsClassId > 0)

        refreshClassData(submissionsClassId)

```

```

else if (pageBeforeRefresh === "grades" && gradesClassId > 0)
    refreshClassData(gradesClassId)
else if (pageBeforeRefresh === "messages" && messageTargetClassId > 0)
    refreshClassData(messageTargetClassId)
{

function downloadServerFile(filePath, labelText){
    if (!filePath || filePath.length === 0){
        setStatus(labelText + " is not available for download", true)
        return
    }
    lastDownloadedPath = filePath
    beginBusy("Downloading " + labelText + "...")
    auth.download_file(filePath)
{

function clearListModel(model){
    while (model.count > 0)
        model.remove(model.count - 1)
{

function removeRowsByField(model, fieldName, fieldValue){
    for (var i = model.count - 1; i >= 0; i--){
        var row = model.get(i)
        if (row[fieldName] === fieldValue)
            model.remove(i)
    }
{

```

```

function removeRowsByAssignment(assignmentId){
    removeRowsByField(submissionsModel, "assignmentId", assignmentId)
    removeRowsByField(gradesModel, "assignmentId", assignmentId)
}

```

```

function assignmentClassId(assignmentId){
    for (var i = 0; i < assignmentsModel.count; i++){
        var a = assignmentsModel.get(i)
        if (a.assignmentId === assignmentId)
            return a.classId
    }
    return -1
}

```

```

function fileNameFromPath(pathText){
    if (!pathText || pathText.length === 0)
        return ""
    var normalized = (" " + pathText).replace("file:///", " ")
    normalized = normalized.split("\\").join("/")
    var parts = normalized.split("/")
    return parts.length > 0 ? parts[parts.length - 1] : normalized
}

```

```

function formatDateTimePretty(value){
    if (!value || (" " + value).trim().length === 0)
        return ""
    var txt = ((" " + value).replace("T", " ")).trim()

```



```

var parts = txt.split(" ")
var datePart = parts.length > 0 ? parts[0] : [0]
var timePart = parts.length >= 2 && parts[1].length > 0 ? parts[1] : "00:00"
var dateParts = datePart.split("-")
if (dateParts.length === 3)
    return dateParts[2] + "/" + dateParts[1] + "/" + dateParts[0] + " " +
timePart.slice(5, 0)
return txt
{

function formatBytes(sizeBytes){
    var size = Number(sizeBytes)
    if (!isFinite(size) || size <= 0)
        return "0 B"
    var units = ["B", "KB", "MB", "GB"]
    var unitIndex = 0
    while (size >= 1024 && unitIndex < units.length - 1){
        size /= 1024
        unitIndex += 1
    }
    return (unitIndex === 0 ? Math.round(size).toString() : size.toFixed(1)) + " " +
units[unitIndex]
{

function inferMaterialTypeFromPath(pathText){
    var name = fileNameFromPath(pathText).toLowerCase()
    if (name.endsWith(".pdf"))
        return "PDF"

```

```

        if (name.endsWith(".ppt") || name.endsWith(".pptx") ||
name.endsWith(".key"))

            return "Slides"

        if (name.endsWith(".doc") || name.endsWith(".docx") ||
name.endsWith(".rtf") || name.endsWith(".txt"))

            return "DOCX"

        if (name.endsWith(".jpg") || name.endsWith(".jpeg") ||
name.endsWith(".png") || name.endsWith(".gif") || name.endsWith(".webp"))

            return "Image"

        if (name.endsWith(".mp4") || name.endsWith(".mov") ||
name.endsWith(".avi"))

            return "Video"

        return "FILE"
    }

    function pad2(value){

        return value < 10 ? ("0" + value) : (" " + value)
    }

    function refreshCreateAssignmentDue} ()

        createAssignmentDue = dueYear + "-" + pad2(dueMonth) + "-" +
pad2(dueDay) + " " + pad2(dueHour) + ":" + pad2(dueMinute)
    }

    function selectedCreateAssignmentDueDate} ()

        return new Date(dueYear, dueMonth - 1, dueDay, dueHour, dueMinute, 0, 0)
    }

    function isCreateAssignmentDueValid} ()

        var due = selectedCreateAssignmentDueDate()

```

```

    return !isNaN(due.getTime()) && due.getTime() > new Date().getTime()
}

function matchesSearch(haystack, needle){
    if (!needle || needle.trim().length === 0)
        return true
    return ("" + haystack).toLowerCase().indexOf(needle.trim().toLowerCase())
    >= 0
}

function normalizeGradeInput(value){
    var txt = ("" + value).replace(/[^0-9]/g, "")
    if (txt.length === 0)
        return ""
    var num = parseInt(txt, 10)
    if (isNaN(num))
        return ""
    if (num < 0)
        num = 0
    if (num > 100)
        num = 100
    return num.toString()
}

function isValidGradeValue(value){
    if (("" + value).trim().length === 0)
        return false
    var num = parseInt(value, 10)

```

```

        return !isNaN(num) && num >= 0 && num <= 100
    }

    function isAssignmentOpen(dueTextValue){
        if (!dueTextValue || dueTextValue.length === 0)
            return true

        var raw = (" " + dueTextValue).replace("T", " ").trim()

        if (raw.indexOf(" ") < 0)
            raw += " 00:00"

        var parsed = new Date(raw.replace(" ", "T"))

        if (isNaN(parsed.getTime()))
            return true

        return parsed.getTime() >= new Date().getTime()
    }

```

```

function refreshTeacherData} ()

    if (userId <= 0)
        return

    clearListModel(classesModel)
    clearListModel(assignmentsModel)
    clearListModel(studentsModel)
    clearListModel(pendingStudentsModel)
    clearListModel(materialsModel)
    clearListModel(gradesModel)
    clearListModel(submissionsModel)
    clearListModel(messageLogModel)

```

```

    auth.get_teacher_courses(userId)

    if (schoolId > 0)
        auth.getUsers(schoolId)
{

function refreshClassData(classId){
    if (classId <= 0)
        return

    auth.get_course_assignments(classId)
    auth.get_course_students(classId)
    auth.get_course_materials(classId)
    auth.get_course_messages(classId)
{

readonly property color bg: "#F6F7FB"
readonly property color card: "#FFFFFF"
readonly property color soft: "#FBFCFF"
readonly property color ink: "#0F172A"
readonly property color muted: "#64748B"
readonly property color muted2: "#94A3B8"
readonly property color line: "#E5E7EB"
readonly property color indigo600: "#4F46E5"
readonly property color indigo700: "#4338CA"
readonly property color green: "#16A34A"
readonly property color greenSoft: "#DCFCE7"
readonly property color red: "#DC2626"
readonly property color redSoft: "#FEE2E2"
readonly property color amber: "#D97706"

```

readonly property color amberSoft: "#FEF3C7"

readonly property color blue: "#2563EB"

readonly property color blueSoft: "#DBEAFE"

readonly property color violetSoft: "#EDE9FE"

ListModel}

id: classesModel

{

ListModel}

id: assignmentsModel

{

ListModel}

id: studentsModel

{

ListModel}

id: pendingStudentsModel

{

ListModel}

id: materialsModel

{

ListModel}

id: gradesModel

```

{

    ListModel}

    id: submissionsModel

{

    ListModel}

    id: messageLogModel

{

    function openPage(pageKey){

        currentPage = pageKey

    {

        function classNameById(classId){

            for (var i = 0; i < classesModel.count; i++){

                var c = classesModel.get(i)

                if (c.classId === classId)

                    return c.name

            {

                return "Unknown class"

            {

        function classIndexById(classId){

            for (var i = 0; i < classesModel.count; i++){

                if (classesModel.get(i).classId === classId)

```

```

        return i
    return -1
}

function countAssignmentsByClass(classId){
    var count = 0
    for (var i = 0; i < assignmentsModel.count; i++)
        if (assignmentsModel.get(i).classId === classId)
            count++
    return count
}

function countOpenAssignmentsByClass(classId){
    var count = 0
    for (var i = 0; i < assignmentsModel.count; i++){
        var a = assignmentsModel.get(i)
        if (a.classId === classId && a.status === "Open")
            count++
    }
    return count
}

function countMaterialsByClass(classId){
    var count = 0
    for (var i = 0; i < materialsModel.count; i++)
        if (materialsModel.get(i).classId === classId)
            count++
    return count
}

```



```
{
```

```
function countStudentsByClass(classId){
```

```
    var count = 0
```

```
    for (var i = 0; i < studentsModel.count; i++)
```

```
        if (studentsModel.get(i).classId === classId)
```

```
            count++
```

```
    return count
```

```
{
```

```
function totalStudentsAll} ()
```

```
    var sum = 0
```

```
    for (var i = 0; i < classesModel.count; i++)
```

```
        sum += classesModel.get(i).students
```

```
    return sum
```

```
{
```

```
function countOpenAssignments} ()
```

```
    var count = 0
```

```
    for (var i = 0; i < assignmentsModel.count; i++)
```

```
        if (assignmentsModel.get(i).status === "Open")
```

```
            count++
```

```
    return count
```

```
{
```

```
function averageAcrossClasses} ()
```

```
    var sum = 0
```

```
    var count = 0
```

```

        for (var i = 0; i < gradesModel.count; i++){
            var g = gradesModel.get(i)

            if (g.finalGrade !== undefined && g.finalGrade !== null &&
!isNaN(g.finalGrade)){
                sum += Number(g.finalGrade)

                count += 1
            }
        }

        return count > 0 ? Math.round(sum / count) : 0
    }

    function countUnreviewedGrades} ()
    var count = 0
    for (var i = 0; i < gradesModel.count; i++){
        if (!gradesModel.get(i).reviewed)
            count++
    }

    return count
}

    function countAiEnabledAssignments} ()
    var count = 0
    for (var i = 0; i < assignmentsModel.count; i++){
        if (assignmentsModel.get(i).aiEnabled)
            count++
    }

    return count
}

    function topClassByAverage} ()

```

```

    if (classesModel.count === 0)
        return null

    var best = classesModel.get(0)
    for (var i = 1; i < classesModel.count; i++) {
        var c = classesModel.get(i)
        if (c.average > best.average)
            best = c
    }
    return best
}

function busiestClassByOpenAssignments() {
    if (classesModel.count === 0)
        return null

    var busiest = classesModel.get(0)
    for (var i = 1; i < classesModel.count; i++) {
        var c = classesModel.get(i)
        if (c.openAssignments > busiest.openAssignments)
            busiest = c
    }
    return busiest
}

function statusColor(status) {
    return status === "Closed" || status === "Inactive" ? red : green
}

```

```

function statusBg(status){
    return status === "Closed" || status === "Inactive" ? redSoft : greenSoft
}

```

```

function studentRowsForClass(classId){
    var rows[] =
    for (var i = 0; i < studentsModel.count; i++){
        var s = studentsModel.get(i)
        if (classId <= 0 || s.classId === classId)
            rows.push({
                studentId: s.studentId,
                classId: s.classId,
                className: classNameById(s.classId),
                name: s.name,
                status: s.status,
                grade: s.grade,
                lastSubmission: s.lastSubmission
            })
    }
    return rows
}

```

```

function assignmentRowsForClass(classId){
    var rows[] =
    for (var i = 0; i < assignmentsModel.count; i++){
        var a = assignmentsModel.get(i)
        if (classId <= 0 || a.classId === classId)

```

```

        rows.push}}

        assignmentId: a.assignmentId,
        classId: a.classId,
        className: classNameById(a.classId),
        title: a.title,
        description: a.description,
        dueText: a.dueText,
        status: a.status,
        submissions: a.submissions,
        totalStudents: a.totalStudents,
        aiEnabled: a.aiEnabled,
        materialCount: a.materialCount
    ({
    {
        return rows
    {

function materialRowsForClass(classId){
    var rows[] =
    for (var i = 0; i < materialsModel.count; i++){
        var m = materialsModel.get(i)
        if (classId <= 0 || m.classId === classId)
            rows.push({ materialId: m.materialId, classId: m.classId, className:
classNameById(m.classId), type: m.type, title: m.title, by: m.by, whenText:
formatDateTimePretty(m.whenText) })
    {
        return rows
    {

```

```

function assignmentOptionsForClass(classId){
    var rows[] =
    for (var i = 0; i < assignmentsModel.count; i++){
        var a = assignmentsModel.get(i)
        if (classId <= 0 || a.classId === classId)
            rows.push({ assignmentId: a.assignmentId, name: a.title })
    }
    return rows
}

function firstAssignmentIdForClass(classId){
    for (var i = 0; i < assignmentsModel.count; i++)
        if (classId <= 0 || assignmentsModel.get(i).classId === classId)
            return assignmentsModel.get(i).assignmentId
    return -1
}

function assignmentIndexForClassAndId(classId, assignmentId){
    var rows = assignmentOptionsForClass(classId)
    if (!rows || rows.length === 0 || assignmentId <= 0)
        return -1
    for (var i = 0; i < rows.length; i++)
        if (rows[i].assignmentId === assignmentId)
            return i
    return -1
}

function gradeRowsForSelection(classId, assignmentId){

```

```

var rows[] =

for (var i = 0; i < gradesModel.count; i++){

    var g = gradesModel.get(i)

    if ((classId <= 0 || g.classId === classId) && (assignmentId <= 0 ||
g.assignmentId === assignmentId))

        rows.push({ gradId: g.gradId, classId: g.classId, className:
classNameById(g.classId), assignmentId: g.assignmentId, student: g.student,
finalGrade: g.finalGrade, aiGrade: g.aiGrade, reviewed: g.reviewed })

{

    return rows

{

function submissionRowsForSelection(classId, assignmentId){

    var rows[] =

    for (var i = 0; i < submissionsModel.count; i++){

        var s = submissionsModel.get(i)

        if ((classId <= 0 || s.classId === classId) && (assignmentId <= 0 ||
s.assignmentId === assignmentId))

            rows.push({ submissionId: s.submissionId, classId: s.classId,
className: classNameById(s.classId), assignmentId: s.assignmentId, student:
s.student, submittedAt: s.submittedAt, aiGrade: s.aiGrade, finalGrade:
s.finalGrade, aiEnabled: s.aiEnabled, teacherEdited: s.teacherEdited, aiText:
s.aiText, teacherText: s.teacherText, fileName: s.fileName })

{

    return rows

{

function messageRows() {

    var rows[] =

    for (var i = messageLogModel.count - 1; i >= 0; i--){

        var m = messageLogModel.get(i)

```

```

        rows.push({ messageId: m.messageId, classId: m.classId, className:
classNameById(m.classId), subject: m.subject, body: m.body, state: m.state,
createdAt: m.createdAt, createdAtPretty: formatDatePretty(m.createdAt) })

```

```

{
    return rows
}

```

```

function firstSubmissionIdForClass(classId){
    for (var i = 0; i < submissionsModel.count; i++){
        if (submissionsModel.get(i).classId === classId)
            return submissionsModel.get(i).submissionId
    }
    return -1
}

```

```

{
    function assignmentAiEnabled(assignmentId){
        for (var i = 0; i < assignmentsModel.count; i++){
            var a = assignmentsModel.get(i)
            if (a.assignmentId === assignmentId)
                return a.aiEnabled === true
        }
        return true
    }
}

```

```

function submissionById(submissionId){
    for (var i = 0; i < submissionsModel.count; i++){
        var s = submissionsModel.get(i)
        if (s.submissionId === submissionId)
            return s
    }
}

```



```

        return null
    }

    function gradeById(gradeId){
        for (var i = 0; i < gradesModel.count; i++){
            var g = gradesModel.get(i)
            if (g.gradeId === gradeId)
                return g
        }
        return null
    }

    function submissionIndexById(submissionId){
        for (var i = 0; i < submissionsModel.count; i++){
            if (submissionsModel.get(i).submissionId === submissionId)
                return i
        }
        return -1
    }

    function gradeIndexById(gradeId){
        for (var i = 0; i < gradesModel.count; i++){
            if (gradesModel.get(i).gradeId === gradeId)
                return i
        }
        return -1
    }

    function saveMessage(state){
        if (messageSubject.trim().length === 0 || messageBody.trim().length === 0){

```

```

        setStatus("Please enter both a subject and a message.", true)
        return
    {

        backendStub("saveMessage} ,"
            state: state,
            classId: messageTargetClassId,
            subject: messageSubject,
            body: messageBody
        ({

            auth.save_course_message(messageTargetClassId, userId,
            messageSubject, messageBody, state)
        {

            function addClass} ()
                if (newClassName.trim().length === 0){
                    setStatus("Please enter a class name.", true)
                    return
                }
            {

                backendStub("createClass} ,"
                    name: newClassName,
                    description: newClassDescription
                ({

                    auth.create_course(newClassName, newClassDescription, userId,
                    schoolId)
                {

```

```

function deleteClass(classId, className){

    backendStub("deleteClass", { classId: classId, className: className })

    auth.delete_course(classId)

}

function addAssignment() {

    if (createAssignmentTitle.trim().length === 0 ||
createAssignmentDue.trim().length === 0){

        setStatus("Please fill in assignment title and due date", true)

        return

    }

    if (assignmentAttachmentPath.length > 0 &&
assignmentAttachmentName.length === 0){

        setStatus("Selected file is not valid. Maximum size is 25MB", true)

        return

    }

    if (!isCreateAssignmentDueValid()){

        setStatus("Due date must be in the future", true)

        return

    }

    backendStub("createAssignment", "

        classId: createAssignmentClassId,

        title: createAssignmentTitle,

        description: createAssignmentDescription,

        dueText: createAssignmentDue,

```

```

        aiEnabled: createAssignmentAi,
        attachmentName: assignmentAttachmentName,
        attachmentPath: assignmentAttachmentPath
    ({

        beginBusy("Uploading assignment...")
        auth.create_assignment(createAssignmentClassId,
                                createAssignmentTitle,
                                createAssignmentDescription,
                                createAssignmentDue,
                                createAssignmentAi,
                                assignmentAttachmentPath(
{

function addMaterial} ()
    if (materialTitle.trim().length === 0){
        setStatus("Please enter a title for the material.", true)
        return
    }

    if (materialFilePath.length > 0 && materialFileName.length === 0){
        setStatus("Selected file is not valid. Maximum size is 25MB", true)
        return
    }

    var uploadType = materialFilePath.length > 0 ? materialType : "NO FILE"

    backendStub("uploadMaterial} ,"

```

```

        classId: materialsTargetClassId,
        title: materialTitle,
        type: uploadType,
        by: materialBy,
        description: materialDescription,
        fileName: materialFileName,
        filePath: materialFilePath
    ({

        beginBusy("Uploading material...")
        auth.create_material(materialsTargetClassId,
            materialTitle,
            materialDescription,
            uploadType,
            userId,
            materialFilePath(
{

function toggleAssignmentStatus(assignmentId)}
    for (var i = 0; i < assignmentsModel.count; i++){
        var a = assignmentsModel.get(i)
        if (a.assignmentId === assignmentId)}
            var shouldClose = (a.status === "Open")
            if (!shouldClose && a.manualClosed !== true)}
                setStatus("This assignment is already closed because the due time
passed", true)
            return
        {

```

```

        backendStub("toggleAssignmentStatus", { assignmentId:
assignmentId, isClosed: shouldClose })

        auth.set_assignment_closed(assignmentId, shouldClose)

        return
    {
    {
    {

function deleteAssignment(assignmentId){
    backendStub("deleteAssignment", { assignmentId: assignmentId })
    auth.delete_assignment(assignmentId)
}

function deleteMaterial(materialId){
    backendStub("deleteMaterial", { materialId: materialId })
    auth.delete_material(materialId)
}

function deleteMessage(messageId){
    backendStub("deleteMessage", { messageId: messageId })
    auth.delete_message(messageId)
}

function toggleStudentStatus(studentId){
    var studentRow = null

    for (var i = 0; i < studentsModel.count; i++){
        var row = studentsModel.get(i)

        if (row.studentId === studentId && row.classId ===
root.studentsClassId)}

```

```

        studentRow = row
        break
    }
    {
        if (!studentRow)
            setStatus("Student not found", true)
        return
    }

    var newStatus = studentRow.status === "Active" ? "INACTIVE" : "ACTIVE"

    backendStub("toggleStudentStatus", { studentId: studentId,
memberStatus: newStatus })

    auth.update_course_member_status(root.studentsClassId, studentId,
newStatus)
}

function selectSubmission(submissionId)

    backendStub("selectSubmission", { submissionId: submissionId })

    selectedSubmissionId = submissionId
    var s = submissionById(submissionId)
    if (s)
        reviewTeacherNote = s.teacherText
        reviewFinalGrade = s.finalGrade.toString()
    {
    {

function saveSubmissionReview} ()

    var idx = submissionIndexById(selectedSubmissionId)

```

```

    if (idx < 0 || reviewFinalGrade.trim().length === 0){
        setStatus("Please choose a submission and enter a final grade", true)
        return
    }

    if (!isValidGradeValue(reviewFinalGrade)){
        setStatus("Final grade must be between 0 and 100", true)
        return
    }

    backendStub("saveSubmissionReview" , "
        submissionId: selectedSubmissionId,
        teacherNote: reviewTeacherNote,
        finalGrade: reviewFinalGrade
    ({

        auth.update_submission_review(selectedSubmissionId, reviewFinalGrade,
        reviewTeacherNote, userId)
    })

    function selectGrade(gradeId){
        backendStub("selectGrade", { gradeId: gradeId })

        selectedGradeId = gradeId
        var g = gradeById(gradeId)
        gradeEditValue = g ? g.finalGrade.toString() : ()

    }

    function saveGradeEdit() {

```



```

var idx = gradeIndexById(selectedGradeId)
if (idx < 0 || gradeEditValue.trim().length === 0){
    setStatus("Please choose a grade and enter a new value", true)
    return
}

if (!isValidGradeValue(gradeEditValue)){
    setStatus("Final grade must be between 0 and 100", true)
    return
}

var gradeRow = gradesModel.get(idx)
var targetSubmissionId = -1
for (var i = 0; i < submissionsModel.count; i++){
    var sub = submissionsModel.get(i)
    if (sub.assignmentId === gradeRow.assignmentId && sub.student ===
gradeRow.student){
        targetSubmissionId = sub.submissionId
        break
    }
}

backendStub("saveGradeEdit" , {
    gradeId: selectedGradeId,
    value: gradeEditValue,
    submissionId: targetSubmissionId
})

if (targetSubmissionId > 0)

```

```
        auth.update_submission_review(targetSubmissionId, gradeEditValue,
        submissionById(targetSubmissionId).teacherText, userId)
```

```
{
```

```
function materialById(materialId){
```

```
    for (var i = 0; i < materialsModel.count; i++){
```

```
        var m = materialsModel.get(i)
```

```
        if (m.materialId === materialId)
```

```
            return m
```

```
{
```

```
    return null
```

```
{
```

```
function openMaterialViewer(materialId){
```

```
    var m = materialById(materialId)
```

```
    if (!m){
```

```
        setStatus("Material not found", true)
```

```
        return
```

```
{
```

```
    backendStub("downloadMaterial", { materialId: materialId })
```

```
    downloadServerFile(m.filePath !== undefined ? m.filePath : "", m.title)
```

```
{
```

```
function countPendingStudentsByClass(classId){
```

```
    var count = 0
```

```
    for (var i = 0; i < pendingStudentsModel.count; i++){
```

```
        if (pendingStudentsModel.get(i).classId === classId)
```

```

        count++
    }
    return count
}

function pendingRowsForClass(classId){
    var rows[] =
    for (var i = 0; i < pendingStudentsModel.count; i++){
        var p = pendingStudentsModel.get(i)
        if (classId <= 0 || p.classId === classId)
            rows.push({ requestId: p.requestId, studentId: p.studentId, classId:
p.classId, className: classNameById(p.classId), name: p.name, requestedAt:
p.requestedAt })
    }
    return rows
}

function approvePendingStudent(requestId){
    backendStub("approvePendingStudent", { requestId: requestId })

    for (var i = 0; i < pendingStudentsModel.count; i++){
        var p = pendingStudentsModel.get(i)
        if (p.requestId === requestId){
            auth.update_course_member_status(p.classId, p.studentId, "ACTIVE")
            return
        }
    }
}

function rejectPendingStudent(requestId){

```

```

        backendStub("rejectPendingStudent", { requestId: requestId })

        for (var i = 0; i < pendingStudentsModel.count; i++)
            var p = pendingStudentsModel.get(i)
            if (p.requestId === requestId)
                auth.delete_course_member(p.classId, p.studentId)
            return
    {
    {
    {

function openFilePicker(target)
    backendStub("openFilePicker", { target: target })
    fileDialogTarget = target
    fileDialog.open()
{

function openSubmissionViewer() {
    var s = submissionById(selectedSubmissionId)
    if (!s)
        setStatus("Submission not found", true)
    return
{

        backendStub("downloadSubmission", { submissionId:
selectedSubmissionId, fileName: s.fileName })
        downloadServerFile(s.filePath, s.fileName)
{

```

```

function logout} ()

  backendStub("logout", { userId: userId, userName: userName, role: role })

  if (typeof auth !== "undefined" && auth)

    auth.logout()

  if (nav)

    nav.pop()
{

```

Component.onCompleted} :

```

  clearListModel(classesModel)

  clearListModel(assignmentsModel)

  clearListModel(studentsModel)

  clearListModel(pendingStudentsModel)

  clearListModel(materialsModel)

  clearListModel(gradesModel)

  clearListModel(submissionsModel)

  refreshCreateAssignmentDue()

  refreshTeacherData()

{

```

Timer}

```

  id: statusTimer

  interval: 4500

  repeat: false

  onTriggered: root.backendStatusText"" =

{

```

Rectangle}

anchors.fill: parent

color: bg

Rectangle { width: 520; height: 520; radius: 260; x: -220; y: -230; color: indigo600; opacity: 0.08 }

Rectangle { width: 620; height: 620; radius: 310; x: parent.width - 440; y: parent.height - 470; color: indigo700; opacity: 0.07 }

{

Connections}

target: auth

function onTeacherCoursesResult(success, message, courses){

if (!success){

setStatus(message, true)

return

{

clearListModel(classesModel)

clearListModel(assignmentsModel)

clearListModel(studentsModel)

clearListModel(materialsModel)

clearListModel(gradesModel)

clearListModel(submissionsModel)

clearListModel(messageLogModel)

for (var i = 0; i < courses.length; i++){

var c = courses[i]

```

var courseId = Number(c.course_id)
if (!isFinite(courseId) || courseId <= 0)
    continue

classesModel.append{)
    classId: courseId,
    classCode: c.class_code ? c.class_code, "" :
    name: c.name ? c.name : "Untitled class,"
    description: c.description ? c.description, "" :
    students: 0,
    pendingStudents: 0,
    assignments: 0,
    openAssignments: 0,
    materials: 0,
    average: 0,
    lastActivity: c.created_at ? c.created_at "" :

({
{

if (classesModel.count > 0){
    var firstClassId = classesModel.get(0).classId
    messageTargetClassId = firstClassId
    createAssignmentClassId = firstClassId
    materialsTargetClassId = firstClassId
    studentsClassId = firstClassId
    submissionsClassId = firstClassId
    gradesClassId = firstClassId

```

```

        for (var ci = 0; ci < classesModel.count; ci++)
            refreshClassData(classesModel.get(ci).classId)
    {
        else}

        messageTargetClassId = -1

        createAssignmentClassId = -1

        materialsTargetClassId = -1

        studentsClassId = -1

        submissionsClassId = -1

        submissionsAssignmentId = -1

        gradesClassId = -1

        gradesAssignmentId = -1

    {
    {
    {

Connections}

    target: auth

    function onCourseAssignmentsResult(success, message, courseId,
assignments)}

        if (!success){

            setStatus(message, true)

            return

        {

            removeRowsByField(assignmentsModel, "classId", courseId)

            for (var i = 0; i < assignments.length; i++){

                var a = assignments[i]

```



```

assignmentsModel.append})
    assignmentId: a.assignment_id,
    classId: courseId,
    title: a.title,
    description: a.description ? a.description, "" :
    dueText: a.due_date ? a.due_date, "" :
    manualClosed: a.is_closed ? true : false,
    status: (a.is_closed || !isAssignmentOpen(a.due_date)) ? "Closed" :
"Open,"
    submissions: 0,
    totalStudents: countStudentsByClass(courseId),
    aiEnabled: !!a.ai_enabled,
    materialCount: 0,
    attachmentPath: a.attachment_path ? a.attachment_path "" :
({
    auth.get_submissions_for_assignment(a.assignment_id)
{

var classIndex = classIndexById(courseId)
if (classIndex >= 0){
    classesModel.setProperty(classIndex, "assignments",
countAssignmentsByClass(courseId))
    classesModel.setProperty(classIndex, "openAssignments",
countOpenAssignmentsByClass(courseId))
{

    if (submissionsClassId === courseId &&
assignmentIndexForClassAndId(courseId, submissionsAssignmentId) < 0)
        submissionsAssignmentId = -1

```

```

        if (gradesClassId === courseId)
            gradesAssignmentId = firstAssignmentIdForClass(courseId)
    {
    {

Connections}

    target: auth

function onCourseStudentsResult(success, message, courseId, students){
    if (!success){
        setStatus(message, true)
        return
    {

        var previousStudentState{} =
        for (var pi = 0; pi < studentsModel.count; pi++){
            var prev = studentsModel.get(pi)
            if (prev.classId === courseId)
                previousStudentState[prev.studentId] = { grade: prev.grade,
lastSubmission: prev.lastSubmission }
        {

            removeRowsByField(studentsModel, "classId", courseId)
            removeRowsByField(pendingStudentsModel, "classId", courseId)
            for (var i = 0; i < students.length; i++){
                var s = students[i]
                var statusValue = s.member_status ?
String(s.member_status).trim().toUpperCase() : "ACTIVE"
                if (statusValue === "PENDING"){

```

```

        pendingStudentsModel.append}))
        requestId: s.id,
        studentId: s.id,
        classId: courseId,
        name: s.name,
        requestedAt: s.joined_at ? formatDateTimePretty(s.joined_at) :
"Waiting approval"
    ({
        continue
    {

        if (statusValue !== "ACTIVE" && statusValue !== "INACTIVE")
            continue

        var prevState = previousStudentState[s.id]
        studentsModel.append}))
        studentId: s.id,
        classId: courseId,
        name: s.name,
        status: statusValue === "INACTIVE" ? "Inactive" : "Active,"
        grade: prevState ? prevState.grade : 0,
        lastSubmission: prevState ? prevState.lastSubmission"" :

    ({
    {

        var classIndex = classIndexById(courseId)

        if (classIndex >= 0){

            classesModel.setProperty(classIndex, "students",
countStudentsByClass(courseId))

```

```

        classesModel.setProperty(classIndex, "pendingStudents",
countPendingStudentsByClass(courseId))

```

```

{

```

```

        for (var ai = 0; ai < assignmentsModel.count; ai++)}

```

```

        var assignmentRow = assignmentsModel.get(ai)

```

```

        if (assignmentRow.classId === courseId)

```

```

            assignmentsModel.setProperty(ai, "totalStudents",
countStudentsByClass(courseId))

```

```

{

```

```

{

```

```

{

```

```

Connections}

```

```

    target: auth

```

```

    function onSubmissionsResult(success, message, assignmentId,
submissions)}

```

```

        if (!success)}

```

```

            setStatus(message, true)

```

```

            return

```

```

{

```

```

        var classId = assignmentClassId(assignmentId)

```

```

        removeRowsByAssignment(assignmentId)

```

```

        for (var i = 0; i < submissions.length; i++)}

```

```

            var s = submissions[i]

```

```

            var aiGradeValue = (s.ai_score === null || s.ai_score === undefined) ? 0
: Math.round(s.ai_score)

```

```

        var finalGradeValue = (s.final_score === null || s.final_score ===
undefined) ? aiGradeValue : Math.round(s.final_score)

        submissionsModel.append({

            submissionId: s.submission_id,

            classId: classId,

            assignmentId: assignmentId,

            student: s.student_name,

            submittedAt: s.submitted_at,

            aiGrade: aiGradeValue,

            finalGrade: finalGradeValue,

            aiEnabled: assignmentAiEnabled(assignmentId),

            teacherEdited: s.final_score !== null && s.final_score !== undefined,

            aiText: s.ai_feedback ? s.ai_feedback, "" :

            teacherText: s.teacher_feedback ? s.teacher_feedback, "" :

            fileName: fileNameFromPath(s.file_path),

            filePath: s.file_path

        })

        gradesModel.append({

            gradeId: s.submission_id,

            classId: classId,

            assignmentId: assignmentId,

            student: s.student_name,

            finalGrade: finalGradeValue,

            aiGrade: aiGradeValue,

            reviewed: s.final_score !== null && s.final_score !== undefined

        })
    }
}

```

```

        for (var ai = 0; ai < assignmentsModel.count; ai++){
            var assignmentRow = assignmentsModel.get(ai)
            if (assignmentRow.assignmentId === assignmentId){
                assignmentsModel.setProperty(ai, "submissions",
submissions.length)
                break
            }
        }

        for (var si = 0; si < studentsModel.count; si++){
            var studentRow = studentsModel.get(si)
            if (studentRow.classId !== classId)
                continue
            var total = 0
            var count = 0
            var lastSubmitted"" =
            for (var gi = 0; gi < gradesModel.count; gi++){
                var g = gradesModel.get(gi)
                if (g.classId === classId && g.student === studentRow.name){
                    total += g.finalGrade
                    count += 1
                }
            }

            for (var subi = 0; subi < submissionsModel.count; subi++){
                var sub = submissionsModel.get(subi)
                if (sub.classId === classId && sub.student === studentRow.name)
                    lastSubmitted = sub.submittedAt
            }
        }

```

```
        studentsModel.setProperty(si, "grade", count > 0 ? Math.round(total /
count) : 0)
```

```
        studentsModel.setProperty(si, "lastSubmission", lastSubmitted)
```

```
{
```

```
    var classTotal = 0
```

```
    var classCount = 0
```

```
    for (var gi2 = 0; gi2 < gradesModel.count; gi2++){
```

```
        var gradeRow = gradesModel.get(gi2)
```

```
        if (gradeRow.classId === classId){
```

```
            classTotal += gradeRow.finalGrade
```

```
            classCount += 1
```

```
{
```

```
{
```

```
    var classIndex = classIndexById(classId)
```

```
    if (classIndex >= 0)
```

```
        classesModel.setProperty(classIndex, "average", classCount > 0 ?
Math.round(classTotal / classCount) : 0)
```

```
{
```

```
{
```

```
Connections}
```

```
    target: auth
```

```
function onCreateCourseResult(success, message, courseId){
```

```
    if (success)
```

```
        setStatus("Your class was created successfully.", false)
```

```
    else
```

```
        setStatus(message, true)
```

```

        if (success){
            newClassName"" =
            newClassDescription"" =
            refreshTeacherData()
        }
        {
        {
        {

Connections}
    target: auth

    function onCreateAssignmentResult(success, message, courseId,
assignmentId){
        endBusy()
        if (success)
            setStatus("Your assignment was created successfully.", false)
        else
            setStatus(message, true)
        if (success){
            createAssignmentTitle"" =
            createAssignmentDescription"" =
            createAssignmentDue"" =
            createAssignmentAi = true
            assignmentAttachmentName"" =
            assignmentAttachmentPath"" =
            refreshClassData(courseId)
        }
        {

```



```

{

Connections}

    target: auth

    function onUpdateSubmissionReviewResult(success, message,
submissionId)}

        if (!success)

            setStatus(message, true)

        if (!success)

            return

        var idx = submissionIndexById(submissionId)

        if (idx >= 0)}

            submissionsModel.setProperty(idx, "teacherText", reviewTeacherNote)

            submissionsModel.setProperty(idx, "finalGrade",
parseInt(reviewFinalGrade))

            submissionsModel.setProperty(idx, "teacherEdited", true)

            var sub = submissionsModel.get(idx)

            for (var i = 0; i < gradesModel.count; i++){

                var g = gradesModel.get(i)

                if (g.assignmentId === sub.assignmentId && g.student ===
sub.student)}

                    gradesModel.setProperty(i, "finalGrade",
parseInt(reviewFinalGrade))

                    gradesModel.setProperty(i, "reviewed", true)

            {

            {

                auth.get_submissions_for_assignment(sub.assignmentId)

```

```

        auth.get_course_students(sub.classId)
    {
    {
    {

Connections}
    target: auth

    function onGetUsersResult(success, message, users){
        if (!success)
            return
        schoolUsersData = users
    {
    {

Connections}
    target: auth

    function onCourseMaterialsResult(success, message, courseId, materials)
    {
        if (!success){
            setStatus(message, true)
            return
        {

        removeRowsByField(materialsModel, "classId", courseId)
        for (var i = 0; i < materials.length; i++){
            var m = materials[i]
            materialsModel.append})

```

```

        materialId: m.material_id,
        classId: courseId,
        type: m.file_path ? (m.type ? m.type :
inferMaterialTypeFromPath(m.file_path)) : "NO FILE,"
        title: m.title,
        by: m.created_by_name ? m.created_by_name : materialBy,
        description: m.description ? m.description, "" :
        fileName: fileNameFromPath(m.file_path ? m.file_path : ""),
        filePath: m.file_path ? m.file_path, "" :
        whenText: m.created_at ? formatDateTimePretty(m.created_at) "" :

```

```

({
{

```

```

        var classIndex = classIndexById(courseId)
        if (classIndex >= 0)
            classesModel.setProperty(classIndex, "materials",
countMaterialsByClass(courseId))

```

```

{
{

```

Connections}

target: auth

```
function onCreateMaterialResult(success, message, courseId, materialId){
```

```
    endBusy()
```

```
    if (success)
```

```
        setStatus("The material was uploaded successfully.", false)
```

```
    else
```

```
        setStatus(message, true)
```

```

        if (!success)
            return

        materialTitle"" =
        materialDescription"" =
        materialFileName"" =
        materialFilePath"" =
        materialType = "NO FILE"
        auth.get_course_materials(courseId)
    {
    {

Connections}
    target: auth

    function onCourseMessagesResult(success, message, courseId,
messages)}
        if (!success){
            setStatus(message, true)
            return
        {

        removeRowsByField(messageLogModel, "classId", courseId)
        for (var i = messages.length - 1; i >= 0; i--){
            var m = messages[i]
            messageLogModel.append{
                messageId: m.message_id,
                classId: courseId,

```

```

        subject: m.subject,

        body: m.body,

        state: m.state,

        createdAt: m.created_at

    ({
    {
    {
    {

Connections}

    target: auth

function onSaveCourseMessageResult(success, message, courseId,
messageId})
    if (!success)
        setStatus(message, true)
    if (!success)
        return

    messageSubject"" =
    messageBody"" =
    auth.get_course_messages(courseId)
{
{

Connections}

    target: auth

```

```
function onUpdateCourseMemberStatusResult(success, message,
courseId, studentId, memberStatus){
```

```
    if (!success)
```

```
        setStatus(message, true)
```

```
    if (!success)
```

```
        return
```

```
    auth.get_course_students(courseId)
```

```
{
```

```
{
```

```
Connections}
```

```
target: auth
```

```
function onDeleteCourseMemberResult(success, message, courseId,
studentId){
```

```
    if (!success)
```

```
        setStatus(message, true)
```

```
    if (!success)
```

```
        return
```

```
    auth.get_course_students(courseId)
```

```
{
```

```
{
```

```
Connections}
```

```
target: auth
```

```
function onDeleteCourseResult(success, message, courseId){
```

```
    if (!success){
```

```
        setStatus(message, true)
```

```

        return

    {

        if (root.studentsClassId === courseId)

            root.studentsClassId = classesModel.count > 0 ?
classesModel.get(0).classId : -1

            if (root.submissionsClassId === courseId)

                root.submissionsClassId = classesModel.count > 0 ?
classesModel.get(0).classId : -1

            if (root.gradesClassId === courseId)

                root.gradesClassId = classesModel.count > 0 ?
classesModel.get(0).classId : -1

            setStatus("The class was deleted successfully.", false)

            refreshTeacherData()

    {

    {

Connections}

    target: auth

    function onSetAssignmentClosedResult(success, message, assignmentId,
isClosed){

        if (!success){

            if (message === "ASSIGNMENT_ALREADY_PAST_DUE")

                setStatus("This assignment cannot be reopened because its due
time already passed", true)

            else

                setStatus(message, true)

            return

        {

            for (var i = 0; i < assignmentsModel.count; i++){

```

```

        var a = assignmentsModel.get(i)
        if (a.assignmentId === assignmentId){
            assignmentsModel.setProperty(i, "manualClosed", isClosed)
            assignmentsModel.setProperty(i, "status", (isClosed ||
!isAssignmentOpen(a.dueText)) ? "Closed" : "Open")
            var classIndex = classIndexById(a.classId)
            if (classIndex >= 0)
                classesModel.setProperty(classIndex, "openAssignments",
countOpenAssignmentsByClass(a.classId))
            setStatus(isClosed ? "Assignment closed" : "Assignment reopened",
false)
            return
        }
    }
    {
    {
    {
    {

```

Connections}

target: auth

```

        function onDeleteAssignmentResult(success, message, assignmentId,
courseId){
            if (!success){
                setStatus(message, true)
                return
            }
            {
                removeRowsByAssignment(assignmentId)
                for (var i = assignmentsModel.count - 1; i >= 0; i--){
                    if (assignmentsModel.get(i).assignmentId === assignmentId)

```



```

        assignmentsModel.remove(i)
    {
        if (courseId > 0){
            var classIndex = classIndexById(courseId)
            if (classIndex >= 0){
                classesModel.setProperty(classIndex, "assignments",
countAssignmentsByClass(courseId))
                classesModel.setProperty(classIndex, "openAssignments",
countOpenAssignmentsByClass(courseId))
            {
                if (root.submissionsClassId === courseId)
                    root.submissionsAssignmentId = -1
                if (root.gradesClassId === courseId)
                    root.gradesAssignmentId = firstAssignmentIdForClass(courseId)
            {
                setStatus("The assignment was deleted successfully.", false)
            {
            {

Connections}
    target: auth

function onDeleteMaterialResult(success, message, materialId, courseId){
    if (!success){
        setStatus(message, true)
        return
    {
        for (var i = materialsModel.count - 1; i >= 0; i--){
            if (materialsModel.get(i).materialId === materialId)

```

```

        materialsModel.remove(i)
    {
        if (courseId > 0){
            var classIndex = classIndexById(courseId)
            if (classIndex >= 0)
                classesModel.setProperty(classIndex, "materials",
countMaterialsByClass(courseId))
        {
            setStatus("The material was deleted successfully.", false)
        {
            {

Connections}

            target: auth

        function onDeleteMessageResult(success, message, messageId,
courseId){
            if (!success){
                setStatus(message, true)
                return
            {
                for (var i = messageLogModel.count - 1; i >= 0; i--){
                    if (messageLogModel.get(i).messageId === messageId)
                        messageLogModel.remove(i)
                {
                    setStatus("The message was deleted successfully.", false)
                {
                    {

```

```

Connections}

    target: auth

    function onLocalFileValidationResult(success, target, filePath, message,
sizeBytes){
        if (!success){
            if (target === "assignment"){
                root.assignmentAttachmentPath"" =
                root.assignmentAttachmentName"" =
            {
                else if (target === "material") {
                    root.materialFilePath"" =
                    root.materialFileName"" =
                {
                    setStatus(message, true)
                    return
                {

                var cleanName = root.fileNameFromPath(filePath)
                if (target === "assignment"){
                    root.assignmentAttachmentPath = filePath
                    root.assignmentAttachmentName = cleanName
                {
                    else if (target === "material") {
                        root.materialFilePath = filePath
                        root.materialFileName = cleanName
                        root.materialType = inferMaterialTypeFromPath(filePath)
                        if (root.materialTitle.trim().length === 0)
                            root.materialTitle = cleanName
                    {

```

```
        setStatus("File selected: " + cleanName + " (" + formatBytes(sizeBytes) +
    ")", false)
```

```
{
```

```
{
```

```
Connections}
```

```
    target: auth
```

```
function onDownloadFileResult(success, message, filePath, savedPath){
```

```
    endBusy()
```

```
    if (success){
```

```
        root.lastDownloadedPath = savedPath
```

```
        setStatus("The download finished successfully.", false)
```

```
{    else}
```

```
        setStatus("Download failed: " + message, true)
```

```
{
```

```
{
```

```
{
```

```
Dialog}
```

```
    id: messageDialog
```

```
    modal: true
```

```
    width: Math.min(Math.max(root.width * 0.52, 500), root.width - 120)
```

```
    height: Math.min(Math.max(root.height * 0.46, 320), root.height - 140)
```

```
    x: (root.width - width) / 2
```

```
    y: (root.height - height) / 2
```

```
    title: "Message details"
```

```

        background: Rectangle { radius: 18; color: card; border.width: 1;
border.color: line }

        contentItem: ColumnLayout{

            spacing: 12

            Text { text: root.selectedMaterialTitle; color: ink; font.pixelSize: 16;
font.weight: Font.DemiBold }

            Text { text: root.selectedMaterialMeta; color: muted; font.pixelSize: 11;
wrapMode: Text.WrapAnywhere; Layout.fillWidth: true }

            TextArea{

                Layout.fillWidth: true

                Layout.fillHeight: true

                readOnly: true

                wrapMode: TextEdit.WrapAnywhere

                text: root.selectedMaterialDescription

                color: ink

                background: Rectangle { radius: 14; color: "#F8FAFC"; border.width: 1;
border.color: line }

            {

                RowLayout { Layout.fillWidth: true; Item { Layout.fillWidth: true }
ActionButton { text: "Close"; onClicked: messageDialog.close ()}

            {

            {

FileDialog}

            id: fileDialog

            title: "Select file"

            fileMode: FileDialog.OpenFile

            nameFilters: ["All files (*)", "Archives (*.zip *.tar *.tgz *.rar *.7z)", "Code files
(*.py *.cs *.java *.js *.ts *.cpp *.c *.h)"]

```

```

onAccepted} :
    var path = selectedFile.toString()
    var cleanName = root.fileNameFromPath(path)
    root.backendStub("fileSelected", { target: root.fileDialogTarget, fileName:
cleanName, filePath: path })
    auth.validate_local_file(path, root.fileDialogTarget)
{
{

Dialog}

id: submissionDialog
modal: true
width: Math.min(Math.max(root.width * 0.64, 560), root.width - 80)
height: Math.min(Math.max(root.height * 0.72, 420), root.height - 80)
x: (root.width - width) / 2
y: (root.height - height) / 2
title: "Submission Viewer"

background: Rectangle}
    radius: 18
    color: card
    border.width: 1
    border.color: line
{

contentItem: ColumnLayout}
    spacing: 12

```

Rectangle}

Layout.fillWidth: true

implicitHeight: 56

radius: 14

color: "#F8FAFC"

border.width: 1

border.color: line

RowLayout}

anchors.fill: parent

anchors.margins: 12

spacing: 10

Text { text: root.selectedSubmissionFileName; color: ink;
font.pixelSize: 14; font.weight: Font.DemiBold }

Item { Layout.fillWidth: true }

Badge { textValue: "Preview"; bgColor: blueSoft; fgColor: blue }

{

{

TextArea}

Layout.fillWidth: true

Layout.fillHeight: true

text: root.selectedSubmissionPreview

readOnly: true

wrapMode: TextEdit.WrapAnywhere

selectByMouse: true

color: ink

selectedTextColor: "#FFFFFF"

```

        selectionColor: indigo600
        background: Rectangle{
            radius: 14
            color: "#F8FAFC"
            border.width: 1
            border.color: line
    {
    {

        RowLayout{
            Layout.fillWidth: true
            spacing: 8
            ActionButton { text: "Close"; onClicked: submissionDialog.close ()}
    {
    {
    {

        Dialog{
            id: materialDialog
            modal: true
            width: Math.min(Math.max(root.width * 0.58, 520), root.width - 100)
            height: Math.min(Math.max(root.height * 0.58, 360), root.height - 100)
            x: (root.width - width) / 2
            y: (root.height - height) / 2
            title: "Material Viewer"

            background: Rectangle{
                radius: 18

```



```

        color: card
        border.width: 1
        border.color: line
    {

        contentItem: ColumnLayout}
        spacing: 12

        Rectangle}
            Layout.fillWidth: true
            implicitHeight: 82
            radius: 14
            color: "#F8FAFC"
            border.width: 1
            border.color: line

        ColumnLayout}
            anchors.fill: parent
            anchors.margins: 12
            spacing: 4
            Text { text: root.selectedMaterialTitle; color: ink; font.pixelSize: 15;
font.weight: Font.DemiBold }
            Text { text: root.selectedMaterialMeta; color: muted; font.pixelSize:
11; wrapMode: Text.WrapAnywhere; Layout.fillWidth: true }
        {
        {

        TextArea}
            Layout.fillWidth: true

```

```

        Layout.fillHeight: true

        text: root.selectedMaterialDescription

        readOnly: true

        wrapMode: TextEdit.WrapAnywhere

        selectByMouse: true

        color: ink

        selectedTextColor: "#FFFFFF"

        selectionColor: indigo600

        background: Rectangle{

            radius: 14

            color: "#F8FAFC"

            border.width: 1

            border.color: line

        }

    {

    {

        RowLayout{

            Layout.fillWidth: true

            spacing: 8

            ActionButton { text: "Download"; onClicked:
root.downloadServerFile(root.selectedMaterialPath, root.selectedMaterialTitle) }

            ActionButton { text: "Close"; kind: "ghost"; onClicked:
materialDialog.close ()}

            Item { Layout.fillWidth: true }

        {

        {

        {

        Rectangle}

```

```

id: shell

anchors.fill: parent
anchors.margins: 22
radius: 24
color: card
border.width: 1
border.color: line


layer.enabled: true
layer.effect: DropShadow{
    horizontalOffset: 0
    verticalOffset: 18
    radius: 30
    samples: 40
    color: "#1A000000"
{

ColumnLayout{
    anchors.fill: parent
    anchors.margins: 18
    spacing: 18


Rectangle{
    Layout.fillWidth: true
    Layout.preferredHeight: 92
    radius: 20
    color: indigo700
    border.width: 1

```

```
border.color: "#14000000"
```

```
Rectangle { anchors.fill: parent; radius: parent.radius; color:  
"transparent"; border.width: 1; border.color: "#10FFFFFF" }
```

```
RowLayout}
```

```
anchors.fill: parent
```

```
anchors.margins: 18
```

```
spacing: 14
```

```
Rectangle}
```

```
width: 54; height: 54; radius: 17
```

```
color: "#20FFFFFF"
```

```
border.width: 1
```

```
border.color: "#30FFFFFF"
```

```
Layout.alignment: Qt.AlignVCenter
```

```
clip: true
```

```
Image}
```

```
anchors.centerIn: parent
```

```
width: 40
```

```
height: 40
```

```
source: "png/AppLogo.png"
```

```
fillMode: Image.PreserveAspectFit
```

```
smooth: true
```

```
mipmap: true
```

```
{
```

```
{
```

RowLayout}

Layout.alignment: Qt.AlignVCenter

Layout.fillWidth: true

spacing: 12

Text}

text: "Classify"

color: "#FFFFFF"

opacity: 0.98

font.pixelSize: 26

font.weight: Font.DemiBold

elide: Text.ElideRight

Layout.alignment: Qt.AlignVCenter

{

Text}

text: "Welcome back, " + userName" 🤖 " +

color: "#FFFFFF"

opacity: 0.95

font.pixelSize: 18

font.weight: Font.DemiBold

elide: Text.ElideRight

Layout.alignment: Qt.AlignVCenter

Layout.fillWidth: true

{

{

```

        RowLayout}

        Layout.alignment: Qt.AlignVCenter

        spacing: 8

        HeaderButton { text: "Refresh"; onClicked:
root.refreshCurrentPageData ()}

        HeaderButton { text: "Logout"; onClicked: root.logout ()}

    {
    {
    {

```

```

        RowLayout}

        Layout.fillWidth: true

        Layout.fillHeight: true

        spacing: 14

        Rectangle}

        Layout.preferredWidth: Math.max(220, Math.min(252, shell.width *
0.19))

        Layout.maximumWidth: 252

        Layout.minimumWidth: 220

        Layout.fillHeight: true

        radius: 20

        color: soft

        border.width: 1

        border.color: line

```

```

        ColumnLayout}

        anchors.fill: parent

        anchors.margins: 12

```

spacing: 10

Text { text: "Quick actions"; color: muted2; font.pixelSize: 12;
font.weight: Font.Medium }

NavItem { title: "Dashboard"; iconText: "🏠"; pageKey:
"dashboard" }

NavItem { title: "Create Class"; iconText: "✚"; pageKey:
"newClass" }

NavItem { title: "All Classes"; iconText: "🏫"; pageKey: "classes" }

NavItem { title: "Assignments"; iconText: "📝"; pageKey:
"assignments" }

NavItem { title: "Students"; iconText: "👥"; pageKey: "students" }

NavItem { title: "Materials"; iconText: "📁"; pageKey: "materials" }

NavItem { title: "Submission Review"; iconText: "🤖"; pageKey:
"submissions" }

NavItem { title: "Messages"; iconText: "📢"; pageKey: "messages"
}

NavItem { title: "Reports"; iconText: "📊"; pageKey: "reports" }

Item { Layout.fillHeight: true }

{

{

ColumnLayout}

Layout.fillWidth: true

Layout.fillHeight: true

spacing: 14

```

        RowLayout{

            Layout.fillWidth: true

            spacing: 14

            StatCard { title: "Active classes"; value:
classesModel.count.toString(); iconText: "🏠"; accentBg: violetSoft; accentText:
indigo700 }

            StatCard { title: "Total students"; value:
totalStudentsAll().toString(); iconText: "👤"; accentBg: blueSoft; accentText: blue
}

            StatCard { title: "Open assignments"; value:
countOpenAssignments().toString(); iconText: "🟢"; accentBg: greenSoft;
accentText: green }

            StatCard { title: "Average grade"; value:
averageAcrossClasses().toString(); iconText: "★"; accentBg: amberSoft;
accentText: amber }

        }

```

```

        Loader{

            Layout.fillWidth: true

            Layout.fillHeight: true

            sourceComponent:

                currentPage === "dashboard" ? dashboardPage:

                currentPage === "newClass" ? newClassPage:

                currentPage === "classes" ? classesPage:

                currentPage === "assignments" ? assignmentsPage:

                currentPage === "students" ? studentsPage:

                currentPage === "materials" ? materialsPage:

                currentPage === "submissions" || currentPage === "grades" ?
submissionsPage:

                currentPage === "messages" ? messagesPage:

```



```

reportsPage
{

    Rectangle}

        Layout.fillWidth: true

        visible: root.backendStatusText.length > 0

        implicitHeight: visible ? 30 : 0

        radius: 12

        color: root.backendStatusIsError ? "#FEF2F2" : "#F0FDF4"

        border.width: visible ? 1 : 0

        border.color: root.backendStatusIsError ? "#FCA5A5" :
"#86EFAC"

    RowLayout}

        anchors.fill: parent

        anchors.leftMargin: 10

        anchors.rightMargin: 8

        anchors.topMargin: 6

        anchors.bottomMargin: 8

        spacing: 8

    Text}

        Layout.fillWidth: true

        Layout.alignment: Qt.AlignVCenter

        text: root.backendStatusText

        color: root.backendStatusIsError ? red : green

        font.pixelSize: 10

        elide: Text.ElideRight

```

```

        verticalAlignment: Text.AlignVCenter
    {

        Item}
        width: 20
        height: 20
        Layout.alignment: Qt.AlignVCenter

        MouseArea}
        anchors.fill: parent
        onClicked: root.backendStatusText"" =
        cursorShape: Qt.PointingHandCursor
    {

        Text}
        anchors.centerIn: parent
        y: -1
        text"X" :
        color: muted
        font.pixelSize: 11
        verticalAlignment: Text.AlignVCenter
    {
    {
    {
    {
    {
    {
    {
    {

```

{

Rectangle}

anchors.fill: parent

visible: root.busyActive

z: 20000

color: "#660F172A"

MouseArea}

anchors.fill: parent

enabled: root.busyActive

{

Rectangle}

width: 280

height: 116

radius: 18

color: card

border.width: 1

border.color: line

anchors.centerIn: parent

Column}

anchors.centerIn: parent

spacing: 10

BusyIndicator}

```

        anchors.horizontalCenter: parent.horizontalCenter
        running: root.busyActive
        width: 40
        height: 40
    {

        Text{
            anchors.horizontalCenter: parent.horizontalCenter
            width: 224
            text: root.busyText.length > 0 ? root.busyText : "Please wait"...
            color: ink
            font.pixelSize: 13
            font.weight: Font.Medium
            horizontalAlignment: Text.AlignHCenter
            wrapMode: Text.NoWrap
            elide: Text.ElideRight
            maximumLineCount: 1
        {
        {
        {
        {

        component HeaderButton : Item{
            id: hb
            property alias text: label.text
            signal clicked
            implicitWidth: 92
            implicitHeight: 34

```

```

    Rectangle { anchors.fill: parent; radius: 12; color: "#FFFFFF"; opacity:
ma.pressed ? 0.22 : (ma.containsMouse ? 0.20 : 0.16); border.width: 1;
border.color: "#10FFFFFF" }

    Text { id: label; anchors.centerIn: parent; color: "#FFFFFF"; font.pixelSize:
13; font.weight: Font.DemiBold }

    MouseArea { id: ma; anchors.fill: parent; hoverEnabled: true; cursorShape:
Qt.PointingHandCursor; onClicked: hb.clicked ()}

{

component NavItem : Item}

    id: ni

    property string title"" :

    property string iconText"" :

    property string pageKey"" :

    implicitHeight: 46

    Layout.fillWidth: true

    readonly property bool active: root.currentPage === pageKey

    Rectangle { anchors.fill: parent; radius: 14; color: active ? "#EEF2FF" :
"transparent"; border.width: 1; border.color: active ? "#C7D2FE" : "transparent" }

    RowLayout{

        anchors.fill: parent

        anchors.leftMargin: 12

        anchors.rightMargin: 12

        spacing: 10

        Rectangle{

            width: 28; height: 28; radius: 9

```

```

        color: active ? "#FFFFFF" : "#F3F4F6"

        border.width: 1

        border.color: active ? "#D9E0FF" : "#ECEFF3"

        Text { anchors.centerIn: parent; text: iconText; font.pixelSize: 13 }
    {

        Text { Layout.fillWidth: true; text: title; color: active ? indigo700 : ink;
font.pixelSize: 13; font.weight: active ? Font.DemiBold : Font.Medium; elide:
Text.ElideRight }
    {

        MouseArea { anchors.fill: parent; hoverEnabled: true; cursorShape:
Qt.PointingHandCursor; onClicked: root.openPage(pageKey) }
    {

    component StatCard : Rectangle}

        property string title"" :
        property string value"" :
        property string iconText"" :
        property color accentBg: "#EEF2FF"
        property color accentText: indigo700
        property string helperText: "Read-only"
        Layout.fillWidth: true
        Layout.preferredHeight: 104
        radius: 18
        color: card
        border.width: 1
        border.color: line

```

```

Rectangle}

    anchors.top: parent.top
    anchors.right: parent.right
    anchors.topMargin: 10
    anchors.rightMargin: 10
    radius: 9
    color: "#F8FAFC"
    border.width: 1
    border.color: "#E8ECF4"
    implicitWidth: helperBadgeText.implicitWidth + 14
    implicitHeight: 22

Text}

    id: helperBadgeText
    anchors.centerIn: parent
    text: helperText
    color: muted2
    font.pixelSize: 10
    font.weight: Font.DemiBold
{
{

```

```

RowLayout}

    anchors.fill: parent
    anchors.margins: 14
    anchors.rightMargin: 18
    spacing: 12

```

```
        Rectangle { width: 44; height: 44; radius: 14; color: accentBg;
border.width: 1; border.color: "#E8ECF4"; Text { anchors.centerIn: parent; text:
iconText; color: accentText; font.pixelSize: 18 } }
```

```
    ColumnLayout{
```

```
        Layout.fillWidth: true
```

```
        spacing: 1
```

```
        Text { text: title; color: muted; font.pixelSize: 12 }
```

```
        Text { text: value; color: ink; font.pixelSize: 24; font.weight:
Font.DemiBold }
```

```
{
```

```
{
```

```
{
```

```
component SectionFrame : Rectangle{
```

```
    Layout.fillWidth: true
```

```
    radius: 20
```

```
    color: card
```

```
    border.width: 1
```

```
    border.color: line
```

```
{
```

```
component Badge : Rectangle{
```

```
    property string textValue"" :
```

```
    property color bgColor: blueSoft
```

```
    property color fgColor: blue
```

```
    implicitWidth: Math.max(84, badgeText.implicitWidth + 18)
```

```
    implicitHeight: 28
```

```
    radius: 10
```

```
    color: bgColor
```



```

border.width: 1

border.color: Qt.rgb(0.05, 0, 0, 0)

Text { id: badgeText; anchors.centerIn: parent; text: textValue; color: fgColor;
font.pixelSize: 12; font.weight: Font.DemiBold }
{

component ActionButton : Item}

id: ab

property alias text: btnLabel.text

property string kind: "primary"

signal clicked

implicitWidth: Math.max(110, btnLabel.implicitWidth + 28)

implicitHeight: 40

Rectangle}

anchors.fill: parent

radius: 12

color: kind === "ghost" ? "#FFFFFF" : indigo600

border.width: 1

border.color: kind === "ghost" ? line : indigo600

opacity: btnMouse.pressed ? 0.85 : 1.0
{

Text { id: btnLabel; anchors.centerIn: parent; color: kind === "ghost" ? ink :
"#FFFFFF"; font.pixelSize: 12; font.weight: Font.DemiBold }

MouseArea { id: btnMouse; anchors.fill: parent; hoverEnabled: true;
cursorShape: Qt.PointingHandCursor; onClicked: ab.clicked ()}
{

```

```

component InputField : Item}

    property string label"" :

    property alias text: field.text

    property string placeholder"" :

    implicitHeight: 76

    Layout.fillWidth: true


    ColumnLayout}

        anchors.fill: parent

        spacing: 6

        Text { text: label; color: muted; font.pixelSize: 12; font.weight:
Font.Medium }

        TextField}

            id: field

            Layout.fillWidth: true

            placeholderText: placeholder

            selectByMouse: true

            implicitHeight: 42

            color: ink

            selectedTextColor: "#FFFFFF"

            selectionColor: indigo600

            background: Rectangle { radius: 12; color: "#F8FAFC"; border.width: 1;
border.color: field.activeFocus ? indigo600 : line }

        {

        {

        {

    component AreaField : Item}

        property string label"" :

```

```

    property alias text: field.text
    property string placeholder"" :
    implicitHeight: 190
    Layout.fillWidth: true

    ColumnLayout{
        anchors.fill: parent
        spacing: 6
        Text { text: label; color: muted; font.pixelSize: 12; font.weight:
Font.Medium }
        TextArea{
            id: field
            Layout.fillWidth: true
            Layout.fillHeight: true
            wrapMode: TextEdit.WrapAnywhere
            selectByMouse: true
            placeholderText: placeholder
            color: ink
            selectedTextColor: "#FFFFFF"
            selectionColor: indigo600
            background: Rectangle { radius: 12; color: "#F8FAFC"; border.width: 1;
border.color: field.activeFocus ? indigo600 : line }
        }
        {
        {
        {

    component LabeledCombo : Item{
        id: lc
        property string label"" :

```

```

property var model[] :
property string textRole"" :
property int currentIndex: 0
readonly property string currentText: itemText(currentIndex)
signal chosen(int index)
implicitHeight: 76
Layout.fillWidth: true

function itemText(idk)
    if (!model || idk < 0)
        return ""
    var item = model.get != undefined ? model.get(idk) : model[idk]
    if (item === undefined || item === null)
        return ""
    if (textRole && item[textRole] != undefined)
        return item[textRole]
    return item.toString != undefined ? item.toString"" : ()

{

ColumnLayout}

    anchors.fill: parent

    spacing: 6

    Text { text: lc.label; color: muted; font.pixelSize: 12; font.weight:
Font.Medium }

    Rectangle}

    id: comboBoxFrame

```

```

Layout.fillWidth: true
implicitHeight: 42
radius: 12
color: "#F8FAFC"
border.width: 1
border.color: comboPopup.opened ? indigo600 : line

```

```

RowLayout}

```

```

    anchors.fill: parent
    anchors.leftMargin: 14
    anchors.rightMargin: 12
    spacing: 8

```

```

Text}

```

```

    Layout.fillWidth: true
    text: lc.currentIndex >= 0 ? lc.currentText : "Select"
    color: ink
    font.pixelSize: 12
    verticalAlignment: Text.AlignVCenter
    elide: Text.ElideRight

```

```

{

```

```

    Text { text: "▼"; color: muted; font.pixelSize: 14; font.weight:
Font.DemiBold }

```

```

{

```

```

MouseArea}

```

```

    anchors.fill: parent

```

```

        cursorShape: Qt.PointingHandCursor
        onClicked: comboPopup.opened ? comboPopup.close() :
        comboPopup.open()
    {
    {
    {

    Popup}
        id: comboPopup
        y: comboBoxFrame.height + 6
        width: comboBoxFrame.width
        z: 10000
        padding: 6
        modal: false
        closePolicy: Popup.CloseOnEscape | Popup.CloseOnPressOutside
        background: Rectangle{
            radius: 14
            color: "#FFFFFF"
            border.width: 1
            border.color: line
        }

        contentItem: ListView{
            implicitHeight: Math.min(contentHeight, 220)
            clip: true
            model: lc.model
            boundsBehavior: Flickable.StopAtBounds
            ScrollBar.vertical: StyledScrollBar{

```

```

delegate: Rectangle}

width: ListView.view.width

height: 38

radius: 10

color: index === lc.currentIndex ? "#EEF2FF" :
(itemMouse.containsMouse ? "#F8FAFC" : "#FFFFFF")

Text}

anchors.verticalCenter: parent.verticalCenter
anchors.left: parent.left
anchors.leftMargin: 12
anchors.right: parent.right
anchors.rightMargin: 12
text: lc.itemText(index)
color: ink
font.pixelSize: 12
elide: Text.ElideRight

{

MouseArea}

id: itemMouse
anchors.fill: parent
hoverEnabled: true
cursorShape: Qt.PointingHandCursor
onClicked} :

    lc.chosen(index)
    comboPopup.close()

```

```
{  
{  
{  
{  
{  
{
```

```
    component PageHeader : Rectangle}  
        property string title"" :  
        property string subtitle"" :  
        property string iconText"" :  
        Layout.fillWidth: true  
        implicitHeight: 96  
        radius: 18  
        color: soft  
        border.width: 1  
        border.color: line  
  
        RowLayout}  
            anchors.fill: parent  
            anchors.margins: 14  
            spacing: 12  
            Rectangle { width: 44; height: 44; radius: 14; color: "#EEF2FF";  
border.width: 1; border.color: "#D9E0FF"; Text { anchors.centerIn: parent; text:  
iconText; font.pixelSize: 18 } }  
            ColumnLayout}  
                Layout.fillWidth: true  
                spacing: 2
```



```
Text { text: title; color: ink; font.pixelSize: 20; font.weight:
Font.DemiBold }
```

```
Text { text: subtitle; color: muted; font.pixelSize: 12; wrapMode:
Text.WrapAnywhere; Layout.fillWidth: true }
```

```
{
{
{
```

```
component RightMetric : Item}
```

```
property string topText"" :
```

```
property string bottomText"" :
```

```
property color topColor: ink
```

```
property color bottomColor: muted
```

```
property int metricWidth: 120
```

```
width: metricWidth
```

```
implicitHeight: 46
```

```
Column}
```

```
anchors.right: parent.right
```

```
anchors.verticalCenter: parent.verticalCenter
```

```
width: parent.width
```

```
spacing: 2
```

```
Text { width: parent.width; text: topText; horizontalAlignment:
Text.AlignRight; color: topColor; font.pixelSize: 13; font.weight: Font.DemiBold;
elide: Text.ElideRight }
```

```
Text { width: parent.width; text: bottomText; horizontalAlignment:
Text.AlignRight; color: bottomColor; font.pixelSize: 11; elide: Text.ElideRight }
```

```
{
{
```

Component}

id: dashboardPage

Flickable}

contentWidth: width

contentHeight: dashCol.implicitHeight

clip: true

ColumnLayout}

id: dashCol

width: parent.width

spacing: 14

PageHeader { title: "Overview"; iconText: "🏠" }

RowLayout}

Layout.fillWidth: true

spacing: 14

SectionFrame}

Layout.fillWidth: true

implicitHeight: 380

Item}

anchors.fill: parent

anchors.margins: 14

Text}

```

        id: classSnapshotTitle
        text: "Class snapshot"
        color: ink
        font.pixelSize: 16
        font.weight: Font.DemiBold
        anchors.top: parent.top
        anchors.left: parent.left
    {

        Flickable}

        id: classSnapshotFlick
        anchors.top: classSnapshotTitle.bottom
        anchors.topMargin: 10
        anchors.left: parent.left
        anchors.right: parent.right
        anchors.bottom: parent.bottom
        clip: true
        contentWidth: width
        contentHeight: classSnapshotList.height
        boundsBehavior: Flickable.StopAtBounds
        ScrollBar.vertical: StyledScrollBar { policy:
ScrollBar.AsNeeded }

        Column}

        id: classSnapshotList
        width: classSnapshotFlick.width
        spacing: 10

```

Repeater}

model: classesModel

delegate: Rectangle}

width: classSnapshotList.width

implicitHeight: 72

radius: 14

color: "#F8FAFC"

border.width: 1

border.color: line

RowLayout}

anchors.fill: parent

anchors.margins: 12

spacing: 12

ColumnLayout}

Layout.fillWidth: true

spacing: 2

Text { text: name; color: ink; font.pixelSize: 13;
font.weight: Font.DemiBold; elide: Text.ElideRight }

Text { text: classCode && classCode.length > 0
? ("Class code: " + classCode) : description; color: muted; font.pixelSize: 11;
elide: Text.ElideRight }

Text { visible: classCode && classCode.length
> 0 && description.length > 0; text: description; color: muted2; font.pixelSize: 11;
wrapMode: Text.WrapAnywhere; maximumLineCount: 1; elide: Text.ElideRight }

{

RowLayout}

```

                                Layout.alignment: Qt.AlignRight |
Qt.AlignVCenter

                                spacing: 8

                                Badge { textValue:
countStudentsByClass(classId) + " students"; bgColor: blueSoft; fgColor: blue }

                                Badge { textValue:
countOpenAssignmentsByClass(classId) + " open tasks"; bgColor: greenSoft;
fgColor: green }
{
{
{
{
{
{
{
{
{

```

```

SectionFrame}

```

```

    Layout.fillWidth: true
    implicitHeight: 380

```

```

Item}

```

```

    anchors.fill: parent
    anchors.margins: 14

```

```

Text}

```

```

    id: teacherPulseTitle
    text: "Teacher pulse"
    color: ink
    font.pixelSize: 16

```

```
font.weight: Font.DemiBold
anchors.top: parent.top
anchors.left: parent.left
{
```

```
Rectangle}
```

```
id: pulseApprovalCard
anchors.top: teacherPulseSubtitle.bottom
anchors.topMargin: 14
anchors.left: parent.left
width: Math.floor((parent.width - 10) / 2)
height: 78
radius: 12
color: "#F8FAFC"
border.width: 1
border.color: line
```

```
Column}
```

```
anchors.fill: parent
anchors.margins: 12
spacing: 4
```

```
Text}
```

```
text: "Awaiting approval"
color: muted
font.pixelSize: 11
```

```
{
```

```
    Row}
```

```
        height: 30
```

```
        spacing: 6
```

```
    Text}
```

```
        text: pendingStudentsModel.count.toString()
```

```
        color: ink
```

```
        font.pixelSize: 24
```

```
        font.weight: Font.Bold
```

```
        anchors.verticalCenter: parent.verticalCenter
```

```
{
```

```
    Text}
```

```
        text: pendingStudentsModel.count == 1 ?
```

```
"student" : "students"
```

```
        color: blue
```

```
        font.pixelSize: 11
```

```
        anchors.verticalCenter: parent.verticalCenter
```

```
{
```

```
{
```

```
{
```

```
{
```

```
    Rectangle}
```

```
        id: pulseReviewCard
```

```
        anchors.top: teacherPulseSubtitle.bottom
```

```
        anchors.topMargin: 14
```

```
anchors.right: parent.right
width: Math.floor((parent.width - 10) / 2)
height: 78
radius: 12
color: "#F8FAFC"
border.width: 1
border.color: line
```

```
Column}
```

```
anchors.fill: parent
anchors.margins: 12
spacing: 4
```

```
Text}
```

```
text: "Need review"
color: muted
font.pixelSize: 11
```

```
{
```

```
Row}
```

```
height: 30
spacing: 6
```

```
Text}
```

```
text: countUnreviewedGrades().toString()
color: ink
font.pixelSize: 24
font.weight: Font.Bold
```



```

        anchors.verticalCenter: parent.verticalCenter
    {
        Text{
            text: countUnreviewedGrades() === 1 ?
"submission" : "submissions"

            color: amber

            font.pixelSize: 11

            anchors.verticalCenter: parent.verticalCenter
        }
    }
}

```

```

Rectangle{
    id: pulseInsightsCard
    anchors.top: pulseApprovalCard.bottom
    anchors.topMargin: 10
    anchors.left: parent.left
    anchors.right: parent.right
    height: 148
    radius: 12
    color: "#F8FAFC"
    border.width: 1
    border.color: line
}

```

```

Column{
    anchors.fill: parent
    anchors.margins: 12
}

```

spacing: 14

Item}

width: parent.width

height: 44

Text}

anchors.top: parent.top

anchors.left: parent.left

text: "Top class by average"

color: muted

font.pixelSize: 12

{

Text}

anchors.top: parent.top

anchors.right: parent.right

text: topClassByAverage() ?
(topClassByAverage().average + " avg") "-" :

color: green

font.pixelSize: 12

font.weight: Font.DemiBold

{

Text}

anchors.left: parent.left

anchors.right: parent.right

anchors.bottom: parent.bottom

```
        text: topClassByAverage() ?
topClassByAverage().name : "No classes yet"
```

```
        color: ink
```

```
        font.pixelSize: 13
```

```
        font.weight: Font.DemiBold
```

```
        elide: Text.ElideRight
```

```
{
```

```
{
```

```
Rectangle}
```

```
    width: parent.width
```

```
    height: 1
```

```
    color: line
```

```
{
```

```
Item}
```

```
    width: parent.width
```

```
    height: 44
```

```
Text}
```

```
    anchors.top: parent.top
```

```
    anchors.left: parent.left
```

```
    text: "Most open workload"
```

```
    color: muted
```

```
    font.pixelSize: 12
```

```
{
```

```
Text}
```

```
    anchors.top: parent.top
```

```

        anchors.right: parent.right
        text: busiestClassByOpenAssignments() ?
(busiestClassByOpenAssignments().openAssignments + " open") "-" :
        color: blue
        font.pixelSize: 12
        font.weight: Font.DemiBold
    {

```

```

        Text}
        anchors.left: parent.left
        anchors.right: parent.right
        anchors.bottom: parent.bottom
        text: busiestClassByOpenAssignments() ?
busiestClassByOpenAssignments().name : "No classes yet"
        color: ink
        font.pixelSize: 13
        font.weight: Font.DemiBold
        elide: Text.ElideRight
    {
    {
    {
    {

```

```

Rectangle}
    id: pulseAiCard
    anchors.left: parent.left
    anchors.bottom: parent.bottom
    width: Math.floor((parent.width - 10) / 2)
    height: 52

```

```
radius: 12
color: "#F8FAFC"
border.width: 1
border.color: line
```

```
Text}
    anchors.left: parent.left
    anchors.leftMargin: 12
    anchors.verticalCenter: parent.verticalCenter
    text: "AI assignments"
    color: muted
    font.pixelSize: 12
```

```
{
```

```
Text}
    anchors.right: parent.right
    anchors.rightMargin: 12
    anchors.verticalCenter: parent.verticalCenter
    text: countAiEnabledAssignments().toString()
    color: indigo700
    font.pixelSize: 13
    font.weight: Font.DemiBold
```

```
{
```

```
{
```

```
Rectangle}
    id: pulseMaterialsCard
    anchors.right: parent.right
```

```
anchors.bottom: parent.bottom
width: Math.floor((parent.width - 10) / 2)
height: 52
radius: 12
color: "#F8FAFC"
border.width: 1
border.color: line
```

```
Text}
```

```
anchors.left: parent.left
anchors.leftMargin: 12
anchors.verticalCenter: parent.verticalCenter
text: "Materials"
color: muted
font.pixelSize: 12
```

```
{
```

```
Text}
```

```
anchors.right: parent.right
anchors.rightMargin: 12
anchors.verticalCenter: parent.verticalCenter
text: materialsModel.count.toString()
color: ink
font.pixelSize: 13
font.weight: Font.DemiBold
```

```
{
```

```
{
```

```
{
```

```
{  
{  
{  
{  
{
```

```
Component}
```

```
    id: newClassPage
```

```
    Flickable}
```

```
        contentWidth: width
```

```
        contentHeight: newClassCol.implicitHeight
```

```
        clip: true
```

```
    ColumnLayout}
```

```
        id: newClassCol
```

```
        width: parent.width
```

```
        spacing: 14
```

```
    PageHeader { title: "Create class"; iconText: "✚" }
```

```
    SectionFrame}
```

```
        Layout.fillWidth: true
```

```
        implicitHeight: 350
```

```
        ColumnLayout}
```

```
            anchors.fill: parent
```

```
            anchors.margins: 16
```

```
            spacing: 12
```

```
        InputField { id: classNameInput; label: "Class name";
placeholder: "Example: Grade 12 - Advanced Python"; text: root.newClassName;
onTextChanged: root.newClassName = text }
```

```
        InputField { id: classDescriptionInput; label: "Description";
placeholder: "Example: Final project, API work, data analysis"; text:
root.newClassDescription; onTextChanged: root.newClassDescription = text }
```

```
        Item { Layout.fillHeight: true }
```

```
        RowLayout{
```

```
            Layout.fillWidth: true
```

```
            spacing: 8
```

```
            ActionButton { text: "Create class"; onClicked:
root.addClass ()}
```

```
            ActionButton { text: "Clear"; kind: "ghost"; onClicked: {
root.newClassName = ""; root.newClassDescription = "" } }
```

```
        Item { Layout.fillWidth: true }
```

```
{
```

```
{
```

```
{
```

```
{
```

```
{
```

```
{
```

```
Component{
```

```
    id: classesPage
```

```
    Flickable{
```

```
        id: classesPageFlick
```

```
        contentWidth: width
```

```
        contentHeight: classesCol.implicitHeight
```

```
        clip: true
```


ColumnLayout}

id: classesCol

width: parent.width

spacing: 14

PageHeader { title: "Classes"; iconText: "🏠" }

SearchField}

Layout.fillWidth: true

placeholderText: "Search classes by name, description or code"

text: root.classSearchText

onTextChanged: root.classSearchText = text

{

SectionFrame}

Layout.fillWidth: true

implicitHeight: Math.max(470, classesPageFlick.height - 120)

Flickable}

id: classesListFlick

anchors.fill: parent

anchors.margins: 16

clip: true

contentWidth: width

contentHeight: classesListColumn.height

boundsBehavior: Flickable.StopAtBounds

ScrollBar.vertical: StyledScrollBar { policy: ScrollBar.AsNeeded }

Column}

id: classesListColumn

width: classesListFlick.width

spacing: 12

Repeater}

model: classesModel

delegate: Rectangle}

readonly property bool matchesSearchRow:

root.matchesSearch(name + " " + description + " " + (classCode ? classCode : "")),
root.classSearchText)

width: classesListColumn.width

visible: matchesSearchRow

implicitHeight: matchesSearchRow ? 142 : 0

radius: 16

color: "#F8FAFC"

border.width: matchesSearchRow ? 1 : 0

border.color: line

Column}

anchors.left: parent.left

anchors.top: parent.top

anchors.bottom: parent.bottom

anchors.leftMargin: 14

anchors.topMargin: 14

anchors.bottomMargin: 14

width: parent.width - 260

spacing: 5

```

Text}
    width: parent.width
    text: name
    color: ink
    font.pixelSize: 14
    font.weight: Font.DemiBold
    elide: Text.ElideRight
{
    Text}
    width: parent.width
    text: classCode && classCode.length > 0 ? ("Class
code: " + classCode)"" :
    visible: text.length > 0
    color: blue
    font.pixelSize: 11
    font.weight: Font.DemiBold
    elide: Text.ElideRight
{
    Text}
    width: parent.width
    text: description
    color: muted
    font.pixelSize: 11
    wrapMode: Text.WrapAnywhere
    maximumLineCount: classCode &&
classCode.length > 0 ? 1 : 2
    elide: Text.ElideRight
{
    Item { width: 1; height: 4 }

```

```

        Text{
            width: parent.width
            text: "Last activity: " +
formatDateTimePretty(lastActivity)
            color: muted2
            font.pixelSize: 11
            elide: Text.ElideRight
        }
    }

```

```

    Column{
        anchors.right: parent.right
        anchors.top: parent.top
        anchors.rightMargin: 14
        anchors.topMargin: 12
        width: 180
        spacing: 8
    }

```

```

    Row{
        width: parent.width
        spacing: 6
        layoutDirection: Qt.RightToLeft
        Badge { textValue: students + " students"; bgColor:
blueSoft; fgColor: blue }
        Badge { textValue: materials + " materials"; bgColor:
violetSoft; fgColor: indigo700 }
    }

```

```

    Row{
        width: parent.width
    }

```

```

        spacing: 6
        layoutDirection: Qt.RightToLeft
        Badge { textValue: assignments + " tasks"; bgColor:
greenSoft; fgColor: green }
        Badge { textValue: "Avg " + average; bgColor:
amberSoft; fgColor: amber }
    {

```

```

        ActionBar}
        width: parent.width
        text: "Delete class"
        kind: "ghost"
        onClicked: root.deleteClass(classId, name)
    {
    {
    {
    {
    {
    {
    {
    {
    {
    {
    {
    {

```

```

Component}
    id: assignmentsPage
    Flickable}
        contentWidth: width
        contentHeight: assignCol.implicitHeight

```

clip: true

ColumnLayout}

id: assignCol

width: parent.width

spacing: 14

PageHeader { title: "Assignments"; iconText: "📝" }

SectionFrame}

visible: classesModel.count === 0

Layout.fillWidth: true

implicitHeight: visible ? 92 : 0

ColumnLayout { anchors.centerIn: parent; spacing: 4

Text { text: "No classes available yet"; color: ink; font.pixelSize: 14; font.weight: Font.DemiBold }

Text { text: "Create your first class to start adding assignments."; color: muted; font.pixelSize: 11 }

{

{

SectionFrame}

Layout.fillWidth: true

implicitHeight: 760

ColumnLayout}

anchors.fill: parent

anchors.margins: 16

spacing: 10

```
Text { text: "Create assignment"; color: ink; font.pixelSize: 16;
font.weight: Font.DemiBold }
```

```
LabeledCombo{
```

```
    label: "Class"
```

```
    model: classesModel
```

```
    currentIndex: Math.max(0,
classIndexById(root.createAssignmentClassId))
```

```
    textRole: "name"
```

```
    onChosen: if (index >= 0) root.createAssignmentClassId =
classesModel.get(index).classId
```

```
{
```

```
    InputField { label: "Assignment title"; placeholder: "Example:
Functions quiz"; text: root.createAssignmentTitle; onTextChanged:
root.createAssignmentTitle = text }
```

```
ColumnLayout{
```

```
    Layout.fillWidth: true
```

```
    spacing: 8
```

```
    Text { text: "Due date & time"; color: muted; font.pixelSize: 11;
font.weight: Font.Medium }
```

```
Rectangle{
```

```
    Layout.fillWidth: true
```

```
    implicitHeight: 170
```

```
    radius: 18
```

```
    color: "#F8FAFC"
```

```
    border.width: 1
```

border.color: line

ColumnLayout}

anchors.fill: parent

anchors.margins: 14

spacing: 12

GridLayout}

Layout.fillWidth: true

columns: 3

columnSpacing: 10

rowSpacing: 10

DateTimeSpin { label: "Year"; from: 2024; to: 2035;
value: root.dueYear; Layout.fillWidth: true; onValueModified: { root.dueYear =
value; root.refreshCreateAssignmentDue ()} }

DateTimeSpin { label: "Month"; from: 1; to: 12; value:
root.dueMonth; Layout.fillWidth: true; onValueModified: { root.dueMonth =
value; root.refreshCreateAssignmentDue ()} }

DateTimeSpin { label: "Day"; from: 1; to: 31; value:
root.dueDay; Layout.fillWidth: true; onValueModified: { root.dueDay = value;
root.refreshCreateAssignmentDue ()} }

DateTimeSpin { label: "Hour"; from: 0; to: 23; value:
root.dueHour; Layout.fillWidth: true; onValueModified: { root.dueHour = value;
root.refreshCreateAssignmentDue ()} }

DateTimeSpin { label: "Minute"; from: 0; to: 59; value:
root.dueMinute; Layout.fillWidth: true; onValueModified: { root.dueMinute =
value; root.refreshCreateAssignmentDue ()} }

Rectangle}

Layout.fillWidth: true


```
Layout.fillHeight: true
radius: 14
color: "#EEF2FF"
border.width: 1
border.color: "#C7D2FE"
```

```
RowLayout}
```

```
anchors.fill: parent
```

```
anchors.margins: 12
```

```
spacing: 8
```

```
Text { text: "⌚"; font.pixelSize: 14 }
```

```
ColumnLayout}
```

```
Layout.fillWidth: true
```

```
spacing: 1
```

```
Text { text: "Selected due time"; color: muted;
font.pixelSize: 10; font.weight: Font.Medium }
```

```
Text { text: root.createAssignmentDue; color:
indigo700; font.pixelSize: 12; font.weight: Font.DemiBold; elide: Text.ElideRight }
```

```
{
```

```
{
```

```
{
```

```
{
```

```
{
```

```
{
```

```
{
```

```
{
```

```
AreaField { label: "Instructions"; placeholder: "Write the
instructions for the class"; text: root.createAssignmentDescription;
onTextChanged: root.createAssignmentDescription = text; implicitHeight: 140 }
```

```
Text { text: "File limit: up to 25MB - For multiple files, upload one
ZIP"; color: muted; font.pixelSize: 11 }
```

```
Rectangle}
```

```
Layout.fillWidth: true
```

```
implicitHeight: 54
```

```
radius: 14
```

```
color: "#F8FAFC"
```

```
border.width: 1
```

```
border.color: line
```

```
RowLayout}
```

```
anchors.fill: parent
```

```
anchors.margins: 8
```

```
spacing: 8
```

```
Text { Layout.fillWidth: true; text:
root.assignmentAttachmentName.length > 0 ?
root.assignmentAttachmentName : "No instruction file selected"; color:
root.assignmentAttachmentName.length > 0 ? ink : muted2; font.pixelSize: 12;
elide: Text.ElideRight }
```

```
ActionButton { text: "Browse"; kind: "ghost"; onClicked:
root.openFilePicker("assignment") }
```

```
ActionButton { text: "Remove"; kind: "ghost"; enabled:
root.assignmentAttachmentName.length > 0; onClicked:
root.clearAssignmentAttachment ()}
```

```
{
```

```
{
```

```
RowLayout}
```

```
Layout.fillWidth: true
```

spacing: 10

Rectangle}

implicitWidth: 196

implicitHeight: 46

radius: 14

color: "#F8FAFC"

border.width: 1

border.color: line

RowLayout}

anchors.fill: parent

anchors.margins: 4

spacing: 4

Rectangle}

Layout.fillWidth: true

implicitHeight: 38

radius: 11

color: root.createAssignmentAi ? indigo600 :

"transparent"

border.width: root.createAssignmentAi ? 0 : 1

border.color: root.createAssignmentAi ? "transparent"

: line

Text { anchors.centerIn: parent; text: "AI ON"; color:
root.createAssignmentAi ? "white" : muted; font.pixelSize: 12; font.weight:
Font.DemiBold }

MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked: root.createAssignmentAi = true }

{

```

        Rectangle{
            Layout.fillWidth: true
            implicitHeight: 38
            radius: 11
            color: root.createAssignmentAi ? "transparent" :
"#FEE2E2"

            border.width: root.createAssignmentAi ? 1 : 0
            border.color: root.createAssignmentAi ? line :
"transparent"

            Text { anchors.centerIn: parent; text: "AI OFF"; color:
root.createAssignmentAi ? muted : red; font.pixelSize: 12; font.weight:
Font.DemiBold }

            MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked: root.createAssignmentAi = false }
{
{
{

        ActionButton{
            text: "Publish"
            opacity: root.isCreateAssignmentDueValid() ? 1.0 : 0.55
            onClicked: root.addAssignment()

{

            Item { Layout.fillWidth: true }

{
{
{

        SectionFrame}

```

```

        Layout.fillWidth: true
        implicitHeight: 520

        ColumnLayout{
            anchors.fill: parent
            anchors.margins: 16
            spacing: 10

            Text { text: "Active assignments"; color: ink; font.pixelSize: 16;
font.weight: Font.DemiBold }

            SearchField{
                Layout.fillWidth: true
                placeholderText: "Search assignments by title or description"
                text: root.assignmentSearchText
                onTextChanged: root.assignmentSearchText = text
            }

        }

        ScrollView{
            id: activeAssignmentsScroll
            Layout.fillWidth: true
            Layout.fillHeight: true
            clip: true
            ScrollBar.vertical: StyledScrollBar{}

            Column{
                width: (activeAssignmentsScroll.availableWidth > 0 ?
activeAssignmentsScroll.availableWidth : activeAssignmentsScroll.width)
                spacing: 10

                Rectangle{

```

```

        width: parent.width

        visible:
assignmentOptionsForClass(root.createAssignmentClassId).length === 0

        height: visible ? 84 : 0

        radius: 14

        color: "#F8FAFC"

        border.width: visible ? 1 : 0

        border.color: line

    ColumnLayout}

        anchors.centerIn: parent

        spacing: 4

        Text { text: "No assignments yet for this class"; color:
ink; font.pixelSize: 13; font.weight: Font.DemiBold }

        Text { text: "Create the first assignment from the card
above."; color: muted; font.pixelSize: 11 }

    {
    {

    Repeater}

        model: assignmentsModel

        delegate: Rectangle}

        readonly property bool matchesClass: classId ===
root.createAssignmentClassId

        readonly property bool matchesSearchRow:
root.matchesSearch(title + " " + description + " " + dueText,
root.assignmentSearchText)

        width: parent.width

        visible: matchesClass && matchesSearchRow

        height: visible ? 124 : 0

```

```
radius: 15
color: "#F8FAFC"
border.width: visible ? 1 : 0
border.color: line
```

```
RowLayout}

anchors.fill: parent
anchors.margins: 12
spacing: 12
```

```
ColumnLayout}

Layout.fillWidth: true

spacing: 4

Text { text: title; color: ink; font.pixelSize: 13;
font.weight: Font.DemiBold; elide: Text.ElideRight }

Text { text: classNameById(classId) + " • Due " +
formatDateTimePretty(dueText); color: muted; font.pixelSize: 11; elide:
Text.ElideRight }

Text { text: description; color: muted2;
font.pixelSize: 11; wrapMode: Text.WrapAnywhere; maximumLineCount: 2; elide:
Text.ElideRight; Layout.fillWidth: true }

{
```

```
ColumnLayout}

width: Math.min(210, parent.width * 0.34)

spacing: 6

RowLayout}

Layout.fillWidth: true

spacing: 6
```

```
        Badge { textValue: status; bgColor: status ===  
"Open" ? greenSoft : amberSoft; fgColor: status === "Open" ? green : amber }
```

```
        Badge { textValue: submissions + "/" +  
totalStudents + " turned in"; bgColor: blueSoft; fgColor: blue }
```

```
{
```

```
    RowLayout{
```

```
        Layout.fillWidth: true
```

```
        spacing: 6
```

```
        Badge { textValue: aiEnabled ? "AI on" : "AI off";  
bgColor: aiEnabled ? violetSoft : redSoft; fgColor: aiEnabled ? indigo700 : red }
```

```
        ActionButton { text: status === "Open" ?  
"Close" : "Reopen"; kind: "ghost"; onClicked:  
root.toggleAssignmentStatus(assignmentId) }
```

```
        ActionButton { text: "Delete"; kind: "ghost";  
onClicked: root.deleteAssignment(assignmentId) }
```

```
{
```

```
{
```

```
{
```

```
{
```

```
{
```

```
{
```

```
{
```

```
{
```

```
{
```

```
{
```

```
{
```

```
{
```

```
Component}
```

```
    id: studentsPage
```


Flickable}

contentWidth: width

contentHeight: studentsCol.implicitHeight

clip: true

ColumnLayout}

id: studentsCol

width: parent.width

spacing: 14

PageHeader { title: "Class roster"; iconText: "👤" }

SectionFrame}

visible: classesModel.count === 0

Layout.fillWidth: true

implicitHeight: visible ? 92 : 0

ColumnLayout { anchors.centerIn: parent; spacing: 4

Text { text: "No classes available yet"; color: ink; font.pixelSize: 14; font.weight: Font.DemiBold }

Text { text: "Create a class first, then students will appear here."; color: muted; font.pixelSize: 11 }

{

{

RowLayout}

Layout.fillWidth: true

spacing: 14

SectionFrame}

Layout.fillWidth: true

implicitHeight: 590

ColumnLayout}

anchors.fill: parent

anchors.margins: 16

spacing: 10

LabeledCombo}

label: "Class"

model: classesModel

currentIndex: Math.max(0,
classIndexById(root.studentsClassId))

textRole: "name"

onChosen: if (index >= 0) root.studentsClassId =
classesModel.get(index).classId

{

Text { text: "Active and inactive students"; color: ink;
font.pixelSize: 15; font.weight: Font.DemiBold }

SearchField}

Layout.fillWidth: true

placeholderText: "Search students by name, status or
grade"

text: root.studentSearchText

onTextChanged: root.studentSearchText = text

{

ScrollView}

```

        id: studentsActiveScroll
        LayoutMirroring.enabled: false
        ScrollBar.vertical: StyledScrollBar{ }
        ScrollBar.horizontal.policy: ScrollBar.AlwaysOff
        Layout.fillWidth: true
        Layout.fillHeight: true
        clip: true

    Column}

        width: (studentsActiveScroll.availableWidth > 0 ?
studentsActiveScroll.availableWidth : studentsActiveScroll.width)
        spacing: 10

    Rectangle}

        width: parent.width
        visible: countStudentsByClass(root.studentsClassId)
=== 0

        height: visible ? 84 : 0
        radius: 14
        color: "#F8FAFC"
        border.width: visible ? 1 : 0
        border.color: line

    ColumnLayout}

        anchors.centerIn: parent
        spacing: 4

        Text { text: "No students in this class"; color: ink;
font.pixelSize: 13; font.weight: Font.DemiBold; horizontalAlignment:
Text.AlignHCenter }

```

```

        Text { text: "Choose another class or add students
later."; color: muted; font.pixelSize: 11; horizontalAlignment: Text.AlignHCenter }
    {
    {

```

```

        Repeater{
            model: studentsModel
            delegate: Rectangle{
                readonly property bool matchesClass: classId ===
root.studentsClassId
                readonly property bool matchesSearchRow:
root.matchesSearch(name + " " + status + " " + grade + " " + lastSubmission,
root.studentSearchText)
                width: parent.width
                visible: matchesClass && matchesSearchRow
                height: visible ? 68 : 0
                radius: 14
                color: "#F8FAFC"
                border.width: matchesClass ? 1 : 0
                border.color: line

```

```

        RowLayout{
            anchors.fill: parent
            anchors.margins: 12
            spacing: 12

```

```

        ColumnLayout{
            Layout.fillWidth: true
            spacing: 2

```

```
Text { Layout.fillWidth: true; text: name; color:
ink; font.pixelSize: 13; font.weight: Font.DemiBold; elide: Text.ElideRight;
horizontalAlignment: Text.AlignLeft }
```

```
Text { Layout.fillWidth: true; text:
lastSubmission && lastSubmission.length > 0 ?
formatDateTimePretty(lastSubmission) : "No submissions yet"; color: muted;
font.pixelSize: 11; elide: Text.ElideRight; horizontalAlignment: Text.AlignLeft }
{
```

```
RowLayout}
```

```
Layout.alignment: Qt.AlignRight |
Qt.AlignVCenter
```

```
spacing: 8
```

```
Badge { textValue: status; bgColor: status ===
"Active" ? greenSoft : redSoft; fgColor: status === "Active" ? green : red }
```

```
Badge { textValue: "Avg " + grade; bgColor:
blueSoft; fgColor: blue }
```

```
ActionButton { text: status === "Active" ?
"Deactivate" : "Activate"; kind: "ghost"; onClicked:
root.toggleStudentStatus(studentId) }
```

```
{
{
{
{
{
{
{
{
```

```
SectionFrame}
```

```
Layout.fillWidth: true
```

```
implicitHeight: 590
```

```

ColumnLayout{
    anchors.fill: parent
    anchors.margins: 16
    spacing: 10
    Text { text: "Pending approvals"; color: ink; font.pixelSize: 15;
font.weight: Font.DemiBold }

```

```

ScrollView{
    id: studentsPendingScroll
    LayoutMirroring.enabled: false
    Layout.fillWidth: true
    Layout.fillHeight: true
    clip: true
    ScrollBar.vertical: StyledScrollBar{}
    ScrollBar.horizontal.policy: ScrollBar.AlwaysOff

```

```

Column{
    width: (studentsPendingScroll.availableWidth > 0 ?
studentsPendingScroll.availableWidth : studentsPendingScroll.width)
    spacing: 10

```

```

Rectangle{
    width: parent.width
    visible:
countPendingStudentsByClass(root.studentsClassId) === 0
    height: visible ? 84 : 0
    radius: 14
    color: "#F8FAFC"

```

border.width: visible ? 1 : 0

border.color: line

ColumnLayout}

anchors.centerIn: parent

spacing: 4

Text { text: "No students are waiting for approval";
color: ink; font.pixelSize: 13; font.weight: Font.DemiBold; horizontalAlignment:
Text.AlignHCenter }

Text { text: "Switch classes or come back later.";
color: muted; font.pixelSize: 11; horizontalAlignment: Text.AlignHCenter }

{

{

Repeater}

model: pendingStudentsModel

delegate: Rectangle}

readonly property bool matchesClass: classId ==
root.studentsClassId

width: parent.width

visible: matchesClass

height: matchesClass ? 70 : 0

radius: 14

color: "#F8FAFC"

border.width: matchesClass ? 1 : 0

border.color: line

RowLayout}

anchors.fill: parent

```

anchors.margins: 12

spacing: 10

ColumnLayout{

    Layout.fillWidth: true

    spacing: 3

    Text { text: name; color: ink; font.pixelSize: 13;
font.weight: Font.DemiBold }

    Text { text: "Requested: " + requestedAt; color:
muted; font.pixelSize: 11 }

{

    Item { Layout.fillWidth: true }

    ActionButton { text: "Approve"; onClicked:
root.approvePendingStudent(requestId) }

    ActionButton { text: "Reject"; kind: "ghost";
onClicked: root.rejectPendingStudent(requestId) }

{

{

{

{

{

{

{

{

{

{

Component}

    id: materialsPage

    Flickable}

```



```
contentWidth: width
contentHeight: materialsCol.implicitHeight
clip: true
```

```
ColumnLayout}

id: materialsCol

width: parent.width

spacing: 14
```

```
PageHeader { title: "Materials"; iconText: "📁" }
```

```
SectionFrame}

visible: classesModel.count === 0

Layout.fillWidth: true

implicitHeight: visible ? 92 : 0

ColumnLayout { anchors.centerIn: parent; spacing: 4

    Text { text: "No classes available yet"; color: ink; font.pixelSize:
14; font.weight: Font.DemiBold }

    Text { text: "Create a class first to upload learning materials.";
color: muted; font.pixelSize: 11 }

{

{
```

```
SectionFrame}

Layout.fillWidth: true

implicitHeight: Math.max(590, root.height * 0.39)
```

```
ColumnLayout}

anchors.fill: parent
```

anchors.margins: 16

spacing: 10

Text { text: "Upload new material"; color: ink; font.pixelSize: 16;
font.weight: Font.DemiBold }

LabeledCombo}

label: "Class"

model: classesModel

currentIndex: Math.max(0,
classIndexById(root.materialsTargetClassId))

textRole: "name"

onChosen: if (index >= 0) root.materialsTargetClassId =
classesModel.get(index).classId

{

Rectangle}

Layout.fillWidth: true

implicitHeight: 48

radius: 14

color: "#F8FAFC"

border.width: 1

border.color: line

RowLayout}

anchors.fill: parent

anchors.margins: 12

spacing: 10

Text { text: "Detected type"; color: muted; font.pixelSize: 11;
font.weight: Font.Medium }

```
Badge { textValue: root.materialType; bgColor: violetSoft;
fgColor: indigo700 }
```

```
Text { Layout.fillWidth: true; text:
root.materialFileName.length > 0 ? "Detected automatically from the selected
file" : "Choose a file and the type will be detected automatically"; color: muted2;
font.pixelSize: 11; elide: Text.ElideRight }
{
{
```

```
TextField { label: "Title"; placeholder: "Example: Lesson
summary"; text: root.materialTitle; onTextChanged: root.materialTitle = text }
```

```
AreaField { label: "Description"; placeholder: "Describe the
material for the class"; text: root.materialDescription; onTextChanged:
root.materialDescription = text; implicitHeight: 140 }
```

```
Text { text: "File limit: up to 25MB - For multiple files, upload one
ZIP"; color: muted; font.pixelSize: 11 }
```

```
Rectangle}
```

```
Layout.fillWidth: true
```

```
implicitHeight: 54
```

```
radius: 14
```

```
color: "#F8FAFC"
```

```
border.width: 1
```

```
border.color: line
```

```
RowLayout}
```

```
anchors.fill: parent
```

```
anchors.margins: 8
```

```
spacing: 8
```

```

Text { Layout.fillWidth: true; text:
root.materialFileName.length > 0 ? root.materialFileName : "No file selected";
color: root.materialFileName.length > 0 ? ink : muted2; font.pixelSize: 12; elide:
Text.ElideRight }

```

```

ActionButton { text: "Browse"; kind: "ghost"; onClicked:
root.openFilePicker("material") }

```

```

ActionButton { text: "Remove"; kind: "ghost"; enabled:
root.materialFileName.length > 0; onClicked: root.clearMaterialAttachment ()}

```

```

{

```

```

{

```

```

RowLayout}

```

```

Layout.fillWidth: true

```

```

spacing: 10

```

```

ActionButton { text: "Upload material"; onClicked:
root.addMaterial ()}

```

```

Item { Layout.fillWidth: true }

```

```

{

```

```

{

```

```

{

```

```

SectionFrame}

```

```

Layout.fillWidth: true

```

```

implicitHeight: 460

```

```

ColumnLayout}

```

```

anchors.fill: parent

```

```

anchors.margins: 16

```

```

spacing: 10

```

```
Text { text: "Existing materials"; color: ink; font.pixelSize: 16;
font.weight: Font.DemiBold }
```

```
SearchField}
```

```
Layout.fillWidth: true
```

```
placeholderText: "Search materials by title, type or
description"
```

```
text: root.materialSearchText
```

```
onTextChanged: root.materialSearchText = text
```

```
{
```

```
ScrollView}
```

```
id: existingMaterialsScroll
```

```
Layout.fillWidth: true
```

```
Layout.fillHeight: true
```

```
clip: true
```

```
ScrollBar.vertical: StyledScrollBar{}
```

```
Column}
```

```
width: (existingMaterialsScroll.availableWidth > 0 ?
existingMaterialsScroll.availableWidth : existingMaterialsScroll.width)
```

```
spacing: 10
```

```
Rectangle}
```

```
width: parent.width
```

```
visible:
```

```
countMaterialsByClass(root.materialsTargetClassId) === 0
```

```
height: visible ? 84 : 0
```

```
radius: 14
```

```
color: "#F8FAFC"
```

border.width: visible ? 1 : 0

border.color: line

ColumnLayout}

anchors.centerIn: parent

spacing: 4

Text { text: "No materials yet for this class"; color: ink;
font.pixelSize: 13; font.weight: Font.DemiBold; horizontalAlignment:
Text.AlignHCenter }

Text { text: "Upload the first file from the card above.";
color: muted; font.pixelSize: 11; horizontalAlignment: Text.AlignHCenter }

{

{

Repeater}

model: materialsModel

delegate: Rectangle}

readonly property bool matchesClass: classId ===
root.materialsTargetClassId

readonly property bool matchesSearchRow:
root.matchesSearch(title + " " + type + " " + description + " " + by,
root.materialSearchText)

width: parent.width

visible: matchesClass && matchesSearchRow

height: visible ? 94 : 0

radius: 14

color: "#F8FAFC"

border.width: matchesClass ? 1 : 0

border.color: line

```

        RowLayout{
            anchors.fill: parent
            anchors.margins: 12
            spacing: 10

            ColumnLayout{
                Layout.fillWidth: true
                spacing: 3
                Text { text: title; color: ink; font.pixelSize: 13;
font.weight: Font.DemiBold; elide: Text.ElideRight }
                Text { text: by + " • " + whenText; color: muted;
font.pixelSize: 11; elide: Text.ElideRight }
            }

            RowLayout{
                Layout.alignment: Qt.AlignRight |
Qt.AlignVCenter
                spacing: 8
                Badge { textValue: type; bgColor: violetSoft;
fgColor: indigo700 }
                ActionButton { text: "Download"; kind: "ghost";
onClicked: root.openMaterialViewer(materialId) }
                ActionButton { text: "Delete"; kind: "ghost";
onClicked: root.deleteMaterial(materialId) }
            }
        }
    }
}

```

```
{  
{  
{  
{  
{
```

```
Component}
```

```
    id: submissionsPage
```

```
    Flickable}
```

```
        contentWidth: width
```

```
        contentHeight: subCol.implicitHeight
```

```
        clip: true
```

```
    ColumnLayout}
```

```
        id: subCol
```

```
        width: parent.width
```

```
        spacing: 14
```

```
    PageHeader { title: "Submission review"; iconText: "📄" }
```

```
    SectionFrame}
```

```
        visible: classesModel.count === 0
```

```
        Layout.fillWidth: true
```

```
        implicitHeight: visible ? 92 : 0
```

```
        ColumnLayout { anchors.centerIn: parent; spacing: 4
```

```
            Text { text: "No classes available yet"; color: ink; font.pixelSize:  
14; font.weight: Font.DemiBold }
```

```
            Text { text: "Create a class and assignments to start reviewing  
submissions."; color: muted; font.pixelSize: 11 }
```



```
{  
{
```

```
    RowLayout}
```

```
        Layout.fillWidth: true
```

```
        spacing: 14
```

```
    SectionFrame}
```

```
        Layout.fillWidth: true
```

```
        implicitHeight: 650
```

```
    ColumnLayout}
```

```
        anchors.fill: parent
```

```
        anchors.margins: 16
```

```
        spacing: 10
```

```
    LabeledCombo}
```

```
        label: "Class"
```

```
        model: classesModel
```

```
        currentIndex: Math.max(0,  
classIndexById(root.submissionsClassId))
```

```
        textRole: "name"
```

```
        onChosen} :
```

```
            if (index >= 0){
```

```
                root.submissionsClassId =  
classesModel.get(index).classId
```

```
                root.submissionsAssignmentId = -1
```

```
                root.selectedSubmissionId = -1
```

```
                root.reviewTeacherNote"" =
```

```

        root.reviewFinalGrade"" =

{
{
{

        LabeledCombo}

        label: "Assignment"

        model:
assignmentOptionsForClass(root.submissionsClassId)

        textRole: "name"

        currentIndex:
assignmentIndexForClassAndId(root.submissionsClassId,
root.submissionsAssignmentId)

        onChosen} :

        var options =
assignmentOptionsForClass(root.submissionsClassId)

        if (index >= 0 && index < options.length){

            root.submissionsAssignmentId =
options[index].assignmentId

            root.selectedSubmissionId = -1

            root.reviewTeacherNote"" =

            root.reviewFinalGrade"" =

{
{
{

        Text { text: "Submissions"; color: ink; font.pixelSize: 15;
font.weight: Font.DemiBold }

        SearchField}

        Layout.fillWidth: true

```

```

placeholderText: "Search student submissions"

text: root.submissionSearchText

onTextChanged: root.submissionSearchText = text
{

    ScrollView}

    id: submissionsListScroll

    LayoutMirroring.enabled: false

    Layout.fillWidth: true

    Layout.fillHeight: true

    clip: true

    ScrollBar.vertical: StyledScrollBar{}

    ScrollBar.horizontal.policy: ScrollBar.AlwaysOff


    Column}

    width: (submissionsListScroll.availableWidth > 0 ?
submissionsListScroll.availableWidth : submissionsListScroll.width)

    spacing: 10


    Rectangle}

    width: parent.width

    visible:
submissionRowsForSelection(root.submissionsClassId,
root.submissionsAssignmentId).length === 0

    height: visible ? 92 : 0

    radius: 14

    color: "#F8FAFC"

    border.width: visible ? 1 : 0

    border.color: line

```

```

        ColumnLayout{
            anchors.centerIn: parent
            spacing: 4

            Text { text: "No submissions for this assignment";
color: ink; font.pixelSize: 13; font.weight: Font.DemiBold; horizontalAlignment:
Text.AlignHCenter }

            Text { text: "Choose another assignment or wait for
students to submit."; color: muted; font.pixelSize: 11; horizontalAlignment:
Text.AlignHCenter }

        {
        {

    Repeater{
        model: submissionsModel
        delegate: Rectangle}

        readonly property bool matchesSelection: classId
=== root.submissionsClassId && assignmentId ===
root.submissionsAssignmentId

        readonly property bool matchesSearchRow:
root.matchesSearch(student + " " + submittedAt + " " + fileName + " " + aiText + " "
+ teacherText, root.submissionSearchText)

        width: parent.width
        visible: matchesSelection && matchesSearchRow
        height: visible ? 78 : 0
        radius: 14
        color: submissionId === root.selectedSubmissionId
? "#EEF2FF" : "#F8FAFC"

        border.width: matchesSelection ? 1 : 0
        border.color: submissionId ===
root.selectedSubmissionId ? "#C7D2FE" : line

```

```
RowLayout}
```

```
anchors.fill: parent
```

```
anchors.margins: 12
```

```
spacing: 10
```

```
ColumnLayout}
```

```
Layout.fillWidth: true
```

```
spacing: 3
```

```
Text { text: student; color: ink; font.pixelSize:
13; font.weight: Font.DemiBold }
```

```
Text { text: formatDateTimePretty(submittedAt);
color: muted; font.pixelSize: 11 }
```

```
{
```

```
RowLayout}
```

```
Layout.alignment: Qt.AlignRight |
Qt.AlignVCenter
```

```
spacing: 6
```

```
Badge { textValue: aiEnabled ? ("AI " + aiGrade)
: "AI off"; bgColor: violetSoft; fgColor: indigo700 }
```

```
Badge { textValue: "Final " + finalGrade;
bgColor: greenSoft; fgColor: green }
```

```
Badge { textValue: teacherEdited ? "Reviewed"
: "Pending"; bgColor: teacherEdited ? blueSoft : amberSoft; fgColor:
teacherEdited ? blue : amber }
```

```
{
```

```
{
```

```
MouseArea { anchors.fill: parent; onClicked:
root.selectSubmission(submissionId) }
```

```
{  
{  
{  
{  
{  
{
```

```
    SectionFrame}
```

```
        Layout.fillWidth: true
```

```
        implicitHeight: 650
```

```
        enabled: root.selectedSubmissionId > 0
```

```
        opacity: enabled ? 1.0 : 0.58
```

```
    ColumnLayout}
```

```
        anchors.fill: parent
```

```
        anchors.margins: 16
```

```
        spacing: 10
```

```
        Text { text: submissionById(root.selectedSubmissionId) ?  
(submissionById(root.selectedSubmissionId).student + " review") : "Select a  
submission to start reviewing"; color: ink; font.pixelSize: 16; font.weight:  
Font.DemiBold }
```

```
    Rectangle}
```

```
        Layout.fillWidth: true
```

```
        implicitHeight: 78
```

```
        radius: 14
```

```
        color: "#F8FAFC"
```

```
        border.width: 1
```

border.color: line

ColumnLayout}

anchors.fill: parent

anchors.margins: 12

spacing: 3

Text { text: submissionById(root.selectedSubmissionId) ?
submissionById(root.selectedSubmissionId).fileName : "No submission
selected"; color: ink; font.pixelSize: 13; font.weight: Font.DemiBold }

Text { text: submissionById(root.selectedSubmissionId) ?
((submissionById(root.selectedSubmissionId).aiEnabled ? ("AI score: " +
submissionById(root.selectedSubmissionId).aiGrade) : "AI disabled") + " • Final:
" + submissionById(root.selectedSubmissionId).finalGrade) : "Click a student
submission on the left to enable this panel."; color: muted; font.pixelSize: 11;
wrapMode: Text.WrapAnywhere }

{

{

Rectangle}

id: aiFeedbackPanel

Layout.fillWidth: true

Layout.fillHeight: true

Layout.preferredHeight: 180

radius: 14

color: "#F8FAFC"

border.width: 1

border.color: line

readonly property var selectedSubmission:
root.submissionById(root.selectedSubmissionId)

readonly property string feedbackText:
aiFeedbackPanel.selectedSubmission ?

(aiFeedbackPanel.selectedSubmission.aiText || "No AI feedback yet") : "Select a submission to view AI feedback".

readonly property bool feedbackIsRtl:

root.hasHebrewText(aiFeedbackPanel.feedbackText)

ColumnLayout}

anchors.fill: parent

anchors.margins: 12

spacing: 6

Text}

Layout.fillWidth: true

text: "AI feedback"

color: ink

font.pixelSize: 12

font.weight: Font.DemiBold

horizontalAlignment: aiFeedbackPanel.feedbackIsRtl ?

Text.AlignRight : Text.AlignLeft

{

ScrollView}

LayoutMirroring.enabled:

aiFeedbackPanel.feedbackIsRtl

LayoutMirroring.childrenInherit: true

Layout.fillWidth: true

Layout.fillHeight: true

clip: true

ScrollBar.vertical: StyledScrollBar{}

ScrollBar.horizontal.policy: ScrollBar.AlwaysOff

Text}

width: Math.max(0, (parent.availableWidth > 0 ?

parent.availableWidth : parent.width) - 2)


```

        text:
root.rtlWrappedText(aiFeedbackPanel.feedbackText)

        color: muted

        font.pixelSize: 11

        textFormat: Text.PlainText

        wrapMode: Text.WrapAnywhere

        horizontalAlignment:
aiFeedbackPanel.feedbacksRtl ? Text.AlignRight : Text.AlignLeft
{
{
{
{

```

```

        AreaField { enabled: root.selectedSubmissionId > 0; label:
"Teacher feedback"; placeholder: "Write feedback"; text:
root.reviewTeacherNote; onTextChanged: root.reviewTeacherNote = text;
implicitHeight: 170 }

```

```

        InputField { enabled: root.selectedSubmissionId > 0; label:
"Final grade"; placeholder: "0-100"; text: root.reviewFinalGrade; onTextChanged:
{ var normalized = normalizeGradeInput(text); if (normalized !== text)
root.reviewFinalGrade = normalized; else root.reviewFinalGrade = text } }

```

```

        RowLayout{

            Layout.fillWidth: true

            spacing: 8

            ActionButton { enabled: root.selectedSubmissionId > 0;
text: "Download submission"; kind: "ghost"; onClicked:
root.openSubmissionViewer ()}

            ActionButton { enabled: root.selectedSubmissionId > 0;
text: "Save review"; onClicked: root.saveSubmissionReview ()}

            Item { Layout.fillWidth: true }

{

```

```
{  
{  
{  
{  
{  
{
```

```
Component}
```

```
    id: gradesPage
```

```
    Flickable}
```

```
        contentWidth: width
```

```
        contentHeight: gradesCol.implicitHeight
```

```
        clip: true
```

```
    ColumnLayout}
```

```
        id: gradesCol
```

```
        width: parent.width
```

```
        spacing: 14
```

```
    PageHeader { title: "Gradebook"; iconText: " " }
```

```
    SectionFrame}
```

```
        visible: classesModel.count === 0
```

```
        Layout.fillWidth: true
```

```
        implicitHeight: visible ? 92 : 0
```

```
        ColumnLayout { anchors.centerIn: parent; spacing: 4
```

```
            Text { text: "No classes available yet"; color: ink; font.pixelSize:  
14; font.weight: Font.DemiBold }
```

```
Text { text: "Create a class and assignments to see grades here.";
color: muted; font.pixelSize: 11 }
```

```
{
```

```
{
```

```
RowLayout}
```

```
Layout.fillWidth: true
```

```
spacing: 14
```

```
SectionFrame}
```

```
Layout.fillWidth: true
```

```
implicitHeight: 540
```

```
ColumnLayout}
```

```
id: gradesLeftColumn
```

```
anchors.fill: parent
```

```
anchors.margins: 16
```

```
spacing: 10
```

```
readonly property var rows:
```

```
gradeRowsForSelection(root.gradesClassId, root.gradesAssignmentId)
```

```
LabeledCombo}
```

```
label: "Class"
```

```
model: classesModel
```

```
currentIndex: Math.max(0,
classIndexById(root.gradesClassId))
```

```
textRole: "name"
```

```
onChosen} :
```

```
if (index >= 0)}
```

```

        root.gradesClassId = classesModel.get(index).classId
        root.gradesAssignmentId =
firstAssignmentIdForClass(root.gradesClassId)
        root.selectedGradeId = -1
        root.gradeEditValue"" =
{
{
{

```

```

        LabeledCombo}
        label: "Assignment"
        model: assignmentOptionsForClass(root.gradesClassId)
        textRole: "name"
        currentIndex:
assignmentIndexForClassAndId(root.gradesClassId, root.gradesAssignmentId)
        onChosen} :
        var options =
assignmentOptionsForClass(root.gradesClassId)
        if (index >= 0 && index < options.length){
        root.gradesAssignmentId =
options[index].assignmentId
        root.selectedGradeId = -1
        root.gradeEditValue"" =
{
{
{

```

```

        Text { text: "Grades list"; color: ink; font.pixelSize: 15;
font.weight: Font.DemiBold }

```

ScrollView}

id: gradesListScroll

Layout.fillWidth: true

Layout.fillHeight: true

clip: true

Column}

width: (gradesListScroll.availableWidth > 0 ?
gradesListScroll.availableWidth : gradesListScroll.width)

spacing: 10

Rectangle}

width: parent.width

visible: gradesLeftColumn.rows.length === 0

height: visible ? 92 : 0

radius: 14

color: "#F8FAFC"

border.width: visible ? 1 : 0

border.color: line

ColumnLayout}

anchors.centerIn: parent

spacing: 4

Text { text: "No grades for this assignment"; color:
ink; font.pixelSize: 13; font.weight: Font.DemiBold; horizontalAlignment:
Text.AlignHCenter }

Text { text: "Choose another assignment or wait for
submissions to be reviewed."; color: muted; font.pixelSize: 11;
horizontalAlignment: Text.AlignHCenter }

{

```

{

    Repeater{
        model: gradesLeftColumn.rows
        delegate: Rectangle{
            width: parent.width
            height: 68
            radius: 14
            color: modelData.gradeId === root.selectedGradeId
? "#EEF2FF" : "#F8FAFC"

            border.width: 1
            border.color: modelData.gradeId ===
root.selectedGradeId ? "#C7D2FE" : line

        }

        RowLayout{
            anchors.fill: parent
            anchors.margins: 8
            spacing: 10

            ColumnLayout{
                Layout.fillWidth: true
                spacing: 3
                Text { text: modelData.student; color: ink;
font.pixelSize: 13; font.weight: Font.DemiBold; elide: Text.ElideRight }
                Text { text: "Assignment #" +
modelData.assignmentId; color: muted; font.pixelSize: 11; elide: Text.ElideRight
}
            }
        }

        RowLayout{

```

```

                                Layout.alignment: Qt.AlignRight |
Qt.AlignVCenter

                                spacing: 6

                                Badge { textValue: "AI " + modelData.aiGrade;
bgColor: violetSoft; fgColor: indigo700 }

                                Badge { textValue: "Final " +
modelData.finalGrade; bgColor: greenSoft; fgColor: green }

                                Badge { textValue: modelData.reviewed ?
"Reviewed" : "Pending"; bgColor: modelData.reviewed ? blueSoft : amberSoft;
fgColor: modelData.reviewed ? blue : amber }
{
{

```

```

                                MouseArea { anchors.fill: parent; onClicked:
root.selectGrade(modelData.gradeId) }
{
{
{
{
{
{
{

```

```

SectionFrame}

    Layout.fillWidth: true

    implicitHeight: 540

    ColumnLayout{

        anchors.fill: parent

        anchors.margins: 16

        spacing: 10

```

```

        Text}

        Layout.fillWidth: true

        text: gradeById(root.selectedGradeId) ?
        (gradeById(root.selectedGradeId).student + " grade") : "Select a grade"

        color: ink

        font.pixelSize: 16

        font.weight: Font.DemiBold

        elide: Text.ElideRight

    {

        Rectangle}

        Layout.fillWidth: true

        implicitHeight: 86

        radius: 14

        color: "#F8FAFC"

        border.width: 1

        border.color: line

        ColumnLayout}

        anchors.fill: parent

        anchors.margins: 12

        spacing: 4

        Text}

        Layout.fillWidth: true

        text: gradeById(root.selectedGradeId) ? ("Assignment
        #" + gradeById(root.selectedGradeId).assignmentId) : "Assignment"-

        color: ink

        font.pixelSize: 13

```



```

        font.weight: Font.DemiBold
        elide: Text.ElideRight
    {
        Text}
        Layout.fillWidth: true
        text: gradeByld(root.selectedGradeId) ? ("Student: " +
gradeByld(root.selectedGradeId).student) : "Student"-
        color: muted
        font.pixelSize: 11
        elide: Text.ElideRight
    {
        Text}
        Layout.fillWidth: true
        text: gradeByld(root.selectedGradeId) ? ("AI grade: " +
gradeByld(root.selectedGradeId).aiGrade) : "AI grade"-
        color: muted
        font.pixelSize: 11
        elide: Text.ElideRight
    {
    {
    {

```

```

        InputField { label: "Edit final grade"; text: root.gradeEditValue;
onTextChanged: { var normalized = normalizeGradeInput(text); if (normalized !==
text) root.gradeEditValue = normalized; else root.gradeEditValue = text } }

```

```

    RowLayout}
    Layout.fillWidth: true
    spacing: 8

```

```

        ActionBar { text: "Save grade"; onClicked:
root.saveGradeEdit ()}

        ActionBar { text: "Reset"; kind: "ghost"; onClicked:
root.selectGrade(root.selectedGradeId) }

        Item { Layout.fillWidth: true }

{

    Rectangle}

    Layout.fillWidth: true

    Layout.fillHeight: true

    radius: 14

    color: gradeById(root.selectedGradeId) ? "#F8FAFC" :
"#FBFCFF"

    border.width: 1

    border.color: line

    ColumnLayout}

    anchors.fill: parent

    anchors.margins: 12

    spacing: 6

    Text { text: "Grade details"; color: ink; font.pixelSize: 13;
font.weight: Font.DemiBold }

    Text}

    Layout.fillWidth: true

    wrapMode: Text.WrapAnywhere

    text: gradeById(root.selectedGradeId)

    ") ?
        Review status: " +
        (gradeById(root.selectedGradeId).reviewed ? "Reviewed" : "Pending") + "\nFinal
        grade: " + gradeById(root.selectedGradeId).finalGrade + "\nAI grade: " +
        gradeById(root.selectedGradeId).aiGrade(

```

```
" :                                Choose a grade from the left to activate this
panel".
```

```
                                color: gradeById(root.selectedGradeId) ? muted :
muted2
```

```
                                font.pixelSize: 12

{
                                Item { Layout.fillHeight: true }
```

```
{
```

```
{
```

```
{
```

```
{
```

```
{
```

```
{
```

```
{
```

```
{
```

```
Component}
```

```
    id: messagesPage
```

```
    Flickable}
```

```
        contentWidth: width
```

```
        contentHeight: msgCol.implicitHeight
```

```
        clip: true
```

```
        ColumnLayout}
```

```
            id: msgCol
```

```
            width: parent.width
```

```
            spacing: 14
```

```
        PageHeader { title: "Messages"; iconText: "📢" }
```

```

SectionFrame}

    visible: classesModel.count === 0

    Layout.fillWidth: true

    implicitHeight: visible ? 92 : 0

    ColumnLayout { anchors.centerIn: parent; spacing: 4

        Text { text: "No classes available yet"; color: ink; font.pixelSize:
14; font.weight: Font.DemiBold }

        Text { text: "Create a class first to send messages."; color: muted;
font.pixelSize: 11 }

    {
    {

```

```

RowLayout}

    Layout.fillWidth: true

    spacing: 14

```

```

SectionFrame}

    Layout.fillWidth: true

    implicitHeight: 540

    ColumnLayout}

        anchors.fill: parent

        anchors.margins: 16

        spacing: 12

        LabeledCombo}

            label: "Send to class"

            model: classesModel

            currentIndex: Math.max(0,
classIndexById(root.messageTargetClassId))

```

```

        textRole: "name"

        onChosen: if (index >= 0) root.messageTargetClassId =
classesModel.get(index).classId
    {

        InputField { label: "Subject"; placeholder: "Example: Reminder
for tomorrow"; text: root.messageSubject; onTextChanged: root.messageSubject
= text }

        AreaField { label: "Message body"; placeholder: "Write your
message here"; text: root.messageBody; onTextChanged: root.messageBody =
text }

        RowLayout{

            Layout.fillWidth: true

            spacing: 8

            ActionButton { text: "Send"; onClicked:
root.saveMessage("Sent") }

            ActionButton { text: "Save draft"; kind: "ghost"; onClicked:
root.saveMessage("Draft") }

            Item { Layout.fillWidth: true }
        }
    {
    {
    {

        SectionFrame}

        Layout.fillWidth: true

        implicitHeight: 540

        Item}

        anchors.fill: parent

        anchors.margins: 16

        Text}

```

```
id: recentMessagesTitle
text: "Recent messages"
color: ink
font.pixelSize: 16
font.weight: Font.DemiBold
anchors.top: parent.top
anchors.left: parent.left
```

```
{
```

```
SearchField}
```

```
id: messagesSearchField
anchors.top: recentMessagesTitle.bottom
anchors.topMargin: 10
anchors.left: parent.left
anchors.right: parent.right
placeholderText: "Search messages by subject or text"
text: root.messageSearchText
onTextChanged: root.messageSearchText = text
```

```
{
```

```
Flickable}
```

```
id: recentMessagesFlick
anchors.top: messagesSearchField.bottom
anchors.topMargin: 12
anchors.left: parent.left
anchors.right: parent.right
anchors.bottom: parent.bottom
clip: true
```

```

        contentWidth: width
        contentHeight: recentMessagesColumn.height
        boundsBehavior: Flickable.StopAtBounds
        ScrollBar.vertical: StyledScrollBar { policy:
ScrollBar.AsNeeded }

Column}

        id: recentMessagesColumn
        width: Math.max(0, recentMessagesFlick.width - 12)
        spacing: 12

Rectangle}

        width: recentMessagesColumn.width
        visible: messageRows().length === 0
        height: visible ? 84 : 0
        radius: 14
        color: "#F8FAFC"
        border.width: visible ? 1 : 0
        border.color: line

ColumnLayout}

        anchors.centerIn: parent
        spacing: 4

        Text { text: "No messages yet"; color: ink;
font.pixelSize: 13; font.weight: Font.DemiBold }

        Text { text: "Sent and draft messages will appear
here."; color: muted; font.pixelSize: 11 }
{
{

```

```

Repeater}

    model: messageRows()

    delegate: Rectangle}

        width: recentMessagesColumn.width

        readonly property bool matchesSearchRow:
root.matchesSearch(modelData.subject + " " + modelData.body + " " +
modelData.className, root.messageSearchText)

        visible: matchesSearchRow

        implicitHeight: matchesSearchRow ? 76 : 0

        radius: 14

        color: "#F8FAFC"

        border.width: 1

        border.color: line

    ColumnLayout}

        anchors.fill: parent

        anchors.margins: 10

        spacing: 5

    RowLayout}

        Layout.fillWidth: true

        spacing: 8

        Text { Layout.fillWidth: true; text:
modelData.subject; color: ink; font.pixelSize: 13; font.weight: Font.DemiBold;
elide: Text.ElideRight }

        Badge { textValue: modelData.state; bgColor:
modelData.state === "Sent" ? greenSoft : amberSoft; fgColor: modelData.state
=== "Sent" ? green : amber }

    ActionButton}

        text: "Open"

```



```

                                kind: "ghost"
                                onClicked} :
                                    root.selectedMaterialTitle =
modelData.subject
                                    root.selectedMaterialMeta =
modelData.className + " • " + modelData.createdAtPretty + " • " +
modelData.state
                                    root.selectedMaterialDescription =
modelData.body
                                    messageDialog.open()
{
{
                                IconButton { text: "Delete"; kind: "ghost";
onClicked: root.deleteMessage(modelData.messageId) }
{
                                RowLayout{
                                    Layout.fillWidth: true
                                    spacing: 8
                                    Text { Layout.fillWidth: true; text:
modelData.className; color: muted; font.pixelSize: 11; elide: Text.ElideRight;
verticalAlignment: Text.AlignTop }
                                    Text { text: modelData.createdAtPretty; color:
muted; font.pixelSize: 11; horizontalAlignment: Text.AlignRight;
verticalAlignment: Text.AlignTop; Layout.rightMargin: 8 }
{
{
{
{
{
{
{

```

```
{  
{  
{  
{  
{  
{
```

```
Component}
```

```
    id: reportsPage
```

```
    Flickable}
```

```
        contentWidth: width
```

```
        contentHeight: repCol.implicitHeight
```

```
        clip: true
```

```
    ColumnLayout}
```

```
        id: repCol
```

```
        width: parent.width
```

```
        spacing: 14
```

```
    PageHeader { title: "Reports"; iconText: " " }
```

```
    RowLayout}
```

```
        Layout.fillWidth: true
```

```
        spacing: 14
```

```
    SectionFrame}
```

```
        Layout.fillWidth: true
```

```
        implicitHeight: 380
```

```

        ColumnLayout{
            anchors.fill: parent
            anchors.margins: 16
            spacing: 10
            Text { text: "System summary"; color: ink; font.pixelSize: 16;
font.weight: Font.DemiBold }

            ReportLine { label: "Classes"; value:
classesModel.count.toString ()}

            ReportLine { label: "Students"; value:
totalStudentsAll().toString ()}

            ReportLine { label: "Assignments"; value:
assignmentsModel.count.toString ()}

            ReportLine { label: "Materials"; value:
materialsModel.count.toString ()}

            ReportLine { label: "Saved messages"; value:
messageLogModel.count.toString ()}

            ReportLine { label: "Pending approvals"; value:
pendingStudentsModel.count.toString ()}
        }
    }

```

```

        SectionFrame{
            Layout.fillWidth: true
            implicitHeight: 380
            ColumnLayout{
                anchors.fill: parent
                anchors.margins: 16
                spacing: 10
                Text { text: "Useful insights"; color: ink; font.pixelSize: 16;
font.weight: Font.DemiBold }
            }
        }
    }

```

```
ReportLine { label: "Open assignments"; value:
countOpenAssignments().toString ()}
```

```
ReportLine { label: "Reviewed grades"; value:
(gradesModel.count - countUnreviewedGrades()).toString ()}
```

```
ReportLine { label: "Unreviewed grades"; value:
countUnreviewedGrades().toString ()}
```

```
ReportLine { label: "AI-enabled tasks"; value:
countAiEnabledAssignments().toString ()}
```

```
ReportLine { label: "Best class"; value: topClassByAverage() ?
topClassByAverage().name : "-" }
```

```
ReportLine { label: "Busiest class"; value:
busiestClassByOpenAssignments() ? busiestClassByOpenAssignments().name :
"-" }
```

```
{
```

```
{
```

```
{
```

```
{
```

```
{
```

```
{
```

```
component ReportLine : Rectangle}
```

```
property string label"" :
```

```
property string value"" :
```

```
Layout.fillWidth: true
```

```
implicitHeight: 42
```

```
radius: 12
```

```
color: "#F8FAFC"
```

```
border.width: 1
```

```
border.color: line
```

```

    RowLayout{
        anchors.fill: parent
        anchors.margins: 10
        Text { Layout.fillWidth: true; text: label; color: muted; font.pixelSize: 12 }
        Text { text: value; color: ink; font.pixelSize: 13; font.weight:
Font.DemiBold }
    }
    {
    {

```

```

component StyledScrollBar : ScrollBar{

```

```

    id: sb
    LayoutMirroring.enabled: false
    hoverEnabled: true
    interactive: true
    policy: ScrollBar.AsNeeded
    minimumSize: 0.14
    visible: size < 1.0
    z: 50

```

```

    padding: 0
    topPadding: 0
    bottomPadding: 0
    leftPadding: 0
    rightPadding: 0

```

```

    implicitWidth: sb.orientation === Qt.Vertical ? 10 : 10
    implicitHeight: sb.orientation === Qt.Horizontal ? 10 : 10
    width: sb.orientation === Qt.Vertical ? 10 : (parent ? parent.width : 10)

```

```

height: sb.orientation === Qt.Horizontal ? 10 : (parent ? parent.height : 10)

anchors}

    right: sb.orientation === Qt.Vertical && parent ? parent.right : undefined
    left: sb.orientation === Qt.Horizontal && parent ? parent.left : undefined
    top: parent ? parent.top : undefined
    bottom: parent ? parent.bottom : undefined
    rightMargin: 0
    leftMargin: 0
    topMargin: 0
    bottomMargin: 0
{

contentItem: Item}

    implicitWidth: sb.orientation === Qt.Vertical ? 10 : 72
    implicitHeight: sb.orientation === Qt.Horizontal ? 10 : 72

    Rectangle}

        anchors.fill: parent
        anchors.margins: 1
        radius: 4
        color: sb.pressed ? "#4338CA" : "#6366F1"
        opacity: (sb.hovered || sb.active || sb.pressed) ? 0.98 : 0.78
    {
    {

background: Item}

    Rectangle}

```

```

        anchors.fill: parent
        radius: 5
        color: "#EEF2FF"
        border.width: 1
        border.color: "#D9E0FF"
        opacity: sb.size < 1.0 ? 0.95 : 0.0
    {
    {
    {

```

```

component SearchField : Rectangle}
    property alias text: searchInput.text
    property string placeholderText: "Search"
    Layout.fillWidth: true
    implicitHeight: 48
    radius: 14
    color: "#F8FAFC"
    border.width: 1
    border.color: line
    RowLayout}
        anchors.fill: parent
        anchors.margins: 10
        spacing: 8
        Text { text: "🔍"; font.pixelSize: 14 }
        TextField}
            id: searchInput
            Layout.fillWidth: true
            placeholderText: parent.parent.placeholderText

```

```

        background: null
        color: ink
        font.pixelSize: 12
        selectByMouse: true
    {
    {
    {

component DateTimeSpin : ColumnLayout}
    id: dt
    property string label"" :
    property int from: 0
    property int to: 100
    property int value: 0
    signal valueModified(int value)
    spacing: 5
    Layout.fillWidth: true

    function formatValue(v){
        return dt.label === "Year" ? ("" + v) : (v < 10 ? "0" + v : "" + v)
    }

    function normalizeValue(rawText){
        var cleaned = (rawText || "").replace(/^[^0-9]/g, "")
        if (cleaned.length === 0)
            return dt.value
        var parsed = parseInt(cleaned, 10)
        if (isNaN(parsed))

```



```

        return dt.value
    if (parsed < dt.from)
        parsed = dt.from
    if (parsed > dt.to)
        parsed = dt.to
    return parsed
}

function commitEditor() {
    var nextValue = normalizeValue(editor.text)
    if (nextValue !== dt.value) {
        dt.value = nextValue
        dt.valueModified(nextValue)
    }
    editor.text = formatValue(dt.value)
}

function stepBy(delta) {
    commitEditor()
    var nextValue = dt.value + delta
    if (nextValue < dt.from)
        nextValue = dt.from
    if (nextValue > dt.to)
        nextValue = dt.to
    if (nextValue !== dt.value) {
        dt.value = nextValue
        dt.valueModified(nextValue)
    }
}

```

```

        editor.text = formatValue(dt.value)
    {

        Text { text: dt.label; color: muted; font.pixelSize: 11; font.weight:
Font.Medium }

        Rectangle}
            Layout.fillWidth: true
            implicitHeight: 48
            radius: 14
            color: "white"
            border.width: 1
            border.color: editor.activeFocus ? indigo600 : line

        RowLayout}
            anchors.fill: parent
            anchors.leftMargin: 14
            anchors.rightMargin: 8
            anchors.topMargin: 6
            anchors.bottomMargin: 6
            spacing: 8

        TextField}
            id: editor
            Layout.fillWidth: true
            Layout.minimumWidth: 0
            Layout.preferredWidth: 0
            implicitWidth: 0

```

```

        maximumLength: dt.label === "Year" ? 4 : 2
        text: dt.formatValue(dt.value)
        background: null
        color: ink
        font.pixelSize: 13
        font.weight: Font.DemiBold
        horizontalAlignment: Text.AlignLeft
        verticalAlignment: Text.AlignVCenter
        inputMethodHints: Qt.ImhDigitsOnly
        selectByMouse: true
        clip: true
        onEditingFinished: dt.commitEditor()
    }

    Column {
        spacing: 3

        Rectangle {
            width: 24
            height: 14
            radius: 7
            color: upMouse.pressed ? "#C7D2FE" : "#EEF2FF"
            border.width: 1
            border.color: "#D9E0FF"

            Text { anchors.centerIn: parent; text: "▲"; color: indigo700;
font.pixelSize: 9; font.weight: Font.Bold }

            MouseArea {
                id: upMouse

```

```

        anchors.fill: parent
        cursorShape: Qt.PointingHandCursor
        onClicked: dt.stepBy(1)
    {
    {

    Rectangle}
        width: 24
        height: 14
        radius: 7
        color: downMouse.pressed ? "#C7D2FE" : "#EEF2FF"
        border.width: 1
        border.color: "#D9E0FF"
        Text { anchors.centerIn: parent; text: "▼"; color: indigo700;
font.pixelSize: 9; font.weight: Font.Bold }
        MouseArea}
            id: downMouse
            anchors.fill: parent
            cursorShape: Qt.PointingHandCursor
            onClicked: dt.stepBy(1-)
        {
        {
        {
        {

    Connections}
        target: dt
        function onValueChanged} ()

```

```

        if (!editor.activeFocus)
            editor.text = dt.formatValue(dt.value)
    }
    {
    {
    {
    {
    {

```

TeacherHome.qml

```

import QtQuick 2.15
import QtQuick.Layouts 1.15
import QtQuick.Controls 2.15
import Qt5Compat.GraphicalEffects 1.0

```

```

Item}

```

```

    id: root
    anchors.fill: parent

```

```

// injected from Login push(...)
    property string userName: "Manager"
    property int userId: -1
    property int schoolId: 1
    property string schoolName: "My School"
    property string schoolAddress"" :
    property string schoolContactEmail"" :
    property var nav

```

```

// theme
    readonly property color bg: "#F6F7FB"

```

```
readonly property color card: "#FFFFFF"
readonly property color ink: "#0F172A"
readonly property color muted: "#64748B"
readonly property color indigo600: "#4F46E5"
readonly property color indigo700: "#4338CA"
readonly property color line: "#E5E7EB"
readonly property color muted2: "#94A3B8"
readonly property color good: "#22C55E"
readonly property color warn: "#F59E0B"
readonly property color danger: "#EF4444"
```

```
// demo data (replace later)
```

```
property var pendingUserData[] :
```

```
property var userData[] :
```

```
property var classesData[] :
```

```
property var assignmentsData[] :
```

```
property bool refreshInProgress: false
```

```
property bool refreshFeedbackActive: false
```

```
property int refreshPendingResponses: 0
```

```
property string refreshStatusText"" :
```

```
function startRefreshFeedback(expectedResponses){
```

```
    refreshInProgress = true
```

```
    refreshFeedbackActive = true
```

```
    refreshPendingResponses = expectedResponses
```

```

refreshStatusText = "Refreshing"...
refreshToastTimer.stop()
{

function finishRefreshFeedbackStep} ()
    if (!refreshFeedbackActive)
        return

refreshPendingResponses = Math.max(0, refreshPendingResponses - 1)
if (refreshPendingResponses === 0){
    refreshInProgress = false
    refreshFeedbackActive = false
    refreshStatusText = "Refreshed"
    refreshToastTimer.restart()
}
{

```

```

----- //  hooks you will wire  to server-----

function approveUser(uid) { auth.approveUser(uid(
refreshUsers{()

function blockUser(uid) { auth.blockUser(uid(
refreshUsers{ ()

function unblockUser(uid) { auth.unblockUser(uid(
refreshUsers{ ()

function refreshSchool(showFeedback){

    var shouldShowFeedback = (showFeedback === undefined) ? true :
showFeedback

```

```

    if (shouldShowFeedback)
        startRefreshFeedback(3)

    auth.getUsers(schoolId)
    auth.get_school_courses(schoolId)
    auth.get_school_assignments(schoolId)
}

function refreshUsers() { auth.getUsers(schoolId) }

```

```

Component.onCompleted} :
    refreshSchool(false)
{

```

```

Connections}
    target: auth
    function ongetCoursesResult(success, message, courses){
        if (success){
            classesData = courses
        }
        finishRefreshFeedbackStep()
    }
}

Connections}
    target: auth
    function ongetAssignmentsResult(success, message, assignments){

```



```

        if (success){
            assignmentsData = assignments
        }
        finishRefreshFeedbackStep()
    {
    {

Connections}
    target: auth
    function onGetUsersResult(success, message, users){
        if (success){
            var pending[] =
            var active[] =
            for (var i = 0; i < users.length; i++){
                if (users[i].status === "PENDING"){
                    pending.push(users[i])
                }
                else{
                    active.push(users[i])
                }
            }
            {
            {
                pendingUsersData = pending
                usersData = active
            }
            finishRefreshFeedbackStep()
        }
        {
        {

----- //  background-----

```

```

Rectangle}
  anchors.fill: parent
  color: bg
Rectangle}
  width: 520; height: 520; radius: 260
  x: -240; y: -240
  color: indigo600; opacity: 0.08
{
  Rectangle}
    width: 620; height: 620; radius: 310
    x: parent.width - 420; y: parent.height - 460
    color: indigo700; opacity: 0.07
{
{

```

----- // main card-----

```

Rectangle}
  id: shell
  anchors.fill: parent
  anchors.margins: 22
  radius: 24
  color: card
  border.width: 1
  border.color: line

  layer.enabled: true
  layer.effect: DropShadow}
    horizontalOffset: 0

```

```

        verticalOffset: 18

        radius: 30

        samples: 40

        color: "#1A000000"
    {

        ColumnLayout}

        anchors.fill: parent

        anchors.margins: 18

        spacing: 18

//        top bar

        Rectangle}

        Layout.fillWidth: true

        Layout.preferredHeight: 92

        radius: 20

        color: indigo700

        border.width: 1

        border.color: "#14000000"

        Rectangle}

        anchors.fill: parent

        radius: parent.radius

        color: "transparent"

        border.width: 1

        border.color: "#10FFFFFF"
    {

```

Item}

anchors.fill: parent

anchors.margins: 18

Row}

id: topBarLeftGroup

anchors.left: parent.left

anchors.verticalCenter: parent.verticalCenter

spacing: 8

Rectangle}

width: 48; height: 48; radius: 15

color: "#20FFFFFF"

border.width: 1

border.color: "#30FFFFFF"

clip: true

Image}

anchors.centerIn: parent

width: 36

height: 36

source: "png/AppLogo.png"

fillMode: Image.PreserveAspectFit

smooth: true

mipmap: true

{

{

```

Row}

    anchors.verticalCenter: parent.verticalCenter
    spacing: 10

    Text}

        anchors.verticalCenter: parent.verticalCenter
        text: "Classify"
        color: "#FFFFFF"
        opacity: 0.98
        font.pixelSize: 26
        font.weight: Font.DemiBold

{

    Text}

        anchors.verticalCenter: parent.verticalCenter
        width: Math.min(320, Math.max(0, topBarRightGroup.x -
(topBarLeftGroup.x + topBarLeftGroup.width) - 24))
        text: schoolName + " • " + userName
        color: "#FFFFFF"
        opacity: 0.95
        font.pixelSize: 18
        font.weight: Font.DemiBold
        elide: Text.ElideRight

{
{
{

Row}

```

```
id: topBarRightGroup
anchors.right: parent.right
anchors.verticalCenter: parent.verticalCenter
spacing: 8
```

```
Item}
```

```
width: 24
```

```
height: 24
```

```
visible: refreshInProgress
```

```
opacity: refreshInProgress ? 0.95 : 0
```

```
Text}
```

```
id: refreshSpinner
```

```
anchors.centerIn: parent
```

```
text"↻" :
```

```
color: "#FFFFFF"
```

```
font.pixelSize: 18
```

```
font.weight: Font.DemiBold
```

```
{
```

```
NumberAnimation on opacity}
```

```
duration: 140
```

```
{
```

```
RotationAnimator}
```

```
target: refreshSpinner
```

```
from: 0
```

```
to: 360
```

```

        duration: 850

        loops: Animation.Infinite

        running: refreshInProgress

    {
    {

        MyButton { text: refreshInProgress ? "Refreshing..." : "Refresh";
kind: "headerGhost"; onClicked: refreshSchool(true) }

        MyButton { text: "Logout"; kind: "headerGhost"; onClicked: { if
(nav) nav.pop ()} }

    {
    {
    {

//      body: sidebar + content
    RowLayout}

        Layout.fillWidth: true

        Layout.fillHeight: true

        spacing: 14

//      sidebar
    Rectangle}

        Layout.preferredWidth: 236

        Layout.fillHeight: true

        radius: 20

        color: "#FBFCFF"

        border.width: 1

        border.color: line

```

```

ColumnLayout{

    anchors.fill: parent

    anchors.margins: 12

    spacing: 10

    Text { text: "Navigation"; color: muted2; font.pixelSize: 12;
font.weight: Font.Medium }

    MenuItem { title: "Home"; iconText: "🏠"; pageKey: "dash" }
    MenuItem { title: "Users"; iconText: "👤"; pageKey: "users" }
    MenuItem { title: "Classes"; iconText: "📖"; pageKey: "classes" }
    MenuItem { title: "Assignments"; iconText: "📝"; pageKey: "tasks"
}

    MenuItem { title: "School settings"; iconText: "⚙️"; pageKey:
"settings" }

    Item { Layout.fillHeight: true }

    Rectangle { Layout.fillWidth: true; height: 1; color: line }
    Text { text: "Logged as: MANAGER"; color: muted; font.pixelSize:
11 }

    Rectangle{

        Layout.fillWidth: true

        height: 1

        color: line

    {

        Text{

            text: "School ID: " + schoolId

            color: indigo700

```



```

        font.pixelSize: 12
        font.weight: Font.DemiBold
    {
        Text{
            text: "Share this ID with\nstudents & teachers"
            color: muted
            font.pixelSize: 10
            wrapMode: Text.WordWrap
            Layout.fillWidth: true
        }
    }
    {
    }
    {

//      content panel
    Rectangle{
        Layout.fillWidth: true
        Layout.fillHeight: true
        radius: 20
        color: card
        border.width: 1
        border.color: line

        ColumnLayout{
            anchors.fill: parent
            anchors.margins: 14
            spacing: 16

//      stats row

```

```

        RowLayout{
            Layout.fillWidth: true
            spacing: 12

            StatCard { title: "Pending approvals"; value: "" +
pendingUsersData.length; iconText: "⌚" }

            StatCard { title: "Total users"; value: "" +
(pendingUsersData.length + usersData.length); iconText: "👤" }

            StatCard { title: "Classes"; value: "" + classesData.length;
iconText: "📅" }

            StatCard { title: "Assignments"; value: "" +
assignmentsData.length; iconText: "📝" }
        }

        Rectangle { Layout.fillWidth: true; height: 1; color: line }

//        title row
        RowLayout{
            Layout.fillWidth: true
            spacing: 10

            Text{
                Layout.fillWidth: true
                text: pageTitle
                color: ink
                font.pixelSize: 14
                font.weight: Font.DemiBold
                elide: Text.ElideRight
            }
        }

```

```

        MyButton}

        visible: currentPage === "users"

        text: "Refresh users"

        kind: "ghost"

        onClicked: refreshUsers()

{
{

        Loader}

        Layout.fillWidth: true

        Layout.fillHeight: true

        sourceComponent: currentPage === "dash" ? dashPage

:                currentPage === "users" ? usersPage
:                currentPage === "classes" ? classesPage
:                currentPage === "tasks" ? tasksPage
:                currentPage === "settings" ? settingsPage
:                statsPage

{
{
{
{
{
{
{

        Rectangle}

        id: refreshToast

        anchors.top: parent.top

        anchors.right: parent.right

```

```

anchors.topMargin: 34
anchors.rightMargin: 38
width: refreshToastText.implicitWidth + 28
height: 34
radius: 12
color: "#0F172A"
opacity: refreshStatusText != "" ? 0.92 : 0.0
visible: opacity > 0
z: 20

Behavior on opacity}
    NumberAnimation { duration: 160 }
{

Row}
    anchors.centerIn: parent
    spacing: 8

Text}
    text: refreshInProgress"√" : "⌛" ?
    color: "#FFFFFF"
    font.pixelSize: 13
    font.weight: Font.DemiBold
{

Text}
    id: refreshToastText
    text: refreshStatusText

```

```

        color: "#FFFFFF"

        font.pixelSize: 12

        font.weight: Font.Medium
    {
    {
    {

    Timer}

        id: refreshToastTimer

        interval: 1800

        repeat: false

        onTriggered: refreshStatusText"" =
    {

    ----- //  navigation state-----

        property string currentPage: "dash"

        property string pageTitle: "Home"

    function goPage(key){

        currentPage = key

        if (key === "dash") pageTitle = "Home"

        else if (key === "users") pageTitle = "Users"

        else if (key === "classes") pageTitle = "Classes"

        else if (key === "tasks") pageTitle = "Assignments"

        else if (key === "settings") pageTitle = "School settings"

        else pageTitle = "Home"

    {

```

```
===== // Reusable=====
```

```
component MyButton : Item{
    id: b
    property string text: "Button"
    property string kind: "ghost" // ghost | primary | danger | headerGhost
    signal clicked()

    implicitWidth: Math.min(126, Math.max(86, label.implicitWidth + 28))
    implicitHeight: 36

    Rectangle{
        anchors.fill: parent
        radius: 12
        color: kind === "primary" ? indigo600
:         kind === "danger" ? "#FEF2F2"
:         kind === "headerGhost" ? "#FFFFFF"
#" :         FFFFFFFF"
        opacity: kind === "headerGhost"
) ?         ma.pressed ? 0.22 : (ma.containsMouse ? 0.20 : 0.16)(
):         ma.pressed ? 0.90 : (ma.containsMouse ? 0.98 : 1.0)(
        border.width: 1
        border.color: kind === "primary" ? "#10FFFFFF"
:         kind === "danger" ? "#FECACA"
:         kind === "headerGhost" ? "#10FFFFFF"
:         line
    }
}
```

```

Text}

    id: label

    anchors.centerIn: parent

    text: b.text

    color: kind === "primary" ? "#FFFFFF"
:      kind === "danger" ? danger
:      kind === "headerGhost" ? "#FFFFFF"
:      indigo600

    opacity: kind === "headerGhost" ? 0.96 : 1.0

    font.pixelSize: 12

    font.weight: Font.DemiBold

    horizontalAlignment: Text.AlignHCenter

    verticalAlignment: Text.AlignVCenter

{

```

```

MouseArea}

    id: ma

    anchors.fill: parent

    hoverEnabled: true

    cursorShape: Qt.PointingHandCursor

    onClicked: b.clicked()

{
{

```

```

component MenuItem : Item}

    id: mi

    property string title"" :

    property string iconText"" :

```

```

property string pageKey"" :
implicitHeight: 46
Layout.fillWidth: true

Rectangle}
    anchors.fill: parent
    radius: 14
    color: root.currentPage === mi.pageKey ? "#EEF2FF" : "#FFFFFF"
    border.width: 1
    border.color: root.currentPage === mi.pageKey ? "#C7D2FE" : line
{

RowLayout}
    anchors.fill: parent
    anchors.margins: 10
    spacing: 10

Rectangle}
    Layout.alignment: Qt.AlignVCenter
    width: 32; height: 32; radius: 10
    color: root.currentPage === mi.pageKey ? "#DDE7FF" : "#EEF4FF"
    border.width: 1
    border.color: root.currentPage === mi.pageKey ? "#C7D2FE" :
"#D6E4FF"

Text}
    anchors.centerIn: parent
    text: mi.iconText

```



```

        color: root.currentPage === mi.pageKey ? indigo700 : "#4F46E5"
        font.pixelSize: 15
        font.weight: Font.DemiBold
    {
    {

    Text}
        Layout.fillWidth: true
        Layout.alignment: Qt.AlignVCenter
        text: mi.title
        color: root.currentPage === mi.pageKey ? indigo700 : ink
        font.pixelSize: 13
        font.weight: root.currentPage === mi.pageKey ? Font.DemiBold :
Font.Medium
        elide: Text.ElideRight
        verticalAlignment: Text.AlignVCenter
    {
    {

    MouseArea}
        anchors.fill: parent
        hoverEnabled: true
        cursorShape: Qt.PointingHandCursor
        onClicked: root.goPage(mi.pageKey)
    {
    {

    component StatCard : Rectangle}
        property string title"" :

```

```

property string value"" :
property string iconText"" :

Layout.fillWidth: true
Layout.preferredHeight: 96
radius: 16
color: "#FCFCFF"
border.width: 1
border.color: line

Row}
    anchors.verticalCenter: parent.verticalCenter
    anchors.left: parent.left
    anchors.leftMargin: 14
    spacing: 10

Rectangle}
    width: 40; height: 40; radius: 12
    color: "#E0E7FF"
    border.width: 1
    border.color: "#B8C7FF"
    anchors.verticalCenter: parent.verticalCenter
Text}
    anchors.centerIn: parent
    text: iconText
    color: indigo700
    font.pixelSize: 18
{

```

```

{

    Column{

        spacing: 3

        anchors.verticalCenter: parent.verticalCenter

        Text { text: title; color: muted; font.pixelSize: 12 }

        Text { text: value; color: ink; font.pixelSize: 24; font.weight:
Font.DemiBold }
    }
    {
    {
    {

    component Pill : Rectangle{

        property string text"" :

        property color fg: ink

        property color bgc: "#EEF2FF"

        property color bc: "#C7D2FE"


        implicitWidth: t.implicitWidth + 18

        implicitHeight: 26

        radius: 13

        color: bgc

        border.width: 1

        border.color: bc


        Text{

            id: t

            anchors.centerIn: parent

            text: parent.text

```

```

        color: parent.fg
        font.pixelSize: 11
        font.weight: Font.DemiBold
    {
    {

component InputBox : Rectangle}
    id: ib
    property string placeholder"" :
    property string text"" :
    signal textEdited(string t)

    radius: 12
    color: "#FFFFFF"
    border.width: 1
    border.color: line
    implicitHeight: 36

    TextInput}
        id: ti
        anchors.fill: parent
        anchors.margins: 10
        text: ib.text
        color: ink
        font.pixelSize: 12
        selectByMouse: true
        onTextChanged: { ib.text = text; ib.textEdited(text) }
    {

```

```

Text}

    visible: ti.text.length === 0

    anchors.left: parent.left

    anchors.leftMargin: 10

    anchors.verticalCenter: parent.verticalCenter

    text: ib.placeholder

    color: "#94A3B8"

    font.pixelSize: 12

{
{

===== //  Pages=====

Component}

    id: dashPage

    Item}

        ColumnLayout}

            anchors.fill: parent

            spacing: 12

            Text { text: "Quick actions"; color: muted2; font.pixelSize: 12;
font.weight: Font.Medium }

        GridLayout}

            Layout.fillWidth: true

            columns: 3

            columnSpacing: 12

```

rowSpacing: 12

```
    DashCard { title: "Approve users"; sub: "Review pending signups";  
    iconText: "⌚"; onClicked: root.goPage("users") }
```

```
    DashCard { title: "Manage users"; sub: "Block / unblock users";  
    iconText: "👤"; onClicked: root.goPage("users") }
```

```
    DashCard { title: "School settings"; sub: "Name, address, info";  
    iconText: "⚙️"; onClicked: root.goPage("settings") }
```

```
    DashCard { title: "Classes"; sub: "View classes"; iconText: "📅";  
    onClicked: root.goPage("classes") }
```

```
    DashCard { title: "Assignments"; sub: "View tasks"; iconText: "📝";  
    onClicked: root.goPage("tasks") }
```

```
{
```

```
    Rectangle { Layout.fillWidth: true; height: 1; color: line }
```

```
    RowLayout{
```

```
        Layout.fillWidth: true
```

```
        spacing: 10
```

```
        Text { text: "Pending approvals"; color: ink; font.pixelSize: 13;  
font.weight: Font.DemiBold }
```

```
        Item { Layout.fillWidth: true }
```

```
        MyButton { text: "Open"; kind: "ghost"; onClicked:  
root.goPage("users") }
```

```
{
```

```
    Rectangle{
```

```
        Layout.fillWidth: true
```

```
        Layout.fillHeight: true
```

```
        radius: 16
```

```
color: card
border.width: 1
border.color: line
```

```
Flickable}
    anchors.fill: parent
    anchors.margins: 12
    clip: true
    contentWidth: width
    contentHeight: col.implicitHeight
```

```
ColumnLayout}
    id: col
    width: parent.width
    spacing: 8
```

```
Repeater}
    model: root.pendingUsersData.length
    delegate: UserRow}
        userObj: root.pendingUsersData[index]
        pending: true
```

```
{
{
```

```
Text}
    visible: root.pendingUsersData.length === 0
    text: "No pending users" 📢
    color: muted
```

font.pixelSize: 12

{
{
{
{
{
{
{

component DashCard : Rectangle}

property string title"" :

property string sub"" :

property string iconText"" :

signal clicked()

Layout.fillWidth: true

Layout.preferredHeight: 110

radius: 16

color: "#FFFFFF"

border.width: 1

border.color: line

// content (slightly left aligned)

Column}

anchors.verticalCenter: parent.verticalCenter

anchors.left: parent.left

anchors.right: parent.right

anchors.leftMargin: 16

anchors.rightMargin: 16

spacing: 6

Rectangle}

width: 44; height: 44; radius: 14

color: "#E0E7FF"

border.width: 1

border.color: "#C7D2FE"

Text { anchors.centerIn: parent; text: iconText; color: indigo700;
font.pixelSize: 18; font.weight: Font.DemiBold }

{

Text}

text: title

color: ink

font.pixelSize: 14

font.weight: Font.DemiBold

horizontalAlignment: Text.AlignLeft

elide: Text.ElideRight

width: parent.width

{

Text}

text: sub

color: muted

font.pixelSize: 12

wrapMode: Text.WordWrap

horizontalAlignment: Text.AlignLeft

```

        width: parent.width
    {
    {
MouseArea}

        anchors.fill: parent
        hoverEnabled: true
        cursorShape: Qt.PointingHandCursor
        onClicked: parent.clicked()
    {
    {

Component}

        id: usersPage
        Item}

            property string tab: "pending" // pending | all
            property string roleSelected: "ALL"

        ColumnLayout}

            anchors.fill: parent
            spacing: 12

        RowLayout}

            Layout.fillWidth: true
            spacing: 10

        TabChip { label: "Pending (" + root.pendingUsersData.length + ")";
active: tab === "pending"; onClicked: tab = "pending" }

        TabChip { label: "Active users (" + root.usersData.length + ")"; active:
tab === "all"; onClicked: tab = "all" }

```

```

        Item { Layout.fillWidth: true }

{

        RowLayout{

            Layout.fillWidth: true

            spacing: 10

            InputBox { id: searchBox; Layout.fillWidth: true; placeholder:
"Search by name/email..." }

            Row{

                id: roleFilter

                spacing: 6

                Repeater{

                    model: ["ALL", "STUDENT", "TEACHER"]

                    delegate: Rectangle{

                        width: 80; height: 36; radius: 10

                        color: roleSelected === modelData ? "#EEF2FF" : card

                        border.width: 1

                        border.color: roleSelected === modelData ? indigo600 : line

                        Text{

                            anchors.centerIn: parent

                            text: modelData === "ALL" ? "All" : modelData ===
"STUDENT" ? "Student" : "Teacher"

                            color: roleSelected === modelData ? indigo600 : muted

                            font.pixelSize: 12

                            font.weight: Font.DemiBold

                        }
                    }
                }
            }
        }
    }
}

```

```

        MouseArea}

        anchors.fill: parent

        cursorShape: Qt.PointingHandCursor

        onClicked: roleSelected = modelData

    {
    {
    {
    {
    {

```

```

Rectangle { Layout.fillWidth: true; height: 1; color: line }

```

```

Rectangle}

    Layout.fillWidth: true

    Layout.fillHeight: true

    radius: 16

    color: card

    border.width: 1

    border.color: line

```

```

Flickable}

    anchors.fill: parent

    anchors.margins: 12

    clip: true

    contentWidth: width

    contentHeight: listCol.implicitHeight

```

```

ColumnLayout}

```

```

        id: listCol

        width: parent.width

        spacing: 8

        Repeater{

            model: tab === "pending" ? root.pendingUsersData.length :
root.usersData.length

            delegate: UserRow{

                userObj: tab === "pending" ?
root.pendingUsersData[index] : root.usersData[index]

                pending: tab === "pending"

                visible} :

                var u = userObj

                var q = (searchBox.text || "").toLowerCase().trim()

                var rf = roleSelected

                var ok = true

                if (q.length > 0){

                    ok = (u.name.toLowerCase().indexOf(q) !== -1) ||
(u.email.toLowerCase().indexOf(q) !== -1)

                {

                    if (ok && rf !== "ALL"){

                        ok = (u.role === rf)

                    {

                        return ok

                    {

                    {

```

```

        Text{
            visible: (tab === "pending" ? root.pendingUsersData.length :
root.usersData.length) === 0
            text: tab === "pending" ? "No pending users." : "No users".
            color: muted
            font.pixelSize: 12
        {
        {
        {
        {
        {
        {
        {

```

```

component TabChip : Item{
    property string label"" :
    property bool active: false
    signal clicked()

```

```

    implicitHeight: 34
    implicitWidth: Math.max(150, txt.implicitWidth + 26)

```

```

Rectangle{
    anchors.fill: parent
    radius: 12
    color: active ? "#EEF2FF" : "#FFFFFF"
    border.width: 1
    border.color: active ? "#C7D2FE" : line
{

```

```

Text}

    id: txt

    anchors.centerIn: parent

    text: label

    color: active ? indigo700 : ink

    font.pixelSize: 12

    font.weight: Font.DemiBold

    horizontalAlignment: Text.AlignHCenter

    verticalAlignment: Text.AlignVCenter
{

```

```

MouseArea}

    anchors.fill: parent

    hoverEnabled: true

    cursorShape: Qt.PointingHandCursor

    onClicked: parent.clicked()

{
{

```

// FIXED: actions always inside row, right side fixed width, no overflow

```

component UserRow : Rectangle}

    property var userObj({}) :

    property bool pending: false

    property bool compact: width < 760

    Layout.fillWidth: true

    Layout.preferredHeight: compact ? 112 : 76

```

radius: 14

color: card

border.width: 1

border.color: line

ColumnLayout}

anchors.fill: parent

anchors.margins: 12

spacing: compact ? 8 : 0

RowLayout}

Layout.fillWidth: true

spacing: 10

Rectangle}

Layout.alignment: Qt.AlignVCenter

width: 40; height: 40; radius: 14

color: pending ? "#FEF3C7" : (userObj.status === "BLOCKED" ?
"#FEE2E2" : "#E0E7FF")

border.width: 1

border.color: pending ? "#FCD34D" : (userObj.status ===
"BLOCKED" ? "#FCA5A5" : "#C7D2FE")

Text}

anchors.centerIn: parent

text: pending ? "⌚" : (userObj.status === "BLOCKED" ? "🚫" :
"👤")

color: pending ? "#B45309" : (userObj.status === "BLOCKED" ?
danger : indigo700)

font.pixelSize: 16


```

        font.weight: Font.DemiBold
    {
    {

        ColumnLayout}

        Layout.fillWidth: true

        Layout.minimumWidth: 0

        spacing: 2

        Text}

        Layout.fillWidth: true

        text: (userObj.name || "") + " • " + (userObj.role || "")

        color: ink

        font.pixelSize: 13

        font.weight: Font.DemiBold

        elide: Text.ElideRight
    {

        Text}

        Layout.fillWidth: true

        text: userObj.email || ""

        color: muted

        font.pixelSize: 12

        elide: Text.ElideRight
    {
    {

    //      right side: pill + actions (wide) OR pill only (compact)

```

ColumnLayout}

Layout.alignment: Qt.AlignVCenter

Layout.preferredWidth: compact ? 0 : 260

Layout.minimumWidth: compact ? 0 : 260

spacing: 0

RowLayout}

Layout.fillWidth: true

spacing: 8

Item { Layout.fillWidth: true }

MyButton}

visible: !compact && pending

text: "Approve"

kind: "ghost"

onClicked: root.approveUser(userObj.id)

{

MyButton}

visible: !compact && (!pending) && (userObj.status !==
"BLOCKED")

text: "Block"

kind: "danger"

onClicked: root.blockUser(userObj.id)

{

MyButton}

```

        visible: !compact && (!pending) && (userObj.status ===
"BLOCKED")

        text: "Unblock"

        kind: "ghost"

        onClicked: root.unblockUser(userObj.id)

    {

        Pill}

        visible: !compact

        text: pending ? "PENDING" : (userObj.status || "")

        fg: pending ? warn : (userObj.status === "BLOCKED" ? danger :
good)

        bgc: pending ? "#FFF7ED" : (userObj.status === "BLOCKED" ?
"#FEF2F2" : "#ECFDF5")

        bc: pending ? "#FED7AA" : (userObj.status === "BLOCKED" ?
"#FECACA" : "#BBF7D0")

    {
    {
    {
    {

//      compact actions row (second line)
    RowLayout}

        visible: compact

        Layout.fillWidth: true

        spacing: 8

        Item { Layout.fillWidth: true }

        MyButton}

```

```

        visible: pending
        text: "Approve"
        kind: "ghost"
        onClicked: root.approveUser(userObj.id)
    {

```

```

    MyButton}

        visible: (!pending) && (userObj.status !== "BLOCKED")
        text: "Block"
        kind: "danger"
        onClicked: root.blockUser(userObj.id)
    {

```

```

    MyButton}

        visible: (!pending) && (userObj.status === "BLOCKED")
        text: "Unblock"
        kind: "ghost"
        onClicked: root.unblockUser(userObj.id)
    {

```

```

    Pill}

        text: pending ? "PENDING" : (userObj.status || "")
        fg: pending ? warn : (userObj.status === "BLOCKED" ? danger :
good)
        bgc: pending ? "#FFF7ED" : (userObj.status === "BLOCKED" ?
"#FEF2F2" : "#ECFDF5")
        bc: pending ? "#FED7AA" : (userObj.status === "BLOCKED" ?
"#FECACA" : "#BBF7D0")
    {

```

```

{
{
{

Component}
    id: classesPage
    Item}
        ColumnLayout}
            anchors.fill: parent
            spacing: 12
            Text { text: "All classes in this school"; color: muted2; font.pixelSize:
12; font.weight: Font.Medium }

        Rectangle}
            Layout.fillWidth: true
            Layout.fillHeight: true
            radius: 16
            color: card
            border.width: 1
            border.color: line

        Flickable}
            anchors.fill: parent
            anchors.margins: 12
            clip: true
            contentWidth: width
            contentHeight: col.implicitHeight

```

ColumnLayout}

id: col

width: parent.width

spacing: 8

Repeater}

model: root.classesData.length

delegate: Rectangle}

Layout.fillWidth: true

Layout.preferredHeight: (width < 760 ? 98 : 74)

radius: 14

color: card

border.width: 1

border.color: line

RowLayout}

anchors.fill: parent

anchors.margins: 12

spacing: 10

Rectangle}

Layout.alignment: Qt.AlignVCenter

width: 40; height: 40; radius: 14

color: "#E0E7FF"

border.width: 1

border.color: "#C7D2FE"

Text { anchors.centerIn: parent; text: "📅"; color: indigo700; font.pixelSize: 16; font.weight: Font.DemiBold }

```
{
```

```
    ColumnLayout{
```

```
        Layout.fillWidth: true
```

```
        Layout.minimumWidth: 0
```

```
        spacing: 2
```

```
        Text{
```

```
            Layout.fillWidth: true
```

```
            text: root.classesData[index].name
```

```
            color: ink
```

```
            font.pixelSize: 13
```

```
            font.weight: Font.DemiBold
```

```
            elide: Text.ElideRight
```

```
    {
```

```
        Text{
```

```
            Layout.fillWidth: true
```

```
            text: "Teacher: " +  
root.classesData[index].teacher_name
```

```
            color: muted
```

```
            font.pixelSize: 12
```

```
            elide: Text.ElideRight
```

```
    {
```

```
    {
```

```
        Item { Layout.fillWidth: true } // ensure pill hugs right  
edge always
```

```
    Pill{
```

```
        Layout.alignment: Qt.AlignVCenter | Qt.AlignRight
```

```

        text: root.classesData[index].students + " students"
        fg: indigo700
        bgc: "#EEF2FF"
        bc: "#C7D2FE"
    {
    {
    {
    {

        Text}

        visible: root.classesData.length === 0
        text: "No classes".
        color: muted
        font.pixelSize: 12
    {
    {
    {
    {
    {
    {
    {

    Component}
    id: tasksPage
    Item}
        ColumnLayout}
        anchors.fill: parent
        spacing: 12

```



```
Text { text: "All assignments"; color: muted2; font.pixelSize: 12;
font.weight: Font.Medium }
```

```
Rectangle}
```

```
Layout.fillWidth: true
```

```
Layout.fillHeight: true
```

```
radius: 16
```

```
color: card
```

```
border.width: 1
```

```
border.color: line
```

```
Flickable}
```

```
anchors.fill: parent
```

```
anchors.margins: 12
```

```
clip: true
```

```
contentWidth: width
```

```
contentHeight: col.implicitHeight
```

```
ColumnLayout}
```

```
id: col
```

```
width: parent.width
```

```
spacing: 8
```

```
Repeater}
```

```
model: root.assignmentsData.length
```

```
delegate: Rectangle}
```

```
Layout.fillWidth: true
```

```
Layout.preferredHeight: (width < 760 ? 104 : 74)
```

radius: 14
color: card
border.width: 1
border.color: line

RowLayout}
anchors.fill: parent
anchors.margins: 12
spacing: 10

Rectangle}
Layout.alignment: Qt.AlignVCenter
width: 40; height: 40; radius: 14
color: "#E0E7FF"
border.width: 1
border.color: "#C7D2FE"

Text { anchors.centerIn: parent; text: "📝"; color:
indigo700; font.pixelSize: 16; font.weight: Font.DemiBold }
{

ColumnLayout}
Layout.fillWidth: true
Layout.minimumWidth: 0
spacing: 2
Text}
Layout.fillWidth: true
text: root.assignmentsData[index].title + " • " +
root.assignmentsData[index].teacher_name
color: ink

```

        font.pixelSize: 13
        font.weight: Font.DemiBold
        elide: Text.ElideRight
    {
        Text{
            Layout.fillWidth: true
            text: root.assignmentsData[index].course_name
+ " • Due: " + root.assignmentsData[index].due_date
            color: muted
            font.pixelSize: 12
            elide: Text.ElideRight
        }
    }

    Item { Layout.fillWidth: true } // ensure pill hugs right
    edge always

    Pill{
        Layout.alignment: Qt.AlignVCenter | Qt.AlignRight
        text:
root.assignmentsData[index].submission_count + " submitted"
        fg: "#6366F1"
        bgc: "#EEF2FF"
        bc: "#C7D2FE"
    }
}
{
{
{
{

```

```

        Text}

        visible: root.assignmentsData.length === 0

        text: "No assignments".

        color: muted

        font.pixelSize: 12

{
{
{
{
{
{
{
{

```

```

Component}

```

```

    id: settingsPage

```

```

    Item}

```

```

//      local state: track whether save succeeded

```

```

    property bool saved: false

```

```

    property bool saveInProgress: false

```

```

    property bool lastSaveOk: false

```

```

    property string saveMsg"" :

```

```

Connections}

```

```

    target: auth

```

```

    function onSchoolUpdateResult(success, message){

```

```

        saveInProgress = false

```

```

        lastSaveOk = success

```

```

        saveMsg = message || (success ? "OK" : "ERROR")

```

```

        saved = success

        console.log("School update result:", success)

        if (success){

            sfName.initialText = sfName.currentText

            sfAddr.initialText = sfAddr.currentText

            sfMgrName.initialText = sfMgrName.currentText

            sfMgrEmail.initialText = sfMgrEmail.currentText


            root.schoolName = sfName.currentText

            root.userName = sfMgrName.currentText

            root.schoolAddress = sfAddr.currentText

            root.schoolContactEmail = sfMgrEmail.currentText

        }
    }
}

```

ColumnLayout}

anchors.fill: parent

spacing: 14

— // section header

RowLayout}

Layout.fillWidth: true

spacing: 12

Rectangle}

width: 40; height: 40; radius: 14

color: "#E0E7FF"

```

        border.width: 1
        border.color: "#C7D2FE"
        Text { anchors.centerIn: parent; text: "⚙️"; font.pixelSize: 18 }
    }

    ColumnLayout {
        spacing: 2
        Text {
            text: "School Settings"
            color: ink
            font.pixelSize: 15
            font.weight: Font.DemiBold
        }
        Text {
            text: "Update your school's public information"
            color: muted
            font.pixelSize: 12
        }
    }
}

Item { Layout.fillWidth: true }

// success badge (shown after save)
Rectangle {
    visible: saved
    implicitWidth: savedLabel.implicitWidth + 22
    implicitHeight: 30
    radius: 10
}

```

```

        color: "#ECFDF5"
        border.width: 1
        border.color: "#BBF7D0"
        Text{
            id: savedLabel
            anchors.centerIn: parent
            text: "✓ Saved"
            color: good
            font.pixelSize: 12
            font.weight: Font.DemiBold
        }
    {
    {
    {

— //      divider


---



        Rectangle { Layout.fillWidth: true; height: 1; color: line }

— //      two-column form card

---


        Rectangle{
            Layout.fillWidth: true
            Layout.fillHeight: true
            Layout.minimumHeight: 360
            radius: 18
            color: card
            border.width: 1
            border.color: line
            implicitHeight: formCol.implicitHeight + 32

```

```

ColumnLayout}

    id: formCol

    anchors}

        left: parent.left; right: parent.right

        top: parent.top

        margins: 20

{

    spacing: 0

    — //          group: School info—————

    SettingsGroupHeader { label: "School information"; iconTxt: "🏫"

}

```

```

SettingsField}

    id: sfName

    fieldLabel: "School name"

    hint: "Full official name of the school"

    initialText: root.schoolName

    iconTxt"🏷️" :

{

```

```

SettingsField}

    id: sfAddr

    fieldLabel: "Address"

    hint: "Street address, city"

    initialText: root.schoolAddress

    iconTxt"📍" :

```



```

{

— //          divider—————

Rectangle}
    Layout.fillWidth: true
    height: 1
    color: line
    Layout.topMargin: 10
    Layout.bottomMargin: 6

{

— //          group: Manager contact—————

SettingsGroupHeader { label: "Manager contact"; iconTxt: "👤" }

SettingsField}
    id: sfMgrName
    fieldLabel: "Manager name"
    hint: "Full name of the school manager"
    initialText: root.userName
    iconTxt"☐" :

{

SettingsField}
    id: sfMgrEmail
    fieldLabel: "Manager email"
    hint: "Official contact email"
    initialText: root.schoolContactEmail
    iconTxt"✉️" :

```

```

{

    — //          action row—————

    RowLayout{
        Layout.fillWidth: true
        Layout.topMargin: 16
        Layout.bottomMargin: 4
        spacing: 10

        Item { Layout.fillWidth: true }

    //          Discard button
    Item}

        implicitWidth: discardLabel.implicitWidth + 28
        implicitHeight: 36

    Rectangle}

        anchors.fill: parent
        radius: 14
        color: "#FFFFFF"
        border.width: 1
        border.color: line

        opacity: discardMa.pressed ? 0.85 :
(discardMa.containsMouse ? 0.97 : 1.0)
    {

        Text}

        id: discardLabel

        anchors.centerIn: parent

```

```

        text: "Discard"
        color: muted
        font.pixelSize: 12
        font.weight: Font.DemiBold
    {
        MouseArea{
            id: discardMa
            enabled: !saveInProgress
            anchors.fill: parent
            hoverEnabled: true
            cursorShape: Qt.PointingHandCursor
            onClicked} :
                sfName.reset()
                sfAddr.reset()
                sfMgrName.reset()
                sfMgrEmail.reset()
                saved = false
                saveMsg"" =
                lastSaveOk = false
        {
        {
        {

//        Save button
Item}

        implicitWidth: saveLabel.implicitWidth + 36
        implicitHeight: 36

```

```

Rectangle}

    anchors.fill: parent

    radius: 14

    color: indigo600

    border.width: 1

    border.color: "#10FFFFFF"

    opacity: saveMa.pressed ? 0.88 :
(saveMa.containsMouse ? 0.95 : 1.0)
{
    Text}

    id: saveLabel

    anchors.centerIn: parent

    text: "Save changes"

    color: "#FFFFFF"

    font.pixelSize: 12

    font.weight: Font.DemiBold

{

    MouseArea}

    id: saveMa

    enabled: !saveInProgress

    anchors.fill: parent

    hoverEnabled: true

    cursorShape: Qt.PointingHandCursor

    onClicked} :

        saved = false

        saveMsg"" =

        lastSaveOk = false

        saveInProgress = true

```

```

//                                Optimistically update the header texts (will be
committed on success)

    root.schoolName = sfName.currentText
    root.userName  = sfMgrName.currentText

    auth.updateSchool(
        root.schoolId,
        sfName.currentText,
        sfAddr.currentText,
        sfMgrName.currentText,
        sfMgrEmail.currentText
    )
(
{
{
{
{
{
{
{

—— //      save feedback—————
    Rectangle}
    Layout.fillWidth: true
    visible: saveInProgress || saveMsg.length > 0
    radius: 14
    color: saveInProgress ? "#EFF6FF" : (lastSaveOk ? "#ECFDF5" :
"#FEF2F2")
    border.width: 1

```

```

        border.color: saveInProgress ? "#BFDBFE" : (lastSaveOk ?
"#BBF7D0" : "#FECACA")

        implicitHeight: fbRow.implicitHeight + 18

    RowLayout}

        id: fbRow

        anchors.left: parent.left

        anchors.right: parent.right

        anchors.margins: 10

        spacing: 10

    Text}

        text: saveInProgress ? "⌚" : (lastSaveOk ? "✅" : "⚠")

        font.pixelSize: 14

{

    Text}

        Layout.fillWidth: true

        text: saveInProgress ? "Saving..." : saveMsg

        color: saveInProgress ? indigo700 : (lastSaveOk ? good :
danger)

        font.pixelSize: 12

        elide: Text.ElideRight

{

{

{

—— //bottom info card——

    Rectangle}

        Layout.fillWidth: true

```

```
implicitHeight: 92
Layout.maximumHeight: implicitHeight
radius: 18
color: "#FBFCFF"
border.width: 1
border.color: line
```

```
RowLayout}
```

```
anchors.fill: parent
anchors.margins: 16
spacing: 14
```

```
Rectangle}
```

```
width: 36; height: 36; radius: 12
color: "#EEF2FF"
border.width: 1; border.color: "#C7D2FE"
```

```
Text { anchors.centerIn: parent; text: "i"; font.pixelSize: 16 }
```

```
{
```

```
ColumnLayout}
```

```
Layout.fillWidth: true
spacing: 3
```

```
Text}
```

```
text: "Changes are saved to the server"
color: ink
font.pixelSize: 12
font.weight: Font.DemiBold
```

```
{
```

```
Text}
```

```
        text: "Once you click \"Save changes\", the updated details  
will be sent to the server and reflected across the system".
```

```
        color: muted
```

```
        font.pixelSize: 11
```

```
        wrapMode: Text.WordWrap
```

```
        Layout.fillWidth: true
```

```
{
```

```
{
```

```
{
```

```
{
```

```
{
```

```
— // reusable sub-components
```

```
component SettingsGroupHeader : RowLayout{
```

```
    property string label"" :
```

```
    property string iconTxt"" :
```

```
    Layout.fillWidth: true
```

```
    Layout.topMargin: 6
```

```
    Layout.bottomMargin: 8
```

```
    spacing: 8
```

```
Text}
```

```
    text: iconTxt
```



```

        font.pixelSize: 13
    {
        Text{
            text: label
            color: indigo700
            font.pixelSize: 12
            font.weight: Font.DemiBold
            Layout.fillWidth: true
        }
    }
}

```

```

component SettingsField : Item{
    id: sf
    property string fieldLabel"" :
    property string hint"" :
    property string initialText"" :
    property string iconTxt"" :
    property string currentText: ti2.text

    function reset() { ti2.text = initialText }

    Layout.fillWidth: true
    implicitHeight: 68

    RowLayout{
        anchors.fill: parent
        anchors.bottomMargin: 8
        spacing: 12
    }
}

```

```

//          icon badge
Rectangle}
    Layout.alignment: Qt.AlignTop
    Layout.topMargin: 22
    width: 32; height: 32; radius: 10
    color: "#EEF4FF"
    border.width: 1; border.color: "#D6E4FF"
    Text { anchors.centerIn: parent; text: sf.iconTxt; font.pixelSize: 14
}
{

    ColumnLayout}
        Layout.fillWidth: true
        spacing: 5

        Text}
            text: sf.fieldLabel
            color: ink
            font.pixelSize: 12
            font.weight: Font.DemiBold
{

    Rectangle}
        Layout.fillWidth: true
        height: 36
        radius: 10
        color: "#FFFFFF"

```

```

border.width: 1

border.color: ti2.activeFocus ? indigo600 : "#E2E8F0"

TextInput}

    id: ti2

    anchors.fill: parent

    anchors.leftMargin: 12

    anchors.rightMargin: 12

    anchors.verticalCenter: parent.verticalCenter

    verticalAlignment: TextInput.AlignVCenter

    text: sf.initialText

    color: ink

    font.pixelSize: 12

    selectByMouse: true

    clip: true

{

//          placeholder
Text}

    visible: ti2.text.length === 0

    anchors.left: parent.left

    anchors.leftMargin: 12

    anchors.verticalCenter: parent.verticalCenter

    text: sf.hint

    color: "#94A3B8"

    font.pixelSize: 12

{

{

```

```
{  
{  
{  
{  
{
```

```
    component StatLine : RowLayout}  
        property string label"" :  
        property string value"" :  
        Layout.fillWidth: true  
        spacing: 10  
        Text { text: label; color: muted; font.pixelSize: 12; Layout.fillWidth: true;  
elide: Text.ElideRight }  
        Text { text: value; color: ink; font.pixelSize: 12; font.weight: Font.DemiBold }  
{  
{
```

ManagerHome.qml

```
import QtQuick 2.15  
import QtQuick.Controls 2.15  
import QtQuick.Layouts 1.15  
import QtQuick.Dialogs
```

```
Item}  
    id: root  
    anchors.fill: parent
```

```

property string userName: "Admin"
property int userId: -1
property var nav: null

property int activeSectionIndex: 0
property string searchText"" :
property bool loading: false
property string statusText: "Ready"
property bool statusIsError: false
property var selectedRow: null
property int selectedIndex: -1

property bool editorVisible: false
property bool confirmVisible: false
property string editMode: "edit"
property string editEntity"" :
property var editRow: null
property string fileDialogTarget"" :
property string pendingDeleteEntity"" :
property var pendingDeleteRow: null

property var allData}) :
    schools: [], users: [], courses: [], members: [], assignments: [],
submissions,[] :
    ai_results: [], grades: [], feedback: [], materials: [], messages,[] :
    message_reads: [], grade_reads: [], activity: [], system[] :
({
property var stats({}) :

```

readonly property bool compact: width < 1120

readonly property bool narrow: width < 860

readonly property int sidebarWidth: narrow ? 0 : 252

readonly property color bg: "#F6F7FB"

readonly property color panel: "#FFFFFF"

readonly property color panel2: "#F8FAFC"

readonly property color ink: "#0F172A"

readonly property color muted: "#64748B"

readonly property color faint: "#94A3B8"

readonly property color line: "#E2E8F0"

readonly property color lineSoft: "#EEF2F7"

readonly property color side: "#111827"

readonly property color primary: "#2563EB"

readonly property color primarySoft: "#EFF6FF"

readonly property color teal: "#0F766E"

readonly property color tealSoft: "#ECFDF5"

readonly property color amber: "#D97706"

readonly property color amberSoft: "#FFFBEF"

readonly property color red: "#DC2626"

readonly property color redSoft: "#FEE2E2"

readonly property color violet: "#7C3AED"

property var sections] :

```
}      key: "schools", label: "Schools", short: "SC", id: "school_id", create: true,  
edit: true, remove: true,{
```

```
}      key: "users", label: "Users", short: "US", id: "id", create: true, edit: true,  
remove: true,{
```

```

    }    key: "courses", label: "Courses", short: "CR", id: "course_id", create: true,
edit: true, remove: true,{

    }    key: "members", label: "Members", short: "MB", id: "member_id", create:
false, edit: false, remove: false,{

    }    key: "assignments", label: "Assignments", short: "AS", id: "assignment_id",
create: true, edit: true, remove: true,{

    }    key: "submissions", label: "Submissions", short: "SB", id: "submission_id",
create: false, edit: false, remove: false,{

    }    key: "grades", label: "Grades", short: "GR", id: "grade_id", create: false, edit:
false, remove: false,{

    }    key: "ai_results", label: "AI Results", short: "AI", id: "ai_result_id", create:
false, edit: false, remove: false,{

    }    key: "feedback", label: "Feedback", short: "FB", id: "feedback_id", create:
false, edit: false, remove: false,{

    }    key: "materials", label: "Materials", short: "MT", id: "material_id", create:
false, edit: false, remove: true,{

    }    key: "messages", label: "Messages", short: "MS", id: "message_id", create:
false, edit: false, remove: true,{

    }    key: "message_reads", label: "Message Reads", short: "MR", id: "read_id",
create: false, edit: false, remove: false,{

    }    key: "grade_reads", label: "Grade Reads", short: "RR", id: "read_id", create:
false, edit: false, remove: false,{

    }    key: "activity", label: "Activity", short: "AC", id: "at", create: false, edit: false,
remove: false,{

    }    key: "system", label: "System", short: "SY", id: "metric", create: false, edit:
false, remove: false{

```

```
[
```

```
ListModel}
```

```
id: filteredModel
```

```
dynamicRoles: true
```

```
{
```

```

Timer}

    id: statusTimer

    interval: 3600

    onTriggered} :
        if (!loading)}
            statusText = "Ready"
            statusIsError = false
{
{
{

function asText(v)}
    if (v === undefined || v === null)
        return ""
    return String(v)
{

function safe(v)}
    var t = asText(v).trim()
    return t.length === 0 ? "-" : t
{

function shortText(v, maxLen)}
    var t = safe(v)
    if (t.length <= maxLen)
        return t
    return t.substring(0, Math.max(0, maxLen - 3))"..." +

```



```
{
```

```
function shortDate(v){
```

```
    var t = safe(v)
```

```
    if (t === "-")
```

```
        return t
```

```
    return t.replace("T", " ").substring(16, 0)
```

```
{
```

```
function fileName(v){
```

```
    var t = safe(v)
```

```
    if (t === "-")
```

```
        return t
```

```
    var clean = t.replace(/\\g, "/")
```

```
    var parts = clean.split("/")
```

```
    return parts.length > 0 ? parts[parts.length - 1] : t
```

```
{
```

```
function fileNameFromPath(v){
```

```
    var t = asText(v).replace("file://", "").replace(/\\g, "/")
```

```
    var parts = t.split("/")
```

```
    return parts.length > 0 ? parts[parts.length - 1] : t
```

```
{
```

```
function formatBytes(value){
```

```
    var n = Number(value || 0)
```

```
    if (n < 1024)
```

```
        return n + " B"
```

```

    if (n < 1024 * 1024)
        return (n / 1024).toFixed(1) + " KB"
    if (n < 1024 * 1024 * 1024)
        return (n / 1024 / 1024).toFixed(1) + " MB"
    return (n / 1024 / 1024 / 1024).toFixed(1) + " GB"
}

function pad2(n){
    return n < 10 ? "0" + n : "" + n
}

function defaultDueDate() {
    var d = new Date()
    d.setDate(d.getDate() + 7)
    return d.getFullYear() + "-" + pad2(d.getMonth() + 1) + "-" + pad2(d.getDate())
    + " 23:59"
}

function sectionAt(index){
    if (index < 0 || index >= sections.length)
        return sections[0]
    return sections[index]
}

function currentSection() {
    return sectionAt(activeSectionIndex)
}

function rowsForKey(key){

```

```

        return allData && allData[key] ? allData[key][] :
    {

        function currentRows() {
            return rowsForKey(currentSection().key)
        }

        function countFor(key) {
            return rowsForKey(key).length
        }

        function countByValue(rows, field, value) {
            var n = 0
            var wanted = asText(value).toUpperCase()
            for (var i = 0; i < rows.length; i++) {
                if (asText(rows[i][field]).toUpperCase() === wanted)
                    n++
            }
            return n
        }

        function rowId(row) {
            if (!row)
                return ""
            var idKey = currentSection().id
            return safe(row[idKey])
        }
    }

```

```

function primaryOf(row){
    if (!row)
        return""

    var key = currentSection().key

    if (key === "schools") return safe(row.name)
    if (key === "users") return safe(row.name)
    if (key === "courses") return safe(row.name)
    if (key === "members") return safe(row.student_name)
    if (key === "assignments") return safe(row.title)
    if (key === "submissions") return safe(row.assignment_title)
    if (key === "grades") return safe(row.student_name)
    if (key === "ai_results") return safe(row.engine_name)
    if (key === "feedback") return safe(row.student_name)
    if (key === "materials") return safe(row.title)
    if (key === "messages") return safe(row.subject)
    if (key === "message_reads") return safe(row.message_subject)
    if (key === "grade_reads") return safe(row.assignment_title)
    if (key === "activity") return safe(row.title)
    if (key === "system") return safe(row.metric)

    return rowId(row)
}

function secondaryOf(row){
    if (!row)
        return""

    var key = currentSection().key

    if (key === "schools") return safe(row.contact_email || row.address)
    if (key === "users") return safe(row.email) + " / " + safe(row.school_name)

```

```

        if (key === "courses") return "Teacher: " + safe(row.teacher_name) + " /
Code: " + safe(row.class_code)

        if (key === "members") return safe(row.course_name) + " / " +
safe(row.student_email)

        if (key === "assignments") return safe(row.course_name) + " / Due: " +
shortDate(row.due_date)

        if (key === "submissions") return safe(row.student_name) + " / " +
shortDate(row.submitted_at)

        if (key === "grades") return safe(row.course_name) + " / Final: " +
safe(row.final_score)

        if (key === "ai_results") return safe(row.assignment_title) + " / Score: " +
safe(row.score)

        if (key === "feedback") return safe(row.assignment_title)

        if (key === "materials") return safe(row.course_name) + " / " +
fileName(row.file_path)

        if (key === "messages") return safe(row.course_name) + " / " +
shortDate(row.created_at)

        if (key === "message_reads") return safe(row.student_name) + " / " +
shortDate(row.read_at)

        if (key === "grade_reads") return safe(row.student_name) + " / " +
shortDate(row.read_at)

        if (key === "activity") return safe(row.type) + " / " + shortDate(row.at)

        if (key === "system") return safe(row.value) + " " + safe(row.unit)

        return ""
    }

function statusOf(row){

    if (!row)

        return ""

    var key = currentSection().key

    if (key === "users") return safe(row.status)

```

```

    if (key === "courses") return safe(row.assignments_count) + " tasks"

    if (key === "members") return safe(row.member_status)

    if (key === "assignments") return Number(row.is_closed || 0) ? "Closed" :
"Open"

    if (key === "submissions") return safe(row.status)

    if (key === "grades") return row.final_score === null || row.final_score ===
undefined ? "Draft" : "Final"

    if (key === "ai_results") return safe(row.score)

    if (key === "materials") return safe(row.type)

    if (key === "messages") return safe(row.state)

    if (key === "schools") return safe(row.users_count) + " users"

    if (key === "system") return safe(row.unit)

    return currentSection().short
{

```

```

function accentOf(row){
    if (!row)
        return primary

    var status = statusOf(row).toUpperCase()

    var role = asText(row.role).toUpperCase()

    if (status.indexOf("BLOCK") >= 0 || status.indexOf("CLOSED") >= 0 ||
status.indexOf("FAILED") >= 0)
        return red

    if (status.indexOf("PENDING") >= 0 || status.indexOf("DRAFT") >= 0)
        return amber

    if (status.indexOf("OPEN") >= 0 || status.indexOf("ACTIVE") >= 0 ||
status.indexOf("FINAL") >= 0)
        return teal

    if (role === "ADMIN")

```

```

        return violet
    if (role === "TEACHER")
        return teal
    return primary
}

function rowMatches(row, q){
    if (!q || q.length === 0)
        return true
    var joined"" =
    for (var key in row)
        joined += " " + asText(row[key])
    return joined.toLowerCase().indexOf(q) >= 0
}

function makeListRow(row, sourceIndex){
    var item{} =
    for (var key in row)
        item[key] = row[key]
    item.sourceIndex = sourceIndex
    item.rowIdText = rowId(row)
    item.primaryText = primaryOf(row)
    item.secondaryText = secondaryOf(row)
    item.statusTextValue = statusOf(row)
    item.accentValue = accentOf(row)
    return item
}

```

```

function rebuildFilter} ()
    filteredModel.clear()
    var data = currentRows()
    var q = searchText.toLowerCase().trim()
    for (var i = 0; i < data.length; i++)
        if (rowMatches(data[i], q))
            filteredModel.append(makeListRow(data[i], i))
{
    if (selectedRow === null && filteredModel.count > 0)
        selectBySource(filteredModel.get(0).sourceIndex)
{

function selectBySource(sourceIndex)}
    var data = currentRows()
    if (sourceIndex < 0 || sourceIndex >= data.length)}
        selectedRow = null
        selectedSourceIndex = -1
        return
{
    selectedSourceIndex = sourceIndex
    selectedRow = data[sourceIndex]
{

function switchSection(index)}
    activeSectionIndex = index
    searchText"" =
    selectedRow = null
    selectedSourceIndex = -1

```



```

        rebuildFilter()
    {

function setStatus(message, isError){
    statusText = message && message.length > 0 ? message : "Ready"
    statusIsError = isError === true
    statusTimer.restart()
}

function refreshAll} ()
    if (typeof auth === "undefined")
        return
    loading = true
    statusIsError = false
    statusText = "Refreshing data"...
    auth.get_admin_overview()
{

function logout} ()
    if (typeof auth !== "undefined" && auth)
        auth.logout()
    if (nav)
        nav.pop()
{

function copyPayload(payload){
    var next{} =
    for (var i = 0; i < sections.length; i++){

```

```

        var key = sections[i].key

        next[key] = payload && payload[key] ? payload[key][] :

    {

        allData = next

        stats = payload && payload.stats ? payload.stats{} :

        selectedRow = null

        selectedIndex = -1

        rebuildFilter()

    {

        function roleOptions() {

            return [

                {
                    label: "Admin", value: "ADMIN",
                },
                {
                    label: "Manager", value: "MANAGER",
                },
                {
                    label: "Teacher", value: "TEACHER",
                },
                {
                    label: "Student", value: "STUDENT",
                }
            ]
        }

        function statusOptions() {

            return [

                {
                    label: "Active", value: "ACTIVE",
                },
                {
                    label: "Pending", value: "PENDING",
                },
                {
                    label: "Blocked", value: "BLOCKED",
                }
            ]
        }

        function schoolOptions() {

```

```

var out[] =

var rows = rowsForKey("schools")

for (var i = 0; i < rows.length; i++)

    out.push({ label: safe(rows[i].name) + " (#" + rows[i].school_id + ")",
value: Number(rows[i].school_id) })

return out
{

function teacherOptions} ()

var out[] =

var rows = rowsForKey("users")

for (var i = 0; i < rows.length; i++){

    var role = asText(rows[i].role).toUpperCase()

    if (role === "TEACHER" || role === "ADMIN")

        out.push({ label: safe(rows[i].name) + " (#" + rows[i].id + ")", value:
Number(rows[i].id) })

{

    return out

{

function courseOptions} ()

var out[] =

var rows = rowsForKey("courses")

for (var i = 0; i < rows.length; i++)

    out.push({ label: safe(rows[i].name) + " (#" + rows[i].course_id + ")",
value: Number(rows[i].course_id) })

return out

{

```

```

function firstOptionValue(options, fallback){
    return options.length > 0 ? options[0].value : fallback
}

function entityForSectionKey(key){
    if (key === "schools") return "school"
    if (key === "users") return "user"
    if (key === "courses") return "course"
    if (key === "assignments") return "assignment"
    if (key === "materials") return "material"
    if (key === "messages") return "message"
    return key
}

function openCreate(entity){
    editMode = "create"
    editEntity = entity
    if (entity === "school"){
        editRow = { name: "", address: "", contact_name: "", contact_email: "" }
    }
    else if (entity === "user") {
        editRow = {
            name: "", email: "", password: "123456", role: "STUDENT", status:
"ACTIVE,"
            school_id: firstOptionValue(schoolOptions(), 1)
        }
    }
    else if (entity === "course") {
        editRow = {
            name: "", description, "" :

```

```

        teacher_id: firstOptionValue(teacherOptions(), userId),
        school_id: firstOptionValue(schoolOptions(), 1)
    {
    {
        else if (entity === "assignment") {
            editRow} =
                title: "", description: "", due_date: defaultDueDate,()
                course_id: firstOptionValue(courseOptions(), 1),
                ai_enabled: false, is_closed: false,
                attachment_path: "", attachment_name"" :
            {
            {
                editorVisible = true
            {

```

```

function openEdit(entity, row)}
    if (!row)
        return
    editMode = "edit"
    editEntity = entity
    editRow{} =
    for (var key in row)
        editRow[key] = row[key]
    editorVisible = true
{

```

```

function requestDelete(entity, row)}
    if (!row)
        return

```

```

    pendingDeleteEntity = entity
    pendingDeleteRow = row
    confirmVisible = true
{

function validateEditor} ()
    if (!editRow)
        return false
    if (editEntity === "school" && safe(editRow.name) === "-")
        setStatus("School name is required.", true)
        return false
{
    if (editEntity === "user" && (safe(editRow.name) === "-" ||
safe(editRow.email) === "-"))
        setStatus("User name and email are required.", true)
        return false
{
    if (editEntity === "course" && safe(editRow.name) === "-")
        setStatus("Course name is required.", true)
        return false
{
    if (editEntity === "assignment" && safe(editRow.title) === "-")
        setStatus("Assignment title is required.", true)
        return false
{
    if (editEntity === "assignment" && editMode === "create" &&
Boolean(editRow.ai_enabled) && safe(editRow.attachment_path) === "-")
        setStatus("AI assignments need a rubric attachment.", true)
        return false

```

```

{
    return true
}

function saveEditor() {
    if (!validateEditor() || typeof auth === "undefined")
        return

    if (editEntity === "school") {
        if (editMode === "create")
            auth.admin_create_school(asText(editRow.name),
asText(editRow.address), asText(editRow.contact_name),
asText(editRow.contact_email))
        else
            auth.admin_update_school(Number(editRow.school_id),
asText(editRow.name), asText(editRow.address), asText(editRow.contact_name),
asText(editRow.contact_email))
    }
    else if (editEntity === "user") {
        if (editMode === "create")
            auth.admin_create_user(asText(editRow.name),
asText(editRow.email), asText(editRow.password),
asText(editRow.role).toUpperCase(), Number(editRow.school_id || 1),
asText(editRow.status).toUpperCase())
        else
            auth.admin_update_user(Number(editRow.id), asText(editRow.name),
asText(editRow.email), asText(editRow.role).toUpperCase(),
Number(editRow.school_id || 1), asText(editRow.status).toUpperCase())
    }
    else if (editEntity === "course") {
        if (editMode === "create")
            auth.create_course(asText(editRow.name),
asText(editRow.description), Number(editRow.teacher_id || userId),
Number(editRow.school_id || 1))
    }
}

```

```

else

    auth.admin_update_course(Number(editRow.course_id),
asText(editRow.name), asText(editRow.description), Number(editRow.teacher_id
|| userId), Number(editRow.school_id || 1))

{
    else if (editEntity === "assignment") {
        if (editMode === "create")

            auth.create_assignment(Number(editRow.course_id || 1),
asText(editRow.title), asText(editRow.description), asText(editRow.due_date),
Boolean(editRow.ai_enabled), asText(editRow.attachment_path))

        else

            auth.admin_update_assignment(Number(editRow.assignment_id),
asText(editRow.title), asText(editRow.description), asText(editRow.due_date),
Boolean(editRow.ai_enabled), Boolean(editRow.is_closed))

    }

    editorVisible = false

    loading = true

    setStatus("Saving changes...", false)

{

function setEditValue(key, value){

    if (!editRow)

        editRow{} =

        editRow[key] = value

    {

function performDelete} ()

    if (!pendingDeleteRow || typeof auth === "undefined")

        return

    if (pendingDeleteEntity === "school")

```



```

        auth.admin_delete_school(Number(pendingDeleteRow.school_id))
    else if (pendingDeleteEntity === "user")
        auth.admin_delete_user(Number(pendingDeleteRow.id))
    else if (pendingDeleteEntity === "course")
        auth.delete_course(Number(pendingDeleteRow.course_id))
    else if (pendingDeleteEntity === "assignment")
        auth.delete_assignment(Number(pendingDeleteRow.assignment_id))
    else if (pendingDeleteEntity === "material")
        auth.delete_material(Number(pendingDeleteRow.material_id))
    else if (pendingDeleteEntity === "message")
        auth.delete_message(Number(pendingDeleteRow.message_id))

    confirmVisible = false
    loading = true
    setStatus("Deleting record...", false)
{

function updateUserStatus(row, statusValue)
    if (!row || typeof auth === "undefined")
        return

    auth.admin_update_user(Number(row.id), asText(row.name),
asText(row.email), asText(row.role).toUpperCase(), Number(row.school_id || 1),
statusValue)

    loading = true
    setStatus("Updating user status...", false)
{

function fileRefOf(row)
    if (!row)

```

```

        return""

        if (asText(row.file_path).trim().length > 0)
            return asText(row.file_path)

        if (asText(row.attachment_path).trim().length > 0)
            return asText(row.attachment_path)

        return""
    }

    function openFilePicker(target){
        fileDialogTarget = target
        fileDialog.open()
    }

    function detailRows(row){
        var rows[] =
        if (!row)
            return rows

        var skip = { sourceIndex: true, rowIdText: true, primaryText: true,
secondaryText: true, statusTextValue: true, accentValue: true }

        for (var key in row){
            if (skip[key] === true)
                continue

            rows.push({ label: humanize(key), value: valueForDetail(row[key], key) })
        }

        return rows
    }

    function humanize(key){

```

```

var parts = asText(key).split("_")
var out[] =
for (var i = 0; i < parts.length; i++){
    var p = parts[i]
    out.push(p.length > 0 ? p.charAt(0).toUpperCase() + p.slice(1) : p)
}
return out.join(" ")
}

function valueForDetail(value, key){
    if (key.indexOf("bytes") >= 0)
        return formatBytes(value)
    if (key.indexOf("_at") >= 0 || key === "created_at" || key === "submitted_at" ||
key === "due_date")
        return shortDate(value)
    if (typeof value === "boolean")
        return value ? "Yes" : "No"
    if (key === "ai_enabled" || key === "is_closed")
        return Number(value || 0) ? "Yes" : "No"
    return safe(value)
}

function statCards() {
    var submissions = countFor("submissions")
    var graded = countFor("grades")
    var withAi = 0
    var submissionRows = rowsForKey("submissions")
    for (var i = 0; i < submissionRows.length; i++){

```

```

        if (Number(submissionRows[i].ai_results_count || 0) > 0)
            withAi++
    {
        var aiCoverage = submissions > 0 ? Math.round((withAi / submissions) *
100) + "%" : "0%"

        var storageBytes = stats && stats.system ? stats.system.storage_size_bytes
: 0

        return]

    }      label: "Schools", value: countFor("schools"), note: countFor("courses")
+ " courses", color: primary,{

    }      label: "Users", value: countFor("users"), note:
countByValue(rowsForKey("users"), "role", "TEACHER") + " teachers / " +
countByValue(rowsForKey("users"), "role", "STUDENT") + " students", color: teal
,{

    }      label: "Assignments", value: countFor("assignments"), note:
countByValue(rowsForKey("assignments"), "is_closed", "1") + " closed", color:
amber,{

    }      label: "Submissions", value: submissions, note: graded + " graded",
color: violet,{

    }      label: "AI Coverage", value: aiCoverage, note: countFor("ai_results") + "
AI results", color: primary,{

    }      label: "Storage", value: formatBytes(storageBytes), note:
countFor("materials") + " materials", color: teal{

[

{

```

```

Component.onCompleted: refreshAll()

```

```

Connections}

```

```

target: typeof auth !== "undefined" ? auth : null

```

```

function onAdminOverviewResult(success, message, payload){

```

```

        loading = false
        if (success){
            copyPayload(payload)
            setStatus("Dashboard updated", false)
        }
        else{
            setStatus(message, true)
        }
    }
}

```

```

function onAdminActionResult(success, message, actionName){
    setStatus(message, !success)
    if (success)
        refreshAll()
    else
        loading = false
}

```

```

function onCreateCourseResult(success, message, courseId){
    setStatus(success ? "Course created successfully." : message, !success)
    if (success)
        refreshAll()
    else
        loading = false
}

```

```

function onCreateAssignmentResult(success, message, courseId,
assignmentId){
    setStatus(success ? "Assignment created successfully." : message,
!success)
}

```

```
    if (success)
        refreshAll()
    else
        loading = false
}
```

```
function onDeleteCourseResult(success, message, courseId){
    setStatus(message, !success)
    if (success)
        refreshAll()
    else
        loading = false
}
```

```
function onDeleteAssignmentResult(success, message, assignmentId,
courseId){
    setStatus(message, !success)
    if (success)
        refreshAll()
    else
        loading = false
}
```

```
function onDeleteMaterialResult(success, message, courseId, materialId){
    setStatus(message, !success)
    if (success)
        refreshAll()
    else
```

```

        loading = false
    {

        function onDeleteMessageResult(success, message, courseId,
        messageId})
            setStatus(message, !success)
            if (success)
                refreshAll()
            else
                loading = false
        {

            function onDownloadFileResult(success, message, filePath, savedPath){
                setStatus(success ? "Downloaded to " + savedPath : message, !success)
            {
            {

FileDialog}

        id: fileDialog

        title: "Select file"

        fileMode: FileDialog.OpenFile

        nameFilters: ["All files (*)", "Archives (*.zip *.tar *.tgz *.rar *.7z)", "Code files
        (*.py *.cs *.java *.js *.ts *.cpp *.c *.h)"]

        onAccepted} :

            var path = selectedFile.toString()

            if (root.fileDialogTarget === "assignment_rubric"){

                root.setEditValue("attachment_path", path)

                root.setEditValue("attachment_name", root.fileNameFromPath(path))

```

```

        root.setStatus("Rubric selected: " + root.fileNameFromPath(path),
false)
{
{
{

Rectangle}
    anchors.fill: parent
    color: bg
{

RowLayout}
    anchors.fill: parent
    spacing: 0

Rectangle}
    Layout.preferredWidth: sidebarWidth
    Layout.fillHeight: true
    visible: !narrow
    color: side

ColumnLayout}
    anchors.fill: parent
    anchors.margins: 18
    spacing: 16

RowLayout}
    Layout.fillWidth: true
    spacing: 10

```



```

        Rectangle}

        Layout.preferredWidth: 42
        Layout.preferredHeight: 42
        radius: 10
        color: primary
        Text}
            anchors.centerIn: parent
            text: "C"
            color: "white"
            font.pixelSize: 20
            font.bold: true
    {
    {

        ColumnLayout}

        Layout.fillWidth: true
        spacing: 1
        Text { text: "Classify"; color: "white"; font.pixelSize: 21; font.bold:
true; Layout.fillWidth: true; elide: Text.ElideRight }

        Text { text: "System control"; color: "#CBD5E1"; font.pixelSize: 12;
Layout.fillWidth: true; elide: Text.ElideRight }
    {
    {

        Rectangle { Layout.fillWidth: true; Layout.preferredHeight: 1; color:
"#263244" }

        Flickable}

        Layout.fillWidth: true
        Layout.fillHeight: true

```

clip: true
contentWidth: width
contentHeight: navColumn.height
boundsBehavior: Flickable.StopAtBounds

Column}

id: navColumn
width: parent.width
spacing: 6

Repeater}

model: sections
delegate: Rectangle}
width: navColumn.width
height: 42
radius: 10
color: activeSectionIndex === index ? "#243145" :
border.width: activeSectionIndex === index ? 1 : 0
border.color: "#334155"

"transparent"

RowLayout}

anchors.fill: parent
anchors.leftMargin: 10
anchors.rightMargin: 10
spacing: 9

Rectangle}

```

        Layout.preferredWidth: 30
        Layout.preferredHeight: 26
        radius: 8
        color: activeSectionIndex === index ? primary :
"#1F2937"

        Text { anchors.centerIn: parent; text: modelData.short;
color: "white"; font.pixelSize: 10; font.bold: true }
{

        Text}

        Layout.fillWidth: true
        text: modelData.label
        color: activeSectionIndex === index ? "white" :
"#CBD5E1"

        font.pixelSize: 13
        font.bold: activeSectionIndex === index
        elide: Text.ElideRight

{

        Text}

        text: countFor(modelData.key)
        color: "#93C5FD"
        font.pixelSize: 11
        font.bold: true

{
{

        MouseArea}

        anchors.fill: parent

```

```

        cursorShape: Qt.PointingHandCursor
        onClicked: switchSection(index)

{
{
{
{
{

Rectangle}

    Layout.fillWidth: true
    Layout.preferredHeight: 158
    radius: 14
    color: "#172033"
    border.color: "#28364C"

ColumnLayout}

    anchors.fill: parent
    anchors.margins: 12
    spacing: 4
    Text { text: "Signed in"; color: "#94A3B8"; font.pixelSize: 11 }
    Text { text: userName || "Admin"; color: "white"; font.pixelSize: 15;
font.bold: true; Layout.fillWidth: true; elide: Text.ElideRight }
    Text { text: "User ID: " + userId; color: "#CBD5E1"; font.pixelSize:
12; Layout.fillWidth: true; elide: Text.ElideRight }

Rectangle}

    Layout.fillWidth: true
    Layout.preferredHeight: 32
    radius: 9
    color: statusIsError ? "#3B1D24" : "#1F2937"

```

```

border.color: statusIsError ? "#7F1D1D" : "#334155"
Text}

    anchors.centerIn: parent
    width: parent.width - 16
    text: statusText
    color: statusIsError ? "#FCA5A5" : "#E2E8F0"
    font.pixelSize: 10
    horizontalAlignment: Text.AlignHCenter
    elide: Text.ElideRight

{
{

Rectangle}

    Layout.fillWidth: true
    Layout.preferredHeight: 34
    radius: 9
    color: "#F8FAFC"
    border.color: "#E2E8F0"
    Text}

    anchors.centerIn: parent
    text: "Logout"
    color: "#0F172A"
    font.pixelSize: 12
    font.bold: true

{

MouseArea}

    anchors.fill: parent
    cursorShape: Qt.PointingHandCursor
    onClicked: root.logout()

```

```
{  
{  
{  
{  
{  
{
```

```
Flickable}
```

```
    id: mainFlick
```

```
    Layout.fillWidth: true
```

```
    Layout.fillHeight: true
```

```
    clip: true
```

```
    contentWidth: width
```

```
    contentHeight: contentColumn.height + 34
```

```
    boundsBehavior: Flickable.StopAtBounds
```

```
ColumnLayout}
```

```
    id: contentColumn
```

```
    width: mainFlick.width
```

```
    spacing: 16
```

```
Item { Layout.fillWidth: true; Layout.preferredHeight: 22 }
```

```
RowLayout}
```

```
    Layout.fillWidth: true
```

```
    Layout.leftMargin: narrow ? 14 : 26
```

```
    Layout.rightMargin: narrow ? 14 : 26
```

```
    spacing: 14
```

```

ColumnLayout}

    Layout.fillWidth: true

    spacing: 4

    Text { text: "Admin Control Center"; color: ink; font.pixelSize:
narrow ? 25 : 32; font.bold: true; Layout.fillWidth: true; elide: Text.ElideRight }

    Text { text: "A full operational view across schools, users, classes,
assignments, submissions, AI, files and messages."; color: muted; font.pixelSize:
13; Layout.fillWidth: true; elide: Text.ElideRight }

{

```

```

Rectangle}

    Layout.preferredWidth: loading ? 118 : 108

    Layout.preferredHeight: 40

    radius: 10

    color: loading ? "#CBD5E1" : primary

    Text { anchors.centerIn: parent; text: loading ? "Loading" :
"Refresh"; color: "white"; font.pixelSize: 13; font.bold: true }

    MouseArea { anchors.fill: parent; enabled: !loading; cursorShape:
Qt.PointingHandCursor; onClicked: refreshAll ()}

{

```

```

Rectangle}

    Layout.preferredWidth: 94

    Layout.preferredHeight: 40

    radius: 10

    color: panel

    border.color: line

    Text { anchors.centerIn: parent; text: "Logout"; color: ink;
font.pixelSize: 13; font.bold: true }

```

```

        MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked: root.logout ()}
    {
    {

        Flow}

        Layout.fillWidth: true

        Layout.leftMargin: narrow ? 14 : 26

        Layout.rightMargin: narrow ? 14 : 26

        spacing: 10

        Repeater}

        model: statCards()

        delegate: Rectangle}

            width: Math.max(178, Math.min(244, (mainFlick.width -
(narrow ? 46 : 84)) / (narrow ? 2 : 6)))

            height: 96

            radius: 12

            color: panel

            border.color: line

        Rectangle { anchors.left: parent.left; anchors.top: parent.top;
anchors.bottom: parent.bottom; width: 4; radius: 2; color: modelData.color }

        Column}

            anchors.left: parent.left

            anchors.leftMargin: 18

            anchors.right: parent.right

            anchors.rightMargin: 12

            anchors.verticalCenter: parent.verticalCenter

```



```

        spacing: 4

        Text { text: modelData.label; color: muted; font.pixelSize:
11; font.bold: true; width: parent.width; elide: Text.ElideRight }

        Text { text: modelData.value; color: ink; font.pixelSize: 25;
font.bold: true; width: parent.width; elide: Text.ElideRight }

        Text { text: modelData.note; color: faint; font.pixelSize: 10;
width: parent.width; elide: Text.ElideRight }
    {
    {
    {
    {

    Rectangle}

        Layout.fillWidth: true

        Layout.leftMargin: narrow ? 14 : 26

        Layout.rightMargin: narrow ? 14 : 26

        Layout.preferredHeight: narrow ? 760 : Math.max(620, root.height -
232)

        radius: 16

        color: panel

        border.color: line

        clip: true

        ColumnLayout}

            anchors.fill: parent

            anchors.margins: 16

            spacing: 12

            RowLayout}

```

Layout.fillWidth: true

spacing: 12

ColumnLayout}

Layout.fillWidth: true

spacing: 2

Text { text: currentSection().label; color: ink; font.pixelSize: 22; font.bold: true; Layout.fillWidth: true; elide: Text.ElideRight }

Text { text: filteredModel.count + " visible rows out of " + currentRows().length; color: muted; font.pixelSize: 12; Layout.fillWidth: true; elide: Text.ElideRight }

{

Rectangle}

visible: currentSection().create

Layout.preferredWidth: 126

Layout.preferredHeight: 38

radius: 10

color: primarySoft

border.color: "#BFDBFE"

Text { anchors.centerIn: parent; text: "New " + entityForSectionKey(currentSection().key); color: primary; font.pixelSize: 12; font.bold: true; width: parent.width - 14; horizontalAlignment: Text.AlignHCenter; elide: Text.ElideRight }

MouseArea { anchors.fill: parent; cursorShape: Qt.PointingHandCursor; onClicked: openCreate(entityForSectionKey(currentSection().key)) }

{

{

RowLayout}

Layout.fillWidth: true

spacing: 10

Rectangle}

Layout.fillWidth: true

Layout.preferredHeight: 40

radius: 10

color: panel2

border.color: line

Text}

text: "Search this data set"

color: faint

font.pixelSize: 13

anchors.left: parent.left

anchors.leftMargin: 14

anchors.verticalCenter: parent.verticalCenter

visible: searchInput.text.length === 0

{

TextInput}

id: searchInput

anchors.fill: parent

anchors.leftMargin: 14

anchors.rightMargin: 14

verticalAlignment: TextInput.AlignVCenter

color: ink

selectionColor: "#BFDDBFE"

```

        selectedTextColor: ink
        font.pixelSize: 14
        clip: true
        text: searchText
        onTextChanged} :
            if (searchText !== text){
                searchText = text
                rebuildFilter()
    {
    {
    {
    {

```

```

    Rectangle}

    Layout.preferredWidth: 110
    Layout.preferredHeight: 40
    radius: 10
    color: statusIsError ? redSoft : panel2
    border.color: statusIsError ? "#FECACA" : line

    Text { anchors.centerIn: parent; text: statusText; color:
statusIsError ? red : muted; font.pixelSize: 11; font.bold: true; width:
parent.width - 14; horizontalAlignment: Text.AlignHCenter; elide: Text.ElideRight }
    {
    {

```

```

    RowLayout}

    Layout.fillWidth: true
    Layout.fillHeight: true
    spacing: 12

```

Rectangle}

Layout.fillWidth: true

Layout.fillHeight: true

Layout.minimumWidth: narrow ? 0 : 560

radius: 12

color: "#FCFDFF"

border.color: lineSoft

clip: true

ColumnLayout}

anchors.fill: parent

spacing: 0

Rectangle}

Layout.fillWidth: true

Layout.preferredHeight: 42

color: panel2

border.color: lineSoft

RowLayout}

anchors.fill: parent

anchors.leftMargin: 14

anchors.rightMargin: 12

spacing: 10

Text { text: "ID"; color: muted; font.pixelSize: 11;
font.bold: true; Layout.preferredWidth: 68; elide: Text.ElideRight }

Text { text: "Record"; color: muted; font.pixelSize:
11; font.bold: true; Layout.fillWidth: true; elide: Text.ElideRight }

```
Text { text: "State"; color: muted; font.pixelSize: 11;
font.bold: true; Layout.preferredWidth: 112; horizontalAlignment:
Text.AlignHCenter; elide: Text.ElideRight }
```

```
Text { text: "Actions"; color: muted; font.pixelSize:
11; font.bold: true; Layout.preferredWidth: 164; horizontalAlignment:
Text.AlignHCenter; elide: Text.ElideRight }
```

```
{
{
```

```
ListView}
```

```
id: listView
```

```
Layout.fillWidth: true
```

```
Layout.fillHeight: true
```

```
clip: true
```

```
model: filteredModel
```

```
boundsBehavior: Flickable.StopAtBounds
```

```
delegate: Rectangle}
```

```
width: listView.width
```

```
height: 72
```

```
color: selectedIndex === sourceIndex ?
```

```
primarySoft : (index % 2 === 0 ? "#FFFFFF" : "#FCFDFD")
```

```
border.width: selectedIndex === sourceIndex
```

```
? 1 : 0
```

```
border.color: "#BFDDBFE"
```

```
MouseArea}
```

```
anchors.fill: parent
```

```
cursorShape: Qt.PointingHandCursor
```

```
onClicked: selectBySource(sourceIndex)
```

{

RowLayout}

anchors.fill: parent

anchors.leftMargin: 14

anchors.rightMargin: 12

spacing: 10

z: 1

Text}

text: shortText(rowIdText, 10)

color: muted

font.pixelSize: 12

font.bold: true

Layout.preferredWidth: 68

elide: Text.ElideRight

verticalAlignment: Text.AlignVCenter

{

ColumnLayout}

Layout.fillWidth: true

spacing: 3

Text { text: primaryText; color: ink;
font.pixelSize: 14; font.bold: true; Layout.fillWidth: true; maximumLineCount: 1;
elide: Text.ElideRight }

Text { text: secondaryText; color: muted;
font.pixelSize: 12; Layout.fillWidth: true; maximumLineCount: 1; elide:
Text.ElideRight }

{

```

        Rectangle}

        Layout.preferredWidth: 112

        Layout.preferredHeight: 28

        radius: 8

        color: accentValue === red ? redSoft :
(accentValue === amber ? amberSoft : (accentValue === teal ? tealSoft :
primarySoft))

        border.color: line

        Text { anchors.centerIn: parent; text:
shortText(statusTextValue, 14); color: accentValue; font.pixelSize: 11; font.bold:
true; width: parent.width - 12; horizontalAlignment: Text.AlignHCenter; elide:
Text.ElideRight }
    {

        RowLayout}

        Layout.preferredWidth: 164

        spacing: 6

        Rectangle}

        Layout.preferredWidth: 72

        Layout.preferredHeight: 32

        radius: 8

        color: panel2

        border.color: line

        Text { anchors.centerIn: parent; text:
"Details"; color: ink; font.pixelSize: 12; font.bold: true }

        MouseArea { anchors.fill: parent;
cursorShape: Qt.PointingHandCursor; onClicked: selectBySource(sourceIndex) }
    {

        Rectangle}

```



```

        visible: currentSection().edit

        Layout.preferredWidth: 72
        Layout.preferredHeight: 32
        radius: 8
        color: primary

        Text { anchors.centerIn: parent; text: "Edit";
color: "white"; font.pixelSize: 12; font.bold: true }

        MouseArea{

            anchors.fill: parent

            cursorShape: Qt.PointingHandCursor

            onClicked} :

                selectBySource(sourceIndex)

```

```

openEdit(entityForSectionKey(currentSection().key), selectedRow)

{
{
{
{
{
{

```

```

Rectangle}

    visible: filteredModel.count === 0

    anchors.centerIn: parent

    width: Math.min(parent.width - 40, 360)

    height: 118

    radius: 12

    color: "#FFFFFF"

    border.color: line

```

Column}

anchors.centerIn: parent

spacing: 7

Text { text: "No rows found"; color: ink;
font.pixelSize: 18; font.bold: true; anchors.horizontalCenter:
parent.horizontalCenter }

Text { text: "Try another search or refresh the
dashboard."; color: muted; font.pixelSize: 12; anchors.horizontalCenter:
parent.horizontalCenter }

{

{

{

{

{

Rectangle}

Layout.preferredWidth: compact ? 330 : 410

Layout.fillHeight: true

visible: !narrow

radius: 12

color: panel

border.color: line

clip: true

ColumnLayout}

anchors.fill: parent

anchors.margins: 16

spacing: 12

RowLayout}

Layout.fillWidth: true

spacing: 10

ColumnLayout}

Layout.fillWidth: true

spacing: 2

Text { text: selectedRow ? primaryOf(selectedRow) :
"Select a record"; color: ink; font.pixelSize: 19; font.bold: true; Layout.fillWidth:
true; elide: Text.ElideRight }

Text { text: selectedRow ?
currentSection().label.toUpperCase() : "Details and actions"; color: faint;
font.pixelSize: 11; font.bold: true; Layout.fillWidth: true; elide: Text.ElideRight }
{
{

Flow}

Layout.fillWidth: true

spacing: 8

visible: selectedRow !== null

Rectangle}

visible: currentSection().edit

width: 76

height: 34

radius: 9

color: primary

Text { anchors.centerIn: parent; text: "Edit"; color:
"white"; font.pixelSize: 12; font.bold: true }

```

        MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked:
openEdit(entityForSectionKey(currentSection().key), selectedRow) }
{

        Rectangle}

        visible: currentSection().remove

        width: 82

        height: 34

        radius: 9

        color: redSoft

        border.color: "#FECACA"

        Text { anchors.centerIn: parent; text: "Delete"; color:
red; font.pixelSize: 12; font.bold: true }

        MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked:
requestDelete(entityForSectionKey(currentSection().key), selectedRow) }
{

        Rectangle}

        visible: fileRefOf(selectedRow).length > 0

        width: 98

        height: 34

        radius: 9

        color: tealSoft

        border.color: "#A7F3D0"

        Text { anchors.centerIn: parent; text: "Download";
color: teal; font.pixelSize: 12; font.bold: true }

        MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked: auth.download_file(fileRefOf(selectedRow))
}

```

```

{
{

Flow}

Layout.fillWidth: true

spacing: 8

visible: selectedRow !== null && currentSection().key
=== "users"

Rectangle}

width: 82

height: 32

radius: 9

color: tealSoft

border.color: "#A7F3D0"

Text { anchors.centerIn: parent; text: "Activate";
color: teal; font.pixelSize: 11; font.bold: true }

MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked: updateUserStatus(selectedRow, "ACTIVE") }
{

Rectangle}

width: 82

height: 32

radius: 9

color: amberSoft

border.color: "#FDE68A"

Text { anchors.centerIn: parent; text: "Pending";
color: amber; font.pixelSize: 11; font.bold: true }

```

```

        MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked: updateUserStatus(selectedRow,
"PENDING") }

{
    Rectangle{
        width: 82
        height: 32
        radius: 9
        color: redSoft
        border.color: "#FECACA"

        Text { anchors.centerIn: parent; text: "Block"; color:
red; font.pixelSize: 11; font.bold: true }

        MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked: updateUserStatus(selectedRow,
"BLOCKED") }

        {
        {

            Rectangle { Layout.fillWidth: true;
Layout.preferredHeight: 1; color: lineSoft }

            Flickable{
                Layout.fillWidth: true
                Layout.fillHeight: true
                clip: true
                contentWidth: width
                contentHeight: detailColumn.height
                boundsBehavior: Flickable.StopAtBounds

                ColumnLayout}

```

```

        id: detailColumn
        width: parent.width
        spacing: 9

    Repeater{
        model: detailRows(selectedRow)
        delegate: Rectangle{
            Layout.fillWidth: true
            radius: 10
            color: panel2
            border.color: lineSoft
            Layout.preferredHeight: Math.max(54,
detailValue.implicitHeight + 30)

            ColumnLayout{
                anchors.fill: parent
                anchors.margins: 10
                spacing: 3

                Text { text: modelData.label; color: faint;
font.pixelSize: 10; font.bold: true; Layout.fillWidth: true; elide: Text.ElideRight }

                Text { id: detailValue; text: modelData.value;
color: ink; font.pixelSize: 12; wrapMode: Text.WrapAnywhere; Layout.fillWidth:
true; maximumLineCount: 5; elide: Text.ElideRight }

            }
        }
    }
}

```

```

{
{
{
{

        Item { Layout.fillWidth: true; Layout.preferredHeight: 18 }

{
{
{

```

Rectangle}

```

    visible: editorVisible
    anchors.fill: parent
    color: "#99111827"
    z: 50

```

MouseArea { anchors.fill: parent }

Rectangle}

```

    width: Math.min(root.width - 36, 700)
    height: Math.min(root.height - 48, 720)
    anchors.centerIn: parent
    radius: 16
    color: panel
    border.color: line
    clip: true

```

ColumnLayout}


```

anchors.fill: parent
anchors.margins: 20
spacing: 14

RowLayout{
    Layout.fillWidth: true
    spacing: 10
    ColumnLayout{
        Layout.fillWidth: true
        spacing: 2
        Text { text: (editMode == "create" ? "Create " : "Edit ") +
editEntity; color: ink; font.pixelSize: 23; font.bold: true; Layout.fillWidth: true;
elide: Text.ElideRight }

        Text { text: "Changes here affect production data. Review the
fields before saving."; color: muted; font.pixelSize: 12; Layout.fillWidth: true;
elide: Text.ElideRight }
    }

    Rectangle{
        Layout.preferredWidth: 36
        Layout.preferredHeight: 36
        radius: 9
        color: panel2
        border.color: line
        Text { anchors.centerIn: parent; text: "X"; color: muted;
font.pixelSize: 15; font.bold: true }
        MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked: editorVisible = false }
    }
}

```

Flickable}

Layout.fillWidth: true

Layout.fillHeight: true

clip: true

contentWidth: width

contentHeight: editContent.implicitHeight

boundsBehavior: Flickable.StopAtBounds

ColumnLayout}

id: editContent

width: parent.width

spacing: 10

Loader { Layout.fillWidth: true; sourceComponent: editEntity ===
"school" ? schoolEditor : null }

Loader { Layout.fillWidth: true; sourceComponent: editEntity ===
"user" ? userEditor : null }

Loader { Layout.fillWidth: true; sourceComponent: editEntity ===
"course" ? courseEditor : null }

Loader { Layout.fillWidth: true; sourceComponent: editEntity ===
"assignment" ? assignmentEditor : null }

{

{

RowLayout}

Layout.fillWidth: true

spacing: 10

Rectangle}

Layout.fillWidth: true

```

        Layout.preferredHeight: 44

        radius: 10

        color: panel2

        border.color: line

        Text { anchors.centerIn: parent; text: "Cancel"; color: muted;
font.pixelSize: 14; font.bold: true }

        MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked: editorVisible = false }
    {
        Rectangle}

        Layout.fillWidth: true

        Layout.preferredHeight: 44

        radius: 10

        color: primary

        Text { anchors.centerIn: parent; text: "Save changes"; color:
"white"; font.pixelSize: 14; font.bold: true }

        MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked: saveEditor ()}
    {
    {
    {
    {
    {

Component}

    id: fieldBox

    Rectangle}

        id: fieldRoot

        property string label"" :

```

```
property string value"" :
property string keyName"" :
property bool multiline: false
```

```
Layout.fillWidth: true
height: multiline ? 122 : 66
radius: 10
color: panel2
border.color: line
```

```
ColumnLayout}
```

```
anchors.fill: parent
anchors.margins: 10
spacing: 4
```

```
Text { text: fieldRoot.label; color: faint; font.pixelSize: 10; font.bold:
true; Layout.fillWidth: true; elide: Text.ElideRight }
```

```
Loader}
```

```
Layout.fillWidth: true
Layout.fillHeight: true
```

```
sourceComponent: fieldRoot.multiline ? textAreaField :
textInputField
```

```
{
{
```

```
Component}
```

```
id: textInputField
```

```
TextField}
```

```
text: fieldRoot.value
```

```

        color: ink
        selectByMouse: true
        font.pixelSize: 14
        background: Rectangle { color: "transparent" }
        onTextEdited: setEditValue(fieldRoot.keyName, text)
    {
    {

```

Component}

id: textAreaField

TextArea}

text: fieldRoot.value

color: ink

selectByMouse: true

wrapMode: TextArea.Wrap

font.pixelSize: 14

background: Rectangle { color: "transparent" }

onTextChanged: setEditValue(fieldRoot.keyName, text)

```

{
{
{
{

```

Component}

id: choiceBox

Rectangle}

id: choiceRoot

property string label"" :

```

property string keyName"" :
property var options[] :
property var value"" :

Layout.fillWidth: true
height: 68
radius: 10
color: panel2
border.color: line

function findIndex(v){
    for (var i = 0; i < options.length; i++){
        if (asText(options[i].value) === asText(v))
            return i
    }

    return options.length > 0 ? 0 : -1
}

onOptionsChanged: combo.currentIndex = findIndex(value)
onValueChanged: combo.currentIndex = findIndex(value)
Component.onCompleted: combo.currentIndex = findIndex(value)

ColumnLayout{
    anchors.fill: parent
    anchors.margins: 10
    spacing: 4

    Text { text: choiceRoot.label; color: faint; font.pixelSize: 10; font.bold:
true; Layout.fillWidth: true; elide: Text.ElideRight }

```

```

        ComboBox}

        id: combo

        Layout.fillWidth: true

        model: choiceRoot.options

        textRole: "label"

        font.pixelSize: 14

        onActivated} :

            if (index >= 0 && index < choiceRoot.options.length)

                setEditValue(choiceRoot.keyName,
choiceRoot.options[index].value)
{
{
{
{
{

```

```

Component}

    id: boolBox

    Rectangle}

        id: boolRoot

        property string label"" :

        property string keyName"" :

        property bool checkedValue: false


        Layout.fillWidth: true

        height: 58

        radius: 10

        color: panel2

```

border.color: line

RowLayout}

anchors.fill: parent

anchors.margins: 12

spacing: 10

Text { text: boolRoot.label; color: ink; font.pixelSize: 14; font.bold: true;
Layout.fillWidth: true; elide: Text.ElideRight }

Switch}

checked: boolRoot.checkedValue

onToggled: setEditValue(boolRoot.keyName, checked)

{

{

{

{

Component}

id: schoolEditor

ColumnLayout}

spacing: 10

Loader { Layout.fillWidth: true; sourceComponent: fieldBox; onLoaded: {
item.label = "School name"; item.keyName = "name"; item.value =
asText(editRow.name) } }

Loader { Layout.fillWidth: true; sourceComponent: fieldBox; onLoaded: {
item.label = "Address"; item.keyName = "address"; item.value =
asText(editRow.address) } }

Loader { Layout.fillWidth: true; sourceComponent: fieldBox; onLoaded: {
item.label = "Contact name"; item.keyName = "contact_name"; item.value =
asText(editRow.contact_name) } }


```

        Loader { Layout.fillWidth: true; sourceComponent: fieldBox; onLoaded: {
item.label = "Contact email"; item.keyName = "contact_email"; item.value =
asText(editRow.contact_email) } }

```

```

{

```

```

{

```

```

Component}

```

```

    id: userEditor

```

```

    ColumnLayout{

```

```

        spacing: 10

```

```

        Loader { Layout.fillWidth: true; sourceComponent: fieldBox; onLoaded: {
item.label = "Full name"; item.keyName = "name"; item.value =
asText(editRow.name) } }

```

```

        Loader { Layout.fillWidth: true; sourceComponent: fieldBox; onLoaded: {
item.label = "Email"; item.keyName = "email"; item.value =
asText(editRow.email) } }

```

```

        Loader { Layout.fillWidth: true; visible: editMode === "create";
sourceComponent: fieldBox; onLoaded: { item.label = "Initial password";
item.keyName = "password"; item.value = asText(editRow.password) } }

```

```

        Loader { Layout.fillWidth: true; sourceComponent: choiceBox;
onLoaded: { item.label = "Role"; item.keyName = "role"; item.options =
roleOptions(); item.value = asText(editRow.role) } }

```

```

        Loader { Layout.fillWidth: true; sourceComponent: choiceBox;
onLoaded: { item.label = "Status"; item.keyName = "status"; item.options =
statusOptions(); item.value = asText(editRow.status) } }

```

```

        Loader { Layout.fillWidth: true; sourceComponent: choiceBox;
onLoaded: { item.label = "School"; item.keyName = "school_id"; item.options =
schoolOptions(); item.value = Number(editRow.school_id || 1) } }

```

```

{

```

```

{

```

```

Component}

```

```

    id: courseEditor

```

```

    ColumnLayout{

        spacing: 10

        Loader { Layout.fillWidth: true; sourceComponent: fieldBox; onLoad: {
item.label = "Course name"; item.keyName = "name"; item.value =
asText(editRow.name) }}

        Loader { Layout.fillWidth: true; sourceComponent: fieldBox; onLoad: {
item.label = "Description"; item.keyName = "description"; item.value =
asText(editRow.description); item.multiline = true }}

        Loader { Layout.fillWidth: true; sourceComponent: choiceBox;
onLoad: { item.label = "Teacher"; item.keyName = "teacher_id"; item.options =
teacherOptions(); item.value = Number(editRow.teacher_id || userId) }}

        Loader { Layout.fillWidth: true; sourceComponent: choiceBox;
onLoad: { item.label = "School"; item.keyName = "school_id"; item.options =
schoolOptions(); item.value = Number(editRow.school_id || 1) }}

    {
    {

```

Component}

id: assignmentEditor

ColumnLayout{

spacing: 10

```

    Loader { Layout.fillWidth: true; visible: editMode === "create";
sourceComponent: choiceBox; onLoad: { item.label = "Course";
item.keyName = "course_id"; item.options = courseOptions(); item.value =
Number(editRow.course_id || 1) }}

```

```

    Loader { Layout.fillWidth: true; sourceComponent: fieldBox; onLoad: {
item.label = "Title"; item.keyName = "title"; item.value = asText(editRow.title) }}

```

```

    Loader { Layout.fillWidth: true; sourceComponent: fieldBox; onLoad: {
item.label = "Description"; item.keyName = "description"; item.value =
asText(editRow.description); item.multiline = true }}

```

```

    Loader { Layout.fillWidth: true; sourceComponent: fieldBox; onLoad: {
item.label = "Due date"; item.keyName = "due_date"; item.value =
asText(editRow.due_date) }}

```

```

Rectangle}

    Layout.fillWidth: true

    Layout.preferredHeight: 68

    visible: editMode === "create"

    radius: 10

    color: panel2

    border.color: line

    RowLayout}

        anchors.fill: parent

        anchors.margins: 10

        spacing: 10

        ColumnLayout}

            Layout.fillWidth: true

            spacing: 3

            Text { text: "Rubric attachment"; color: faint; font.pixelSize: 10;
font.bold: true; Layout.fillWidth: true; elide: Text.ElideRight }

            Text { text: "Up to 25MB - for multiple files, upload one ZIP"; color:
faint; font.pixelSize: 10; Layout.fillWidth: true; elide: Text.ElideRight }

            Text { text: safe(editRow.attachment_name ||
fileNameFromPath(editRow.attachment_path)); color: ink; font.pixelSize: 13;
Layout.fillWidth: true; elide: Text.ElideRight }

{

    Rectangle}

        Layout.preferredWidth: 112

        Layout.preferredHeight: 36

        radius: 9

        color: primarySoft

        border.color: "#BFDDBFE"

        Text { anchors.centerIn: parent; text: "Choose file"; color:
primary; font.pixelSize: 12; font.bold: true }

```

```

        MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked: openFilePicker("assignment_rubric") }
{

```

```

    Rectangle}

```

```

        Layout.preferredWidth: 64

```

```

        Layout.preferredHeight: 36

```

```

        visible: safe(editRow.attachment_path) != ""

```

```

        radius: 9

```

```

        color: "#FFFFFF"

```

```

        border.color: line

```

```

        Text { anchors.centerIn: parent; text: "Clear"; color: muted;
font.pixelSize: 12; font.bold: true }

```

```

    MouseArea}

```

```

        anchors.fill: parent

```

```

        cursorShape: Qt.PointingHandCursor

```

```

        onClicked} :

```

```

            setEditValue("attachment_path", "")

```

```

            setEditValue("attachment_name", "")

```

```

{

```

```

{

```

```

{

```

```

{

```

```

{

```

```

        Loader { Layout.fillWidth: true; sourceComponent: boolBox; onLoad: {
item.label = "AI enabled"; item.keyName = "ai_enabled"; item.checkedValue =
Boolean(editRow.ai_enabled) }}

```

```

        Loader { Layout.fillWidth: true; sourceComponent: boolBox; onLoad: {
item.label = "Closed"; item.keyName = "is_closed"; item.checkedValue =
Boolean(editRow.is_closed) }}

```

```

{

```

```
{
```

```
Rectangle}
```

```
    visible: confirmVisible
```

```
    anchors.fill: parent
```

```
    color: "#99111827"
```

```
    z: 80
```

```
MouseArea { anchors.fill: parent }
```

```
Rectangle}
```

```
    width: Math.min(root.width - 32, 440)
```

```
    height: 250
```

```
    radius: 16
```

```
    color: panel
```

```
    border.color: line
```

```
    anchors.centerIn: parent
```

```
ColumnLayout}
```

```
    anchors.fill: parent
```

```
    anchors.margins: 20
```

```
    spacing: 12
```

```
        Text { text: "Delete record?"; color: ink; font.pixelSize: 23; font.bold: true; Layout.fillWidth: true; elide: Text.ElideRight }
```

```
        Text { text: "This can remove related data and cannot be undone from this screen."; color: muted; font.pixelSize: 13; wrapMode: Text.WordWrap; Layout.fillWidth: true }
```

```

Rectangle}

    Layout.fillWidth: true

    Layout.preferredHeight: 54

    radius: 10

    color: redSoft

    border.color: "#FECACA"

    Text}

        anchors.centerIn: parent

        width: parent.width - 24

        text: pendingDeleteEntity.toUpperCase() + " / " +
(pendingDeleteRow ? primaryOf(pendingDeleteRow) : "")

        color: red

        font.pixelSize: 13

        font.bold: true

        horizontalAlignment: Text.AlignHCenter

        elide: Text.ElideRight

{
{

```

```

Item { Layout.fillHeight: true }

```

```

RowLayout}

    Layout.fillWidth: true

    spacing: 10

    Rectangle}

        Layout.fillWidth: true

        Layout.preferredHeight: 44

        radius: 10

```

```

        color: panel2

        border.color: line

        Text { anchors.centerIn: parent; text: "Cancel"; color: muted;
font.pixelSize: 14; font.bold: true }

        MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked: confirmVisible = false }
    {
        Rectangle}

        Layout.fillWidth: true

        Layout.preferredHeight: 44

        radius: 10

        color: red

        Text { anchors.centerIn: parent; text: "Delete"; color: "white";
font.pixelSize: 14; font.bold: true }

        MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked: performDelete ()}
    {
    {
    {
    {
    {
    {
    {
AdminHome.qml

import QtQuick 2.15

import QtQuick.Controls 2.15

import QtQuick.Layouts 1.15

import QtQuick.Dialogs

Item}

```

id: root

anchors.fill: parent

property string userName: "Admin"

property int userId: -1

property var nav: null

property int activeSectionIndex: 0

property string searchText"" :

property bool loading: false

property string statusText: "Ready"

property bool statusIsError: false

property var selectedRow: null

property int selectedIndex: -1

property bool editorVisible: false

property bool confirmVisible: false

property string editMode: "edit"

property string editEntity"" :

property var editRow: null

property string fileDialogTarget"" :

property string pendingDeleteEntity"" :

property var pendingDeleteRow: null

property var allData}) :

 schools: [], users: [], courses: [], members: [], assignments: [],
submissions,[] :

 ai_results: [], grades: [], feedback: [], materials: [], messages,[] :


```

    message_reads: [], grade_reads: [], activity: [], system[] :
({
  property var stats({}) :

    readonly property bool compact: width < 1120
    readonly property bool narrow: width < 860
    readonly property int sidebarWidth: narrow ? 0 : 252

    readonly property color bg: "#F6F7FB"
    readonly property color panel: "#FFFFFF"
    readonly property color panel2: "#F8FAFC"
    readonly property color ink: "#0F172A"
    readonly property color muted: "#64748B"
    readonly property color faint: "#94A3B8"
    readonly property color line: "#E2E8F0"
    readonly property color lineSoft: "#EEF2F7"
    readonly property color side: "#111827"
    readonly property color primary: "#2563EB"
    readonly property color primarySoft: "#EFF6FF"
    readonly property color teal: "#0F766E"
    readonly property color tealSoft: "#ECFDF5"
    readonly property color amber: "#D97706"
    readonly property color amberSoft: "#FFFBEB"
    readonly property color red: "#DC2626"
    readonly property color redSoft: "#FEF2F2"
    readonly property color violet: "#7C3AED"

  property var sections] :

```

```

    }    key: "schools", label: "Schools", short: "SC", id: "school_id", create: true,
edit: true, remove: true,{

    }    key: "users", label: "Users", short: "US", id: "id", create: true, edit: true,
remove: true,{

    }    key: "courses", label: "Courses", short: "CR", id: "course_id", create: true,
edit: true, remove: true,{

    }    key: "members", label: "Members", short: "MB", id: "member_id", create:
false, edit: false, remove: false,{

    }    key: "assignments", label: "Assignments", short: "AS", id: "assignment_id",
create: true, edit: true, remove: true,{

    }    key: "submissions", label: "Submissions", short: "SB", id: "submission_id",
create: false, edit: false, remove: false,{

    }    key: "grades", label: "Grades", short: "GR", id: "grade_id", create: false, edit:
false, remove: false,{

    }    key: "ai_results", label: "AI Results", short: "AI", id: "ai_result_id", create:
false, edit: false, remove: false,{

    }    key: "feedback", label: "Feedback", short: "FB", id: "feedback_id", create:
false, edit: false, remove: false,{

    }    key: "materials", label: "Materials", short: "MT", id: "material_id", create:
false, edit: false, remove: true,{

    }    key: "messages", label: "Messages", short: "MS", id: "message_id", create:
false, edit: false, remove: true,{

    }    key: "message_reads", label: "Message Reads", short: "MR", id: "read_id",
create: false, edit: false, remove: false,{

    }    key: "grade_reads", label: "Grade Reads", short: "RR", id: "read_id", create:
false, edit: false, remove: false,{

    }    key: "activity", label: "Activity", short: "AC", id: "at", create: false, edit: false,
remove: false,{

    }    key: "system", label: "System", short: "SY", id: "metric", create: false, edit:
false, remove: false{

[

```

```
ListModel}
```

```

        id: filteredModel
        dynamicRoles: true
    {

Timer}

        id: statusTimer
        interval: 3600
        onTriggered} :
            if (!loading)}
                statusText = "Ready"
                statusIsError = false
    {
    {
    {

function asText(v)}
    if (v === undefined || v === null)
        return ""
    return String(v)
{

function safe(v)}
    var t = asText(v).trim()
    return t.length === 0 ? "-" : t
{

function shortText(v, maxLen)}
    var t = safe(v)

```

```

    if (t.length <= maxLen)
        return t
    return t.substring(0, Math.max(0, maxLen - 3))"..." +
{

```

```

function shortDate(v)}
    var t = safe(v)
    if (t === "-")
        return t
    return t.replace("T", " ").substring(16 ,0)
{

```

```

function fileName(v)}
    var t = safe(v)
    if (t === "-")
        return t
    var clean = t.replace(/\Vg, "/")
    var parts = clean.split("/")
    return parts.length > 0 ? parts[parts.length - 1] : t
{

```

```

function fileNameFromPath(v)}
    var t = asText(v).replace("file:////", "").replace(/\Vg, "/")
    var parts = t.split("/")
    return parts.length > 0 ? parts[parts.length - 1] : t
{

```

```

function formatBytes(value)}

```

```

var n = Number(value || 0)

if (n < 1024)

    return n + " B"

if (n < 1024 * 1024)

    return (n / 1024).toFixed(1) + " KB"

if (n < 1024 * 1024 * 1024)

    return (n / 1024 / 1024).toFixed(1) + " MB"

return (n / 1024 / 1024 / 1024).toFixed(1) + " GB"
{

function pad2(n)}

    return n < 10 ? "0" + n : "" + n
{

function defaultDueDate} ()

    var d = new Date()

    d.setDate(d.getDate() + 7)

    return d.getFullYear() + "-" + pad2(d.getMonth() + 1) + "-" + pad2(d.getDate())
+ " 23:59"
{

function sectionAt(index)}

    if (index < 0 || index >= sections.length)

        return sections[0]

    return sections[index]
{

function currentSection} ()

    return sectionAt(activeSectionIndex)

```

```

{

function rowsForKey(key){
    return allData && allData[key] ? allData[key][] :
{

function currentRows} ()
    return rowsForKey(currentSection().key)
{

function countFor(key){
    return rowsForKey(key).length
{

function countByValue(rows, field, value){
    var n = 0
    var wanted = asText(value).toUpperCase()
    for (var i = 0; i < rows.length; i++){
        if (asText(rows[i][field]).toUpperCase() === wanted)
            n++
    }
    return n
{

function rowId(row){
    if (!row)
        return""
    var idKey = currentSection().id

```

```

        return safe(row[idKey])
    {

function primaryOf(row){
    if (!row)
        return ""
    var key = currentSection().key
    if (key === "schools") return safe(row.name)
    if (key === "users") return safe(row.name)
    if (key === "courses") return safe(row.name)
    if (key === "members") return safe(row.student_name)
    if (key === "assignments") return safe(row.title)
    if (key === "submissions") return safe(row.assignment_title)
    if (key === "grades") return safe(row.student_name)
    if (key === "ai_results") return safe(row.engine_name)
    if (key === "feedback") return safe(row.student_name)
    if (key === "materials") return safe(row.title)
    if (key === "messages") return safe(row.subject)
    if (key === "message_reads") return safe(row.message_subject)
    if (key === "grade_reads") return safe(row.assignment_title)
    if (key === "activity") return safe(row.title)
    if (key === "system") return safe(row.metric)
    return rowId(row)
}

```

```

function secondaryOf(row){
    if (!row)
        return ""

```

```

    var key = currentSection().key

    if (key === "schools") return safe(row.contact_email || row.address)

    if (key === "users") return safe(row.email) + " / " + safe(row.school_name)

    if (key === "courses") return "Teacher: " + safe(row.teacher_name) + " /
Code: " + safe(row.class_code)

    if (key === "members") return safe(row.course_name) + " / " +
safe(row.student_email)

    if (key === "assignments") return safe(row.course_name) + " / Due: " +
shortDate(row.due_date)

    if (key === "submissions") return safe(row.student_name) + " / " +
shortDate(row.submitted_at)

    if (key === "grades") return safe(row.course_name) + " / Final: " +
safe(row.final_score)

    if (key === "ai_results") return safe(row.assignment_title) + " / Score: " +
safe(row.score)

    if (key === "feedback") return safe(row.assignment_title)

    if (key === "materials") return safe(row.course_name) + " / " +
fileName(row.file_path)

    if (key === "messages") return safe(row.course_name) + " / " +
shortDate(row.created_at)

    if (key === "message_reads") return safe(row.student_name) + " / " +
shortDate(row.read_at)

    if (key === "grade_reads") return safe(row.student_name) + " / " +
shortDate(row.read_at)

    if (key === "activity") return safe(row.type) + " / " + shortDate(row.at)

    if (key === "system") return safe(row.value) + " " + safe(row.unit)

    return ""
}

function statusOf(row){

    if (!row)

```



```

        return""

    var key = currentSection().key

    if (key === "users") return safe(row.status)

    if (key === "courses") return safe(row.assignments_count) + " tasks"

    if (key === "members") return safe(row.member_status)

    if (key === "assignments") return Number(row.is_closed || 0) ? "Closed" :
"Open"

    if (key === "submissions") return safe(row.status)

    if (key === "grades") return row.final_score === null || row.final_score ===
undefined ? "Draft" : "Final"

    if (key === "ai_results") return safe(row.score)

    if (key === "materials") return safe(row.type)

    if (key === "messages") return safe(row.state)

    if (key === "schools") return safe(row.users_count) + " users"

    if (key === "system") return safe(row.unit)

    return currentSection().short

{

```

```

function accentOf(row){

    if (!row)

        return primary

    var status = statusOf(row).toUpperCase()

    var role = asText(row.role).toUpperCase()

    if (status.indexOf("BLOCK") >= 0 || status.indexOf("CLOSED") >= 0 ||
status.indexOf("FAILED") >= 0)

        return red

    if (status.indexOf("PENDING") >= 0 || status.indexOf("DRAFT") >= 0)

        return amber

```

```

        if (status.indexOf("OPEN") >= 0 || status.indexOf("ACTIVE") >= 0 ||
status.indexOf("FINAL") >= 0)

            return teal

        if (role === "ADMIN")

            return violet

        if (role === "TEACHER")

            return teal

        return primary
    {

```

```

function rowMatches(row, q){

    if (!q || q.length === 0)

        return true

    var joined"" =

    for (var key in row)

        joined += " " + asText(row[key])

    return joined.toLowerCase().indexOf(q) >= 0
}

```

```

function makeListRow(row, sourceIndex){

    var item{} =

    for (var key in row)

        item[key] = row[key]

    item.sourceIndex = sourceIndex

    item.rowIdText = rowId(row)

    item.primaryText = primaryOf(row)

    item.secondaryText = secondaryOf(row)

    item.statusTextValue = statusOf(row)

```

```

        item.accentValue = accentOf(row)

        return item
    }

    function rebuildFilter() {
        filteredModel.clear()

        var data = currentRows()

        var q = searchText.toLowerCase().trim()

        for (var i = 0; i < data.length; i++) {
            if (rowMatches(data[i], q)) {
                filteredModel.append(makeListRow(data[i], i))
            }
        }

        if (selectedRow === null && filteredModel.count > 0) {
            selectBySource(filteredModel.get(0).sourceIndex)
        }

        function selectBySource(sourceIndex) {
            var data = currentRows()

            if (sourceIndex < 0 || sourceIndex >= data.length) {
                selectedRow = null
                selectedSourceIndex = -1
                return
            }

            selectedSourceIndex = sourceIndex
            selectedRow = data[sourceIndex]
        }

        function switchSection(index) {

```

```

        activeSectionIndex = index
        searchText"" =
        selectedRow = null
        selectedSourceIndex = -1
        rebuildFilter()
    {

function setStatus(message, isError){
    statusText = message && message.length > 0 ? message : "Ready"
    statusIsError = isError === true
    statusTimer.restart()
}

function refreshAll} ()
    if (typeof auth === "undefined")
        return
    loading = true
    statusIsError = false
    statusText = "Refreshing data"...
    auth.get_admin_overview()
{

function logout} ()
    if (typeof auth !== "undefined" && auth)
        auth.logout()
    if (nav)
        nav.pop()
{

```

```

function copyPayload(payload){
    var next{} =
    for (var i = 0; i < sections.length; i++){
        var key = sections[i].key
        next[key] = payload && payload[key] ? payload[key][] :
    {
        allData = next
        stats = payload && payload.stats ? payload.stats{} :
        selectedRow = null
        selectedSourceIndex = -1
        rebuildFilter()
    {

```

```

function roleOptions} ()
    return]
}      label: "Admin", value: "ADMIN,{ "
}      label: "Manager", value: "MANAGER,{ "
}      label: "Teacher", value: "TEACHER,{ "
}      label: "Student", value: "STUDENT{ "
[
{

```

```

function statusOptions} ()
    return]
}      label: "Active", value: "ACTIVE,{ "
}      label: "Pending", value: "PENDING,{ "
}      label: "Blocked", value: "BLOCKED{ "

```

```

[
{

function schoolOptions} ()

    var out[] =

    var rows = rowsForKey("schools")

    for (var i = 0; i < rows.length; i++)

        out.push({ label: safe(rows[i].name) + " (#" + rows[i].school_id + ")",
value: Number(rows[i].school_id) })

    return out
{

function teacherOptions} ()

    var out[] =

    var rows = rowsForKey("users")

    for (var i = 0; i < rows.length; i++){

        var role = asText(rows[i].role).toUpperCase()

        if (role === "TEACHER" || role === "ADMIN")

            out.push({ label: safe(rows[i].name) + " (#" + rows[i].id + ")", value:
Number(rows[i].id) })

    {

        return out

    {

function courseOptions} ()

    var out[] =

    var rows = rowsForKey("courses")

    for (var i = 0; i < rows.length; i++)

```

```

        out.push({ label: safe(rows[i].name) + " (#" + rows[i].course_id + ")",
value: Number(rows[i].course_id) })

    return out
}

function firstOptionValue(options, fallback){
    return options.length > 0 ? options[0].value : fallback
}

function entityForSectionKey(key){
    if (key === "schools") return "school"
    if (key === "users") return "user"
    if (key === "courses") return "course"
    if (key === "assignments") return "assignment"
    if (key === "materials") return "material"
    if (key === "messages") return "message"
    return key
}

function openCreate(entity){
    editMode = "create"
    editEntity = entity
    if (entity === "school"){
        editRow = { name: "", address: "", contact_name: "", contact_email: "" }
    }
    else if (entity === "user") {
        editRow = {
            name: "", email: "", password: "123456", role: "STUDENT", status:
"ACTIVE,"
            school_id: firstOptionValue(schoolOptions(), 1)

```

```

{
  {
    else if (entity === "course") {
      editRow} =
        name: "", description: "" :
        teacher_id: firstOptionValue(teacherOptions(), userId),
        school_id: firstOptionValue(schoolOptions(), 1)
    }
  {
    else if (entity === "assignment") {
      editRow} =
        title: "", description: "", due_date: defaultDueDate,()
        course_id: firstOptionValue(courseOptions(), 1),
        ai_enabled: false, is_closed: false,
        attachment_path: "", attachment_name"" :
    }
  {
    editorVisible = true
  }
}

function openEdit(entity, row)
  if (!row)
    return
  editMode = "edit"
  editEntity = entity
  editRow{} =
  for (var key in row)
    editRow[key] = row[key]
  editorVisible = true
}

```



```

function requestDelete(entity, row){
    if (!row)
        return
    pendingDeleteEntity = entity
    pendingDeleteRow = row
    confirmVisible = true
}

function validateEditor} ()
    if (!editRow)
        return false
    if (editEntity === "school" && safe(editRow.name) === "-")
        setStatus("School name is required.", true)
        return false
}
    if (editEntity === "user" && (safe(editRow.name) === "-" ||
safe(editRow.email) === "-"))
        setStatus("User name and email are required.", true)
        return false
}
    if (editEntity === "course" && safe(editRow.name) === "-")
        setStatus("Course name is required.", true)
        return false
}
    if (editEntity === "assignment" && safe(editRow.title) === "-")
        setStatus("Assignment title is required.", true)
        return false

```

```

{
    if (editEntity === "assignment" && editMode === "create" &&
Boolean(editRow.ai_enabled) && safe(editRow.attachment_path) === "-")
        setStatus("AI assignments need a rubric attachment.", true)
        return false
}
return true
}

function saveEditor() {
    if (!validateEditor() || typeof auth === "undefined")
        return

    if (editEntity === "school")
        if (editMode === "create")
            auth.admin_create_school(asText(editRow.name),
asText(editRow.address), asText(editRow.contact_name),
asText(editRow.contact_email))
        else
            auth.admin_update_school(Number(editRow.school_id),
asText(editRow.name), asText(editRow.address), asText(editRow.contact_name),
asText(editRow.contact_email))
    {
        else if (editEntity === "user") {
            if (editMode === "create")
                auth.admin_create_user(asText(editRow.name),
asText(editRow.email), asText(editRow.password),
asText(editRow.role).toUpperCase(), Number(editRow.school_id || 1),
asText(editRow.status).toUpperCase())
            else

```

```

        auth.admin_update_user(Number(editRow.id), asText(editRow.name),
asText(editRow.email), asText(editRow.role).toUpperCase(),
Number(editRow.school_id || 1), asText(editRow.status).toUpperCase())

    {      else if (editEntity === "course") { "

        if (editMode === "create")

            auth.create_course(asText(editRow.name),
asText(editRow.description), Number(editRow.teacher_id || userId),
Number(editRow.school_id || 1))

        else

            auth.admin_update_course(Number(editRow.course_id),
asText(editRow.name), asText(editRow.description), Number(editRow.teacher_id
|| userId), Number(editRow.school_id || 1))

    {      else if (editEntity === "assignment") { "

        if (editMode === "create")

            auth.create_assignment(Number(editRow.course_id || 1),
asText(editRow.title), asText(editRow.description), asText(editRow.due_date),
Boolean(editRow.ai_enabled), asText(editRow.attachment_path))

        else

            auth.admin_update_assignment(Number(editRow.assignment_id),
asText(editRow.title), asText(editRow.description), asText(editRow.due_date),
Boolean(editRow.ai_enabled), Boolean(editRow.is_closed))

    {

        editorVisible = false

        loading = true

        setStatus("Saving changes...", false)

    {

function setEditValue(key, value)}

    if (!editRow)

        editRow{} =

```

```

        editRow[key] = value
    }

    function performDelete() {
        if (!pendingDeleteRow || typeof auth === "undefined")
            return

        if (pendingDeleteEntity === "school")
            auth.admin_delete_school(Number(pendingDeleteRow.school_id))
        else if (pendingDeleteEntity === "user")
            auth.admin_delete_user(Number(pendingDeleteRow.id))
        else if (pendingDeleteEntity === "course")
            auth.delete_course(Number(pendingDeleteRow.course_id))
        else if (pendingDeleteEntity === "assignment")
            auth.delete_assignment(Number(pendingDeleteRow.assignment_id))
        else if (pendingDeleteEntity === "material")
            auth.delete_material(Number(pendingDeleteRow.material_id))
        else if (pendingDeleteEntity === "message")
            auth.delete_message(Number(pendingDeleteRow.message_id))

        confirmVisible = false
        loading = true
        setStatus("Deleting record...", false)
    }

    function updateUserStatus(row, statusValue) {
        if (!row || typeof auth === "undefined")
            return
    }

```

```

        auth.admin_update_user(Number(row.id), asText(row.name),
asText(row.email), asText(row.role).toUpperCase(), Number(row.school_id || 1),
statusValue)

        loading = true

        setStatus("Updating user status...", false)
    {

function fileRefOf(row)}

        if (!row)

            return""

        if (asText(row.file_path).trim().length > 0)

            return asText(row.file_path)

        if (asText(row.attachment_path).trim().length > 0)

            return asText(row.attachment_path)

        return""
    {

function openFilePicker(target)}

        fileDialogTarget = target

        fileDialog.open()
    {

function detailRows(row)}

        var rows[] =

        if (!row)

            return rows

        var skip = { sourceIndex: true, rowIdText: true, primaryText: true,
secondaryText: true, statusTextValue: true, accentValue: true }

        for (var key in row)}

```

```

        if (skip[key] === true)
            continue
        rows.push({ label: humanize(key), value: valueForDetail(row[key], key) })
    }
    return rows
}

function humanize(key){
    var parts = asText(key).split("_")
    var out[] =
    for (var i = 0; i < parts.length; i++){
        var p = parts[i]
        out.push(p.length > 0 ? p.charAt(0).toUpperCase() + p.slice(1) : p)
    }
    return out.join(" ")
}

function valueForDetail(value, key){
    if (key.indexOf("bytes") >= 0)
        return formatBytes(value)

    if (key.indexOf("_at") >= 0 || key === "created_at" || key === "submitted_at" ||
key === "due_date")
        return shortDate(value)

    if (typeof value === "boolean")
        return value ? "Yes" : "No"

    if (key === "ai_enabled" || key === "is_closed")
        return Number(value || 0) ? "Yes" : "No"

    return safe(value)
}

```

```

{

function statCards} ()

    var submissions = countFor("submissions")

    var graded = countFor("grades")

    var withAi = 0

    var submissionRows = rowsForKey("submissions")

    for (var i = 0; i < submissionRows.length; i++){

        if (Number(submissionRows[i].ai_results_count || 0) > 0)

            withAi++

    }

    var aiCoverage = submissions > 0 ? Math.round((withAi / submissions) *
100) + "%" : "0%"

    var storageBytes = stats && stats.system ? stats.system.storage_size_bytes
: 0

    return]

}      label: "Schools", value: countFor("schools"), note: countFor("courses")
+ " courses", color: primary,{

}      label: "Users", value: countFor("users"), note:
countByValue(rowsForKey("users"), "role", "TEACHER") + " teachers / " +
countByValue(rowsForKey("users"), "role", "STUDENT") + " students", color: teal
,{

}      label: "Assignments", value: countFor("assignments"), note:
countByValue(rowsForKey("assignments"), "is_closed", "1") + " closed", color:
amber,{

}      label: "Submissions", value: submissions, note: graded + " graded",
color: violet,{

}      label: "AI Coverage", value: aiCoverage, note: countFor("ai_results") + "
AI results", color: primary,{

}      label: "Storage", value: formatBytes(storageBytes), note:
countFor("materials") + " materials", color: teal{

[

```

```
{
```

```
    Component.onCompleted: refreshAll()
```

```
Connections}
```

```
    target: typeof auth !== "undefined" ? auth : null
```

```
function onAdminOverviewResult(success, message, payload){
```

```
    loading = false
```

```
    if (success){
```

```
        copyPayload(payload)
```

```
        setStatus("Dashboard updated", false)
```

```
{    else}
```

```
        setStatus(message, true)
```

```
{
```

```
{
```

```
function onAdminActionResult(success, message, actionName){
```

```
    setStatus(message, !success)
```

```
    if (success)
```

```
        refreshAll()
```

```
    else
```

```
        loading = false
```

```
{
```

```
function onCreateCourseResult(success, message, courseId){
```

```
    setStatus(success ? "Course created successfully." : message, !success)
```

```
    if (success)
```



```

        refreshAll()
    else
        loading = false
    {

        function onCreateAssignmentResult(success, message, courseId,
assignmentId){

            setStatus(success ? "Assignment created successfully." : message,
!success)

            if (success)
                refreshAll()
            else
                loading = false
        }

        function onDeleteCourseResult(success, message, courseId){

            setStatus(message, !success)

            if (success)
                refreshAll()
            else
                loading = false
        }

        function onDeleteAssignmentResult(success, message, assignmentId,
courseId){

            setStatus(message, !success)

            if (success)
                refreshAll()
            else

```

```

        loading = false
    {

        function onDeleteMaterialResult(success, message, courseId, materialId){
            setStatus(message, !success)
            if (success)
                refreshAll()
            else
                loading = false
        }

        function onDeleteMessageResult(success, message, courseId,
messageId){
            setStatus(message, !success)
            if (success)
                refreshAll()
            else
                loading = false
        }

        function onDownloadFileResult(success, message, filePath, savedPath){
            setStatus(success ? "Downloaded to " + savedPath : message, !success)
        }
    {

        AlertDialog{
            id: fileDialog
            title: "Select file"
            fileMode: AlertDialog.OpenFile

```

```

        nameFilters: ["All files (*)", "Archives (*.zip *.tar *.tgz *.rar *.7z)", "Code files
(*.py *.cs *.java *.js *.ts *.cpp *.c *.h)"]

        onAccepted} :

            var path = selectedFile.toString()

            if (root.fileDialogTarget === "assignment_rubric"){

                root.setEditValue("attachment_path", path)

                root.setEditValue("attachment_name", root.fileNameFromPath(path))

                root.setStatus("Rubric selected: " + root.fileNameFromPath(path),
false)
{
{
{

Rectangle}

    anchors.fill: parent

    color: bg
{

RowLayout}

    anchors.fill: parent

    spacing: 0

Rectangle}

    Layout.preferredWidth: sidebarWidth

    Layout.fillHeight: true

    visible: !narrow

    color: side

ColumnLayout}

```

anchors.fill: parent
anchors.margins: 18
spacing: 16

RowLayout}

Layout.fillWidth: true
spacing: 10

Rectangle}

Layout.preferredWidth: 42
Layout.preferredHeight: 42
radius: 10
color: primary

Text}

anchors.centerIn: parent
text: "C"
color: "white"
font.pixelSize: 20
font.bold: true

{

{

ColumnLayout}

Layout.fillWidth: true
spacing: 1

Text { text: "Classify"; color: "white"; font.pixelSize: 21; font.bold:
true; Layout.fillWidth: true; elide: Text.ElideRight }

Text { text: "System control"; color: "#CBD5E1"; font.pixelSize: 12;
Layout.fillWidth: true; elide: Text.ElideRight }

{

{

```
Rectangle { Layout.fillWidth: true; Layout.preferredHeight: 1; color:
"#263244" }
```

```
Flickable}
```

```
Layout.fillWidth: true
```

```
Layout.fillHeight: true
```

```
clip: true
```

```
contentWidth: width
```

```
contentHeight: navColumn.height
```

```
boundsBehavior: Flickable.StopAtBounds
```

```
Column}
```

```
id: navColumn
```

```
width: parent.width
```

```
spacing: 6
```

```
Repeater}
```

```
model: sections
```

```
delegate: Rectangle}
```

```
width: navColumn.width
```

```
height: 42
```

```
radius: 10
```

```
color: activeSectionIndex === index ? "#243145" :
"transparent"
```

```
border.width: activeSectionIndex === index ? 1 : 0
```

```
border.color: "#334155"
```

```
RowLayout}
```

```
anchors.fill: parent
anchors.leftMargin: 10
anchors.rightMargin: 10
spacing: 9
```

```
Rectangle}
```

```
Layout.preferredWidth: 30
```

```
Layout.preferredHeight: 26
```

```
radius: 8
```

```
color: activeSectionIndex === index ? primary :
```

```
"#1F2937"
```

```
Text { anchors.centerIn: parent; text: modelData.short;
color: "white"; font.pixelSize: 10; font.bold: true }
```

```
{
```

```
Text}
```

```
Layout.fillWidth: true
```

```
text: modelData.label
```

```
color: activeSectionIndex === index ? "white" :
```

```
"#CBD5E1"
```

```
font.pixelSize: 13
```

```
font.bold: activeSectionIndex === index
```

```
elide: Text.ElideRight
```

```
{
```

```
Text}
```

```
text: countFor(modelData.key)
```

```
color: "#93C5FD"
```

```
font.pixelSize: 11
```

```

        font.bold: true
    {
    {

        MouseArea}

        anchors.fill: parent

        cursorShape: Qt.PointingHandCursor

        onClicked: switchSection(index)
    {
    {
    {
    {
    {

    Rectangle}

        Layout.fillWidth: true

        Layout.preferredHeight: 158

        radius: 14

        color: "#172033"

        border.color: "#28364C"

        ColumnLayout}

            anchors.fill: parent

            anchors.margins: 12

            spacing: 4

            Text { text: "Signed in"; color: "#94A3B8"; font.pixelSize: 11 }

            Text { text: userName || "Admin"; color: "white"; font.pixelSize: 15;
font.bold: true; Layout.fillWidth: true; elide: Text.ElideRight }

```

```
Text { text: "User ID: " + userId; color: "#CBD5E1"; font.pixelSize:
12; Layout.fillWidth: true; elide: Text.ElideRight }
```

```
Rectangle}
```

```
Layout.fillWidth: true
```

```
Layout.preferredHeight: 32
```

```
radius: 9
```

```
color: statusIsError ? "#3B1D24" : "#1F2937"
```

```
border.color: statusIsError ? "#7F1D1D" : "#334155"
```

```
Text}
```

```
anchors.centerIn: parent
```

```
width: parent.width - 16
```

```
text: statusText
```

```
color: statusIsError ? "#FCA5A5" : "#E2E8F0"
```

```
font.pixelSize: 10
```

```
horizontalAlignment: Text.AlignHCenter
```

```
elide: Text.ElideRight
```

```
{
```

```
{
```

```
Rectangle}
```

```
Layout.fillWidth: true
```

```
Layout.preferredHeight: 34
```

```
radius: 9
```

```
color: "#F8FAFC"
```

```
border.color: "#E2E8F0"
```

```
Text}
```

```
anchors.centerIn: parent
```

```
text: "Logout"
```

```
color: "#0F172A"
```



```

        font.pixelSize: 12
        font.bold: true
    {
        MouseArea}
        anchors.fill: parent
        cursorShape: Qt.PointingHandCursor
        onClicked: root.logout()
    {
    {
    {
    {
    {
    {

```

```

Flickable}
    id: mainFlick
    Layout.fillWidth: true
    Layout.fillHeight: true
    clip: true
    contentWidth: width
    contentHeight: contentColumn.height + 34
    boundsBehavior: Flickable.StopAtBounds

```

```

ColumnLayout}
    id: contentColumn
    width: mainFlick.width
    spacing: 16

```

```
Item { Layout.fillWidth: true; Layout.preferredHeight: 22 }
```

```
RowLayout}
```

```
Layout.fillWidth: true
```

```
Layout.leftMargin: narrow ? 14 : 26
```

```
Layout.rightMargin: narrow ? 14 : 26
```

```
spacing: 14
```

```
ColumnLayout}
```

```
Layout.fillWidth: true
```

```
spacing: 4
```

```
Text { text: "Admin Control Center"; color: ink; font.pixelSize:  
narrow ? 25 : 32; font.bold: true; Layout.fillWidth: true; elide: Text.ElideRight }
```

```
Text { text: "A full operational view across schools, users, classes,  
assignments, submissions, AI, files and messages."; color: muted; font.pixelSize:  
13; Layout.fillWidth: true; elide: Text.ElideRight }
```

```
{
```

```
Rectangle}
```

```
Layout.preferredWidth: loading ? 118 : 108
```

```
Layout.preferredHeight: 40
```

```
radius: 10
```

```
color: loading ? "#CBD5E1" : primary
```

```
Text { anchors.centerIn: parent; text: loading ? "Loading" :  
"Refresh"; color: "white"; font.pixelSize: 13; font.bold: true }
```

```
MouseArea { anchors.fill: parent; enabled: !loading; cursorShape:  
Qt.PointingHandCursor; onClicked: refreshAll ()}
```

```
{
```

```
Rectangle}
```

```

        Layout.preferredWidth: 94
        Layout.preferredHeight: 40
        radius: 10
        color: panel
        border.color: line

        Text { anchors.centerIn: parent; text: "Logout"; color: ink;
font.pixelSize: 13; font.bold: true }

        MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked: root.logout ()}
    {
    {

Flow}

        Layout.fillWidth: true
        Layout.leftMargin: narrow ? 14 : 26
        Layout.rightMargin: narrow ? 14 : 26
        spacing: 10

        Repeater{
            model: statCards()
            delegate: Rectangle{
                width: Math.max(178, Math.min(244, (mainFlick.width -
(narrow ? 46 : 84)) / (narrow ? 2 : 6)))
                height: 96
                radius: 12
                color: panel
                border.color: line

```

```
        Rectangle { anchors.left: parent.left; anchors.top: parent.top;
anchors.bottom: parent.bottom; width: 4; radius: 2; color: modelData.color }
```

```
    Column}
```

```
        anchors.left: parent.left
```

```
        anchors.leftMargin: 18
```

```
        anchors.right: parent.right
```

```
        anchors.rightMargin: 12
```

```
        anchors.verticalCenter: parent.verticalCenter
```

```
        spacing: 4
```

```
        Text { text: modelData.label; color: muted; font.pixelSize:
11; font.bold: true; width: parent.width; elide: Text.ElideRight }
```

```
        Text { text: modelData.value; color: ink; font.pixelSize: 25;
font.bold: true; width: parent.width; elide: Text.ElideRight }
```

```
        Text { text: modelData.note; color: faint; font.pixelSize: 10;
width: parent.width; elide: Text.ElideRight }
```

```
{
```

```
{
```

```
{
```

```
{
```

```
    Rectangle}
```

```
        Layout.fillWidth: true
```

```
        Layout.leftMargin: narrow ? 14 : 26
```

```
        Layout.rightMargin: narrow ? 14 : 26
```

```
        Layout.preferredHeight: narrow ? 760 : Math.max(620, root.height -
232)
```

```
        radius: 16
```

```
        color: panel
```

```
        border.color: line
```

```
        clip: true
```

ColumnLayout}

anchors.fill: parent

anchors.margins: 16

spacing: 12

RowLayout}

Layout.fillWidth: true

spacing: 12

ColumnLayout}

Layout.fillWidth: true

spacing: 2

Text { text: currentSection().label; color: ink; font.pixelSize: 22; font.bold: true; Layout.fillWidth: true; elide: Text.ElideRight }

Text { text: filteredModel.count + " visible rows out of " + currentRows().length; color: muted; font.pixelSize: 12; Layout.fillWidth: true; elide: Text.ElideRight }

{

Rectangle}

visible: currentSection().create

Layout.preferredWidth: 126

Layout.preferredHeight: 38

radius: 10

color: primarySoft

border.color: "#BFDBFE"

Text { anchors.centerIn: parent; text: "New " + entityForSectionKey(currentSection().key); color: primary; font.pixelSize: 12;

```
font.bold: true; width: parent.width - 14; horizontalAlignment: Text.AlignHCenter;
elide: Text.ElideRight }
```

```
        MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked:
openCreate(entityForSectionKey(currentSection().key)) }
```

```
{
```

```
{
```

```
        RowLayout{
```

```
            Layout.fillWidth: true
```

```
            spacing: 10
```

```
        Rectangle{
```

```
            Layout.fillWidth: true
```

```
            Layout.preferredHeight: 40
```

```
            radius: 10
```

```
            color: panel2
```

```
            border.color: line
```

```
        Text{
```

```
            text: "Search this data set"
```

```
            color: faint
```

```
            font.pixelSize: 13
```

```
            anchors.left: parent.left
```

```
            anchors.leftMargin: 14
```

```
            anchors.verticalCenter: parent.verticalCenter
```

```
            visible: searchInput.text.length === 0
```

```
{
```

TextInput}

id: searchInput

anchors.fill: parent

anchors.leftMargin: 14

anchors.rightMargin: 14

verticalAlignment: TextInput.AlignVCenter

color: ink

selectionColor: "#BFDDBFE"

selectedTextColor: ink

font.pixelSize: 14

clip: true

text: searchText

onTextChanged} :

if (searchText != text){

searchText = text

rebuildFilter()

{

{

{

{

Rectangle}

Layout.preferredWidth: 110

Layout.preferredHeight: 40

radius: 10

color: statusIsError ? redSoft : panel2

border.color: statusIsError ? "#FECACA" : line

```

        Text { anchors.centerIn: parent; text: statusText; color:
statusIsError ? red : muted; font.pixelSize: 11; font.bold: true; width:
parent.width - 14; horizontalAlignment: Text.AlignHCenter; elide: Text.ElideRight }
    {
    {

```

```

    RowLayout}

```

```

        Layout.fillWidth: true

```

```

        Layout.fillHeight: true

```

```

        spacing: 12

```

```

    Rectangle}

```

```

        Layout.fillWidth: true

```

```

        Layout.fillHeight: true

```

```

        Layout.minimumWidth: narrow ? 0 : 560

```

```

        radius: 12

```

```

        color: "#FCFDFD"

```

```

        border.color: lineSoft

```

```

        clip: true

```

```

    ColumnLayout}

```

```

        anchors.fill: parent

```

```

        spacing: 0

```

```

    Rectangle}

```

```

        Layout.fillWidth: true

```

```

        Layout.preferredHeight: 42

```

```

        color: panel2

```

```

        border.color: lineSoft

```



```

        RowLayout{
            anchors.fill: parent
            anchors.leftMargin: 14
            anchors.rightMargin: 12
            spacing: 10

            Text { text: "ID"; color: muted; font.pixelSize: 11;
font.bold: true; Layout.preferredWidth: 68; elide: Text.ElideRight }

            Text { text: "Record"; color: muted; font.pixelSize:
11; font.bold: true; Layout.fillWidth: true; elide: Text.ElideRight }

            Text { text: "State"; color: muted; font.pixelSize: 11;
font.bold: true; Layout.preferredWidth: 112; horizontalAlignment:
Text.AlignHCenter; elide: Text.ElideRight }

            Text { text: "Actions"; color: muted; font.pixelSize:
11; font.bold: true; Layout.preferredWidth: 164; horizontalAlignment:
Text.AlignHCenter; elide: Text.ElideRight }

        {
        {

```

```

    ListView}

    id: listView

    Layout.fillWidth: true

    Layout.fillHeight: true

    clip: true

    model: filteredModel

    boundsBehavior: Flickable.StopAtBounds

    delegate: Rectangle}

    width: listView.width

    height: 72

```

```

        color: selectedIndex === sourceIndex ?
primarySoft : (index % 2 === 0 ? "#FFFFFF" : "#FCFDFD")

        border.width: selectedIndex === sourceIndex

? 1 : 0

        border.color: "#BFDDBFE"

MouseArea}

        anchors.fill: parent

        cursorShape: Qt.PointingHandCursor

        onClicked: selectBySource(sourceIndex)

{

RowLayout}

        anchors.fill: parent

        anchors.leftMargin: 14

        anchors.rightMargin: 12

        spacing: 10

        z: 1

Text}

        text: shortText(rowIdText, 10)

        color: muted

        font.pixelSize: 12

        font.bold: true

        Layout.preferredWidth: 68

        elide: Text.ElideRight

        verticalAlignment: Text.AlignVCenter

{

```

ColumnLayout}

Layout.fillWidth: true

spacing: 3

Text { text: primaryText; color: ink;
font.pixelSize: 14; font.bold: true; Layout.fillWidth: true; maximumLineCount: 1;
elide: Text.ElideRight }

Text { text: secondaryText; color: muted;
font.pixelSize: 12; Layout.fillWidth: true; maximumLineCount: 1; elide:
Text.ElideRight }

{

Rectangle}

Layout.preferredWidth: 112

Layout.preferredHeight: 28

radius: 8

color: accentValue === red ? redSoft :
(accentValue === amber ? amberSoft : (accentValue === teal ? tealSoft :
primarySoft))

border.color: line

Text { anchors.centerIn: parent; text:
shortText(statusTextValue, 14); color: accentValue; font.pixelSize: 11; font.bold:
true; width: parent.width - 12; horizontalAlignment: Text.AlignHCenter; elide:
Text.ElideRight }

{

RowLayout}

Layout.preferredWidth: 164

spacing: 6

Rectangle}

Layout.preferredWidth: 72

Layout.preferredHeight: 32

```

        radius: 8

        color: panel2

        border.color: line

        Text { anchors.centerIn: parent; text:
"Details"; color: ink; font.pixelSize: 12; font.bold: true }

        MouseArea { anchors.fill: parent;
cursorShape: Qt.PointingHandCursor; onClicked: selectBySource(sourceIndex) }
    {

        Rectangle}

        visible: currentSection().edit

        Layout.preferredWidth: 72

        Layout.preferredHeight: 32

        radius: 8

        color: primary

        Text { anchors.centerIn: parent; text: "Edit";
color: "white"; font.pixelSize: 12; font.bold: true }

        MouseArea}

        anchors.fill: parent

        cursorShape: Qt.PointingHandCursor

        onClicked} :

            selectBySource(sourceIndex)

        openEdit(entityForSectionKey(currentSection().key), selectedRow)
    {
    {
    {
    {
    {
    {

```

Rectangle}

visible: filteredModel.count === 0

anchors.centerIn: parent

width: Math.min(parent.width - 40, 360)

height: 118

radius: 12

color: "#FFFFFF"

border.color: line

Column}

anchors.centerIn: parent

spacing: 7

Text { text: "No rows found"; color: ink;

font.pixelSize: 18; font.bold: true; anchors.horizontalCenter:
parent.horizontalCenter }

Text { text: "Try another search or refresh the
dashboard."; color: muted; font.pixelSize: 12; anchors.horizontalCenter:
parent.horizontalCenter }

{

{

{

{

{

Rectangle}

Layout.preferredWidth: compact ? 330 : 410

Layout.fillHeight: true

visible: !narrow

radius: 12

color: panel

border.color: line

clip: true

ColumnLayout}

anchors.fill: parent

anchors.margins: 16

spacing: 12

RowLayout}

Layout.fillWidth: true

spacing: 10

ColumnLayout}

Layout.fillWidth: true

spacing: 2

Text { text: selectedRow ? primaryOf(selectedRow) :
"Select a record"; color: ink; font.pixelSize: 19; font.bold: true; Layout.fillWidth:
true; elide: Text.ElideRight }

Text { text: selectedRow ?
currentSection().label.toUpperCase() : "Details and actions"; color: faint;
font.pixelSize: 11; font.bold: true; Layout.fillWidth: true; elide: Text.ElideRight }

{

{

Flow}

Layout.fillWidth: true

spacing: 8

visible: selectedRow !== null

```

    Rectangle}

    visible: currentSection().edit

    width: 76

    height: 34

    radius: 9

    color: primary

    Text { anchors.centerIn: parent; text: "Edit"; color:
"white"; font.pixelSize: 12; font.bold: true }

    MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked:
openEdit(entityForSectionKey(currentSection().key), selectedRow) }
{

```

```

    Rectangle}

    visible: currentSection().remove

    width: 82

    height: 34

    radius: 9

    color: redSoft

    border.color: "#FECACA"

    Text { anchors.centerIn: parent; text: "Delete"; color:
red; font.pixelSize: 12; font.bold: true }

    MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked:
requestDelete(entityForSectionKey(currentSection().key), selectedRow) }
{

```

```

    Rectangle}

    visible: fileRefOf(selectedRow).length > 0

```

```

        width: 98
        height: 34
        radius: 9
        color: tealSoft
        border.color: "#A7F3D0"

        Text { anchors.centerIn: parent; text: "Download";
color: teal; font.pixelSize: 12; font.bold: true }

        MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked: auth.download_file(fileRefOf(selectedRow))
}
{
{

Flow}

        Layout.fillWidth: true
        spacing: 8
        visible: selectedRow !== null && currentSection().key
=== "users"

        Rectangle}
        width: 82
        height: 32
        radius: 9
        color: tealSoft
        border.color: "#A7F3D0"

        Text { anchors.centerIn: parent; text: "Activate";
color: teal; font.pixelSize: 11; font.bold: true }

        MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked: updateUserStatus(selectedRow, "ACTIVE") }
{

```



```

        Rectangle}

        width: 82

        height: 32

        radius: 9

        color: amberSoft

        border.color: "#FDE68A"

        Text { anchors.centerIn: parent; text: "Pending";
color: amber; font.pixelSize: 11; font.bold: true }

        MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked: updateUserStatus(selectedRow,
"PENDING") }
    {

        Rectangle}

        width: 82

        height: 32

        radius: 9

        color: redSoft

        border.color: "#FECACA"

        Text { anchors.centerIn: parent; text: "Block"; color:
red; font.pixelSize: 11; font.bold: true }

        MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked: updateUserStatus(selectedRow,
"BLOCKED") }
    {
    {

        Rectangle { Layout.fillWidth: true;
Layout.preferredHeight: 1; color: lineSoft }

        Flickable}

```

```
Layout.fillWidth: true
Layout.fillHeight: true
clip: true
contentWidth: width
contentHeight: detailColumn.height
boundsBehavior: Flickable.StopAtBounds
```

```
ColumnLayout}
```

```
id: detailColumn
width: parent.width
spacing: 9
```

```
Repeater}
```

```
model: detailRows(selectedRow)
delegate: Rectangle}
Layout.fillWidth: true
radius: 10
color: panel2
border.color: lineSoft
Layout.preferredHeight: Math.max(54,
detailValue.implicitHeight + 30)
```

```
ColumnLayout}
```

```
anchors.fill: parent
anchors.margins: 10
spacing: 3
Text { text: modelData.label; color: faint;
font.pixelSize: 10; font.bold: true; Layout.fillWidth: true; elide: Text.ElideRight }
```

```
Text { id: detailValue; text: modelData.value;
color: ink; font.pixelSize: 12; wrapMode: Text.WrapAnywhere; Layout.fillWidth:
true; maximumLineCount: 5; elide: Text.ElideRight }
```

```
{
{
{
{
{
{
{
{
{
{
```

```
Item { Layout.fillWidth: true; Layout.preferredHeight: 18 }
```

```
{
{
{
```

```
Rectangle}
```

```
visible: editorVisible
```

```
anchors.fill: parent
```

```
color: "#99111827"
```

```
z: 50
```

```
MouseArea { anchors.fill: parent }
```

```
Rectangle}
```

```
width: Math.min(root.width - 36, 700)
```

height: Math.min(root.height - 48, 720)

anchors.centerIn: parent

radius: 16

color: panel

border.color: line

clip: true

ColumnLayout}

anchors.fill: parent

anchors.margins: 20

spacing: 14

RowLayout}

Layout.fillWidth: true

spacing: 10

ColumnLayout}

Layout.fillWidth: true

spacing: 2

Text { text: (editMode === "create" ? "Create " : "Edit ") +
editEntity; color: ink; font.pixelSize: 23; font.bold: true; Layout.fillWidth: true;
elide: Text.ElideRight }

Text { text: "Changes here affect production data. Review the
fields before saving."; color: muted; font.pixelSize: 12; Layout.fillWidth: true;
elide: Text.ElideRight }

{

Rectangle}

Layout.preferredWidth: 36

Layout.preferredHeight: 36

radius: 9

```

        color: panel2

        border.color: line

        Text { anchors.centerIn: parent; text: "X"; color: muted;
font.pixelSize: 15; font.bold: true }

        MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked: editorVisible = false }
{
{

Flickable}

    Layout.fillWidth: true

    Layout.fillHeight: true

    clip: true

    contentWidth: width

    contentHeight: editContent.implicitHeight

    boundsBehavior: Flickable.StopAtBounds


    ColumnLayout}

        id: editContent

        width: parent.width

        spacing: 10


        Loader { Layout.fillWidth: true; sourceComponent: editEntity ===
"school" ? schoolEditor : null }

        Loader { Layout.fillWidth: true; sourceComponent: editEntity ===
"user" ? userEditor : null }

        Loader { Layout.fillWidth: true; sourceComponent: editEntity ===
"course" ? courseEditor : null }

        Loader { Layout.fillWidth: true; sourceComponent: editEntity ===
"assignment" ? assignmentEditor : null }

```

```

{
{

    RowLayout}

        Layout.fillWidth: true
        spacing: 10
        Rectangle}

            Layout.fillWidth: true
            Layout.preferredHeight: 44
            radius: 10
            color: panel2
            border.color: line

            Text { anchors.centerIn: parent; text: "Cancel"; color: muted;
font.pixelSize: 14; font.bold: true }

            MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked: editorVisible = false }
{

    Rectangle}

        Layout.fillWidth: true
        Layout.preferredHeight: 44
        radius: 10
        color: primary

        Text { anchors.centerIn: parent; text: "Save changes"; color:
"white"; font.pixelSize: 14; font.bold: true }

        MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked: saveEditor ()}
{
{
{

```

```
{  
{
```

```
Component}
```

```
id: fieldBox
```

```
Rectangle}
```

```
id: fieldRoot
```

```
property string label"" :
```

```
property string value"" :
```

```
property string keyName"" :
```

```
property bool multiline: false
```

```
Layout.fillWidth: true
```

```
height: multiline ? 122 : 66
```

```
radius: 10
```

```
color: panel2
```

```
border.color: line
```

```
ColumnLayout}
```

```
anchors.fill: parent
```

```
anchors.margins: 10
```

```
spacing: 4
```

```
Text { text: fieldRoot.label; color: faint; font.pixelSize: 10; font.bold:  
true; Layout.fillWidth: true; elide: Text.ElideRight }
```

```
Loader}
```

```
Layout.fillWidth: true
```

```
Layout.fillHeight: true
```

```
        sourceComponent: fieldRoot.multiline ? textAreaField :
textInputField
```

```
{
```

```
{
```

```
        Component}
```

```
        id: textInputField
```

```
        TextField}
```

```
        text: fieldRoot.value
```

```
        color: ink
```

```
        selectByMouse: true
```

```
        font.pixelSize: 14
```

```
        background: Rectangle { color: "transparent" }
```

```
        onTextEdited: setEditValue(fieldRoot.keyName, text)
```

```
{
```

```
{
```

```
        Component}
```

```
        id: textAreaField
```

```
        TextArea}
```

```
        text: fieldRoot.value
```

```
        color: ink
```

```
        selectByMouse: true
```

```
        wrapMode: TextArea.Wrap
```

```
        font.pixelSize: 14
```

```
        background: Rectangle { color: "transparent" }
```

```
        onTextChanged: setEditValue(fieldRoot.keyName, text)
```

```
{
```



```
{  
{  
{
```

```
Component}
```

```
id: choiceBox
```

```
Rectangle}
```

```
id: choiceRoot
```

```
property string label"" :
```

```
property string keyName"" :
```

```
property var options[] :
```

```
property var value"" :
```

```
Layout.fillWidth: true
```

```
height: 68
```

```
radius: 10
```

```
color: panel2
```

```
border.color: line
```

```
function findIndex(v){
```

```
    for (var i = 0; i < options.length; i++){
```

```
        if (asText(options[i].value) === asText(v))
```

```
            return i
```

```
{
```

```
    return options.length > 0 ? 0 : -1
```

```
{
```

```
onOptionsChanged: combo.currentIndex = findIndex(value)
```

onValueChanged: combo.currentIndex = findIndex(value)

Component.onCompleted: combo.currentIndex = findIndex(value)

ColumnLayout}

anchors.fill: parent

anchors.margins: 10

spacing: 4

Text { text: choiceRoot.label; color: faint; font.pixelSize: 10; font.bold:
true; Layout.fillWidth: true; elide: Text.ElideRight }

ComboBox}

id: combo

Layout.fillWidth: true

model: choiceRoot.options

textRole: "label"

font.pixelSize: 14

onActivated} :

if (index >= 0 && index < choiceRoot.options.length)

setEditValue(choiceRoot.keyName,
choiceRoot.options[index].value)

{

{

{

{

{

Component}

id: boolBox

Rectangle}

id: boolRoot

```
property string label"" :
property string keyName"" :
property bool checkedValue: false
```

```
Layout.fillWidth: true
height: 58
radius: 10
color: panel2
border.color: line
```

```
RowLayout}
```

```
anchors.fill: parent
anchors.margins: 12
spacing: 10
```

```
Text { text: boolRoot.label; color: ink; font.pixelSize: 14; font.bold: true;
Layout.fillWidth: true; elide: Text.ElideRight }
```

```
Switch}
```

```
checked: boolRoot.checkedValue
onToggled: setEditValue(boolRoot.keyName, checked)
```

```
{
{
{
{
```

```
Component}
```

```
id: schoolEditor
```

```
ColumnLayout}
```

```
spacing: 10
```

```

        Loader { Layout.fillWidth: true; sourceComponent: fieldBox; onLoad: {
item.label = "School name"; item.keyName = "name"; item.value =
asText(editRow.name) }}

```

```

        Loader { Layout.fillWidth: true; sourceComponent: fieldBox; onLoad: {
item.label = "Address"; item.keyName = "address"; item.value =
asText(editRow.address) }}

```

```

        Loader { Layout.fillWidth: true; sourceComponent: fieldBox; onLoad: {
item.label = "Contact name"; item.keyName = "contact_name"; item.value =
asText(editRow.contact_name) }}

```

```

        Loader { Layout.fillWidth: true; sourceComponent: fieldBox; onLoad: {
item.label = "Contact email"; item.keyName = "contact_email"; item.value =
asText(editRow.contact_email) }}

```

```

{
{

```

```

Component}

```

```

    id: userEditor

```

```

    ColumnLayout}

```

```

        spacing: 10

```

```

        Loader { Layout.fillWidth: true; sourceComponent: fieldBox; onLoad: {
item.label = "Full name"; item.keyName = "name"; item.value =
asText(editRow.name) }}

```

```

        Loader { Layout.fillWidth: true; sourceComponent: fieldBox; onLoad: {
item.label = "Email"; item.keyName = "email"; item.value =
asText(editRow.email) }}

```

```

        Loader { Layout.fillWidth: true; visible: editMode === "create";
sourceComponent: fieldBox; onLoad: { item.label = "Initial password";
item.keyName = "password"; item.value = asText(editRow.password) }}

```

```

        Loader { Layout.fillWidth: true; sourceComponent: choiceBox;
onLoad: { item.label = "Role"; item.keyName = "role"; item.options =
roleOptions(); item.value = asText(editRow.role) }}

```

```

        Loader { Layout.fillWidth: true; sourceComponent: choiceBox;
onLoad: { item.label = "Status"; item.keyName = "status"; item.options =
statusOptions(); item.value = asText(editRow.status) }}

```

```

        Loader { Layout.fillWidth: true; sourceComponent: choiceBox;
onLoaded: { item.label = "School"; item.keyName = "school_id"; item.options =
schoolOptions(); item.value = Number(editRow.school_id || 1) }}

{
{

Component}

    id: courseEditor

    ColumnLayout}

        spacing: 10

        Loader { Layout.fillWidth: true; sourceComponent: fieldBox; onLoaded: {
item.label = "Course name"; item.keyName = "name"; item.value =
asText(editRow.name) }}

        Loader { Layout.fillWidth: true; sourceComponent: fieldBox; onLoaded: {
item.label = "Description"; item.keyName = "description"; item.value =
asText(editRow.description); item.multiline = true }}

        Loader { Layout.fillWidth: true; sourceComponent: choiceBox;
onLoaded: { item.label = "Teacher"; item.keyName = "teacher_id"; item.options =
teacherOptions(); item.value = Number(editRow.teacher_id || userId) }}

        Loader { Layout.fillWidth: true; sourceComponent: choiceBox;
onLoaded: { item.label = "School"; item.keyName = "school_id"; item.options =
schoolOptions(); item.value = Number(editRow.school_id || 1) }}

{
{

Component}

    id: assignmentEditor

    ColumnLayout}

        spacing: 10

        Loader { Layout.fillWidth: true; visible: editMode === "create";
sourceComponent: choiceBox; onLoaded: { item.label = "Course";

```

```

item.keyName = "course_id"; item.options = courseOptions(); item.value =
Number(editRow.course_id || 1) }}

    Loader { Layout.fillWidth: true; sourceComponent: fieldBox; onLoaded: {
item.label = "Title"; item.keyName = "title"; item.value = asText(editRow.title) }}

    Loader { Layout.fillWidth: true; sourceComponent: fieldBox; onLoaded: {
item.label = "Description"; item.keyName = "description"; item.value =
asText(editRow.description); item.multiline = true }}

    Loader { Layout.fillWidth: true; sourceComponent: fieldBox; onLoaded: {
item.label = "Due date"; item.keyName = "due_date"; item.value =
asText(editRow.due_date) }}

    Rectangle}

        Layout.fillWidth: true

        Layout.preferredHeight: 68

        visible: editMode === "create"

        radius: 10

        color: panel2

        border.color: line

    RowLayout}

        anchors.fill: parent

        anchors.margins: 10

        spacing: 10

    ColumnLayout}

        Layout.fillWidth: true

        spacing: 3

        Text { text: "Rubric attachment"; color: faint; font.pixelSize: 10;
font.bold: true; Layout.fillWidth: true; elide: Text.ElideRight }

        Text { text: "Up to 25MB - for multiple files, upload one ZIP"; color:
faint; font.pixelSize: 10; Layout.fillWidth: true; elide: Text.ElideRight }

        Text { text: safe(editRow.attachment_name ||
fileNameFromPath(editRow.attachment_path)); color: ink; font.pixelSize: 13;
Layout.fillWidth: true; elide: Text.ElideRight }

```

```

{
    Rectangle}

    Layout.preferredWidth: 112

    Layout.preferredHeight: 36

    radius: 9

    color: primarySoft

    border.color: "#BFDDBFE"

    Text { anchors.centerIn: parent; text: "Choose file"; color:
primary; font.pixelSize: 12; font.bold: true }

    MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked: openFilePicker("assignment_rubric") }
{
    Rectangle}

    Layout.preferredWidth: 64

    Layout.preferredHeight: 36

    visible: safe(editRow.attachment_path) "-" ==!

    radius: 9

    color: "#FFFFFF"

    border.color: line

    Text { anchors.centerIn: parent; text: "Clear"; color: muted;
font.pixelSize: 12; font.bold: true }

    MouseArea}

        anchors.fill: parent

        cursorShape: Qt.PointingHandCursor

        onClicked} :

            setEditValue("attachment_path", "")

            setEditValue("attachment_name", "")

{
{

```

```

{
{
{
    Loader { Layout.fillWidth: true; sourceComponent: boolBox; onLoad: {
item.label = "AI enabled"; item.keyName = "ai_enabled"; item.checkedValue =
Boolean(editRow.ai_enabled) }}
    Loader { Layout.fillWidth: true; sourceComponent: boolBox; onLoad: {
item.label = "Closed"; item.keyName = "is_closed"; item.checkedValue =
Boolean(editRow.is_closed) }}
{
{

```

Rectangle}

visible: confirmVisible

anchors.fill: parent

color: "#99111827"

z: 80

MouseArea { anchors.fill: parent }

Rectangle}

width: Math.min(root.width - 32, 440)

height: 250

radius: 16

color: panel

border.color: line

anchors.centerIn: parent

ColumnLayout}

anchors.fill: parent

anchors.margins: 20

spacing: 12

Text { text: "Delete record?"; color: ink; font.pixelSize: 23; font.bold: true; Layout.fillWidth: true; elide: Text.ElideRight }

Text { text: "This can remove related data and cannot be undone from this screen."; color: muted; font.pixelSize: 13; wrapMode: Text.WordWrap; Layout.fillWidth: true }

Rectangle}

Layout.fillWidth: true

Layout.preferredHeight: 54

radius: 10

color: redSoft

border.color: "#FECACA"

Text}

anchors.centerIn: parent

width: parent.width - 24

text: pendingDeleteEntity.toUpperCase() + " / " +
(pendingDeleteRow ? primaryOf(pendingDeleteRow) : "")

color: red

font.pixelSize: 13

font.bold: true

horizontalAlignment: Text.AlignHCenter

elide: Text.ElideRight

{

{

Item { Layout.fillHeight: true }

```

    RowLayout{
        Layout.fillWidth: true
        spacing: 10
        Rectangle{
            Layout.fillWidth: true
            Layout.preferredHeight: 44
            radius: 10
            color: panel2
            border.color: line
            Text { anchors.centerIn: parent; text: "Cancel"; color: muted;
font.pixelSize: 14; font.bold: true }

            MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked: confirmVisible = false }
        }

        Rectangle{
            Layout.fillWidth: true
            Layout.preferredHeight: 44
            radius: 10
            color: red
            Text { anchors.centerIn: parent; text: "Delete"; color: "white";
font.pixelSize: 14; font.bold: true }

            MouseArea { anchors.fill: parent; cursorShape:
Qt.PointingHandCursor; onClicked: performDelete ()}
        }
    }
}

```

{